TECHNICAL DEEP DIVE

How AArchX works

An x86-64 to arm64 binary translator for macOS, from the instruction decoder to Steam, Wine and native frameworks.

AArchX 0.8 (formerly Ocerz) · source at commit **6e6c64b5**, 2026-10-18
Written from the source tree, its explanation blocks, its documentation and its git history.

About this document

This is a technical deep dive into AArchX, a userspace binary translator that runs x86-64 macOS programs, and 32-bit Intel Mac programs, which macOS has refused to run since Catalina in 2019, on Apple silicon. It is written for systems engineers who know C, can read some x86 and some arm64, and want to understand how this particular translator works: not only what each part does, but why it has the shape it has, which failures shaped it, and what it costs.

It follows a program through the translator from the command line to its exit, then takes each layer in turn: the decoder, the reference interpreter, the arm64 JIT, guest memory, the system-call layer, the Mach-O loader, the dynamic linker that binds against Apple's x86-64 shared cache, native mode, which binds the same programs against the Mac's own arm64 frameworks instead, and m32, which runs i386 Mac programs on those frameworks. Later chapters cover the real software that runs (Cocoa applications, the macOS Steam client, games, Windows programs through Wine), how correctness is held, how speed is measured, the debugging tools, and what is still missing.

How it was written

Everything here describes the source tree at commit `6e6c64b5` of 18 October 2026, AArchX 0.8. This is the second edition; the first described commit `90fe4d8e` of 4 October 2026, before releases 0.4 to 0.8, and Chapter 13 is new in this one. On 14 October 2026 the project's history was rewritten: every commit ID changed and the contents did not. The IDs quoted here are those of the current history. The material comes from four places, in this order of authority: the source itself, where every file opens with a prose block that explains what it owns and why; the commit history, whose messages state the rule each change establishes; the project's own documentation under `docs/`; and a set of engineering notes that were checked against the code. Numbers were taken from the tree on that date (line counts, test counts, knob counts) or from dated measurements recorded in the history, and each says when it was measured. Where something could not be verified, the text says so or leaves it out.

The document describes Apple's software only from the outside: public interfaces, published file formats, error messages and behaviour that can be observed by running programs. AArchX's own source is quoted freely; it is free software under the GNU Lesser General Public License, version 2.1 or later.

Names and conventions

AArchX
: The product. It was called Ocerz until 4 September 2026, and the old name survives in the binary, the source paths and every environment variable.

`ocerz`
: The executable you run, and the prefix of its environment variables, `OCERZ_*`, and its C symbols, `ocerz_*`.

guest
: The x86-64 (or, inside Wine and under m32, i386) program being translated, and everything it sees: guest registers, guest memory, guest threads, guest addresses.

host

: The arm64 process AArchX actually is, and the macOS kernel underneath it. Host code is arm64; host addresses are what the processor really uses.

cache mode, native mode

: The two ways a guest's system libraries are supplied: Apple's x86-64 shared cache, translated like any other code, or the Mac's own arm64 frameworks reached through generated bridges.

Code, file names, functions and environment variables are set in `monospace`. Ratios against Rosetta are AArchX's time divided by Rosetta's, so lower is better and anything under 1.00x is faster. Sizes use KB, MB and GB in the binary sense. Callouts mark a *Key idea*, a *Note*, a *Warning*, or *How it was found*: the story of a bug whose fix explains a rule.

How to read it

- **If you want to run software:** Chapter 1 and Chapters 13, 14 and 15 for what runs, Chapter 18 for diagnosing a failure, and Appendices A and B for the switches and messages.
- **If you want to contribute:** Chapters 2 and 3 first, then the chapters for the layer you will touch, then Chapter 16 on testing; every change is expected to carry the test that pins it.
- **If you want to understand binary translation:** read Chapters 2, 4, 5, 6 and 7 in order, then Chapter 12 for the less common idea of running translated code against native libraries.

Contents

PART I · ORIENTATION

1 What AArchX is 12

A translator written from scratch 12
Why it exists 13
What it is not 13
Two ways to supply the system libraries 13
What runs today 15
How fast 16
Installing it 17
History 17
Where to go from here 20

2 The life of a program 21

1. The command line 22
2. Reserving guest memory 23
3. Mapping the shared cache 24
4. Mapping, binding and protecting the program 24
5. The initial frame, and the pieces the guest expects 25
6. Initializers, in dyld's order 25
7. The first block 27
8. A system call 28
9. A second thread 29
10. A signal 29
11. Exit 30
The same program in native mode 31
A static program 32
A 32-bit program 33
Exit codes along the way 33

3 The source tree and its conventions 35

The shape of the repository 35
Every source file 36
The headers and how they depend on each other 40
Three structures hold the state 41
Building 42
The conventions 44
The switches 45
Documentation in the tree 45

PART II · THE CPU

4 The decoder 47

One function, one format 47
Anatomy of an x86-64 instruction 48
The decode pipeline 48
Prefixes 49
ModRM, SIB and addressing 50
The X86Insn record 51

Operand and address widths ... 52
560 internal operations ... 52
The opcode maps ... 53
x87 decoding ... 54
VEX ... 55
i386 mode ... 55
The cost of decoding ... 57
What it refuses, and why that is right ... 58
Proving a decoder correct ... 58
Bugs that shaped it ... 59

5 The interpreter and x86 flags ... 61

Three jobs for one engine ... 61
The CPU state ... 62
Executing one instruction ... 64
x86 flags, computed eagerly ... 66
The deferred record ... 68
Flag liveness ... 69
Integer corners ... 71
The x87 unit ... 72
SSE and AVX ... 75
i386 execution ... 77
What the JIT leaves to the interpreter ... 77

6 The JIT: translation, registers and control flow ... 79

Where the design came from ... 79
The code arena ... 80
Finding a block ... 81
The run loop and the dispatcher ... 81
Building a block ... 82
Register pinning ... 84
Anatomy of a block ... 87
Addressing guest memory ... 88
Flags without computing flags ... 89
Exits and chains ... 95
Calls and returns ... 99
Superblocks and trace inversion ... 102
Lane caches and the edges of a loop ... 103
32-bit blocks ... 104
The emitter ... 106
When a block faults ... 106
Lessons, in one table ... 107

7 The JIT: floating point, SIMD, memory ordering and changing code ... 109

Exact floating point ... 109
The x87 unit ... 113
SIMD coverage ... 117
Memory ordering ... 121
Addressing in Wine's memory layout ... 126
Code that changes ... 128
When the arena fills ... 131
In-place string and memory routines ... 133

Keeping translations: the on-disk cache ... 135
Switches for this chapter ... 137

PART III · MEMORY AND THE OPERATING SYSTEM

8 Guest memory ... 141
The constraints ... 141
Three address maps ... 141
Translating an address ... 144
4 KB guest pages on 16 KB host pages ... 145
Files and shared memory ... 147
Armed pages and kernel writes ... 148
What the guest is told about its memory ... 148
Host memory inside the guest's world ... 148
The Wine layout's guard and its hoist ... 150
The translation store ... 152
Native mode ... 153
Footprint ... 153
Switches ... 154

9 System calls, Mach, threads and signals ... 155
How a guest enters the kernel ... 155
The return convention is Rosetta's ... 156
Pointers the kernel reads and writes ... 157
Mach messages ... 158
Threads ... 160
Suspending threads ... 161
Signals ... 161
Faults ... 165
Callbacks from native code ... 166
The unstick monitor ... 167
Processes ... 167
Diagnostics ... 168

PART IV · LINKING

10 The Mach-O loader ... 171
Two kinds of program, one front door ... 171
What main() settles first ... 172
Where an i386 program goes ... 174
Choosing the slice ... 175
The static path ... 176
The initial stack ... 176
The dynamic frame ... 177
Where a dynamic image lands ... 178
From here to the mini-dyld ... 179

11 The mini-dyld and the shared cache ... 180
Why a linker of its own ... 180
From exec to main ... 180
Finding and mapping the cache ... 181
Resolving symbols ... 183
Fixups ... 186

Initializers ... 186
dlopen, dlsym and the image list ... 189
The dyld API table ... 191
Objective-C in cache mode ... 193
Thread-local variables ... 194
Keeping the launch small ... 194
Smaller things that had large effects ... 196

12 Native mode ... 197
The idea ... 197
Synthesized libraries ... 199
The API database ... 200
The ABI engine ... 203
Calls from translated code ... 206
Calls back into guest code ... 207
Objective-C ... 210
Blocks ... 215
Swift ... 216
CoreFoundation and callback structures ... 217
Everything else a process does ... 218
When a crossing fails ... 219
What it costs ... 219
Steam in native mode ... 220
Older Mac applications ... 220
Under Wine, and for 32-bit programs ... 222
What native mode cannot run yet ... 224

13 32-bit Mac programs (m32) ... 225
Where m32 comes from ... 225
How a program arrives at m32 ... 226
The 4 GB window ... 228
Running i386 code ... 229
Calls into the host ... 230
Calls from inside translated code ... 235
Calls back into guest code ... 236
Objective-C 1 ... 237
Carbon on what Apple silicon keeps ... 238
OpenGL ... 240
The libc and libSystem gaps ... 240
The Steam bridge ... 241
What runs ... 242
Testing ... 243
What does not work yet ... 243

PART V · REAL SOFTWARE

14 macOS applications, Steam, games and the GPU ... 245
How an application is run ... 245
Command-line tools ... 246
Applications in cache mode ... 247
The GPU ... 250
The macOS Steam client ... 252

Games ... 254
The AArchX app ... 258
What is still open ... 260

15 Wine and WoW64 ... 261
Why Wine is the hardest guest ... 261
The segment-register contract ... 263
WoW64 and 32-bit code ... 264
Memory: the low shadow ... 265
The walls, in order ... 270
Wine under native mode ... 273
Signals, thread suspension and fault loops ... 274
CrossOver ... 276
Games through Wine ... 276
What is still open ... 279

PART VI · QUALITY, SPEED AND TOOLING

16 Testing ... 281
The shape of the problem ... 281
Unit harnesses ... 282
The guest suite ... 283
The x86-64 differential gate ... 285
The i386 differential gate ... 285
The dynamic suite ... 286
The native suite and its companions ... 288
The m32 gate ... 289
Real software, outside the gates ... 290
Where expected outputs come from ... 290
Conventions ... 291
Current results ... 291
Conventions ... 292
Adding a test ... 293

17 Performance ... 294
How it is measured ... 294
Results ... 295
What makes it fast ... 297
What is still slower ... 298
SIMD: AVX2, FMA and MMX ... 298
x87 ... 299
Exactness and its price ... 300
Memory ordering ... 300
Native-mode crossings ... 301
Wine's layout ... 301
7-Zip under Wine ... 303
Startup: where real applications lose ... 304
Games ... 307
Native mode on a real application ... 308
Callbacks and sends ... 308
32-bit Mac programs ... 309
Reading these numbers honestly ... 310

18 The debugging toolkit ... 311
The method ... 311
Bisecting a JIT bug ... 312
Reading a crash report ... 314
A hung process ... 315
Where the time goes ... 316
Loader and library diagnostics ... 317
System calls, Mach and IOKit ... 317
Diagnosing a 32-bit program ... 318
Native-mode diagnostics ... 319
Case studies ... 320

19 Limitations and roadmap ... 324
What closed since 2026-10-04 ... 324
Programs that stop ... 326
32-bit programs (m32) ... 327
The GPU and Steam ... 327
Slower than Rosetta ... 328
Semantic deviations ... 329
Loader gaps in cache mode ... 330
Translator internals still open ... 330
Native mode ... 331
The direction: after Rosetta ... 332
Open fronts ... 333
The June plan, scored ... 333

APPENDICES

A Environment variables ... 335
Command line ... 335
Choosing what runs ... 336
Process trees ... 336
Keeping translations ... 336
Memory ordering ... 337
Turning off what makes it fast ... 337
Behaviour ... 339
Native mode ... 340
32-bit programs (m32) ... 341
Diagnostics ... 342
Switches that restore old behaviour ... 342
The diagnostic knobs, counted ... 343

B Exit codes and messages ... 345
Exit codes ... 345
Starting up ... 346
Loading and binding ... 346
Running ... 347
Native mode ... 348
32-bit programs (m32) ... 349

C Glossary ... 351

D Timeline ... 359

June: from static binaries to Wine 359
July: a JIT that keeps up 360
August: the pinned JIT and 32-bit code 360
September: Steam, AVX, native mode and games 360
October: games under Wine, and 32-bit programs 361

PART I

Orientation

What AArchX is, how a program travels through it from exec to exit, and how the source tree is laid out.

CHAPTER 1

What AArchX is

AArchX runs Intel macOS programs on Apple silicon by translating their x86-64 machine code to arm64 as they run, the job Rosetta 2 does. This chapter says what it is and is not, the two ways it supplies a program's system libraries, what runs on it today, how it is installed, and how it got here in four and a half months.

114,486
LINES OF C, ASSEMBLY AND HEADERS

1,082
COMMITS, 3 JUN TO 18 OCT 2026

14 / 15
BENCHMARK KERNELS FASTER THAN ROSETTA

40,044
I386 SEQUENCES CHECKED PER MEMORY LAYOUT

A translator written from scratch

An x86-64 Mach-O executable cannot run on an Apple silicon processor: the instructions mean nothing to it. Rosetta 2, Apple's answer, translates them ahead of time and at run time, and does so invisibly enough that most people never notice it is there. AArchX does the same job with none of Rosetta's code. It is one arm64 program, `ocerz`, that is given an x86-64 program on its command line and runs it inside its own process:

```
$ ./ocerz /usr/bin/sw_vers
ProductName:            macOS
ProductVersion:         26.7.1
BuildVersion:           25G241
```

Everything between that command and the output is AArchX's own: it parses the Mach-O file, reserves and lays out the guest's memory, loads and binds the program's libraries, decodes the x86 instructions, executes them in an interpreter or translates them to arm64 in a JIT, and answers every system call, Mach message, thread creation and signal the program makes. The source is about 114,000 lines of C11 and a little arm64 assembly, in 64 source files and 36 headers, built with nothing but the Xcode Command Line Tools. (On 4 October, at the commit this book was first written against, it was 76,178 lines in 30 source files and 28 headers; the difference is mostly the 32-bit runtime of Chapter 13, 24,669 lines of which 6,282 are Doug Lea's public-domain dlmalloc, vendored for its heap, and the rest is the x87 unit, the work for Wine and the JIT, and native mode.) It is free software under the GNU Lesser General Public License, version 2.1 or later.

The project was called Ocerz until 4 September 2026, when it was renamed AArchX. The rename touched the documentation and the version string; the binary is still `ocerz`, the source lives under `include/ocerz/`, and the several hundred environment variables that control it are all `OCERZ_*`. Running `./ocerz version` prints `AArchX 0.8`; it printed `0.3-dev` until the first numbered release, 0.4, on 5 October.

Why it exists

Rosetta 2 is closed, managed by the operating system, and on a deprecation track: the project's documentation is built around Apple ending general-purpose Rosetta after macOS 27. When it goes, so does the x86-64 copy of the system libraries that every Intel program on a Mac links against, because that copy is shipped by the Rosetta package. Intel-only software, from old command-line tools to games and their launchers, then has nowhere to run.

AArchX started for a narrower reason. Its first purpose, in June 2026, was to run Wine, and through Wine the Windows Steam client and its games, under a translator whose every decision could be read, instrumented and changed. Rosetta's behaviour can only be observed from outside; when a Wine process died inside it, there was nothing to bisect. A translator written in the open turns "Rosetta does something odd here" into "this function does this, and here is the commit that changed it". By September the project had become a general runtime for Intel macOS software, and native mode, described below, became its plan for the time after Rosetta.

> **KEY IDEA**
>
> AArchX is built around inspectability. Its interpreter is a complete reference implementation that every translation is checked against byte for byte; nearly every subsystem has switches that turn one optimisation off or print one kind of event; and every source file opens with a prose explanation of what it owns and why. The rest of this book leans on all three.

What it is not

- **Not a virtual machine.** AArchX emulates a processor and a user-space process, not a computer. There is one kernel, the Mac's own, and the guest's system calls are answered by AArchX on top of it, translating arguments and results where the two architectures differ.
- **Not Rosetta.** It never invokes Rosetta's translator and contains none of its code. In cache mode it does read the x86-64 shared cache file that the Rosetta package installs, because that file is where the Intel system libraries live; it maps and relocates that file itself.
- **Not a Windows layer.** Windows programs run through Wine, unmodified, and AArchX runs Wine the way it runs any other Intel program. What AArchX adds is the processor: x86-64, and i386 for 32-bit Windows code inside Wine's WoW64 process.
- **Not an ahead-of-time translator.** Code is translated when it is first run. Since 27 September 2026 those translations are kept on disk and reused by later processes, but nothing is translated before a program asks for it.
- **Not finished.** The project is experimental, version 0.8, with no stability guarantees. What runs is listed below and in Chapter 14; what does not is in Chapter 19.

Two ways to supply the system libraries

Translating a program's own instructions is only half the problem. A real program calls `printf`, `malloc`, `CFRelease` and `-[NSString length]` thousands of times a second, and those functions live in system libraries. AArchX can supply them in two ways, chosen once per process by a flag.

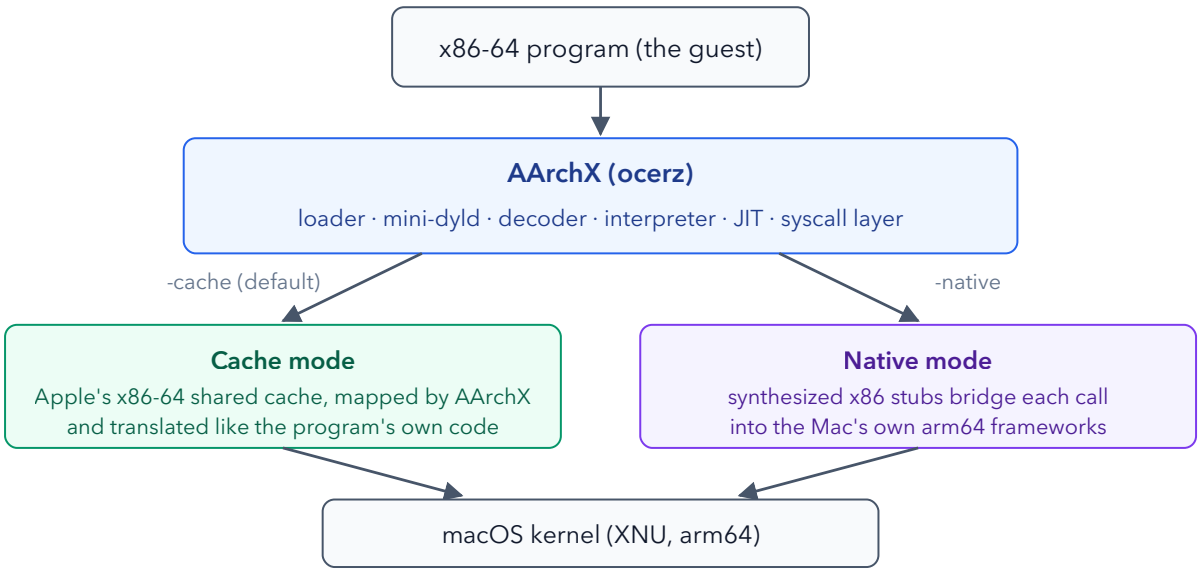


Figure 1.1. One translator, two sources of system libraries. Both modes translate the program's own code identically; they differ in what happens when it calls into the system.

Cache mode is the default. On macOS, the system libraries do not exist as separate files; they are pre-linked into one large "shared cache" per architecture, and the x86-64 one is installed with Rosetta. AArchX maps that file into the guest's address space, applies its relocations, registers its Objective-C classes and runs its initializers, and from then on the guest's `printf` is Apple's x86-64 `printf`, translated exactly like the program's own code. This is the mode that runs the most software, because the libraries are the real ones and behave exactly as they do under Rosetta. Chapter 11 describes it.

Native mode (`-native`) maps no x86 libraries at all. For each system library the guest links against, AArchX builds an x86-64 Mach-O image in memory whose exports are tiny stubs; calling one crosses into the host's real arm64 function, moving arguments between the two calling conventions on the way. A table generated from the macOS SDK says what each export's signature is. In this mode, `strlen` or `CFDictionaryGetValue` runs as native arm64 code at full speed, and no x86 system libraries need to exist on the machine, which is what makes native mode the answer to Rosetta's end. It runs a narrower set of programs so far. Chapter 12 describes it.

	Cache mode	Native mode
Flag	`-cache`, the default	`-native`, or `OCERZ_MODE=native`
System libraries	Apple's x86-64 shared cache, translated	the Mac's own arm64 frameworks, run as they are
Needs Rosetta installed	yes, for the cache file only	no
Library coverage	everything in the cache	59 libraries with a generated API database on 4 October 2026; 179 are listed in `tools/sdkgen/libraries` now, and 32 further databases, generated as i386, serve 32-bit programs
Cost of a library call	translated x86 code	a crossing of about 17 ns, or 2 ns for eleven string and memory routines

	Cache mode	Native mode
Runs	the most software	command-line programs, Cocoa apps including older ones such as Transmission and HandBrake, the Steam client, and every 32-bit Mac program

The mode is fixed before the guest starts, holds for the whole process, and is inherited by every program the guest launches through `OCERZ_MODE`.

There is a third kind of guest that neither mode was built for: a 32-bit Mac program, an executable with an i386 slice and no x86-64 one. macOS stopped running these in Catalina and Rosetta never ran them, so until 10 October 2026 AArchX refused them as well. Since commit `0ca24d64` they run through **m32**, which arrived from the MacShack project written by Jfishin, in native mode against the Mac's own arm64 frameworks, in a flat 4 GB window the guest never leaves, with a second set of API databases, generated from the SDK as i386, that say how each 32-bit call crosses. An i386 program is put in native mode by default; since 18 October (`1ae3140a`) that needs no flag. Chapter 13 describes it.

What runs today

"Runs" here means the program did its job: a command-line tool printed what it prints natively, an application drew its main window and stayed up, a game reached its menus. Everything below was seen working on the date given; Chapters 13 to 15 tell each story in detail.

Software	Result	When
Command-line tools	16 of 16 checked tools match native output byte for byte, on macOS 26.7.1 and 27; a sweep of 74 system tools agrees with Rosetta	2026-10-03
Bundled Cocoa apps	Chess (with its engine), Calculator, Dictionary, Font Book, Grapher, Digital Color Meter, Activity Monitor, Console, TextEdit, Preview, Script Editor open their windows	2026-09-11, macOS 26.6
Safari	runs and browses	2026-09-21
macOS Steam client	full UI in cache mode with software rendering; in native mode on macOS 27 with its GPU process on hardware graphics	2026-09-12, 2026-09-21
Discord	the Electron client reaches its signed-in view	2026-09-22
Brawlhalla	an Adobe AIR game, launched from Steam, reaches its menus	2026-09-22
Ollama	command line, HTTP server and menu-bar app	2026-09-13
Wine	boots in about 14 s; 32-bit Notepad and WineMine through WoW64; Windows Steam reaches its main window; 7-Zip 25.01 compresses on Wine Staging 11.18	September 2026
R.E.P.O.	a Unity game under Wine with DXMT renders: roughly 45-65 frames a second on its main menu, varying from run to run, and about 50 in a level, where Rosetta manages about 178 on the menu	2026-10-03, 2026-10-04
Older Intel apps, native mode	Transmission, Sequel Pro, HandBrake and VLC 2.2 open their windows	2026-10-16
Batman: Arkham Asylum, a 32-bit Mac game (m32)	plays with sound, launched from Steam and signed in to it: about 31 frames a second on an M4 Max when m32 landed, about 61 after the 32-bit speed work that day	2026-10-10

Software	Result	When
Other 32-bit games from Steam (m32)	Call of Duty: Modern Warfare 3 and Left 4 Dead 2 play	2026-10-12, 2026-10-13
Free 32-bit games (m32)	AssaultCube, SuperTuxKart, Neverball and Neverputt, Battle for Wesnoth, Teeworlds and Nexuiz play, with sound	October 2026
32-bit Mac apps (m32)	Adium, VLC 1.1.12 and Kega Fusion open their windows	2026-10-16, 2026-10-18
Counter-Strike 2	under Wine with D3DMetal, reaches its own window, then stops on an error Rosetta does not hit	2026-09-26

On macOS 27 the bundled applications in the second row ship as arm64 only, with no Intel code in them, so there is nothing for a translator to run; AArchX says so and exits, and a guest that launches one gets the native program, as it would under Rosetta. Safari, Steam, Discord and the games still carry Intel code on macOS 27.

How fast

On the project's fifteen-kernel benchmark, measured on an Apple M5 under macOS 27 on 21 September 2026, the build linked against libSystem runs faster than Rosetta on fourteen of the fifteen kernels in cache mode, from 0.72x of Rosetta's time on small `memcpy` calls to 0.99x on integer division; the one loss is a loop of tiny function calls at 1.03x. A freestanding build that makes no library calls wins eleven and loses four, none by more than five percent. AVX2 and FMA loops mostly beat Rosetta too. Chapter 17 has every number and the method.

The weak spot is start-up. Everything a large program runs has to be translated the first time it runs, and a Windows Steam session under Wine is a dozen processes running the same system and Wine code. Its main window took about 33 seconds under AArchX on 27 September, against 11 to 12 under Rosetta; the on-disk translation cache added that week is the main lever on that number.

Since the first edition of this book the translator itself has become faster in four places that the benchmark above does not show. The x87 unit was rewritten and given a JIT on 7 October (commit `6c6679f0`, issue #6): the project's x87 benchmark kernel took 1.147 s before, 0.161 s after and 0.731 s under Rosetta, on an M2 Max. The cost of translating dropped: a cold start of the system's perl loading ten modules, with the translation store off, spent about 275 ms in translation on the morning of 13 October on a Mac Studio and 208 ms after `f4ef8900`, and an empty CoreFoundation program, which carried 505 images until `85b248fc` because AArchX followed links dyld leaves out, launched in 133 ms, then 67 ms, then 42 ms by `16f65d59` on 9 October, against 9 ms under Rosetta (commit messages). A callback from native code into guest code, a `qsort` comparator for instance, fell from 536 ns to 100-112 ns on 18 October (`ad51f4b3`). And 32-bit Batman, at 31 frames a second when m32 landed, ran at about 61 on the same day.

The costs that remain are visible in one measurement taken on 14 October on a MacBook Air M5: 7-Zip 25.01 under Wine Staging 11.18, one thread, compressed at about 8,200 KiB/s under AArchX against 11,600 under Rosetta, and a 26 MB archive took 6.85 s against 4.8 s. Seventy percent of that compression is LZMA's binary-tree match finder. Ordered memory, which Wine always needs, costs that kernel about 17% against plain memory, and the archive came out identical either way. Split apart, in 7-Zip's own

benchmark the acquire loads cost 14% and the release stores 2.6%, but in the real run the barriers cost only 1 to 3%, and most of the rest is code generation, because the ordered load and store forms of arm64 take no index register. Chapter 17 follows that lead.

Installing it

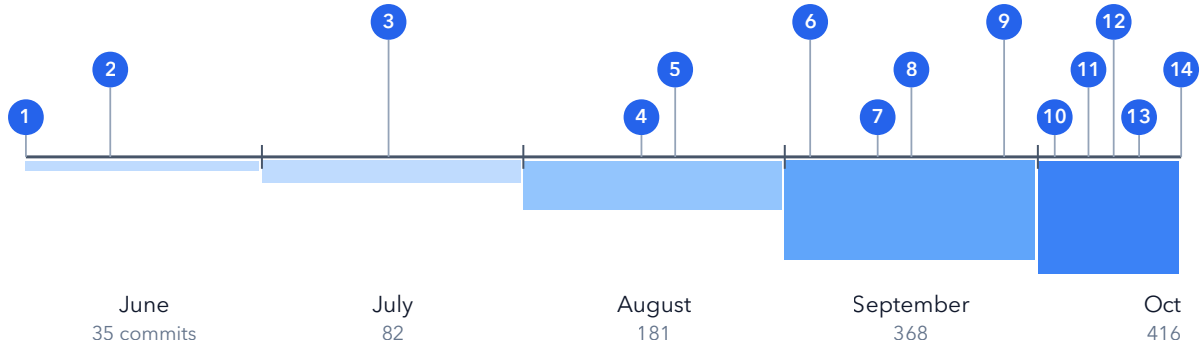
For its first four months AArchX was something you built: `make` with the Command Line Tools, then `./ocerz`. In the second week of October, with release 0.7 (`9a4da5bf`, 13 October), it gained ways to install it without Xcode. Releases themselves began as version bumps: 0.4 on 5 October (`de872c96`), 0.5 on 6 October (`33e00a13`), 0.6 on 7 October (`7fc00672`), 0.7 on 13 October, three patch releases that day (0.7.1 `1c4811dc`, 0.7.2 `2206cc7a`, 0.7.3 `1a316328`), and 0.8 on 18 October (`a454f2fd`).

- **The prebuilt binary and the one-line installer.** `curl -fsSL https://mont127.github.io/AArchX/install.sh | sh` (the script is `docs/install.sh`, from `3bf6e6aa`) downloads the latest release's `aarchx-macos-arm64.tar.gz`, checks it against the `.sha256` beside it, unpacks it into `~/.aarchx` and adds `~/.aarchx/bin` to `PATH` in the shell profile. Because `curl` does not quarantine what it fetches, Gatekeeper does not stop the unsigned binary. Running it again updates, and `--uninstall` removes the install and the profile line. `ocerz` finds `runtime/` through `realpath`, which is why a symlink in `bin` works.
- **The Homebrew tap.** `brew install --cask mont127/aarchx/aarchx` installs the same prebuilt from the `mont127/homebrew-aarchx` tap (`962f9c7c`); the cask strips the quarantine flag that Homebrew sets on its downloads.
- **AArchX.app.** A small Objective-C program, `app/AArchX.m`, built by `make app` with no Xcode project and signed ad hoc (`2dce5f07`, `a246dc47`). An Intel program reaches it by being dropped on its window or Dock icon, through Open With in Finder, or through Choose, and runs as `ocerz [-native] -- <path>` with its output under `~/Library/Logs/AArchX`; a 32-bit program goes to native mode by itself. It keeps a *library*, a grid of tiles for the user's programs, and runs a Windows `.exe` or `.msi` through a Wine build it finds (WineHQ's, Whisky's, the Game Porting Toolkit or one in Homebrew's `bin`), in cache mode and a prefix of its own. The installer puts it in `~/Applications` (`b1c999d4`), and `AArchX.zip` is a self-contained copy for people who never use Terminal; it is not notarized, so macOS refuses it the first time.
- **CrossOver bottles.** When CrossOver is installed, the app's *CrossOver Bottles* sheet ticks bottles over to AArchX one at a time by setting the bottle's documented `[Wine] BinPath` to two launcher scripts that run CrossOver's own `wineloader` and `wineserver` under `ocerz`, or as they are if `ocerz` is gone; nothing inside CrossOver.app changes (`a246dc47`, `e3a26326`).

Cache mode still needs Rosetta installed, for the cache file alone; the 32-bit Steam bridge needs a build from source (`docs/32-bit-steam-games.md`).

History

The first commit is dated 3 June 2026. By 18 October the history held 1,082 commits, 210 of them the routine follow-up that refreshes the committed unit-test binaries (the first edition of this book counted 698 commits at 4 October). Commits per month: 35 in June, 82 in July, 181 in August, 368 in September, and 416 in the first eighteen days of October.



1 First commit Jun 3
2 wineboot runs Jun 13
3 First Cocoa apps Jul 16
4 Full register pinning Aug 15
5 i386 and WoW64 Aug 19
6 Renamed AArchX Sep 4
7 macOS Steam full UI Sep 12
8 Native mode Sep 16
9 Translation cache Sep 27
10 R.E.P.O. renders Oct 3
11 x87 rewrite and JIT, 0.6 Oct 7
12 m32: 32-bit Mac programs Oct 10
13 Installer and app, 0.7 Oct 13
14 Release 0.8 Oct 18

Figure 1.2. Selected milestones from 3 June to 18 October 2026, numbered along the axis and listed below it. The bars under the axis show commits per month; September and the first eighteen days of October together hold more than 70% of the history.

June: from static binaries to Wine

The first version ran statically linked x86-64 programs: a Mach-O loader, a decoder, an interpreter and a small call-threaded JIT that inlined only moves and called back into the interpreter for everything else. Within days it was running dynamically linked programs against the shared cache with its own dynamic linker: real Objective-C programs on 5 June, guest threads and the C++ standard library on 6 June, `fork`, `posix_spawn`, shells and the core utilities on 7 June, thread-local variables on 8 June, and delivered guest signals on 11 June. On 13 June Wine's `wineboot` ran end to end into the GUI layer, and the next day Notepad built its GUI.

July and August: speed, threads and 32-bit code

July made the memory model honest (x86's ordering guarantees on 16 July) and ran the first real macOS applications. On 17 July the JIT gained register allocation and deferred flags. August rebuilt it: on 15 August every guest general-purpose register and every `xmm` register became permanently pinned in a host register, the next day blocks could continue past conditional branches as superblocks, and on 17 August ordered memory became cheap enough to use by default once a program has a second thread. On 19 and 20 August the decoder, interpreter and JIT learned i386, with a 32-bit differential gate, so that 32-bit Windows code inside Wine's WoW64 process could run.

September: Steam, native mode and the world outside

September is more than half the history. The project was renamed AArchX on 4 September, and its license moved from LGPL-2.1 to a proprietary non-commercial license the next day. On 8 September the Windows Steam client's login window appeared under Wine, rendered with software compositing. On 12 September the macOS Steam client came up with its full UI, and the source tree adopted its rule that each file explains itself in one prose block at the top, with no comments in the code bodies. AVX2 and FMA reached the JIT on 13 September. Native mode began on 16 September and ran the Steam client five days later, the same day the in-place string routines and hardware NaN handling put the dynamic benchmark ahead of Rosetta on fourteen kernels. On 22 September a bug in the loaded-image list was found to have hidden the GPU from every translated program, and the same day Brawlhalla and Discord ran. On 23 September the license returned to LGPL-2.1-or-later and the documentation moved into the tree, and games launched from Steam began running under AArchX. Counter-Strike 2 reached D3DMetal under Wine on 26 September, and on 27 September translations started being kept on disk and shared between processes.

1 to 4 October

Some time between 19 September and 2 October the project's public history was rewritten (the rewrite kept every original commit date, so it cannot be dated more closely): the old and new histories share no commit, every hash changed, and the generated API databases were taken out of all of it. It was rewritten again on 14 October, with the same effect on the identifiers and none on the contents; every commit identifier in this book is the first eight characters of one from the current history. On 1 October Steam ran on stock Wine Staging 11.16 with the DXMT graphics layer. On 3 October cache mode learned to find the shared cache where macOS 26 keeps it again, an initializer-order bug that crashed `sw_vers` on macOS 26.7.1 was fixed by matching the order Rosetta's dyld uses, and R.E.P.O., a Unity game, rendered under Wine. On 4 October MMX instructions joined the JIT, a full code arena started being flushed instead of overflowing, and a cheaper address translation for Wine; together they took R.E.P.O.'s menu from 28-30 frames a second to roughly 45-65.

5 to 18 October: releases, x87, the Wine layout and 32-bit programs

The fortnight after the first edition's commit, 4 to 18 October, was the busiest in the history, 384 commits, and it went in five directions.

Native mode grew a long tail. On 4 and 5 October, native mode gained databases for 74 more libraries, Objective-C exceptions, Swift programs on a guest copy of the Swift runtime and the calls Wine, Steam and R.E.P.O. make; on 6 October its memory below 12 GB, which Wine's layout needs, and a D3D11 path through MacNdCheese's D3DMetal backend (`0aae84f0`), worked out from what the loader and bridge report and from return values. On 16 October it opened Transmission, Sequel Pro, HandBrake and VLC 2.2 (`334a5e1f`).

x87 was rewritten. A Rosetta comparison on 7 October found that register forms of x87 arithmetic computed wrong values (`fadd %st(1),%st` gave 20 for 10 + 3), comparisons always said equal and `fild` and `fistp` of 64-bit integers corrupted memory. `8f310eb8` made `src/x87.c` a unit of its own with exact 80-bit register images, and `6c6679f0` translated the common x87 instructions the same day, so that the x87 kernel took 0.161 s against Rosetta's 0.731 s (issue #6, release 0.6). 32-bit inlining went in with it.

The JIT was tuned against 7-Zip and the Wine layout. Across the fortnight some 95 commits with the subject area `jit` shaped how flags travel (a compare's NZCV reaches consumers up to eight instructions on, `adc` and `sbb` become `csinc`), how the stack pointer is kept, the hoisting of base checks in Wine's

low-window layout, self-modifying-code fixes for spliced callees and chains through veneers, and translation time itself (the decoder's one-lookup prefix classification and single-switch opcode map, `39271608`, the flag look-ahead that stops once every flag is written, a per-block NZCV producer table, one barrier pair per chain install). A wrong-code bug in a `dec`/`jnz` loop carrying a double was fixed on 13 October (`2cf48a7e`). On 14 October pending signals started reaching threads in translated loops (`a7fd0f3f`).

32-bit Mac programs arrived. m32 landed on 10 October (`0ca24d64`) and in the next eight days ran Batman: Arkham Asylum, Left 4 Dead 2 and Call of Duty: Modern Warfare 3 from Steam, a UPX-packed Kega Fusion, AssaultCube, SuperTuxKart, Neverball, Wesnoth, Nexuiz, VLC 1.1.12 and Adium. Chapter 13 describes how.

It became installable. Release 0.7 on 13 October came with the prebuilt binary, the installer, the Homebrew tap, AArchX.app and CrossOver bottles, described above, and 0.8 followed on 18 October. That week's last commits cut a callback's cost (`ad51f4b3`), limited the same-address fault-loop guard to Wine processes (`ff6ea313`) and gave every thread a cache for the questions an Objective-C send asks (`beed19af`).

> **NOTE**
>
> The project's two development machines worked on the tree in parallel through September and October, one on macOS 26 and an M5 on macOS 27. That is why several results in this book name the macOS version they were measured on, and why both versions are supported. The 32-bit work was done by a second contributor, Jfishin, on a separate machine and merged by pull request (#8, #9 and #12).

Where to go from here

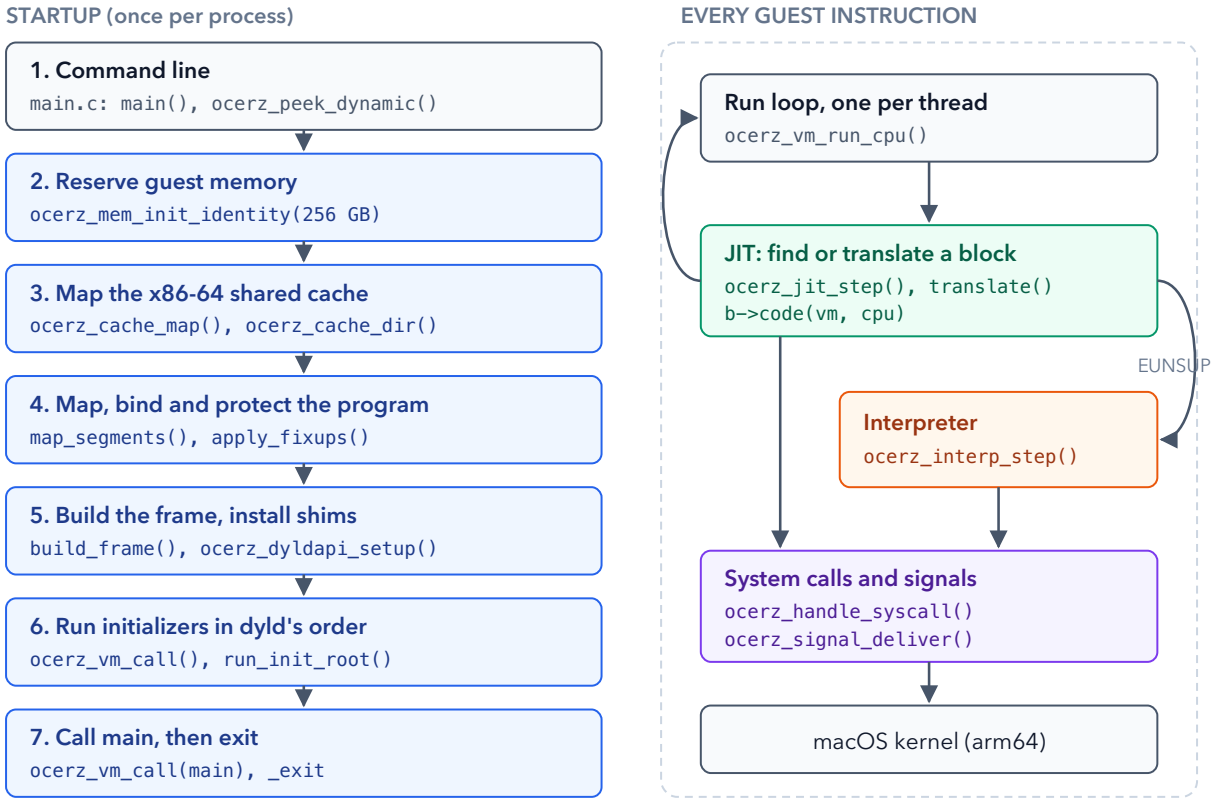
Chapter 2 follows one program from the command line to its exit and names the functions it passes through; it is the map for everything after it. Chapter 3 lays out the source tree and the conventions it is written in. From there, the chapters take the layers one at a time, from the instruction decoder up to native mode and, in Chapter 13, the 32-bit runtime built on it.

CHAPTER 2

The life of a program

This chapter follows one program, `./ocerz /usr/bin/sw_vers` in cache mode, from the command line to its exit, and names the function that does each step. It is the map the rest of the book fills in: every later chapter is a closer look at one stop on this route. The same program in native mode, and a statically linked one, close the chapter.

`sw_vers` is a good subject because it is small and ordinary. It is a universal binary with an x86-64 slice, it links only CoreFoundation and libSystem, it reads one property list and prints three lines. Yet getting it to start exercises the whole translator: a 256 GB memory reservation, the x86-64 shared cache with several thousand libraries in it, the initializers of a few hundred of them in the right order, a few hundred thousand translated instructions, dozens of system calls, and an exit.



Steps 6 and 7 do not leave the run loop: ocerz_vm_call() runs guest code through it and returns when the guest function does. Threads get a run loop of their own; signals are delivered at system-call edges; faults land in the handler that ocerz_vm_install_handlers() installed.

Figure 2.1. The route of a dynamically linked program in cache mode. Startup runs once; the right-hand loop runs for every block of guest code, on every guest thread.

1. The command line

Everything starts in `main()` in `src/main.c`, a 575-line file whose explanation block is about the three decisions it makes before anything else runs. Before parsing a single argument, though, it deals with two pieces of the environment that would otherwise do damage.

Standard error first. If `OCERZ_STDERR_FILE` names a file, standard error is reopened onto it in append mode. Chromium and others start their children with standard error closed or pointing nowhere, and a child that dies early under AArchX would otherwise leave no trace.

Then `DYLD_INSERT_LIBRARIES`. That variable is meant for the guest: Steam sets it to its loader and overlay for every game it launches. But the host's own dyld reads it first, and would load the arm64 slices of those libraries into `ocerz` itself, which broke startup within half a second. So `reexec_without_host_insertion()` moves the value to `OCERZ_GUEST_DYLD_INSERT_LIBRARIES`, removes the original, and re-executes `ocerz` with `execv`. Because it is an exec rather than a fork, the process ID Steam is tracking survives. In the clean copy, `restore_guest_insertion()` puts the variable back before the guest's environment is captured, so the guest sees it exactly as it was set and the mini-dyld later inserts the libraries into the guest alone.

Argument parsing is plain: `-v` (repeatable), `-trace`, `-strace`, `-no-jit`, `-native`, `-cache`, `-path file` and `--`, with the first non-option argument being the program. Any other option prints the usage and exits with status 64. `./ocerz version` prints `AArchX 0.8` and stops (the string is `OCERZ_VERSION` in `include/ocerz/version.h`; it read `0.3-dev` when this book was first written, on 4 October 2026, and the releases since are `de872c96`, `33e00a13`, `7fc00672`, `9a4da5bf`, `1c4811dc`, `2206cc7a`, `1a316328` and, on 18 October, `a454f2fd`, which made it 0.8). `main()` also sets `MallocNanoZone=0` for the process before the guest's environment is taken.

The **mode** comes from the last of `-native` or `-cache` on the command line; with neither, `OCERZ_MODE` decides, and a value other than `native` or `cache` is refused with exit 64 rather than ignored. That variable is how a child inherits its parent's mode across a spawn or an exec. `ocerz_afp_enable()` then checks whether the processor reports `FEAT_AFP`, the arm64 feature that lets the hardware produce x86's NaN results (Chapter 7).

Three more adjustments can change what will run. If the program is an application bundle, `bundle_executable()` reads `CFBundleExecutable` from `Contents/Info.plist` (or falls back to the bundle's own name) and runs `Contents/MacOS/<name>` instead, with that path as the guest's `argv[0]`; that is how Steam's `ocerz %command%` launch option works for games. If the program is one of Wine's loaders, `apply_wine_defaults()` marks the process as Wine's, which fixes the base of its memory arena (Chapter 8), and sets the default list of Objective-C images to preload. A loader is recognised by its real path ending in `wine` or `wine64`, and, since `74e6140d` (13 October 2026), also by carrying a `WINE_RESERVE` segment, the range Wine's preloader keeps for Windows. The name alone failed under CrossOver, which starts each process through a hard link named after the Windows program: `realpath` keeps a hard link's name, so its `cmd.exe` started with the ordinary memory layout and died reserving the low-shadow base its Wine parent had handed down. `OCERZ_NO_WINE_SEGMENT_DETECT=1` goes back to the name alone. One further adjustment is made for a single program: when the guest is a binary named `Steam Helper`, `mock_keychain_argv()` adds `--use-mock-keychain` to its arguments unless it has one already or `OCERZ_NO_MOCK_KEYCHAIN` is set. And three variables single out one process of a large tree by a substring of its command line: `OCERZ_EXE_ENV` sets variables in it, `OCERZ_STRACE_EXE` traces its system calls, and `OCERZ_NOJIT_EXE` runs it under the interpreter.

Two more steps sit between the bundle and the classification, both added in October 2026. First, `run_launch_script()` checks whether the file is a script, that is, starts with `#!`. Steam's `%command%` for a Source game on the Mac is `hl2.sh`, which sets `DYLD_LIBRARY_PATH` and starts the game as `${GAME_DEBUGGER} <game> <args>`. Until `46fade2e` (13 October) ocerz took the script for a program and stopped with "no x86_64 slice", so Left 4 Dead 2 needed a launch option of its own; since then a script that expands `GAME_DEBUGGER` runs as itself with that variable naming this `ocerz` and the flags that preceded the script, so `ocerz -native %command%` serves those games as it serves the rest (a `GAME_DEBUGGER` already set wins, and an `ocerz` whose path contains a space is refused with a message, because the script expands the variable unquoted). Any other script, such as the one that is `Nexuiz.app`'s executable (it sets `DYLD_LIBRARY_PATH` and then execs the game), is run since `88b7e20c` (18 October) on its interpreter's x86-64 slice under this `ocerz`, with the same flags, by an `execv` of `ocerz` itself; run natively, `/bin/sh` had handed the i386 game to the kernel, which refused it with "Bad CPU type in executable". An interpreter with no x86-64 slice runs the script natively, as before. Second, a program with no x86-64 slice that is an i386 Mach-O file is put in native mode unless the command line or `OCERZ_MODE` says otherwise. That default came with `1ae3140a` (18 October): an i386 program has no x86 shared cache to map and crosses into arm64 frameworks on every call, and in cache mode the hardware NaN results cost two FPCR writes per call, about 17 ns on an M5, more than the checks they save (Nexuiz's benchmark demo took 8% more cycles).

Finally `ocerz_peek_dynamic()` reads just enough of the file to classify it. It returns -2 for a file with no x86-64 slice, a negative value for a file it cannot read (exit 65), 0 for a static image and 1 for one that names a dynamic linker. On -2, `m32_is_i386()` decides: an i386 Mach-O file goes to `m32_run()`, described in the section on 32-bit programs below and in Chapter 13, and anything else exits 64 with "it is not an Intel program, so there is nothing to translate". Native mode refuses a static image (exit 64), since there is nothing for it to bridge. `sw_vers` is dynamic, so `main()` hands it to `ocerz_dyld_run()` in `src/dyld.c`, together with a private copy of the environment made by `env_snapshot()`.

> **HOW IT WAS FOUND**
>
> The environment is copied because of a bug that depended on which shell launched `ocerz`. In native mode, loading the guest opens the host's real CoreFoundation, whose initializer calls `setenv`, which reallocates `environ` and frees the array a pointer taken earlier still named. The guest's stack was built from that pointer afterwards, and whether the stale read crashed depended on whether the freed block had been reused, which it was when the environment happened to be the size bash passes and not the size zsh passes. The copy owns its strings and lives as long as the process.

2. Reserving guest memory

The first thing `ocerz_dyld_run()` does is `ocerz_mem_init_identity(DYN_ARENA_SIZE)`, with `DYN_ARENA_SIZE` defined as 256 GB. This reserves one contiguous region of host address space for everything the guest will map, starting at 12 GB (`0x300000000`), and it is an *identity* map: a guest address and the host address behind it are the same number. Every guest load and store can then use its address directly, with no base to add, which is the fastest case the JIT has.

The arena is reserved, not committed: it costs address space, not memory. 256 GB is not a typo. Chromium's PartitionAlloc reserves 32 GB at startup, and JavaScriptCore's Gigacage asks for a single 128 GB mapping; the arena grew from 32 GB to its present size for those two. A Wine process puts the arena at a fixed

base instead of wherever the host would place it, and a Wine image that insists on low addresses gets a second map, the low shadow, for its first 12 GB. Chapter 8 explains both, and the 4 KB-guest-pages-on-16 KB-host-pages machinery underneath.

3. Mapping the shared cache

In cache mode the guest's system libraries come from Apple's x86-64 shared cache, and `ocerz_cache_map()` in `src/cache.c` maps it. `ocerz_cache_dir()` first decides where it is: macOS 27 keeps it in the Rosetta cryptex, `/System/Volumes/Preboot/Cryptexes/Rosetta/System/Library/dyld/`, and macOS 26 in the OS cryptex, `/System/Volumes/Preboot/Cryptexes/OS/System/Library/dyld/`. The Rosetta location is tried first, and the main file and all its numbered subcaches are taken from whichever directory holds a cache. If neither does, which normally means Rosetta is not installed, the process stops with "cannot find the x86_64 shared cache".

The main file and its subcaches, `dyld_shared_cache_x86_64`, `.01`, `.02` and so on, are mapped at the cache's preferred base, `0x7FF800000000`, which lies above the arena in host space. The cache's data regions store every pointer in a packed slide-info format that has to be unpacked even when the cache is not slid; unpacking about 500 MB of it eagerly cost about 200 ms per process, so those regions are mapped with no access at all and each 16 KB page is unpacked the first time anything touches it, from the fault handler. The cache's own header then gives the image table: several thousand libraries, from libSystem to AppKit, all present in every process from this point on. Chapter 11 has the details.

4. Mapping, binding and protecting the program

`read_file()` reads `/usr/bin/sw_vers`, `select_slice()` picks its x86-64 slice out of the universal file, and the path is made absolute with `realpath`, because programs derive things from it (Steam derives its bundle root from its executable's path, and a relative one broke it). `map_segments()` lays the segments out in the arena, read-write for now so that the copy and the fixups can land. The image must have an entry point, either `LC_MAIN` or the older `LC_UNIXTHREAD`, or loading stops.

Dependencies come next. `collect_rpaths()` gathers the image's `LC_RPATH` entries and `load_disk_deps()` loads any dependency that is a file on disk rather than a library in the cache, recursively. `sw_vers` has none: CoreFoundation and libSystem are both in the cache. Then `apply_fixups()` walks the image's chained fixups, rebasing internal pointers by the image's slide and binding every import. A cache import is bound in the specific library the image named, following re-exports, rather than by searching every cache image for the name; the difference matters when two libraries export the same symbol, which is how `openssl` once reported the wrong version. A weak-definition lookup, the kind a C++ program makes for its own `operator new` or a template instance, takes the first image in load order that has weak definitions and exports the name, and the main executable comes first; AArchX asked the shared cache first until `230da272` (9 October 2026), so a program defining its own `operator new(size_t, nothrow_t)` got libc++'s. The same commit made the weak-bind stream of an older binary a run of weak-definition lookups instead of 1,743 searches across 3,900 cache images: `ffmpeg -version` fell from 0.71 s to 0.11 s (Rosetta 0.028 s). An older image whose imports are described in the classic `LC_DYLD_INFO` form goes through `apply_classic_fixups()` instead. Last, `protect_ro_segments()` gives the image its final protections; a segment left writable would be treated as possibly self-modifying code (Chapter 7).

5. The initial frame, and the pieces the guest expects

`build_frame()` builds the guest's initial stack in an 8 MB stack region the way XNU's exec path lays it out: every argument and environment string, then `argv`, `envp` and the `apple` array as one contiguous ascending run, each terminated by a null pointer. The contiguity is load-bearing, because libSystem finds `apple[]` by walking off the end of `envp`. The frame also holds a `ProgramVars` block pointing at libdyld's own `NXArgc`, `NXArgv`, `environ` and `__progname`, so the program's `environ` and libc's `_NSGetEnviron()` are one variable, as on a real system.

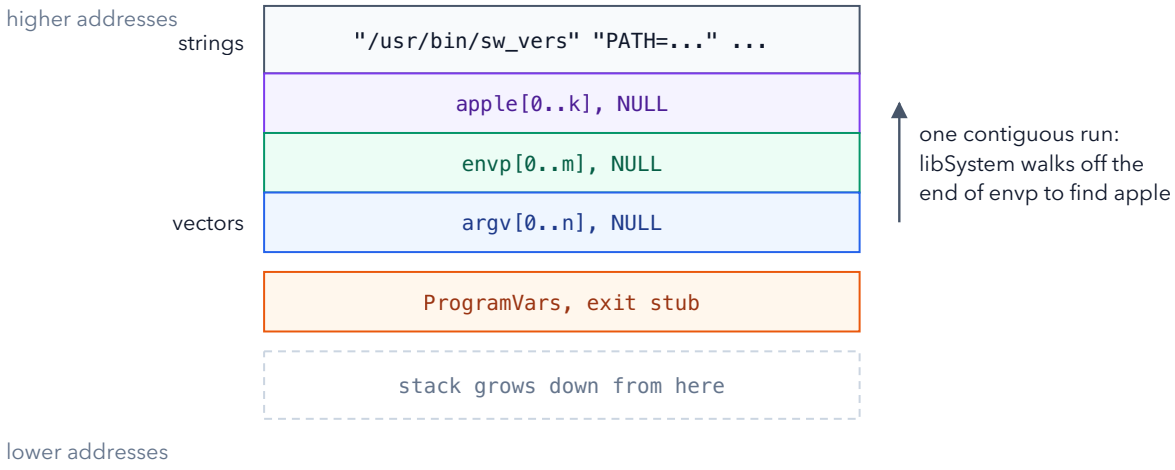


Figure 2.2. The guest's initial stack in dynamic mode. Every string and vector entry is counted and stored; an earlier version capped the environment at 60 entries, which silently dropped the marker Wine appends to stop its loader re-executing, and re-executed every Wine process forever.

Then the pieces a program expects to find already in place. A 32 KB guest thread block is mapped for the main thread, and the guest's `gs` base is pointed into it, because x86-64 macOS code reaches its thread-specific data through `gs`. `ocerz_vm_install_handlers()` installs the host signal and fault handlers that everything later depends on (Chapter 9). `ocerz_commpage_init()` builds the emulated commpage, the small read-only page of system information macOS maps into every process at a fixed address. In cache mode `ocerz_dyldapi_setup()` installs AArchX's stand-in for the object that libdyld's API functions call into: `_dyld_image_count`, `dlopen`, the Objective-C runtime's image callbacks and the rest reach it through a table whose entries point into a reserved window of guest addresses, `0xdda00000` to `0xdda10000`, and AArchX answers each by its position in the table (Chapter 11). Last, libdyld's `environ`, `__progname`, `NXArgc` and `NXArgv` variables in the cache are pointed at the frame.

6. Initializers, in dyld's order

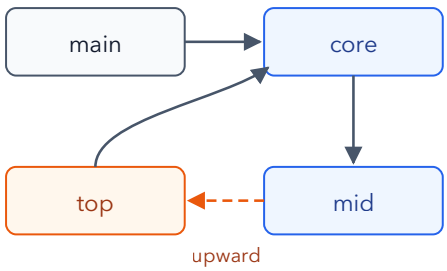
Before `main` can run, every library the program depends on must have run its initializers, and in the right order. This is, in the words of `dyld.c`'s explanation block, "most of the file". Each guest function runs through `ocerz_vm_call()`, which sets up a guest call with a sentinel return address, enters the same run loop that runs everything else, and returns to C when the guest function returns.

1. **libSystem first.** `find_dylib_init()` finds libSystem's initializer in the cache, and it runs with five arguments: `argc`, `argv`, `envp`, `apple` and the `ProgramVars` pointer. It sets up malloc, libdispatch, the Objective-C runtime and the rest of the base system. When it returns, `init_mark_done_closure()`

marks libSystem's whole dependency closure as initialized, which keeps the later walk from descending through it.

2. **Thread-local variables.** `ocerz_tlv_register_closure()` registers every image's `__thread` variables.
3. **The eager set.** For a program whose closure reaches CoreFoundation, `compute_eager_set()` decides which images must be initialized now: those the program reaches, closed under dependencies. An image in the set whose dependency was left out would initialize on top of an uninitialized library. Two changes of 9 October 2026 shaped what is in it. A link marked `DYLIB_USE_DELAYED_INIT` (macOS 15) is not part of the launch: dyld maps and binds the image but keeps it off the image list, runs none of its initializers and does not tell the Objective-C runtime about it, unless an ordinary link reaches it as well, and a client that needs it `dlopen`s it first. AArchX followed delayed links like any other, and on macOS 27 a program that links only CoreFoundation had 505 images and ran their initializers where it has 346 under Rosetta; since `85b248fc` those links stay out of the image list, the eager set, the initializer walk and the upward links, the image list is identical to Rosetta's image for image, and `OCERZ_NO_DELAY_INIT=1` follows them again. And `16f65d59` stopped the pointer scan that finds links the walk did not follow from reading 830,000 pointers of every eager image: the cache builder binds a cache image only to the images it links, so an image whose links all resolved cannot lead anywhere the walk has not been, and only images not in the cache and images with an unresolved link are scanned (`OCERZ_SCAN_ALL_USES=1` scans everything). An empty CoreFoundation program launched in 133 ms before the first change, 67 ms after it and 42 ms after the second, against 9 ms under Rosetta (commit messages, 9 October 2026).
4. **The walk.** `run_load_phase()` delivers the Objective-C `+load` notifications for libSystem's closure, and then `run_init_root()` walks from the main image: per image, bottom-up, every dependency finished first, then that image's `+load` methods, then its own initializers. Neither global order works: running every `+load` first, or every initializer first, both broke real programs.
5. **Inserted libraries.** If `DYLD_INSERT_LIBRARIES` was set, `load_inserted_libraries()` opens each named library with the guest's own `dlopen`.

Upward dependencies are the subtle part. An *upward* link (`LC_LOAD_UPWARD_DYLIB`) is how a library declares the back edge of a dependency cycle: CoreFoundation links Foundation upward, although Foundation depends on CoreFoundation. An upward link is an ordering edge for nothing, so the walk does not follow it on the way down. But its target still has to be initialized at some point, and *when* turned out to matter.



Initializer order

Rosetta's dyld	`mid core main top`
AArchX before	`mid top core main`
AArchX now	`mid core main top`

Before the fix, top ran right after mid, before core, its own dependency. Now upward targets wait until the whole walk from the program has finished.

Figure 2.3. The shape tested by `tests/dynamic/upward_defer.c`: core depends on mid, mid links top upward, and top depends on core. Each initializer logs its name; the expected order was measured under Rosetta.

Until 22 September 2026 AArchX did not initialize upward targets at all, and on macOS 27 that broke `sw_vers`: Foundation's initializer never ran, `NSString` stayed an abstract class cluster, and the first `+[NSString stringWithFormat:]` recursed until the 8 MB guest stack ran out. The fix initialized each upward target right after the library that declared it. On macOS 26.7.1 that order broke the same program the other way: there, libobjc links libswiftCore upward and libswiftCore links Foundation upward, and libobjc is itself a dependency of CoreFoundation, so Foundation's entire subtree initialized while CoreFoundation was still unfinished on the recursion stack, and an allocation through CoreFoundation's allocator recursed between `malloc` and `malloc_zone_malloc` until the stack ran out. Since 3 October each upward target goes on a list owned by the root call and is initialized after the whole walk has finished, the order Figure 2.3 shows Rosetta's dyld using.

7. The first block

With the initializers done, `ocerz_dyld_run()` calls `main` through `ocerz_vm_call()` with `argc`, `argv`, `envp` and `apple`. Like every guest function, it runs in `ocerz_vm_run_cpu()` in `src/vm.c`, the run loop. On entry the loop registers the CPU, clears the host thread's signal mask (AArchX emulates the guest's mask itself, and the host thread must never block asynchronous signals), and sets a recovery point with `sigsetjmp`. Then, until the guest exits or the call returns, it first delivers any signal that is pending and not blocked (Section 10) and then repeats one decision:

```
while (!vm->exited && !cpu->terminated && !cpu->interrupt) {
    ocerz_vm_suspend_point(cpu);
    ...
    } else if (vm->jit_enabled && (vm->jit || (vm->jit = ocerz_jit_create(vm)))) {
        r = ocerz_jit_step(vm, cpu);
        if (r == OCERZ_EUNSUP)
            r = ocerz_interp_step(vm, cpu);
    } else {
        r = ocerz_interp_step(vm, cpu);
    }
    ...
}
```

`ocerz_jit_step()` in `src/jit.c` looks the guest `rip` up with `cache_lookup()`, keyed by address and by whether the code is 32- or 64-bit. The first time, there is nothing there, so it takes the translation lock and calls `translate()`. That function first checks whether this address must stay interpreted (a page that keeps being rewritten, a range named by a debugging switch), then asks the on-disk translation cache whether another process already translated this code and the bytes it was made from are unchanged. If not, the decoder turns the bytes into `X86Insn` records until it reaches the end of the block, possibly continuing past forward conditional branches into a superblock, and the emitter produces arm64 code into the 1 GB code arena. Chapter 4 describes the decoder and Chapters 6 and 7 the translation.

A guest function that native code calls back into, a `qsort` comparator or a Cocoa delegate in native mode, takes the same route through `ocerz_vm_call()`, and its cost was once dominated by this entry. Until `ad51f4b3` (18 October 2026) each such callback made three system calls and copied the 8 KB `OcerzCPU`: it read the host signal mask to clear what a host thread had blocked, called `sigsetjmp(jb, 1)` so that the mask and the signal-stack state were saved for the rare jump back to the recovery point, and copied the whole CPU. Now only a thread's outermost guest loop clears the mask (a callback nested inside another is called by a native function that guest code called, which leaves the mask alone), the recovery point is

taken with `sigsetjmp(jb, 0)` and a recovery then empties the mask itself and takes the thread off the signal stack with the `UC_RESET_ALT_STACK` flag of `sigreturn`, which only clears that flag, as Apple's published XNU source defines it, and the callback's CPU copy leaves out the return-address stack (it starts empty) and the system-call ring, 2.2 KB instead of 8.2. A `qsort` of 200,000 integers with a guest comparator went from 536 ns to 100-112 ns a comparison; `OCERZ_NO_CHEAP_CALLBACK=1` restores the old entry.

A published block is a C-callable function, and running it is a single call: `b->code(vm, cpu)`. Its prologue loads the guest registers from the `OcerzCPU` struct into the host registers they are pinned to, all sixteen general registers and all sixteen `xmm` registers, for the whole time the guest runs in translated code. When the block ends with a branch to another translated block, that branch is patched into a direct arm64 branch into the other block's body, so a hot loop or call chain runs from block to block without coming back to C at all. Only when the next block is not translated yet, or something needs the dispatcher, does the epilogue store the registers back and return to the run loop, which looks up the new `rip` and goes round again. If `ocerz_jit_step()` answers `OCERZ_EUNSUP`, there is nothing it will translate here (the dyld API window is one such place), and `ocerz_interp_step()` executes one instruction instead: the interpreter is complete, so anything the JIT declines still runs.

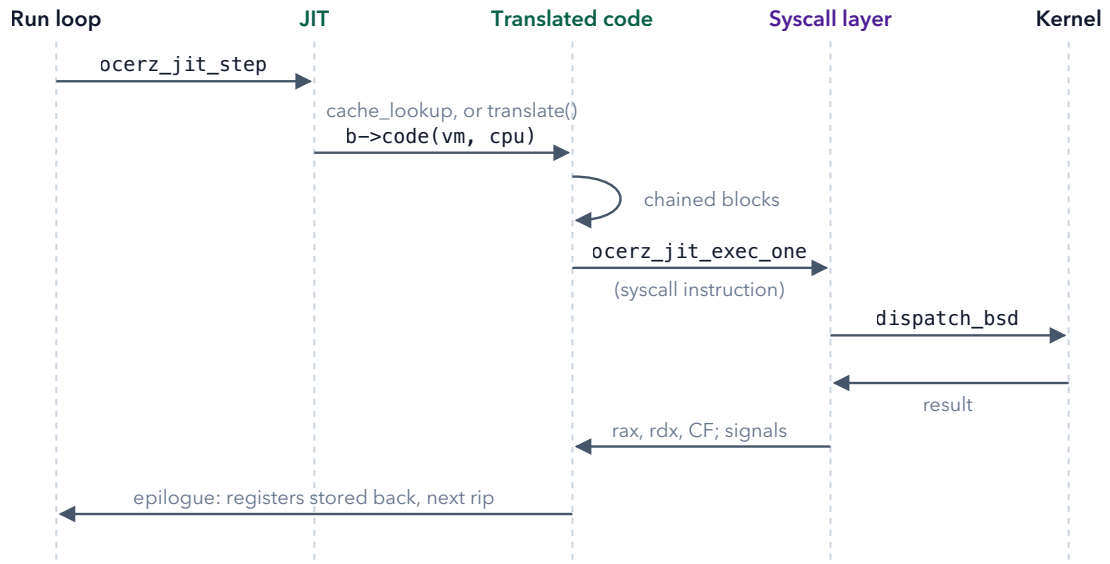


Figure 2.4. One trip through translated code. Most trips never reach the right half: blocks chain into each other and only come back for an untranslated target, an interpreter-only instruction, or a system call.

8. A system call

Sooner or later `sw_vers` opens `SystemVersion.plist`, and libSystem's x86-64 `open` executes a `syscall` instruction. Translated code does not try to emit anything for it: it calls out to `ocerz_jit_exec_one()`, which runs that one instruction through the interpreter, and the interpreter's `OCERZ_OP_SYSCALL` case calls `ocerz_handle_syscall()` in `src/syscall.c`. The 8,967-line syscall layer is the guest-to-host boundary.

On macOS the syscall number in `rax` carries a class in bits 24 to 31. `ocerz_handle_syscall()` splits it: class 1 goes to `dispatch_mach()` (Mach traps), class 2 to `dispatch_bsd()` (BSD system calls such as `open`), class 3 to `dispatch_machdep()` (machine-dependent calls such as setting the thread's segment base). Most BSD calls are passed straight to the host kernel through `sys_raw.h`, the one place AArchX enters the kernel directly; the rest are intercepted because an argument or result has to be rewritten, or

because AArchX must keep its own state in step (memory mappings, signals, threads, processes). The result comes back the way Rosetta returns it, which is not quite the way the kernel does: unix calls return through `rax` and `rdx` with the carry flag set on error, and the other arithmetic flags take the fixed pattern Rosetta leaves; machine-dependent calls touch `rax` alone, because Wine keeps a frame pointer in `rdx` across one of them. Pending signals are offered both on the way in and on the way out (Section 10). Then the interpreter returns to the translated block, which carries on.

One call takes a different path. A `fork` is never performed from inside translated code: `jit_exec_one()` recognises it and returns `OCERZ_EUNSUP`, so the run loop performs it from a clean state. And a blocking call made from translated code is recorded as *parked*, so that a code-arena flush (Chapter 7) knows the thread is safely outside the arena while it waits.

9. A second thread

`sw_vers` itself is single-threaded, but libSystem's initializer and libdispatch start threads in nearly every Cocoa program, so consider one. `pthread_create` in the guest's libc ends in the `bsdthread_create` system call, and `sys_bsdthread_create()` answers it by creating a real host thread with an `OcerzCPU` of its own and its own guest stack and thread block, which then runs `ocerz_vm_run_cpu()` from the guest's start routine. Guest threads are host threads; they run in parallel on real cores.

That has one process-wide consequence. A program with a single thread cannot observe the difference between x86's strong memory ordering and arm64's weaker one, so AArchX starts every process with plain loads and stores. The first sign of a second observer (a thread, a `fork` or spawn, a workqueue worker, a writable shared mapping or an attached System V segment) calls `ocerz_jit_require_ordered()`, and from then on, for the rest of the process's life, scalar memory accesses are translated with acquire and release semantics. Chapter 7 explains what that costs and what it deliberately leaves plain.

libdispatch's worker threads are created by the kernel rather than by `pthread_create`; the host workqueue bridge hands those requests to the host kernel and gives each worker a guest personality when it starts running guest code (Chapter 9).

10. A signal

Suppose the guest installs a handler with `sigaction` and something sends it `SIGUSR1`. The guest's handler table, mask and alternate stacks are AArchX's own state, kept by the syscall layer; plain asynchronous signals are also mirrored onto the host, because the host kernel decides, for instance, whether `SIGPIPE` kills the process. When the host signal arrives, AArchX's handler records it as pending on the CPU of the thread it arrived on, and nothing more. For the first four months it was delivered only at the next system-call edge (on entry, with `rip` rewound so the call runs again afterwards, or on return) or bridged call, and nowhere else, so a thread spinning in a loop never saw it: a loop of `dec` and `jnz` with a 5 ms `ITIMER_REAL` got no `SIGALRM` until it ended, and under Wine, where `SuspendThread` is a `SIGUSR1` sent to the thread, Mono's collector waited for any busy thread to finish its loop. Since `a7fd0f3f` (14 October 2026) both run loops also deliver a pending, unblocked signal between two steps, where all the guest state is in the CPU, and for translated code the handler helps the loop reach that point. Translated code returns to the run loop only when a block exits to it, so the handler calls `ocerz_jit_signal_stop()` with the interrupted host pc: it does to the block the thread is in, and to the blocks its chained edges lead to a few hops on, what retirement does, unchaining their edges and rewriting their stop sites, so the next jump out

of any of them goes to the dispatcher. The dispatch stub, which enters the next block without returning to C, goes back to the run loop while such a signal is pending, and `ocerz_jit_signal_unstop()` restores the blocks from the run loop under the lock. The handler itself takes no lock; every word it writes is a whole valid branch. A block whose stop sites are not in their running state is not saved to the translation store, since it would come back stopped for good. `OCERZ_NO_SIGNAL_STOP=1` goes back to the old delivery, and an m32 process keeps it, since delivery there would enter a 64-bit handler. The dynamic test `loop_signal` pins it under both engines with seven loop shapes, plain and ordered memory, and a thread that twenty `pthread_kill` calls must each reach while it spins. Delivery, in `ocerz_signal_deliver()`, builds the x86 signal frame (`siginfo`, `ucontext` and a machine context that includes the AVX upper halves) on the guest's stack or alternate stack and enters the guest's signal trampoline, which calls the handler and then `sigreturn`. Guest masks are honoured exactly; Chapter 9 explains why getting that wrong once desynchronised Wine's server protocol.

Faults are different. A translated load from a bad address raises a real host `SIGSEGV` or `SIGBUS` in the middle of arm64 code, and the fault handler installed in step 5 decides what it was: the first touch of a lazily unpacked cache page (unpack and retry), a store into a page whose code was translated (drop the stale translations and retry), an unaligned ordered access (patch that one access and retry), and so on. Only a fault that is none of these is turned back into a guest fault, with every guest register reconstructed from the host registers holding it, and delivered to the guest's handler, or reported as a crash. One guard counts faults at the same guest address in a row and, past sixteen, ends the thread as a Wine foreign worker looping on a dispatcher state it cannot repair; until `ff6ea313` (18 October 2026) it did so in every program without a guest signal stack, which is every program that is not Wine, so a handler that unprotects a page and returns, as a garbage collector's write barrier or a JVM safepoint poll does, ended its thread at the seventeenth fault (a sort whose comparator read a protected page exited 241 after sixteen of its 338 faults). The count now applies only to Wine processes.

11. Exit

`main` returns its status in `rax`, and `ocerz_vm_call()` hands it back to `ocerz_dyld_run()`, which resolves `_exit` in the cache and calls it with the low byte of the status. If the program called `exit` itself, its `atexit` handlers and `stdio` flushes have already run as guest code. Either way the guest ends in the exit system call, and `sys_exit()` calls `ocerz_vm_request_exit()`, which marks the VM exited and kicks every other guest thread out of translated code. On the main thread the run loop then stops at its next check, `ocerz_dyld_run()` returns the status and `main()` returns it; on any other host thread, which cannot unwind the main thread's C frames, `sys_exit()` flushes the profiler and the translation store and calls `_exit` on the spot. Translations of code that sits at the same address in every run (here, the shared cache's) have been streaming to the on-disk store all along, and the next process that runs the same code loads them instead of translating it again.

> **NOTE**
>
> A dynamic image whose entry point is `LC_UNIXTHREAD` rather than `LC_MAIN`, as older programs have, takes a slightly different last step: its initial stack is rearranged into the form the kernel would hand to `start` (`argc` followed by the vectors), `rip` is set to the thread entry, and `ocerz_vm_run()` runs it like a static program.

The same program in native mode

Run as `./ocerz -native /usr/bin/sw_vers`, the first two steps are identical: the same command line, the same 256 GB identity arena. From there the route diverges.

- **No cache is mapped.** `ocerz_dyld_run()` logs that native mode is independent of Rosetta and moves on.
- **The system libraries are synthesized.** When the loader reaches `/usr/lib/libSystem.B.dylib` or CoreFoundation, `src/vdylib.c` builds a real x86-64 Mach-O image for it in memory, with load commands, segments and an export trie, from the library's API database. Each export is twelve bytes of x86: load the export's number into `r11`, jump through a slot that points into the same reserved trap window the dyld API uses.
- **Binding is checked strictly.** An import that no synthesized library exports is collected, and if any are left after binding, they are listed and the process exits with status 71 before the guest runs a single instruction.
- **Objective-C is the host's.** `ocerz_objcbridge_fix_selrefs()` rewrites the image's selector references to the native runtime's selectors, and `ocerz_objcbridge_define_image()` realizes the guest's own classes in the native runtime.
- **Initializers are simpler.** There is no libSystem initializer to run, because the host's libSystem is already initialized in the `ocerz` process. `ocerz_objcbridge_run_loads()` runs the guest's `+load` methods, then each loaded guest image's initializers in reverse load order, then the program's own.
- **A library call is a crossing.** When `sw_vers` calls `CFStringCreateWithCString`, the stub's jump lands in the trap window. From translated code the JIT does not take the trap: it calls the bridge directly from inside the block, `src/abi.c` moves the arguments from System V registers to arm64 ones according to the function's signature, `src/abicall.s` calls the host's real CoreFoundation, and the result comes back the same way. That costs about 17 ns; eleven string and memory routines such as `strlen` are answered in place by arm64 code in `src/leaf.s` for about 2 ns. A function with no crossing stops with status 72 and its name.

Two details of native mode's startup were added later. Since `2d59b306` (6 October 2026), when the process is Wine's and runs under the low shadow (Chapter 8), readable host memory below 12 GB, the main image, the first heap, the main stack and the shared cache, is aliased into the shadow at the same offset and pinned, because native mode hands the guest host pointers from that range all the time. And a child the guest starts that is a system tool with an arm64 slice of its own, and is not a shell or another launcher, runs as itself rather than under AArchX, since nothing about it needs translating; `OCERZ_NO_NATIVE_CHILDREN=1` turns that off (`docs/modes.md`).

Everything below the libraries is unchanged: the same JIT translates `sw_vers`'s own code, the same syscall layer answers the system calls that do not go through a library, and the exit is the same. Chapter 12 describes native mode in full.

A static program

The test programs under `tests/guest/` are freestanding: built with `-nostdlib -static`, they make system calls directly and link nothing. For such a file `ocerz_peek_dynamic()` returns 0, and `main()` takes the oldest path in the tree:

1. `ocerz_mem_init(0x100000000, 0x900000000)` reserves a fixed arena for guest addresses from 4 GB to 36 GB. Unlike the identity map, a guest address here is translated by adding a base, because the host will not always grant the low addresses a static image asks for.
2. `ocerz_load_image()` in `src/loader.c` picks the x86-64 slice and maps the segments, reading every segment's bytes before applying any protection, because two 4 KB guest segments can share one 16 KB host page.
3. `ocerz_setup_stack()` in `src/stack.c` builds the initial stack exactly as XNU's exec path does for `start`: strings at the top, then `apple`, `envp`, `argv` and `argc` below them.
4. `rip` is set to the entry point from `LC_UNIXTHREAD` and `ocerz_vm_run()` runs it, with no dynamic linker, no initializers and no shared cache.

Native mode refuses such a program, and cache mode does not need the cache for it: a static test program runs on any Mac with nothing but `ocerz`, which is why the guest test suite, the oldest gate in the tree, still uses this path. Chapter 10 describes the loader.

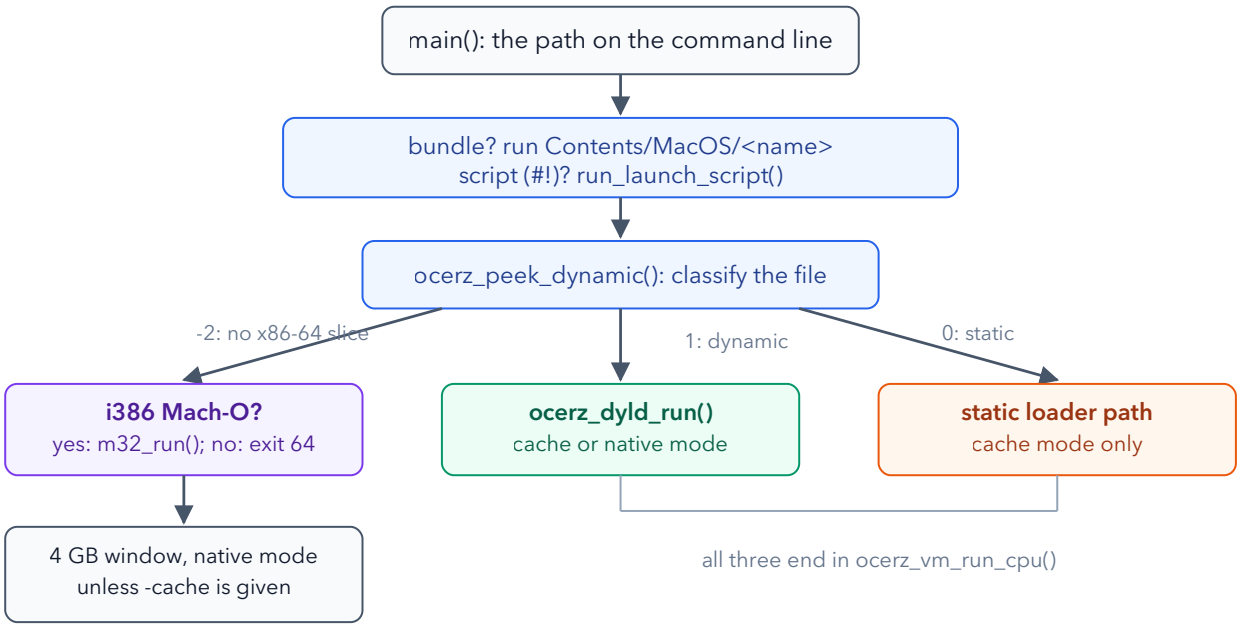


Figure 2.5. The three routes out of `main()` at 6e6c64b5. The first two stops are new since the first edition: scripts are resolved before the file is classified, and a file with no x86-64 slice may be an i386 program rather than an error.

A 32-bit program

Until 10 October 2026, `main()` refused any executable with no x86-64 slice. `0ca24d64` changed that: an executable with an i386 slice and no x86-64 one is handed to `m32_run()` in `src/m32.c` instead. m32 comes from MacShack, written to bring Feral's Batman: Arkham Asylum, a 2014 game built for i386 only, to Apple silicon, and it runs such programs in native mode against the Mac's own arm64 frameworks; Chapter 13 is about it. Only the route is described here, because it is a short, separate life.

The i386 default described in Section 1 means that `./ocerz "Some Game.app"` needs no flag. `m32_run()` reserves a flat 4 GB window with `ocerz_mem_init(0, 4 GB)`, so the guest never holds a host address, and it fences off three ranges that are never handed out: the trap window that imports are addresses in, the handle cells at `0xE0000000` to `0xF0000000` through which host objects reach the guest as 8-byte cells, and the top of the window. It tells CoreFoundation that the guest is the main bundle, loads the program and its dependent images with `m32_load_program()`, starts the dlmalloc heap, and resolves every import with `m32_fixup_all()`; an import that cannot be resolved ends the process with exit 71 and "required imports are unresolved" unless `OCERZ_M32_KEEP_GOING` is set. Then it maps an 8 MB stack, builds the frame the way the kernel and dyld leave it (`argc`, `argv`, `envp` and an `apple` vector holding `executable_path`), resets the CPU into 32-bit mode with an LDT code selector, installs the same host fault handlers as any other process, and calls `m32_start_program()`. For an `LC_MAIN` program that sets up dyld's frame for `main(argc, argv, envp, apple)` with the exit trap as its return address; for an `LC_UNIXTHREAD` program it starts at the program's own start routine with `argc` at the stack pointer. Every image's initializers run first, dependencies before dependents, and then `ocerz_vm_run()` enters the ordinary run loop on the ordinary interpreter and JIT, which decode i386 (Chapter 4).

One kind of program has no dyld and no libraries at all: a UPX-packed executable, whose stub unpacks the real program with raw system calls and then opens `/usr/lib/dyld` to start it. `m32_run()` recognises it by an `LC_UNIXTHREAD` entry with no dylinker and no dependencies, keeps the low 256 MB of the window free for the unpacking, and answers the stub's open with a dyld of its own whose entry is a trap, where it takes the unpacked program from memory and starts it like any other (`src/m32_selfload.c`). `OCERZ_M32_NO_SELFLOAD` turns that off.

Exit codes along the way

Code	Where	Meaning
64	`main()`	bad AArchX arguments, an unknown `OCERZ_MODE`, a file with neither an x86-64 slice nor an i386 one, a launch script whose `ocerz` path has a space in it, or a static image in native mode
65	`main()`	the program could not be read or loaded, or an i386 program has no entry point
70	static path	the guest arena, window, heap or stack could not be set up
71	native binding, m32	an import no synthesized library exports, or an i386 import left unresolved
72	native crossing	a bound export with no crossing was called
126	launch script	the script could not be run
guest's own	exit	the program ran and exited

Every one of these is preceded by a line on standard error that begins with `ocerz:` and says what went wrong. Appendix B lists the messages.

CHAPTER 3

The source tree and its conventions

AArchX is 64 source files and 36 headers, built by one Makefile with nothing but the system toolchain. This chapter lists every file with its size and its job, shows how the headers depend on each other and which three structures hold the state, and describes the build, the test gates it drives, and the conventions the tree is written in: one prose explanation per file, commit messages that state a rule, a test for every change, and several hundred environment switches.

109,858
LINES IN SRC/ (60 .C, 3 .S, 1 .M, 1 .H)

4,628
LINES IN 35 HEADERS

26,593
LINES IN JIT.C ALONE

545
DISTINCT OCERZ_* VARIABLES READ

The shape of the repository

All counts in this chapter are from commit `6e6c64b5` (18 October 2026); the first edition counted at `90fe4d8e` (4 October), and where a number changed the old one is given. The repository tracks 1,427 files, up from 722; most of them are tests.

Directory	What lives there
`src/`	the translator: 30 C files and 3 arm64 assembly files, plus the 32-bit runtime, m32: 30 more C files, one Objective-C file and one vendored header (Chapter 13)
`include/` `ocerz/`	35 headers, each opening with a prose block that states its contract; six of them belong to m32
`tests/`	the gates: `unit/` (30 C harnesses that link the translator's objects), `guest/` (freestanding x86-64 programs with expected outputs), `dynamic/` (240 files of dynamically linked fixtures), `m32/` (267 files: 79 i386 programs with their expected outputs, and their sources and binaries), `win/` (7 Windows programs for Wine), `corpus/`, and the runner scripts
`tools/`	`sdkgen.sh` and `sdkgen/`, which generate native mode's API databases from the SDK (and, with `--guest i386`, m32's); `steamgen.sh` and `sdkgen/steamgen.c`, which write m32's Steam interface database; `aptest.sh`, which compares real tools against native output; `dyldslots.sh`, which identifies every dyld API slot by running code; the guest runtime scripts `build_guest_cxx.sh`, `fetch_guest_gnu_cxx.sh`, `fetch_guest32_cxx.sh` and `install_guest_swift.sh`; the i386 decode oracles and the corpus generator
`app/`	the source of AArchX.app: `AArchX.m`, `Info.plist` and `icon.m`, which draws the icon when `make app` runs (Chapter 1)
`runtime/`	not committed: `apis/` and `apis32/`, the API databases `make apis` and `make apis32` generate, and `guest/`, `guest32/` and `guest-gnu/`, the optional C++ runtimes the fetch and build scripts install
`docs/`	user and contributor documentation, `install.sh`, the Atlas (a guided tour of the source), and the PDF of this book's first edition

Every source file

Grouped by layer, in the order a program meets them. The line counts are the whole file, explanation block included. The 33 files below hold 85,632 lines; the 32-bit runtime, listed after them, holds another 24,226.

File	Lines	Role
Entry and process		
`main.c`	575	Command line, the startup decisions (Objective-C preload list, plain memory model, mode, launch scripts, i386 programs to m32), bundle launch, the `DYLD_INSERT_LIBRARIES` re-exec, the per-process switches for Wine's process tree.
`vm.c`	4,613	The run loop, `ocerz_vm_call` and the cheap callback entry, crash containment and the fault handler, signal plumbing and delivery between steps, guest thread suspension, the unstick monitor, native-thread attach, the `kill -INFO` thread dump and the sampling profiler.
`cpu.c`	189	CPU reset state and the register dump used by fatal paths and `-trace`; propagates the guest's MXCSR rounding mode to the host FPCR and clears or sets `FPCR.AH` around host calls.
`globals.c`	17	Process-wide definitions no single module should own.
The CPU		
`decode.c`	4,442	x86-64 and i386 decoding, legacy, VEX and MMX forms, into one `X86Insn` per instruction (Chapter 4).
`interp.c`	1,939	The core single-step interpreter: reference semantics and the oracle the JIT is checked against (Chapter 5).
`interp_sse.c`	2,542	SSE through SSE4.1 and the AVX forms layered on them.
`interp_ext.c`	763	The long tail: string instructions, CPUID, CRC-32C, FXSAVE and XSAVE, and the rest; the x87 instructions are in `x87.c`.
`x87.c`	1,277	The x87 unit, new on 7 October 2026 (`8f310eb8`): registers as doubles with an exact 80-bit image beside each, the tag word, control and status words, and the arithmetic, checked against Rosetta; the specification the JIT's x87 translation follows.
`flags.c`	230	Eager RFLAGS computation, bit for bit, which the JIT's deferred flags must agree with.
`flags_live.c`	298	Per-instruction flag definition and use, so the JIT can skip flags nothing reads.
`jit.c`	26,593	The JIT: translation, register pinning, deferred flags, chaining, superblocks, floating-point batches, x87 runs, memory ordering, the Wine layout's hoisted base checks, self-modifying code, signal stops, the arena and its flush (Chapters 6 and 7).
`a64emit.c`	954	The arm64 encoder: one function per instruction, no scheduling, no register allocation.
`leaf.s`	600	arm64 `strlen`, `memcpy` and nine more that translated code calls with the guest's registers left in place.

File	Lines	Role
`tcache.c`	897	The on-disk store that keeps translations between processes and runs.
Memory and the system		
`mem.c`	2,342	Guest address maps, the 4 KB-in-16 KB slot machinery, write-arming for self-modifying code, the low shadow (Chapter 8).
`syscall.c`	8,967	The guest-to-host boundary: BSD calls, Mach traps and messages, threads and the workqueue bridge, signals, fork, exec and spawn (Chapter 9).
Linking		
`loader.c`	378	Static Mach-O loading: slice selection and segment layout (Chapter 10).
`stack.c`	137	The initial process stack, laid out the way XNU's exec path does.
`dyld.c`	6,563	The mini-dyld: loading, binding, initializer order, delayed-init links, `dlopen` and `dlsym`, in both modes (Chapter 11).
`cache.c`	1,591	Mapping the x86-64 shared cache, lazy slide, and symbol resolution out of it.
`dyldapi.c`	2,731	The dyld API object libdyld calls into, the image list, Objective-C image callbacks; also the trap window m32 shares.
Native mode (Chapter 12)		
`vdylib.c`	1,424	Synthesizes the x86-64 system libraries native mode has no files for.
`apidb.c`	1,108	Reads the API database: each library's exports and signatures.
`bridge.c`	3,692	What each export does when called: crossings, the functions AArchX answers itself, fault attribution.
`abi.c`	1,975	Moves arguments and results between System V x86-64 and Apple arm64, both ways; the callback trampoline bank.
`abicall.s`	237	Makes an arm64 call whose argument registers arrive as data.
`sysbridge.c`	2,116	The libSystem calls a generic crossing cannot make: fixed-argument variadics, memory mapping, non-local jumps, processes.
`objcbridge.c`	3,760	Objective-C message sends and formatted output across the boundary, with the per-thread send cache added on 18 October.
`objcclass.c`	1,658	The guest's own classes, categories and protocols, realized in the native runtime.
`blocks.c`	914	Blocks in both directions.

File	Lines	Role
`objcguard.s`	60	Catches a native Objective-C exception where a crossing returns to the guest (`46349987` , 5 October): the frame whose personality routine claims the exception.
`compl exdiv.c`	50	`__divdc3` and `__divsc3` , the compiler-rt complex divisions the arm64 libSystem does not export, which Unity's Mono imports (`275843d4`).

The 32-bit runtime

The thirty-two files of m32 were added in October 2026 (the first, `0ca24d64` on 10 October, came from the MacShack project) and are the largest single addition since the first edition. They are named `m32_*` so that a directory listing keeps them together; `ls src/m32_*` shows more than that, because it also matches the `.o` and `.d` files a build leaves beside them. Six headers in `include/ocerz/` belong to them too. Chapter 13 explains how the pieces fit; this table says only where each lives.

File	Lines	Role
`m32.c`	223	Entry: `m32_run` reserves the 4 GB window, loads the program, builds the initial frame, starts the CPU in 32-bit mode and runs the initializers (Chapter 2).
`m32_selfload.c`	205	Programs that load themselves, such as a UPX-packed executable: raw system calls, then a dyld of m32's own whose entry is a trap.
`m32_dyld.c`	1,509	i386 Mach-O images: loading from `@executable_path` , `@loader_path` and `@rpath` , binding, flat namespaces, initializer order, `dlopen` , 10.4-era `_dyld_func_lookup` .
`m32_db.c`	184	Reads m32's i386 API database, each export's guest and host notation.
`m32_bridge.c`	453	Every import becomes an export with an id; its address in the trap window is its trap, and this file dispatches it.
`m32_cross.c`	1,220	The generic crossing from an i386 cdecl call to an arm64 AAPCS64 one, driven by the two notations.
`m32_handle.c`	243	The pointer rule: the guest never holds a host address; host objects reach it as handle cells and aliases.
`m32_mem.c`	171	The window's memory: static allocation and the dlmalloc heap.
`m32_leaf.c`	209	Imports the JIT calls straight from an import stub with no crossing: small libc functions, CGL calls, integer-only OpenGL.
`m32_callback.c`	331	Calls into i386 guest code from the host, and guest function pointers handed to host functions.
`m32_libsystem.c`	1,550	Hand-written libSystem entry points whose 32- and 64-bit forms differ in layout.
`m32_stdio.c`	920	The variadic printf and scanf families, the heap's entry points and `errno` .
`m32_thread.c`	576	pthreads, keys and thread-local variables, with i386-sized pthread objects.
`m32_cf.c`	748	Constant strings and the CoreFoundation calls that take arrays of object pointers.
`m32_cg.c`	229	CoreGraphics calls whose i386 shapes the notation cannot carry (callbacks, patterns).

File	Lines	Role
`m32_zlib.c`	226	zlib and bzip2 streams, which differ between 56 and 112 bytes.
`m32_tsearch.c`	201	`tsearch` and its relatives, with the tree kept in guest memory.
`m32_unwind.c`	500	C++ exception unwinding: the two-phase Itanium unwinder over the guest's `__eh_frame`.
`m32_gl.c`	521	Mapped OpenGL buffers and buffer-object offsets.
`m32_audio.c`	481	CoreAudio render callbacks and audio units.
`m32_atsu.c`	1,137	ATS fonts and ATSUI text, on CoreText, because both are shells on current macOS.
`m32_carbon.c`	578	Carbon on the HIToolbox Apple silicon still has: windows, events, universal procedure pointers.
`m32_controls.c`	665	Carbon menus and controls that arm64 HIToolbox dropped, made from their HIView successors.
`m32_qd.c`	1,360	QuickDraw on CoreGraphics: ports, GWorlds, drawing, pen and text state.
`m32_agl.c`	352	AGL, Carbon's OpenGL, on the arm64 AGL.
`m32_steam.c`	451	The Steam bridge: objects whose vtables cross into the 64-bit Steam client.
`m32_objc.c`	1,960	The Objective-C 1 runtime of i386 guests: classes, proxies, paired host classes, categories, sends.
`m32_objc_types.c`	374	Objective-C type encodings turned into m32's notation.
`m32_objc_exc.c`	98	Objective-C 1 exceptions, setjmp-based, rethrown into the guest.
`m32_blocks.c`	227	i386 blocks and host blocks, in both directions.
`m32_catch.m`	42	The one Objective-C file of m32: stops an `NSException` at the crossing.
`m32_dlmalloc.h`	6,282	Doug Lea's dlmalloc 2.8.6, public domain, unmodified, for the guest heap; vendored, so its text is its author's rather than an explanation block.

The i386 guest needs no separate decoder or interpreter: `decode.c`, the interpreter files and the JIT handle 32-bit mode themselves, which is why the m32 files hold only the runtime around them.

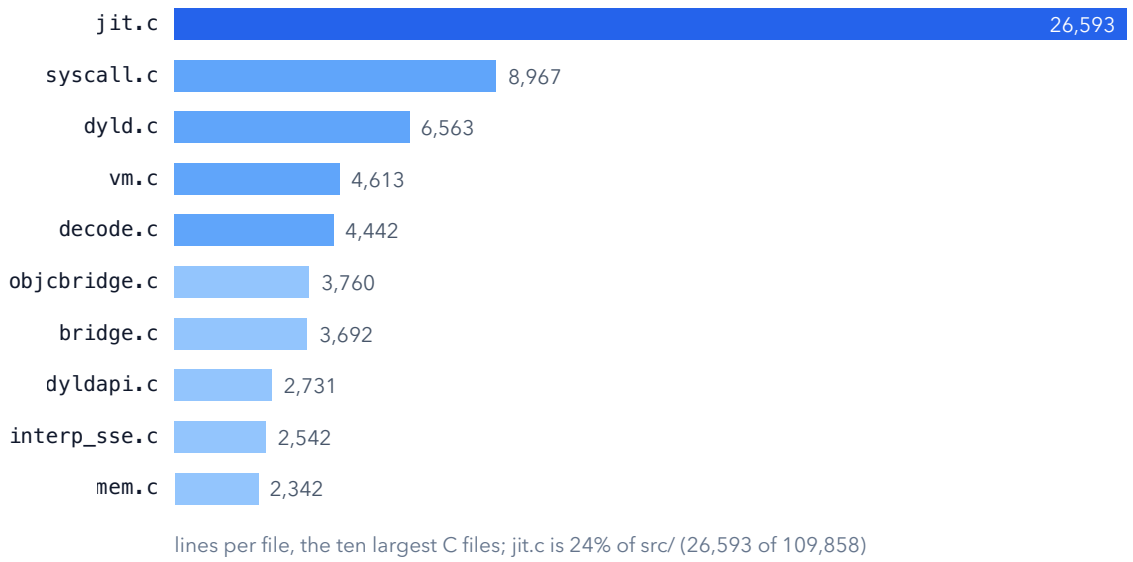


Figure 3.1. Where the code is. The JIT dominates, then the system-call layer and the dynamic linker; every other C file is under 4,700 lines, and none of the m32 files is among the ten (the largest, `m32_objc.c`, has 1,960; the vendored `m32_dlmalloc.h`, at 6,282, is a header and not counted).

The headers and how they depend on each other

The 35 headers in `include/ocerz/` are the contracts between the files; 29 belong to the translator and 6 to m32. Most are small; the large ones are `decode.h` (655 lines, the instruction record and the list of operations), `abi.h` (492 lines, the signature language native mode's crossings are written in) and `objcbridge.h` (435). The include graph is shallow and acyclic: almost everything rests on `types.h` and `cpu.h`.

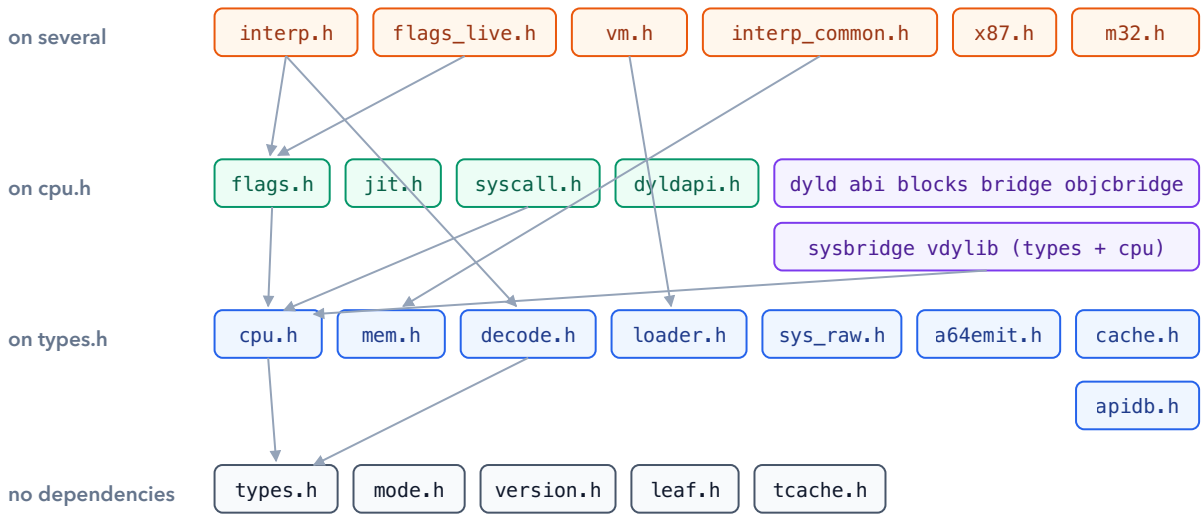


Figure 3.2. The header include graph, simplified: each row depends only on rows below it. `interp_common.h` pulls in `cpu.h`, `decode.h`, `flags.h`, `mem.h` and `interp.h`; `vm.h` needs `cpu.h` and `loader.h`; `dyldapi.h` includes only `cpu.h`. The two boxes at the right of the top row are new: `x87.h` needs `cpu.h` and `decode.h`, and `m32.h` needs `cpu.h`, `mem.h` and `dyldapi.h`; the five other m32 headers are used only by the m32 files and are left out.

Header	Lines	Contract
`types.h`	48	Fixed-width types, error codes, logging macros, the 128-bit vector type.
`cpu.h`	199	`OcerzCPU`, one guest thread's architectural state; its field order is load-bearing for the JIT.
`decode.h`	655	`X86Insn`, the only format the decoder, interpreter and JIT share, and the list of operations (560 values, 12 more than at the first edition: eleven x87 forms and `sysenter`).
`mem.h`	268	The guest address maps and sized guest loads and stores; the commpage window.
`vm.h`	111	`OcerzVM`, one guest process; the run loop and `ocerz_vm_call`.
`jit.h`	94	The JIT's interface, and the retirement counter callers use to tell whether code they will return to is still current.
`syscall.h`	123	The system-call entry and the helpers other layers call.
`abi.h`	492	Native mode's signature language and the calling-convention engine.
`objcbridge.h`	435	Objective-C sends, selectors and guest classes across the boundary.
`a64emit.h`	304	The arm64 encoders.
`apidb.h`	177	The API database format and where its files are read from.
`interp_common.h`	166	Helpers shared by the three interpreter files.
`bridge.h`, `vdylib.h`, `blocks.h`, `sysbridge.h`, `dyld.h`	166, 141, 129, 129, 100	The rest of native mode and the dynamic linker.
`flags.h`, `leaf.h`, `cache.h`, `tcache.h`, `sys_raw.h`	72, 62, 53, 50, 39	Flag semantics, the in-place routines, the shared cache, the translation store, the one direct path into the kernel.
`interp.h`, `dyldapi.h`, `mode.h`, `loader.h`, `flags_live.h`, `version.h`	31, 31, 29, 25, 17, 10	Small interfaces; `version.h` defines `"AArchX"` and `"0.8"` (it defined `"0.3-dev"` at the first edition).
`x87.h`	29	The x87 unit's entry points for the interpreter, FXSAVE, and the bridge, which pushes a returned `long double`.
`m32.h`, `m32_image.h`, `m32_objc.h`, `m32_objcrt.h`, `m32_sig.h`, `m32_db.h`	174, 75, 53, 81, 37, 23	m32's window, trap ids and entry points; its image records; its Objective-C 1 runtime and the host runtime it resolves on first use; its parsed notations; its API database.

Three structures hold the state

Nearly everything AArchX does reads or writes one of three structures.

`OcerzVM` (`vm.h`) is one guest process: the main thread's CPU, the loaded image, the JIT, the exit flag and code, the trace switches, and the two memory-model flags, `jit_plain_mem` and `jit_ordered_required`, that record whether the process has acquired a second observer yet. There is one per `ocerz` process.

`OcerzCPU` (`cpu.h`) is one guest thread. It holds the sixteen general registers in x86 encoding order (`rax`, `rcx`, `rdx`, `rbx`, `rsp`, `rbp`, `rsi`, `rdi`, then `r8` to `r15`), `rip`, the flags with the deferred-flag record beside them (`cc_op`, `cc_src`, `cc_dst`), the `fs` and `gs` bases, the sixteen `xmm` registers, the AVX upper halves, the x87 registers held as doubles with an exact 80-bit image beside each (`fpr_xm`, `fpr_xe` and a validity bit, since 7 October: `8f310eb8`), the MMX registers, MXCSR, the signal mask and pending set, the alternate stack, and the segment selectors 32-bit code needs. The field order matters because the JIT addresses this struct with immediate offsets: `xmm[]` is 16-byte aligned for scaled vector loads, and the 256-entry return-address stack, 4 KB by itself and most of the struct, sits immediately after it so that every entry stays within reach of a single `ldp` or `stp`. New fields are appended at the end so nothing that exists moves by a byte. The newest fields serve signals and the Wine layout: the blocks and edges a signal rewrote so the thread would leave a loop (`sig_stop_*`, `a7fd0f3f`), and the stack delta the Wine layout keeps beside its host-pointer `rsp`. A fair part of the rest is per-thread instrumentation: the block-entry trace ring, the fields the unstick monitor reads, the thread-suspension handshake, a ring of the last system calls, and the form of the instruction the interpreter is running, for the sampling profiler.

`OcerzJit` (private to `jit.c`) is the translator: the code arena, 1 GB by default (`OCERZ_JIT_CODE_MB` or `OCERZ_JIT_CODE_KB` change it, and the test suite runs some programs in a tiny one on purpose), the hash table of translated blocks, the live-block array, per-granule block lists, and the tables that chaining, return prediction and invalidation use. There is one per process, shared by every thread under a translation lock.

Between them sits **`X86Insn`** (`decode.h`), the 96-byte record the decoder produces for each instruction: its address, length, a 16-bit operation number, size and prefix fields, the condition code, the mode, the VEX fields, and up to three operands. It is the only representation the decoder, interpreter and JIT share. The operation field was once eight bits wide while the operation list had 364 entries; operations above 255 truncated into the wrong interpreter unit, every consumer honoured the flawed header identically, and only the end-to-end differential exposed it. The field is 16 bits now and there are 560 operation values.

Building

The Makefile compiles every `src/` file with `clang -arch arm64 -std=c11 -O2 -g -Wall -Wextra` and links `ocerz` against `libcompression`, which the translation store uses for its LZ4 records, and `libobjc`, which m32's Objective-C 1 runtime and its one `.m` file need. The Objective-C file is compiled with `-fobjc-exceptions`, `src/m32_leaf.c` with `-mgeneral-regs-only`, and the unit harnesses are linked with `-Wl,-S` so that the binaries committed to the tree carry no debug map and with it no build path (`5e09eb4b`). There are no other dependencies. A full build takes well under a minute.

Target	What it does
`make` or `make ocerz`	Builds the translator.
`make unit`	Builds the 30 unit harnesses, which link every object except `main.o`, and runs each with `OCERZ_NO_ARM_EXEC=1`, because they drive the JIT without a run loop and the self-modifying-code trap would have no fault recovery to land on.
`make guest`	Builds the freestanding x86-64 test programs in `tests/guest` with `-arch x86_64 -nostdlib -static`.

Target	What it does
`make apis`	Generates native mode's API databases under `runtime/apis` from this machine's SDK, with `tools/sdkgen.sh`; a stamp named for the SDK version keeps it from rerunning until the generator or its inputs change. They are derived from the SDK and so are never committed.
`make apis32`	Generates m32's i386 API databases under `runtime/apis32` for 32 libraries, with `tools/sdkgen.sh --guest i386`; it depends on `make apis`, which m32 needs too.
`make steam32`	Runs `tools/steamgen.sh`, which fetches the Steamworks interface headers from Valve's Proton at a pinned commit (about 50 MB, kept out of the tree) and writes the Steam interface database m32's bridge reads.
`make check`	Depends on `ocerz`, `unit`, `guest` and `apis`, then runs eleven gates in order: the guest suite interpreted, the guest suite translated, the x86-64 differential, the i386 differential, the dynamic suite, the native suite, the guest-library tests, the native C++ tests, the native framework tests, the native format tests and the native Swift tests (the Swift gate was added after the first edition, with the Swift runtime support of `06d1b4da`).
`make check-m32`	The gate for 32-bit programs, not part of `check`: it needs `apis32`, then runs `tests/m32/dbcheck.sh`, `test_m32 carbon` and `tests/run_m32_tests.sh`, and the last again with the high-byte and shift-by-immediate translations switched off. The 79 programs under `tests/m32` are committed as binaries, because linking i386 needs Xcode 26's `ld-classic`.
`make app`	Builds `AArchX.app` from `app/AArchX.m` with no Xcode project, draws the icon with `app/icon.m`, and signs the bundle ad hoc.
`make guest-swift`	Installs the guest copy of the Swift runtime native mode uses for Swift programs (`tools/install_guest_swift.sh`).
`make guest-cxx`	Builds LLVM's libc++, libc++abi and libunwind for x86-64 into `runtime/guest`, for native mode. m32 programs built with GCC's libstdc++ take theirs from `tools/fetch_guest32_cxx.sh`, which since `97c3d7c7` (16 October) takes GCC 15.2's runtime from MacPorts' universal package and since `9fcf0476` (12 October) fetches the i386 LLVM libc++ as well; it fills `runtime/guest32`, and the two m32 tests that need it are skipped without it.
`make i386diff`	32-bit decode conformance against an independent third-party decoder, reported as a coverage percentage; deliberately not part of `check`.

The Makefile exports `OCERZ_TCACHE=off` unless the environment says otherwise, so no test loads a translation an earlier test stored and none writes to `~/Library/Caches`; the translation-store tests turn it on themselves against a directory of their own.

The test binaries are **committed**: 30 unit harnesses under `tests/unit/bin` and the guest programs and benchmarks under `tests/guest/bin` and `tests/guest/benchbin`, 135 files, and 82 more under `tests/m32/bin`. A change to the translator is then measured against exactly the binaries the previous run used, not against whatever the local compiler produces today. Because the unit harnesses link the translator's objects, they change whenever it does, and they are refreshed by a separate commit with the fixed subject `tests: rebuild unit binaries`; 210 of the 1,082 commits are that. Chapter 16 describes every gate.

The conventions

One explanation block per file

Since commit `3037b859` of 12 September 2026, which changed 154 files and removed 5,501 lines of comments, every source file and header opens with one prose block explaining what it owns and why it is shaped the way it is, and the code bodies carry no comments at all. The blocks are written in the present tense, in full sentences, and they record the failures that shaped each rule: why the environment is copied, why initializers run in a particular order, why vector accesses stay plain in ordered mode. They are the deepest documentation in the project, and they are long where the subject is: 558 lines in `dyld.c` (519 at the first edition), 680 in `jit.c` (517), 387 in `syscall.c` (377), 333 in `vm.c` (302). The vendored `m32_dlmalloc.h` is the one file that keeps its author's text and carries no such block.

```
/*
 * Command-line parsing and the entry point.
 *
 * Three startup decisions are made here rather than lazily.  Some shared-cache
 * images' Objective-C categories must be visible before any class is realized,
 * because a dlopen registers them too late for classes that already exist:
 * ...
 */
```

The rule is that a block changes in the same commit as the behaviour it describes. It is not always kept: at `6e6c64b5`, as at `90fe4d8e`, the blocks of `decode.c` and `cpu.h` still say the JIT declines every VEX instruction, which stopped being true on 13 September.

Commit messages that state the rule

A subject line has the form `area: what is now true`, not a description of the activity: `jit: a chain that cannot reach goes through a veneer instead of being dropped`, `dyld: an upward dependency is initialized after the whole walk from the program, not right after the library that declares it`. The body is prose: the symptom, what was measured, the mechanism, the fix and the test. These messages are long, often several paragraphs, and together with the explanation blocks they are the project's design record; this book quotes from both. By area, the most frequent subjects at `6e6c64b5` are `jit` (268 commits), `tests` (236, of which 210 are binary refreshes), `native` (80), `syscall` (59), `dyld` (31), `vm` (20), `m32` (16) and `mem` (11). The m32 work was committed under `native` until 16 October and under its own area after that, and the counts are of the rewritten history, so they differ slightly from those the first edition gave for the same commits.

A test that pins every change

Every behavioural change carries the test that would have failed before it: a freestanding guest program with an expected output, a dynamically linked fixture, a unit harness, or a native-mode case. Expected outputs come from a real run, under Rosetta or natively, never from memory. The project goes further for its hardest rules: a test is expected to fail on the build just before the fix and pass after it, and several commit messages record that check.

Sign-off, and nothing else

Commits are signed off under the Developer Certificate of Origin (`git commit -s`); 464 commits carry a `Signed-off-by` line (83 at the first edition), the first on 23 September 2026. No other trailers are used. `CONTRIBUTING.md` writes all of this down, along with two more rules: `make check` must be green before a change is sent, and a claim in the README that a change invalidates is corrected in the same change.

The switches

AArchX is unusually configurable at run time, by design. At `6e6c64b5` the source reads 545 distinct `OCERZ_*` environment variables through `getenv` or its two caching wrappers, 239 of them in `jit.c` alone, and 216 are `OCERZ_NO_*` switches that each turn one optimisation or one behaviour off; at `90fe4d8e`, counted the same way, they were 410, 162 and 124. They fall into a few families:

- **`OCERZ_NO_<FEATURE>`** turns one optimisation off: `OCERZ_NO_FLIP` (superblock retranslation), `OCERZ_NO_AFP` (hardware NaN results), `OCERZ_NO_LEAF_INPLACE` (the arm64 string routines), and from October `OCERZ_NO_SIGNAL_STOP` (delivery of signals to threads in loops), `OCERZ_NO_CHEAP_CALLBACK` (the full-cost callback entry), `OCERZ_NO_DELAY_INIT` (delayed-init links followed like any other) and `OCERZ_NO_M32_TRAP_BLOCKS`. If a program misbehaves and one of these fixes it, the bug is in what that switch controls.
- **`OCERZ_<AREA>LOG`** and their relatives print one kind of event: `OCERZ_DLPATH`, `OCERZ_EXCLOG`, `OCERZ_JITLOCKLOG`, `OCERZ_PERFSTAT`, `OCERZ_GUESTPROF`, and for 32-bit programs `OCERZ_M32LOG`, whose value names what to print (`imports`, `steam`, failed `dlsym` lookups).
- **Ranges** select part of a program: `OCERZ_INTERP_LO` and `OCERZ_INTERP_HI` keep an address range in the interpreter, which is how a miscompiled block is bisected in about an hour.
- **Per-process** switches pick one process of a tree by its command line: `OCERZ_NOJIT_EXE`, `OCERZ_STRACE_EXE`, `OCERZ_EXE_ENV`.

Each is read once, at first use, and cached. Most exist because one real bug needed exactly that view, and they stay because the next bug usually needs it too. `docs/reference.md` documents the ones meant to be used; Appendix A lists them, and Chapter 18 shows how they are combined.

Documentation in the tree

Since 23 September 2026 the documentation lives in `docs/`: getting started, the two modes, native mode in depth, the command-line and environment reference, architecture, performance, compatibility, troubleshooting and testing. The **Atlas** (`docs/atlas/index.html`, also published at mont127.github.io/AArchX/atlas/) is a guided tour of the source for contributors that assumes no C: the vocabulary first, then the life of a process, executing x86, the run loop, memory, the kernel boundary, cache mode's loader, native mode's bridge, how correctness is held, how to read a debugging session, and seven case files of real fixes. `CONTRIBUTING.md`, `CODE_OF_CONDUCT.md` (the Contributor Covenant 2.1) and `SECURITY.md` complete the set, with issue and pull-request templates under `.github/`. Three files arrived with the October work: `docs/m32.md` on 32-bit programs, `docs/32-bit-steam-games.md` on playing a 32-bit Mac game from Steam step by step, and `docs/install.sh`, the installer Chapter 1 describes. The README links the first edition of this book, as a PDF in `docs/`.

PART II

The CPU

Turning x86 bytes into behaviour: the decoder, the reference interpreter, and the arm64 JIT that does the real work.

CHAPTER 4

The decoder

Every x86 instruction AArchX ever runs, interpreted or translated, 64-bit or 32-bit, first passes through one function that turns raw bytes into one fixed-size record. This chapter takes that function apart: the byte layout it parses, the prefix rules that are easy to get wrong, the record it produces, the 560 internal operations it maps onto, and the machinery that proves a change to it moved nothing it should not have.

4,442	560	96 B	2^{24}
LINES IN DECODE.C	INTERNAL OPERATIONS	PER DECODED INSTRUCTION	OPENINGS COMPARED PER CHANGE

One function, one format

The decoder lives in `src/decode.c` (4,442 lines) with its contract in `include/ocerz/decode.h` (655 lines). Its whole public surface is two functions:

```
int ocerz_decode(const uint8_t *code, size_t avail, uint64_t rip, X86Insn *out);
int ocerz_decode_mode(const uint8_t *code, size_t avail, uint64_t rip,
                      X86Insn *out, int mode32);
```

`ocerz_decode` is the 64-bit long-mode decoder and always has been; `ocerz_decode_mode` takes the mode explicitly, with `mode32 = 1` selecting i386 semantics for the 32-bit code Wine runs inside a WoW64 process. Both are pure: they read the byte buffer and nothing else. They never look at CPU registers, never touch guest memory through a side door, and never consult global emulator state, which is what makes it possible to test them exhaustively from a harness that links `decode.o` and libc and nothing more.

The result is an `X86Insn`, and that struct is the single exchange format between the decoder, the interpreter and the JIT. The interpreter executes an `X86Insn`; the JIT reads a run of them to plan a block; the diagnostic printers format them. Because both engines consume the same decode, they cannot disagree about what an instruction *is*, only about what it *does*, and the second kind of disagreement is exactly what the differential gates catch (Chapter 16).

A call returns one of four results:

Result	Meaning
`OCERZ_OK`	Decoded; `out->len` is the full length including prefixes.
`OCERZ_EUNDEF` (−1)	The byte string is not an instruction in this mode, or is one AArchX deliberately refuses.
`OCERZ_ETRUNC` (−2)	The bytes ran out before the instruction ended and fewer than 16 were available.

Result	Meaning
`OCERZ_ETOOLONG` (−3)	The instruction would be longer than 15 bytes, the architectural ceiling. A truncation with 16 or more bytes available is reported as this rather than as `ETRUNC`, because more bytes would not have helped.

Every read goes through four fetch helpers (`fetch8/16/32/64`) that stop at the end of a window of at most 16 bytes, so no path can read past what the caller said was there. A `NULL` code pointer is treated as what it is, an emulator bug upstream (every caller obtains the pointer through `ocerz_g2h`, the guest-to-host translation of Chapter 8): the decoder says so once on standard error with the address-map bases, and fails the decode instead of faulting.

Anatomy of an x86-64 instruction

An x86-64 instruction is a variable-length sequence of up to 15 bytes in a fixed order: optional legacy prefixes, an optional REX byte, one to three opcode bytes, and then, depending on the opcode, a ModRM byte, a SIB byte, a displacement and an immediate. AVX instructions replace the REX byte, the escape bytes and the mandatory prefix with a two- or three-byte VEX prefix. Figure 4.1 shows both forms.

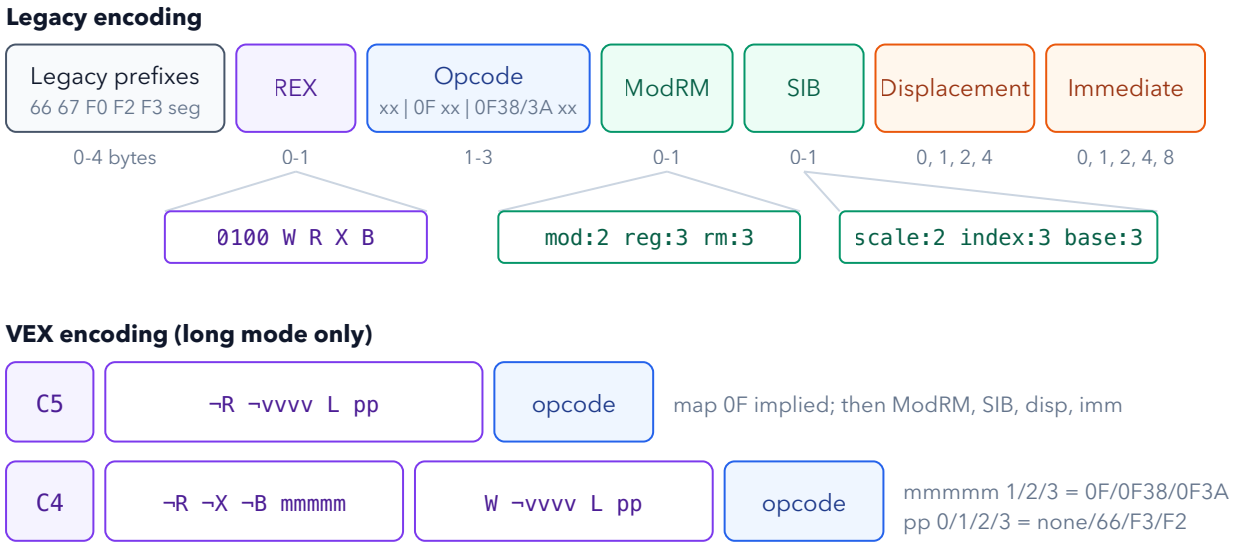


Figure 4.1. The two shapes of an x86-64 instruction. A VEX prefix folds the REX bits (stored inverted), the opcode map, the mandatory prefix, a third register operand (vvvv, also inverted) and the vector length L into two or three bytes.

The decode pipeline

A call proceeds in a fixed order, shown in Figure 4.2. A `DecState` carries the cursor and everything learned so far: which legacy prefixes were seen, the mandatory-prefix choice, the REX bits, the address-size mode, the VEX fields, and a pointer to the output record, which is zeroed first so that every field a path does not set has a defined value.

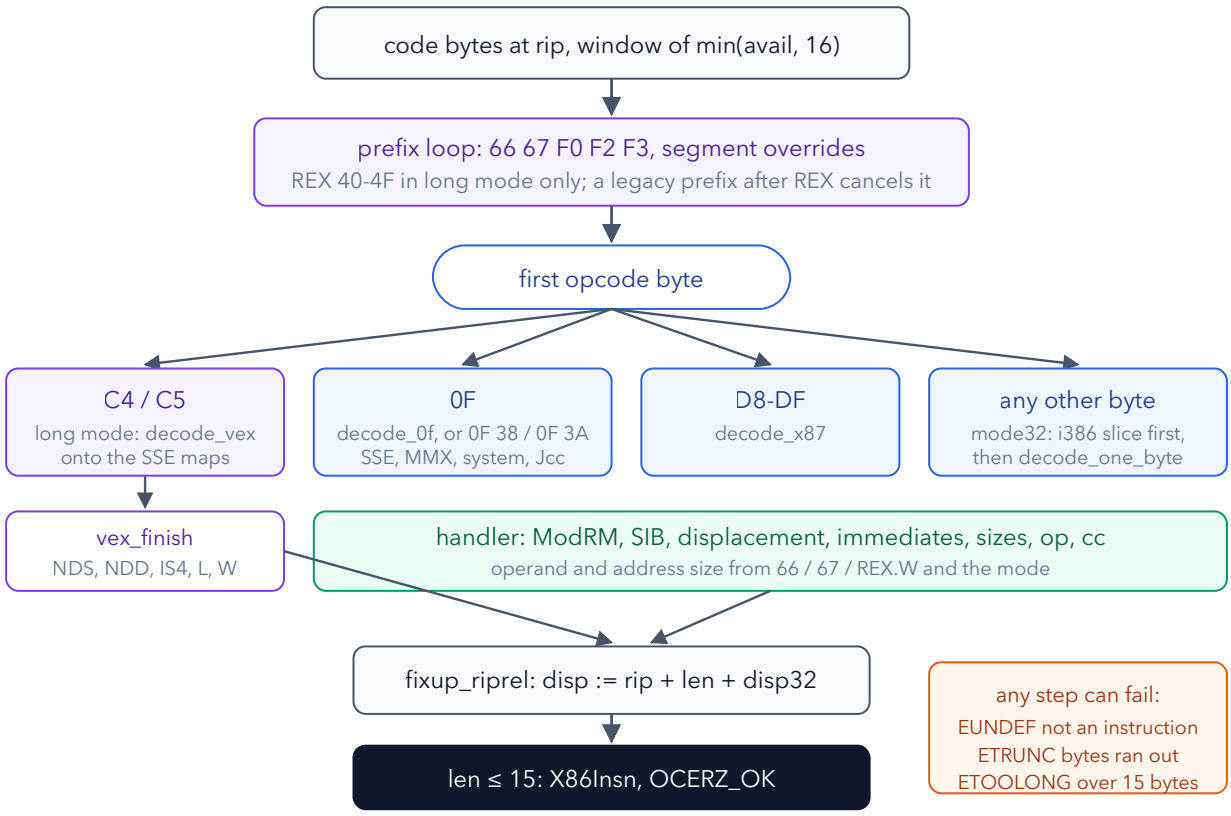


Figure 4.2. The path of one decode. Every opcode handler parses its own ModRM and immediates, because whether an operand exists, how wide it is and which register file it names all depend on the opcode.

Two design choices show in the figure. First, there is no table-driven generic decoder: each opcode or opcode group has a handler that knows its operand shapes, built from a small set of helpers (`alu_rm_r`, `alu_acc_imm`, `branch_rel`, `decode_sse_rr`, `decode_mmx_rr` and so on). The integer ALU block at `0x00`–`0x3D` is the one place where the encoding is regular enough to compute: the operation is `op >> 3` (add, or, adc, sbb, and, sub, xor, cmp) and the form is `op & 7` (register/memory, direction, byte or full width, accumulator with immediate). Second, the RIP-relative fix-up happens once, at the very end, because only then is the instruction length known. Since 2026-10-13 (39271608) it also happens only when it has something to do: `decode_modrm` records that it produced a RIP-relative operand, and only then does `fixup_riprel` run, where before every decode paid a pass over its operands.

Prefixes

The prefix loop is short, but nearly every line encodes a rule that has bitten someone.

- **REX must be last.** In long mode, bytes `0x40`–`0x4F` are REX prefixes and set the W, R, X and B bits. Hardware honours a REX only when it immediately precedes the opcode, so when a legacy prefix follows a REX, the loop clears the REX state it had recorded. In 32-bit mode the same bytes are `INC` and `DEC` of a 32-bit register, so the loop leaves them alone and the opcode dispatcher sees them.
- **Mandatory prefixes.** For SSE instructions, `66`, `F3` and `F2` are not modifiers but part of the opcode: `0F 58` is `addps`, `66 0F 58` `addpd`, `F3 0F 58` `addss`, `F2 0F 58` `addsd`. The loop records the choice in `mand`. An `F2` or `F3` always wins over `66`, and once one of them has been seen a later `66` no longer changes the mandatory prefix (it still sets the operand-size flag). Between `F2` and `F3` the last one seen wins.

- **Segment overrides.** In both modes only FS and GS carry a base that applies to addresses. `2E`, `36`, `3E` and `26` (CS, SS, DS, ES) are consumed and recorded as no override; `64` and `65` set `insn.seg` to FS or GS. Thread-local storage on macOS uses GS, and Wine on macOS uses GS for the Windows TEB.
- `67` swaps the mode's default address size for the other size the mode can name: 64 and 32 in long mode, 32 and 16 in 32-bit mode. The result is `insn.addrsize`: 8, 4 or 2.
- `F0` (LOCK) is recorded in `insn.lock`. The decoder does not check that the instruction is one LOCK may legally prefix; the engines act on the flag.
- **REP** state is held in `DecState` and copied into `insn.rep` only by the string instructions that use it, so an `F3` that served as a mandatory prefix does not also make an SSE instruction look repeated.

ModRM, SIB and addressing

The ModRM byte splits into `mod` (2 bits), `reg` (3) and `rm` (3). REX.R extends `reg` to four bits and REX.B extends `rm`; `mod = 3` means `rm` names a register, and every other `mod` describes a memory operand. `decode_modrm` handles the special cases in one place:

Encoding	Long mode	32-bit mode
`rm = 100`	a SIB byte follows: `scale`, `index`, `base`	same
SIB `index = 100`, no REX.X	no index register	no index register
SIB `base = 101`, `mod = 00`	no base, `disp32`	same
`mod = 00`, `rm = 101`	**RIP-relative** `disp32`	absolute `disp32`
`mod = 01` / `10`	`disp8` / `disp32`, sign-extended	same
under `67` in 32-bit mode	(cannot occur)	16-bit table: no SIB; bases BX/BP with optional SI/DI; `disp16` at `mod = 00, rm = 110`

The scale is stored as its two-bit logarithm, so the effective address is `disp + base + (index << scale)`. A RIP-relative operand is **pre-resolved**: after the whole instruction has been parsed, `fixup_riprel` rewrites the operand's displacement into the absolute guest address `rip + len + disp32`, clears base and index, and leaves the `riprel` bit set. No consumer ever re-derives the program counter, which matters to the JIT: a RIP-relative load is a load from a constant address, and Chapter 7 shows how translation-time constants let it skip run-time checks entirely.

The 16-bit addressing table is a different beast from the 32- and 64-bit one, with a fixed set of eight base/index pairs:

```
rm:   000      001      010      011      100  101  110      111
      BX+SI    BX+DI    BP+SI    BP+DI    SI   DI   BP*      BX
      (* mod = 00, rm = 110 is disp16 with no base instead)
```

It exists only for 32-bit code under a `67` prefix, which real WoW64 programs rarely execute. The JIT leaves it entirely to the interpreter.

The X86Insn record

Figure 4.3 shows the record a decode produces. It is 96 bytes: a 24-byte header and three 24-byte operands.

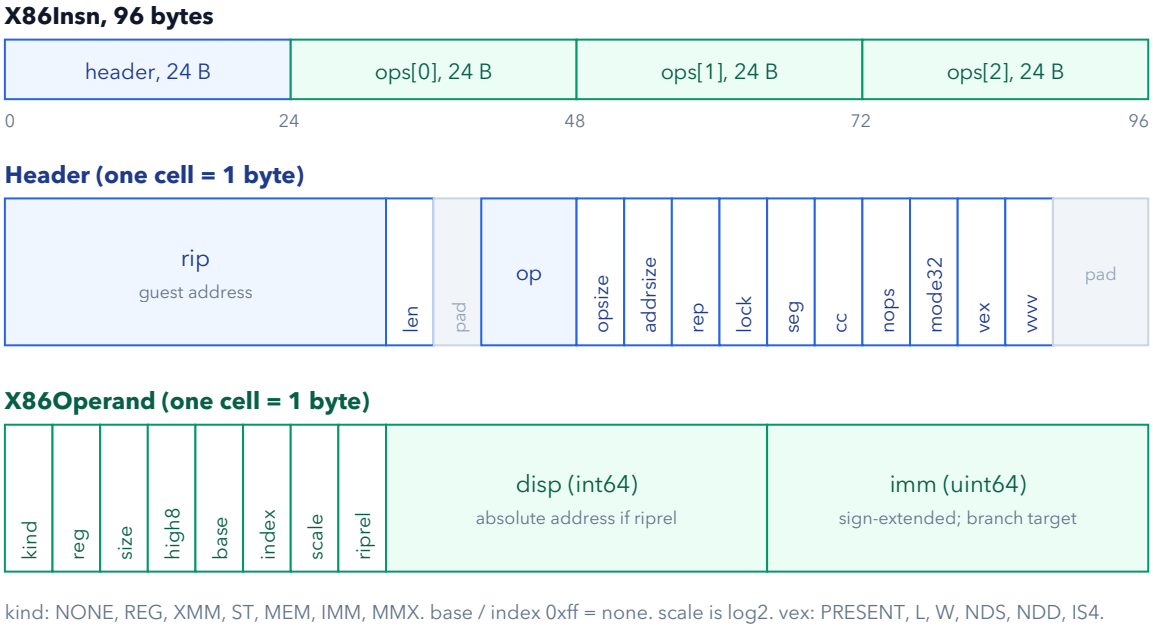


Figure 4.3. The decoded-instruction record. The header holds what applies to the whole instruction; each operand carries its own kind and size, which consumers trust over the header's operand size.

The conventions behind the record:

- **Each operand carries its own size** in bytes: 1, 2, 4, 8, 10 for an x87 extended-precision memory operand, 16 for an xmm register or a 128-bit memory operand. Consumers read `ops[i].size`, not `insn.opsize`, because the two differ in exactly the instructions that matter: `movzx` has a 2-, 4- or 8-byte destination and a 1- or 2-byte source.
- **The high-byte registers.** An 8-bit register encoding 4–7 with no REX prefix names AH, CH, DH or BH; it is stored as the base register (RAX, RCX, RDX, RBX) with `high8 = 1`. With any REX prefix the same encodings name SPL, BPL, SIL and DIL instead.
- **Immediates** arrive sign-extended to a 64-bit pattern, so a consumer can use them without knowing which immediate form was encoded. Only `mov r64, imm64` and the `moffs` forms carry raw 64-bit values, and the `moffs` offset width follows the *address* size, not always 64 bits (a bug fixed on 2026-08-19).
- **Branches carry their target.** A direct `jmp`, `jcc`, `call` or `loop` has a single immediate operand holding the absolute target, already masked to 32 or 16 bits when the operand size calls for it. Conditional instructions (`jcc`, `setcc`, `cmovcc`, `fcmovcc`) store the four-bit condition in `insn.cc`.
- **Segment registers have no operand kind.** `mov Sreg, r/m` (`MOVSEG`) encodes its destination as a one-byte immediate holding the segment index, `PUSHSEG` and `POPSEG` follow the same convention,

and the width actually moved on the stack is `insn.opsize`. The reverse direction, `mov r/m, Sreg`, is its own operation, `MOVFROMSEG`.

Twenty-four bytes per operand is generous, and the JIT does not keep it. A published translation stores 16 bytes per instruction (address, length and operation) and copies a full `X86Insn` only for the few instructions it still runs through the interpreter or must inspect during fault recovery. A GUI Wine process once carried about 870 bytes of decoded instructions per block across 215,000 blocks, 180 MB in all, before that change (Chapter 6).

Operand and address widths

The default operand size is 4 bytes; `66` makes it 2 and REX.W makes it 8, with REX.W winning when both are present. Stack and branch widths do not follow that rule, and they split three ways. The split exists because the i386 work had to be done without moving 64-bit decoding by a single bit (see the decodiff gate below):

Class	Examples	Long mode	32-bit mode
Explicit stack operand	`push` / `pop r`, `push imm`, `push` / `pop r/m`	8; `66` gives 2	4; `66` gives 2
Implicit stack traffic	`ret`, `leave`, `pushf` / `popf`	always 8 (the 16-bit forms are not implemented, and the gate requires that)	4; `66` gives 2
Near branches	`jmp` / `call` rel and r/m, `jcc`, `loop`, `jrcxz`	forced to 8, `66` ignored, as the SDM specifies	4; `66` gives 2

560 internal operations

The decoder does not hand its consumers x86 opcodes. It maps every instruction onto an internal operation, an `OcerzOp` enum value in a 16-bit field, and the engines dispatch on that. The enum's final member, `OCERZ_OP_COUNT`, is 560 (it was 548 at 90fe4d8e; twelve operations arrived in October, below). New operations are appended rather than inserted wherever the layout allows, so the enum reads as a history of the instruction set's arrival in AArchX. The one exception is the x87 block, whose region the interpreter and the JIT test as a range, so its eleven new members went at the end of that range and everything after it shifted:

Region	Values	Contents
Integer	1-102	moves, ALU, shifts and rotates, bit operations, control flow, far transfers, string operations, system and miscellany
x87	103-183	from `OCERZ_OP_X87_FIRST`: loads, stores, arithmetic, compares, transcendental functions, control; 103-172 until 2026-10-07, when eleven more were appended at the end of the region (`ficom`, `ficomp`, `fisttp`, `fbld`, `fbstp`, `fxam`, `fxtract`, `fyl2xp1`, `fnsave`, `frstor`, `ffreep`)
SSE	184-405	from `OCERZ_OP_SSE_FIRST`: SSE through most of SSE4.1, then AES-NI and PCLMULQDQ
i386-only	406-420	`pusha` / `popa`, `push` / `pop` sreg, the BCD adjusts, `bound`, `les`, `lds`, `into`, `salc` (August 2026)

Region	Values	Contents
SSE4.1/4.2 completion	421-430	`pmuldq`, `mpsadbw`, `dpps`, `crc32`, the four `pcmpXstrY` (2026-08-25)
Segment read	431	`MOVFROMSEG` (2026-09-05)
AVX and AVX2	432-481	VEX-only operations with no SSE twin: broadcasts, 128-bit inserts and extracts, permutes, `vzeroupper`, masks, gathers
FMA block	482-541	`OCERZ_OP_VFMA_FIRST` to `VFMA_LAST`: sixty reserved values, one per fused multiply-add form and operand order
Late extensions	542-559	BMI1/BMI2 (`andn`, `blsr`, `bzhi`, `pdep`, `pext`, `mulx`, `rorx`, `sarx`, `shlx`, `shrx` and the rest), `movbe`, `xsave` / `xrstor`, `rdrand`, and since 2026-10-16 `sysenter` (`0F 34`)

The region table is therefore the enum as of 6e6c64b5; every operation after the x87 block moved up by eleven when those were appended, and `decodiff` is indifferent to that because it records names, not numbers (below). Two rules govern how the enum grows, and both protect existing behaviour. The i386-only one-byte opcodes were appended at the end of the enum as it then stood, so no pre-existing operation was renumbered and none of them is reachable from 64-bit decoding. AVX-only operations are always placed above `OCERZ_OP_SSE_FIRST`, because the interpreter's last dispatch layer, the SSE unit, takes everything at or above that value that the integer layers did not claim. The field holding the operation has been 16 bits wide since the enum passed 255: in June, when it held 364 values in an 8-bit field, every operation above 255 was silently truncated and dispatched to the wrong handler.

Several encodings that differ only in ways no program can observe collapse onto one operation, which keeps both engines smaller:

Encodings	Decoded as	Why it is safe
`andpd`, `orpd`, `xorpd`, `andnpd`	the `ps` forms	bitwise operations are identical on any element type
`movapd`, `movupd`	`movaps`, `movups`	moves do not interpret their data
`lddqu`	`movdqu`	an unaligned 16-byte load either way
`movntps`, `movntpd`, `movntdq`	ordinary stores	the non-temporal hint has no observable semantics
`endbr64`, the `0F 1F` multi-byte NOPs, `fnop`, every `prefetch*`	`nop`	no architectural effect

The opcode maps

x86 has four opcode maps reachable from legacy encoding: the one-byte map, the `0F` two-byte map, and the `0F 38` and `0F 3A` three-byte maps. AArchX implements all four, plus the x87 escape bytes `D8`–`DF`, which select a further eight-way table by the ModRM `reg` field and, for register forms, by the whole ModRM byte.

The integer SIMD opcodes in the `0F` map are shared between MMX and SSE2: with no mandatory prefix, `0F FE` is `paddd mm, mm/m64`; with `66` it is `paddd xmm, xmm/m128`. AArchX decodes both onto the same operation and lets the operand kind tell them apart: an MMX form has `OCERZ_OPK_MMX` operands of size 8,

an SSE form `OCERZ_OPK_XMM` operands of size 16. That was done on 2026-09-13, and since 2026-10-04 most of those MMX forms are translated by the JIT as well. `punpcklqdq` and `punpckhqdq` exist only in the SSE2 form, so they are the two exceptions to that sharing.

x87 decoding

The x87 escape bytes `D8` to `DF` were the part of the decoder that was most wrong, and on 2026-10-07 (8f310eb8, for issue #6) they were rewritten from the SDM's tables. A Rosetta comparison, run to find why `fild` and `fistp` of a quadword corrupted data, found that the register forms were broken as well, so most x87 code computed wrong values. The decoder had, among other things:

- decoded `fadd st(0), st(i)` and the other register forms of `D8` as `st(0) op st(0)`, and swapped the operands of `DC`'s forms, where `st(i)` is the destination;
- decoded `fcomi`, `fucomi` and their popping forms as a compare of `st(0)` with itself, and `fcmovcc` as a copy of `st(0)` onto itself;
- decoded `fxam` as `fxch`, `fyl2xp1` as `fyl2x`, and `frstor` and `fnsave` as `fnstenv`;
- read the integer of `ficom` and `ficomp` as a float, and not decoded `fisttp`, `fbld`, `fbstp`, `fxtract` and `ffreep` at all.

The eleven operations added to the enum are the ones that had no name. The decoder is now one handler per escape byte (`x87_d8` to `x87_df`), each switching on the ModRM `reg` field for memory forms and on the whole ModRM byte for the register forms that name no register, and every case builds its record through four small helpers, which fix the operand convention for the whole unit:

Helper	Record it builds	Used for
`x87_mem`	one memory operand of the given size; `opsize` set to match	loads and stores (4, 8 or 10 bytes), integer forms (2, 4 or 8), `fnstcw` (2), `fbld` / `fbstp` (10)
`x87_env`	as `x87_mem`, with the size chosen by the `66` prefix	`fldenv` / `fnstenv` (14 or 28 bytes), `frstor` / `fnsave` (94 or 108)
`x87_st1`	one `OCERZ_OPK_ST` operand, size 10	compares, loads, stores and exchanges with `st(i)`
`x87_st2`	two ST operands, **destination first**: the interpreter reads `ops[1]` into `ops[0]`	arithmetic, `fcomi`, `fcmovcc` (with the condition in `insn.cc`)

Destination-first is the rule that makes the `D8` and `DC` pairs fall out: `D8 C1` is `fadd st(0), st(1)` and `DC C1` is `fadd st(1), st(0)`, and the handler for `DC` says so by passing the register as the destination. The reversed subtract and divide forms of `DC` and `DE` are the other trap: the encodings for `fsub` and `fsubr` swap meaning between the two register-destination tables, and the handler keeps a separate table for each. The 8087 and 287 control instructions (`fneni`, `fndisi`, `fnsetpm`) that a 387 ignores decode to `nop`, like `fnop`. What the operations then do belongs to the interpreter's x87 unit (Chapter 5) and, for the common forms, to the JIT (Chapter 7).

VEX

VEX prefixes exist only in long mode for AArchX's purposes: in 32-bit mode `C4` and `C5` are `LES` and `LDS`. In long mode, `decode_vex` reads the prefix bytes and turns them into the state the legacy decoder already understands, then calls into the ordinary `0F`, `0F 38` or `0F 3A` handler:

- The inverted R, X and B bits become the REX extension bits (the two-byte `C5` form has only R; X and B are zero), and REX is marked present.
- `mmmmm` selects the map: 1, 2 and 3 are `0F`, `0F 38` and `0F 3A`; anything else is undefined.
- `pp` becomes the mandatory prefix, exactly as if `66`, `F3` or `F2` had been written.
- W becomes REX.W, L the vector length, and the inverted `vvvv` the extra register operand.

Because the legacy handlers do the work, a VEX instruction decodes into the same operation as its SSE twin. `vex_finish` then tags the record in `insn.vex` with bits saying how its operands relate:

Flag	Meaning	Examples
`PRESENT`	encoded with VEX: write the full register and zero what lies above the result	every VEX instruction
`L`	256-bit (ymm) form	`vaddps ymm`
`W`	VEX.W	gather and permute variants
`NDS`	non-destructive source: the first source is `vvvv`, not the destination	arithmetic, logic, compares, unpacks, shuffles, FMA
`NDD`	the destination is `vvvv`; the ModRM register is a source	shifts by an immediate (`vpsllw ymm, ymm, imm`); the decoder moves the operands so `ops[0]` is the destination
`IS4`	a fourth register operand in the immediate's high nibble	`vblendvps`, `vblendvpd`, `vpblendvb`

A 256-bit register operand is still an `OCERZ_OPK_XMM` of size 16: the width lives in the L bit. That is deliberate. The interpreter executes most VEX.256 instructions as two 128-bit halves (Chapter 5) and the CPU state keeps the upper halves of the ymm registers in a separate array, so "xmm register n, plus its upper half when L is set" is exactly the shape both engines want. Broadcasts and half-precision converts are the exception that needs a size fix-up: their memory operand is narrower than the register, so `vex_finish` sets it to 1, 2, 4, 8 or 16 bytes as the instruction requires.

i386 mode

Wine runs 32-bit Windows programs inside a 64-bit process by switching the CPU into 32-bit compatibility mode with a far transfer to a 32-bit code selector, and back again for system calls. AArchX therefore decodes i386 too, and the way it was added is the most instructive design decision in the file.

The i386 decoding had to be added without changing long-mode decoding by a single bit. Two earlier i386 proposals each shipped a silent 64-bit regression that the entire test suite missed, so the rule became structural: **the whole i386-only slice lives in one function, `decode_i386_only`, which the one-byte dispatcher calls only when `mode32` is set**. Every byte it owns is undefined in long mode, and it sets a

`handled` flag even for the architecturally invalid forms (`BOUND`, `LES` and `LDS` with a register operand), so those report `OCERZ_EUNDEF` from there instead of falling through to a generic path. Where a width differs between the modes, the 64-bit arm keeps the literal it always had and only the 32-bit arm is new.

Difference	Long mode	32-bit mode
`40` – `4F`	REX prefixes	`inc` / `dec r32`
Default operand / address size	4 / 8	4 / 4
`mod = 00, rm = 101`	RIP-relative	absolute `disp32`
`67`	32-bit addressing	16-bit addressing table
`06 0E 16 1E` / `07 17 1F`	undefined	`push` / `pop` ES, CS, SS, DS
`C4`, `C5`	VEX	`les`, `lds` (far pointer: a 6-byte memory operand for the 32-bit form)
`27 2F 37 3F`, `D4 D5`	undefined	BCD adjust: `daa`, `das`, `aaa`, `aas`, `aam`, `aad`
`62`	undefined (EVEX prefix, not supported)	`bound`
`82`	undefined	alias of `80` (group 1, byte immediate)
`60`, `61`, `CE`, `D6`	undefined	`pusha`, `popa`, `into`, `salc`
`9A`, `EA`	undefined	far `call` / `jmp ptr16:32`, recorded selector first

Segment registers are the subtle part. In long mode, reading a segment register is uninteresting, and for a long time `mov r/m, Sreg` was decoded as a load of a constant: `0x2B` for CS, `0x23` for SS, the values macOS uses. That is only true in long mode. In a WoW64 process, 32-bit code runs under a code selector from the local descriptor table, and Wine's `RtlCaptureContext` stores what it reads there into the context record that later decides which mode an `iretq` returns to. With the constant folded in, every 32-bit exception was dispatched as 64-bit code, overflowed its stack, and killed the process within a second of starting, which is what made Steam relaunch itself in a loop. Since 2026-09-05 selectors are CPU state that the interpreter reads, and `MOVFROMSEG` is a real operation.

Wine was the only customer of the 32-bit decoder until 2026-10-10, when 32-bit Mac programs arrived as a second one (`0ca24d64`, Chapter 13). They need nothing new in the tables, since an i386 Mac binary is ordinary compatibility-mode code, but they do exercise the 32-bit paths far more than Wine's thin 32-bit side does, and one instruction was added for them. On 2026-10-16 (0144d669) `0F 34`, `sysenter`, decodes to its own operation, `OCERZ_OP_SYSENTER`, with no operands: a program that loads itself, as a UPX-packed one does, makes its system calls directly, and the interpreter hands this one to m32's raw system-call path when an i386 program is running and stops with "sysenter outside an i386 program" otherwise. `test_decode32.c` pins the two-byte form. `int 0x80` needed no decoder change, being an ordinary `int` with an immediate.

The cost of decoding

The decoder is not only on the interpreter's path. A decode runs for every instruction the interpreter executes, and several times for each instruction the JIT translates: once for the block itself, again for each successor block the flag look-ahead reads (Chapter 5), for each callee the translator splices in, and for the fusion and if-conversion look-aheads (Chapter 6). So the decoder's cost is paid in translation time as well as in interpreter speed, and in October it became the object of two measured passes.

One lookup for the prefixes, one switch for the opcode

On 2026-10-13 (39271608) the decoder's shape was changed for speed and nothing else. Measured by compiling for arm64 and counting instructions under an emulator over 600 KB of code swept linearly from the `.text` of bash, libc, python3 and libclang-cpp, a decode took about 276 instructions, 158 of them in `ocerz_decode_mode`'s own code rather than in an opcode handler: setting up the eighteen integer fields of `DecState`, copying the prefix state in and out of it around the prefix loop, a range test and a switch for every byte, a pass over the operands for the RIP-relative fix-up on every instruction, and then, in `decode_one_byte`, a chain of range tests and three switches before most opcodes were reached. The rewrite changed four things:

- `prefix_kind` classifies a byte with one table load, so a byte that is not a prefix, which is almost every first byte, ends the prefix loop at its first test.
- The prefixes are gathered in locals, and `DecState`, whose flags are now bytes, is built once after the loop. Five fields that nothing read after the loop (`avail`, `has_67`, `has_f0`, `rex`, `seg`) are gone.
- The RIP-relative fix-up runs only when `decode_modrm` says it produced a RIP-relative operand.
- `decode_one_byte` is a single switch, so a common opcode is one indirect branch away.

Measure (2026-10-13)	Before	After
arm64 instructions per decode, long mode	276.4	**213.8**
arm64 instructions per decode, 32-bit mode	247.9	**191.1**
decode speed, x86-64 host, 3.46 million instructions	1.00x	**1.20x long, 1.18x 32-bit**
interpreter CPU time, M2 Max, `-no-jit`, xbench depchain / mixed	1.000	**0.888 / 0.882**
start with nothing cached (`OCERZ_TCACHE=off`)	1.00	**0.97 to 0.99**

The point of the work is the sentence in the commit that "nothing a decode produces changes", and the evidence for it is the one this chapter's last section describes. decodiff reported IDENTICAL over its 16,777,216 openings. In addition, a harness that linked the old and the new decoder together compared the return code and every field of the `X86Insn`, on success and on failure, at every byte offset of the same 13.7 MB of code in both modes, with a random buffer length from 1 to 16 at every eighth offset, and on 20 million random byte strings with prefixes mixed in. Of 49,071,175 decodes, none differed, and the bash sweep gave the same result on an arm64 build with signed `char`.

Decoding less

The other half of the saving is decoding fewer instructions. On 2026-10-13 a profile of translation found the flag look-ahead (`xlive_decode_entry_d`) at 20% of translation time, three quarters of it decoding, and 57.6 decodes per translated block. e34ad68f stopped the look-ahead reading a successor once the in-

structions decoded so far have written all six flags, since nothing later could then change the answer (the argument is in Chapter 5), which cut decodes per translated block from 57.6 to 26.4 for perl and from 43.2 to 23.8 for ruby, and the look-ahead from 20.0% to 7.5% of translation time. The decoder was the same; it was simply asked half as often.

What it refuses, and why that is right

An emulator that guesses is worse than one that stops. A byte string AArchX does not implement decodes to `OCERZ_EUNDEF`, and the interpreter then stops the process with the address, the mode and the raw bytes. Decoded and rejected, as of this writing:

- **EVEX and AVX-512** (`62` in long mode). Rosetta does not run AVX-512 either.
- **SHA extensions** and `rdseed`.
- **Privileged and I/O instructions**: `cli`, `in`, `out`, `lgdt` and the rest of `0F 00` and `0F 01`, which a user-mode program cannot execute anyway.
- **The BCD adjust instructions in long mode**, where they do not exist (they decode in i386 mode).

Decoding is also separate from *advertising*. CPUID (Chapter 5) reports SSE3, SSSE3, CX16, SSE4.1, SSE4.2 and POPCNT, every one implemented in full, and the synthesized `hw.optional` sysctls agree. AVX2, FMA, BMI1, BMI2, F16C, LZCNT, MOVBE and XSAVE decode and execute but are not advertised, which is exactly how Rosetta runs x86-64-v3 code on macOS 15 and later. Programs that probe CPUID see a conservative processor; programs compiled for x86-64-v3, or that use AVX2 without asking (Go does), still run. The project's rule since SSE4.2 became advertised is never to set a CPUID bit without the whole family behind it.

Proving a decoder correct

Length is the property that matters most. If one instruction's length is wrong by a byte, every later instruction in the block is decoded from the wrong place, and the result is garbage that may still look plausible for a while. Four layers of checking hold the decoder to account:

1. **Hand-written cases.** `tests/unit/test_decode.c` asserts length, operation and operand fields for 246 encodings (last counted in September 2026; it was 190 in June).
2. **The assembler as a length oracle.** `tools/gen_corpus.sh` assembles a list of instructions with the host's own `clang -arch x86_64 -c`, reads the bytes back out of the object file, and writes `tests/corpus/corpus.inc`: 511 rows of the form `X("add r/m32 r32", 2, 0x01, 0x08)`. `tests/unit/test_corpus.c` decodes each byte string and requires the decoder's length to equal the assembler's. Nothing is executed; the assembler is trusted only for how long an instruction is.
3. **decodiff, the no-change gate.** `tools/decodiff.c` links `decode.o` alone and sweeps every one of the 2^{24} three-byte openings, each followed by the same 13-byte tail so 16 bytes are always available. That covers every opcode, every prefix combination up to three deep and every ModRM and SIB pairing. Each decode is rendered as canonical *text*, with the operation by name rather than by number, and hashed. `tools/decodiff-cmp.sh` builds the harness against two trees and compares the digests: it prints `IDENTICAL -- 16777216/16777216 64-bit decodes match` or lists every difference. Recording names rather than numbers makes the digest immune to renumbering the enum, which is a legitimate change, while any change in what a byte string decodes to still shows. Since 2026-10-13 (f2010b4e)

the harness defines the three arena bases that `decode.c`'s NULL-code diagnostic prints, which live in `mem.c`, so that it still links against `decode.o` and libc alone. The script always rebuilds `decode.o`, because a stale object silently produces a false IDENTICAL, the worst failure a gate can have.

4. **dec32, the correctness gate for i386.** decodiff proves 64-bit decoding did not change, but is blind to whether new 32-bit answers are right. `tools/dec32probe.c` and `tools/dec32-oracle.py` feed the same byte strings to an independent open-source x86 decoding library (version 5.0.7) in 32-bit mode and compare field by field, including exhaustive 2^{24} length sweeps. The unit tests `test_decode32.c` and `test_decode32_addr.c` pin 103 decodes, 26 rejections and 122 addressing cases (the 103rd, `sysenter`, arrived on 2026-10-16).

> **WHERE THE ORACLE IS WRONG**
>
> That library is the reference for the i386 tests, and it is documented as wrong in two places. With a 16-bit operand size the SDM has a near branch clear the upper two bytes of EIP, which the library does not model, so the oracle-checked rows keep their targets below 64 KB. And `LES` / `LDS` load a far pointer, so their memory operand is two bytes plus the operand size, six bytes for the 32-bit form, while the library reports four and prints no size keyword. A reference tool is a witness, not a judge.

Bugs that shaped it

Date	Bug	What it broke
June 2026	The operation field was 8 bits wide while the enum held 364 values	Every operation above 255 was dispatched as a different one.
2026-08-19	`moffs` offsets were always read as 64 bits	32-bit `mov eax, [moffs]` consumed four bytes too many.
2026-09-05	`mov r/m, Sreg` folded to a long-mode constant	32-bit exceptions in WoW64 ran as 64-bit code; every Steam process died at once.
2026-09-06	`lddqu` (`F2 0F F0`) was not decoded	Chromium's renderer uses it to scan UTF-8; every renderer Steam launched died with "decode failed".
2026-09-08	`movntps` / `movntpd` (`0F 2B`) missing	SDL3 stopped on them.
2026-09-08	`0F 6D` decoded as `punpcklqdq`	Every Cocoa window was squashed to 74 pixels high (below).
2026-10-07	x87 register forms with the wrong operands; `fxam` as `fxch`; `fcomi` comparing `st(0)` with itself; no decode for `fisttp`, `fbld`, `fbstp`, `fxtract`, `ffreep`	Most x87 code computed wrong values (`fadd %st(1),%st` gave 20 for 10 + 3); found by comparing against Rosetta while chasing issue #6 (x87 decoding, above).

HOW IT WAS FOUND: THE 74-PIXEL WINDOWS

For a long time every titled window in every translated Cocoa process came up 74 pixels high, and Steam's login window logged `AfterCreated ... 700x74`. The `0x60 – 0x6D` case in the `0F` map mapped `0x6C` to `punpcklqdq` and let `0x6D` fall through a default arm to the same operation: a bug present since the initial commit. With the high quadword never moved down, a geometry calculation inside the window system came out wrong, the screen's visible frame became a 74-pixel strip, and AppKit constrained every window to it.

It was found with three five-second probes instead of six-minute Steam runs: a small native AppKit program that printed the visible frame (wrong, and identically wrong under the interpreter, so not a JIT bug), then `OCERZ_MSGHEX` showing that the kernel's reply to the window server request was correct. If the data arrived right and the interpreter was wrong too, only the shared front end was left. The guest test `punpck_qdq` now pins every `punpck[hl]qdq` form, with the low and REX-extended registers.

The pattern in that table is the reason the decoder is held to such a strict standard. A decoding bug is shared by both engines, so the differential gate cannot see it; it shows up as a wrong result far away from the instruction that caused it. The defences are the ones described above: an assembler that knows the lengths, an independent decoder that knows the 32-bit fields, an exhaustive sweep that proves nothing else moved, and, for meaning, guest tests whose expected output comes from running the same binary under Rosetta.

CHAPTER 5

The interpreter and x86 flags

The interpreter is slow on purpose and correct on purpose. It defines what every x86 instruction means inside AArchX, it runs whatever the JIT declines, and it is the oracle every translation is checked against. This chapter covers the CPU state it operates on, how one instruction is executed, the exact flag formulas and the deferred form the JIT stores instead, and the corners of the x86 architecture where getting the semantics right took a real program failing first.

5,244	1,277	8,224 B	40,044
LINES ACROSS INTERP, EXT AND SSE	LINES IN X87.C, ITS OWN UNIT	PER GUEST CPU (OCERZCPU)	I386 SEQUENCES CHECKED AGAINST IT

Three jobs for one engine

A translator could exist without an interpreter, by translating everything. AArchX keeps one, and gives it three distinct jobs.

1. **Reference semantics.** For every instruction there is exactly one C implementation of what it means, in `src/interp.c`, `src/interp_ext.c` or `src/interp_sse.c`. The JIT's emitted arm64 must produce the same architectural state, and the differential gate checks it: every guest test binary is run with `-no-jit` and with the JIT, and the outputs must match byte for byte. Because the interpreter is the reference, any divergence is a JIT bug by construction (Chapter 16).
2. **Fallback.** The JIT does not translate everything. the x87 forms the JIT leaves out (transcendentals, BCD, the 80-bit memory and environment forms, and any operation on a NaN or an infinity), the SSE4.2 string instructions, string operations, `cpuid`, system calls, far transfers, misaligned locked atomics and a list of rarer forms are executed by the interpreter, one instruction at a time, from inside translated code. The JIT calls these *slow calls*.
3. **Vantage point.** Because it is the slow path, the interpreter is the right place to watch from. Trace switches that need to see each instruction live here, and `OCERZ_INTERP_LO` / `HI` can force any address range out of the JIT and into the interpreter, which is how a JIT miscompile is narrowed to one block (Chapter 18).

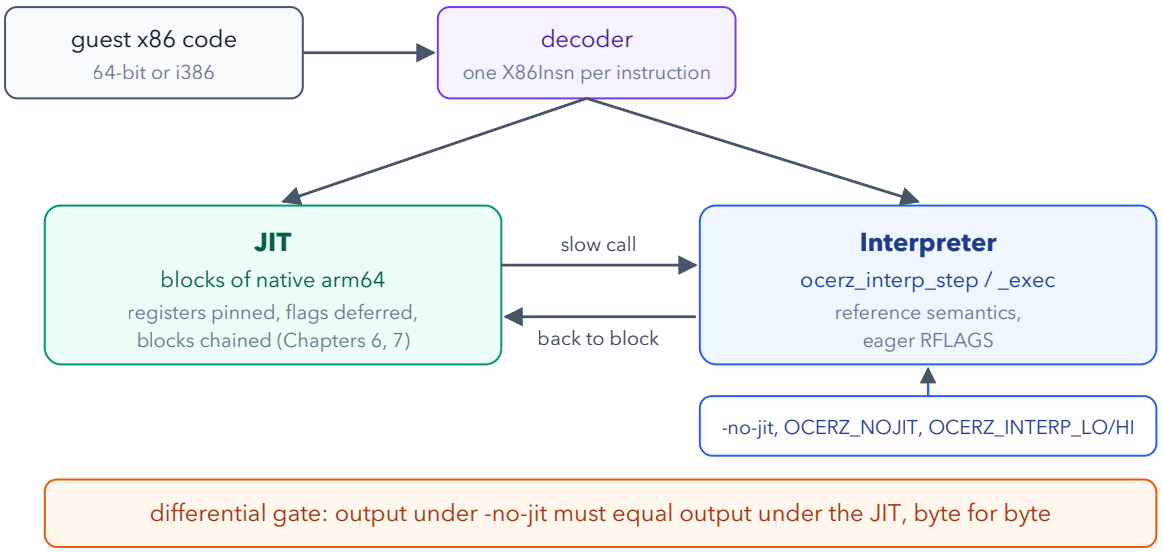


Figure 5.1. One decoder, two engines. The JIT runs what it can translate and calls the interpreter for single instructions it cannot; the interpreter can also run a whole process or chosen address ranges, which is what the differential gate and JIT bisection rely on.

The CPU state

Each guest thread has one `OcerzCPU`, declared in `include/ocerz/cpu.h`. Measured with `sizeof`, it was 7,968 bytes at 90fe4d8e and is 8,224 at 6e6c64b5. Most of it is not architectural state at all: it is per-thread bookkeeping that has to live where the JIT can reach it with one instruction. Figure 5.2 maps it as it now stands. The 256 bytes it grew by in October are three additions, each appended where its neighbours allowed and none moving a field the JIT already addressed: the exact 80-bit images of the x87 registers, with the scratch words the x87 translation keeps (`fpr_xm`, `fpr_xe`, `fpr_x_ok`, `jit_x87_top0`, `jit_fcmp_mem`, 2026-10-07 and 2026-10-08); `low_stack_delta`, which holds the Wine layout's stack delta while the register that carries `rsp` holds a host address (ab86121d, 2026-10-10, Chapter 6); and a table of up to eight blocks and chained edges that a signal rewrote so the thread would leave a loop (`sig_stop_*`, a7fd0f3f, 2026-10-14, below and Chapter 9).

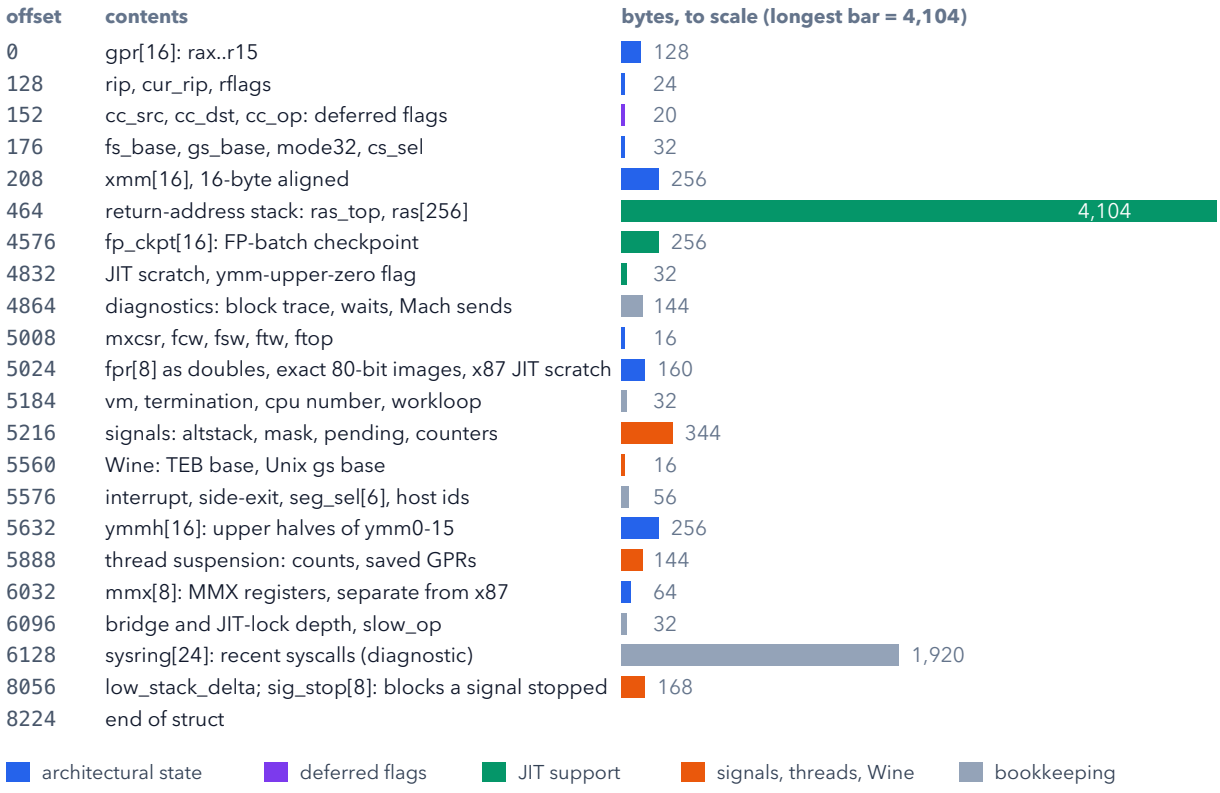


Figure 5.2. The per-thread CPU state at 6e6c64b5, by region. The return-address stack dominates; the architectural registers are a small part of it. Compared with the layout at 90fe4d8e, the x87 images and the signal-stop table are new, and every field after the x87 registers has moved up by 96 bytes.

The field order is load-bearing, and `cpu.h` says so first. The JIT reaches every field with immediate-offset loads and stores off one pinned base register, and arm64 immediates have limited reach: `xmm[]` is 16-byte aligned so that scaled 128-bit loads and stores can address it, and the return-address stack sits right after it so its first entries stay within the ±504-byte reach of a load-pair or store-pair instruction on the call and return paths. When the segment selectors were needed for WoW64, they were appended at the end for the same reason: nothing that already existed, above all neither `xmm[]` nor the stack window, could move by a byte. The same discipline added the latest diagnostic, a 24-entry ring of recent system calls, at the very end on 2026-10-02.

A few conventions matter for everything that follows:

- **General registers** are indexed in x86 encoding order (RAX = 0, RCX = 1, RDX = 2, RBX = 3, RSP = 4, RBP = 5, RSI = 6, RDI = 7, then R8–R15), so a decoded register number is directly an index.
- **Vector registers** are split: `xmm[16]` holds the low 128 bits and `ymmh[16]` the upper 128 bits of each ymm register. Legacy SSE instructions leave `ymmh` alone, VEX.128 instructions zero it, and VEX.256 instructions write it, exactly as the hardware's upper-state rules say.
- **x87** lives in `fpr[8]` as C `double`s with `ftop`, `fcw`, `fsw` and `ftw` beside them, plus, since 2026-10-07, an exact 80-bit image of each register that counts while its validity bit is set; the eight MMX registers have their own array, `mmx[8]`, rather than aliasing the x87 stack (both choices are discussed below).

- **Reset state** is what a fresh Darwin x86-64 thread sees: RFLAGS `0x202` (the always-one bit and IF), x87 control word `0x037F`, MXCSR `0x1F80`, CS selector `0x2B` and SS `0x23`.

Executing one instruction

`ocerz_interp_step` executes the instruction at `cpu->rip`. Stripped of its diagnostics, it is short:

```
int ocerz_interp_step(OcerzVM *vm, OcerzCPU *cpu)
{
    ocerz_flags_materialize(cpu);          /* a JIT-deferred flag record becomes RFLAGS */
    if (rip is in the dyld-API trap window)
        return ocerz_dyldapi_dispatch(vm, cpu);   /* Chapter 11 */

    X86Insn insn;
    rc = ocerz_decode_mode(ocerz_g2h(cpu->rip), 15, cpu->rip, &insn, cpu->mode32);
    if (rc != OCERZ_OK)
        report the bytes, the module, recent rips; return OCERZ_STEP_FATAL;

    vm->insn_count++;
    cpu->cur_rip = cpu->rip;
    cpu->rip    += insn.len;                /* rip is ALREADY the next instruction */
    if (insn.mode32) cpu->rip = (uint32_t)cpu->rip;
    rc = ocerz_interp_exec(vm, cpu, &insn);
    if (cpu->mode32) cpu->rip = (uint32_t)cpu->rip;  /* the mode may have changed */
    return rc;
}
```

Three details carry weight. First, **`rip` advances before the handler runs**, so a non-branching handler never touches it, a branch simply overwrites it, `call` pushes it, and `syscall` publishes it in RCX as the hardware does. Second, the **decode mode is chosen in exactly one place**, from `cpu->mode32`; no handler ever asks which mode it is in, because the decoded instruction already says. Third, **EIP is re-wrapped after execution** keyed on the mode the CPU is in *afterwards*, because a far transfer that just left 32-bit mode has legitimately produced a full 64-bit `rip`.

`ocerz_interp_exec` is the dispatch half. It is layered by file: the integer core in `interp.c` handles what it owns; anything else falls to `ocerz_interp_ext` (string operations, `cpuid`, `rdtsc`, CRC-32C, x87, FXSAVE and XSAVE, MXCSR access, the bit-manipulation extensions); operations at or above `OCERZ_OP_SSE_FIRST` fall to `ocerz_interp_sse`; and anything left reaches `ocerz_unimpl`, which stops with the operation name, the address and the raw bytes. The JIT's slow calls enter at `ocerz_interp_exec` with an already-decoded instruction, so for every instruction the JIT hands over there is exactly one implementation. Two of the system-entry instructions are routed by what is running: `int 0x80` and, since 2026-10-16, `sysenter` go to m32's raw system-call path when the process is an i386 Mac program and `m32_active` is set (Chapter 13), and otherwise `int` stops the process as an unexpected trap and `sysenter` as unimplemented ("sysenter outside an i386 program"). `popf` also changed for those programs: it now keeps AC (bit 18) and ID (bit 21) writable, because a 32-bit program's startup code probes for CPUID by toggling ID, and a frozen ID made it take the no-CPUID path (0ca24d64, 2026-10-10).

A step returns one of five results, two of which are not ordinary control flow:

Result	Meaning
`OCERZ_STEP_OK`	executed; continue

Result	Meaning
`OCERZ_STEP_EXIT`	the guest exited
`OCERZ_STEP_FATAL`	undecodable or unimplemented; the process stops with a report
`OCERZ_STEP_REDIRECT`	a non-branching instruction changed `rip` because a guest signal was delivered; the caller must re-dispatch
`OCERZ_STEP_PROFILE`	a profiled JIT side exit came back to C only to be counted; continue

Signals between two steps

The host's signal handler never runs guest code. It only marks the signal pending on the thread's CPU, and the guest handler runs later, from the thread's own run loop, where all of the guest state is in `OcerzCPU`. Until 2026-10-14 that happened after the thread's next system call or bridged call and nowhere else, in the interpreter as much as in the JIT, so a loop of `dec` and `jnz` under a 5 ms `ITIMER_REAL` got no SIGALRM until it ended; under Wine, where `SuspendThread` is a SIGUSR1 sent to the thread, Mono's collector waited for every busy thread to finish its loop. a7fd0f3f made both run loops deliver a pending signal that the thread does not block between two steps. That was all the interpreter needed. Translated code returns to a run loop only when a block exits to it, so the JIT side stops the loop from the signal handler: it unchains the interrupted block's edges and a few blocks beyond, so that the thread's next jump out leaves for the dispatcher, and the dispatch stub itself returns to C while such a signal is pending. The mechanism and its race analysis are in Chapter 9 and Chapter 7; what belongs here is that the interpreter's contract is now "a signal is delivered between any two instructions", which is what the `OCERZ_STEP_REDIRECT` result exists to carry. `OCERZ_NO_SIGNAL_STOP=1` restores the old delivery points, and m32 processes keep them, since delivery there would enter a 64-bit handler. The dynamic test `loop_signal` pins it under both engines with seven loop shapes and a thread that twenty `pthread_kill` calls must each reach while it spins.

Reading and writing operands

No handler touches an operand directly. `include/ocerz/interp_common.h` holds the inline accessors, and it is the one place the easy-to-get-wrong rules live:

- A **32-bit register write zero-extends** into the upper half, including `cmovcc` with a false condition, which still writes its destination. 8- and 16-bit writes merge, and `high8` addresses bits 8–15.
- `ocerz_ea` computes `disp + base + (index << scale)`, truncates to 32 bits under a 32-bit address size, and then adds the FS or GS base. `lea` has its own path, because it must never add a segment base.
- **The stack pointer is RSP in long mode and ESP in i386 mode**, so in 32-bit mode every push and pop wraps at 2^{32}. The mode comes from the decoded instruction, never from a test in the handler.
- **Wine's TLS slot.** The absolute form `gs:[0x58]` is special-cased. Wine on macOS runs 64-bit PE code with GS pointing at the Darwin thread-specific data and mirrors the TEB fields PE code reads into Wine's reserved slots, but `ThreadLocalStoragePointer` is the one field that changes after the mirror is taken, so a thread whose snapshot preceded its TLS setup read zero and MSVC `__declspec(thread)` code crashed. Slot 6 holds the TEB's self pointer on Wine threads and zero elsewhere, so reading through it yields the live field and selects itself.

x86 flags, computed eagerly

x86 arithmetic instructions write six status flags in RFLAGS as a side effect. AArchX's definition of them is `src/flags.c` (230 lines): one helper per instruction family, each taking the operand width in bytes and the inputs and result, and each rewriting the six arithmetic flags while leaving the control bits (TF, IF, DF) alone.

Flag	Bit	Meaning
CF	0	carry out of (or borrow into) the most significant bit
PF	2	even parity of the *low byte* of the result
AF	4	carry or borrow between bits 3 and 4 (used by BCD arithmetic)
ZF	6	the result, at the operation's width, is zero
SF	7	the result's most significant bit
OF	11	signed overflow

The formulas, with all values masked to the operation's width first:

- **ZF, SF, PF** come from the result alone. PF looks only at the low eight bits, whatever the width.
- **ADD and ADC:** CF if `res < a`, or `res == a` with a carry in. OF from the top bit of `~(a ^ b) & (a ^ res)`: two operands of the same sign produced a result of the other sign. AF from bit 4 of `a ^ b ^ res`.
- **SUB, SBB, CMP, NEG:** CF if `b > a`, or `b == a` with a borrow in. OF from the top bit of `(a ^ b) & (a ^ res)`. AF as for add.
- **AND, OR, XOR, TEST** set ZF, SF and PF from the result and clear CF, OF and AF.
- **INC and DEC never touch CF.** OF is positional: INC overflows exactly when it lands on the minimum signed value, DEC on the maximum. AF when the low nibble wrapped to 0 (INC) or to F (DEC).
- **Shifts** are never called with a count of zero, because a zero-count shift leaves every flag unchanged and the caller skips the helper (but see the register write below). SHL: CF is the last bit shifted out of the top, OF is CF xor the result's top bit. SHR: CF is the last bit out of the bottom, OF the original top bit. SAR: CF from the sign-extended value, OF zero.
- **ROL and ROR** write CF and OF only. CF is the bit that wrapped round: the new low bit for ROL, the new top bit for ROR. OF is architecturally undefined past a count of 1, but x86 hardware and Rosetta compute it with the count-1 formula for every count, so the interpreter does too. Until 2026-10-09 (edf0b16f) it left OF unchanged for any other count, and the two engines differed from Rosetta in OF alike.

The worked examples below are 8-bit, where the flags are easy to check by hand:

Instruction	Result	CF	PF	AF	ZF	SF	OF	Why
`add 0x7F, 0x01`	`0x80`	0	0	1	0	1	1	positive plus positive gave negative; nibble carried
`add 0xFF, 0x01`	`0x00`	1	1	1	1	0	0	unsigned wrap; zero has even parity
`sub 0x00, 0x01`	`0xFF`	1	1	1	0	1	0	borrow; eight one-bits is even parity
`sub 0x80, 0x01`	`0x7F`	0	0	1	0	0	1	most negative minus one overflows

Instruction	Result	CF	PF	AF	ZF	SF	OF	Why
`cmp 0x05, 0x05`	(`0x00`)	0	1	0	1	0	0	equal
`and 0xF0, 0x0F`	`0x00`	0	1	0	1	0	0	logic clears CF, OF, AF
`inc 0x7F`	`0x80`	=	0	1	0	1	1	CF unchanged; landed on the minimum
`shl 0x81, 1`	`0x02`	1	0	0	0	0	1	bit 7 shifted out; OF = CF xor top bit
`mul 0x10, 0x10`	`0x0100`	1	0	0	0	0	1	high half non-zero; the other four cleared

Conditions

`ocerz_cc_eval` turns RFLAGS and a four-bit condition code into true or false. x86 arranges its sixteen conditions as eight pairs that differ only in bit 0, so the function computes a base predicate from `cc >> 1` and inverts it when bit 0 is set:

cc >> 1	Conditions (bit 0 = 0 / 1)	Predicate
0	O / NO	OF
1	B / AE	CF
2	E / NE	ZF
3	BE / A	CF or ZF
4	S / NS	SF
5	P / NP	PF
6	L / GE	SF ≠ OF
7	LE / G	ZF or SF ≠ OF

Undefined flags follow Rosetta

The architecture leaves some flags undefined after some instructions. "Undefined" does not mean a program cannot observe them, and AArchX's tests compare against the same binary run under Rosetta, so the rule since 2026-09-03 is to produce what Rosetta produces:

- **MUL and IMUL** define only CF and OF, set when the high half carries information (non-zero for MUL, not the sign extension of the low half for IMUL). SF, ZF, AF and PF are undefined, and Rosetta leaves all four clear whatever the result and the flags going in, so the helpers clear them. The June code had derived them from the low half.
- **CMPXCHG** sets its flags as for `dest − accumulator` and writes the accumulator in both outcomes, so the 32-bit form zero-extends RAX on a match as well.
- **DIV and IDIV** leave flags architecturally undefined, but the interpreter produces specific values and the differential gate compares them, so the liveness table treats them as defined (below).
- **System-call returns** reproduce Rosetta's fixed patterns for the arithmetic flags, probed on 2026-09-03: PF and AF after a successful Unix call, CF, ZF and PF after a failed one, AF after a Mach trap. Only CF carries meaning (libSystem's stubs branch on it), but a program that reads the rest sees what it would see under Rosetta. Machdep calls leave the flags alone (Chapter 9).

A count of zero still writes the register

A 32-bit shift or rotate whose count masks to 0 changes no flag, but it still writes its destination, and a 32-bit write clears the upper half of the 64-bit register, which is what hardware and Rosetta do. The interpreter used to return early and the JIT used to skip an immediate count of 0 entirely, so the upper half stayed stale; a bit buffer shifted by a count that happens to be 0 is the usual way for code to depend on it. `zero_count_write` in `src/interp.c` now writes the truncated value back for a 32-bit register destination in 64-bit mode (a memory destination is left alone, and 32-bit code has no upper half). The JIT's twin is `emit_zero_count_write` (6c6389cd, 2026-10-09; test `shift_cl_flags`).

The deferred record

The interpreter computes flags eagerly, after every instruction that writes them, because it is the reference and simplicity is the point. The JIT cannot afford to: most flags are never read. It computes flags only where something reads them, usually by fusing a compare directly into an arm64 conditional branch (Chapter 6). When it cannot fuse but also cannot prove the flags dead, it stores a **deferred record** instead: three fields in the CPU state.

```
uint64_t cc_src;     /* first input                        */
uint64_t cc_dst;     /* second input, result, or count     */
uint32_t cc_op;      /* kind | size << 8 | carry_in << 16  */

/* kinds: OCERZ_CC_ADD, _SUB, _LOGIC, _INC, _DEC, _SHL, _SHR, _SAR, _MUL, _IMUL */
```

`ocerz_flags_materialize` turns a pending record into RFLAGS by calling the very same helpers the interpreter uses, then marks the record empty. That gives one formula per flag in the whole project, and it is why the interpreter's step function starts by materializing: when execution falls from translated code into the interpreter, the record left behind becomes ordinary RFLAGS before the next instruction can read them. A record is also materialized for `pushf`, for a signal frame, for a fault, and for any condition the JIT did not fuse.

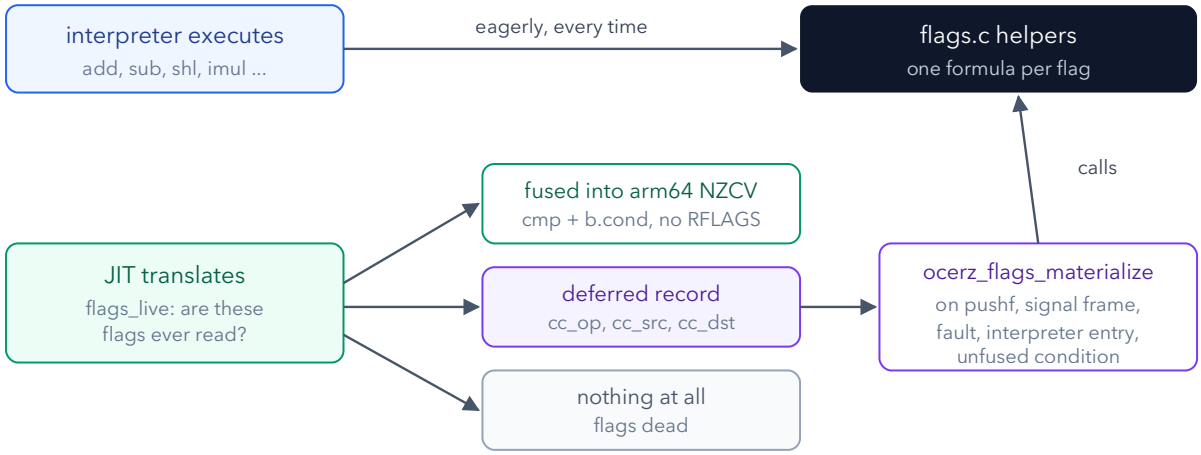


Figure 5.3. Three fates for a flag in translated code, one formula for all of them. Whatever the JIT does, a flag that is finally read is computed by the same `flags.c` helper the interpreter uses.

Flag liveness

Whether a flag is ever read is decided by `src/flags_live.c` (298 lines), a table of what each instruction *defines* and *uses*. The JIT runs a liveness pass over each block with it, and a flag that no later instruction uses before the next definition is dead and never computed. Every entry is a claim about what an instruction may do, so the safe direction is always "**defines less, uses more**": overstating a definition could kill a flag something reads, while overstating a use only costs a little speed.

Instruction class	Defines	Uses
moves, `lea`, `not`, `push`/`pop`, `call`/`ret`, `xchg`, string moves, `cpuid`	nothing	nothing
ADD, SUB, logic, compares, `popcnt`, `rdrand`, `xadd`, `cmpxchg`	all six	(ADC/SBB use CF)
`bt`, `bts`, `btr`, `btc`	CF	nothing
`bsf`, `bsr`	ZF only	the rest stay as they were
`tzcnt`, `lzcnt`	CF, ZF	nothing
BMI: `andn`, `blsr`, `blsmsk`, `blsi`, `bzhi`	CF, ZF, SF, OF	nothing
Shifts and rotates by `%cl`, RCL and RCR, SHLD/SHRD by a register	*may* define	all six
ROL and ROR by an immediate	CF and OF, or nothing if the count masks to zero	nothing
SHLD/SHRD by an immediate	all six, or nothing if the count masks to zero	nothing
SSE and x87 compares into RFLAGS, `ptest`	all six	nothing
`fcmovcc`	nothing	CF, ZF, PF
the rest of x87 and SSE, every VEX instruction	nothing	nothing

Two rows deserve explanation. A shift by `%cl` is a *may*-define, and until 2026-10-09 so was every rotate: a count that masks to zero leaves every flag untouched while any other count writes them, and that cannot be known when translating, so the instruction is treated as both using all flags and not reliably defining them. An audit on 2026-08-18 found stale flags exactly there. With an immediate count the question can be answered, and from edf0b16f on it is: ROL and ROR by a nonzero count write CF and OF and leave the other four flags alone. The old entry, "uses and defines all six", kept the flags of an `add` alive through every rotate that followed it. Apple's x86 SHA-256 in CommonCrypto (run from the shared cache) adds and then rotates three times before an `xor` overwrites every flag, so each round's add saved its operands for lazy flags and the first rotate called out to materialize them, spilling every register on the way. After the change its hottest block went from 1,154 arm64 words to 436 and each rotate became one instruction; `CC_SHA256` over 64 MB in cache mode went from 1.34 s to 0.44 s (Rosetta 0.33 s), measured on 2026-10-09. RCL, RCR and rotates by `%cl` keep the conservative entry. And **DIV and IDIV are treated as defining** flags the architecture calls undefined, because the interpreter produces specific values and the differential gate compares them; "undefined" is not "killed".

The table can also make every memory-touching instruction a **fault barrier**, forcing all flags live across it so that a guest fault handler sees exact flags even where they are dead. That was the original design, and it cost real speed, because it forced flag producers to keep records that nothing in the program's own code ever read. Since 2026-08-15 the barrier is opt-in (`OCERZ_FAULT_FLAG_BARRIER=1`): flags that are live are always recoverable in a fault, and only flags that are dead may be stale in a signal context. The commit that made it opt-in, which also classified most x87 and SSE instructions as flag-neutral, took the `vm` benchmark kernel from 3.9 to 1.6 times Rosetta's time. (The comment at the top of `flags_live.c` still describes the barrier as unconditional.)

Liveness across blocks

The table answers questions inside one instruction stream. The JIT also has to know which flags are live *on entry* to a block it is about to chain to, and that is a question about what the successors read. `xlive_decode_entry_d` in `src/jit.c` answers it by decoding the successor block up to its terminator, following that terminator's targets down to `OCERZ_XLIVE_DEPTH` blocks (3 unless the variable says otherwise), treating everything past that depth as every flag live, and folding the def/use pairs of this file backwards over what it saw. The fold gives a flag at the entry only if some instruction reads it before any earlier one writes it, or if it is live out of the block and no instruction in the block writes it. Chapter 6 shows what the translator does with the answer. Three changes in October made the look-ahead cheaper or more precise without changing what it concludes where it was already right:

- **Stop reading once every flag is written** (e34ad68f, 2026-10-13). Once the instructions decoded so far have written all six flags, neither condition can hold for anything later, so the decode loop stops at that instruction and the block it hands on ends in a non-terminator, which the code after the loop already treats as every flag live out, with no successors followed. The commit's argument is a proof, and it was also checked: from every byte offset of bash's and libc's `.text`, in both modes (4.5 million block starts), where the loop stopped early (64% to 66% of them) the fold over the prefix was compared with the fold over the whole block under 64 live-out values each, 187 million comparisons, none different. The effect was on translation time: decodes per translated block fell from 57.6 to 26.4 for perl and from 43.2 to 23.8 for ruby, and `xlive_decode_entry_d` from 20.0% to 7.5% of translation time (M2 Max, nothing cached). `OCERZ_NO_XLIVE_KILL=1` reads every successor to its end as before.
- **A loop's own start is a least fixed point** (5fc69e49, 2026-10-11). A block whose `jcc` goes back to its own start used to follow that edge like any other, hit the depth limit, and come back with every flag live; an `inc` or `dec` closing the loop leaves CF as it was, so CF stayed live round every trip, and the loop's back edge called `ocerz_flags_materialize` on each iteration. Such a block's own edge now starts with no flags live and the answer is iterated until it stops growing. 7-Zip's LZ copy loop was 18% of its decompression time and 136 of 150 samples in `ocerz_flags_materialize` came from its back edge; under Wine Staging, decompression went from 71,586 to 85,141 KiB/s (two runs each, Rosetta 124,000). `OCERZ_NO_XLIVE_SELF=1` follows the edge as before.
- **inc and dec keep what they must, only when it is read** (edf0b16f, 2026-10-09). `inc` and `dec` leave CF alone, and the JIT used to rebuild the old CF into the lazy-flags record whenever any flag they set was live, a call-out in most cases. The liveness pass now keeps every flag live after each instruction (`fl_live`), not only those the instruction defines, so the emitters rebuild the old CF only when CF is live afterwards; and a branch on their result is decided inline for every condition except the carry and parity ones. `if (--o->refcnt == 0) free(o)`, the pattern behind `CFRelease`, `objc_release` and `shared_ptr`, ran a release loop of 100 million calls in 1.69 s before and 0.21 s after (Rosetta 0.13 s).

These are all consequences of one rule in this file's header: the safe direction is "defines less, uses more", and each October change made a table entry or a look-ahead more exact without crossing it. `tests/dynamic/rot_flags.c`, `incdec_flags.c` and `xlive_self.c` pin them against Rosetta in both layouts, and each commit lists the planted bug the test must catch.

Integer corners

Misaligned locked atomics

x86 makes a LOCK-prefixed access atomic at any alignment, because the bus or cache-line lock covers a split access. arm64's LSE atomics, and clang's `__atomic_*` built on them, fault on a misaligned address. Wine's `CRITICAL_SECTION` inside a packed Valve structure sits at offset `+0x446`, so every `EnterCriticalSection` in Steam's CEF host is a `lock cmpxchg` on a doubleword that is 2 mod 4. The JIT sends misaligned locked accesses to the interpreter, which takes one of 256 spin locks chosen by address (`(addr >> 3) & 255`) and performs the read-modify-write with plain unaligned copies. That is atomic against every other misaligned access to the same location, which is the guarantee that actually matters. Aligned locked accesses use the real atomic instructions.

Divides, fast-fail and the stack pointer

- **A divide fault is a real fault.** `rip` stays at the `div`, and the guest's handler sees SIGFPE with `FPE_INTDIV` or `FPE_INTOVF`, as on Darwin; with no handler the process dies.
- **`int 0x29`** (Windows' `__fastfail`) ends the process in the interpreter rather than being handed to the guest. Dispatching it as an exception so Wine could raise `STATUS_STACK_BUFFER_OVERRUN` was tried and measured worse: the thread that fast-fails is already deep, the dispatch overflowed its stack, and it wedged holding ntdll's loader lock, so the process could never exit.
- **`pop` into `[rsp+disp]`** computes its address *after* the pop has adjusted the stack pointer, as the SDM says. V8's `TailCallRuntime` trampoline moves its return address with `pop qword [rsp+0x98]`; computed with the old `rsp` it landed one slot low and V8 returned into a stale heap pointer (fixed 2026-09-08, guest test `pop_rsp_mem`). `pop rsp` takes its new value from the slot, not from the adjustment.

Strings and bits

String instructions count in rCX and step rSI and rDI at the **address size**: the 64-bit registers at 8, the 32-bit ones at 4, and the 16-bit ones at 2, the last only in 32-bit code with a `67` prefix, written back through the register accessor so only the low half changes. A whole `rep` loop runs inside one step. A forward `rep movs` or `rep stos` runs at the host's `memmove`, `memset` or `memset_pattern4/8` when its range is one piece of host memory (wholly on one side of the low window's edge, clear of the top strip and the commpage, its first and last bytes as far apart on the host as in the guest); a `movs` whose destination starts inside its source (a pattern copy), a backward one, a segment override, a count under 32 and native mode's pinned pages keep the element loop. Pages that hold translated code are disarmed and their translations dropped first, and `rsi`, `rdi` and `rcx` move only once the range is done, so a fault inside it, such as a guard page Wine commits on demand, reaches the guest with the instruction not yet started. Wine's `msvcrt` `memcpy` is `rep movsb` above a few hundred bytes, and a benchmark of two million copies of 8 bytes to 4 KB took 3,874 ms on AArchX against 188 ms under Rosetta before b839cdfa (2026-10-07), and 145 ms after; under WoW64, 188 ms against Rosetta's 446 ms. `cmps` and `scas` take their flags from the last iteration ex-

ecuted, and a zero-count `rep` leaves the flags untouched. `bt` and its relatives with a memory operand and a register bit offset address memory outside the operand: the byte at `ea + (offset >> 3)`, sign-extended. `bsf` and `bsr` on a zero source set ZF and leave the destination unchanged; `tzcnt` and `lzcnt` return the operand width with CF set.

CPUID and the clock

Leaf	Value
0	highest leaf `0xD`, vendor `GenuineIntel`
1	EAX `0x000306A9` (family 6, model 58), EBX `0x00100800`, ECX `0x00982201`, EDX `0x078BFBFF`
7	zero: no extended features advertised
`0xD`	XSAVE state: subleaf 0 EAX 7 (x87, SSE, AVX), area `0x340` bytes; subleaf 2 AVX state `0x100` bytes at `0x240`
`0x80000001`	ECX `0x1` (LAHF/SAHF in long mode), EDX `0x28100800`
`0x80000002 - 4`	brand string `Ocerz x86_64 Emulated CPU`

ECX `0x00982201` is SSE3, SSSE3, CX16, SSE4.1, SSE4.2 and POPCNT, each implemented in full. SSE4.2 has been advertised since 2026-08-25, because Steam's bootstrapper refuses to start the client on a processor without it; the rule since then is never to advertise a bit without the whole family behind it. AVX2, FMA, BMI1, BMI2, F16C, LZCNT and MOVBE execute without being advertised, matching what Rosetta does on macOS 15 and later. `rdtsc` returns `mach_absolute_time()` converted to nanoseconds: monotonic, but not a cycle counter.

The x87 unit

Until 2026-10-07 the x87 code was a few hundred lines of `interp_ext.c` that treated the register stack as eight C `double`s and was wrong in many places. 8f310eb8 (issue #6) moved it into `src/x87.c` (1,277 lines, with its entry points in `include/ocerz/x87.h`), rewrote it from the SDM, and checked it against Rosetta; it shipped in AArchX 0.6 (7fc00672, the same day). 542 lines left `interp_ext.c` in the same commit, which is why that file is now 763 lines. The file's explanation block is the specification, and the translated forms in the JIT are checked against it bit for bit (Chapter 7).

Doubles with an exact twin

The registers are still IEEE doubles, indexed by physical register (`ST(i)` is `fpr[(ftop + i) & 7]`), because a double keeps x87 arithmetic on the host's floating-point unit, and because `rosettax87_jit`, which 32-bit games ran on before, makes the same choice. What a double cannot hold is a value that was *loaded* as more than a double. A 64-bit integer is not a matter of precision at all: Delphi's and Free Pascal's `Move()` copy eight bytes at a time as `fild qword` and `fistp qword`, and a rounded integer is corrupted memory. An 80-bit value moved with `fld tbyte` and `fstp tbyte`, which is how compilers copy a `long double`, is the same problem. So each register also has an exact 80-bit image, a 64-bit significand in `fpr_xm` and a sign and exponent in `fpr_xe`, and the image counts only while its bit in `fpr_x_ok` is set *and* the double is still, bit for bit, the image rounded to double. Figure 5.4 shows which operations fill, carry and read it.

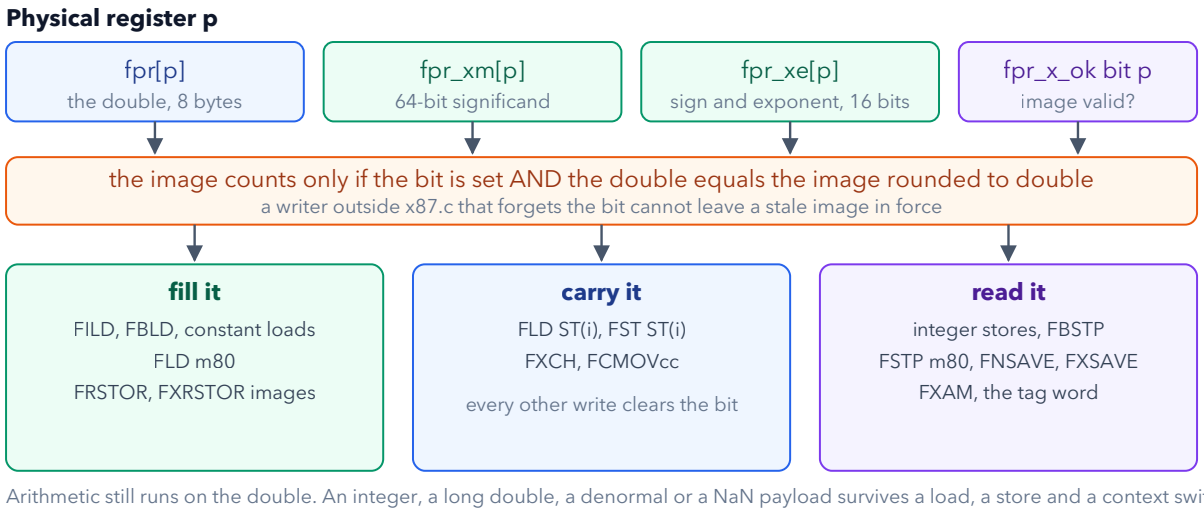


Figure 5.4. The x87 register model since 2026-10-07. Each physical register is a double plus an exact 80-bit image whose validity is a bit and a consistency check; loads fill the image, copies carry it, and the operations that need more than a double read it.

HOW IT WAS FOUND: EIGHT BYTES THROUGH A DOUBLE

The game OMSI 2 died reading its own form resource. Delphi's and Free Pascal's `Move()` copy a block eight bytes at a time through the x87 stack, as `fild qword` followed by `fistp qword`, and a double holds only 53 bits of a 64-bit integer, so every qword with a high bit pattern came out rounded: not an inaccurate number but corrupted memory. Issue #6 asked for exact `fild` and `fistp` of a quadword, and a comparison against Rosetta turned up the register forms the decoder had wrong as well (Chapter 4). The issue's check now reports 0 of 9 qwords corrupted under both engines.

The tag word and the control word

`ftw` is the abridged tag word, bit *p* set while physical register *p* holds a value, as FXSAVE stores it; FNSTENV and FNSAVE expand it to the two-bit form from the register contents. Pushes, pops, FFREE and FXCH keep it, which is what lets FXAM report an empty register. A push onto a full register or a read of an empty one is deliberately not a stack fault: the value passes through, because a tag that went wrong somewhere outside the unit must never turn data into NaNs. TOP is `ftop`; the TOP field of `fsw` is ignored on write and synthesized whenever `fsw` is read.

Each operation runs on the host's floating-point unit with the FPCR set from the x87 control word, rounding from RC and flush-to-zero off, whatever MXCSR asked of SSE, and with the FPSR saved around it, so the exception flags an operation raises reach `fsw` and never MXCSR, as on x86. Precision control 24, which Direct3D 9 sets, rounds the results of FADD, FSUB, FMUL, FDIV and FSQRT to a 24-bit significand without narrowing the exponent; precision control 53 and 64 both leave the double. NaNs follow the x87 rules, which are neither arm64's nor SSE's: an invalid operation with no NaN operand gives the negative default NaN, a single NaN operand comes back quieted, and of two NaNs a QNaN beats an SNaN and otherwise the larger significand wins, the positive one on a tie, as Rosetta does.

The corners, and where Rosetta wins

- **Integer stores.** FIST, FISTP and FISTTP store the integer indefinite (`0x8000`, `0x80000000` or `0x8000000000000000`) for a NaN or an out-of-range value, with IE set. They also set C1 when the store was inexact and rounded away from zero, and clear it otherwise, FISTTP and exact and invalid stores

included (b3e701b9, 2026-10-08). Before that, 54 of the 241 lines of the `fist_rc` test differed from Rosetta, all of them in C1, in the interpreter and the JIT alike.

- **FPREM and FPREM1** reduce in one step when the exponents differ by less than 64, with C2 clear and the low three quotient bits in C0, C3 and C1; otherwise they take a partial step and set C2, as the hardware does. The partial remainder is `fmod` against the divisor scaled by a power of two, which is exact.
- **FSIN, FCOS, FPTAN and FSINCOS** leave an operand of magnitude 2^{63} or more alone and set C2.
- **The control word**'s bit 6 always reads as one.
- **Not modelled:** unmasked exceptions do not trap, and the instruction and operand pointers in the environment images are zero.
- **Where Rosetta and the SDM part, Rosetta wins**, since it is what these programs ran on: FNSTENV leaves the exception masks alone, and the pad words of the environment images are zero.

Arithmetic is still in doubles, so `max_double * max_double` computed in `long double` returns infinity rather than about 3.2×10^{616}. That is a documented deviation, not an oversight: values are exact when they are moved and rounded to 53 bits when they are computed on.

Saving the state

FXSAVE and XSAVE used to store raw doubles in physical order, and FXRSTOR never restored TOP. They now store the eighty-bit registers `ST(0)` to `ST(7)` in stack order and the restore sets TOP, which is the format Wine copies into 32-bit thread contexts, so a context switch of a 32-bit Windows thread now keeps an x87 register exactly, denormals and NaN payloads included. A `long double` that a native call returns reaches the guest through `ocerz_x87_push`, which is why that function is in the unit's header. The MMX registers are kept apart from the x87 stack, in `mmx[8]`, rather than aliasing its mantissas as the hardware does. Programs that use both in one stretch without `emms` are broken on real hardware anyway, and the separation keeps both units simple; the cost is that FXSAVE and signal frames do not carry the MMX registers.

How it is tested

The tests are five dynamic programs, `x87_arith`, `x87_compare`, `x87_int`, `x87_trans` and `x87_state`, whose expected output is 1,759 lines recorded from Rosetta, run under the JIT and the interpreter in both memory layouts. Three things are masked because Rosetta's answer is not one to copy: C1 after an inexact divide, which depends on precision; a loaded tag word, which Rosetta echoes; and Rosetta's tag word after FXRSTOR, which contradicts the registers it just restored. The JIT's translated forms are compared with this unit by the i386 differential gate, which since 9f0b7087 (2026-10-07) generates 20,000 random x87 sequences besides the 20,000 integer ones, with NaNs, infinities, zeros, double denormals and integers above 2^{53} at stack depths 1 to 8, comparing each register's double, image, image bit, tags, TOP, `fcw` and `fsw` (Chapter 16).

SSE and AVX

`src/interp_sse.c` (2,542 lines) covers scalar and packed floating point, every compare predicate, the conversions with x86's "integer indefinite" results, the integer SIMD set, AES-NI, PCLMULQDQ, the SSE4.2 string instructions, and the AVX forms layered on top. Alignment is not enforced: `movaps` on a misaligned address behaves like `movups` instead of faulting. `rcpps` and `rsqrtps` return exact results instead of x86's 12-bit approximations, a deliberate deviation listed in Chapter 19.

NaNs the x86 way

x86 and arm64 disagree about NaNs, and programs do notice. The interpreter applies x86's rule to add, subtract, multiply, divide and square root: if the result is a NaN, then if the first source (the destination register) was a NaN the result is that NaN, quieted; otherwise if the second source was a NaN, that one quieted; otherwise the operation was invalid and the result is the *negative* default NaN, which x86 calls the real indefinite.

Operation (double)	x86 and AArchX	arm64 without FPCR.AH
`0.0 / 0.0`	`0xFFF8000000000000`	`0x7FF8000000000000`
`inf − inf`	`0xFFF8000000000000`	`0x7FF8000000000000`
`1.0 + SNaN 0x7FF0000000000001`	`0x7FF8000000000001`	`0x7FF8000000000001`
`QNaN 0x7FF8000000000005 + SNaN 0x7FF0000000000001`	`0x7FF8000000000005` : the first operand	`0x7FF8000000000001` : the signalling one wins

MIN and MAX keep x86's own rule: the second operand is returned when either operand is a NaN, or when both are zero. On processors that report `FEAT_AFP`, one bit of the arm64 floating-point control register (FPCR.AH) makes the hardware itself produce the x86 column of this table, and the JIT then emits arithmetic without software checks (Chapter 7). The interpreter's C code produces x86's answers either way.

Rounding modes

The rounding-control field of MXCSR is honoured since 2026-09-11: `ocerz_apply_mxcsr_round` drives the host's rounding mode, which both the interpreter's C arithmetic and the JIT's arm64 instructions obey. The mapping goes through the C library's `fe*` constants rather than copying bits, because the two architectures number the directed modes the other way round:

MXCSR RC	Meaning	FPCR RMode for the same mode
`00`	to nearest, ties to even	`00`
`01`	toward −∞	`10`
`10`	toward +∞	`01`
`11`	toward zero	`11`

Before that change, `divsd`, `sqrtsd` and friends always rounded to nearest whatever a program asked for. A second bug was found while testing it on 2026-09-21: the main thread of a dynamically linked program never received the guest's floating-point control state at all, because it enters through the guest-call loop

and only the run loop applied it. `tests/dynamic/nan_contexts.c` now checks NaN results and rounding on the main thread, a pthread, inside and after a signal handler, on libdispatch's threads, after an MXCSR write, and across `fork`.

VEX: one implementation, two halves

A VEX.128 instruction is the legacy SSE operation with its first source taken from `vvvv` instead of the destination, and the upper half of the destination ymm zeroed. So the legacy implementations are reused, with the first source chosen per encoding, and the merge forms take their untouched part from `vvvv` rather than from the destination. A VEX.256 instruction is executed twice: once on the low 128-bit halves, and once with the involved registers swapped for their entries in `ymmh[]` and memory operands 16 bytes further on. Operations that cross the 128-bit lanes cannot be split that way and are written out in full: widening and narrowing conversions, broadcasts, 128-bit inserts and extracts, permutes, half-precision conversions, and the mask and test instructions.

The SSE4.2 string instructions

`pcmpestri`, `pcmpestrm`, `pcmpistri` and `pcmpistrm` compare a 16-byte "needle" against a 16-byte "haystack" block in one of four ways. AArchX implements them from the architectural immediate layout directly rather than as a list of named mnemonics:

imm8 bits	Field	Values
[1:0]	element format	unsigned or signed, bytes or words
[3:2]	aggregation	equal any, ranges, equal each, **equal ordered** (substring search)
[5:4]	polarity	positive, negative, masked positive, masked negative
[6]	output selection	least or most significant index (`i` forms); bit or byte mask (`m` forms)

The explicit-length forms take the lengths from EAX and EDX; the implicit forms find them at the first zero element. The JIT does not translate these instructions, so this C code is their only implementation.

HOW IT WAS FOUND: STEAM'S MISSING LOCALIZATION

Under AArchX, Steam logged in and then stopped on an assertion, `strtools.cpp (898) : pSrc != NULL`, after logging `ILocalize::AddFile() failed to load file resource/vgui_%language%.txt`. Under Rosetta the same installation loaded its localization. Steam builds that file name by finding `%language%` in `public\steambootstrapper_%language%.txt` with its own SSE4.2 `strstr`, and in the buffer it searched, the needle started five bytes before the end of a 16-byte block.

In equal-ordered aggregation the architecture compares the needle against the block only while the needle position is still inside the block, so a needle that starts near the end counts as a partial match there, and SSE4.2 `strstr` relies on exactly that to find a needle that straddles two blocks. AArchX kept comparing past the end of the block, treated the missing bytes as a mismatch, and cleared the bit; `pcmpistri` answered 16 (not found) where the hardware answers 11. The fix (2026-09-26) is one loop bound, `j + i < n`. The guest test `sse42_test` gained three cases whose expected values were recorded under Rosetta.

i386 execution

The interpreter never asks per instruction whether it is in 32-bit mode; the decoded instruction carries the mode, and the step function chooses the decode mode once from `cpu->mode32`. Mode changes happen only through far transfers, and every far transfer (`jmp far`, `call far`, `retf`, `iret`) funnels through one helper, `far_transfer`, so the mode, the CS selector and EIP/RIP can never disagree. Which mode a selector means is decided from its descriptor in the local descriptor table that Wine installs through the `i386_set_ldt` call: a descriptor with the long-mode bit selects 64-bit code, one with the default-size bit 32-bit code.

The details that bite are the ones the SDM states and that are easy to miss:

- **A selector slot holds 16 bits** and hardware ignores the rest. Wine's `I386_CONTEXT` stores `SegCs` and `SegSs` as doublewords that `RtlCaptureContext` fills with a 16-bit store, so their upper halves are stack garbage by the time the WoW64 layer pushes them for an `iretq`. Taking the full 32 bits picked a garbage selector.
- **A zero CS or SS on the stack** is a frame AArchX built itself, not a mode decision, so the current selector is kept.
- **FS and GS** are the two segments whose base applies to addresses, so loading one moves its base; ES, SS and DS are flat and carry no base.
- **BCD**: `das`'s second test has no else branch, so a CF raised by the first adjustment survives when the second does not fire, and the adjust instructions leave every flag the SDM calls undefined untouched.

The i386 path is checked by its own differential gate: 40,044 instruction sequences (44 hand-written, 20,000 random integer ones and, since 2026-10-07, 20,000 random x87 ones; the count was 20,033 before) run under both engines, comparing all sixteen registers at full width, the flags, the instruction pointer, the mode, the selectors, the whole floating-point and vector register file and every byte of memory touched (Chapter 16).

The same i386 interpreter runs 32-bit Mac programs, which arrived on 2026-10-10 (0ca24d64, Chapter 13). Nothing in `far_transfer` or the segment rules changed for them: they never leave 32-bit mode, they use flat segments, and they enter the kernel through `int 0x80` or `sysenter` instead of through Wine's far transfers, which is the routing described under "Executing one instruction".

What the JIT leaves to the interpreter

The interpreter's share of a real program's time is small, but its list of responsibilities is long, and it changes as the JIT grows. As of 2026-10-04, with the x87 and string rows revised on 2026-10-18 and the rest checked against `docs/compatibility.md`:

Executed by the interpreter	Why
The x87 forms the JIT does not translate: the transcendentals, BCD, 80-bit memory and environment forms, and any operation that meets a NaN, gives an infinity or a tiny product or quotient, or rounds other than to nearest	until 2026-10-07 this was every x87 instruction; the common forms are translated now (6c6679f0, 9f0b7087 for 32-bit blocks, Chapter 7), and what they cannot reproduce they hand to this unit
`pcmpXstrY`	complex, data-dependent control; rarely hot

Executed by the interpreter	Why
String instructions (`movs` , `stos` , `lods` , `scas` , `cmps`), with or without `rep`	a whole `rep` loop runs in one call, and a forward `rep movs` or `rep stos` over one piece of host memory is one `memmove` or fill; the hot string routines are replaced by native ones anyway (Chapter 7)
`cpuid` , `rdtsc` , `rdtscp` , `xgetbv`	answered by emulator state
`syscall` and `int`	enter the system-call layer (Chapter 9)
Far transfers	change the mode; must stay in one helper
Misaligned locked atomics	arm64 atomics cannot do them
MMX leftovers: `pmaddwd` , `pmovmskb` , `emms` , the SSSE3 MMX forms, MMX/SSE conversions, shifts by a register	most MMX has been translated since 2026-10-04
VEX leftovers: 256-bit byte and integer shuffles, unpacks, permutes and lane inserts, `vptest` , immediate blends, mask-producing compares, gathers, `bextr` , `pdep` , `pext`	not yet translated
16-bit addressing; i386 instructions outside the JIT's 32-bit whitelist	rare

A slow call costs far more than the instruction it executes: the JIT has to write back the guest registers it keeps in host registers, call into C, and reload them. That is why the list above shrinks whenever a profile shows one of its entries in a hot loop. Most of the 2026-09-27 and 2026-10-04 additions to the JIT came from exactly such profiles, and so did the October x87 translation (a 32-bit game's float arithmetic sits on the x87 stack, so every x87 instruction was a spill, a call and a reload), taken with `OCERZ_GUESTPROF` , which names the interpreted instruction forms a thread spends its time in (Chapter 18).

> **A NOTE ON THE SOURCE COMMENTS**
>
> Several explanation blocks in these files predate the JIT's growth. `interp_sse.c` and `cpu.h` still say the JIT translates no VEX instruction, and `interp_ext.c` that it does not translate CRC-32C. Both stopped being true in September 2026 (VEX on 2026-09-13; `crc32` , AES-NI and PCLMULQDQ on 2026-09-27). The code, not those sentences, is what this chapter describes.

CHAPTER 6

The JIT: translation, registers and control flow

The interpreter defines what an x86 instruction means; the JIT is what makes AArchX fast enough to run Steam and games. This chapter covers how guest code becomes arm64: the code arena, the dispatcher, how a block is built, the register layout every block shares, how flags are avoided rather than computed, how blocks are stitched together by chains, return stacks and superblocks, how a thread spinning in translated code is stopped for a signal, and how 32-bit code is translated. Floating point, SIMD, memory ordering, code that changes under the translator and the translation cache are in Chapter 7.

26,593	1 GB	1,048,576	256
LINES IN SRC/JIT.C	CODE ARENA (MAP_JIT)	HASH BUCKETS	INSTRUCTIONS PER BLOCK, AT MOST

Where the design came from

The first JIT, in June 2026, was call-threaded. It decoded a block once, emitted a native function for it with `vm` in x19 and `cpu` in x20, inlined only the no-op family and register moves against a memory-backed register file at `[x20 + reg*8]`, and called the interpreter through `ocerz_jit_exec_one` for every other instruction. It ran about 2.9 times faster than the interpreter, with one 64 MB arena, a 65,536-bucket hash and no invalidation at all.

The real translator arrived in two waves. Between 16 and 23 July the JIT gained lazy flags, call and return inlining, chaining infrastructure, a return-address predictor and a first register allocator; the commit of 17 July recorded the `alu` kernel going from 4.46 to 2.42 times Rosetta's time, and the one of 23 July claimed rough parity. Then, on 15 August, a single day of more than twenty commits rebuilt it around the ideas this chapter describes: an in-arena dispatch stub, inline indirect calls with per-site caches, inline predicate evaluation from deferred flags, compare-and-branch fusion, xmm registers pinned in v16-v31, and full pinning of all sixteen general registers in fixed host registers ("pin class 3"). Superblocks followed on 16 and 17 August, the host-stack return predictor on 17 August, guest `rsp` as a host pointer on 25 August, trace inversion on 3 September. On an Apple M5 in September, the libSystem-linked benchmark build beat Rosetta on 14 of its 15 kernels in cache mode (Chapter 17).

The fortnight from `90fe4d8e` (4 October) to `6e6c64b5` (18 October), releases 0.4 (`de872c96`) to 0.8 (`a454f2fd`), changed the translator's inputs more than its structure. 108 commits touched `src/jit.c`, nine of them by Jfishin, and the file grew from 20,555 lines to 26,593. Four workloads drove most of them: 7-Zip's LZMA decoder under Wine, which reads one compare's flags up to eight instructions later; the Wine memory layout, measured with xbench linked into it and with winbench under Wine, which gave `rsp`'s register, x0 and x29 new roles; 32-bit code under WoW64 and m32, which got most of the 64-bit

machinery on 7 October, the day of 0.6 (`7fc00672`); and translation time itself, cut by about a quarter on 13 October. Beside them came a fix no benchmark shows: a signal sent to a thread spinning in translated code was not delivered until the loop ended (`a7fd0f3f`, 14 October).

The interpreter never went away. Anything the JIT declines to encode runs through it one instruction at a time, from inside a translated block, and every guest test runs both ways and must agree byte for byte (Chapter 16). That pairing is what made the aggressive optimizations below safe to attempt: each one can be switched off by an `OCERZ_NO_*` variable, and a miscompile can be cornered by bisecting addresses and switches against the interpreter (Chapter 18).

The code arena

`ocerz_jit_create` reserves one region with `mmap(PROT_READ|PROT_WRITE|PROT_EXEC, MAP_PRIVATE|MAP_ANON|MAP_JIT)`, 1 GB by default (`OCERZ_JIT_CODE_MB` or `OCERZ_JIT_CODE_KB` override it; the test suite runs some programs in a 64 KB arena on purpose, so that almost every block takes the slow path). If `MAP_JIT` fails, the process simply runs interpreted. On Apple silicon a `MAP_JIT` region is writable or executable per thread, never both, so every write to the arena is bracketed by `pthread_jit_write_protect_np(0)` and `(1)`, and every written range is pushed through `sys_icache_invalidate` before it can run. That applies to the one-word patches as much as to whole blocks. Until 13 October a chain activation wrote four bytes and invalidated four bytes; since then the words one translation's chain install patches are made visible together, with one pair of barriers (see "Installing chains").

The arena fills front to back and is never compacted in place:

- **The head** holds a copy of `src/leaf.s`, the hand-written arm64 string and memory routines (Chapter 7), so that a translated block can reach them with a single direct branch.
- **Two dispatch stubs**, one for 64-bit and one for 32-bit code, are emitted the first time a block of that mode is translated.
- **Veneer pools**, 256 KB each, are carved out every 64 MB of code (up to sixteen of them) for chains that cannot reach their target with a plain branch.
- **Translated blocks** are appended at `code_cur`. A block that is retired stays where it is, on a retired list, so a thread still running inside it runs to its next exit; its bytes are only reclaimed when the whole arena is flushed, which since 4 October happens when less than an eighth of it, at most 8 MB, is left (Chapter 7).

Everything that describes a block lives outside the arena, in ordinary `malloc` memory. A `JitBlock` holds the block's key, three entry points into its code (`code`, `body_code` and `body_noreload`, explained below), its outgoing edges (at most eight), the list of blocks that chain into it, its index in the array of live blocks, per-instruction lane and fault-recovery records, and a compact description of its instructions. Since 11 October it also keeps the least and the end of the greatest guest address its instructions cover, spliced callees and far superblock legs included, which is what a write to guest code is checked against (Chapter 7). Until 6 September a block kept the full 96-byte decoded `X86Insn` of every instruction; a GUI Wine process carried about 870 bytes of that per block across 215,000 blocks, 180 MB in all. A published block now keeps 16 bytes per instruction (address, length, operation) and full copies only of the instructions it still sends to the interpreter or inspects during fault recovery.

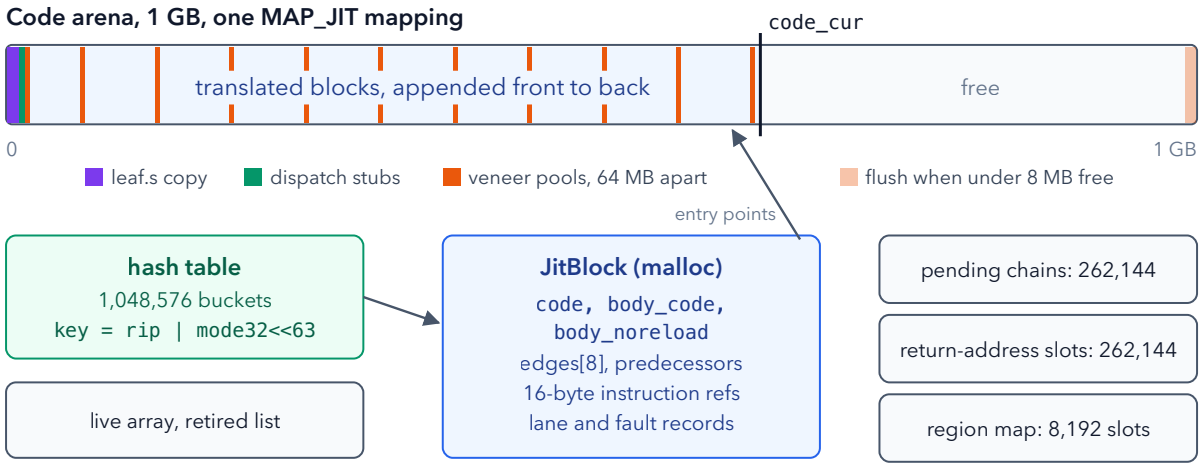


Figure 6.1. The arena holds only machine code; everything that describes a block lives in malloc memory beside it. The tables at the bottom are the ones a lookup, a chain activation, a return continuation and an invalidation consult.

Finding a block

Blocks are keyed by `jit_key(rip, mode32)`, the guest address with bit 63 set for 32-bit code. The same address in 32-bit and 64-bit mode is different code, and under Wine's WoW64 both run in one process, so the two may never share a cache entry, an invalidation mark or a commpage mark. The key is hashed with a 64-bit finalizer (shift right by 33 and XOR, multiply by `0xff51afd7ed558ccd`, shift right by 29 and XOR) into 2^{20} buckets, each a singly linked chain of `JitBlock`s.

Lookups take no lock: a bucket head is read with an acquire load, and a block is published into its bucket with a release store only after its code and metadata are complete, so a reader on another thread sees either nothing or a finished block. Translation does take a lock, the JIT lock, and `ocerz_jit_step` looks the key up a second time after acquiring it, because another thread may have translated the same block meanwhile. Since 12 September every acquisition of that lock is a `trylock` loop with yields and short sleeps that never parks the thread in the kernel: a thread parked on it could lose its wakeup and wedge translation for the whole process, which is what some Steam launches did. `OCERZ_JITLOCKLOG=1` reports any wait over three seconds with the holder's thread, the site that took the lock, the holder's guest `rip` and its kernel thread state.

The run loop and the dispatcher

Every guest thread runs a loop in `src/vm.c`. If the thread owes one interpreted instruction (`cpu->interp_once`, set by fault recovery and a few other paths), it interprets one. Otherwise it calls `ocerz_jit_step`, and if that answers `OCERZ_EUNSUP` it interprets one instruction with `ocerz_interp_step` and goes round again. That fallback is how the JIT declines things: a block that cannot be translated, an address that is kept in the interpreter on purpose, an arena flush in progress. Since 14 October the loop also delivers, before each step, any pending signal the thread does not block, because between two steps the whole guest state is in the cpu (see "Signals in translated loops").

`ocerz_jit_step` itself does little. It refuses addresses inside the dyld API trap window (Chapter 11), credits a superblock side exit that came back to it for profiling, checks whether an arena flush has been requested, looks the block up, translates it under the lock if it is missing, and then calls the block as a C function, `b->code(vm, cpu)`. Around that call it bumps a per-thread count of translated frames, which is how the arena flusher knows when no thread is inside translated code (Chapter 7). A block that was translated with no code, because the translator gave up on it, runs through `jit_interp_block` instead.

The value a block returns is a step code: `OCERZ_STEP_OK` (zero) means "the guest `rip` in the cpu is where to continue", `OCERZ_STEP_PROFILE` means a probed side exit was taken, and the others mean exit or a fatal error. The common case never reaches C at all. A block's shared exit tests the code it is about to return; if it is zero, instead of returning it branches to the **dispatch stub**, a small arm64 routine in the arena (since 15 August) that does what `ocerz_jit_step` would have done for a hit. It loads the cpu's interrupt word, its termination flag and the VM's exit flag and returns to C if any is set; since 14 October it also returns while the cpu's pending signals, less the ones it blocks, are not zero; it returns to C if the new `rip` is in the dyld API window (in an m32 process a 32-bit trap address is looked up instead, since a trap is translated as the import stub it stands for, Chapter 13); otherwise it hashes the `rip` exactly as `hash_key` does, walks at most six entries of the bucket, and branches straight to the matching block's full entry. Only a miss, or a block without code, goes back to the C loop. The branch from a block's exit to the stub is range-checked: until 18 August it was assumed to be in range, and after 128 MB of appended code it silently encoded a different instruction and the process died of `SIGILL`.

Building a block

`translate` does the same steps for every block. It first declines outright in a few cases: the address is in a range that has churned too often (Chapter 7), or a developer has asked for it to be interpreted with `OCERZ_INTERP_LO/HI` or `OCERZ_INTERP_RIP`, or it is the throw site an exception trap is watching. If the translation cache is on and holds a record for this key, the record is loaded instead (Chapter 7).

Decoding

The translator decodes up to 256 instructions with `jit_decode`, under a `sigsetjmp` guard so that a block running into an unmapped page simply ends there instead of crashing the translator. A block normally ends at its first terminator: any jump, conditional jump, `loop` or `jrcxz`, call, return, far transfer, `syscall`, `int`, `int3`, `ud2` or `hlt`. Three exceptions make blocks longer than the x86 basic block:

- A **direct call** to a short straight-line function may be spliced in place, its body decoded into the caller's block (see "Calls and returns").
- A **conditional branch** may be passed over, making the block a superblock: decoding continues on the fall-through path and the taken side becomes an out-of-line exit stub, up to six per block.
- A conditional branch that profiling has marked for **inversion** is rewritten while decoding: its condition is complemented and its target swapped with the fall-through, so the hot path continues inline.

A block that reaches 256 instructions without a terminator stops there. Until 9 October such a block materialized the flags, a C call-out that spills every register, stored `rip` and returned to the dispatcher; unrolled code such as SHA-256's compression rounds paid that every 256 instructions. Since `61b26d66` it ends as if

a `jmp` to the next instruction stood there: a chained edge, with the flags left lazy for the next block. Four rounds of SHA-256 over 64 MB in cache mode went from 1.08 to 0.87 seconds (Rosetta 0.74; `OCERZ_NO_CAP_CHAIN=1` restores the old ending).

Planning

Before any code is emitted, the translator decides the block's shape. It chooses the pin class (class 3, full pinning, for 64-bit code), which guest base registers are used often enough to hoist into dedicated host registers, which xmm registers are pinned (all sixteen, in practice), whether the block needs the guarded memory forms because it has faulted on the commpage or on a misaligned ordered access before, whether it should use the fixed-lane mode reserved for self-looping blocks, which spliced call regions may keep `rsp`'s register still (see "Stack traffic"), which instruction hands its NZCV to each consumer (worked out once, into a per-block table, since 13 October), and where the flags each instruction defines are live. It also makes sure the dispatch stub exists and that a veneer pool has been carved if the arena has crossed another 64 MB mark.

Emitting and publishing

The body is emitted one guest instruction at a time. Each emitter either produces inline arm64 or emits a *slow call*: spill the pinned registers the C call would clobber, call `ocerz_jit_exec_one` with the decoded instruction, reload. After the body come the exit stubs and the shared exit; then the code is made executable, the instruction cache is invalidated, the block goes into the hash table, and any chains that were waiting for this address are activated (see "Chains").

What translation costs

Translation is paid again whenever nothing is cached: on a first start, for code the translation store does not keep (Chapter 7), and on every start with the store off. On 13 October the translator was profiled on such starts on an M2 Max (the system's perl loading ten modules and its ruby loading the standard library, cache mode, `OCERZ_TCACHE=off`, with `OCERZ_JITMEASURE=1` giving the translation time of the first 32,768 blocks). Three costs stood out, and each was removed that day without changing what any block translates to:

- **The flag look-ahead** was 20% of translation time, three quarters of it decoding: each translated block caused 57.6 decodes. It now stops reading a successor once that successor has written all six flags (`e34ad68f`, see "Liveness").
- **Finding NZCV producers.** `nzcv_fuse_producer`, which finds the instruction a consumer may take NZCV from, was asked up to nine times per instruction by the emission loop alone, each time scanning back through its window, and came to about a tenth of translation time. Its answer depends only on the block's instructions and on state fixed before the liveness pass, so `nzcv_prod_fill` now works it out once per instruction into a table that every later question reads (`6d7e051a`). `OCERZ_NZCV_PROD_CHECK=1` asks again at every read and aborts on a difference.
- **Instruction-cache maintenance** was 10.5%, half of it chain installs flushing one patched word at a time (`f4ef8900`, see "Installing chains").

Change, 13 October	perl before	after	ruby before	after
look-ahead stops early (`e34ad68f`), against `OCERZ_NO_XLIVE_KILL=1`, best of three	275 ms	**237 ms**	207 ms	**181 ms**

Change, 13 October	perl before	after	ruby before	after
producer table and batched chain install (`6d7e051a`, `f4ef8900`), against the tree before both	232 ms	**208 ms**	185 ms	**168 ms**
batched chain install alone, against `OCERZ_NO_CHAIN_SYNC_BATCH=1`, best of seven	215 ms	**208 ms**	171 ms	**168 ms**

Each row is that commit's own paired measurement; over the day the perl start's translation went from about 275 ms to 208 ms. The decoder was made cheaper the same day too (`39271608`, Chapter 4), and two switches that had been looked up with `getenv` on every use, one of them on every look-ahead into a block ending in `ret`, are now read once like every other (`669bf3df`).

Register pinning

Full pinning is the single decision that shapes everything else. Every guest general register lives in a fixed host register for the whole body of every block, and the slot equals the guest register number:

Guest	Host	Saved across C calls?
`rax rcx rdx rbx rsp rbp rsi rdi` (0-7)	`x21` - `x28`	yes, callee-saved
`r8` - `r13` (8-13)	`x3` - `x8`	no: spilled and reloaded around every call-out
`r14`, `r15`	`x1`, `x2`	no: spilled and reloaded around every call-out
`xmm0` - `xmm15`	`v16` - `v31`	no: spilled by contract (Chapter 7)

Because every block uses the same layout, a transition between two blocks costs nothing. A chained branch can jump from the body of one block straight into the body of the next with every guest register already where the next block expects it; a return-address stack entry can be a plain pointer into a body; no block boundary pays for frame setup. The prologue and epilogue are paid only when C enters translated code and when translated code goes back to C.

The rest of the register file is assigned too:

- **x19 and x20** hold the `vm` and `cpu` pointers for the whole body. Every access to architectural state that is not pinned (the flag record, `rip`, the interrupt word, the software return stack) is an offset from x20.
- **x0** holds the guest base, called `JGB` in the source (since 15 August). For a dynamically linked program, whose guest addresses are host addresses, it is zero; for a static program it is the arena's offset.
- **x9-x15** are the translator's temporaries, named `JT0`, `JT1`, `JT2`, `JTF`, `JTT`, `JTU` and `JTA`. x15, the computed-address temporary, is never live across a block edge, which is why veneers may use it.
- **x16, x17, x29 and x30** hold hoisted memory bases inside a body (`JMEMBASE2`, `JMEMBASE`, `JMEMAUX`, `JMEMBASE3`). x30 is free there because a guest `ret` takes its host continuation from the shadow stack and the epilogue restores the real link register from the frame. When no base is hoisted, x16, x17 and x30 also serve as rename registers for spliced pushes and pops.
- **x18** is never touched: it is the platform register on Apple's ABI.

- **v0-v3** are vector temporaries and **v4-v15** are twelve lane registers that hold scalar floating-point results and the upper halves of `ymm` registers inside loops. Their callee-saved lower halves, d8-d15, are saved by the prologue.

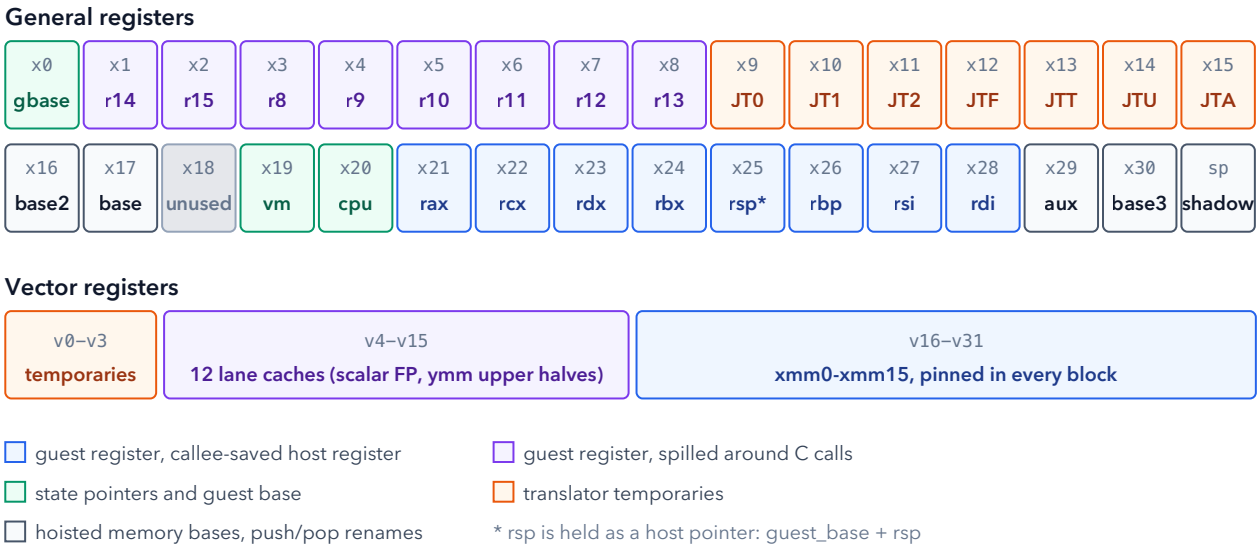


Figure 6.2. Pin class 3. Every block agrees on this layout, so control passes from body to body without saving or restoring anything. In the Wine layout x0 holds the stack delta and x29 holds 12 GB instead (see below); 32-bit blocks use the same layout and leave x1-x8 to registers i386 code cannot name.

Registers in the Wine layout

Wine's processes keep their first 12 GB of guest memory at a different host address (Chapter 8), so there is no single guest base to keep in x0, and a guest address below 12 GB needs `low_base` or'ed in. Three registers took on roles of their own there in October; Chapter 7, "Addressing in Wine's memory layout", covers the guards and hoists that use them.

- **x0 holds the stack delta** (since 7 October, `e7a1cd62`). A thread's stack lies wholly on one side of 12 GB, so while `rsp` is below it every stack slot is `rsp + low_base`, and otherwise `rsp` itself. x0 keeps that delta, computed without touching the flags, which may be live: `(rsp >> 32) - 3` is negative exactly below 12 GB, and its sign spread over the word masks `low_base`. It is recomputed when a body is entered from the dispatcher, after every call-out, before a chain, and after any write of `rsp` other than a push, pop, call, return or (since 8 October) a step of under 64 KB, which no valid stack can carry across 12 GB. In 32-bit blocks x0 holds `low_base` itself, since a 32-bit stack slot is always below 4 GB (see "32-bit blocks").
- **rsp's register is a host pointer here too** (since 10 October, `ab86121d`): it holds `rsp` plus the delta, so pushes, pops, calls, returns and `rsp`-relative operands are single pre- and post-indexed accesses, as in the other layouts. The cpu keeps a copy of the delta, `low_stack_delta`, for where x0 has been reused. A step by a constant moves the pointer in place; an `add` or `sub` of a register may cross to the other stack and is treated as a write of a new value.
- **x29 holds 12 GB** (since 10 October, `5b02be18`). Twelve gigabytes is not a power of two, so no single test says which side of it an address is on, and until then no register was free to hold the limit. Every block's prologue sets x29, nothing translated writes it, and host code called out to preserves it because it is callee-saved, so a guard is one compare against x29; `JMEMAUX`, which shares x29, is not used in

this layout. An object-update loop with its image at 8 GB and its stack above 12 GB went from 0.369 to 0.334 seconds (Rosetta 0.278).

`OCERZ_LOWSTACK_CHECK=1` traps wherever x0, the cpu's copy of the delta and the pointer disagree, and `OCERZ_WLIM_CHECK=1` before any guard whose x29 is not 12 GB.

Guest rsp as a host pointer

Since 25 August the pinned `rsp` does not hold the guest value at all; it holds `guest_base + rsp`, a host pointer. A guest `push` then becomes one pre-indexed store, `str x10, [x25, #-8]!`, and a `pop` one post-indexed load, with no base addition and no separate update of the stack pointer. An `rsp`-relative memory operand needs no base either. The price is paid only where the value of `rsp` is read or written as data (a `mov rax, rsp`, an `lea` from `rsp`), where the accessor subtracts or adds the base, and at the boundaries: spilling to the cpu, filling from it, and rebuilding guest state after a fault. `OCERZ_RSP_VALUE` restores the older form. When the guest base is zero, as in the identity map, those conversions are a plain move since 10 October (`da605c52`). A read-modify-write on an `rsp`-relative operand was refused from the day `rsp` became a pointer until 4 October (`635c2081`); MSVC-built code addresses its locals off `rsp` and uses `lock or [rsp], 0` as its memory barrier, so those were the hottest interpreted forms on R.E.P.O.'s main thread.

Stack traffic: pairs, runs and lazy rsp

A pre-indexed store writes its base register back, and the next one cannot form its address until that write has happened. A run of pushes is therefore a serial chain through `rsp`'s register, however independent the values being pushed: `fib`'s three pushes, three pops, call and return made a chain of eight dependent updates on every call. October removed that chain in three steps (Figure 6.3).

- **Pairs** (8 October, `b2a1db64`, in the Wine layout): two pushes or two pops in a row share one address, one `stp` or `ldp` and one update of `rsp`.
- **Runs** (9 October, `17bfc689`, both layouts): `emit_stack_run` takes up to sixteen pushes, or pops, of 64-bit registers. Every access is at a fixed offset from `rsp` as it was before the run, and `rsp` moves once, after them, so a fault restarts the run at its first instruction with nothing changed but registers a pop will load again. A pop run stops before a register it already loads, since an `ldp` with two equal destinations is unpredictable, and a push run stays within sixteen so that its stores lie inside the 128-byte red zone until `rsp` covers them. Since `5a3dbc8c` a push run may also take the `mov rbp, rsp` of a frame prologue. `fib(40)` went from 0.320 to 0.222 seconds (Rosetta 0.187).
- **Lazy rsp** (11 October, `256faca8`): inside a call spliced into its caller (see "Splicing small callees") whose stack traffic is its own, `rsp`'s register does not move at all.

Lazy rsp applies where `lazy_rsp_scan` finds a spliced region whose call had its return-address push elided, whose nested calls are spliced with theirs elided too, and in which every instruction that names `rsp` is a push or pop of a 64-bit register or an immediate or a `mov` of `rsp` into a 64-bit register. There `rsp`'s register keeps the value it had before the call, and each push, pop, call and return moves an offset the translator keeps, `g_rsp_pend`, instead: a push is a store at the offset, a pop a load from it, a `mov` of `rsp` the register plus the offset. The deepest offset must fit an unscaled access, and the region ends at its return with the offset back at zero. The guest's `rsp` inside the region is the register plus the offset, which the recovery paths work out from the region's instructions, as they already did for elided return addresses: `ocerz_jit_fault_recover_regs` for a fault, and `ocerz_jit_guest_gprs_at` for a suspended thread whose stack a garbage collector is about to scan. An instruction interpreted inside the region moves the register

to the true `rsp` for the call-out and back, and no out-of-line slow arm is placed inside a region. xbench's `leafcall`, whose iteration had been a chain of thirteen dependent stack updates, went from 1.398 to 0.730 billion cycles for 100 million iterations (Rosetta 1.316), and from 1.435 to 0.743 in the Wine layout. `OCERZ_NO_LAZY_RSP=1`, `OCERZ_NO_STACK_RUN=1` and `OCERZ_NO_STACK_PAIR=1` undo the three steps.

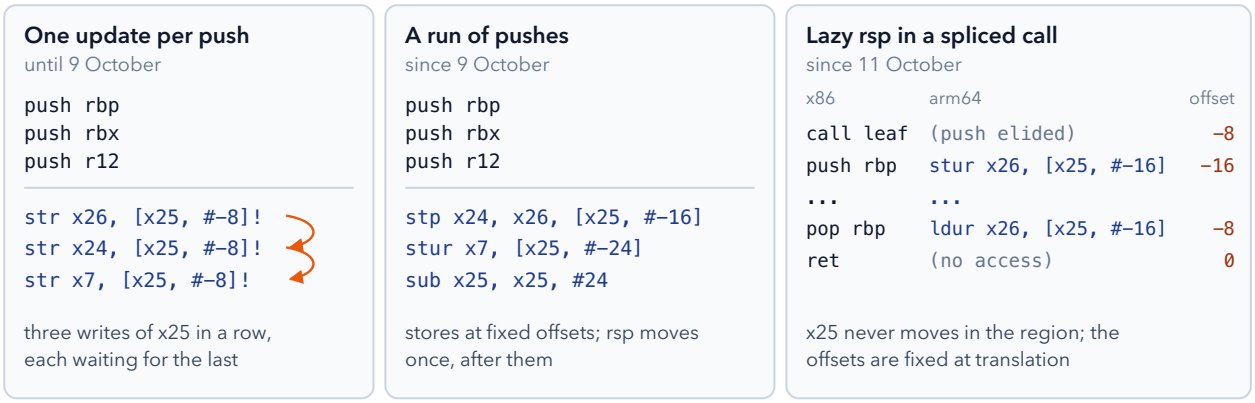


Figure 6.3. Three ways to push registers, in pin class 3 (rsp's register is x25). A pre-indexed store per push makes a serial chain through x25; a run writes at offsets from where rsp started and moves it once; inside a spliced call whose stack traffic is its own, rsp's register stays put and the translator carries the offset.

What happens around a call into C

A slow call, a helper call or a bridged call has to respect the C calling convention, and the C callee is free to destroy x0-x18 and the vector registers. The translator therefore spills exactly what is not callee-saved: the six pinned registers in x3-x8 and the two in x1-x2 go to the cpu struct and come back afterwards, x0 is reloaded with the guest base, any hoisted memory bases are recomputed, and the xmm pins are written back and reloaded. The eight registers in x21-x28 survive without being touched, which is why the most used guest registers were put there. Every call-out also increments a sequence number at translation time; the push/pop rename logic below uses it to know whether a call-out happened between a push and its pop.

Anatomy of a block

Each translated block is one C-callable function with three entry points, laid out as in Figure 6.4.

- **The full entry** (`JitBlock.code`) is where C, or the dispatch stub, enters. It saves x29/x30 and x19/x20, moves the `vm` and `cpu` arguments into x19 and x20, loads the guest base into x0, saves d8-d15 and the callee-saved x21-x28 it is about to overwrite, loads all sixteen pinned guest registers from the cpu, converts `rsp` into a host pointer, records the current host stack pointer in `cpu->jit_fp` as the frame base and pushes a sentinel pair `{0, 0}` onto the host stack, and loads the sixteen xmm registers into v16-v31. In the Wine layout it also sets x29 to 12 GB and computes the stack delta in x0.
- **The body entry** (`body_code`) is where chained blocks enter. It reloads the hoisted memory bases and nothing else.
- **The no-reload entry** (`body_noreload`) skips even that. A chained predecessor whose hoisted bases have the same *hoist signature* (which guest registers, which index and scale, which displacement) branches here, because its host registers already hold exactly what this block would load.

A self-looping block has its loop head aligned to 32 bytes (`OCERZ_LOOP_ALIGN`) by sliding the body forward and filling the gap with `nop`s, and it does not poll for interrupts on every iteration: as described under "Stop sites", a patched branch serves instead.

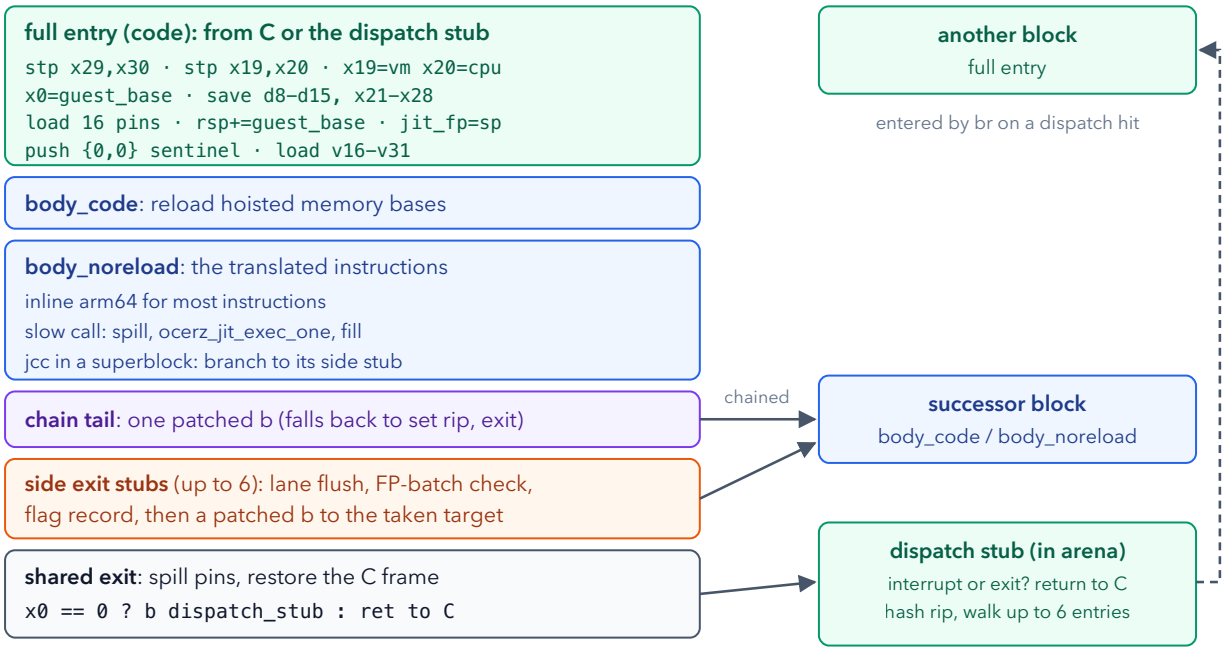


Figure 6.4. One block's code. Chained edges go body to body and never pass the prologue or the epilogue; only unchained exits reach the dispatch stub, and only misses there reach C.

Addressing guest memory

Most of what a block does is load and store, so the shape of an address computation matters more than almost anything else. How a guest address becomes a host address depends on the memory map the process uses (Chapter 8):

- **Identity map.** A dynamically linked program, the usual case, runs with a guest base of zero: every pinned register already holds a host address, and `mov eax, [rdi+16]` is a single `ldr w21, [x28, #16]`. With a zero base, the pinned register itself serves as the hoisted base.
- **Static arena.** A statically linked program has a non-zero guest base in x0, and an access becomes an addition and a register-offset load, or uses a hoisted base: a guest register used as a base two or more times in a block is held already offset in x17, x16 or x30, and `base + index<<scale` can be kept in x29 and recomputed at the loop head.
- **The low shadow.** Wine's processes keep their first 12 GB of guest memory at a different host address, so every access that is not provably to the stack needs its address translated by range; Chapter 7 covers how that went from about twenty instructions to one `orr` on 4 October, and how since 8 October a block settles the question once, where it starts, for the base registers its operands use most.

The commpage guard

One page range breaks the identity map even in the best case: the guest's commpage at `[0x7fffffe00000, 0x7fffffe04000)`, which the host will not let a process map at that address, so AArchX keeps its contents elsewhere (Chapter 8). Guarding every load against it would be ruinous, so guarding is adaptive. Blocks

are translated with plain accesses; one that touches the commpage faults on an unmappable address, the fault handler sees the address inside the commpage range, serves the access from AArchX's copy, and marks the block, which is retranslated with a guard in front of each access that might reach the range. Since 23 September a first commpage or alignment fault retires only the faulting block, through its list of predecessors, instead of scanning every live block: Electron has about 140,000 blocks live at startup, and one scan per fault had made retiring blocks the largest single cost on Discord's main thread.

Constant addresses

For a rip-relative or absolute operand, the address is a constant of the translation, so the questions a guard asks at run time can be answered when the block is built (since 22 September). If the constant is not in the commpage, the guard is skipped; if it is, it becomes a plain redirect. In ordered memory mode, where an acquire load or release store faults if it crosses a 16-byte granule, an aligned constant goes straight to the ordered access. Measured over Adobe AIR, a rip-relative load had been 18 host instructions and a frame-relative one 17; a global load or store now takes four in either mode. A global access is the most common single form in compiled code, and Brawlhalla ran 13% more blocks per second at the same CPU. The "known address" fact is set by the guard and consumed by the very next ordered access, and building any other effective address clears it, so it can never describe the wrong address.

Flags without computing flags

Nearly every x86 arithmetic and logic instruction writes six status flags; almost none of those values are ever read. AArchX's interpreter computes them eagerly, because it is the reference (Chapter 5). The JIT computes them only when something reads them, and most of the time not even then.

The deferred record

An instruction that defines flags records what would be needed to compute them: a kind (add, subtract, logic, increment, decrement, the three shifts, the two multiplies), an operand size and a carry-in, packed into `cc_op` as `kind | size<<8 | cin<<16`, plus the result and source operand in `cc_dst` and `cc_src`. On the hot path the record is usually not even written to memory: the values sit in the registers that produced them, and the record is stored into the cpu struct only on an exit, a side stub or a call-out that leaves the block while the flags are still live. A consumer inside the block, a `jcc`, `setcc`, `cmovcc` or `adc`, evaluates its predicate inline from the record's operands. Anything else that needs the full RFLAGS value (a `pushf`, a signal frame, the interpreter) calls `ocerz_flags_materialize`, which computes them from the record exactly as the interpreter would have.

The inline evaluation used to know only subtraction and logic records, and every other kind called out. That mattered most after `inc` and `dec`, which leave CF alone: to keep the old CF in their record they rebuilt it whenever any of their flags was live, which for a record of another kind was a call-out, and a branch on their result had no inline path either. `if (--o->refcnt == 0) free(o)`, the shape behind `CFRelease`, `objc_release` and `shared_ptr`, paid both on every release. Since 9 October (`edf0b16f`) the old CF is rebuilt only when CF itself is live afterwards, and a condition after `inc` or `dec` is decided inline by turning the result back into the operand and redoing the add or subtract of one, which gives ZF, SF and OF; since 10 October CF is read straight from the record's source field, where an `inc` or `dec` record keeps the CF from before it (`3262e147`). A release loop of 100 million calls went from 1.69 to 0.21 seconds (Rosetta 0.13).

Liveness

Whether a flag is live is decided by a static pass over the block using the def/use table in `src/flags_live.c`, and the emitters consult the same predicates, so the pass and the code generator cannot disagree. The table errs in one direction only: an instruction may be described as defining less than it does and using more, never the reverse. Flags that x86 leaves undefined after `div` are treated as defined, because the interpreter produces specific values and the differential gate compares them. A variable shift by `cl` only *may* define flags, since a count that masks to zero leaves them alone. Any instruction that touches memory is a fault barrier: a guest fault handler must see the flags the hardware would have left, so they are live across it. To avoid paying for that on the hot path, inlined memory accesses carry a cold *fault recipe* that rebuilds the flags from the producer's operands if the access faults.

Liveness also reaches past the block (since 18 July): the flags at an exit are dead if the successor, decoded up to three blocks deep, overwrites them before reading them. That lookahead turned out to be about a fifth of all translation time during Steam's startup, because every block branching to an untranslated successor decoded it again; since 27 September its answers are memoized by address, depth and mode in 8,192 slots. An entry is trusted only while no translation has been retired since it was made and while the successor's first sixteen bytes are unchanged, which also catches code being rewritten before it was ever translated. Translation per block went from 5.78 to 4.85 microseconds over a Steam startup.

In October the pass learned five things, each because a hot loop paid for a flag nothing read:

- **Rotates by an immediate write CF and OF only** (9 October, `edf0b16f`). `flags_live.c` had described `rol` and `ror` as using and defining every flag, which is true only of a count in `cl`. Apple's x86 SHA-256, run from the shared cache, adds and then rotates three times before an `xor` that overwrites everything, so each rotate "used" the add's flags and the first one materialized them in C. `CC_SHA256` over 64 MB went from 1.34 to 0.44 seconds (Rosetta 0.33).
- **The return rule reaches across blocks** (9 October, `5deb74cf`). A block that ends in `ret` without writing flags, an epilogue of a store, some pops and the `ret`, cannot know whether the flags reaching it come from a compare that returns an answer. Its entry liveness is now a mark, `XLIVE_RETDEP`, which the block jumping to it settles by the same rule from its own last flag writer. In libmalloc's free path a `shld`'s CF had stayed live across an `inc` to such an epilogue, which sent the `shld` to the interpreter 635,000 times in a CoreFoundation dictionary build.
- **A jump through a rip-relative slot is followed** (10 October, `765891d8`). A call into another image lands on a stub, `jmp [rip + slot]`. A block ending in a call takes its live-out flags from the callee's entry, and the stub's indirect jump made every flag live there, so the caller kept them all and a `dec` after the return rebuilt the old CF each time round. The look-ahead now reads what the slot holds at translation time (`slot_value_now`, through `mach_vm_read_overwrite`, so an unreadable slot is an answer rather than a fault; not in native mode, whose slots may hold the host's own functions) and continues there. A loop calling a one-line function in a dylib 300 million times went from 1.94 to 0.41 seconds (Rosetta 0.30). The jump itself now chains through the same knowledge (see "Indirect jumps and calls").
- **A loop's own start is a least fixed point** (11 October, `5fc69e49`). A block whose `jcc` or `jmp` goes back to its own start used to follow that edge like any other, three deep, and came back with every flag live; an `inc` or `dec` closing the loop, which leaves CF alone, then rebuilt CF in C on every trip. The edge now starts with nothing live, and the block's entry liveness is iterated from there until it stops growing. 7-Zip's LZ copy loop (`mov byte`, `movzx`, `inc rdx`, `jne`) was the hottest block of its

decompression, 18% of the time, and 136 of the 150 samples in `ocerz_flags_materialize` came from its back edge, though nothing reads CF after it.

- **The look-ahead stops early** (13 October, `e34ad68f`). Folding def/use backwards, a flag is live at a successor's entry only if some instruction reads it before an earlier one writes it, or if it is live out and nothing writes it. Once the instructions decoded so far have written all six flags, neither can happen for anything after them, so the decode stops at that instruction and the successor's own successors are never followed. Compared with the full fold from every byte offset of bash's and libc's text, 187 million comparisons, no answer differed; following real branch and call sites without the memos, the look-ahead made 5.4 to 7.0 decodes per exit where it had made 25.5 to 32.9. `OCERZ_NO_XLIVE_KILL=1` reads every successor to its end.

Four fusions

1. **NZCV forwarding.** When a compare-like producer (`cmp`, `sub`, `add`, `test`, `and`, `or`, `xor`, `bsf`, `bsr` or a bit test) is followed, within a window of a few "gap" instructions, by a `jcc`, `setcc`, `cmovcc`, `adc` or `sbb`, the producer is emitted as an arm64 instruction that sets NZCV and the consumer reads NZCV directly: `cmp` then `b.ne`. The window was three instructions until 11 October and is eight now (see "The NZCV window"). A compare or test that ends a block fuses with its closing `jcc` across at most one gap instruction.
2. **Value-based conditions.** When the producer wrote a 32- or 64-bit pinned register and the condition only asks about the result (zero, sign), the condition is taken from the result register itself: `cbz`, `cbnz` or a compare against zero, with no flags at all. Additions, subtractions, logic, increments, decrements, negation and shifts by a non-zero immediate qualify.
3. **The comis fusion.** After `ucomisd` or `comisd`, a `jcc`, `setcc` or `cmovcc` re-derives its condition by redoing the floating-point compare with `fcmp`, mapping x86's unordered rules onto arm64's. Since 8 October (`951b6977`) the second operand may be memory, the usual `ucomiss xmm0, [rip+limit] ; ja` of game code: the producer still performs the load, so a fault stays its own, and keeps the value in a cpu slot, `jit_fcmp_mem`, which the consumer compares against instead of guest memory that a store in between may have changed.
4. **The ptest fusion** (since 9 October, `ff8ce8b9`). A `jcc`, `setcc` or `cmovcc` on ZF or CF after a `ptest` of two pinned registers that nothing in between writes tests the registers itself, and the `ptest` emits nothing. `deno --version` spent 98% of its 3.4 seconds in one loop of them; it takes 0.25 seconds fused (Rosetta 0.19).

A smaller pairing rides on the same machinery. `setg al ; setl dl ; movzx edx, dl ; movzx eax, al` is how compilers turn a comparison into an integer; since 8 October a `setcc` whose register a `movzx` widens a few instructions later, with nothing in between reading or writing that register or touching memory, writes the whole register with one `cset`, which zero-extends, and the `movzx` is skipped (`08215acc`).

Deferred: the flags leave the block

x86

```
add rax, rbx
mov rcx, [rdi]
jmp next
```

arm64

```
add x21, x21, x24
ldr x22, [x28]
exit stub:
  cc_op = ADD | 64
  cc_dst = x21 (result)
  cc_src = x24
```

A successor that reads ZF evaluates it from the record.

Fused: NZCV forwarded over a gap

x86

```
cmp rax, rbx
lea r15, [rsi+8]
jne L
```

arm64

```
cmp x21, x24
add x2, x27, #8
b.ne L
```

A closing jcc's gap: one instruction, flag-neutral, writing only pinned registers, never a register the compare read. It is emitted into a scratch buffer first and refused there if it breaks a rule.

Figure 6.5. Two ways the JIT avoids computing x86 flags. Left: a record that is written only when the flags leave the block. Right: the compare's own NZCV result consumed directly by the branch. Consumers inside a block may sit further on; Figure 6.6 shows the wider window.

The NZCV window

7-Zip's LZMA decoder, which ran at half Rosetta's speed under Wine, decodes each bit with a `sub` and then reads its carry five times: two `cmovae`, two `cmovb` and an `sbb`, the last of them eight instructions after the `sub`, with moves in between and with the `cmov`s rewriting the `sub`'s own operands. In half the steps a load of the next probability sits among them as well. With a three-instruction window only the first consumers took the forwarded NZCV; the others read the flag record back and rebuilt the carry, 15 to 18 instructions each, and over half the code in the decoder's hottest blocks was those reads. Four commits on 11 October took the step apart (Figure 6.6):

- **Eight instructions** (`e853c450`). The window grew from three to eight. Its gap may hold what it held before: moves and `lea`s between pinned registers, and `setcc` or `cmov` siblings reading the same producer. A sibling may now write the producer's operands when the consumer is a `setcc` or `cmov` (since 8 October, `fa5f1f3c`) or an `adc` or `sbb`: all of them take the flags from NZCV and never rebuild them from those registers. A `jcc` keeps the old rule, because a fused `jcc` may re-derive its condition from the registers. `OCERZ_NZCV_GAP=`*n* sets the window from 0 to 8.
- **Loads** (`a8773e55`). A plain load into a pinned register (`mov`, `movzx`, `movsx`, `movsxd`) may sit in the window, although its translation may clobber NZCV: in the Wine layout its guard compares against x29, and a slow path calls out. The producer must then be a `cmp`, `sub` or `add` of two 32- or 64-bit registers, or of a register and an immediate, whose record is just its two operands. Such a producer keeps writing its record, and after each load in the window NZCV is rebuilt from it with an `ldp` and a `cmp` or `cmn` (`nzcv_regen_emit`) before the next consumer; the record also gives a fault at the load the producer's flags. A consumer of ZF or SF after an `add` or `sub`, which would normally test the result register and leave the producer with no record, takes the record path when its window holds a load.
- **A 32-bit `lea`** (`571a7295`). MSVC writes some steps' `lea` with an address-size prefix, `lea edi, [ebx-8]`. Emitted as an `add` or `sub` on w registers, it leaves NZCV alone, so it may sit in the window and in the gap of a fused `jcc`.
- **`adc` and `sbb` of an immediate as one `csinc`** (`d68898a5`). With the carry a condition on NZCV, `sbb r, imm` is `r - (imm + 1)` plus one where CF is clear, and `adc r, imm` is `r + imm` plus one where CF is set: a `csinc` after at most one `add` or `sub`, where a `cset` and two additions had been. The decoder steps its symbol with `sbb r, -1`, which is the `csinc` alone.

window: 8

	x86 (one decoder step)	arm64, in outline	role
0	`sub ebp, eax`	`subs w26, w26, w21`	producer; keeps its record
1	`cmovae eax, edi`	`csel w21, w28, w21, hs`	consumer; rewrites eax
2	`movzx edi, word [r11+6]`	`ldrh w28, [x6, #6]`	load: may clobber NZCV
		`ldp x9,x10,[rec]; cmp w9,w10`	NZCV rebuilt from the record
3	`cmovae edx, edi`	`csel w23, w28, w23, hs`	consumer
4	`mov edi, 0x1f`	`mov w28, #0x1f`	flag-neutral move
5	`cmovb ebp, esi`	`csel w26, w27, w26, lo`	consumer; rewrites ebp
6	`cmovb edi, r10d`	`csel w28, w5, w28, lo`	consumer
7	`mov ebx, 2`	`mov w24, #2`	flag-neutral move
8	`sbb ebx, -1`	`csinc w24, w24, w24, lo`	8 on: ebx + 1 - CF in one csinc

Until 11 October the window was three: consumers past it read the record back, 15 to 18 instructions each.

Figure 6.6. One bit of 7-Zip's LZMA decoder, as `tests/dynamic/nzcv_window.c` reproduces it, and how its flags travel. The `sub` 's carry reaches five consumers up to eight instructions on; siblings may rewrite its operands, and the load in the middle costs a rebuild of NZCV from the record. The arm64 is in outline, with registers as pin class 3 assigns them.

Commit, 11 October	Change	KiB/s before	after
`e853c450`	a window of eight; `adc` and `sbb` past sibling `cmov` s	58,634	64,167
`a8773e55`	a load may sit in the window	64,005	70,805
`d68898a5`	`adc` or `sbb` of an immediate as one `csinc`	70,549	71,325
`5fc69e49`	flags live at a loop's own start as a least fixed point	71,586	85,141
`571a7295`	a 32-bit `lea` in the window	84,586	**85,552**

The table gives 7-Zip 25.01's own benchmark (`b -mmt1 -md22` , x64) under Wine Staging, decompression rate, as each commit measured it; Rosetta did 124,000 KiB/s. Compression did not move with any of these: 70% of it is LZMA's binary-tree match finder, whose cost lies in memory ordering and address forms rather than flags (Chapter 7).

Memory compares had joined the window earlier in the month. Since 8 October a `cmp` or `test` whose first operand is memory forwards its NZCV to a `setcc` , `cmovcc` , `adc` or `sbb` (`fa5f1f3c` , `50121f1c` ; winbench's `qsort` comparator, `cmp [rcx], eax ; setg al ; setl dl` , went from 47 host instructions to 21), and since 10 October so does any compare with a memory operand in the Wine layout (`a4b19a7d`). The fused closing `jcc` took a stack-slot load as its gap on 8 October (`2194344b`), for ucrtbase's `qsort` , which reloads a slot between `test eax, eax` and `jle` ; because a fault at that load resumes there and the `jcc` then reads the record, the compare writes its record before the gap. The same commit lets `test` drive every condition that reads only N and Z, since after `ands` , as after `test` , C and V are clear. And on 18 October `adc` and `sbb` into memory with their flags dead stopped going to the interpreter (`b77d67dc`): i386 code adds a 64-bit value to memory as `add [m], lo ; adc [m+4], hi` , and Nexuiz's benchmark demo, whose engine averages textures that way, went from 104.1 to 96.9 billion cycles.

Rules written by bugs

Every legality rule around these fusions was learned from a miscompile in real software:

- **A gap may not write a register the producer read.** A byte compare through an index register that the very next instruction overwrote made the forwarding fallback re-load the byte through the new index value, and a loop spun forever.
- **A gap's emission may touch only pinned registers.** In `cmp byte [rdi+0x210], 0 ; lea r15, [rsp+0x290] ; jne`, the `lea`'s pointer-`rsp` form loaded its base through the temporary that held the compared byte, so the branch tested `rsp` instead of the byte. Steam's CEF browser process copied an unengaged `std::optional` and crashed on its first navigation (6 September). Since 7 September a gap is emitted into a scratch buffer first and refused there, instead of asserting after the compare is already in the block.
- **Flags can cross a return.** No ABI passes flags out of a function, but clang's outliner does: it turns a repeated compare into a helper that ends in a compare and a `ret`, and the caller branches right after the call. The JIT treated every `ret` as killing the flags, and every Wine window painted black (fixed 5 September, `6073adee`). A pure flag producer that reaches a `ret` now keeps its flags live; a tail whose last flag writer is arithmetic (`xor eax, eax ; ret`) is returning a value, and keeps the dead seam. `tests/guest/ret_flags.c` pins it. Since 9 October the same rule is settled across blocks, for an epilogue that writes no flags itself (see "Liveness").
- **A self-loop's compare is not fused when the loop head reads the flags.** libc++'s partition loop starts with a `jbe` on the previous iteration's compare. Fusing the compare at the bottom with the branch back to the top skipped recording the flags the top needed, and Steam's CEF renderers died on libc++'s "does your comparator satisfy the strict-weak ordering requirement?" assertion (fixed 13 September, `bbb476e4`; `tests/dynamic/cef_partition.s`).

October's rules came mostly from generated tests rather than from real software. When condition forwarding and the pair fusions were switched on in 32-bit blocks on 7 October (`c5fe05f4`), the 32-bit differential's first run failed 98 of 20,000 random cases, on two rules:

- **A fused pair names its second instruction as the latest flag producer.** After a fused pair such as `add ; inc`, the translate loop skipped the bookkeeping for the second instruction and went on naming the first, so a `jb`, `setb` or `adc` evaluated the `inc`'s record as an `add`'s. That was true in 64-bit blocks too, wherever the `inc`'s own flags were live: `tests/guest/pair_flags.c` prints 3, and the build before the fix printed 4294967299.
- **Forwarding needs a translated consumer.** The producer leaves its flags in NZCV and nowhere else, so a consumer that goes to the interpreter reads a stale record. A 32-bit `cmov` through 16-bit addressing did that, and so did a 64-bit `cmov` whose memory operand carried a `0x67` prefix. The consumer check now asks the 32-bit whitelist about a 32-bit consumer, and wants the address size the 64-bit memory forms accept.

The window itself is held by `tests/dynamic/nzcv_window.c`, run in both layouts against Rosetta, which catches each bug planted while it was written: a fused `adc` reading the inverted carry, no rebuild after a load (caught only in the Wine layout, whose guard clobbers NZCV), and the record dropped when the window has a load.

Exits and chains

A block that ends in a direct jump or falls into another block exits through a *chain tail*. The tail ends in a single `b` instruction which initially points at its own fallback: set `cpu->rip` to the target, record the side-exit tag if profiling, and leave through the shared exit. When the target block exists, that one word is patched to branch to it directly. There are three kinds of edge. A **body edge** (pin class 3 on both sides) jumps from body to body with every guest register in place. A **cross-block edge** runs the full epilogue and enters the target's full entry. A **self-loop edge** branches back to the block's own loop head.

If the target is not translated yet, the edge is registered in a table of pending chains keyed by the target (2^{18} buckets since 21 September, 2^{12} before), and activated when the target is published. Every activation also records the source block and edge on the target's list of predecessors, so that retiring a block can unchain exactly the branches that point into it rather than scanning every block in the process (since 27 September; the scan had been 18% of a CEF process's samples while V8 rewrote its code).

A pending record keeps the patch site, the conditional branch that may later be short-circuited to the target, the source block and edge, and the source's hoist signature; return-address slots that wait for a block wait in the same table. Retiring a block walks its predecessors, patches each chained branch that lands in the block back to its fallback, and registers the edge as pending again, so it chains to the retranslation when that is published. Three holes in that walk were closed in October:

- **Chained calls** (8 October, `dd5717fd`). The decoder of patched words knew `b`, `b.cond`, `cbz` and `cbnz` but not `bl`, and a chained direct call is a `bl` into the callee's body, so callers went on calling a retired callee. A retired translation still computes the right thing, which kept this quiet until a retirement had to take effect: a test of x87 blocks that retire themselves saw one function retired 5,999 times.
- **Chains through veneers** (11 October, `bfaca27a`). Unchaining compared each branch's target with the retired block's code, and a branch chained through a veneer (see "Veneers") lands on the veneer, so such chains were never cut and the retired block went on running from them. `chain_target` now follows a veneer to the address it loads. In the three busiest of Steam's CEF processes under Wine, probe exits from retired blocks fell from 1,905,762, 1,159,135 and 527,613 to 323,620, 67,584 and 24,701. A follow-up the same evening (`e778db6a`) recorded that this was a correctness fix as well: with every chain made through a veneer, `tests/dynamic/smc_ras.c` fails without it, because a far chain left in place reached the old translation after a write to guest code (Chapter 7).
- **Finding a block that has already left** (the same commit). A block leaves the live array by the last block moving into its place, so one whose recorded index does not point back at it is known at once not to be live; the retirement path used to search the whole array for it. Steam's CEF browser process had run 169,984 such searches of a 716,148-block array, 29.4 seconds of its CPU.

Installing chains

Publishing a block runs `blk_chain_install` under the JIT lock: each of the new block's edges whose target exists is patched to it, the rest are registered as pending, and every pending edge that was waiting for the new block is drained and patched. Each patched word must be visible to instruction fetch before the lock is released, and until 13 October each went through its own `sys_icache_invalidate`. On Apple silicon a call over one word costs about what a call over a whole block costs: the price is in the barriers, not in the cache lines. Since `f4ef8900`, `chain_batch_begin` opens write access once, `chain_activate` and

`chain_cond_short` record every word they patch (up to 64; past that a word is flushed on its own), and `chain_batch_end` restores write protection and makes them all visible with the sequence the stop-site patcher already used:

```
dc cvau, word        // each word: clean its data cache line
dsb ish
ic ivau, word        // each word: invalidate its instruction cache line
dsb ish
isb
```

Every word is still invalidated, the barriers complete before `translate` returns, and an install that patched nothing issues none. Patches made outside an install (retiring, flips, unchaining, a signal stop) keep their per-word flush. A barrier pair costs about what a call did, so a block now pays one pair for its install where it paid one per patched word; on the perl start that came to 215 against 208 ms of translation (see "What translation costs"). `OCERZ_NO_CHAIN_SYNC_BATCH=1` flushes word by word again.

Retargeting conditional branches

A conditional branch in a superblock normally goes to its out-of-line side stub, which then branches to the target. Once the target is translated, the conditional branch itself may be retargeted straight at the target's body, saving the extra branch, but only when nothing on the stub's path is still needed. Three things can be: a lane flush that writes cached scalar values back into an xmm register (Chapter 7), an FP-batch check, and the producer's flag record. Both halves of that rule came from miscompiles. On 6 September (`8ce197f2`), a stub's flag record was skipped by a retargeted branch (a `cmp ebp, 0xb ; jbe L` whose target begins `L: ja`) and every SQLite open in Steam's `libcef` failed with "out of memory"; the retargeting itself dated from 15 August. On 8 September (`8851958c`), a scalar SSE result still in its lane register was skipped by a side exit chained past its flush, and Cocoa's view transforms came out singular.

Veneers

An arm64 `b` reaches 128 MB in either direction. The arena is 1 GB and fills front to back, so a caller translated late could chain to a callee translated early only if less than 128 MB of code had been produced in between; beyond that the patch was silently dropped and the edge left through the epilogue on every execution. That is exactly the shape of a game, where the engine's helpers are translated in the first minute and the script code calling them keeps being generated afterwards. An attributed profile of Brawlhalla's main thread put 26% of its time in block prologues and epilogues, with 4.5-5.8% of chains dropped, and the garbage collector's mark routine, 18% of the thread by itself, was reached that way.

Since 22 September a 256 KB veneer pool is carved out of the arena at the start of whichever translation first crosses each 64 MB mark. A patch that cannot reach its target is pointed at a 16-byte veneer in the nearest pool instead (pools within 120 MB are considered): `ldr x15, [pc+8] ; br x15` followed by the 64-bit target address. x15 is the computed-address temporary, never live across a block edge. Brawlhalla's main thread fell from a saturated core to 71% of one while executing 60% more blocks per second; every chain activated (`veneered=39010, out_of_range=0` in `OCERZ_PERFSTAT`), and return-stack misses fell from 36 to 11 million a second because fewer exits wiped the stack. The alignment hotpatch, which puts a branch at a faulting access and a stub at the end of the arena, uses the same pools when the end is out of reach. Because only far chains go through veneers, a test rarely exercises them; `OCERZ_FORCE_VENEER=1` makes every chain take one, and since 11 October the dynamic gate runs four tests again under it: two that rewrite chained code, the flip test and the self-loop liveness test.

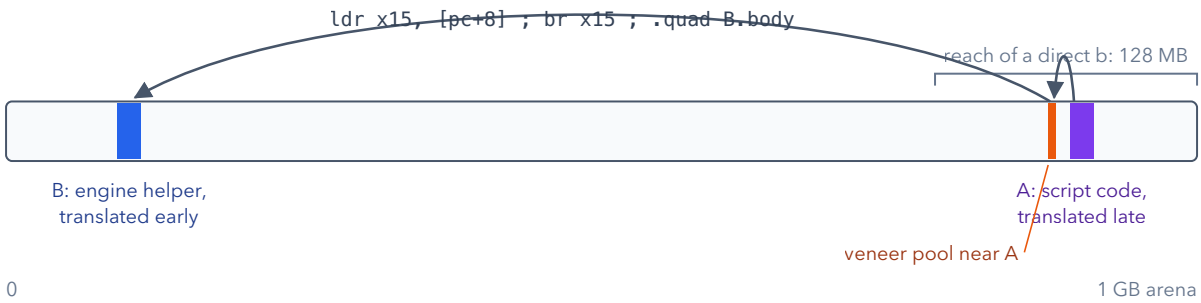


Figure 6.7. A chain that a plain branch cannot reach goes through a 16-byte veneer in the nearest pool, which loads the target's address and branches to it. Before 22 September such chains were dropped and the edge went through the dispatcher every time.

Stop sites and interrupts

Translated code has to notice when another thread wants it to stop: a signal to deliver, a process exit, an arena flush. Polling an interrupt word on every loop iteration costs a load and a branch per iteration, so steady-state loops do not poll. Instead, the first polling edge in a block keeps its chain branch as a *stop site*, and up to six more edges and indirect branches register extra stop sites. To stop the thread, AArchX patches those branches back to their fallbacks, so the next time the loop comes round it leaves through the dispatcher, which checks the interrupt word. The dispatch stub checks the same word on every unchained transition. The arena flush of Chapter 7 relies on the same mechanism.

A block records each stop site's address and the word that stops it, and since 14 October also the word it holds while the loop runs (`stop_run`), so that a stop can be undone. An edge that would poll once a block has used up its six extra sites loads the interrupt word and branches on it instead. The flush patches every stop site of every block on the JIT's stop list, makes them visible with one barrier pair, and puts the old words back if it has to give up.

Signals in translated loops

The host handler for an asynchronous signal only marks it pending on the thread's cpu; the guest gets it when the thread next reaches a point where its whole state is in the cpu. Until 14 October those points were the thread's next system call or bridged call and nowhere else, in the interpreter as much as in the JIT. A loop of `dec` and `jnz` under a 5 ms `ITIMER_REAL` got no `SIGALRM` at all until it ended, and under Wine, where `SuspendThread` is a `SIGUSR1` sent to the thread, Mono's garbage collector waited for any busy thread to finish its loop. `a7fd0f3f` fixed it in three parts (Figure 6.8):

- **Delivery between steps.** Both run loops deliver a pending signal the thread does not block before each step. That was enough for the interpreter; translated code reaches the run loop only when its blocks exit to it.
- **Stopping the loop.** The handler calls `ocerz_jit_signal_stop` with the interrupted host pc. It finds the block the pc is in and does to it what a retirement does, without retiring it: each chained edge goes back to its fallback, each short-circuited conditional branch back to its stub, and each stop site to its stop word. It does the same to the blocks those edges led to, up to eight blocks in all, because the thread may be on its way into one of them, and it records in the cpu every block and edge it changed.
- **The dispatch stub returns.** The stub, which block exits jump into and which enters the next block without returning, now returns to the run loop while a signal the thread does not block is pending.

The run loop then calls `ocerz_jit_signal_unstop`, which, under the JIT lock, puts each stop site back to its running word and chains each edge again as an install would, and delivers the signal. Another thread running the same blocks meanwhile takes a trip or two through its dispatcher. Stop sites alone would not have been enough: a loop whose backward branch is a superblock side exit, chained through its conditional branch, has no stop site anywhere, which is why the walk unchains edges.

The handler takes no lock. In the test that pinned this, the thread holding the JIT lock was the one sending the signal, translating its own wait loop, and a handler that tried the lock and gave up lost the signal. So the handler can race a chain install on the same word; every word either side writes is a whole branch that is valid on its own, so the worst case is an edge chained again before the thread passes it, and the signal then waits for the next stop. The blocks the walk reads stay allocated, because a retired block is freed only with the whole JIT. A signal that interrupts host code rather than a block, or a loop through blocks the walk does not reach (one that returns through the return-address stack into a block it did not visit), still waits for the next return to the dispatcher. m32 processes keep the old delivery. `tests/dynamic/loop_signal.c` runs seven loop shapes that only their handler can end, after 20 signals have reached it, with plain and with ordered memory, and then has a second thread spin while 20 `pthread_kill`s must each reach it, as Wine's `SuspendThread` does to a busy thread. `OCERZ_NO_SIGNAL_STOP=1` restores the old delivery.

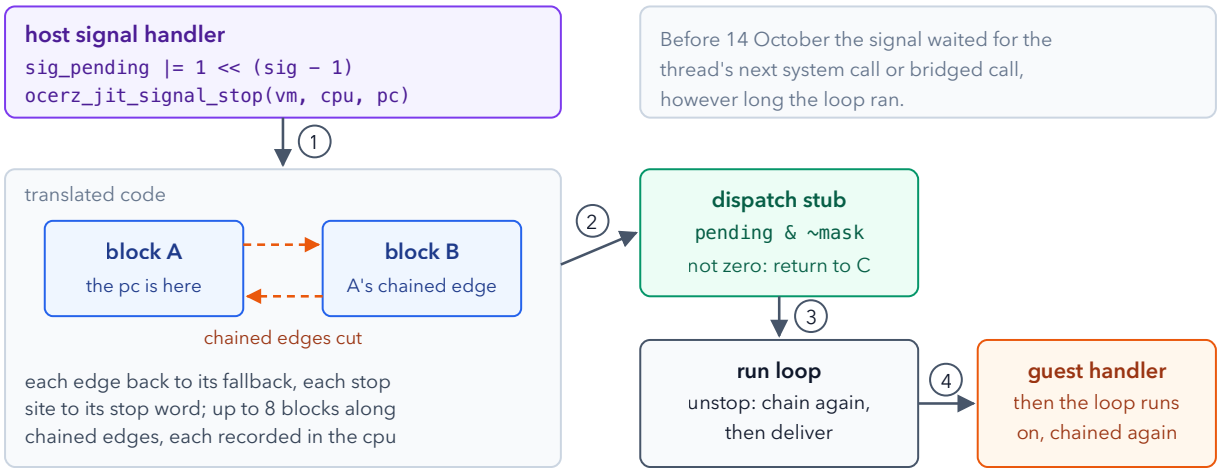


Figure 6.8. A signal for a thread inside a translated loop, since 14 October. The handler marks the signal pending and cuts the loop's chained edges (1); the thread's next edge leaves through the shared exit to the dispatch stub, which sees the pending signal (2) and returns to the run loop (3), which restores the edges and delivers the signal (4).

HOW IT WAS FOUND

The gap showed up as a side note. On 13 October, while testing the `dec / jnz` fix described under "Lane caches and the edges of a loop", a thread spinning for 2.6 seconds in `lea ; dec ; jnz` under a 20 ms `ITIMER_REAL` took one or two of its 130 `SIGALRM`s, before the fix as after it; the commit recorded it as found on the way and not touched. The fix landed the next evening. Earlier gates had logged an occasional missed signal under load: on 8 October `thread_signal-jit` missed a signal in 3 of 48 runs with eight running at once, and the tree before that day's commits missed at the same rate.

Calls and returns

Returns are the hardest branches for a translator, because the target is data on the guest stack. AArchX makes a guest `call` / `ret` pair use the processor's own return predictor.

The host return-address stack

A direct guest `call` under full pinning (Figure 6.9) loads the guest return address into x10, takes the address of the host instruction after the call with `adr`, and pushes the pair `{guest return address, host continuation}` onto the *host* stack. It pushes the guest return address onto the guest stack as x86 requires, and then performs an arm64 `bl` straight into the callee's body; that `bl` starts out pointing at a fallback and is patched when the callee is translated. The callee's `ret` pops the guest return address from the guest stack and the pair from the host stack, checks that the pair's guest address matches the one just popped and that the host continuation is not null, and executes a real arm64 `ret` through x30. Because every `bl` is matched by a `ret`, the hardware return predictor stays in step with the guest's calls. The code at the continuation is an ordinary chain tail to the block at the guest return address.

This has been the design since 17 August. Its weakness is lifetime: an entry lives only until the next trip out of translated code, because every exit resets the host stack pointer to the frame base saved in `cpu->jit_fp`. The sentinel pair `{0, 0}` pushed by the prologue marks that base, so a `ret` that finds it knows its entry is gone.

32-bit code has used the same stack since 8 October (`e7de713b`). A 32-bit `call` pushes `{return address tagged with JIT_KEY_M32, continuation}` and `bl`s or `blr`s into the callee's body, and its `ret` compares and returns with a real `ret`. The tag is the bit that keys a 32-bit block (bit 63), so an entry pushed by one mode can never satisfy a return in the other under WoW64. `call $+5`, the i386 idiom for reading `eip`, pushes no entry, since nothing will return to it. The same commit took winbench32's loop of virtual calls into small frames from 461 to 166 ms (Rosetta 158), though most of that came from its other change, plain accesses to `esp`-relative slots (see "32-bit blocks").

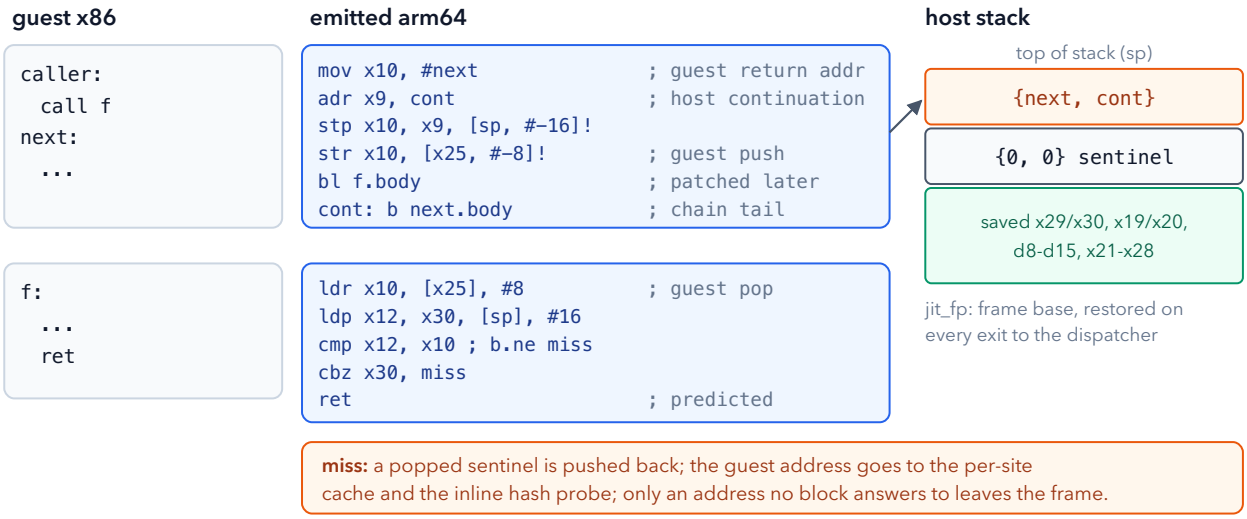


Figure 6.9. A guest call and return through the host return-address stack. Every `bl` is matched by a real `ret`, so the processor's return predictor sees the guest's call structure.

When the entry is gone

Until 22 September, a `ret` whose entry had been lost left the frame through the epilogue, and that exit reset the host stack, discarding every entry still outstanding, so the whole call chain then missed on its way up, each miss doing the same. C++ engine code makes a call every ten instructions or so and leaves translated code every few hundred thousand blocks, which is enough to keep the host stack permanently empty. On Brawlhalla's main thread `OCERZ_PERFSTAT` read 3.2 billion misses in 88 seconds, 36 million a second, one return in seven, all stale; the older software return stack (a 256-entry ring in the cpu struct, `OCERZ_NO_HOST_RAS=1`), which survives exits, cut the misses to 393 thousand but cost 18-26% on the `memcpy` and `str` kernels. Now a miss stays in the body: the popped guest address goes through the same per-site cache and inline hash probe an indirect call uses, with the same body handoff, and only an address no block answers to leaves. A pop that finds the sentinel pushes it back first, because the saved frame registers sit right above it and the next call would overwrite them. Brawlhalla's translated throughput rose from 2.6-2.9 to 3.85 billion guest instructions a second; the kernels the host stack was designed for measure the same. Since 4 October (`d8af2a18`) `OCERZ_PERFSTAT` splits misses by cause, a popped sentinel (the stack was empty), a null entry and a mismatched address, and names the return sites that miss most: on R.E.P.O.'s menu 183.6 of 184.1 million misses were the empty stack, which now costs a cache probe rather than a trip through the dispatcher.

Indirect jumps and calls

An indirect jump or call first probes a per-site cache: 32 direct-mapped pairs of `{guest rip, body address}`, indexed by bits 2-6 of the target, 16-byte aligned so that the `ldp` reading a pair is single-copy atomic and a concurrent update can never produce a torn pair. On a miss it runs the same hash probe the dispatch stub uses, inline, and only then leaves to C. A hit on an indirect call uses `blr`, keeping the return predictor matched as a direct call does.

Retiring a block has to stop these caches naming its body. Until 5 October every retirement walked every per-site table to clear such entries; R.E.P.O. builds about 260,000 of them, 134 MB, and the walk took about 4 ms per retirement with the JIT lock held. Since `d8b7073c` an entry's rip word carries its column's retire generation above bit 48 (the rip XORed with the generation). `struct OcerzVM` begins with the 32 generations, so the hit path reads its column's as `[x19, column, lsl #3]`; retiring bumps the columns of the blocks it removed once they have left the hash table, and the miss path reads the generation with an acquire load before it walks the table, so a refill can never tag a retired body with a current generation. Retirement in R.E.P.O.'s process over a menu session fell from about 6.2 seconds to 75 ms. The hit path is two instructions longer, which costs xbench's `vm`, a twelve-way switch dispatch, 9-11%, still ahead of Rosetta.

Two more kinds of indirect branch joined this path in October. In 32-bit code a `ret`, `call reg`, `call [mem]`, `jmp reg` and `jmp [mem]` left for the dispatcher every time until 7 October, and 32-bit Windows code calls every import as `call dword [IAT]`. They now take the same cache and probe, with the target zero-extended and tagged `JIT_KEY_M32` (`03eb458b`): under winbench32 a vtable-call loop went from 1,321 to 464 ms (Rosetta 159) and `qsort` from 634 to 163 ms (153). And a jump through a rip-relative slot, the `jmp [rip + slot]` of a stub into another image, now compares the target it loads with the value the slot held at translation, with an `eor` and a `cbnz` that leave the flags alone, and chains straight there when they agree (`765891d8`, 10 October); a slot rebound since takes the cache and the probe. xbench's `str` went from 0.504 to 0.472 seconds outside the Wine layout (`OCERZ_NO_SLOT_CHAIN=1` turns this and the look-ahead through slots off).

Splicing small callees

Since 25 August, a direct call to a short function is spliced into the caller's block while decoding. The callee qualifies if it is straight-line code of at most 24 instructions ending in a plain `ret`; calls inside it may themselves be spliced, two levels deep. The `call` becomes a push of the return address and the `ret` becomes a compare of the popped address against the known return address, with a mismatch leaving at the `ret`'s address so the dispatcher can run the real return. Two refinements remove most of the remaining memory traffic. Where the callee's frame is pure register work, the pushed return-address slot is provably never read, so the push becomes a bare `rsp -= 8`. And matched push/pop pairs, the store-to-load chain that dominates call-dense code, become register renames into x16, x17 and x30 when no memory base is hoisted there.

The renames had a hole. Every C call-out clobbers x16, x17 and x30, so a renamed pop whose push had been followed by a call-out read garbage. In the Wine memory map every nested spliced call took the slow path, and a two-level leaf-call loop built at `0x200000000` printed a different wrong sum on every run; xbench's `leafcall` tripped the stack protector. The bug was older than that layout (a build from 26 September fails the same way) and could have struck in any mode. Since 27 September (`5f36d915`) the renamed push still stores its slot, and the pop compares the call-out sequence number recorded at the push with the current one: if they differ, it pops from memory. `tests/dynamic/promo_callout.c` is that loop.

Two October changes touch splices. A spliced callee lies wherever the callee is, not between its caller's first and last instruction, so until 11 October a write to it passed every block's span check, and the callers went on running the old code: a function hooked by patching its first bytes stayed unhooked for every caller that had it spliced in (`60bd6cea`; Chapter 7 covers the span each block now keeps). And since the same day the stack accesses of a spliced region whose stack traffic is its own are made at fixed offsets from a register that does not move (see "Stack traffic").

Returns in Wine's low shadow

Wine's stacks may lie below 12 GB, where guest and host addresses differ, so every stack fast path used to be switched off in that map and every spliced call and return went out to C: xbench's `leafcall`, built as a program at `0x200000000`, took 2.75 seconds against 0.16 outside the window. Nothing in the return-address machinery depends on the memory map; only the two guest-stack accesses do. Since 27 September, with `rsp` pinned as a pointer and a zero guest base, a call stores its return address and a return loads it through the same guarded translation as any other low-window access, and the rest (the host shadow push, the `bl`, the compare, the `ret`) is unchanged. `leafcall` went to 0.35 seconds, `icall` from 0.74 to 0.44, and Windows Steam's main window came up in about 33 seconds instead of 36-38. `OCERZ_NO_LOW_RAS` and `OCERZ_NO_LOW_SPLICE` turn the two halves off.

The guarded translation of the two stack accesses lasted ten days. Since 7 October a stack access in that layout adds the stack delta kept in x0 instead of taking the guard (`e7a1cd62`): in xbench's low-linked build, measured in pairs on one machine, `leafcall` went from 1.27 to 0.65 seconds (Rosetta 0.62) and `str` from 0.84 to 0.75. Since 8 October a call and a return reach their slot with a register offset off `rsp` and x0 (`f6eb8d84`), and since 10 October `rsp`'s register holds the host address itself, so they are the same single pre- and post-indexed accesses as in every other layout (see "Registers in the Wine layout").

In native mode, a call into a synthesized system library is turned into a call made from inside the block as well, with a real `ret` afterwards; that bridged fast call is described in Chapter 12.

Superblocks and trace inversion

Since 16 August a block may run past up to six conditional branches. Decoding continues on the fall-through path and each taken side becomes an out-of-line side stub; since 17 August that includes backward branches other than a branch to the block's own start. A side stub carries everything the taken path needs that the inline path deferred: it replays lane flushes, checks a pending floating-point batch, writes the flag record if the target reads flags, and then chains. Keeping that work in the stub is what makes the inline path cheap, and it is also why the retargeting rule above exists.

Superblocks assume the fall-through is the hot path. When a compiler laid the rare case inline (`if (rare) { ... }` with a jump over it), the hot path is the taken edge, and a loop fragments into a chain of blocks connected through side stubs. Since 3 September such a branch is probed: counters in the arena count both sides, and when the taken counter reaches 1,024 its stub returns to the dispatcher with `OCERZ_STEP_PROFILE`, which hands the edge to `flip_side_hit`. The verdict is "invert" when the taken side ran at least twice as often as the fall-through and at least 512 times. The first window of a loop is often unlike its steady state, so a verdict counts only when the next window repeats it, and the fourth window decides regardless. A decision to keep the branch turns the counters into `nop`s and chains the edge. A decision to invert retires the block, and the retranslation complements the condition while decoding, so the hot side continues inline. Decisions are remembered in a 4,096-entry table so the block is not probed again, at most 65,536 probes are made per process, and a translation shaped by a decision like this is never stored in or loaded from the translation cache. `OCERZ_NO_FLIP=1` turns inversion off, `OCERZ_FLIP_JCC` names branches to invert by hand, and `OCERZ_FLIPLOG=1` logs every verdict.

A decision to keep was final until 8 October. winbench's `structs` fills its array beyond the branch limits on its first pass, so for that whole pass the inline side of both `if (p->x > 100.0f)` branches was the hot one; every window said keep, and for the rest of the run each particle left the superblock by its side exit and crossed three blocks. Since `27e8e609` a kept branch goes on counting its taken side and trips at 65,536 (2^{16}); `flip_side_hit` then restores both probe branches from the words it saved and probes it from scratch, at most three times per translation (`OCERZ_NO_FLIP_REARM=1` turns this off). `structs` went from 23.7 to 14.5 ms (Rosetta 11.3), the same as inverting the two branches by hand.

A kept branch still left its probe behind: the fall-through's counter became a `nop` that the block executed on every pass. Since 18 October (`d7567fc5`), once no edge of a block still probes, or watches through its stub, the block is retired and translated again, and branches already decided are not probed in the retranslation. (A body edge with a short chain does not pass its stub at all, so its watch could never trip.) On `bench br`, a loop of a linear congruential generator and two branches on its random bits, cycles fell from 2.870 to 2.722 billion (Rosetta 2.64-2.74), and xbench's `str` went from 1.074 to 0.967 times Rosetta's time. `OCERZ_NO_FLIP_KEEP_RETIRE=1` leaves the `nop`s in place. Figure 6.10 puts the whole life of a probed branch together.

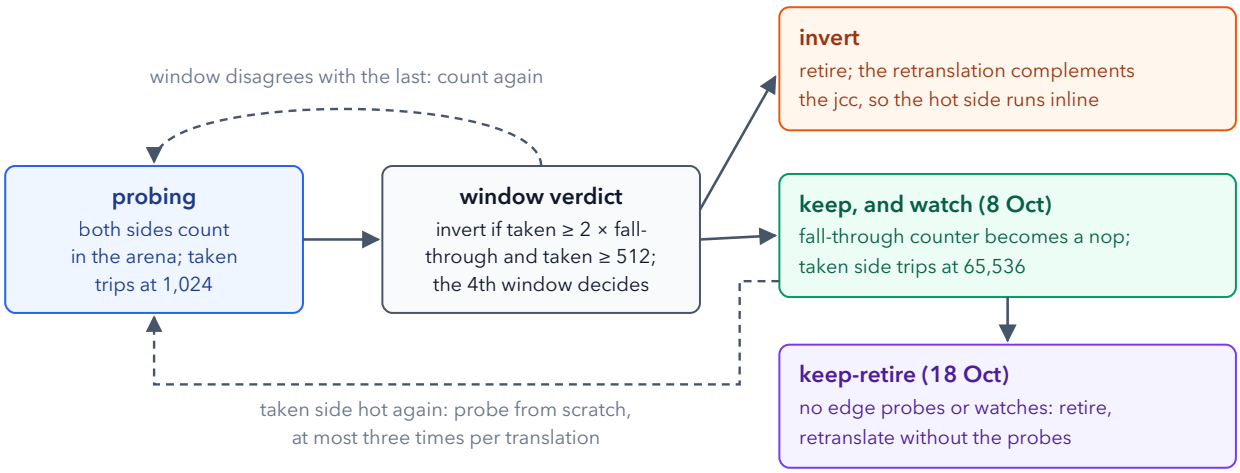


Figure 6.10. The life of a probed superblock branch. Inversion dates from 3 September; the watch after a keep from 8 October; retranslating a block whose branches are all decided, so that it runs without probes, from 18 October.

Retiring a flipped block used to find it in the array of live blocks by a linear search and close the gap with a `memmove`. A Steam webhelper holds about half a million live blocks and flips a thousand or more of them during startup, so each flip moved several megabytes and flip retirement was about 330 ms of every busy process's startup. Since 27 September each block records its index in the live array and a retired block trades places with the last one; nothing depends on the array's order.

The profile exit now carries other lessons as well. A block leaves with `OCERZ_STEP_PROFILE` and a negative side index to say that its translation was wrong for this process rather than for the program: in the Wine layout, a block whose hoisted base failed its check before the loop (index -2) or that needs the hoist's wide form (-4), both Chapter 7's subject, and an x87 block entered with a different TOP from the one it was translated for (-3). `flip_side_hit` marks the block's key and retires it, and the retranslation takes the mark into account. A first version of the hoist's exit returned `OCERZ_STEP_OK` instead, which the in-arena dispatcher answered by entering the same block again.

Lane caches and the edges of a loop

Chapter 7 describes the floating-point machinery; one part of it shapes control flow and belongs here. Inside a loop, a scalar SSE result is kept in one of the twelve lane registers (v4-v15) instead of being merged back into its xmm register after every operation, and since 14 September the upper halves of `ymm` registers are kept in lane registers across a loop too. A self-looping block can use a fixed-lane mode that dedicates four lane registers for the whole body. While a value is cached, the architectural xmm register is stale. Every exit from the loop therefore has to write the cached lanes back first, which is why side stubs "replay lane flushes" and why a conditional branch may not be retargeted past such a stub. Each block also records, for every instruction, which guest registers are in which lanes and which are dirty, so that a fault in the middle of a loop can fold the lanes back into the cpu state before the guest's handler sees it.

The fixed-lane mode had a hole at the most common loop ending of all. Its setup (`l0_fixed_setup`) keeps a scalar double carried round the loop in a fixed lane, and the translate loop then leaves out its usual flush before the block's last two instructions, expecting the branch's emitter to see to the lanes: `emit_jcc`, the fused compare and the nested latch restore them on the way back and flush them on the way out. The emitters for a fused `inc` or `dec` and its `jcc` did neither. Both arms left with the lanes unflushed, the taken one to the block's own body entry, where the setup ran again over what it found, so

```
1:  addsd  -8(%rax,%rcx,8), %xmm0
    mulsd  %xmm1, %xmm0
    dec    %rcx
    jne    1b
```

over 2,048 doubles gave `0x4151b0a708587be7`, where the interpreter, and the same loop closed by `sub $1` or by `cmp`, give `0x41f544f8960ddec8`. It had been wrong since the fixed lanes came in on 25 August (`30e38af7`), and it was found on 13 October (`2cf48a7e`). A self-looping block whose head reads none of the flags the `inc` or `dec` leaves now branches straight back to its loop entry, as `emit_jcc` does, with that branch as the block's stop site and the lanes handled on both arms; on every other path the lanes are flushed before the conditional branch. The direct edge is also the faster one, since the chained edge it replaced re-entered at the body entry and ran the fixed lanes' setup, and in the Wine layout the hoist's checks, on every trip: the loop above takes 1.99 ns an element instead of 2.64 (M2 Max), which is its `addsd` and `mulsd` end to end. `tests/dynamic/incdec_loop.c` covers it, including a head with `setp` reading the `dec`'s own flags, where the direct edge must not be taken (`OCERZ_NO_INCDEC_LOOP=1` keeps the chained edge).

32-bit blocks

32-bit code reaches the JIT from two directions: WoW64 processes under Wine, where 32-bit Windows code runs beside 64-bit code in one process (Chapter 15), and m32, which runs 32-bit Mac programs (Chapter 13). A 32-bit block is keyed with bit 63 set, as described under "Finding a block", and has its own dispatch stub. It uses pin class 3 like any other block: `eax` to `edi` sit in x21-x28, and x1-x8, which hold `r8`-`r15` in 64-bit code, carry nothing i386 code can name. What differs is what a block may contain. Its instructions come from a whitelist, `m32_inline_ok`, which requires a 4-byte address size, so 16-bit addressing and `0x67` prefixes stay in the interpreter. It runs past no conditional branch, so there are no 32-bit superblocks, and every effective address wraps at 2^{32} before the host mapping is applied, as does the fall-through address of a branch. The floating-point batches and lane caches stay off, because the 32-bit differential has no SSE arithmetic corpus to hold them to.

The 32-bit JIT dates from 20 August (`0d8c5593`), when 32-bit code generation arrived together with the 32-bit differential gate, which runs the same generated instruction sequences through the interpreter and the JIT, in the offset arena and in the Wine layout, and compares the results (Chapter 16). It started with every fusion switched off, as a stage rather than for any reason the fusions gave. A run of commits on 7 October, the day of 0.6, brought most of the 64-bit machinery across, each measured on the differential's own benchmark loops:

Loop (diff32 --bench, ns an iteration)	Commit	before	after
SEH frame: two pushes, `fs:[0]` moves	`3bb492c3`	57.1	10.2
the same in the Wine layout	`27e3ca41`	36.7	9.0
`add`, `inc`, `or` on memory	`5378744a`	69.7	7.4
`lock cmpxchg`	`5378744a`	23.9	9.6
`bt eax, 3 ; bts edx, edi ; btr edx, 5`	`294a12ea`	49.0	10.6
`add eax, ebx ; dec edi ; jnz`	`7a21fc53`	7.1	**0.42**

Loop (diff32 --bench, ns an iteration)	Commit	before	after
`cmp`, `setb`, `cmovl`, `add`, `inc`, `sete`	`c5fe05f4`	7.71	**0.76**

- **Segment operands.** An `fs`- or `gs`-relative operand adds the segment base to the wrapped 32-bit address without wrapping the sum, as the interpreter's `ocerz_ea` does. 32-bit Windows code reads `fs:[0]` for every SEH frame it pushes and pops, and `fs:[0x18]` and `fs:[0x2c]` for the TEB and its TLS slots, and each had been a slow call; `push` and `pop` of memory came with them, since `push dword fs:[0]` opens every frame.
- **Read-modify-write and atomics.** Arithmetic on memory, `xchg`, `xadd`, `cmpxchg` and `cmpxchg8b` go through the 64-bit emitter with the wrapped address, which covers every interlocked operation a Windows DLL performs; `cmpxchg8b` is one `casal` of EDX:EAX against ECX:EBX in ordered mode. The same commit fixed a latent bug in both modes: a block has room for 32 out-of-line arms, and when they ran out the emitter gave up after its atomic was already emitted, so the slow call put in its place performed an aligned locked add a second time. Past 32 arms the slow call now goes inline behind a branch.
- **Bit tests** on registers, and `bt` on memory.
- **The fusions** that close a block and the condition forwarding of "Flags without computing flags". The only difference from 64-bit code is that the fall-through address wraps at 2^{32}. Turning them on exposed the two flag rules described under "Rules written by bugs".
- **The stack in the Wine layout.** A 32-bit stack slot is below 4 GB, so its host address is the zero-extended `esp` with `low_base` or'ed in, with no range test; since 8 October x0 holds `low_base` in 32-bit blocks and a slot is one `[x0, w, uxtw]` access. The same day every other 32-bit address got the `orr` alone, and `esp`-relative operands became plain accesses in ordered mode as pushes and pops already were (`e7de713b`).
- **Calls and returns** on the host return-address stack, and returns and indirect branches through the per-site cache (see "Calls and returns").

Jfishin's eight commits to the JIT on 10 October came from running Batman: Arkham Asylum's i386 build through m32 and reading what its render thread still interpreted. Shifts by `cl` were 43% of it at first: the first 32-bit JIT had declined them because a count that masks to zero still wrote the register, and since `45ed22a9` the translation tests the masked count and skips the shift when it is zero, as the interpreter does in 32-bit code. (In 64-bit code a 32-bit shift by zero does write its register, clearing the upper half, as x86 hardware does; both engines were fixed for that on 9 October, `6c6389cd`.) The other forms it turned up are translated now too: MMX moves (Chapter 7), `shld` and `shrd` by `cl` or into memory, shifts of memory by an immediate with their flags dead, and `ah`, `ch`, `dh` and `bh` in arithmetic, logic, compares, tests, moves, `movzx`, `movsx` and `setcc`, in both modes, read with `ubfx` and written back with `bfi` (`358e33fb`, `e7cd1400`). The rest of that work is m32's: an import stub's jump calls its export from inside the block, small libc and OpenGL exports are called directly as leaves, and since 18 October a call through a function pointer to an export is translated as the stub it stands for (Chapter 13). x87 in 32-bit blocks is Chapter 7's.

The emitter

`src/a64emit.c` (954 lines) and its header (304) know how to spell arm64 instructions and nothing else: one function per instruction, each appending a 32-bit word to an `A64Buf`. There is no scheduling, no register allocation and no peephole pass; `jit.c` decides what to emit. An `A64Buf` carries its start, current and end pointers and an overflow flag. Writing past the end sets the flag instead of writing, and asking for a label past the end returns a pointer to a private sink word, so the translator can emit freely and check once at the end whether the block fit.

The encoders that contain logic are the ones where arm64's encoding is not a straight packing of fields. A logical immediate must be a rotated run of ones replicated at some element size; the encoder finds the smallest element that replicates the value and checks that the run, or its complement, does not wrap. `mov_imm64` offers every 64-bit constant it builds to that encoder first, and almost none qualify, guest addresses above all; the failed searches came to about a tenth of translation time. Since 27 September a value is refused at once unless the number of bit transitions going round the register is a power of two, which every valid pattern satisfies; all 6,636 valid size, rotation and width combinations still encode, and a million random values are all refused as before.

October added few encoders (`a64_csinc` on 11 October for `adc` and `sbb`, a vector `umaxv` on 9 October for `ptest`) and changed how one kind of constant is built. A value of three or four non-zero halfwords, such as a host address below 12 GB in the Wine layout or a call's return address, took a `mov` and two or three `movk`; since 8 October it sits in the block's literal pool and is loaded with one `ldr`, while a value of two halfwords or fewer stays a pair of moves (`929592ed`). Such a cell is as portable between processes as the moves were, since `low_base` is part of the translation store's key.

Two range limits matter to callers. `tbz` and `tbnz` carry a 14-bit offset, ±32 KB, far shorter than the ±1 MB of `b.cond`, so a large block can put a stub out of a `tbz`'s reach; a truncated `tbz` offset was a real bug found by audit on 18 August. And the scaled immediate forms of `ldr` and `str` cover only non-negative multiples of the access size, which is why the unscaled `ldur`/`stur` forms exist too, and why the ordered accesses are built from `ldapur` and `stlur`, their FEAT_LRCPC2 counterparts. `tests/unit/test_a64emit.c` does not compare encodings against expected words: it emits each sequence into a `MAP_JIT` page, runs it and checks the result, so an encoding bug cannot pass by agreeing with a wrong table.

When a block faults

A guest fault inside translated code arrives as a host signal with host registers. Before the guest can see it, the guest state is rebuilt: the pinned registers are read from the signal frame, `rsp` is converted back to a guest value (a push whose store faulted has already decremented it, so it is repaired by eight), elided return-address slots are written back so the interpreter can resume in the middle of a spliced frame, the xmm pins and lane caches are recovered from the frame's NEON state because the memory copy is stale, and the flags are rebuilt from the instruction's fault recipe. A faulting `ldp`/`stp` pair is re-run from its first instruction in the interpreter, so the guest sees the signal at the right one. Chapter 9 follows the signal from there to the guest's handler, and Chapter 7 covers the faults the JIT provokes deliberately: write-protected code pages, the commpage and misaligned ordered accesses.

October's changes each added something the rebuild must know: the offset of a lazy-rsp region at the faulting instruction (which `ocerz_jit_guest_gprs_at` also works out for a suspended thread), the restart of a push or pop run at its first instruction, the producer's record that a fault at a load inside an NZCV window or a fused `jcc`'s gap reads its flags from, and, in the Wine layout, which side of 12 GB `rsp`'s host pointer is on.

The translator can fault too. It reads guest code while the thread's JIT pages are writable, a successor's bytes for the flag look-ahead say, and an unreadable byte is a fault that the crash handler turns into a jump back into `translate` through `ocerz_jit_decode_recover`. Skipping `sigreturn` also skipped restoring the thread's write permission, so the next word `translate` stored was a `SIGBUS` inside the translator. It surfaced on 10 October when the look-ahead began to follow stub slots, and crashed 76 of 87 native tests; since `c9362f8e` the translator marks its write window in a thread-local variable and the handler reopens writes before it jumps back.

Lessons, in one table

Symptom	Cause	Rule now	Fixed
Every SQLite open in Steam's CEF failed	retargeted `jcc` skipped its stub's flag record	never retarget past a record, lane flush or batch check	2026-09-06
Every Wine window painted black	flags assumed dead at `ret`	a pure flag producer keeps flags live across `ret`	2026-09-05
CEF crashed on its first navigation	gap `lea` clobbered the compared value's temporary	gaps touch pinned registers only; emitted to scratch first	2026-09-06/07
CEF renderers asserted in libc++ `sort`	self-loop compare fused although the loop head read flags	no fusion when the loop head reads flags	2026-09-13
Cocoa view transforms came out singular	side exit chained past a lane flush	same retargeting rule	2026-09-08
SIGILL after 128 MB of code	out-of-range branch to the dispatch stub	range-check every long branch; veneers	2026-08-18
A game's hottest edges never chained	chains beyond 128 MB silently dropped	veneer pools every 64 MB	2026-09-22
36 million return misses a second	a lost return entry left the frame and wiped the rest	a miss stays in the body	2026-09-22
A leaf-call loop summed garbage	renamed pop after a call-out read a clobbered register	compare call-out sequence numbers	2026-09-27
After `add ; inc`, a `jb` read the wrong carry	a fused pair left its first instruction as the latest flag producer	a fused pair names its second	2026-10-07
A 32-bit `cmov` read stale flags	NZCV forwarded to a consumer that went to the interpreter	forward only to a translated consumer	2026-10-07
An aligned locked add added twice	out-of-line arms ran out after the atomic was emitted	past 32 arms, the slow call goes inline behind a branch	2026-10-07

Symptom	Cause	Rule now	Fixed
Callers kept entering a retired block	a chained `bl` was not recognised when unchaining	unchaining decodes `bl` too	2026-10-08
Retired blocks went on running; a rewritten return site was missed	unchaining did not follow chains through veneers	follow a veneer to the address it loads	2026-10-11
A double carried round a `dec` / `jnz` loop came out wrong	the `inc` / `dec` branch emitters skipped the fixed lanes' flush	flush on every arm; a self-loop branches straight to its loop entry	2026-10-13
A thread spinning in a loop never got its signal	signals were delivered only at system and bridged calls	deliver between steps; cut the loop's chained edges	2026-10-14

Each of those has a test that fails on the build before its fix (Chapter 16), and each optimization involved can be switched off individually, which is how most of them were found (Chapter 18).

CHAPTER 7

The JIT: floating point, SIMD, memory ordering and changing code

Chapter 6 showed how a block of x86 becomes arm64 that keeps every guest register in a host register and runs from one block body straight into the next. This chapter covers the places where the two architectures disagree about meaning rather than encoding: what a NaN looks like, how the x87's 80-bit register stack runs on 64-bit registers, what order memory operations become visible in, what happens when the guest rewrites code that has already been translated, and what to do when the code arena fills. It ends with the two mechanisms that make the JIT cheaper to use than to run: arm64 routines called in place of the system's string functions, and an on-disk cache that lets one process reuse another's translations.

26,593	1,277	897	1 GB
LINES IN SRC/JIT.C	LINES IN SRC/X87.C	LINES IN SRC/TCACHE.C	DEFAULT CODE ARENA

Exact floating point

Where x86 and arm64 disagree

For ordinary numbers the IEEE 754 operations on both architectures produce identical bits, and the rounding mode is the only thing that can change a result. The disagreement is entirely about NaNs, and it is visible to programs. When an SSE operation produces a NaN, x86 follows one fixed rule: if the first operand is a NaN the result is that NaN made quiet, otherwise if the second is a NaN the result is that one made quiet, and otherwise (an invalid operation such as 0/0 or infinity minus infinity) the result is the *default NaN*, whose sign bit is set. arm64 lets a signalling NaN win over a quiet one regardless of operand order, and its default NaN is positive. A program that formats, hashes or compares the bit patterns of its results sees the difference, and the test suite does too: every NaN test in `tests/guest` compares raw bit patterns against the same binary run natively under Rosetta.

The JIT therefore has to produce x86's NaN on every floating-point result, and on most processors it has to do that in software. The obvious method is a check after every instruction: compare the result with itself (a NaN is the only value unequal to itself), and on a NaN branch to an out-of-line *exact fixup arm*, emitted after the block body, that recomputes the result the x86 way. That is correct and it was the first implementation. It is also expensive: the check sits on the dependency chain of every floating-point loop, and in a packed loop it is several vector instructions on top of the few doing the work.

Deferring the check, then batching it

Two observations make most of those checks unnecessary. A NaN propagates: if an operation's input is a NaN its output is a NaN, so a check of the last result in a chain also catches a NaN created earlier in it. And a NaN is rare: real programs produce one in a tiny fraction of operations. The JIT first deferred each check to the instruction that consumes the value, then generalised that into **FP batches**. A batch is a run of floating-point work that is checked once, at its end, with the registers it overwrites checkpointed at its start. If the end check finds a NaN anywhere it should not be, the batch is *replayed*: the checkpoint is restored and the whole run is executed again exactly, with every result correct to the bit. The fast path pays for one check per batch instead of one per instruction, and the slow path pays a replay only when a NaN actually appears.

Early batches covered only maximal runs of arithmetic and register moves, and anything else ended them. A scalar loop such as nbody's pair loop split into five or six batches per iteration, each paying a checkpoint and a check; of its 107 host instructions per iteration, 42 were NaN bookkeeping. Since 2026-09-15 (`a16dc15b`) the scanner builds one batch per run of floating-point work. Members now include self-zeroing `xorps`, `unpckhpd` and `unpcklpd`, `movddup`, `ucomisd` and `comisd` together with the `jcc` that consumes their flags, and stores. A few rules keep that exact:

- A tainted register is checked right before it is stored, because a stored NaN escapes the batch.
- A compare is a free detector. `ucomisd` raises the unordered condition for a NaN in either operand, so the compare's unordered path replays the batch before the flags are read, and the fall-through path needs no check at all.
- The `jcc` after a compare carries its check into the side-exit stub, which runs only when the branch leaves the block.
- At the batch's end only registers still live on the path through the block are checked. A self-loop's back edge is live for the registers the next iteration reads before writing; registers dead in the loop but possibly live after it are checked once, on the loop's fall-through exit.

Stores that alias the batch's loads

A replay re-executes the batch's loads, so a store inside the batch that might overwrite memory an earlier load read used to end the batch: the replay would read the stored value instead of the original. nbody's pair loop updates both bodies' velocities in place and was three batches for that reason. Since `fb62cb88` the batch keeps an **undo log**: the memory pre-image of every such store is saved, and the replay arm writes the pre-images back, in reverse order, before re-executing. Where the pre-image lives decided whether this was worth doing at all. The first version kept it in the CPU structure and was slower than no undo log, because that loop is bound by its stores and four extra stores per iteration ate the gain. The pre-image now lives in a lane register reserved from the top of the v4-v15 pool, and when the guest itself loaded the same operand earlier, the read-modify-write shape, it costs one register move right after that load. The undo log needs addresses the JIT can form without a range check, so it is off under memory guards (the Wine layout described below) and in 32-bit code, where batches still split at such stores; `OCERZ_FPB_NOUNDO=1` switches it off everywhere.

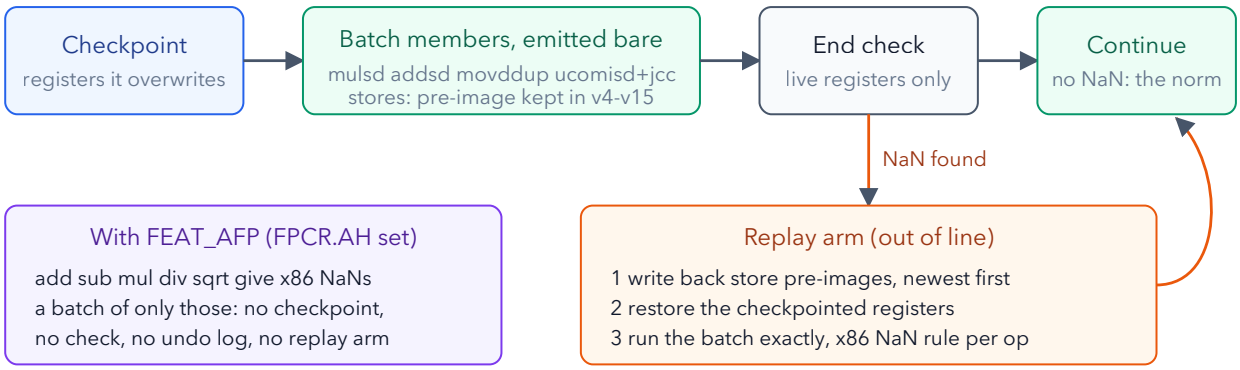


Figure 7.1. An FP batch. The fast path checks once, at the end, and only registers that are still live. A NaN sends the batch to a replay arm that restores memory and registers and recomputes it exactly. On a processor with FEAT_AFP most batches need none of this.

Wide operations and fused multiply-add

A 256-bit packed add, subtract, multiply, divide or square root used to check each 128-bit half separately, five instructions and an out-of-line fixup per half. Since `f051e8be` it takes one check for both halves (two `fcmeq`, an `and`, a `uminv` and a `cbz`), and a NaN sends the whole instruction to the interpreter from its untouched sources. fpvec's inner loop in the x86-64-v3 build went from 287 to 257 host instructions per iteration with that change. Fused multiply-add needs more care than the other operations, because its negated forms negate a NaN operand where x86 returns the operand as it came. VEX.128 packed FMA is a batch member, which made the FMA nbody loop a single batch; outside a batch every FMA keeps its own check.

Testing the slow path on a machine that never takes it

Replay arms run only when a NaN appears, so ordinary tests would leave them almost unexercised, and on a processor with FEAT_AFP most of them are never emitted at all. `tests/run_guest_tests.sh` therefore runs its twelve NaN tests (`fpb_loop_nan`, `fpb_undo_nan`, `fp_loop_nan`, `fpb_defer_nan`, `cvt_nan_batch`, `cvt_merge_nan`, `simd_misc_jit`, `avx_fp_jit`, `avx_cmpblend_jit`, `jit_sse`, `jit_sse2` and `nan_rules`) twice more after the main pass: once with `OCERZ_NO_AFP=1`, so the software checks are emitted, and once more with `OCERZ_FPB_FORCEREPLAY=1` as well, which makes every check take its replay arm whether or not a NaN appeared. A replay that is not exact then shows up as a wrong bit pattern in a golden comparison. `fpb_undo_nan` exists specifically for the undo log: six loop shapes, including aliasing through two base registers, an exact read-modify-write, partial overlap with two stores to one address, and a store whose register is rewritten afterwards.

FEAT_AFP: letting the processor do it

Armv8.7's FEAT_AFP adds a bit to the floating-point control register, FPCR.AH (alternate handling), that changes how arm64 arithmetic treats NaNs. With it set, add, subtract, multiply, divide and square root produce x86's NaN result themselves, as long as the x86 destination is the first arm64 source operand, which is how the JIT already emitted them. AArchX checks `sysctl hw.optional.arm.FEAT_AFP` once (`ocerz_afp` in `src/cpu.c`) and, where it is present, sets the bit on every guest thread in cache mode.

Those five operations then go out bare. A batch made only of them, moves, shuffles, compares and stores keeps its members' fast emission and drops the checkpoint, the checks, the undo log and the replay arms. Minimum and maximum were exact already, being a compare and a select.

The effect is largest in packed loops. In fpvec the end-of-batch check was three vector operations on top of the eight doing the work, and the four vector pipelines were the limit: fpvec went from 1.06x of Rosetta's time to 0.82x, and the x86-64-v3 build's from 0.96x to 0.58x (Apple M5, 2026-09-21). Two places keep the software checks. Fused multiply-add is not covered by the bit, for the negation reason above, so a batch that holds one keeps the whole apparatus. And native mode keeps the translated checks everywhere, because there the host's own arm64 frameworks run on guest threads and would need the bit cleared around every crossing: two FPCR writes cost about 14 ns, roughly what the crossing itself costs. That is why fpvec in native mode is still 1.05x while every other mode is below 0.85x. m32, the runtime for 32-bit Mac programs (Chapter 13), has the same problem: it calls host frameworks from guest threads all the time. AppKit's upgrade of an archived `NSTextView` faulted in host code with the bit set and loads natively without it, so since `281e193d` (2026-10-18) m32 clears the bit for every call into the host and sets it again when host code calls guest code (`ocerz_afp_host`, `ocerz_afp_guest` and `ocerz_afp_restore` in `src/cpu.c`). Those writes cost about 8.5 ns each on an M5, against 0.34 ns for a read, so later the same day `1ae3140a` made an i386 program run in native mode by default, with the translated checks and no FPCR writes, unless `-cache` or `OCERZ_MODE` asks otherwise. `OCERZ_NO_AFP=1` keeps the translated checks everywhere, which is also exactly what a processor without the bit gets.

> **HOW IT WAS FOUND**
>
> Writing `tests/dynamic/nan_contexts.c` for the FEAT_AFP change, which computes NaN results on the main thread, on a pthread, inside and after a signal handler, on libdispatch's threads, after an MXCSR write and across `fork`, turned up an older bug: the main thread of a dynamically linked program had never been given the guest's floating-point control state at all. It enters through the guest-call loop, and only the run loop applied that state.

Rounding modes

Until 2026-09-11 AArchX stored the guest's MXCSR but never applied its rounding-control field to the host, so every SSE result rounded to nearest whatever mode the program selected with `ldmxcsr` or `fesetround`. Both engines take their rounding from the host's FPCR (the JIT through arm64 instructions, the interpreter through C arithmetic), so setting FPCR fixes both at once. `ocerz_apply_mxcsr_round()` maps MXCSR.RC onto the host mode through a table rather than copying bits, because the two architectures number the directed modes differently: MXCSR 01 rounds toward minus infinity and FPCR 01 toward plus infinity. It runs wherever the guest's MXCSR changes (`ldmxcsr`, `fxrstor`, the context restore in `sigreturn`) and at run-loop entry, so a worker or a resuming thread starts in its guest's mode. Until 2026-10-18 the mapping went through the `fe*` constants and `fesetround`, which wrote FPCR every time, and every callback from native code into guest code paid for that write. Since `ad51f4b3` the function reads FPCR, builds the value it wants (with FPCR.AH where FEAT_AFP is in use) and writes it only when it differs, which a callback entering guest code in the mode it already has, nearly every one, never does. The dynamic test `fp_rounding` checks the direction of rounding in each mode. Conversions follow the same mode: `cvtps2dq` rounds with `frinti`, which obeys FPCR, and then shares `cvttps2dq`'s conversion, which gives x86's `0x80000000` for NaN and out-of-range lanes where arm64 would saturate or give zero. In native mode a bridged call writes FPCR only when the guest's rounding mode is not already the default.

The x87 unit

x87 is the older of x86's two floating-point units: eight 80-bit registers used as a stack, a control word that sets rounding and precision, a status word holding the condition codes and the stack's TOP, and a tag word saying which registers hold values. 64-bit compilers leave it to `long double`, but 32-bit Windows programs and games do their floating point on it, and Delphi and Free Pascal copy memory through it. Until October 2026 AArchX only interpreted it, and interpreted parts of it wrongly. Issue #6 asked for exact `fild`/`fistp qword`; the work it started rewrote the unit (`8f310eb8`), translated it (`6c6679f0`) and shipped in 0.6 (`7fc00672`, 2026-10-07), and commits over the following day took much of its remaining cost away.

What was wrong

A comparison against Rosetta found far more than the instruction the issue named. Most x87 code computed wrong values:

- `fadd st(0), st(i)` and the rest of the D8 register forms computed st(0) op st(0) (`fadd %st(1),%st` gave 20 for 10 + 3), and the DC forms wrote st(0) with the operands swapped where st(i) is the destination;
- `fcomi` and `fucomi(p)` compared st(0) with itself and always said equal, and `fcmovcc` copied st(0) onto itself;
- `fxam` was decoded as `fxch`, `fyl2xp1` as `fyl2x`, and `frstor` and `fnsave` as `fnstenv`; `ficom` and `ficomp` read their integer as a float; `fisttp`, `fbld`, `fbstp`, `fxtract` and `ffreep` did not decode;
- an out-of-range `fist` stored the low bits instead of the integer indefinite, `fprem` cleared the quotient bits, and `fxsave` stored raw doubles in physical order while `fxrstor` never restored TOP.

The unit now lives in `src/x87.c`, decoded as Intel's manual has it, and the file's explanation block is the specification that both engines follow.

Doubles with exact images

The eight data registers stay IEEE doubles in `OcerzCPU.fpr`, indexed by physical register, so ST(i) is `fpr[(ftop + i) & 7]`. A double keeps x87 arithmetic on the host's floating-point unit, and rosettax87_jit, which 32-bit games ran on before, makes the same choice. What a double cannot hold is a value loaded as more than a double. One is a 64-bit integer, where rounding is not a loss of precision but corrupted memory: Delphi's and Free Pascal's `Move()` copy eight bytes at a time as `fild qword`/`fistp qword`, and OMSI 2 died reading its own form resource. The other is an 80-bit value moved with `fld tbyte`/`fstp tbyte`, which is how compilers copy a `long double`.

Each register therefore also has an **exact 80-bit image** in `fpr_xm` and `fpr_xe`, which counts while its bit in `fpr_x_ok` is set *and* the double is still, bit for bit, the image rounded to double. FILD, FBLD, the constant loads, an m80 load and the FRSTOR and FXRSTOR images fill it; FLD ST(i), FST ST(i), FXCH and FCMOVcc carry it; every other write clears the bit. Integer stores, FBSTP, the 80-bit images, FXAM and the tag word read it, so an integer, a long double, a denormal or a NaN payload goes through loads, stores and context switches unchanged. The second condition protects the image from code outside the unit that writes a register and forgets the bit. The issue's own check now reports 0 of 9 quadwords corrupted under both engines.

The rest of the model follows x87 rather than SSE or arm64. The tag word is tracked, in the abridged form FXSAVE stores, so FXAM can answer "empty"; a push onto a full register or a read of an empty one is not a stack fault, and the value passes through, since a tag that went wrong elsewhere must never turn data into NaNs. Each operation runs with FPCR's rounding taken from the control word and FPSR saved around it, so x87 flags reach the status word and never MXCSR. Precision control 24, which Direct3D 9 sets, rounds results to a 24-bit significand without narrowing the exponent. NaNs follow the x87 rules: an invalid operation gives the negative default NaN, a single NaN operand comes back quieted, and of two NaNs a quiet one beats a signalling one and otherwise the larger significand wins, the positive one on a tie, as Rosetta does. FXSAVE and XSAVE store the eight 80-bit registers in stack order and FXRSTOR sets TOP, which is the format Wine copies into 32-bit thread contexts. Where Rosetta and the manual disagree the tests follow Rosetta, since it is what these programs ran on: FNSTENV leaves the exception masks alone, and the environment images' pad words are zero. Unmasked exceptions do not trap.

Translating it

Until `6c6679f0` every x87 instruction left translated code through the slow path: all pinned registers and lanes spilled, a call into the interpreter, everything reloaded. The common forms are now translated, with `src/x87.c` as their specification:

- FLD, FST and FSTP of m32, m64 and ST(i), and the constant loads;
- FADD, FSUB, FSUBR, FMUL, FDIV and FDIVR in every register, popping and memory form and on 16- and 32-bit integers; FSQRT, FRNDINT, FCHS, FABS and FXCH;
- FCOM, FUCOM and FICOM with their popping forms, FTST, FNSTSW, FCOMI, FUCOMI and their popping forms, and FCMOVcc;
- FILD, FIST, FISTP and FISTTP of 16, 32 and 64 bits;
- FNSTCW, FLDCW, FNINIT, FNCLEX, FWAIT, FFREE, FFREEP, FINCSTP and FDECSTP.

The transcendental, BCD, 80-bit memory and environment forms stay in the interpreter. A translated instruction leaves exactly what the interpreter would: values, TOP, tags, valid images and their bits, condition codes and exception flags. Before it writes anything it hands the interpreter what it cannot reproduce: a NaN anywhere, an infinite result, a product or quotient within 2^{53} of the subnormal range (an exact zero stays), and an integer store out of range or from a register whose exact image is valid. The status word stays exact without reading FPSR, whose writes cost 15 ns each. Invalid, divide-by-zero, overflow and underflow can only come from results the interpreter takes, and inexact is decided out of line, only while its flag is still clear, with error-free transformations: the result minus each operand of a sum, a fused multiply-subtract for a product, a quotient or a square root. Precision control 24 is one test per operation and an out-of-line tail that rounds the double's significand to 24 bits in integer arithmetic.

Consecutive translated x87 instructions form a **run** with one guard at its head, round to nearest in the control word and in MXCSR (the host's FPCR follows MXCSR), emitted only when something in the run rounds; FLDCW and FNINIT end a run. Every way out of a run, the guard's or an instruction's, enters one call, `ocerz_jit_exec_run_at`, which interprets from that instruction to the end of the run, so a fallback costs one call per run rather than one per instruction. An `fcmovcc` right after `fcomi(p)` branches on that compare's own NZCV. A `fild qword` followed directly by `fistp qword`, Delphi's eight-byte courier, stores the integer rebuilt from the image the FILD just made, its significand shifted right by 16,446 minus the exponent and negated for a negative sign, so the copy is exact without the interpreter (`6a3c32f3`).

32-bit blocks are where x87 lives, and they took every x87 instruction to the interpreter until `9f0b7087` let them ask `x87_inline_ok` too. The same morning, before 0.6, 32-bit blocks had also learned fs- and gs-relative operands, push and pop of memory, the locked and exchanging operations, the bit tests, and the fused compare-and-branch and condition forwarding of 64-bit code, so much more of a 32-bit Windows loop could stay in translated code.

Taking the state off the dependency chain

Translation alone did not make every loop fast. winbench's i686 double kernel, a loop of `fxch`, `fmul`, `fld st(i)`, `faddp` and `fcomi`, was entirely translated and took 2,160 ms under Wine, against 13,050 ms under Rosetta and 483 ms under Rosetta with rosettax87_jit. Removing parts of the emitted code with temporary switches showed where the time went: removing the result checks changed nothing, and removing the store of the control word halved it. Every instruction loaded the 8-byte word holding fcw, fsw, the tag word, TOP and the image bits that the previous instruction had just stored, and took ST(i)'s address from the TOP in it, so the tag and image bits each write set lay on the path to the next instruction's addresses, one serial chain carried from run to run.

Since `2295ec7f` (2026-10-07) a run loads that word into a register, `X87S`, once, and every instruction that changes it still stores it, so every exit sees exact state. The image bits moved into byte 6 of the word, so setting one is a register operation rather than a halfword read-modify-write. TOP inside a run is the run's first TOP plus the pushes and pops counted at translation, read from a halfword the previous run left (`jit_x87_top0`) and checked against `X87S` by a predicted branch, so no address waits for `X87S`. The run also tracks what it knows of each register's tag and image bit and of C1, and emits no update that would change nothing; a register whose image is known invalid is copied and exchanged without its image bytes. The double kernel went from 2,160 to 1,066 ms, and the structs kernel, float arithmetic in 32-bit code, from 867 to 509 ms (Rosetta 2,041, Rosetta with rosettax87_jit 136).

Registers in lanes on a known TOP

A chain of dependent x87 adds still cost about 12 cycles an add (3.50 ns), independent adds about 4 (1.14 ns): every result went to `fpr[]` and the next instruction loaded it back, paying a store-to-load forward on every hop. Keeping ST(i) in vector registers inside a run, written through so every exit stays exact, had been tried with the first translation and measured 6% slower, because runs are short and the moves added instructions. `d1679446` (2026-10-08) keeps the values in registers across runs instead:

- A run is translated for the TOP its block was entered with plus the pushes and pops before it, checked once at the run's open, so every register is a constant physical slot. A block entered with a different TOP leaves through the run's slow path, is retired, and is translated again without the known TOP; that translation counts as a learned one for the translation cache (below).
- A block with translated x87 and no SSE or MMX operand takes eight lane registers, v8-v15, whose low halves C calls preserve, one per physical register. A read uses the lane, loading it the first time; every value stored to `fpr[]` goes to its lane too, so memory stays exact at every exit and the next instruction reads a register instead of waiting on the store. FXCH swaps two lanes' roles, a run's slow path reloads the lanes before rejoining, and MMX, FXRSTOR and x87 outside a run drop them.
- What a run knows of tags and image bits is indexed by physical register and carried into the next run, where with a known slot every update is a constant mask; `X87S` is stored only when something may have written it since memory last matched.

Dependent adds went from 3.50 to 1.62 ns and independent ones from 1.14 to 1.00. The change also exposed an old chaining bug, described below under "Callers that still hold the old code": a block that retires itself because it was entered with the wrong TOP kept being entered by callers that had a direct call chained into it.

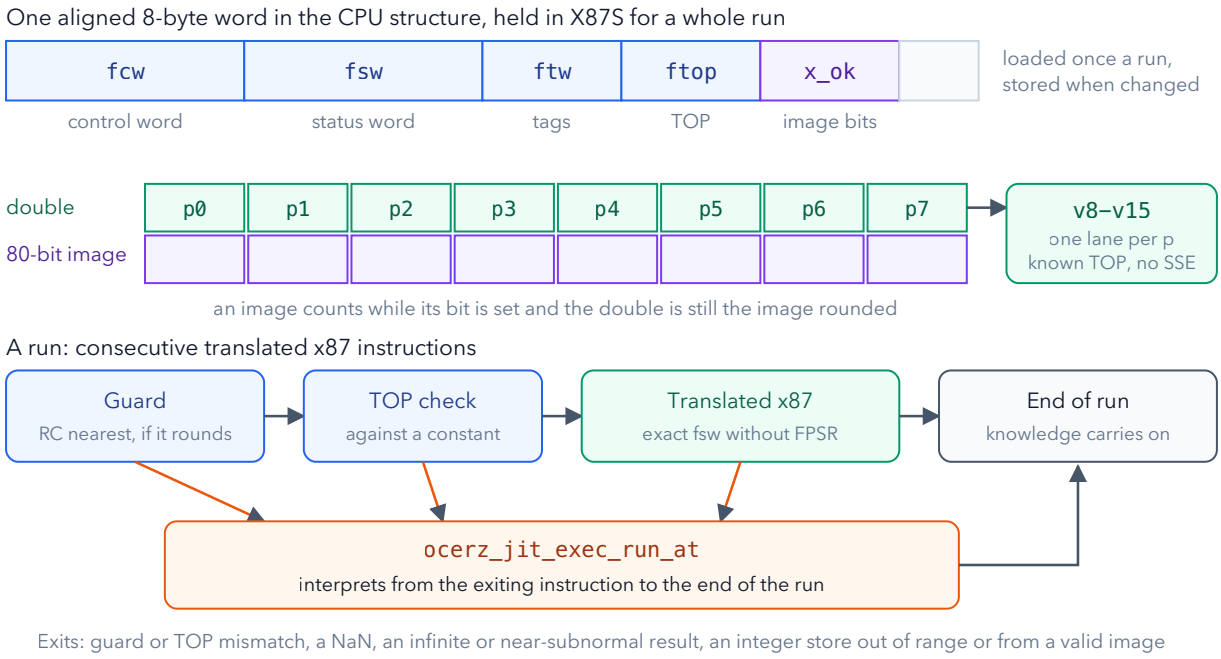


Figure 7.2. The x87 state a translated run works on. The control, status and tag words, TOP and the image bits share one 8-byte word that a run keeps in a register; with a known TOP each physical register also lives in a lane of its own. Anything a run cannot reproduce exactly leaves for one interpreter call that finishes the run.

Integer stores and the rounding control

Compilers write a truncating cast as `fldcw [esp+4]; fistp [esp]; fldcw [esp+6]`. The store's run opens with the rounding control set to chop, and an integer store used to count as rounding, so the run's guard for round to nearest failed and the run was interpreted every time: winbench32's structs kernel spent 58% of its time in the interpreter on those two instructions. Since `969f96b0` (2026-10-08) an integer store does not ask for the guard. In a run without it, the store picks `fcvtns`, `fcvtms`, `fcvtps` or `fcvtzs` from the rounding control in `X87S` as it converts, two `tbnz` and one conversion on any mode, and the kernel went from 506 to 239 ms, all of it translated. The test written alongside it, `tests/dynamic/fist_rc`, found a bug in both engines: x87 sets C1 after an integer store that was inexact and rounded away from zero, Rosetta does the same, and AArchX had left C1 as it found it. 54 of the test's 241 lines differed from Rosetta, all in C1, until `b3e701b9`.

How it is checked, and what it gained

Two harnesses hold the translation to the interpreter and the interpreter to Rosetta. `tests/diff32.c`, the i386 differential, gained 20,000 random x87 sequences, as many as its integer ones, whose values include NaNs, infinities, zeros, double denormals and FILD integers above 2^{53} at stack depths 1 to 8, plus hand-written cases for the quadword courier, FISTTP, precision control 24, the compares, TOP wrapping and the inexact flag. It compares each register's double, its image and image bit, the tags, TOP, fcw and fsw, and its self-test breaks each of those once and must catch it. It passed 40,044 cases in each layout in `9f0b7087` (44 hand-written, 20,000 random integer and 20,000 x87 sequences), and twice again on a MacBook Air

(M5, macOS 27) on 2026-10-16. The dynamic tests `x87_arith`, `x87_compare`, `x87_int`, `x87_trans` and `x87_state` compare 1,759 lines of Rosetta's output under both engines and in both layouts; `x87_top_entry` calls one x87 loop at four stack depths, runs `fucomip` then `fcmovu` with NaNs among the operands and `fninit` inside a block; and `fist_rc` runs every integer store under every rounding control.

x87 kernel	before	after	Rosetta	with rosettax87_jit
xbench x87, 10,000 rounds, interpreted to translated (`6c6679f0`, M2 Max)	1.147 s	**0.161 s**	0.731 s	-
winbench i686 double, under Wine (`2295ec7f`)	2,160 ms	1,066 ms	13,050 ms	483 ms
winbench32 structs, under Wine (`969f96b0`)	506 ms	239 ms	2,015 ms	125 ms
dependent x87 add (`d1679446`)	3.50 ns	1.62 ns	-	-

Translated x87 is now several times faster than Rosetta's own, but at those commits Rosetta with rosettax87_jit was still about twice as fast on both winbench kernels. winbench32 was not run after the lanes went in, because the test bottle and Rosetta were both gone after a macOS 27 update; diff32's copy of the double kernel went from 25.5 to 20.0 ns an iteration with them. `OCERZ_NO_JIT_X87=1` interprets every x87 instruction again, and `OCERZ_X87_CHECK=1` traps wherever `X87S` and memory, or the counted TOP and the real one, disagree.

SIMD coverage

SSE

Most of SSE maps almost one to one onto NEON, and the interesting cases are where it does not. The explanation block at the top of `src/jit.c` lists the translations: widening multiplies followed by a narrowing unzip for the high-half multiplies and `pmaddubsw`; saturating narrows for the packs; `uabd` followed by three pairwise widening adds for `psadbw`; `addp` or an unzip pair for the horizontal operations; `ext` against a zero register for the byte shifts and `palignr`; lane inserts for the immediate blends and `pshuflw`/`pshufhw`. A shift by an xmm or m128 count clamps the 64-bit count to 64 and shifts by a duplicated register, which reproduces x86's saturation: zero for logical shifts and the sign for arithmetic ones. AES rounds use `aese` or `aesd` against a zero key, then `aesmc` or `aesimc`, then the round key, because arm64 adds the key before the substitution and x86 after it; `aeskeygenassist` picks its words out of a zero-key `aese` with a table lookup. `crc32` is arm64's `crc32c` of the same width and `pclmulqdq` is `pmull`.

Many of these forms were emitted only from 2026-09-27 (`53df4711`), when sampling Steam's web helper under Wine showed it interpreting `pinsrw` from memory for 15% of its samples. The reason was general: an emitter that had only the plain memory forms handed every memory operand to the interpreter in the Wine layout, where guest and host addresses differ. Every emitter that takes a memory operand now reaches it through `emit_mem_load_any` or `emit_sse_mem_addr`, which fall back to the guarded address path rather than the interpreter. With that and the new forms the web helper's main process went from 47-67% of its samples in translated code to 78-81%. `tests/guest/simd_pack_jit.c` covers every form with register, memory and aliased sources over random and edge rows, the four rounding modes and eighteen shift counts, against hashes produced under Rosetta, and the dynamic suite builds the same source at `0x200000000` (`dsimd_low`) so every operand sits in the Wine layout.

The forms added since follow the same pattern: a profile shows an instruction in the interpreter, and the fix is a direct mapping, tested against Rosetta in both layouts.

- **Packed float and double conversions** (`61e31dde`, 2026-10-04) are `fcvtl` and `fcvtn`, which quiet a signalling NaN and keep its payload as x86 does and round by the FPCR mode that tracks MXCSR; `fcvtn` also clears the upper half, as x86 does. The single-to-double one alone had been about 0.4% of R.E.P.O.'s main thread, interpreted.
- **`ptest`** (`ff8ce8b9`, 2026-10-09), how SSE4.1 code asks whether a vector is zero, is an AND and a BIC of the two registers and a `umaxv` of each. It sets ZF and CF and clears OF, SF, AF and PF, as Intel's manual says; Rosetta leaves those four as they were, and agrees on everything a branch reads. A `jcc`, `setcc` or `cmovcc` on ZF or CF that follows a `ptest` of two pinned registers, neither written in between, tests the registers itself, and the `ptest` then emits nothing. `deno --version` spent 98% of its 3.4 s in one loop of them, a 16-byte compare slid along a buffer a byte at a time; it now takes 0.25 s (Rosetta 0.19). The 256-bit `vptest` stays a call-out.
- **`shufps`** built every result lane by lane in a scratch register and copied it back, five instructions. With a register and itself (`5c4b1e3d`, 2026-10-08) a broadcast is now one `dup`, a shuffle that changes a single lane one `ins` in place, and the identity nothing; with two registers (`a8d09d44`) the selections that 4x4 transposes and pair packing are made of are one `zip1`, `zip2`, `uzp1`, `uzp2`, `ins` or `ext`. winbench64's structs kernel, whose loop does `shufps xmm5, xmm5, 0xe5`, went from 29.0 to 26.3 ms with the single-lane form (Rosetta 12).
- **A compare against memory** (`951b6977`, 2026-10-08). A `jcc`, `setcc` or `cmovcc` whose flags come from a `comis` or `ucomis` a few instructions earlier already recomputed the comparison itself, but only when both operands were pinned registers. The common case in games is `ucomiss xmm0, [rip+limit]; ...; ja`, where the compare did a read-modify-write of the flags in memory, about 18 instructions, and the branch read them back. Now the compare still performs its load, so a fault stays its own, and keeps the value in a CPU slot (`jit_fcmp_mem`); the fused branch compares the register against that copy, never against guest memory, which a store in between, or another thread, may have changed. winbench64's struct-of-floats loop went from 33.1 to 29.3 ms (Rosetta 12).

AVX2, FMA and BMI

Rosetta on macOS 15 and later runs x86-64-v3 code, AVX2, FMA, BMI1 and BMI2, F16C, LZCNT, MOVBE and XSAVE, even though CPUID advertises none of them, and real programs depend on it: Go turns on its AVX2 paths under Rosetta without checking CPUID. AArchX decoded and interpreted VEX from 2026-09-08 and translated it from 2026-09-13. Before translation, AVX2 and FMA loops ran up to a hundred times slower than under Rosetta; after it, most beat it (Chapter 17 has the kernel table).

The JIT translates VEX moves, integer, bitwise and compare operations and broadcasts; `vpmovmskb`, most sign and zero extensions and shifts by an immediate; `vzeroupper`, FMA, 256-bit packed arithmetic and the variable blends; `vshufps`, `vshufpd`, `vinsertps` and `vmovddup`; the VEX.128 forms of every SSE instruction it translates; and `rorx`, `shlx`, `shrx`, `sarx`, `mulx` and `andn`, plus `blsr`, `blsmsk`, `blsi` and `bzhi` when their flags are dead. Still interpreted: 256-bit byte and integer shuffles, unpacks, permutes and lane inserts, `vptest`, the immediate blends, mask-producing compares, gathers, and `bextr`, `pdep` and `pext`. A `vcmpss` or `vcmpsd` whose mask feeds a `vblendv` is fused into one compare-and-select.

Two details matter for speed. A `vzeroupper` in a block without 256-bit instructions tests a per-thread flag and skips its sixteen stores when the upper halves are already zero. And a block with VEX.128 instructions reserves a zero lane register and clears an upper half with one 16-byte store instead of two, halving the store traffic of every VEX.128 loop (`OCERZ_NO_ZEROREG=1` turns that off).

MMX

Until 2026-10-04 every MMX instruction ran through the slow path: one interpreter call per instruction with all guest state spilled around it. Old code still uses MMX, and Unity's video decoder in R.E.P.O. is written in it, which cost about 4% of the game process in the interpreter at its main menu. `emit_mmx` (`b7449a73`) now translates the moves to and from memory and general registers; the lanewise adds, subtracts, saturating forms, multiplies, logic and compares; the unpacks and packs; `psadbw`, the averages and the minimum and maximum; shifts by an immediate; and `pshufw`, `pextrw` and `pinsrw`.

The eight MMX registers stay in the CPU structure's `mmx` array rather than in host registers. Each instruction loads what it reads into a scratch vector register with a 64-bit load, which clears the upper half; runs the 128-bit form of the operation; and stores the low 64 bits back. For lanewise operations that is all it takes. The others are arranged so their answer lands in the low half: a high unpack is a low zip followed by taking the upper half, the packs join both operands into one register before narrowing, `pmulhw` and `pmulhuw` keep the odd halves of a widening multiply, and `psadbw` reuses the SSE sequence. Each translated MMX instruction leaves the x87 tag word full and TOP at zero, as the interpreter does, because MMX and x87 share state architecturally. `pmaddwd`, `pmovmskb`, `emms`, the SSSE3 forms, conversions to and from SSE and shifts by a register count still go to the interpreter. A loop of the decoder's moves, adds, multiplies and packs runs in 0.142 s instead of 1.395 s; Rosetta takes 0.053 s, which is still 2.7 times faster because every instruction here goes through memory. `tests/guest/mmx_jit.c` covers every translated form on saturating and edge values, and the dynamic suite runs it in the Wine layout too (`dmmx_low`). `OCERZ_NO_JIT_MMX=1` interprets MMX again.

Until 2026-10-10 that applied to 64-bit code only: both places that dispatch to `emit_mmx` answered no for an instruction in a 32-bit block. Bink's video decoder and Feral's IndirectX use MMX in i386 code, and in Batman: Arkham Asylum's render thread, running through m32, `movq` to and from MMX registers was 14% of the instructions still interpreted. Jfishin's `f8fa8aaa` translates MMX in 32-bit blocks as well; the memory operands already came from `emit_sse_mem_addr`, which wraps 32-bit addresses, so nothing else had to change. `tests/m32/mmxjit.c` is the guest test's coverage as an i386 program, checked against the x86-64 build's output under Rosetta.

Lane caches

The xmm registers are pinned in V16-V31 (Chapter 6), but a scalar SSE result has to land in lane 0 of its guest register while lane 1 keeps its old value. Done naively, every scalar operation ends with an `ins` into the pinned register and the next one reads it back out, and that insert sits on the dependency chain of every scalar loop. The **lane cache** keeps scalar results in their own registers instead. Since 2026-09-14 (`b9e71935`) it has twelve lanes, v4-v15; the block prologue saves d8-d15, which are callee-saved for the C code that calls into a block. A self-looping block keeps its lane values live around the back edge: it enters with every fixed lane mapped, a register whose value sits in another register's lane gets a move between lanes rather than a write-back, and only a register that lost its lane is reloaded. A mid-block `jcc` does not flush inline, because its side stub writes back the lanes that were dirty there.

The upper halves of the ymm registers get the same treatment (`a79fbd2b`). They used to live only in `cpu->ymmh`, so a `vpaddd ymm` cost two loads and a store on top of its two adds. A self-loop block now keeps the upper halves it touches in the lanes the scalar cache leaves free, loaded once at the loop head and written back on every way out: the fall-through, the side-exit stubs, the out-of-line slow arms and the FP batch replay arms. In the x86-64-v3 build that took memcpy from 1.16x of Rosetta's time to 0.93x.

A value that lives only in a lane is invisible to a fault handler that rebuilds the guest's xmm state from the pinned registers, so at first both caches were switched off wherever a guest fault could be caught and resumed, which includes the whole Wine layout. Since `8ad9296e` every block records, at each instruction start and at each re-emitted instruction in a replay arm, which registers sit in which lane, in what width, and which lanes are dirty; identical consecutive states share one record. Fault recovery looks up the record for the faulting host program counter and folds the dirty lanes and cached upper halves into the CPU state. That is exact for any fault inside an instruction because every emitter reads all of its memory before it writes a lane, a pinned register or a cache lane; the one emitter that did not, the 256-bit packed arithmetic with a memory source, was changed to read both halves first, low half first, so a fault also reports the address x86 would. `tests/guest/fault_lane`, a scalar SSE2 loop, its AVX2 twin and a 256-bit accumulation loop running into a protected page, caught 25 wrong lines in the build before the fix.

The rules at a loop's edges

The fixed lanes of a self-looping block depend on two rules that October's bugs made explicit. The first is about the back edge. Each register the loop keeps has a home lane, and the back edge moves every register back into its home before branching to the top. That is a parallel move, and its order matters: a register must wait while another pending register still lives in its home lane, since moving it there would overwrite that value. The test in `l0_fixed_restore` was inverted, holding back a register whose current lane was another's home instead (`2d5d5821`, 2026-10-08). clang unrolls `v += step; if (v > 100.0f) { v = 0; wraps++; }` into steps that end in `movaps xmm3, xmm2` and `xorps xmm2, xmm2` before a conditional back edge, so xmm3 lived in xmm2's lane, and the restore wrote xmm2's zero into that lane before taking xmm3 out of it. The loop ended with 0 wraps and `v` = 0, where Rosetta and the interpreter give 738 and 0.740. It needed the lane cache, its fixed lanes and a superblock together; switching off either of the last two hid it.

The second rule is about who flushes. The translator leaves out the lane flush before a block's last two instructions and expects the closing branch's emitter to restore the lanes on the way back and flush them on the way out, as `emit_jcc`, `emit_cmp_test_jcc` and `emit_jmp` do. The two emitters that fuse an `inc` or `dec` with its branch did neither, so a loop ending in `dec` and `jnz` left with its lanes unflushed on both arms, and its taken arm re-entered the body, where the lane setup ran again over what it found. A double summed and multiplied over 2,048 elements came out `0x4151b0a708587be7` where the interpreter, and the same loop ending in `sub` or `cmp`, give `0x41f544f8960ddec8`; it had been wrong since the fixed lanes came in on 2026-08-25. Since `2cf48a7e` (2026-10-13) both emitters flush, and a self-loop whose head reads none of the flags the `inc` or `dec` leaves branches straight back to its loop entry, as `emit_jcc` does, rather than taking the chained edge to its body entry and running the lane setup, and in the Wine layout the hoist's checks, on every trip. That made the loop above 1.99 ns an element against 2.64 through the chained edge (M2 Max). `tests/dynamic/incdec_loop.c` covers both rules in both layouts; `OCERZ_NO_INCDEC_LOOP=1` takes the chained edge always.

x87 code competes for the same registers. A block with translated x87 instructions and no SSE or MMX operand takes eight of the twelve lanes, v8-v15, for the x87 registers (above). 32-bit blocks have no fixed lanes at all.

Memory ordering

The gap between x86-TSO and arm64

x86 promises total store order (TSO). Each core's stores become visible to other cores in program order, loads are not reordered with other loads, and stores are not reordered with earlier loads. The only reordering x86 allows is that a store may become visible after a later load from a different address. arm64 promises far less: without explicit ordering, any two accesses to different addresses may be observed in either order by another core. Multithreaded x86 code is written against TSO, often without a single fence, and translated into plain arm64 loads and stores it breaks in the classic message-passing pattern: one thread writes data and then a flag, another reads the flag and then the data, and on arm64 the reader can see the new flag with the old data.

Plain until a second observer appears

A core always sees its own memory operations in program order, so a program with one thread cannot observe the difference. AArchX therefore starts every process in a **plain** memory model, translating loads and stores as ordinary arm64 accesses, and switches it to the **ordered** model the first time a second observer can exist. The switch is `ocerz_jit_require_ordered()`, and it is one-way: once a process is ordered it stays ordered. The syscall layer calls it from every place a second observer can come from:

- creating a thread (`bsdthread_create`) or registering the host workqueue bridge, whose workers run guest code;
- `fork` and the spawn paths, because parent and child may share memory;
- a writable shared mapping from `mmap` or `mprotect` (a read-only `MAP_SHARED` mapping no longer counts), and System V `shmat`;
- aliasing host memory into the guest, such as IOKit and IOSurface buffers that the kernel or a service also writes;
- in native mode, a host thread created by a framework calling into guest code for the first time.

The aliasing case was too broad at first. ocerz moves a mapping that a Mach VM reply placed outside the guest arena into the arena, and that used to switch the process to ordered memory in case the mapping was shared with another process. Most such mappings are not: the system's logging library maps a private, purgeable 256 KB buffer through `mach_vm_map`, and a read-only copy-on-write table, in nearly every program that logs, so single-threaded programs ran ordered. Since `618d540f` (2026-10-09) the move asks the kernel about every region in the range (`host_range_shared`): private regions, regions aliased only within the task, empty regions, holes, and copy-on-write regions without write permission have no other observer and leave the process plain; anything else, or a range that cannot be inspected, still makes it ordered. A program that only links libsqlite3 ran its inserts and a query in 0.417 s instead of 0.578 s (Rosetta 0.308). `OCERZ_NO_REMAP_PLAIN=1` restores the old rule, and `OCERZ_ORDERLOG=1` prints what first made a process ordered, with a backtrace. A private region shared later through a memory entry sent to another process does not make the process ordered, because ocerz does not watch `mach_make_memory_entry`.

Blocks translated under the plain model are not reused once the process is ordered: the switch invalidates every existing translation, and the memory model is part of a block's key in the translation cache.

Ordered mode: acquire and release, scalars only

In ordered mode a scalar load becomes an acquire load and a scalar store a release store, using the LRCPC2 forms `LDAPUR` and `STLUR`, which take an unscaled immediate offset and so fit most x86 addressing directly. That is enough for TSO and no more than it needs. Release stores are ordered after every earlier access and acquire loads before every later one, which gives load-load, load-store and store-store ordering; the RCpc acquire (the `LDAP` family) is allowed to pass an earlier release store to a different address, and that is precisely the store-load reordering TSO permits. No `dmb` barrier is needed on the ordinary path.

Three classes of access are left plain even in ordered mode, each because ordering them costs much more than it buys:

- **Stack accesses through `rsp`.** `rsp`-relative loads and stores stay plain, on the heuristic, stated in the commit that introduced ordered mode (`2f9087c7`, 2026-08-17), that a thread's stack is private to it; this also keeps `push`, `pop`, `call`, `ret` and the return-stack protocol identical across the switch to ordered mode. `OCERZ_TSO_STRICT=1` orders them too. The Wine layout was the exception until `ed2fd0cf` (2026-10-04): there every stack access went through the address translation and then took the ordered load or store. It is plain after the translation now, as everywhere else. The esp-relative operands of 32-bit code followed on 2026-10-08 (`e7de713b`): push and pop in that code were already plain, but its esp-relative operands took the acquire load and release store, and a WoW64 loop of virtual calls into small frames, which spent most of its time on its stack slots, went from 459 to 163 ms (Rosetta 158). `OCERZ_NO_M32_STACK_PLAIN=1` orders those again.
- **Vector accesses.** Programs do not synchronise through 16-byte accesses (x86 does not even make them atomic), and ordering them cost memcpy 3.3x and fpvec 3.0x of Rosetta's time on an M2 Max on 2026-09-05. Leaving them plain took those to 1.35x and 0.99x. What that gives up is a vector load followed by a scalar re-read of a sequence-lock counter observing newer data than the counter covers, and two vector stores becoming visible out of order. FEX ships the same default, and `OCERZ_TSO_VECTOR=1` orders them.
- **The system's own string and memory routines.** The exported string and memory routines of the platform library are handed buffers at any alignment, and an ordered access that straddles a 16-byte boundary takes a barrier (below). Translated with ordered accesses they made the dynamically linked memcpy kernel 4.6 times slower than Rosetta; translated plain it reached 0.98x. `ocerz_dyldapi_memfn` answers whether an address lies inside one of them, and `OCERZ_NO_MEMFN_PLAIN=1` orders them.

Loads that order themselves

A fourth kind of plain access gives up nothing at all. In ordered memory every scalar load is an acquire, so that no later access passes it, as x86 requires. But arm64 already orders a load before any later load whose address it supplies: the second load cannot be issued before the first has produced its address. Since `9b1d4ec5` (2026-10-09) `dep_plain_mark` lets a `mov`, `movzx`, `movsx` or `movsxd` load in 64-bit code be a plain `ldr` when three things hold:

- its next memory access is another such load;

- that load is addressed through the register this one wrote, as its base or its index, and not off `rsp` (stack accesses are plain already, so they would order nothing that follows);
- between the two there is only register arithmetic that leaves the register alone, at most eight instructions, with no store, branch or other memory access.

The next load is itself an acquire, or plain by the same rule, so nothing later passes either, and the last load of every chain stays an acquire. `p = next[p]`, `obj->a->b` and a virtual call's load of its function pointer are such chains. The decision is keyed to the operand of the one instruction it was made for, so a load fused into another instruction keeps its acquire. Ordered memory is what every program with a second thread runs in, games included, and the change took xbench's `chase` kernel under ordered memory from 1.22 to 1.05 times Rosetta's time, with every other kernel unchanged within noise. `tests/dynamic/dep_order.c` is a message-passing litmus test: a writer fills a fresh node, stores data and publishes the node, and the reader's loads through the node must never see data older than the node. The M5 did not reorder these loads even with every chained load made plain, so the test also checks the decision itself through `OCERZ_DEP_PLAIN_LOG=1`: only the first of the reader's three loads may be plain. `OCERZ_NO_DEP_PLAIN=1` turns the rule off.

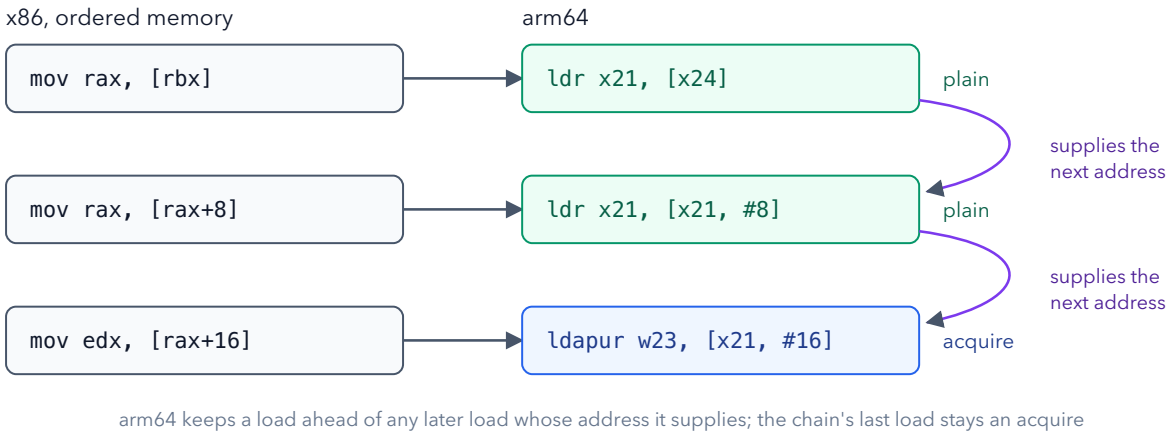


Figure 7.3. A dependent load chain in ordered memory (rax is x21, rdx x23 and rbx x24 under full pinning; guards left out). Only the last load needs the acquire form: the hardware already keeps each earlier load ahead of the one it addresses.

The narrow case: movd and movq

The 8- and 4-byte SSE forms are vector accesses too, and they are plain in ordered mode, but compilers do move pointers and counters through xmm registers. A plain `movq` store after an ordered scalar store can become visible first, and a plain `movq` load can be satisfied after the scalar load that follows it, so a reader can see a published counter newer than the data written before it. `tests/dynamic/tso_narrow.c` catches that hundreds to over a thousand times in three million handoffs, where x86 never shows it. `OCERZ_TSO_NARROW=1` (`975c18fb`, 2026-10-04) orders `movd` and `movq` with two instructions and no barrier each: a store is an `fmov` to a general register and an `stlur`, and a load is a plain load followed by a one-byte acquire load of the same address. That second trick is how any ordered vector load is done: same-address reads are coherent, so the acquire copy sees a value at least as new as the vector, and everything after it is ordered behind it, which is TSO's load ordering without the `dmb ishld` that waited for every outstanding miss (40-60 ns per access on a 4 MB working set). `OCERZ_TSO_NARROW=2` orders every vector access of 8 bytes or fewer. Neither is the default: the first took R.E.P.O.'s main menu from 65 frames a second to 44, and no program is known to depend on it. The test runs with `OCERZ_TSO_NARROW=1`, in the ordinary and the Wine layout.

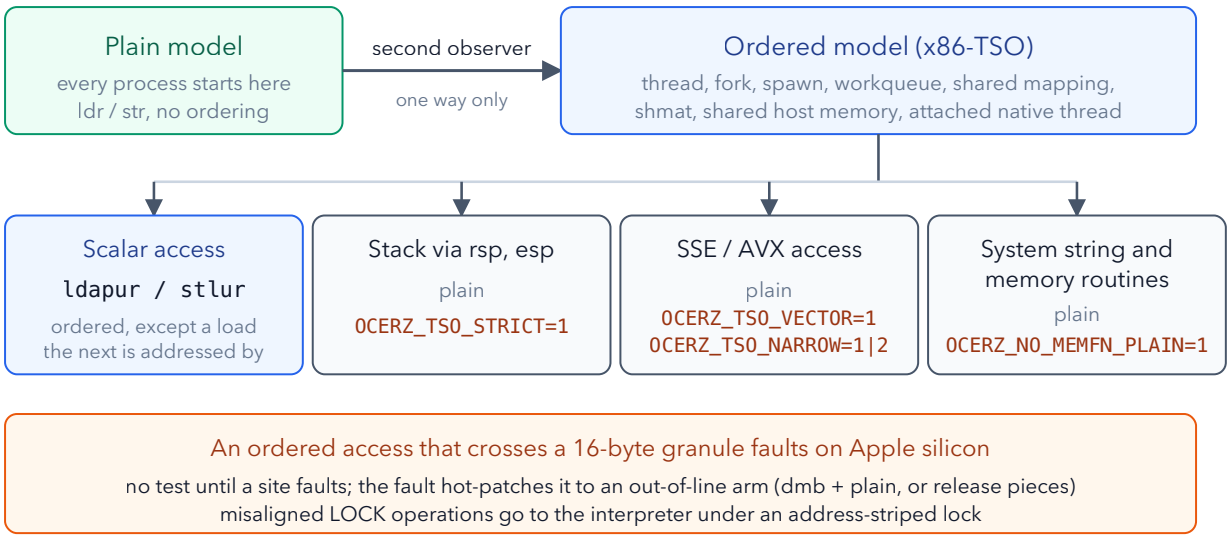


Figure 7.4. The memory model a translation uses. Every process starts plain and switches to the ordered model, for good, when a second observer can exist. Within the ordered model only scalar accesses are ordered by default, less the loads whose only consumer is the next load's address; each plain class has a switch that orders it.

Fences

Until 2026-10-09 the fences went to the interpreter, which spills every register to run one instruction; Apple's x86 libraries, which cache mode runs from the shared cache, use them in hot paths. `ac4424b9` translated all three as one `dmb ish`, as the interpreter's fence is, and `919f1f44` the same day gave each the barrier its ordering needs. `mfence` stays a full `dmb ish`, because a store followed by a load is the one order translated code does not otherwise keep. `lfence` orders earlier loads before everything after it, which is `dmb ishld`, and `sfence` earlier stores before later stores, which is `dmb ishst`. An `lfence` next to `rdtsc` or `rdtscp` is there to keep the counter read from moving past the instructions around it, and the translated read does not move anyway (it uses the commpage's non-speculative timer, or an `isb` goes first), so that `lfence` is dropped. The x86 `mach_absolute_time` is `lfence`, `rdtsc`, `lfence`, SQLite calls it around every statement, and a call went from 15.7 to 12.5 ns (Rosetta 8.1). `OCERZ_NO_LIGHT_FENCE=1` makes every fence a `dmb ish` again.

Granule crossings and the alignment hotpatch

Ordinary arm64 loads and stores accept any alignment, but acquire and release accesses do not. On Apple silicon an acquire or release access faults only when it crosses a 16-byte granule, which was measured rather than assumed, so wherever an ordered access is tested at all, the test is exactly that and nothing stricter: testing natural alignment instead sent three quarters of memcpy's unaligned tail accesses down the slow arm. Most ordered accesses carry no test. An ordered access is a bare `ldapur` or `stlur`, and the first time one crosses a granule, the fault handler hot-patches the site: a branch replaces the access and leads to a stub that does the access a safe way. A store's stub uses `dmb ish` followed by a plain store when the block also does ordered loads, because the drain is cheap then (memcpy 0.31 s against 1.1 s with release pieces), and release-ordered pieces otherwise, which store-only loops prefer (memset 0.10 s against 0.5-1.0 s).

Identity mode always worked that way. The Wine layout's guarded path did not: until `d46f40d8` (2026-10-04) it tested every ordered load and store for a granule crossing first, four instructions in front of every access it translated. It now relies on the patch as identity mode does. Only an instruction whose fault could not be patched carries the test inline, and only that instruction; a second fault there marks the whole block. Marking whole blocks for one fault had put the test in front of every access of UnityPlayer's unrolled texture-slot loop, for one slot that sits across a granule. With patched sites now common, the arm a patch branches to had to become transparent. It borrowed two scratch registers and tested the granule with an `ands`, but a `cmov` from memory keeps its condition in one of those registers across the load, so a `cmov` from a straddling address took the wrong side, in identity mode too. The arm now saves the two registers it borrows on the host stack and tests the granule with a bit test. `tests/dynamic/align_ordered.c` loads, stores, read-modify-writes, `cmov`s and `adc`s at every offset across granules in ordered memory, against Rosetta, in both layouts. On R.E.P.O.'s main menu this and the commit after it took alternating runs from 48.1 and 48.6 frames a second to 61.3 and 59.6, and the main thread from 18.8 to 14.4 ms of CPU a frame. `OCERZ_ALIGN_TEST_ALL=1` tests every access again.

The patch is a `b`, which reaches 128 MB, and its stub originally had to sit at the end of the arena. When the end was out of reach, or the store's data register was 31 (in store encodings, the zero register, which is how a guest store of an immediate zero comes out), the patch used to give up and retranslate the block. Discord's V8 went through 5,391 retranslations in its first ten seconds that way, mostly the same few stores. Since `20e684f4` the stub goes into the nearest veneer pool when the end is out of reach and zero-register stores are patched as well, so it takes none. A first alignment or commpage fault also retires only the faulting block, through the predecessor lists, instead of invalidating its whole range, which at Electron's startup meant scanning about 140,000 live blocks per fault (`OCERZ_FAULT_INV_RANGE=1` restores the range invalidation).

A rip-relative or absolute operand has an address that is a constant of the translation, so for those both questions, whether the address is in the commpage and whether it crosses a granule, are answered when the block is built (`3c9fe1b2`). A global load or store, the most common memory form in compiled code, costs four host instructions instead of fourteen in plain mode and eighteen in ordered mode. The known-address note is set by the guard and consumed by the very next ordered access, and building any other effective address clears it, so it can never describe an address other than the one just guarded.

Locked operations at any alignment

x86 makes a `LOCK`-prefixed read-modify-write atomic at any alignment, because the bus or cache-line lock covers a split access. arm64's LSE atomics fault on a misaligned address, and so does clang's `__atomic_*`. This is not a corner case: Wine's `CRITICAL_SECTION` inside a packed Valve structure sits at offset `0x446`, so every `EnterCriticalSection` in Steam's CEF host is a `lock cmpxchg` on a doubleword that is 2 modulo 4. The JIT sends a misaligned locked access to the interpreter, which takes a lock striped by address and performs the read-modify-write with plain unaligned copies. That is atomic against every other misaligned access to the same location, which is the guarantee that matters in practice. `cmpxchg`'s flags follow Rosetta: flags as for `dest - accumulator`, and the accumulator written in both cases, so the 32-bit form zero-extends `rax` even on a match. An aligned locked or exchanging operation in ordered memory is an LSE atomic, which takes a base register alone, so its displacement has to be added into the address first. Until `737e3fce` (2026-10-04) an operand off `rsp` skipped that, because stack operands count as plain accesses, and in a Wine process `lock xadd`, `lock add`, `xchg` or `lock cmpxchg` on `[rsp+8]` worked on `[rsp]`.

What is ordered now, and what it costs

Access, in ordered memory	Translated as	Switch to order it
scalar load or store, not on the stack	acquire or release (`ldapur` , `stlur`)	-
a 64-bit-code load that supplies the address of the next load, except a chain's last	plain `ldr`	`OCERZ_NO_DEP_PLAIN=1`
`push` , `pop` , `call` , `ret` , rsp-relative operands, in every layout	plain	`OCERZ_TSO_STRICT=1`
esp-relative operands of 32-bit code	plain	`OCERZ_NO_M32_STACK_PLAIN=1`
SSE and AVX loads and stores	plain	`OCERZ_TSO_VECTOR=1` , `OCERZ_TSO_NARROW=1|2`
the system's string and memory routines	plain	`OCERZ_NO_MEMFN_PLAIN=1`
locked and exchanging operations	LSE atomics; misaligned ones in the interpreter	-
`mfence` , `lfence` , `sfence`	`dmb ish` , `ishld` , `ishst`	`OCERZ_NO_LIGHT_FENCE=1` (all `dmb ish`)

The cost of what remains ordered is easiest to see in a program that is always ordered and spends its time in loads and stores. 7-Zip 25.01 compressing under Wine Staging 11.18, on one thread of a MacBook Air (M5), was measured in a working session on 2026-10-14 rather than in a commit: about 8,200 KiB/s under AArchX against 11,600 under Rosetta, and a 26 MB archive in 6.85 s against Rosetta's 4.8 s. 70% of compression is LZMA's binary-tree match finder. Wine always needs ordered memory, and the same kernel with plain memory instead ran about 17% faster, producing an identical archive. Splitting that 17% gave two different answers in two settings. In 7-Zip's built-in benchmark the acquire loads accounted for 14% and the release stores for 2.6%. In the real compression run the ordering itself cost only 1-3%, and most of the rest was code generation: `ldapur` and `stlur` take no index register, so an indexed ordered access needs an `add` in front of it, and the Wine layout's single register-offset access for hoisted indexed operands (`8365b47d` , Chapter 8) is made for plain accesses only.

Addressing in Wine's memory layout

In most processes a guest address either is the host address (the identity map of dynamically linked programs) or differs from it by one constant base. Wine is different. Its main image insists on a low address the host will not hand out, so Chapter 8's *low shadow* puts the guest's first 12 GB at another host base, keeps the middle of the address space as identity, and maps a strip at the very top, which contains the commpage, at a third base. Every guest memory access in a Wine process has to go through that translation.

In its general form the translation builds three 64-bit constants and tests three ranges, about twenty instructions in front of a single load. Since `1b636806` (2026-10-04) there is a fast form whenever the low window's host base is a single run of bits above every low address, as the usual `0x8000000000` and `0x10000000000` are. An address below 12 GB then becomes a host address with a shift, a compare and one `orr` , since or-ing a base whose bits are all above the address is the same as adding it. An address between

12 GB and the top strip was recognised as identity with two shifts and an add, and only the top strip took the general form. In the Wine layout that took memcpy from 6.1x of Rosetta's time to 3.1x, a mixed workload from 3.0x to 1.65x and an interpreter-style loop from 1.5x to 1.0x, and R.E.P.O.'s main menu from 46 frames a second to 65. A debug build that ran both forms at every access and compared the results never saw them differ. `OCERZ_NO_FAST_LOW_GUARD=1` keeps the general form.

The same afternoon took the top strip out of the common path. First the general form, which hot code never runs, moved from inline after the short tests to an arm at the end of the block (`ed2fd0cf`). Then the top strip stopped being tested in advance at all (`1b325b73`). arm64 cannot map anything at the top strip's guest addresses, so an access that reaches it lands on its identity address and faults, and the handler marks the block the way identity mode marks a commpage reader, interprets that one instruction and retires the block. An ordinary block now tests the low window alone: a shift, a compare, an untaken branch and an `orr` for a low address, and the first three for an identity one. A marked block's retranslation tests the identity range with two shifts and an add and takes the general form out of line. `OCERZ_LOW_TOP_GUARD=1` tests the top strip in every block.

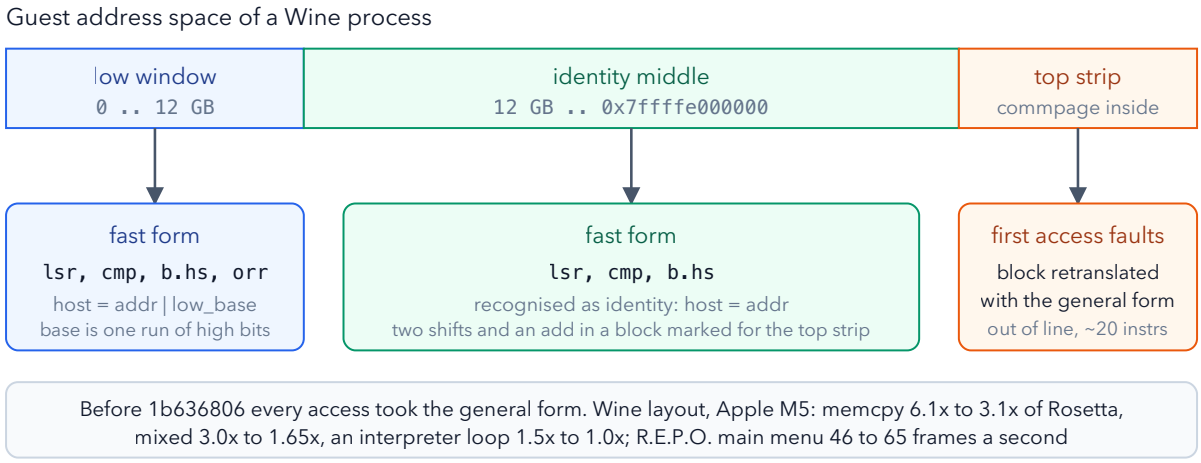


Figure 7.5. Translating a guest address in Wine's layout. Most accesses fall in the low window or the identity middle and take a three- or four-instruction form. The top strip with the commpage is not tested until an access there faults; the block is then translated again with the general twenty-instruction form out of line.

Several of the JIT's other fast paths also needed rework to run in this layout at all. The memory-operand fallback described under SSE above is one. The call and return paths are another: the stack may live below 12 GB, so every stack fast path used to be off there, and xbench's leafcall took 2.75 s against 0.16 s outside the window. Chapter 6 describes how calls and returns now keep the host return stack in the window (leafcall 0.35 s). Stack accesses then left the guard altogether: a thread's stack lies wholly on one side of 12 GB, so x0, which this layout does not need as a guest base, holds the delta between `rsp` and its slots' host addresses, and a push, a pop, a call, a ret or an rsp-relative operand adds it where the guard spent a shift, a compare, a branch and an `orr` (`e7a1cd62`, 2026-10-07). From 2026-10-08 the layout gained a base hoist of its own: a block tests its busiest base registers once, at the head of a loop, over the whole span it reaches from them, and its accesses through them then go straight to the host. Chapter 8 describes the guard, the stack delta and the hoist in full. The FP undo log still needs an address formed with no check at all, so it stays off under memory guards.

Code that changes

Catching writes to translated code

A translation is only valid while the guest bytes it came from are unchanged. Unmapping or remapping code is easy to see, because it goes through `munmap`, `mmap` or `mprotect`, and each of those asks the JIT to drop the code in its range. The hard case is a plain store into a page that is both writable and executable: JavaScript engines do exactly that, rewriting their own code in place, and until 2026-09-07 AArchX kept running the stale translation.

The fix lives in `src/mem.c` and is called **arming**. A host page holds up to four 4 KB guest pages, so its protection is the union of its slots' protections, minus write while the page is armed, meaning translations were made from code in it. A later guest store to an armed page faults; the fault handler drops the translations in the written range, unarms the page and lets the store run again, and the new bytes are translated afresh the next time they execute. Only code that sits in a guest-writable slot can be rewritten by a store, because code in a read-execute slot changes only through `mprotect` or `mmap`, which invalidate on their own. So a page is armed only when a writable slot overlaps translated bytes, which leaves Wine's PE images untouched where the tail of a `.text` section shares a host page with a writable section.

The reverse direction matters as much. A kernel copyout onto a read-only page does not fault: it fails, and with a Mach message the reply is destroyed with it. So the syscall layer unarms every armed page that overlaps a buffer the kernel is about to write, and retries once after unarming when a system call comes back `EFAULT` with armed pages present. A counter of armed pages lets the whole check be skipped when it is zero.

Finding the blocks to retire

Under Wine the live code spans PE images at 32-bit addresses and shared-cache libraries near `0x7ff800000000`, so a global minimum and maximum cannot answer "is there code here". A **region map**, one slot per 4 MB with a bitmap of the 64 KB granules in it, answers in constant time; it may say "maybe" after code is gone but never "no" while code is present. When it says "maybe", the candidate blocks used to come from a walk over every live block, and retiring them rebuilt the whole live array. Unity's Mono JIT writes code all through R.E.P.O.'s startup with about 390,000 blocks live, and those two functions were 30% of the game process. Since `3a4eb467` each 64 KB granule keeps a list of the blocks in it beside its count, an invalidation looks only at the lists of the granules it covers (falling back to the walk past 256 granules or when the list table is full), and a retired block leaves the live array by swapping places with the last one. A benchmark that rewrites one small function at a time among 300,000 translated ones went from 37.9 to 24.7 microseconds per rewrite, against Rosetta's 14.7. `OCERZ_INV_SCAN=1` walks every block again.

Retirement itself was then made precise (`b8ba4796`). Mono patches call sites in code it has already run, over a thousand times a minute in R.E.P.O., and each patch is a write to an armed page. The fault unprotected one 16 KB page, but the invalidation used to be rounded out to the whole 64 KB granule, retiring up to four times the code that could have changed; then it cleared every return-address cell and slot, every site cache and every inline-cache slot in the process, and wiped every thread's return stack, so every thread mispredicted its returns after every patch. Now only the blocks overlapping the written range are retired, a granule's "may hold code" bit is cleared only when no block is left in it, and only the cached entries that point into a retired block are cleared, found by a binary search over the retired blocks' code. The return stacks are filled by translated code, which can race a retirement, so every 256th retirement still wipes

them all. The per-site indirect-branch caches were handled the same way at first, with a full clear every 256th retirement and, on every other one, a walk over every table for the entries naming a retired block. R.E.P.O. builds about 260,000 of those tables, 134 MB, and the walk took about 4 ms per retirement with the JIT lock held, more than all of the game's translation time. Since `d8b7073c` (2026-10-05) an entry's address word carries its column's retire generation (the address xor the generation, which sits above bit 48), the VM structure begins with the 32 generations, and retiring bumps the generations of the columns the retired blocks hash to, after they have left the hash table. A hit compares the cached word with the target xor its column's current generation, so an entry filled before the retirement no longer matches, and the periodic clear is gone. Over a menu session R.E.P.O. spent 75 ms on 1,439 retirements instead of about 6.2 s on 1,446, and its menu rendered after 120 s instead of 139 s. The hit path is two instructions longer, which made xbench's `vm` kernel, a twelve-way switch, 9-11% slower, still ahead of Rosetta (0.82 s against 0.94-0.99 s). The inline-cache slots turned out to be used by no translated code at all and were removed. `OCERZ_INV_GRANULE=1` restores the granule rounding.

A retired block's code is not freed. It stays on a retired list, so a thread that is still inside it runs on to its next exit; that is what makes retirement safe without stopping other threads, and it is also why the arena eventually fills (below).

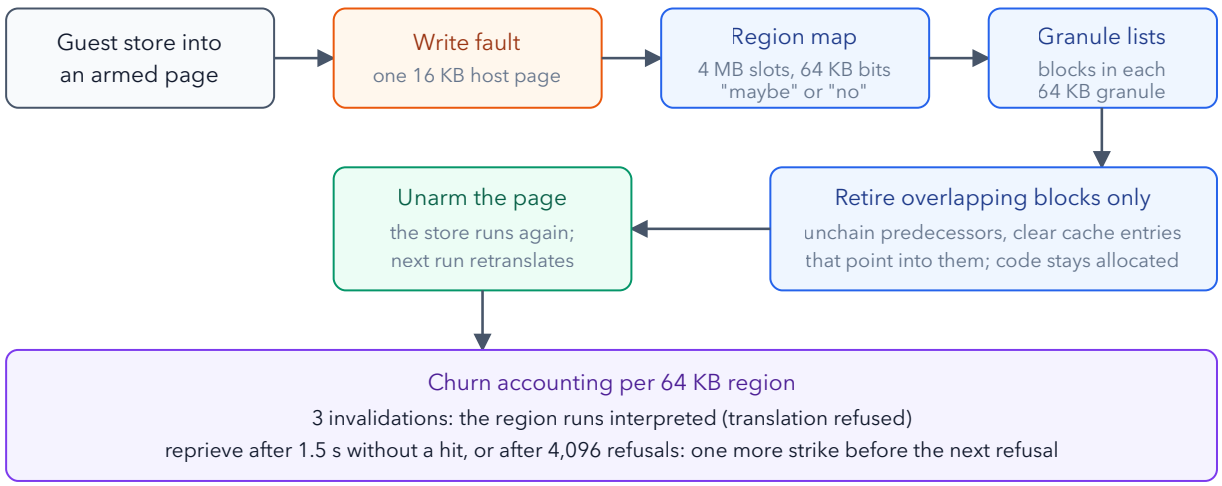


Figure 7.6. From a store into translated code to the next translation. Since October 2026 only the blocks that overlap the written range are retired, found through per-granule lists rather than a walk over every live block.

Callers that still hold the old code

Finding the blocks a write reaches is half the job; every path that still leads into them has to be cut as well. Three such paths were missed until October, two of them hidden because a retired translation of bytes that did not change still computes the right answer.

- **Spliced callees** (`60bd6cea`, 2026-10-11). `ocerz_jit_invalidate_range` checked a write against each block's span from its first instruction to the end of its last. A callee spliced into its caller (Chapter 6) lies wherever the callee is, not inside that span, and so may a superblock's leg at a branch target. A write to such code passed the check for every block that held a copy, so a function hooked by patching its first bytes stayed unhooked for every caller that had it spliced in. Each block now keeps the least and the greatest guest address its instructions cover (`block_gspan`), and the check uses that. `tests/dynamic/smc_chain.c` runs a direct call, a `jcc` and a `jmp` to routines on another page 20,000 times,

rewrites the routines twice and requires the code written last; before the fix the call kept returning the first value in both layouts.

- **Chained calls** (`dd5717fd`, 2026-10-08). A chained direct call is a `bl` patched straight into the callee's body, and the routine that decodes a patched branch's target knew `b`, `b.cond`, `cbz` and `cbnz` but not `bl`, so retiring a block left its callers calling the retired code. It showed when a retirement had to take effect: an x87 block entered with the wrong TOP retires itself, and `x87_top_entry` saw one function retired 5,999 times because its callers kept entering the old block.
- **Chains through veneers** (`bfaca27a`, 2026-10-11). A chain to a target beyond a branch's 128 MB goes through a veneer, a literal load and a `br`, and the chained branch lands on the veneer. Unchaining compared each branch's target with the retired block's code, which a veneer is not in, so a far chain was never cut. In Steam's CEF processes, with over 700,000 blocks live, many chains are far, and blocks a branch flip had retired went on running: once `chain_target` followed a veneer to the address it loads, probe exits from retired blocks in the three busiest processes fell from 1,905,762, 1,159,135 and 527,613 to 323,620, 67,584 and 24,701. Each such exit had also asked for the block's retirement again, and, not finding the block where its index pointed, searched the whole live list: the browser process ran 169,984 searches of a 716,148-block list, 29.4 s of its CPU, none finding its block. A block whose index does not point back at it is not live, and is now recognised as such at once.

The veneer commit took itself for a speed fix, saying its tests also passed without it. The next commit (`e778db6a`) corrected that: `smc_ras`, with every chain forced through a veneer, fails with "stale return site 7800, stale indirect target 0" when unchaining does not follow the veneer, so a far chain left in place had been reaching the old translation after a write to guest code. The dynamic suite now runs `smc_chain`, `smc_ras`, `flip_phase` and `xlive_self` a second time with `OCERZ_FORCE_VENEER=1`, so the veneer paths of chaining, unchaining on a write and a flip's retirement run in every gate.

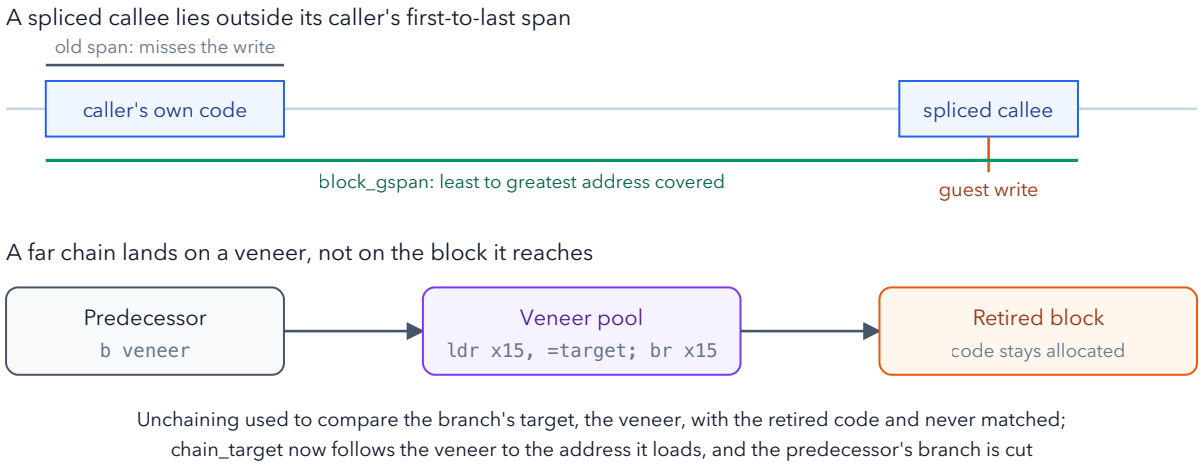


Figure 7.7. Two ways a caller kept running code that a write had made stale. A write to a spliced callee missed the caller's recorded span, and a chain through a veneer was never recognised as leading into the retired block.

Regions that never stop changing

Some code is rewritten so often that translating it is wasted work: a JavaScript engine that flips its code space between writable and executable retires its translations faster than they pay back. The JIT counts invalidations per 64 KB region in a 4,096-slot table, and after three it refuses to translate in that region, so the code runs in the interpreter. A permanent blacklist turned out to be wrong, though. Module-load fixups

also retire blocks a few times and then never again, and a blacklist made the hottest code of a freshly loaded DLL interpret forever. So a region is given a reprieve, dropping back one strike, when it has been quiet for 1.5 seconds (`CHURN_QUIET_NS`). That was still not enough for Steam: hot pages in ICU, libdispatch and Foundation never went quiet for 1.5 s, and one ICU page was refused translation about 30 million times in a two-minute run. Since 2026-09-12 a region also gets its reprieve after 4,096 refusals (`CHURN_REFUSE_MAX`), which took refusals from about 25 million to under 64 thousand per run. `OCERZ_BLACKLOG=1` prints the most-refused pages.

Code that changes is not always where a JavaScript engine's time goes. A function-churn benchmark under Node in Wine, measured in a working session on a MacBook Air on 2026-10-18, ran at 2.2 times Rosetta's time with 90% of it in translated V8 code rather than in translating it. That is an open lead, not a result.

When the arena fills

Translated code goes into one `MAP_JIT` code arena, 1 GB by default (`OCERZ_JIT_CODE_MB` and `OCERZ_JIT_CODE_KB` change it; the test suite uses a 64 KB arena for some tests to force most blocks down the interpreted path). The arena fills front to back, and retired code is never reused in place, because another thread might still be running it. A program that keeps generating code therefore fills the arena eventually, and Unity's Mono JIT in R.E.P.O. does it within two minutes of starting. Until 2026-10-04 every block after that point ran interpreted.

There was a worse problem. R.E.P.O. also crashed in Mono's metadata code in every run whose arena filled while it was still loading. A 2 or 3 GB arena made the crash go away and a 600 MB one brought it back. Running `tests/dynamic/dlopen_cryptex.c` with a 512 KB arena reproduces the same kind of damage within a second whenever a translation overflows the arena and more translating follows. Why an overflowing translation leaves the translator in that state is not understood yet, so the arena is no longer allowed to overflow (`7b82e872`).

Instead, when less than an eighth of the arena, and at most 8 MB, is left, the next translation miss **flushes** it: every block is retired, every table that points into the arena (chains, return-address cells and slots, site caches, veneer pools, the dispatch stubs, the code index) is emptied, and translation starts again at the front. The same flush runs when the return-address slots run out, since only a flush gives those back. The difficulty is that no thread may still be running code in the arena when it is reused, and the flusher cannot simply stop other threads, because a thread may be blocked in a system call made from translated code.

The protocol works with two counts per thread: how many translated frames it is in, and how many of those are parked in a system call made through the slow path, `ocerz_jit_exec_one`. A thread whose two counts are equal is not executing arena code, even if arena frames are on its stack. The flusher bumps a generation number, patches every *stop site* so that a running thread leaves at its next block edge (synchronising one cache line per patched word), and waits until every thread's counts are equal. A thread that reaches the dispatcher meanwhile runs its next instruction in the interpreter rather than waiting, so a garbage collector that has frozen another thread can still get round to resuming it. A parked call that returns into a newer generation does not return into the arena: the slow path has already written the guest state back to the CPU structure, so it goes back to its run loop (`ocerz_vm_jit_escape`) with its result, which is the exit the block itself would have taken. If some thread cannot leave within 500 ms, for example one stopped by `thread_suspend` while it spins, the flush gives up, restores the stop-site words it changed, and is retried two seconds later. A thread that is itself inside translated code never flushes, and a fork child forgets its parent's other threads.

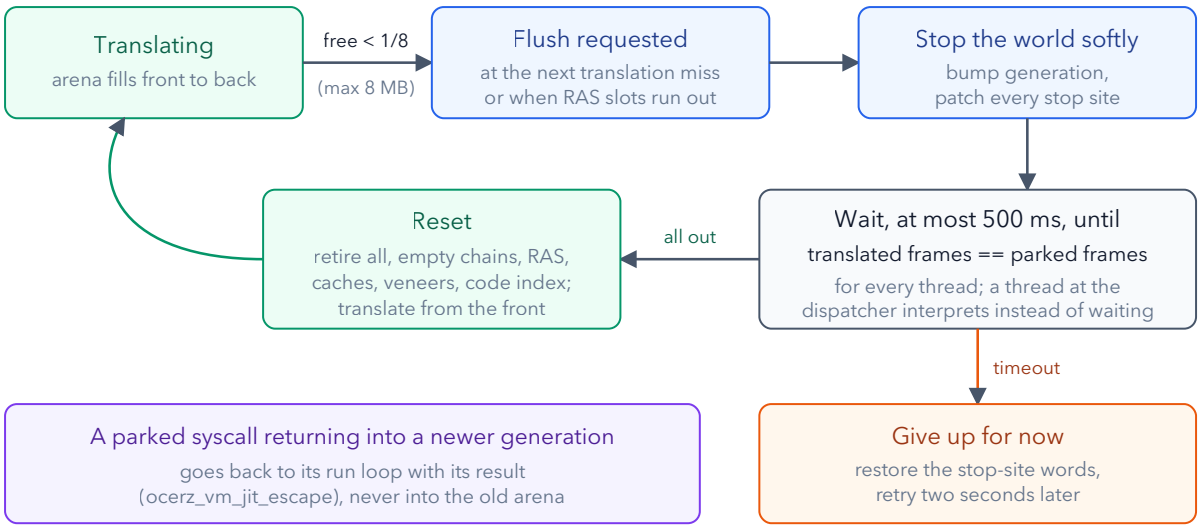


Figure 7.8. The arena flush. Threads leave translated code at their next block edge; a thread blocked in a system call made from translated code counts as out, and is redirected to its run loop when the call returns.

The first version of the flush unchained the old code and pushed the whole arena through the instruction cache, which held every thread for a third of a second; neither is needed, since nothing runs the old code again, and both were removed (`b8ba4796`). Only a system call made through the slow path is parked, because any other slow-path call returns soon enough to leave by a stop site, and parking all of them had cost R.E.P.O.'s worker threads a tenth to a quarter of their busy time in atomics and thread-local lookups. With the flush in place R.E.P.O.'s main menu went from 28-30 frames a second to 46 with nothing else changed, the arena is flushed about once a minute, and the Mono crash did not come back in five runs. With a 512 KB arena the dynamic suite passes everything except the translation-cache cases, which need room to keep records, at thousands of flushes per test. `OCERZ_NO_JIT_FLUSH=1` restores the old behaviour and `OCERZ_FLUSHLOG=1` logs every flush and names the threads it waits on.

Stopping one thread's loop for a signal

The stop sites gained a second user on 2026-10-14 (`a7fd0f3f`). A guest signal used to reach a thread only at its next system call or bridged call, so a thread spinning in a translated loop never saw it; under Wine, `SuspendThread` is a signal sent to the thread, so Mono's collector waited for every busy thread to finish its loop. The run loops now deliver a pending signal between two steps, and translated code is made to get there: the host handler does to the block the interrupted thread is in, and to the blocks its chained edges lead to a few hops on, what a retirement does, unchaining their edges and rewriting their stop sites, so the thread's next jump out of any of them goes to the dispatcher, and the dispatch stub, which enters the next block without returning, goes back to the run loop while such a signal is pending. The thread records each block and edge it changed, and the run loop puts them back. Stop sites alone were not enough, because a loop whose back edge is a side exit chained through its conditional branch has none, which is why the edges are cut too. The handler takes no lock, since the thread holding the JIT lock may be the one that sent the signal; it and a chain install can race on a word, but each side only ever writes a whole valid branch, and the restore runs under the lock. Chapter 9 describes the delivery itself. A block whose stop sites are not in their running state is no longer written to the translation cache, where it would have come back stopped for good. `OCERZ_NO_SIGNAL_STOP=1` goes back to the old delivery, which m32 processes keep, since delivery there would enter a 64-bit handler.

In-place string and memory routines

Eleven routines are called constantly by almost every program and do very little each time: `strlen`, `strnlen`, `strcmp`, `strncmp`, `memcmp`, `bcmp`, `strchr`, `memchr`, `memcpy`, `memmove` and `memset`. In native mode a guest call to one of them crosses into the host's implementation, and the crossing costs far more than the routine: every pinned guest register goes out to the CPU structure and comes back, the arguments are read into an argument array, the rounding mode and `errno` are swapped twice, and the block that made the call is re-entered through its return check. For a routine that reads a few dozen bytes and returns a number, nine tenths of the time went to the crossing. `src/leaf.s` holds arm64 versions of them under a contract that needs none of that.

The contract

A routine reads its first three arguments where full pinning keeps them, `rdi` in x28, `rsi` in x27 and `rdx` in x23, and writes its result to x21, which is `rax`. It changes x9 to x15, v0 to v3 and the flags, and nothing else; it makes no call, uses no stack and never writes x30. The guest base is zero wherever these routines are used, so a guest pointer is dereferenced as it arrives; in the Wine layout, where a pointer below 12 GB is not the host's address, the calling block translates the pointers before the call and puts the guest's back after it (below). A block therefore reaches one with a single direct branch and spills nothing. The routines are copied, being position-independent, to the start of the code arena, so the branch is always in range.

Never writing x30 is what keeps a fault exact. If a guest hands `strlen` a bad pointer, the fault happens at a program counter inside `src/leaf.s`, which belongs to no translated block, but x30 still names the instruction after the branch, and every pinned register is as the guest left it. `ocerz_leaf_site` maps a program counter inside the routines to that calling site, so `src/vm.c` recovers the guest state as if the stub's own instruction had faulted, and the same substitution lets a suspended thread report its registers. Because `rax` is written by the last instructions before the return, which touch no memory, a fault never observes half a result.

The scanning routines read whole 16-byte blocks at 16-byte-aligned addresses. Such a block never spans a page boundary and always holds at least one byte the caller owns, so reading past the end of a string can never touch an unmapped page. The two-string comparisons cannot align both pointers at once; they read unaligned blocks while both pointers are at least 16 bytes short of a 4,096-byte boundary and compare byte by byte for one block otherwise. A 16-byte comparison leaves 0xff or 0 in each byte, `shrn` by four packs that into a 64-bit value with four bits per byte, and the index of the first interesting byte is its count of trailing zeros divided by four.

Declining

Every routine also answers in x9: zero when it did the work, one when it declines, having changed nothing the guest can tell, in which case the caller does what it would have done without the routine. `memmove`, which also serves `memcpy`, declines by itself for one case: ranges that overlap without being identical. The reason is what a fault means here. A fault inside a routine leads, in either mode, to the whole call being made again from its original arguments. Doing part of the work and then all of it is harmless for reads, for copies between ranges that do not touch and for filling a range with one byte, but not for an overlapping move, where the part already moved has overwritten source bytes the second run would read. Up to 64 bytes every load is made before any store; beyond that the copy runs forward in 64-byte blocks and finishes with the last 64 bytes, which the ranges being disjoint makes safe.

The routines that write (`memmove` and `memset`) may store onto a page whose code has been translated; the fault handler resolves that by retiring the translations and letting the store run again, and the caller compares the retire count from before the call with the one after and does not return through its shadow stack when they differ. In native mode these two are used only for lengths up to 16,384 bytes, above which the host's routines are faster.

Two modes, one set of routines

In native mode the routines replace the crossing: the block checks its stub's slot as before, branches to the routine, pops the return address and runs the same return. A fault inside a routine is attributed to the stub, so a guest with a `SIGSEGV` handler gets its handler with `rbx` and `r12` to `r15` holding what the caller left there, where a fault inside an ordinary crossing ends the process.

In cache mode the guest's `strlen` is Apple's x86 code from the shared cache. When a block starts at the exported entry of one of ten of those routines, which `ocerz_dyldapi_leaf_entry` recognises, the same call and return are emitted ahead of the block's first instruction, and the translation of the x86 code follows as what runs when the routine declines. No length limit applies, since the x86 routine it would fall back to is slower at every length. A fault inside a routine is not delivered to the guest from there: `src/vm.c` makes the routine decline instead, and the x86 code then takes the same fault, with every detail the guest can observe already right. A block with a branch back to its own start is left alone, because its lanes may be live where the call would leave.

Guest loop, ns per iteration (Apple M5, 2026-09-21)	trap path	default	arm64 build
`getpid`, an ordinary crossing	40.5	17.0	0.7
`strlen` of a 12-byte string	42.0	**2.0**	0.9
`memcpy` of 48 bytes	45.6	**3.0**	2.7

With `OCERZ_NO_LEAF_INPLACE=1` the `strlen` and `memcpy` rows read 17.3 and 20.8 ns, which is what an ordinary crossing costs. In the dynamically linked benchmark the routines took the `str` kernel from 6.78x of Rosetta's time to 0.82x in native mode and from 1.08x to 0.77x in cache mode, and `memcpy` from 1.06x to 0.73x and from 0.98x to 0.72x. `tests/unit/test_leaf.c` checks every routine against the C library over 6,736,902 inputs, each string placed against an inaccessible page at every alignment so that reading one byte too far dies in the test. `sys_strings` repeats that from inside a guest in both modes and both engines, `sys_strings_inplace` fails unless the translator reports that it answered the calls in place, and `sys_strings_fault` runs each routine into a forbidden page and requires all twelve faults to reach the guest's handler with its callee-saved registers intact.

Finding the routines, and the Wine layout

The routines are found by name, and on macOS 26.7.1 that stopped working: the platform library's x86_64 slice no longer exports the plain names, which libsystem_c now re-exports, so cache mode answered none of the nine entries behind them in place and ran the x86 routines, ordered at that, until `82816498` (2026-10-05) resolved the names from libSystem, the way a program binds them.

The Wine layout refused the routines outright until 2026-10-10, so in every Wine process `memcpy`, `memset`, `strlen` and the rest ran Apple's x86 code translated with the low-window test on every access; xbench's memcpy took 2.26 times Rosetta's time there against 0.72 outside it. Since `a0375541` the block at a routine's entry in cache mode hands it translated pointers. It saves `rdi` in x17 and `rsi` in x29, sets x9

to "declined", and for each pointer checks that the range the routine may touch lies on one side of 12 GB, or above it and below the top strip, before adding that side's delta. A check that fails, a routine that declines and a fault inside one all arrive after the call with x9 set, and `rdi` and `rsi` go back before the x86 translation runs. Only routines that are told how far they may read qualify: `memmove` and `memcpy`, `memset`, `memcmp`, `memchr` and `strncmp`, bounded by `rdx`. `strlen` is asked as `strnlen` up to 12 GB or the top strip and declines when it gets there, as a string running across would be; `strcmp` and `strchr` have no bound and keep their translation. `memchr`'s answer is turned back into a guest pointer, and `memmove` and `memset` answer the guest's first argument. xbench in the Wine layout: memcpy 1.274 to 0.429 s (Rosetta 0.563), str 0.766 to 0.737 s (Rosetta 0.569). `tests/dynamic/leaf_low.c` calls every routine on pointers into the image's data, the heap, a mapping at 5 GB, a page that ends exactly at 12 GB, the top strip and a read-only page whose handler must see the guest's address. `OCERZ_NO_LEAF_LOW=1` turns it off.

32-bit Mac programs have leaves of their own: C functions in `src/m32_leaf.c` that an i386 import stub calls straight from its block, taking their arguments from the guest stack (Jfishin's `b92a3c9d`, 2026-10-10). Chapter 13 describes them.

Keeping translations: the on-disk cache

A short Wine program spends most of its life translating, and every process of a Wine session translates the same system and Wine code again. The translation cache (`src/tcache.c`, `93547a4c` and `08a0458b`, 2026-09-27) stores a finished block, and the next process that needs the same key loads it instead of translating. It is on by default; `OCERZ_TCACHE=off` turns it off, and every test target runs with it off so that no test depends on what an earlier one stored and none writes to the user's caches.

What is kept

Only code that sits at the same address in every run is worth keeping: all of a Wine process, whose low-shadow layout and fixed image bases make addresses stable, and otherwise only the shared cache. A main program, a dylib or a JIT loaded at a random address would fill the store with records nobody loads, and when the guest's own base is chosen at random nothing is kept. A translation made because the process *learned* something, such as a flipped branch or the guard a commpage or alignment fault asked for, is never answered from the store: a key that carries such a mark, or whose block was retired or invalidated, is translated again and the new record replaces the old. Two marks joined that list in October: a block that gave up the Wine layout's base hoist after its check failed (`7394b557`), and an x87 block entered with a TOP other than the one it was translated for. A block whose stop sites are stopped for a signal is not written at all. The key adds the memory model to the block's ordinary key, since a block translates differently once a second thread makes loads ordered.

Making code that can move

A translation is full of values that differ between processes: the addresses of ocerz's own functions and globals, the commpage delta, the block's own `JitBlock`, instruction and profile slots inside it, return-address slots and cells, indirect-call cache tables, the leaf routines and the dispatch stub. When recording, each of these is emitted as a fixed `movz` and three `movk` or as a literal cell, and the translator records where (`TcReloc`); a leaf call and the dispatch-stub exit go through a register instead of a direct branch. A

load copies the code to the same address modulo 64, because the literal pool's padding and a loop head's alignment are absolute, and fills every site from the current process, allocating slots, cells and tables afresh.

A record also names every range of guest bytes the translation read: the block, the callees spliced into it, and the successors whose flag liveness it relied on. That is why a recording translation decodes a successor itself rather than trusting a cached liveness answer, and why a memoised liveness entry keeps the ranges and a hash of the bytes it read and replays them into the record when it answers. The record carries a hash of those bytes, and a load checks the guest's memory against it before using the record, so a changed library or patched code is never answered from stale bytes.

The store

Records live in `~/Library/Caches/ocerz/tc-<fingerprint>`, or under `OCERZ_TCACHE_DIR`. The fingerprint hashes everything that changes what a translation looks like without changing the guest bytes: the ocerz binary's `LC_UUID`, the memory mode and the host bases it maps the guest at, the machine model, and every `OCERZ_` variable apart from the few that only steer logging, the loader or the cache. A different build or a different switch therefore opens a different directory, and nothing inside one needs a version check. Directories of other fingerprints are removed once untouched for a day, and sooner while they hold more than 2 GB together; one touched within the last hour is never removed, because a Wine session uses several fingerprints at once.

A directory holds one index, shared by every process, and one data file per process that has written anything, `d-<n>.td`. The index is an open-addressed table of key and location slots that every process maps shared and changes with atomic operations alone: a key is claimed with a compare-and-swap, and its location is stored only after the record is written, so a reader that finds a location finds a whole record. A process appends only to its own data file. Records are LZ4-compressed, about 1.65 times smaller for 1.6 microseconds to write and 0.3 to read, and carry a checksum checked on every load, so a record torn by a crash is refused rather than run. The checksum was a chain of dependent multiplies, one per 8 bytes, about 2 ms of a CoreFoundation program's launch for the 9 MB of records it loads; since `a02fcc3f` (2026-10-09) it is CRC32C computed in three lanes, so the instructions overlap, and the same launch spends 0.9 ms summing. Each build keeps its own store, so no record written with the old sum is read with the new one. A put copies the record into a 256 KB buffer and nothing more; a full buffer goes to a writer thread, which compresses, appends and publishes. At exit, exec or a non-main-thread exit the last buffer is handed over and the writer gets up to three seconds. Nothing is ever deleted from a store; when its data files reach `OCERZ_TCACHE_MAX_MB` (4,096 by default) writing stops, and a process that finds itself the only user of a full store (an exclusive `flock` on the directory's users file succeeds) removes it and starts again, so a full store costs one cold start rather than every translation from then on.

The store originally never looked at free disk space, and a tester's Mac with a few GB free filled up under a Steam session, could no longer swap, and panicked on the watchdog. Since `93b919f9` (2026-10-07) nothing is written while the volume has less than `OCERZ_TCACHE_MIN_FREE_MB` megabytes free (10,240 by default), checked before every append; a process writes at most half of what was free above that floor when it opened the store; and a lone process that finds the volume already under the floor removes the store, after which the prune keeps no other store older than an hour. Reading is unaffected, so a full disk costs warm starts, not correctness. The directory also keeps one thing that is not a translation: the filter of

names the shared cache's weak-defining images export, about 990,000 names that took 37 ms to build in every process that needed it, is kept there under the cache's UUID with a sum over its bytes (`94614e9c`; Chapter 11).

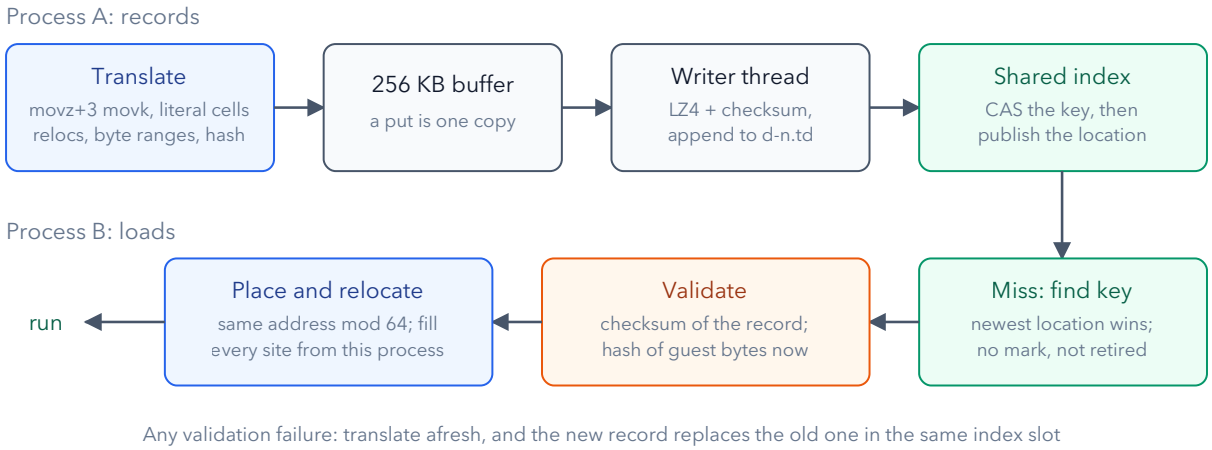


Figure 7.9. The life of a translation-cache record. One process records and publishes; any later process, in the same session or the next, validates the record against the guest's current bytes and relocates it into its own address space.

What it buys, and how it is checked

Measured back to back on the same machine, Windows Steam's window came up in 46.9 s from an empty store and in 35.2 s and 33.7 s from a warm one, and a full Steam session fills about 2 GB of store (2.9 GB before records were compressed). A cold run of `hostname.exe` under Wine, a process that lives about half a second, went from 0.78 s to 0.61 s when records began to be compressed and written by a separate thread and the dependency bookkeeping was made cheaper, against 0.58 s with nothing kept, so recording now costs a cold process almost nothing. With the cache on, the dynamic suite passes twice over one store, the second time loading 1.33 million blocks and storing 365.

Two modes exist to check the cache itself, not to use it. `OCERZ_TCACHE=verify` translates every block anyway and compares it with the record it would have loaded, counting a difference in shape as a variant and any other difference as a bug; over 4.4 million Steam blocks it found none bad. `OCERZ_TCACHE=roundtrip` writes nothing: it moves every translation to a fresh address, fills the original with `BRK`, and reports any PC-relative reference that leaves the block or any host-looking constant that no relocation covers. Across 4.6 million Steam blocks it found none, and building it turned up two older faults, both fixed with it: a fault while the translator read guest bytes used to take a path that wanted the JIT lock the translator already held, and the lane allocator now restarts for each block, so a translation no longer depends on what was translated before it.

Switches for this chapter

Variable	Effect
`OCERZ_NO_AFP=1`	keep the translated NaN checks even where FPCR.AH is available
`OCERZ_FPB_FORCEREPLAY=1`	make every NaN check take its replay arm (a test aid)
`OCERZ_FPB_NOUNDO=1`	no undo log: a possibly aliasing store ends the batch

Variable	Effect
`OCERZ_NO_ZEROREG=1`	clear ymm upper halves with two stores instead of a zero register
`OCERZ_NO_YMMH_CACHE=1`	keep ymm upper halves in memory only
`OCERZ_NO_JIT_MMX=1`	interpret every MMX instruction
`OCERZ_NO_COMIS_MEM_FUSE=1`, `OCERZ_NO_INLINE_PTEST=1`, `OCERZ_NO_PTEST_FUSE=1`	do not fuse a memory `comis` with its branch; interpret `ptest`; keep `ptest` apart from its consumer
`OCERZ_NO_INCDEC_LOOP=1`	close an inc/dec loop through its chained edge rather than straight to its loop entry
`OCERZ_NO_JIT_X87=1`	interpret every x87 instruction
`OCERZ_NO_X87_LIVE=1`, `OCERZ_NO_X87_TOPSPEC=1`, `OCERZ_NO_X87_LANES=1`	load the x87 control word for every instruction; do not translate runs for a known TOP; keep x87 values in memory only
`OCERZ_NO_FIST_RC=1`	guard integer stores for round to nearest instead of reading the rounding control
`OCERZ_X87_CHECK=1`	trap where `X87S` and memory, or the counted and real TOP, disagree
`OCERZ_NO_PLAIN_MEM=1`	start in the ordered memory model
`OCERZ_TSO_STRICT=1`, `OCERZ_TSO_VECTOR=1`, `OCERZ_TSO_NARROW=1|2`	also order stack accesses, every vector access, or `movd` / `movq` (2: every vector access of 8 bytes or fewer)
`OCERZ_NO_MEMFN_PLAIN=1`	order the system's string and memory routines like other code
`OCERZ_NO_M32_STACK_PLAIN=1`	order the esp-relative operands of 32-bit code
`OCERZ_NO_DEP_PLAIN=1`, `OCERZ_DEP_PLAIN_LOG=1`	keep every load of a dependent chain an acquire; name each load made plain
`OCERZ_NO_REMAP_PLAIN=1`, `OCERZ_ORDERLOG=1`	make the process ordered whenever kernel-placed memory is moved into the guest; print what first made it ordered
`OCERZ_NO_LIGHT_FENCE=1`	make every fence a `dmb ish`
`OCERZ_ALIGN_TEST_ALL=1`	test every ordered access for a granule crossing instead of patching the sites that fault
`OCERZ_NO_FAST_LOW_GUARD=1`	use the general address translation everywhere in the Wine layout
`OCERZ_LOW_TOP_GUARD=1`	test for the top strip in every Wine-layout block instead of after a fault there
`OCERZ_FAULT_INV_RANGE=1`	invalidate the whole range on a first alignment or commpage fault
`OCERZ_INV_SCAN=1`, `OCERZ_INV_GRANULE=1`	find blocks by walking every live block; round invalidations out to 64 KB
`OCERZ_NO_VENEER_UNCHAIN=1`, `OCERZ_FORCE_VENEER=1`, `OCERZ_LIVE_SCAN=1`	leave chains through veneers in place on retirement; chain everything through veneers (for tests); search the live list for a block being retired
`OCERZ_BLACKLOG=1`	print the pages most often refused translation
`OCERZ_JIT_CODE_MB=n`, `OCERZ_JIT_CODE_KB=n`	size of the code arena
`OCERZ_NO_JIT_FLUSH=1`, `OCERZ_FLUSHLOG=1`	do not flush a full arena; log every flush and the threads it waits on

Variable	Effect
`OCERZ_NO_SIGNAL_STOP=1`	deliver a signal only at the thread's next system call or bridged call
`OCERZ_NO_LEAF_INPLACE=1`	reach the eleven string and memory routines the ordinary way
`OCERZ_NO_LEAF_LOW=1`	in the Wine layout, leave those routines to their translated x86 code
`OCERZ_TCACHE=off|verify|roundtrip`	disable the translation cache, or run one of its two checks
`OCERZ_TCACHE_DIR`, `OCERZ_TCACHE_MAX_MB`, `OCERZ_TCACHE_LOG`	where stores live, when one stops growing, and per-process load and store counts
`OCERZ_TCACHE_MIN_FREE_MB=n`	free space the store leaves on its disk (10,240 MB by default)

PART III

Memory and the operating system

Where guest memory lives, and how system calls, Mach messages, threads and signals are answered.

CHAPTER 8

Guest memory

An x86-64 macOS program expects to own a 47-bit address space with 4 KB pages, its images where its load commands put them, and a commpage at a fixed address. AArchX runs inside an arm64 process with 16 KB pages that the kernel lays out for its own reasons. This chapter describes how guest addresses become host addresses, how 4 KB guest pages are kept on 16 KB host pages, and what happens when the host kernel or a framework hands the guest memory it never asked for.

2,342	3	256 GB	4 KB / 16 KB
LINES IN SRC/MEM.C	ADDRESS MAPS	IDENTITY ARENA	GUEST PAGE / HOST PAGE

The constraints

Three facts about arm64 macOS shape everything in `src/mem.c` and `include/ocerz/mem.h`.

- **The host lays out the process.** arm64 macOS refuses any mapping below the slid main executable, which here is `ocerz` itself, so the classic x86-64 layout with `__TEXT` at `0x100000000` cannot exist at identical host addresses. The kernel also ignores placement hints for mappings that are not `MAP_FIXED`, which was observed on macOS 26.5 when the first arena was built.
- **Pages are 16 KB.** Apple silicon's host page is `0x4000` bytes; an x86 program believes in `0x1000`-byte pages, sizes its allocations to them, and protects memory at that granularity.
- **The guest must not see the host's view.** A program that asks about its own memory is asking about guest pages, guest protections and guest addresses. Anything the host says about its own arena, its own allocations or its own pointers has to be translated or hidden.

The answer is a set of address maps, chosen per process, plus a table that tracks every 4 KB guest page inside the 16 KB host pages that hold it.

Three address maps

In June there was one map, a 32 GB arena for static binaries. There are now three, and which one a process gets depends on what it is.

Map	Used for	Rule
Affine arena	static `LC_UNIXTHREAD` binaries	guest `[0x100000000, 0x900000000)` is one hintless `PROT_NONE` reservation; `host = guest + guest_base`

Map	Used for	Rule
Identity arena	every dynamically linked program, in both modes	256 GB reserved from host `0x300000000` upward, or at `0x7a0000000000` in a Wine process; `guest_base = 0`, so a guest address is the host address
Low shadow	a program with an image below 12 GB: Wine	guest addresses below 12 GB and a 30 MB strip under the commpage live at their own host bases inside one reserved block; everything else is identity

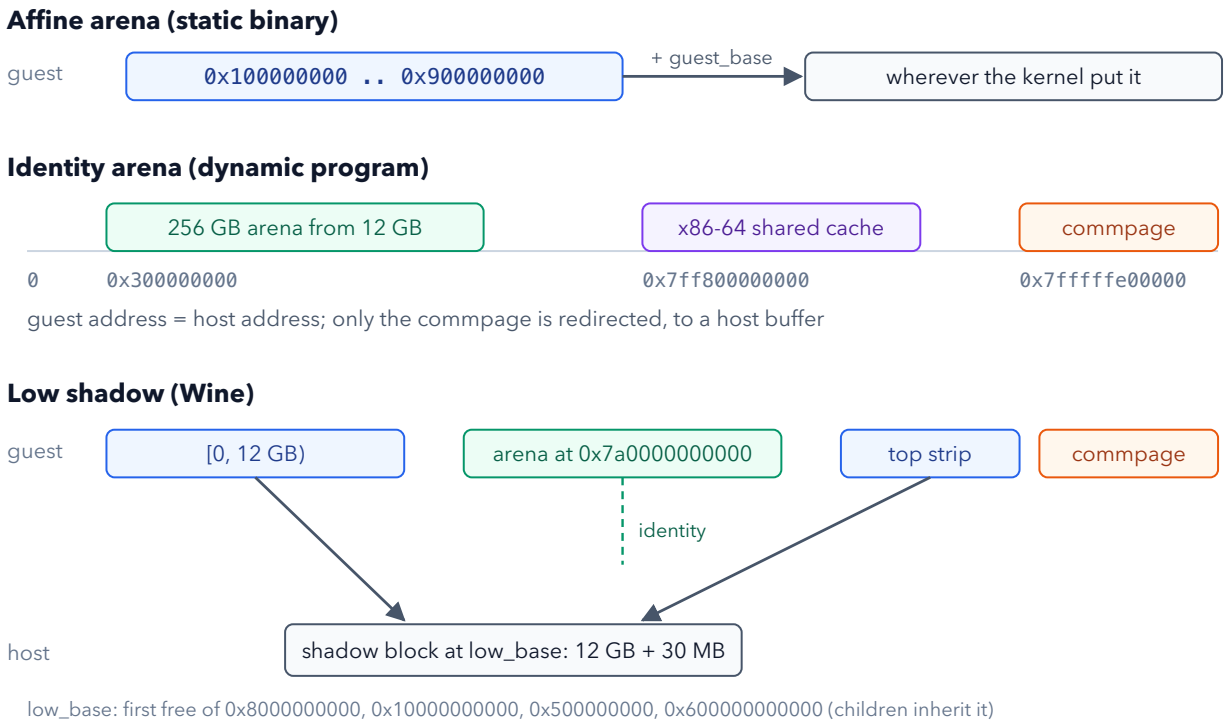


Figure 8.1. The three address maps. A static binary is offset by one constant. A dynamic program uses identity addresses, with only the commpage redirected. A Wine process keeps identity for its arena but moves the bottom 12 GB and the strip below the commpage into one shadow block, because Windows code expects those addresses and the host cannot give them out.

The affine arena

Static test binaries and other `LC_UNIXTHREAD` programs are linked at `0x100000000`, where an arm64 process can never map them. `ocerz_mem_init(0x100000000, 0x900000000)` reserves 32 GB with a plain `mmap(NULL, ..., PROT_NONE)`, takes whatever address the kernel returns, and sets `guest_base = host_base − 0x100000000`. The affine map then works for any base. Because AArchX owns the whole reservation, a `MAP_FIXED` inside it only ever replaces AArchX's own pages, and an unmap re-reserves the range as `PROT_NONE` instead of returning it, so the kernel can never place one of its own allocations inside the guest's address space. The first quarter of the arena is held back for the loader; anonymous allocations bump upward from there.

The identity arena

A dynamically linked program cannot use an offset map at all: its images are position-independent, but the x86-64 shared cache is pre-linked at `0x7ff800000000`, and in native mode the host's own CoreFoundation objects and malloc blocks have to be pointers the guest can hold, compare and hand back. So

`ocerz_mem_init_identity(256 GB)` reserves the arena with a hint at `0x300000000`, refuses one that lands lower, and sets `guest_base = 0`. A guest pointer is then a host pointer, and the JIT never adds a base. The loader keeps the first quarter of the arena, capped at 1 GB, and every other anonymous allocation bumps upward through a first-fit search of the slot table.

The arena has grown twice. In September 2026 Chromium's PartitionAlloc reserved a 32 GB region at startup and did not fit, and a day later JavaScriptCore's Gigacage asked for a single 128 GB mapping. Both are address-space reservations rather than memory, so a 256 GB arena costs nothing until it is touched.

> **HOW IT WAS FOUND**
>
> Every Wine process writes the address of its syscall dispatcher into a slot at `0x7ffe1000`, beside `KUSER_SHARED_DATA`. Under AArchX that slot shares a 16 KB host page with the file-backed shared data, so it is physically shared by every process in the Wine prefix. That is only harmless if the value is the same in all of them, which needs `ntdll.so` at the same address everywhere. With the arena wherever the host put it, each process stored an address the others had never mapped, the last writer won, and Wine's Notepad died while wineboot was still starting services. Since 2026-09-26 a process whose loader is named `wine` reserves its arena at `OCERZ_WINE_ARENA_BASE`, `0x7a0000000000`, and falls back to the usual placement only if that range is taken. `ntdll.so` now loads at the same address in every process of the session, the whole service tree starts, and wineboot finishes in about 15 seconds.

The low shadow

Windows code expects low addresses: PE images at 32-bit bases, `KUSER_SHARED_DATA` at `0x7ffe0000`, a whole 32-bit world for WoW64. Wine's macOS loader is therefore linked with a segment below 12 GB, and when the dynamic loader finds an image segment whose `vmaddr` is below `OCERZ_LOW_LIMIT` (`0x300000000`, 12 GB) it calls `ocerz_mem_init_low_shadow()`. Nothing else about Wine is special-cased here: any non-PIE program linked low gets the same map, which is how the test suite exercises it, by building ordinary tests with `-Wl,-image_base,0x200000000`.

The shadow is one host block large enough for the low 12 GB plus the top strip `[0x7ffffe000000, 0x7fffffe00000)`, 30 MB just below the commpage. It is reserved with `mach_vm_allocate(VM_FLAGS_FIXED)` at the first of four candidate bases that is free: `0x8000000000`, `0x10000000000`, `0x500000000` and `0x600000000000`. The base chosen is exported as `OCERZ_LOWBASE`, and a child process reserves exactly that base or refuses to start, so every process in a Wine tree agrees where everyone's low memory lives. That agreement matters because the wineserver reads and writes its clients' memory with Mach VM calls aimed at their tasks (Chapter 9).

The cost is that a guest address is no longer a host address everywhere, so every memory access in a low-shadow process must decide which of the three ranges it is in. That made Wine the slowest of the three maps. Since 2026-10-04 the JIT does it with a shift, a compare and an `orr` for the common case, which took the `memcpy` kernel in that layout from 6.1 times Rosetta's time to 3.1 times; Chapter 7 has the details. The cost kept falling; the section "The Wine layout's guard and its hoist" below follows it from there.

The commpage

macOS publishes kernel-maintained data, CPU capabilities among it, in a commpage at a fixed address that system libraries read directly. The guest's commpage is `[0x7fffffe00000, 0x7fffffe04000)` and must describe an x86-64 machine, not the arm64 host. AArchX allocates a 16 KB host buffer, fills its start from a fixed x86-64 template (signature, capability bits and CPU fields), and redirects that guest range to

it in `ocerz_g2h`. In identity mode the guest address itself cannot be mapped on the host, so a translated plain-form access to it faults, and the fault handler marks the block for guarded retranslation (Chapter 7). A rip-relative or absolute access whose address is known at translation time is resolved there and then.

Where the shared cache sits

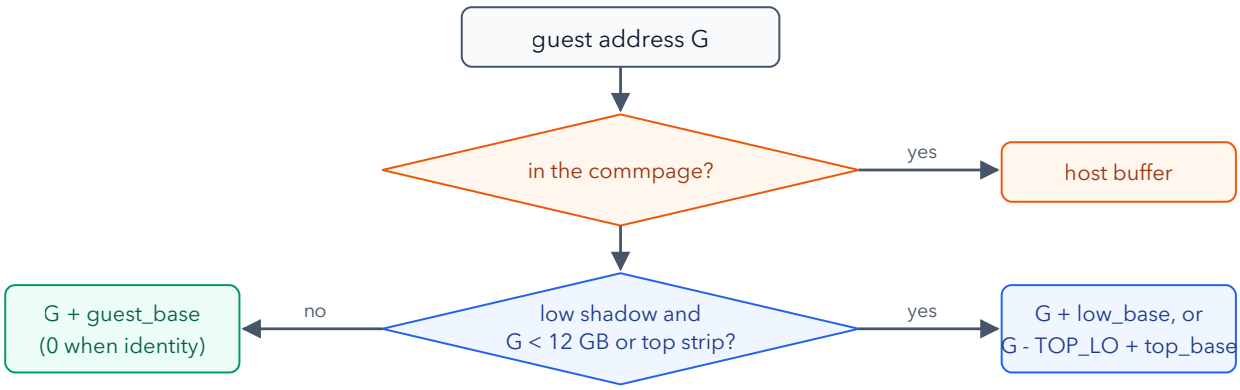
In cache mode the x86-64 shared cache is mapped by `src/cache.c` at its preferred address, `0x7ff800000000` and up, which is outside the arena and, with `guest_base = 0`, also its host address. Its data regions are mapped inaccessible and unpacked one 16 KB page at a time on first touch. Chapter 11 covers it.

Translating an address

Every access the interpreter makes, every pointer a syscall forwards, and every address the JIT cannot prove identity goes through two inline functions in `mem.h`.

```
static inline void *ocerz_g2h(uint64_t gaddr)
{
    if (ocerz_commpage && gaddr >= OCERZ_COMMPAGE_LO && gaddr < OCERZ_COMMPAGE_HI)
        return ocerz_commpage + (gaddr - OCERZ_COMMPAGE_LO);
    if (ocerz_low_base) {
        if (gaddr < OCERZ_LOW_LIMIT)
            return (void *)(uintptr_t)(gaddr + ocerz_low_base);
        if (gaddr - OCERZ_TOP_LO < OCERZ_TOP_HI - OCERZ_TOP_LO)
            return (void *)(uintptr_t)(gaddr - OCERZ_TOP_LO + ocerz_top_base);
    }
    return (void *)(uintptr_t)(gaddr + ocerz_guest_base);
}
```

`ocerz_h2g` is its inverse. Two predicates answer different questions on purpose. `ocerz_host_in_guest_space` is generous: fault attribution counts addresses below the arena as the guest's, because a NULL dereference is still the guest's fault, and in identity mode it counts the commpage range even though nothing is mapped there. `ocerz_host_in_guest_reservation` is strict and covers only memory AArchX actually reserved, which is what decides whether a pointer the kernel returned needs relocating.



The JIT folds these tests into the emitted code: none in the identity map, a guard in the low shadow

Figure 8.2. How `ocerz_g2h` decides where a guest address lives. In a dynamic program without a low shadow only the first test can succeed, which is why the identity map is the fast case.

The C accessors built on top, `ocerz_ld` and `ocerz_st`, use acquire loads and release stores for every naturally aligned access, and a fence around an unaligned copy. The interpreter therefore gives x86's ordering guarantees without help from the JIT's mode switch. The store side also carries software watchpoints: `OCERZ_WATCH` (with `OCERZ_WATCHLEN`) on an address range, `OCERZ_STVAL` on a stored value, and `OCERZ_SHADOWST` on any store of a host pointer into the low shadow.

4 KB guest pages on 16 KB host pages

A host page holds up to four guest pages, each of which can have its own protection, its own backing and its own lifetime. `mem.c` tracks guest memory in 4 KB **slots**. The address space is divided into up to 128 regions (the arena, the low window, the top strip, any extra range a fixed image needs), and each region carries four parallel arrays: a 32-bit word per slot, a bitmap, a byte per host page recording whether it is shared with another mapping, and a byte per host page recording whether it is armed.

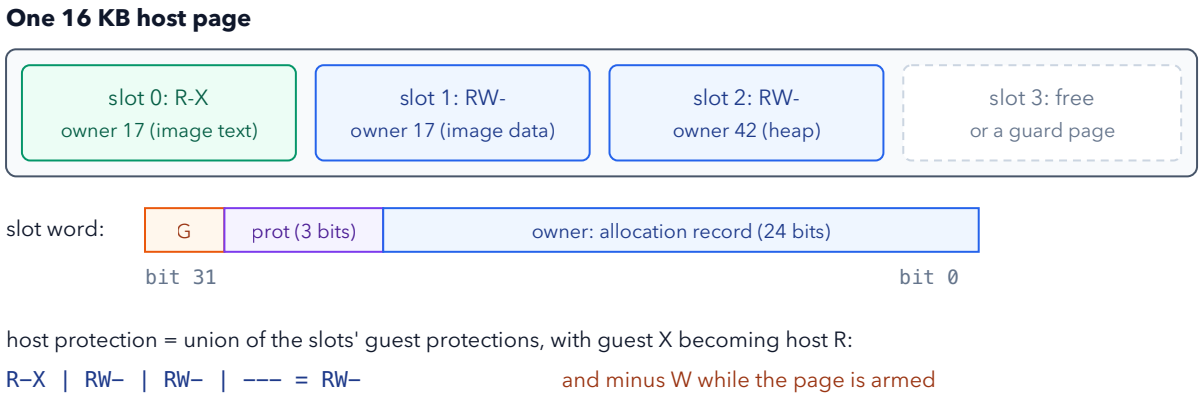


Figure 8.3. A host page shared by four guest slots. Each slot word records its guest protection and the allocation it belongs to; a guard bit marks the uncommitted guard pages that separate allocations. The host page gets the union of its slots' protections, never guest execute, and loses write while code translated from it is live.

Several rules follow from that layout.

- **Guest execute is never host execute.** The host never runs guest bytes. Code is only read, by the decoder, the interpreter and the JIT, and translated code lives in its own `MAP_JIT` arena. A guest `PROT_EXEC` becomes host `PROT_READ`.
- **Protection changes land on 16 KB boundaries.** If one slot of a host page is read-only and its neighbour is writable, the host page is writable, so a stray write into the read-only slot is not caught. This is a documented limitation; making it exact would need a fault on every write to every mixed page.
- **Allocations carry guards.** An allocation the arena places itself ends on a host page boundary and is followed by one whole uncommitted 16 KB guard page, which its allocation record remembers, so two such allocations never share a host page and an overrun past the end of one faults.
- **The guest always sees 4 KB pages.** In cache mode `hw.pagesize` and `vm_page_size` are overridden. The override mirrors the real node's width: `hw.pagesize` by name is a 64-bit node and the legacy `{CTL_HW, HW_PAGESIZE}` MIB a 32-bit one, and writing 4 bytes into the 8 a caller offered left `sysconf(_SC_PHYS_PAGES)` returning 0, which killed `sort`. In native mode, since 2026-09-21, `getpagesize`, `sysconf(_SC_PAGESIZE)`, `host_page_size` and the page-size sysctls answer 4096 too, because V8 checks the page size against PartitionAlloc's compile-time constant.

Mapping in bulk

The general mapping path walks a range one slot at a time, several times over, and V8 reserves and releases gigabytes at startup. Since 2026-09-23 a mapping onto slots that are all free, starting on a host page, with no shared or armed page in reach, takes exactly one host `mmap` of the data pages and fills the slot array directly, because that mmap already leaves the memory zeroed, committed and at its final protection. A replacing mapping over nothing qualifies too, and that is every mapping at an address hint. Unmapping releases slots in runs of one owner, and a fully free unshared range is made inaccessible with one `mmap`. Reserving and releasing 1 GB went from 1.8 ms to 0.45 ms.

Zero runs

Wine's PE loader re-commits over live pages constantly, and the general path costs a `memset` and two `mprotect` calls per host page. A run of whole host pages that must all end up zeroed is replaced with one fresh anonymous mapping. Pages shared with another process are excluded, and so is one sibling slot: the dispatcher slot at `0x7ffe1000` described above. Zero-filling it for one process zeroed it for all of them, and until that process stored its pointer again every syscall elsewhere called address zero; Steam's CEF children died that way on 2026-09-06.

Mach mappings at 4 KB granularity

`mmap` goes through the slot machinery, but `mach_vm_map` is a message to the kernel, which works in 16 KB pages. A `FIXED|OVERWRITE` request at a 4 KB-aligned but not 16 KB-aligned address replaces whole host pages and takes the guest bytes on either side with it. The syscall layer copies those fragments out before the call and back after it. Safari asked for `0x4000` bytes at a `...e000` address inside the shared cache and lost the 8 KB below it, which held a constant a Swift precondition later checked; the program trapped one framework away from the cause.

Files and shared memory

Private file mappings

The host cannot map a file at every 4 KB offset, so a `MAP_PRIVATE` file mapping used to be read into anonymous memory in full. Since 2026-09-05, when the file offset and the guest address agree modulo 16 KB, which they do for nearly everything Wine maps (PE images at 64 KB-aligned bases, fonts, resources), the aligned interior is a real private mapping of the file, and its pages sit in the shared page cache rather than being copied per process. Partial head and tail pages, and anything past the end of the file, are still read in. A GUI Wine process had been carrying 280 MB of such copies, and one WineMine run read 5 GB of a single font file while enumerating fonts. `OCERZ_NO_FILEMAP=1` restores full reads.

Shared mappings

A read-only shared mapping that is not 16 KB-aligned used to become a private snapshot. Steam's UI stream is exactly that: `steam.exe` maps a ring buffer read-only while the browser process writes it. A snapshot never saw the writes, Steam re-created the stream forever, and no window appeared. The real shared overlay is now tried first, and the private copy is only the fallback. Writable shared mappings are communication channels between processes, so guest memory backing them keeps x86 ordering, and mapping one switches the JIT to ordered mode (Chapter 7).

Moving a mapping into the guest used to switch the process to ordered memory as well. AArchX moves a mapping that a Mach VM reply placed outside the guest arena into the arena, in case it was shared with another process. Most are not: libsystem_trace maps a private, purgeable 256 KB buffer through the MIG `mach_vm_map`, and a read-only copy-on-write table, in nearly every program that logs, so a single-threaded program ran with every load an acquire and every store a release. 618d540f (2026-10-09) has `host_range_shared` look at every region in the range: private regions, regions aliased only within this task, empty regions, holes and copy-on-write regions without write permission have no other observer, and anything else, or a range that cannot be inspected, still asks for ordered memory. SQLite inserts and a query went from 0.578 s to 0.417 s (Rosetta 0.308); with CoreFoundation linked, 0.612 s to 0.479 s. `OCERZ_NO_REMAP_PLAIN=1` restores the old rule.

System V shared memory

`shmat` and `shmdt` were not in the syscall table until 2026-09-22, so a guest got `ENOSYS` and continued with a null pointer. Brawlhalla, which talks to the Steam client through a System V segment, crashed at address zero. The host kernel refuses a fixed address for `shmat` even over ground the arena has reserved, so the segment is attached wherever the host wants it and `mach_vm_remap` carries that mapping into guest space as a share, not a copy, so writes from the other process show through. The host attachment is kept, so `shm_nattch` counts the process once, and `shmdt` reverses both halves. A guest-supplied address is honoured when it falls on a host page (`SHM_RND` rounds it down to one); anything else is `EINVAL`. Attaching switches the JIT to ordered mode, because a segment two processes share makes their ordering observable. The dynamic test `sysv_shm` uses a child process on purpose: a copy instead of a share would pass every check a single process can make.

Armed pages and kernel writes

A page that code was translated from is **armed**: its host protection loses write. A later guest store into it, from a JIT such as V8 rewriting its own code with plain stores into read-write-execute pages, faults; the stale translations are dropped, the page is unarmed, and the store retries. Only code in a guest-writable slot can be rewritten by a store, since code in a read-execute slot changes only through `mprotect` or `mmap`, which invalidate on their own. So a page is armed only when a writable slot overlaps the translated bytes. That leaves Wine's PE images alone where a `.text` tail shares a host page with a writable section. Chapter 7 covers the JIT's side of this.

The opposite direction matters as much. A kernel copyout onto a read-only page does not fault; it fails, and for a Mach receive the reply is destroyed with it. wineboot hung forever on exactly that on 2026-09-07. So before any syscall whose buffer the kernel writes, such as a read or a Mach receive, the syscall layer unarms every armed page overlapping the buffer, and an `EFAULT` with armed pages about is retried once after unarming. A counter of currently armed pages lets the whole check be skipped when it is zero, which is almost always.

What the guest is told about its memory

A program that calls `mach_vm_region` on its own memory is asking about guest pages. Until 2026-09-22 only the low-shadow map rewrote the answer; the identity map returned the host's view, in which a whole arena is one read-write-execute region, so an image's `__DATA_CONST` read back as writable. Electron checks a region it expects to be read-only and executes an `int3` otherwise, which is what killed Discord's renderers. The reply is now rebuilt from AArchX's own slot table in every map, by `ocerz_guest_vm_region`; an address AArchX has no mapping for still gets the host's reply, since there is no other answer to give.

For the table to hold the right answer, the loader has to apply each segment's declared protection: `initprot`, with `SG_READ_ONLY` segments made read-only. It cannot do that at load time, because Objective-C still writes into those segments, so each image is queued and the batch protected once Objective-C is done with it. Cache mode now reports 5, 1 and 1 for `__TEXT`, `__DATA_CONST` and `__LINKEDIT`, as a real system does. `max_protection` still reads 7, because AArchX keeps no per-segment maximum. Chapter 11 has the loader's side.

Two diagnostics watch this boundary. `IMAGE-CLOBBER[pid]` is printed, always, when a guest fixed-address `mmap` or a `munmap` lands on a segment of a loaded image, or an `mprotect` takes read or execute away from an executable one; a library unmapped under running code otherwise surfaces only as a wild jump much later. `OCERZ_MAPFAILLOG` names which test in the fixed-mapping path refused a guest `mmap(MAP_FIXED)`, because an `ENOMEM` there is indistinguishable from real memory pressure, and Chromium's allocator treats a refused 4 KB commit as out of memory.

Host memory inside the guest's world

In the identity map, a pointer the kernel or a framework hands the guest is simply usable. Under the low shadow it is not: a host pointer below 12 GB names, from the guest's point of view, the guest's own memory at that number. The kernel and the frameworks produce such pointers constantly. An IOSurface's

per-client page is mapped into the task by the kernel and handed over inline in an IOKit reply. Metal hands back buffer addresses. A Mach reply can carry an address the kernel chose. Most of the work of making Wine's graphics run has been in this one area.

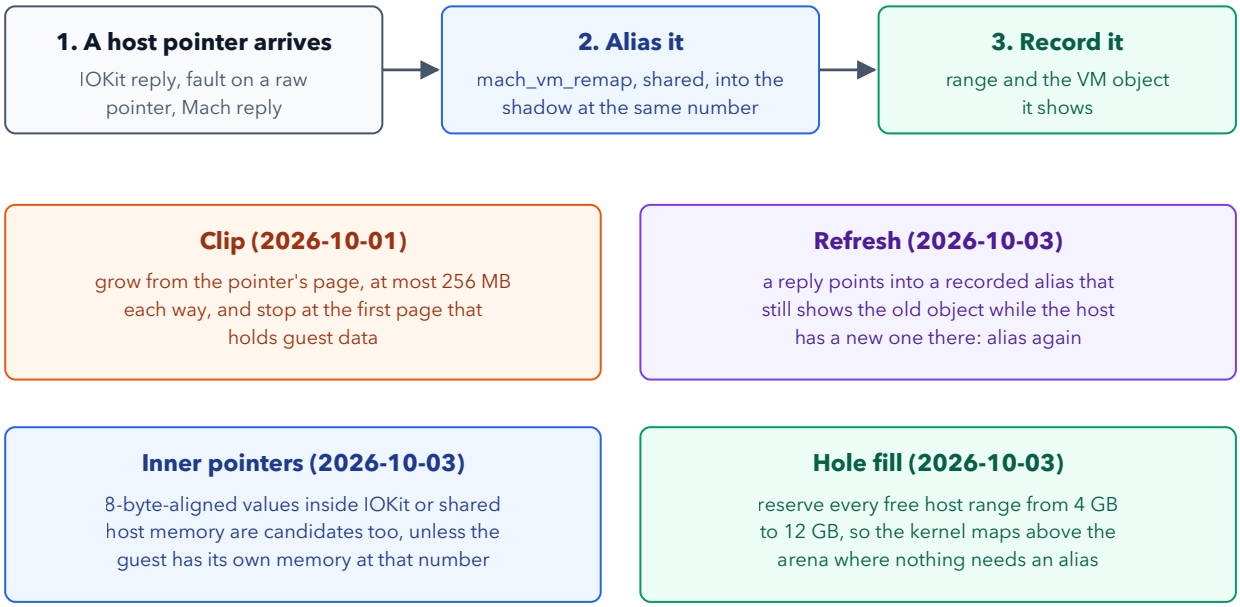


Figure 8.4. The life of an alias under the low shadow. A host region a pointer names is shared into the shadow window at the same number so the guest can read it. Four refinements, all driven by Wine graphics workloads, decide how much is aliased, when an alias goes stale, which values count as pointers, and how to stop the kernel creating the problem in the first place.

Aliasing

`alias_raw_region` in `src/syscall.c` asks the host for the region behind the pointer, maps the matching guest range, and shares the host pages into the shadow at the guest's number with `mach_vm_remap(..., VM_FLAGS_FIXED | VM_FLAGS_OVERWRITE, ..., copy = FALSE)`. The pages are shared, not copied, so the guest sees what the kernel and the GPU see. An alias also switches the JIT to ordered mode, because the other side of shared host memory is a second observer.

Clipping. The first version aliased the whole host region. A pointer into the host's arm64 shared cache once brought two gigabytes of it down on top of Wine's loader, whose header a framework then walked as four billion load commands until Chromium's GPU watchdog fired. Since 2026-10-01 the alias grows outward from the pointer's own page, at most 256 MB each way, and stops at the first page that holds guest data; a pointer whose own page holds guest data is not aliased at all. `OCERZ_NO_ALIAS_CLIP=1` restores the old reach.

Refreshing. The host does not keep its memory where it was. Metal frees a buffer and maps the next one at the same address, and an alias kept the freed buffer's VM object alive: the guest read the previous compute job's results, Metal's own completion state went stale with them, and a job could wait forever. R.E.P.O. got a nil drawable and aborted. Each alias is now recorded with the VM object it shows, and when an IOKit reply hands back a pointer into a recorded alias that still shows that object while the host has a different one there, the region is aliased again. An alias the guest has since covered with its own memory shows a different object and is left alone, and an alias of memory that has no VM object yet is not recorded, because fresh guest memory would look the same. `OCERZ_NO_ALIAS_REFRESH=1` turns it off.

Inner pointers. Creating an IOSurface answers with the surface's pixels and with two records inside memory the kernel shares with the process, at addresses that are not page-aligned. The reply scan took page-aligned values only, so those records were read through whatever the shadow held at their number. R.E.P.O.'s `CAMetalLayer` then fetched a drawable's properties into a buffer of garbage size, four gigabytes with the right size in its low bits, the kernel refused with `kIOReturnVMError`, and the game aborted in about two launches of three. An 8-byte-aligned value inside IOKit or shared host memory is now a candidate as well, skipped when the guest has its own memory at that number. In fifteen launches after the change none failed. `OCERZ_NO_IOKIT_INNER_PTR=1` restores page-aligned values only.

Keeping the kernel out. The identity arena starts at 12 GB, so the kernel placed its own mappings in the free host space below it, which is where Wine and Unity keep their memory too. IOSurface's shared records landed on numbers the guest already used, where they could not be aliased, and R.E.P.O. failed to start in three launches of five. Since 2026-10-03 a low-shadow process reserves every free host range between 4 GB and 12 GB, at startup and again after each alias, so the kernel maps above the arena instead, where guest and host addresses are equal and nothing needs an alias. Eight launches of ten then reached the menu with no wild pointer at all. `OCERZ_NO_LOW_HOLE_FILL=1` turns it off.

Mach VM requests

Three more rules keep Mach memory calls honest under the low shadow.

- A `mach_vm_map(ANYWHERE)` is placed by hand at a guest-visible identity address. Relocating the kernel's answer afterwards would leave dangling host pointers wherever the kernel recorded the original, as IOSurface's bulk-attachment page showed. When AArchX does relocate an `ANYWHERE` reply, the requested alignment is honoured.
- A `FIXED` request without `OVERWRITE` is how Wine asks whether a range is free. The kernel only knows the host's numbers, so a range the guest already used looked free, and Wine mapped over its own loader at `0x200000000`. Since 2026-10-01 such a reply that lands on committed guest pages is given back and answered `KERN_NO_SPACE`, as the kernel would have answered (`OCERZ_NO_VMMAP_TAKEN=1` turns it off).
- A Mach reply must never land in the shared cache: moving it would deallocate a range every image is linked against, and Safari died one `mach_msg` later when the next routine in it faulted.

Requests aimed at another task, the way the wineserver reads and writes its clients, are covered in Chapter 9.

The Wine layout's guard and its hoist

In the Wine layout a guest address below 12 GB has `low_base` added, so every translated memory operand carries a shift, a compare, a branch and an `orr`. Most of the work on Wine's speed in October went into paying that once instead of per access.

Two guards, narrowed

Every access used to test three ranges: the low window, the identity middle and the top strip, the 32 MB below the commpage that arm64 cannot map at its guest address. Only the first two are common. Since 1b325b73 (2026-10-04) a block tests the low window alone. An access that does reach the top strip lands on its identity address, above the highest address an arm64 process can map, and faults. The handler

marks the block the way identity mode marks a commpage reader, interprets that one instruction and retires the block, and the retranslation carries the full test. `ocerz_host_in_guest_space` counts that band as guest space in the Wine layout, so the fault reaches the handler. `OCERZ_LOW_TOP_GUARD=1` tests the top strip everywhere again; the xbench memcpy kernel went from 0.114 s to 0.101 s.

> **HOW IT WAS FOUND**
>
> A second narrowing was tried and reverted. 4450788a (2026-10-04) made the host block behind the low window 16 GB, though the guest could own only the first 12, so that one `tst` of bits 34 and up told low from identity addresses and a guest address from 12 to 16 GB faulted at its own address, as under Rosetta. It measured faster on xbench (memcpy 0.089 s to 0.074 s). 85687480 (2026-10-05) reverted it: the design rested on the guest never owning memory between 12 and 16 GB, and on macOS 26.7.1 it does, because the host places IOSurface and Metal memory there and the guest reaches it at its own address. With the 16 GB window those addresses translated into the reserved part, and the tests `dmetal_nocopy_low` and `diosurface_low_stack` died with an alignment fault inside CarbonCore, the path a Wine game drawing through DXMT takes. macOS 27 placed that memory elsewhere, which is why the gate on it passed. The window is 12 GB on the host again and an address from 12 to 16 GB is identity.

Checking a base once

The remaining cost is per access, so the JIT checks a base register once for a stretch of code and lets every access on it carry only its displacement. The pieces arrived between 2026-10-08 and 2026-10-11:

- **32-bit blocks** (ff7ea1d3, 2026-10-08) keep `low_base` in `x0`, which a 32-bit block has no other use for in this layout, so a stack slot is one register-offset access `[x0, w, uxtw]`. `OCERZ_NO_M32_LOWREG=1` goes back to the `orr`.
- `select_low_hoist` picks the base registers with the most accesses before the block first writes them and tests each once, at the head of a loop, over the whole span the block reaches from it. The test is flag-free, because a block may be entered with the guest's flags live in NZCV. If the test fails the block leaves before running anything, C retires it and remembers its key, and it is translated again without the hoist. 7394b557 (2026-10-08) made "lost its hoist to a bail" a learned translation in the store, which the verify run counts as a variant. ff3e02ea (2026-10-10) raised the limit to three bases (`JMEMBASE`, `JMEMBASE2`, `JMEMBASE3`), covered operands indexed by one register and scale, and added a wide form for a base on either side of 12 GB and an index of either sign; a block is translated first for the usual case, and a failed check retires it and retranslates it wide, and only when that fails does it go without. xbench's fpvec, three arrays under one index, went from 2.63 to 1.07 times its time outside the Wine layout, and mixed from 1.42 to 1.18 (2026-10-10).
- `wine_base_ea` (41a5b5e9, 2026-10-10) handles the operands the hoist does not cover. It guards the base register instead of the whole address and leaves the host address in `x15` (the base itself above 12 GB, plus `low_base` below it); the access carries the displacement, and a later operand on the same base is the access alone while the block writes neither register nor calls out. Lua 5.4 on a mixed script went from 0.870 s to 0.826 s (Rosetta 0.625 s). `OCERZ_NO_WINE_BASE=1` turns it off.
- **Indexed operands** became one register-offset access (8365b47d, 2026-10-10), and e8a26292 (2026-10-11) lowered the number of operands sharing an index that earn a check once a trip from three to two, which 7-Zip's LZ copy loop and byte compare needed. `OCERZ_LOW_HOIST_SHARE=<n>` asks for n.

The wide form checks for the top strip only in blocks marked for it (32481ce6, 2026-10-10), the same lazy rule as the per-access guard. `OCERZ_NO_LOW_HOIST=1` turns the whole hoist off, `OCERZ_NO_LOW_HOIST_INDEX=1` leaves indexed operands out, `OCERZ_LOW_HOIST_WIDE=1` starts every block wide, and `OCERZ_LOW_HOIST_LOG=1` names each block that takes the hoist, moves to the wide form or gives it up. In one test, 316 blocks used to bail and be translated a second time, nearly all shared-cache code holding addresses above 12 GB; after ff3e02ea 17 did.

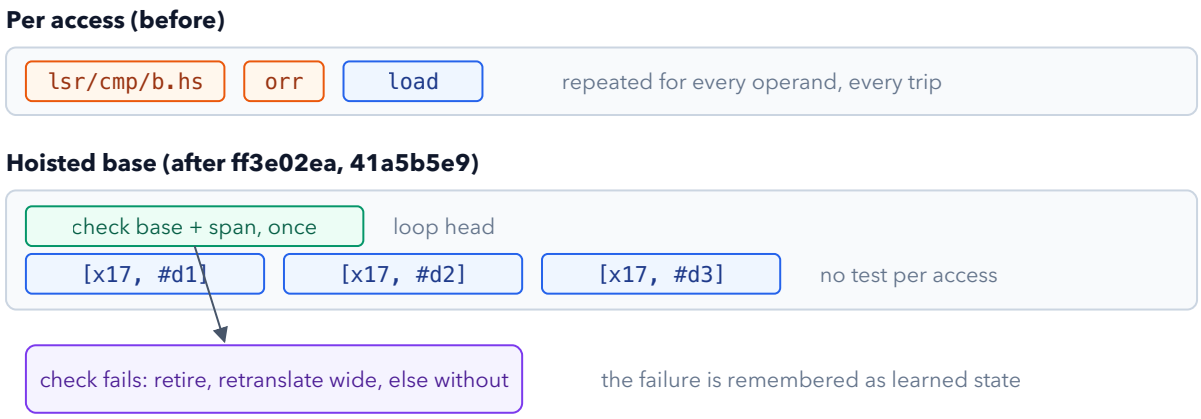


Figure 8.5. The Wine layout's hoist. Instead of testing each access, a block tests its base register once over the span it reaches; a failed test retires the block and it is translated again in a wide form, or without the hoist.

Margins round the low window, which would let the hoist skip even the span check, were tried on a branch and are not part of AArchX.

The translation store

The on-disk store of translated blocks (`src/tcache.c`, Chapter 7 for what a record holds) shares the machine with the guest's memory, and until October it did not look at the disk. A store grew to 4 GB, stores touched in the last hour were never pruned, and a Wine session uses several, one per memory mode, with each build adding more. A tester's Mac with a few GB free filled up under a Steam session, could no longer swap, froze and panicked on the watchdog; the Studio held 6.6 GB in 80 stores with 777 MB left. 93b919f9 (2026-10-07) added a free-space floor:

- nothing is written while the volume has less than `OCERZ_TCACHE_MIN_FREE_MB` free, 10240 by default, checked before every append;
- a process writes at most half of what was free above that floor when it opened the store;
- a lone process that finds the volume already under the floor removes the store, and the prune of other stores then keeps none older than an hour.

Reading is unaffected, so a full disk costs warm starts, not correctness. The test `dtcache_floor` runs with a floor no disk can meet and requires the program to work and no data file to appear; the other store cases run with the floor at 0.

Two later changes made loading cheaper. Every record is summed before it is trusted, and the sum was a chain of dependent multiplies, about 2 ms of a CoreFoundation program's launch for the 9 MB of records it loads; a02fcc3f (2026-10-09) replaced it with CRC32C in three lanes, so the instructions overlap, and the same launch spends 0.9 ms summing. Each build keeps its own store, so no record written with the old

sum is read with the new one. And 94614e9c (2026-10-09) keeps the shared cache's weak-definition name filter beside the store: building it takes 37 ms (990,000 names from 308 images), it depends on nothing but the cache, and every process that made 128 weak lookups built it again. It is stored under the cache's UUID with a sum, written to a temporary name and renamed into place, and it obeys the store's switches: with `OCERZ_TCACHE=off` nothing is read or kept, and under the floor nothing is written. `ollama --version` went from 0.217 s to 0.183 s.

Native mode

In native mode the guest's heap is the host's: native `malloc` and native Mach calls hand the guest memory that no slot table has ever seen. `ocerz_mem_overlaps` answers whether a range touches memory AArchX tracks, and a range wholly outside it goes to the host kernel, after any translations of it are dropped, instead of being refused. `mach_vm_map` and `mach_vm_remap` cross too, with translations dropped first for a fixed target in the process's own task, and `mach_vm_map` never asks the kernel for execute.

The guest also sees views of the host's malloc zones rather than the zones themselves. Chromium writes its own hooks over the default zone's function pointers; those hooks are x86 code, and if they landed in the real zone the next native `calloc` would jump into x86 code. `malloc_default_zone` and zone enumeration therefore hand out views, and the host's real zones are never written. Chapter 12 covers the rest of native mode.

A native process can also run under the low shadow, which is how MacNdCheese's Wine runs in native mode, and then host pointers below 12 GB are handed to the guest constantly (2d59b306, 2026-10-06). Readable host memory there (the main image, the first heap, the main stack, the shared cache) is aliased into the shadow at the same offset and pinned: `ocerz_g2h` answers the host address for a pinned page, fixed maps over it are refused, and a forked child aliases it again. The free host space left there is reserved once pinning is done, so a later host allocation cannot land where the guest sees the shadow. A value in the first 64 KB crosses unchanged, because Objective-C's empty autorelease-pool token is 1 and its shadow alias crashed the pool pop; df5c102a (2026-10-06) made that pass-through native mode's alone, since translated code in cache mode sends that range through the shadow and the two sides would disagree. A 32-bit program's `mach_vm_region` walks only its 4 GB window (8f8bb942, 2026-10-12), because the walk over the host's regions never ended.

Footprint

Steam under Wine is nineteen emulated processes, so per-process memory matters. In early September a GUI Wine process cost about 1.1 GB: roughly 360 MB of JIT metadata, 300 MB of arena, and 280 MB of private copies of files it had mapped. Two changes took most of that away. The file interiors described above now come from the shared page cache, and a published translated block keeps a 16-byte reference per instruction instead of a full 96-byte decode, plus exactly the branch edges it has rather than a fixed table of eight. A WineMine process went from 1.1 GB to 714 MB and the Wine explorer from 860 MB to 611 MB, with the benchmark results unchanged. The test watchdog measures a process group's physical footprint rather than resident size, because resident size counts the shared cache again in every process.

Switches

Variable	Effect
`OCERZ_NO_FILEMAP=1`	read every private file mapping into anonymous memory
`OCERZ_NO_ALIAS_CLIP=1`	alias a host region's whole extent
`OCERZ_NO_ALIAS_REFRESH=1`	never re-alias a stale host region
`OCERZ_NO_IOKIT_INNER_PTR=1`	take only page-aligned pointers from IOKit replies
`OCERZ_NO_LOW_HOLE_FILL=1`	leave free host space below 12 GB to the kernel
`OCERZ_NO_VMMAP_TAKEN=1`	let a non-overwriting fixed `mach_vm_map` land on guest pages in use
`OCERZ_NO_VMMAP_STEER=1`	let `mach_vm_map` place mappings anywhere in host space
`OCERZ_NO_TEXT_RO=1`	leave image segments at the protection they were mapped with
`OCERZ_NO_LOW_HOIST=1`, `OCERZ_NO_WINE_BASE=1`	no base-register hoist in the Wine layout; no once-per-base guard
`OCERZ_LOW_HOIST_LOG=1`	name each block that takes, widens or gives up the hoist
`OCERZ_LOW_TOP_GUARD=1`	test the top strip in every Wine-layout access
`OCERZ_NO_REMAP_PLAIN=1`	ordered memory on every relocated mapping
`OCERZ_TCACHE_MIN_FREE_MB=<n>`	free space the translation store leaves on its disk (10240)
`OCERZ_MAPFAILLOG=1`	name the test that refused a fixed mapping
`OCERZ_SYSFAIL=1`	log every guest `ENOMEM` and `EFAULT`, the signature of a missing pointer translation
`OCERZ_MIGTRACE=1`	log aliases, IOKit reply scans and collisions

CHAPTER 9

System calls, Mach, threads and signals

Everything the guest believes about the operating system is answered by AArchX. Most guest system calls cannot be passed to the kernel as they are: they carry guest pointers, structures whose layout differs between the architectures, thread ports that must name guest threads, or results the guest has to see the way Rosetta presents them. This chapter covers the boundary in `src/syscall.c` and the process machinery in `src/vm.c`: dispatch, the return convention, Mach messages, threads and the workqueue bridge, thread suspension, signals, faults, and processes.

8,967
LINES IN SRC/SYSCALL.C

4,613
LINES IN SRC/VM.C

375
BSD TABLE ENTRIES, 59 INTERCEPTED

53
MACH TRAP NUMBERS HANDLED

How a guest enters the kernel

A guest `syscall` instruction ends its translated block, and the JIT, or the interpreter, calls `ocerz_handle_syscall` with the guest's register file. On macOS the x86-64 convention puts a class in the top byte of `rax` and the call number in the low 24 bits, so the first thing the handler does is split them.

Class	Meaning	Handler	Returns through
1	Mach traps	`dispatch_mach`, a switch over 53 trap numbers	`rax`, a `kern_return_t`
2	BSD (unix) calls	`dispatch_bsd`, a table of 375 entries (room for 600), 59 with their own handler	`rax` and `rdx`, carry flag on error
3	machdep calls	`dispatch_machdep`: `thread_set_tsd_base`, and `i386_set_ldt` / `i386_get_ldt` for WoW64	`rax` only
other	not a macOS call: a Windows system call issued directly by PE code	delivered to the guest as `SIGSYS`	the guest's handler

The last row is how Wine works on macOS: PE code that issues a raw Windows system call is caught by Wine's own `SIGSYS` handler. Before the signal is delivered, a thread whose segment base is not on its Wine thread block is switched to it, because that handler runs on the Windows side.

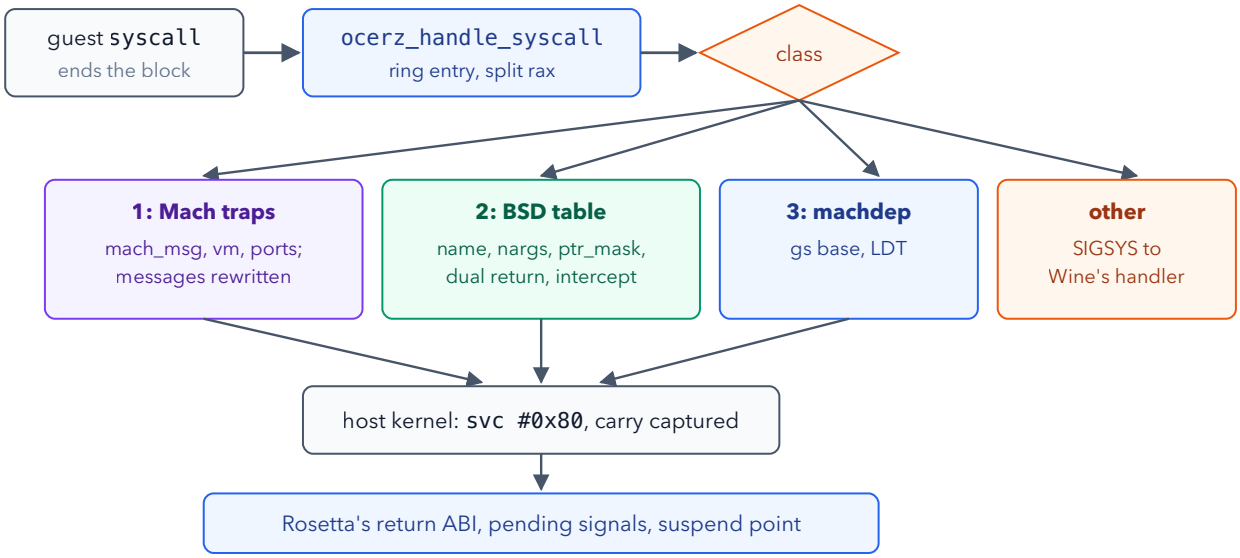


Figure 9.1. A guest system call. The class in the top byte of `rax` picks a dispatcher; most calls end in a raw `svc #0x80` from `sys_raw.h`; on the way back the result is written the way Rosetta writes it, signals the guest mask allows are delivered, and a pending suspension is honoured.

The raw kernel entry

`include/ocerz/sys_raw.h` is the one place AArchX enters the arm64 kernel without libSystem. It loads eight arguments into `x0` to `x7`, the number into `x16`, issues `svc #0x80`, and reads the carry flag with `cset ..., cs` in the same asm block, so nothing can change the flags between the trap and the test. A Mach trap passes its number negated, as it does natively.

The BSD table

Each entry in `bsd_table` carries the call's name, its argument count, a `ptr_mask` whose bits name the arguments that are guest pointers, a flag for calls that return a second result in `rdx`, and an optional intercept function. Arguments come from `rdi`, `rsi`, `rdx`, `r10`, `r8` and `r9` (`r10` rather than `rcx`, because `syscall` overwrites `rcx`), and a seventh and eighth are read from the guest stack. Call number 0, the indirect `syscall(2)`, shifts every argument down one register and re-dispatches. A plain entry is forwarded with its pointers translated through `ocerz_g2h`; an intercepted entry runs AArchX's own implementation. 76 calls became plain pass-throughs on 2026-09-13, when real applications started asking for them.

In the identity map, translating a pointer changes nothing, which is why a few calls whose structures hide further pointers that `ptr_mask` cannot reach (`connectx`, `sendfile`, `recvmsg_x`) are correct only there. Outside identity mode they return `ENOSYS` rather than let the kernel read or write the wrong memory.

The return convention is Rosetta's

The guest's libSystem stubs were written against what XNU does on an Intel Mac, and the applications AArchX runs were tested under Rosetta. So results are returned the way Rosetta returns them, measured with probes on 2026-09-03:

- A unix call returns through `rax` *and* `rdx`, on success and on error alike, with `rdx` zero unless the call fills it. A caller whose wrapper forgot to declare `rdx` clobbered sees zero there, as on Rosetta.

- The arithmetic flags come back in a fixed pattern whatever the caller had: `PF|AF` on unix success, `CF|ZF|PF` on unix error, `AF` after a Mach trap. Only `CF` is meaningful, since the stubs branch on it, but a program that reads the rest sees what it would see under Rosetta.
- Machdep calls are the exception and return through `rax` alone, leaving `rdx` and the flags untouched. Wine's syscall dispatcher keeps its frame pointer in `rdx` across `thread_set_tsd_base`, and applying the unix rule there gave every Wine process a NULL dereference at its first return to PE code (fixed 2026-09-04, guest test `machdep_rdx`).

Failure policy

An unimplemented call reports itself once and returns `ENOSYS`; an unknown Mach trap returns an error. In June the layer aborted instead, which killed the guest thread where it stood, and under Wine that is often inside the loader holding its lock, which deadlocks every other thread. `OCERZ_STRICT_SYSCALL=1` restores the abort for bring-up work. One more rule lives here: a contended `os_unfair_lock` whose owner died is broken the way a robust mutex is, because the system's reaction to the kernel's `EOWNERDEAD` is to crash the process, and that took Steam's web helper down mid-startup in August.

Pointers the kernel reads and writes

Three kinds of argument need more than `ptr_mask`.

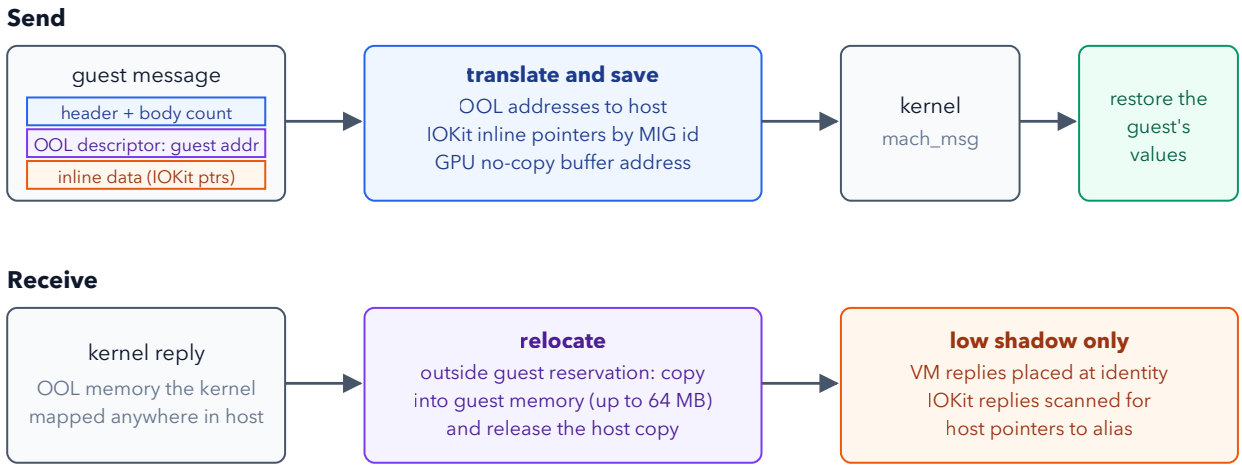
Buffers the kernel writes. A page that code was translated from has lost write on the host (Chapter 8). A kernel copyout onto such a page fails instead of faulting, and a Mach reply is destroyed with it. Before a call whose buffer the kernel writes, every armed page overlapping that buffer is unarmed, and an `EFAULT` with armed pages about is retried once.

Structures whose layout belongs to someone else. `__mac_syscall` carries a policy name and an argument structure whose layout belongs to the policy (Sandbox, Quarantine and others), and that structure points at more memory, usually the caller's out-structures and locals on its stack. In the identity map that works by accident. Under Wine the Unix-side code runs on a stack in the low-shadow window, the kernel received guest addresses, and `sandbox_check` failed with `EFAULT`. The Metal OpenGL renderer took that as no access to the GPU, every `CGLChoosePixelFormat` from a Wine thread returned 10002, wined3d found no GL adapter, and Counter-Strike 2 stopped at an error box. Since 2026-09-26 each of the structure's first twelve words that points from 64 KB below to 1 MB above the caller's `rsp` is rewritten to its host address for the call and put back before the guest runs again. Any other word is left alone, because an integer that happens to look like a low address must not change. dxdiag under Wine then reported the real GPU with an OpenGL 4.1 core context. The dynamic test `mac_syscall_low_stack` pins it.

Information the host has and the guest must not see. `sysctl` is answered with the guest's view: 4 KB pages, at the width of the node the caller asked through, and an x86 CPU description synthesized to agree with AArchX's own CPUID (SSE through SSE4.2, `CX16`, `POPCNT`). The arm64 kernel has no `machdep.cpu` nodes at all, so passed through they failed with `ENOENT`, and code that asks `hw.optional.sse4_1` before taking a fast path took none (dynamic test `x86_sysctl`). `proc_pidpath` and `proc_name` of the process itself name the guest executable, not `ocerz`.

Mach messages

A Mach message is a header followed, in a complex message, by descriptors for ports and out-of-line memory, then inline data. The kernel copies out-of-line memory by address, so every address in a message has to be one the kernel can use, and every address the kernel puts in a reply has to be one the guest can use. In the identity map most of this is free. Under the low shadow, and for a handful of IOKit calls that hide addresses inside their inline data, it is most of the work.



Never rewritten: MACH_RCV_INTERRUPTED on a combined send and receive (turning it into a timeout made CFRunLoop exit). Never placed: a reply inside the shared cache.

Figure 9.2. Translating a Mach message. On send, every guest address the kernel will dereference is replaced with its host address and put back afterwards. On receive, memory the kernel mapped outside the guest's reservation is moved where the guest can see it.

On send

`ocerz_send_xlate_descriptors` walks a complex message's out-of-line descriptors and replaces each guest address with its host address, saving the original so it can be restored after the call. Several IOKit calls hide addresses in their inline data, where no descriptor walk can see them, and are recognised by their MIG message id:

- `io_registry_entry_get_property_bin_buf` and `..._properties_bin_buf` (2888, 2889) carry an inline out-buffer pointer. Untranslated, the call failed with `kIOReturnVMError`, and GPU enumeration under Wine failed with it.
- `io_connect_method` and `io_connect_async_method` (2865, 2866) carry `ool_input` and `ool_output` addresses the kernel copies from directly. Untranslated, `IOSurfaceClientCopyValue` failed in every DXMT swap chain (fixed 2026-09-08).
- Metal's `newBufferWithBytesNoCopy` hands the GPU memory the caller owns, by address, in the structure input of one `io_connect_method` selector. The kernel wires the host pages at that address; under Wine it was a low-shadow guest address, so the GPU read and wrote whatever host memory lay at the guest's number, and R.E.P.O.'s first frame died on a GPU address fault. The call is recognised by the structure's shape, since IOKit will not name the class behind a connection port, and the address is translated for the call (2026-10-03). It need not be page-aligned: DXMT's ring allocator hands Metal memory from the Windows heap.

Mach trap 70, `host_create_mach_voucher_trap`, takes its recipes and the place for the new voucher by address and went to the kernel untranslated. From a Wine thread both live on a low-shadow stack, the kernel answered `KERN_MEMORY_ERROR`, and libdispatch, which creates vouchers for the work it queues, crashed on purpose. Its sibling trap 72 already translated its pointers; trap 70 does now too.

A real-time thread's time-constraint policy needs a unit conversion. `thread_policy_set` and `thread_policy_get` carry the period, computation and constraint in `mach_absolute_time` units. The guest sees a 1/1 timebase, as under Rosetta, so x86 code computes them in nanoseconds; for a Rosetta process the kernel converts them, and for AArchX nothing did. The kernel read CoreAudio's 2 ms of computation as 2,000,000 ticks, refused the policy with `KERN_INVALID_ARGUMENT`, and every output unit and audio queue failed to start with error 89, which silenced Wine's audio driver and R.E.P.O. (b52597f3, 2026-10-06). The three values of a `THREAD_TIME_CONSTRAINT(_WITH_PRIORITY)_POLICY` set now go out in host ticks on both `mach_msg` traps, and the request is put back if the send fails, since the guest's `mach_msg` retries an interrupted send with the same buffer. A get reply for those flavors comes back in nanoseconds. `OCERZ_NO_TC_XLATE=1` sends the values as they are and `OCERZ_POLICYLOG=1` logs each conversion.

On receive

Out-of-line memory in a reply is mapped by the kernel wherever it likes in the host. `ocerz_reply_relocate_ool` checks each descriptor: memory outside the guest's reservation, and not in the shared cache, is copied into a fresh guest mapping (up to 64 MB) and the host copy released, and the descriptor is rewritten to the guest address. Replies to Mach VM calls that return an address (`mach_vm_allocate`, `mach_vm_map`) go through `mig_vm_reply_relocate`, and under the low shadow an IOKit reply (ids 2900 to 2999) is scanned for host pointers to alias into the shadow, as Chapter 8 describes. One reply is deliberately never rewritten: `MACH_RCV_INTERRUPTED` on a combined send and receive. Rewriting it to a timeout made `CFRunLoop` exit (2026-08-24).

Requests aimed at another task

The wineserver reads and writes its clients' memory with `mach_vm_read`, `mach_vm_read_overwrite`, `mach_vm_write`, `mach_vm_protect` and `mach_vm_msync` aimed at their tasks. Those name the other task's guest addresses. Since 2026-10-01 a low address is moved by the shadow base the whole Wine tree inherits in `OCERZ_LOWBASE`, and the top strip by `OCERZ_TOPBASE`, before the kernel sees it. Untranslated, Chromium's sandbox read a child's thread block from the wrong memory and every launch of its GPU process failed with sandbox error 43. `OCERZ_NO_PEER_VM=1` turns it off.

Thread identity

`thread_info(THREAD_IDENTIFIER_INFO)` answers with the thread's TSD base, which the kernel reports as the host's. It is replaced with the guest TSD of the guest cpu whose host thread owns that base, the pthread one even while Wine has `gs` pointing at its Windows thread block, found through the host TSD base each cpu records. Wine saves this value and switches `gs` back to it on every system call, and a host thread whose stack happened to land above 12 GB used to pass its own TSD through untouched, so that Wine thread lost its thread block and died at its first server call.

Threads

Every guest thread is a real host pthread running a guest cpu: an `OcerzCPU` structure holding the x86 register file, the segment bases, the signal mask and pending set, and bookkeeping for the JIT. The host thread runs the master loop in `vm.c`, which executes translated code until something needs the C side.

Creating threads

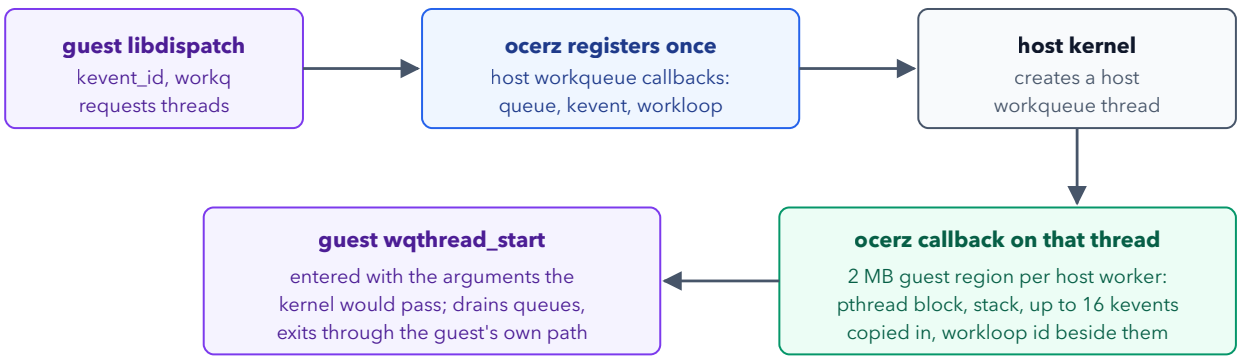
The guest's libpthread starts by calling `bsdthread_register` with the entry points for new threads (`pthread_start`) and for workqueue threads (`wqthread_start`). AArchX records both and answers with a feature word that includes workloop support whenever the host workqueue bridge is on, so that the guest's libdispatch chooses the model the bridge actually delivers.

`bsdthread_create` starts a host pthread and gives it a copy of the creator's cpu, taken inside the creator's own system call. Nothing about the creator's own wait or suspension comes along: a creator suspended by a garbage collector at that moment would otherwise hand the child a suspend count of one, and the child would park at its first safe point with nobody left to resume it. The signal mask is inherited, as POSIX requires. The call blocks until the new thread's kernel port is in its pthread structure, as the real kernel does; without that, a `pthread_join` right after `pthread_create` can read an empty port slot, conclude the thread has exited, and free a live thread's stack. The guest's segment base is set by the machdep call `thread_set_tsd_base`, which updates the cpu's `gs` base.

A process whose main thread AArchX builds by hand has that thread's cached thread id filled in. Without it, `pthread_threadid_np` returned 0, and the mutex protocol that stores that id as the lock owner treated owner 0 as unlocked, so mutexes taken by the main thread had no mutual exclusion at all. That was the Wine explorer freeze of 2026-08-18.

The host workqueue bridge

libdispatch does its work on workqueue threads that the kernel creates on demand, and AppKit hands work to them and waits. Without anything servicing those queues, the first `dispatch_sync` parks the main thread forever: Calculator, TextEdit and Safari all registered as foreground applications without ever reaching the window server. The bridge was opt-in until 2026-09-10 and is on by default now.



Registering retires the plain memory model, since a worker is a second observer of guest memory, and starts the unstick monitor. Native mode leaves the bridge off: the host's own libdispatch needs the process's single workqueue registration.

Figure 9.3. The host workqueue bridge. AArchX takes the process's workqueue registration for itself, and when the kernel starts a worker it builds a guest thread around it and enters the guest's own `wqthread_start`.

On the first guest request that needs it, AArchX registers its own callbacks with `_pthread_workqueue_init_with_workloop`, the libpthread entry point that claims the process's workqueue. When the kernel then starts a host worker and calls back, the callback gives that host thread a 2 MB guest region holding a guest pthread block, a stack and an event buffer, copies in up to 16 kernel events and the workloop id, and enters the guest's `wqthread_start` with the arguments the kernel would have passed. The region is kept for that host thread and reused. A `REQTHREADS` request is always forwarded to the host when the bridge is on (since 2026-09-19). Workqueue threads leave through the guest's own thread-exit path since 2026-09-13; WebKit's allocator could not suspend a thread that had ended without it, which stopped both Safari and the Ollama menu-bar app.

Native mode turns the bridge off outright. Registering claims the process's single workqueue slot, and in native mode the host's own libdispatch is what needs it.

Suspending threads

JavaScriptCore's garbage collector suspends each mutator thread, reads its registers with `thread_get_state` and scans its stack. Chromium's stack sampler does much the same. Handed to the kernel, `thread_suspend` froze an AArchX thread wherever it was, including inside the translator holding the JIT lock, after which every other cpu, the collector included, blocked on that lock for good. And an x86 `thread_get_state` on an arm64 thread fails outright.

So in cache mode `thread_suspend`, `thread_resume` and `thread_get_state` on guest threads are emulated (since 2026-09-11, the main thread since 2026-09-13). A suspend reports done only once the target holds no AArchX lock: parked at a safe point, inside a raw host wait, or stopped by the kernel with its program counter in a translated block, where the pinned host registers are its guest registers. Anywhere else, in the translator, a C helper or the interpreter, it is let go and caught a moment later. `thread_get_state` reports x86 registers reconstructed from the arm64 registers holding them. `OCERZ_NO_THREADACT=1` hands these calls back to the kernel.

In native mode, since 2026-09-21, the calls are bridged and the kernel call is real, because a cooperative stop would leave the suspend count somewhere the caller can tell. What makes it safe is refusing to leave a thread stopped where it would deadlock the caller: while its per-thread critical depth is non-zero, which means it holds the JIT lock or the memory-map lock, or while its program counter, link register or any of twelve frames up its stack is inside the host's allocator, since the suspender's next `malloc` would wait for a lock the target holds. Such a target is resumed and tried again every 100 microseconds, up to 50,000 times.

Signals

AArchX emulates the guest's signal state completely: the handler table, the per-thread mask, the pending sets, the alternate stack, and the frames handlers run on. The host's own signal machinery is used only to receive signals and wake threads.

The host side

Because AArchX emulates the guest's mask itself, the host thread must never block asynchronous signals. The wineserver suspends guest threads and kicks them for asynchronous procedure calls with `SIGUSR1` sent to the host thread, and a host mask that blocked it left the thread suspended server-side forever;

Steam's CEF browser deadlocked that way on 2026-09-06. XNU starts libdispatch workqueue threads with all asynchronous signals blocked, and a pthread created from one inherits that mask, so the mask is cleared whenever a host thread starts running guest code.

A guest `sigaction` for a plain asynchronous signal is also mirrored onto the host, because the host kernel decides what `SIGPIPE` does to the process, and its default is to kill it silently. The wineserver ignores `SIGPIPE` and relies on `EPIPE`; without the mirror the whole server vanished the first time a client died with a reply in flight.

When signals are delivered

A host signal handler only records the signal. Delivery to the guest happens at well-defined points.

- **On the way out of a system call**, every pending signal the guest mask allows is delivered.
- **On the way in**, pending signals are offered too, with `rip` rewound so the call re-executes after the handler. The return edge alone left a signal unnoticed until some later call returned, and if the next call blocked, it parked with the wakeup undelivered.
- **After a bridged call** in native mode, the equivalent of a system call return.
- **Between two steps of either run loop**, where the guest state is all in the cpu. This is the newest point, added on 2026-10-14 (a7fd0f3f), and the next section explains how it reaches a thread that never leaves translated code.

Asynchronous signals obey the guest mask; synchronous faults never come through here and are never gated. Enforcing the mask is what keeps Wine's server protocol in one piece. Wine blocks a signal set around each request and reply to the wineserver, so that a suspend kick lands only between calls; delivering one in the middle nested a second round trip inside the first on the same strictly ordered pipe, until every client parked in `read` forever (2026-08-25).

Since 2026-10-01 a signal that arrives on a thread running a guest cpu stays pending on that cpu, because Wine aims `SIGUSR1` and `SIGQUIT` at one thread and its handler expects to run there. Before, it went to whichever thread reached a system call first. Only a signal that reaches a thread with no guest cpu goes to the process-wide pool, and a thread takes from the pool only the signals its mask accepts. `OCERZ_NO_THREAD_SIGNALS=1` pools them all again.

A self-directed `pthread_kill`, which is how `abort()`, `raise()` and every Chromium CHECK end, is delivered to the guest rather than forwarded. Forwarding raised a real `SIGABRT` on the AArchX process, the system crash reporter started and pegged a core, and under Steam's GPU-process crash loop that starved the emulation past Steam's own watchdogs.

Signals for a thread that never calls

Until 2026-10-14 the delivery points above were the only ones, and a thread spinning in its own code without calls never saw a signal. This held in the interpreter as much as in the JIT: a loop of `dec` and `jnz` with a 5 ms `ITIMER_REAL` got no `SIGALRM` until the loop ended, and a test thread got one or two of 130.

Under Wine it also mattered for suspension, because `SuspendThread` is a `SIGUSR1` sent to the thread: Mono's garbage collector waited for any busy thread to finish its loop. Commit a7fd0f3f (2026-10-14) changed this in three pieces.

1. **The run loops deliver between two steps.** Both the interpreter's loop and the JIT's run loop now deliver a pending signal the thread does not block before the next step. That was all the interpreter needed.
2. **The handler stops the loop.** Translated code reaches the run loop only when its blocks exit to it, and a hot loop can chain its blocks into a cycle that never does. So the host handler calls `ocerz_jit_signal_stop` with the interrupted host program counter. It does to the block the thread is in, and to the blocks that block's chained edges lead to a few hops on, what a retire does: it unchains their chained edges and rewrites their stop sites, so the thread's next jump out of any of them goes to the dispatcher. Unchaining matters because a loop whose backward branch is a side exit chained through its conditional site has no stop site anywhere. The thread records each block and edge it changed.
3. **The dispatch stub returns to C.** The dispatch stub, which block exits jump into and which enters the next block without returning, goes back to the run loop while such a signal is pending. The run loop delivers the signal, and then `ocerz_jit_signal_unstop` puts the recorded blocks back: a stop site returns to the word it was translated with, now kept as `stop_run`, and an edge is chained again the way a chain install would.

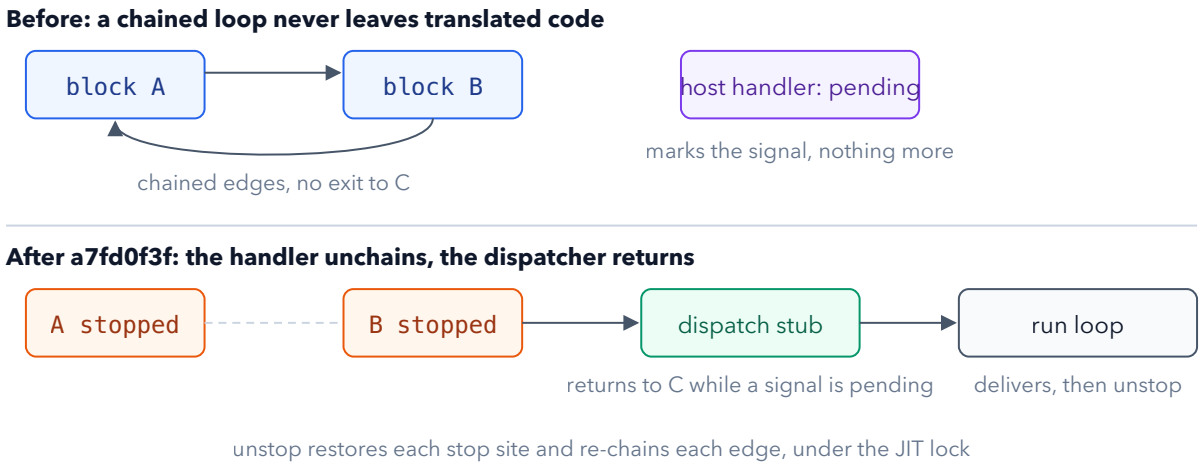


Figure 9.4. Reaching a thread in a translated loop. The handler unchains the blocks the thread is running in, so its next exit goes to the dispatch stub; the stub returns to the run loop, which delivers the signal and restores the blocks.

Details that came from the test failures. The handler takes no lock: in the test the thread holding the JIT lock was the sender, translating its own wait loop, so a try-lock lost the signal. The handler and a chain install can race on a word, but every word either side writes is a whole valid branch, and the restore runs under the lock. The blocks the handler reads stay allocated, because retired blocks are freed only with the JIT. A block whose stop sites are not in their running state is no longer saved to the translation store, since it would come back stopped for good; that could already happen to a block translated after an exit request.

The dynamic test `loop_signal` covers seven loop shapes, each with plain and with ordered memory, under both engines, and a thread that 20 `pthread_kill`s must each reach while it spins. `OCERZ_NO_SIGNAL_STOP=1` goes back to the old delivery. 32-bit Mac programs (Chapter 13) keep that old delivery too, since delivering between steps there would enter a 64-bit handler.

The signal frame

`ocerz_signal_deliver` builds the frame the kernel would build on an Intel Mac, on the alternate stack if the handler asked for it and one is set, otherwise below the interrupted stack pointer after skipping its 128-byte red zone.

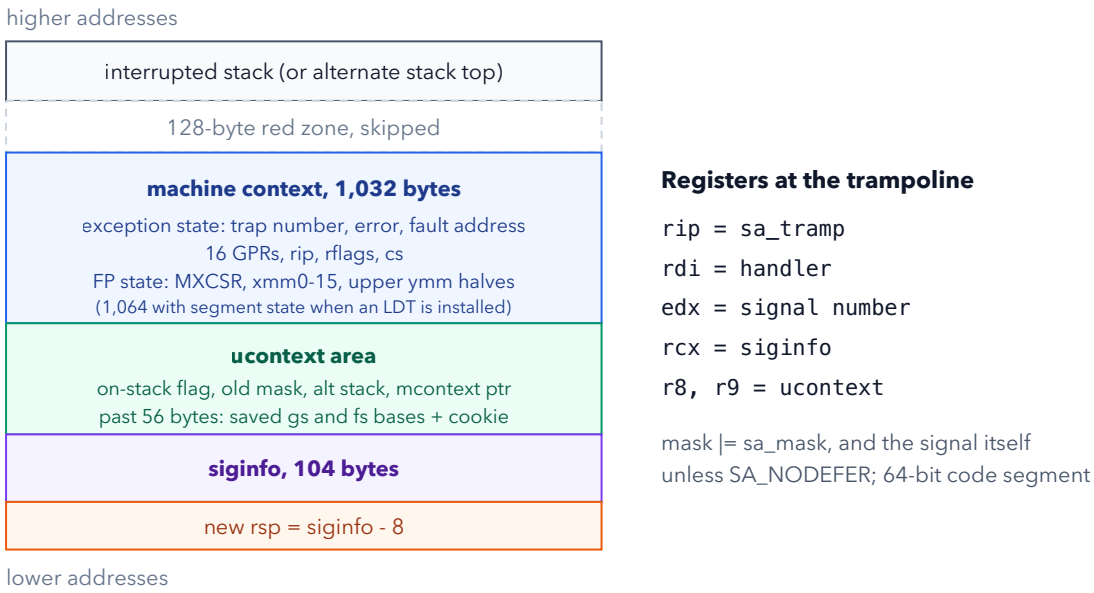


Figure 9.5. The signal frame AArchX builds, in the layout an x86-64 macOS handler expects. The machine context carries the upper halves of the `ymm` registers, so a handler that returns leaves AVX code intact.

The frame layouts are the ones in the public macOS headers: an AVX machine context of 1,032 bytes, or 1,064 with segment state when a 32-bit LDT is installed, a `ucontext`, and a 104-byte `siginfo`. The trampoline the guest's libc registered as `sa_tramp` is entered with the handler in `rdi`, the signal in `edx`, the `siginfo` in `rcx` and the `ucontext` in `r8` and `r9`, and calls the handler and then `sigreturn`. The cpu switches to 64-bit mode for the handler, since a signal can interrupt 32-bit WoW64 code. Frames carry the upper `ymm` halves since 2026-09-13.

Several details have their own history.

- **Two `sigaction` layouts.** The kernel takes a `struct __sigaction`, 24 bytes with the trampoline at offset 8, and returns a `struct sigaction`, 16 bytes without one. Writing the inbound layout to `oldact` overran the caller's buffer by 8 bytes; in `sort` those bytes were a saved `rbx`.
- **`sigreturn` leaves the `gs` base alone.** The real kernel saves and restores the `gs` base across a signal only for a task with an LDT, so a base a handler sets survives the return, and Wine's handler relies on that to install its thread block before it goes back to PE code. AArchX stashes the interrupted bases past the 56-byte `ucontext` of frames it builds, behind a cookie, and had been putting both back, which undid Wine's switch: PE code ran on the pthread base until something read `%gs:0x30`. Since 2026-09-26 only `fs` comes back from the stash.

- **Alternate stacks.** `sigaltstack` reports `SS_DISABLE` when no alternate stack is set, which Go needed, and refuses a stack smaller than 32 KB with `ENOMEM`, as the kernel does.
- **Waiting for signals.** `sigsuspend` and `__sigwait` are emulated against the guest's pending set; passed to the host they waited for a signal the host kernel could never deliver. In native mode they were stubs until 2026-10-05 (19cf543e), because their host functions wait on or read the host's signal state; they now run on the guest's mask and pending set. A `kill` a process sends itself goes to a thread waiting in one of them whose wait accepts the signal, and only otherwise is raised in the caller, as the kernel would give it.

Native mode has no x86 libc in front of these calls, so `sigaction`, `signal`, `sigprocmask`, `sigaltstack`, `raise` and `pthread_kill` arrive as bridged calls and run the same bodies as the system calls, with the register plumbing peeled off, against the same handler table and mask. A raw system call and a bridged call can never disagree. What native mode lacks is the trampoline, so AArchX writes a small x86 one of its own into a guest page and makes it read-execute; it moves the arguments into place, keeps the `ucontext` in callee-saved registers across the handler, and issues `sigreturn` as a system call.

Faults

The host fault handler in `vm.c` is where most of the emulation's hard edges meet. In order, it tries the recoveries that resume the faulting instruction:

1. A first touch of a lazily rebased shared-cache data page: unpack the page and retry (Chapter 11).
2. A fault while the translator reads guest bytes, for a decode or a stored translation's dependency check: abandon the read before anything takes the JIT lock the translator already holds.
3. A raw host pointer the guest was handed inline: alias it into guest space and retry (Chapter 8).
4. An alignment fault in translated code, meaning an ordered access crossed a 16-byte boundary: hot-patch that one access into an alignment-checked form and re-execute (Chapter 7).
5. A host-stack overflow of the return-address shadow: interpret the call and abandon the JIT frame.
6. A store into a page armed because code was translated from it: this is the self-modifying-code notification, so grant write, drop the stale translations and retry.
7. A plain-form access to the emulated commpage: mark the block for guarded retranslation.

Only then does it look for a bridged call in flight. In native mode a guest call into a system library runs the host's own arm64 code on a guest thread, so a bad pointer handed to `strcpy` faults with no guest instruction to blame. Such a thread is several native frames deep in code AArchX can neither resume nor unwind, so the process stops with a report that names the library, symbol and signature, and says whether the fault address was in guest space (the guest passed a bad pointer) or outside it (AArchX marshalled the call wrongly). The order matters: a guest writing into a page it has already executed must still be recovered even when the store came from a bridged `memcpy`.

Anything not recognised is delivered to the guest as an access violation at the faulting instruction, rather than killing the thread. Killing a thread leaves every lock it held taken forever: a V8 background job died that way holding a mutex, and the renderer's garbage-collection safepoint waited on it until the end of time. `SIGILL`, `SIGTRAP` and `SIGSYS` go through the crash reporter too, since a jump to a garbage code pointer used to end the process silently. A fatal fault in 32-bit guest code takes the whole process down on

purpose: it usually lands on a WoW64 thread, and a half-alive process wedges its parent, which waits for the child forever. The crash report's first line names AArchX's own load address, so a host program counter in it can be handed to `atos -l`.

One guard in that last step is Wine's alone. `crash_handler` counts the faults at one guest address in a row, and past 16 (`OCERZ_SIG_MAX_REPEAT`) it takes the thread for one of Wine's foreign workers looping on a dispatcher state it cannot repair and ends it, reporting `WORKER-TERMINATE`. Until 2026-10-18 it applied that to any thread with no guest signal stack, which is every thread of every program that is not Wine. A handler that unprotects a page and returns, as a garbage collector's write barrier or a JVM safepoint poll does, faults at the same address again the next time, and the seventeenth time its thread ended: a sort whose comparator read a protected page on every comparison exited 241 in cache mode after 16 of its 338 faults. A Mac delivers every one. ff6ea313 (2026-10-18) applies the count only to Wine processes. The native case `signal_callback_fault` pins it under both engines, with every fault landing inside a callback from the host's `qsort`, and in cache mode.

Callbacks from native code

In native mode the host's own code calls back into guest code all the time: a `qsort` comparator, a Cocoa delegate, a CoreFoundation callback, and in a 32-bit Mac program (Chapter 13) every AppKit message. Each such call runs a nested guest loop on the calling thread. Measured on 2026-10-18, a call cost 536 ns in native mode, where Rosetta runs the same comparator in 4.2 ns inside the x86 `qsort`. A profile put 59% of the 536 ns in three system calls and 18% in copying the 8 KB `OcerzCPU`. Commit ad51f4b3 (2026-10-18) removed them:

- **The signal mask is read once per thread, not once per callback.** The mask has to be cleared, to undo what a host thread had blocked, only for a thread's outermost guest loop. A callback nested inside `run_cpu` or another callback is called by a native function the guest called, which leaves the mask alone. A per-thread counter, `t_guest_loops`, counts the loops.
- **The recovery point no longer saves the mask.** It used `sigsetjmp(jb, 1)`, which saved the mask and asked for the signal-stack state on every callback, for the rare jump back to the recovery point. It now uses `sigsetjmp(jb, 0)`, and `callback_recovered` does what `siglongjmp` did: it empties the mask and, unless the point is on the signal stack, takes the thread off it with sigreturn's `UC_RESET_ALT_STACK`, which only clears that flag, as Apple's published XNU source defines it and as libc's `longjmp` does.
- **The callback's cpu is a smaller copy.** It copies its caller's cpu less the return-address stack, which starts empty (a translated `ret` below its own pushes finds a top of 0 and looks its target up; with `OCERZ_NO_HOST_RAS=1` the stack is zeroed instead), and less the system-call ring. That is 2.2 KB, not 8.2 KB.
- **Per-call housekeeping went.** Leaving a callback added 62 signal counters back with atomics when they were zero, applying the rounding mode called `fesetround`, which wrote FPCR every time, and the profiler read `OCERZ_GUESTPROF` on every callback.

A `qsort` of 200,000 ints with a guest comparator went from 536 ns to 100-112 ns a comparison (2026-10-18). `OCERZ_NO_CHEAP_CALLBACK=1` clears the mask on every callback and saves it with the recovery point again.

The unstick monitor

A guest thread can park in a blocking host wait whose wakeup was lost, for example through a kernel event edge or an alias between waiter and waker. During bring-up a manual `EINTR` nudge always revived such a session, so the monitor automates it: any cpu inside one blocking host call for more than 800 ms is sent a host signal deliberately installed without `SA_RESTART`. Only waits whose callers already loop on a spurious return are kicked: psynch, kevent, workqueue, ulock and Mach waits. A cpu in `read`, `recvmsg`, `poll` or an `fcntl` lock is marked `block_nokick` and left alone, because those never return `EINTR` on their own and applications rightly do not expect it. Semaphore waits are not kicked, and a timed `__semwait_signal` waits out a kick (2026-09-13). The same no-op handler also serves as a context-synchronisation event, so a thread spinning in translated code observes the stop-site patches the JIT makes when it needs every thread out of the code arena (Chapter 7). `OCERZ_NO_UNSTICK=1` disables the monitor and `OCERZ_UNSTICK_ALL=1` kicks every blocking call, as it once did.

Processes

fork

A fork child keeps the JIT. In August children dropped to the interpreter, which looked like a Steam network hang and was really a child at 96% CPU interpreting. The child does abandon the inherited `MAP_JIT` arena, whose pages read fine but raise `SIGBUS` when executed, and fork is never executed from inside compiled code. The child forgets the parent's other threads. The JIT's, the memory map's, the shared cache's and the initializer gate's locks are all taken around the fork, the way a careful threaded program does, so the child never inherits a lock that nobody will release.

A fork inside a callback is performed (f2e350ab, 2026-10-13). It had been refused by name whenever a translated block's frame lay beneath the callback on the host stack, on the grounds that the child could not return into translations it did not inherit. It can: the child forgets the JIT without unmapping its code (`ocerz_jit_forget`) and translates into a fresh arena, so the code the frame returns into is still there, unchanged, and its exits come back to the child's own JIT. Qt starts every `QProcess` that way from an AppKit callback. `OCERZ_FORK_CALLBACK_REFUSE=1` refuses the fork again. In native mode `pthread_testcancel` returns (fd9ca9e4, 2026-10-13) and `pthread_cancel` stays refused, because cancellation unwinds a host thread through AArchX's own frames, so no cancellation can ever be pending.

exec and spawn

`execve` and `posix_spawn` run the new program under AArchX, keeping the spawn's file actions and attributes. A universal binary runs its x86-64 slice. Since 2026-09-23 a binary with an arm64 slice and no x86-64 slice runs natively in every mode, as Rosetta runs it: Steam spawns `/usr/bin/open` three times a launch, and on macOS 27 `open` ships arm64 only, so every one of those children had died on "cannot read" (dynamic test `spawn_arm64_only`). In native mode a system tool with an arm64 slice runs as itself too, unless it is a shell or launcher whose own children must still be translated. The mode travels to children as `OCERZ_MODE` in the environment the spawn and exec paths build, even for a child started with an empty environment.

launchd

The macOS Steam client starts its `ipcserver` with `launchctl load -S Background` on a property list it writes, which would have launchd run the server natively. AArchX rewrites that plist on the way through, putting itself in front of the program, and enables the job's label first: a legacy `launchctl unload` leaves the label disabled, and every later load failed with an I/O error while the client reported `ipcserver init failed`. Chapter 14 covers the rest of Steam.

System V IPC

Semaphores, shared memory segments and message queues have been implemented since 2026-09-11, because Steam's threading library uses System V semaphores; the bootstrapper ran for the first time that day. `shmat` and `shmdt` followed on 2026-09-22 and are described in Chapter 8.

Diagnostics

This layer has more diagnostics than any other, because a missing translation looks, from inside the guest, like an ordinary failure.

Switch	What it shows
`-strace`, `OCERZ_STRACE_EXE=<text>`	every call with its guest arguments; the second limits it to processes whose command line matches
`OCERZ_SYSFAIL=1`	every `ENOMEM` and `EFAULT`, which is what a missing pointer translation looks like from inside
`kill -INFO <pid>`	every guest thread: host thread id, `rip` and `rsp`, the call it is blocked in, its Windows-side stack, and its last 24 system calls
`OCERZ_SYSRING=<n>`	one process-wide ring of that many system calls, each tagged with its thread; on a wineserver it shows every request, reply and wakeup
`OCERZ_SIGMASKLOG=1`	every change of a guest mask to a wide one, with the caller chain
`OCERZ_MACHMSG=1`, `OCERZ_MSGHEX=<id>`	Mach message results, and full request and reply bytes for one id
`OCERZ_IOKITERR=1`, `OCERZ_IOKITSEL=<sel>`	`io_connect_method` calls that fail with a VM or argument error, or every call with one selector
`OCERZ_MSGNEEDLE=<value>`	every outgoing message that carries a given 64-bit value
`OCERZ_MACSYSLOG=1`	each `__mac_syscall` with its policy, number and argument words
`OCERZ_NO_SIGNAL_STOP=1`	signals wait for the thread to reach the dispatcher on its own, as before 2026-10-14
`OCERZ_NO_CHEAP_CALLBACK=1`	clear the mask on every callback and save it with the recovery point
`OCERZ_HOSTSIG_PROBE=1`	each thread's real host mask and pending set, and the last 512 host signals received

HOW IT WAS FOUND

`OCERZ_SIGMASKLOG` exists because of eight threads spinning in R.E.P.O. with every signal blocked. The log showed that each had been through libc's `abort()`, called from Metal's validation layer, whose `SIGABRT` Wine had turned into an exception while the thread kept `abort`'s mask. The wide mask was a symptom; the cause was the validation failure that called `abort`.

PART IV

Linking

Loading Mach-O images and binding them, against Apple's x86-64 shared cache or against the Mac's own arm64 frameworks, and running 32-bit Mac programs on frameworks that no longer have 32-bit code.

CHAPTER 10

The Mach-O loader

Before a single x86 instruction runs, AArchX has to decide what kind of program it was handed, pull the right slice out of the file, lay its segments into guest memory and build the stack the kernel would have built. This chapter covers that front door: `src/main.c`, the static loader in `src/loader.c` and `src/stack.c`, and the parts of the dynamic path that shape the process before the mini-dyld of Chapter 11 takes over. It also says where a program that is not an x86-64 program at all, an i386 Mac executable, is handed to the 32-bit runtime of Chapter 13.

378	137	575	8 MB
LINES IN LOADER.C	LINES IN STACK.C	LINES IN MAIN.C	INITIAL GUEST STACK

Two kinds of program, one front door

AArchX runs two very different kinds of Mach-O. A **static** program carries an `LC_UNIXTHREAD` command with its initial register state and links against nothing: the freestanding binaries in `tests/guest` are the main examples, built with `-nostdlib -static`. A **dynamic** program names a dynamic linker and libraries, and that is every real macOS program, from `/bin/ls` to Steam. The two take different paths through the code, and the decision is made once, in `main()`, before the virtual machine is set up.

The question is answered by `ocerz_peek_dynamic`, which reads the file, selects the x86-64 slice and scans its load commands without mapping anything. The rule is short:

- an `LC_MAIN` command (0x80000028) makes the program dynamic, and the scan stops there;
- an `LC_UNIXTHREAD` together with `LC_LOAD_DYLINKER` is also dynamic, which is how binaries linked for old releases of macOS start;
- an `LC_UNIXTHREAD` alone is static;
- a file with no x86-64 slice at all returns a distinct answer, which main() then splits in two: an i386-only Mach-O is handed to m32, the 32-bit Mac runtime (below and Chapter 13), and anything else is not an Intel program, so AArchX says so and exits with 64 rather than pretending it cannot read the file;
- a file it cannot read exits with 65.

Native mode (Chapter 12) adds one more refusal. Its whole mechanism is standing in for the libraries a program links, so a static image has nothing to bridge into and is refused with exit 64 before the VM starts.

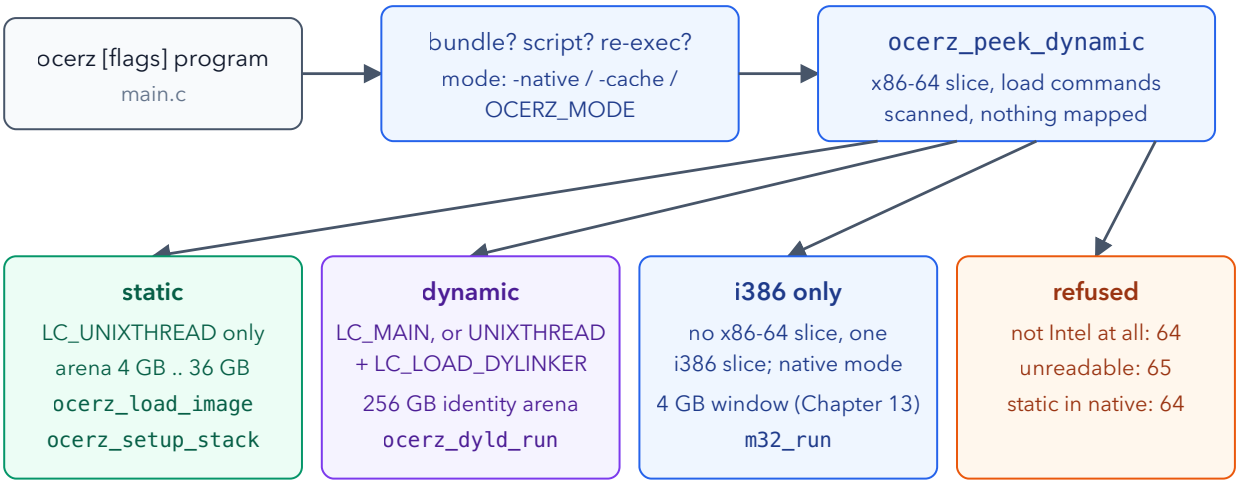


Figure 10.1. How `main()` routes a program. The decision is made from load commands alone, before any guest memory exists. The i386 route was added in 0ca24d64 (2026-10-10); before it, a file with no x86-64 slice was refused.

What main() settles first

`main.c` is small, but several of its decisions shape everything after them, and each exists because something broke without it.

Standard error, before anything else

`OCERZ_STDERR_FILE`, when set, is opened for appending and installed as file descriptor 2 before any other line runs. Chromium and Wine start children with standard error closed or pointed at nothing, and a child that died early under AArchX otherwise left no trace at all. With the variable set, every process of a tree writes into one file.

The host's dyld must not act on DYLD_INSERT_LIBRARIES

Steam launches every game with `DYLD_INSERT_LIBRARIES` naming its loader and overlay. The variable is meant for the guest, but the host's dyld reads it first and loads the arm64 slices of those libraries into AArchX itself, which broke startup within half a second. So `main()` moves the value into `OCERZ_GUEST_DYLD_INSERT_LIBRARIES`, removes the original and `execv`s itself. Re-executing keeps the process ID, which matters because Steam tracks the game by it. The clean copy puts the variable back before the guest's environment is taken, so the guest sees exactly what Steam set, and the mini-dyld opens those libraries in the guest after the startup initializers (Chapter 11).

The mode

`-native` and `-cache` choose outright, and the last one on the command line wins. With neither, `OCERZ_MODE` decides, and that is how a child inherits its parent's mode across a spawn or exec. An `OCERZ_MODE` value that is neither `native` nor `cache` is refused with exit 64 rather than ignored, because a silently ignored mode is a test that measures the wrong thing.

Application bundles

A path that is a directory is treated as an application bundle. `bundle_executable` reads `Contents/Info.plist` and takes the string after the `CFBundleExecutable` key; if the file is not readable as XML, it falls back to the bundle's own name without `.app`. The result must exist as a regular file under `Contents/MacOS`. The guest's `argv[0]` becomes that executable's path, as it is when the system starts a bundle. This exists because Steam's launch options substitute the bundle for `%command%`: a game launched as `ocerz %command%` used to end at once with "cannot read".

Launch scripts

After a bundle has been resolved to its executable, `run_launch_script` looks at the first two bytes of the file. A file that starts with `#!` is a script, not a program, and the loader used to stop on it with "no x86_64 slice". Two kinds matter, and both exist because of a game that Steam starts through a script.

- **A script that starts its game through `GAME_DEBUGGER`.** For a Source game on the Mac, Steam's %command% is hl2.sh, which sets `DYLD_LIBRARY_PATH` and runs `${GAME_DEBUGGER} <game> <args>`. Since 46fade2e (2026-10-13) the script runs as itself, with `GAME_DEBUGGER` set to this ocerz and the flags that came before the script, so `ocerz -native %command%` serves these games as it serves the rest; before it, Left 4 Dead 2 needed a launch option of its own that set the variable. A `GAME_DEBUGGER` already in the environment wins. The scripts expand the variable unquoted, so an ocerz installed under a path with a space would split, and that case is refused with a message and exit 64.
- **Any other script.** Nexuiz.app's executable sets `DYLD_LIBRARY_PATH` and then runs `exec "$0"-bin "$@"`. Run natively, /bin/sh handed the i386 game to the kernel, which refused it with "Bad CPU type in executable". Since 88b7e20c (2026-10-18) a script that does not use `GAME_DEBUGGER`, and whose interpreter has an x86_64 slice, is run on that slice under ocerz, with the same flags, so every program the script starts is ocerz's to run. An interpreter with no x86_64 slice runs the script natively as before. Valve's hl2.sh, which does use `GAME_DEBUGGER`, still runs natively.

Both forms end in an `execv`, so the process keeps its ID. A failed exec exits with 126. `tests/m32/launchscript` is a cut-down hl2.sh that starts a program under `GAME_DEBUGGER`, and `tests/m32/execscript` is a script without the variable that runs and then execs an i386 program.

Wine, Steam Helper and the per-process switches

A process is marked as one of Wine's when its program, after `realpath`, is named wine or wine64, or when the program carries a segment named `WINE_RESERVE`, the address range Wine's preloader keeps for Windows. The name alone was the whole rule until 74e6140d (2026-10-13): CrossOver starts each process through a link named after the Windows program, `realpath` keeps the link's name for a hard link, and its cmd.exe therefore started with the ordinary memory layout and died reserving the low-shadow base its Wine parent had handed down. The segment test reads the Mach-O header of the x86-64 slice and is present in WineHQ's builds and CrossOver's alike; `tests/dynamic/wine_segment` pins it and `OCERZ_NO_WINE_SEGMENT_DETECT=1` goes back to the name alone. Being marked fixes the base of its memory arena (Chapter 8) and, unless the user chose otherwise, sets `OCERZ_PRELOAD_OBJC` to AppKit, QuartzCore, HIToolbox and CoreSpotlight, the Objective-C images whose categories must be visible before any class is realized. A program named `Steam Helper` gets `--use-mock-keychain` inserted after `argv[0]`, unless it is a crash-pad handler or already has the flag, so that macOS does not ask for the login password once per helper process (Chapter 14).

A short command-line summary, the last path component of every argument, is kept in `ocerz_cmdline_summary`. Three switches match against it so that one process of a large tree can be singled out: `OCERZ_NOJIT_EXE` interprets only matching processes, `OCERZ_STRACE_EXE` traces only their system calls, and `OCERZ_EXE_ENV="cs2.exe:K=V,K=V;other.exe:K=V"` sets variables only in them, before any of those variables is read.

A private copy of the environment

The dynamic loader is handed `env_snapshot(environ)`, a copy that owns its strings and lives for the whole process, never `environ` itself. Loading a guest in native mode opens the host's own frameworks, and CoreFoundation's initializer calls `setenv`, which reallocates `environ` and frees the array an earlier pointer still named. The guest's initial stack was built from that pointer only after loading, so it read freed memory, and whether that crashed depended on whether the block had been reused yet. It was reused when the environment had the size `bash` passes and not the size `zsh` passes, which made the bug look like a shell problem.

Where an i386 program goes

macOS stopped running 32-bit Intel programs with Catalina, and Rosetta never ran them. Until 0ca24d64 (2026-10-10) a file with no x86-64 slice was therefore refused. Now `ocerz_peek_dynamic` still answers "no x86-64 slice" for it, but main() asks one more question before giving up: `m32_is_i386`, which reads the first page of the file and is true when the file holds an i386 slice and no x86-64 one. A thin i386 Mach-O qualifies. A universal file with both slices does not, and runs through the x86-64 path as it always did. If the answer is yes, main() calls `m32_run` with the VM, the program, its arguments and the environment snapshot, and the loader of this chapter is never entered; every later step belongs to Chapter 13. A program that is neither x86-64 nor i386 still ends with the message that it is not an Intel program and exit 64.

The mode is settled one step earlier. An i386 program crosses into the Mac's arm64 frameworks on every call, and cache mode has no x86 shared cache to give it, so main() switches it to native mode unless the command line chose a mode (-native or -cache) or `OCERZ_MODE` is set. This is 1ae3140a (2026-10-18). The reason was cost: cache mode's NaN results need `FPCR.AH` cleared for the host and set again around every crossing, about 8.5 ns for each of the two writes on an M5 against 2.5 ns for an unchanged one, and an i386 program run natively used 8% fewer cycles on Nexuiz's benchmark demo. `tests/run_m32_tests.sh` checks the default.

A program that loads itself

One kind of i386 program has no dynamic linker to hand over to. A UPX-packed executable has a single segment, an `LC_UNIXTHREAD` entry, no libraries and no `LC_LOAD_DYLINKER`. On a real system the kernel starts its stub; the stub decompresses the real program into memory using raw system calls, then opens /usr/lib/dyld, maps its i386 slice and jumps to it with the program's Mach header pushed above argc, and dyld carries on as if it had loaded the program itself. Under m32 the stub used to die at its first `sysenter`, an instruction the decoder did not know, and there is no i386 dyld for it to open. 0144d669 (2026-10-16) made it work, and Kega Fusion's Mac build, which is packed this way, now unpacks and starts its own code.

`m32_run` recognizes such a program by its `LC_UNIXTHREAD` with neither a dylinker nor a library. Before anything else is put in the 4 GB window it reserves the low 256 MB around the packed segments, since the stub will map its unpacked program over that range: UPX 3 unpacks the program at its own addresses, UPX 5 maps it wherever it finds room and relies on it being position independent. The program starts at its own entry with argc at the stack pointer, as the kernel leaves it. From then on `src/m32_selfload.c` plays the kernel and dyld:

- the decoder accepts `sysenter` (0F 34), and in an i386 program the interpreter answers it and `int 0x80` as the Darwin system calls they are: the number in the low 16 bits of eax, the arguments above the return address at ecx (`sysenter`) or esp (`int 0x80`), the result in eax, and on failure the carry flag set with errno in eax. Only the calls a loader makes exist (exit, read, write, open, close, getpid, munmap, mprotect, madvise, pread, mmap and lseek); every other number fails with ENOSYS. The JIT leaves `sysenter` to the interpreter, as it does the 64-bit syscall;
- when the stub opens a dyld, it is given a file m32 has just written: an i386 `MH_DYLINKER` with one empty segment and a thread state whose eip is a trap, `M32_ID_DYLD`, and the file is unlinked as soon as it is open. The stub maps it and jumps there;
- the trap lands in `m32_dyld_entry`, which takes the program from the Mach header the stub pushed, puts its file back together from its segments in the window so that load commands, symbols and fixups can be read as for any program, and starts it as `m32_run` starts any other (UPX 5's program, mapped at a new address, is slid and rebased as dyld would).

`OCERZ_M32_NO_SELFLOAD`=1 starts such a program as before. The test `selfload` is the project's objc_basic program packed with UPX 5.2.1, with objc_basic's own output expected. Chapter 13 covers everything after the hand-off: the window, the handle cells, the crossing of calls and the Objective-C 1 runtime.

Choosing the slice

A universal (fat) binary starts with a big-endian header listing one record per architecture. `select_fat_slice` accepts both the 32-bit (`FAT_MAGIC`) and 64-bit (`FAT_MAGIC_64`) forms, rejects a count of zero or more than 64 records as implausible, byte-swaps each record and takes the first whose CPU type is `CPU_TYPE_X86_64`. Only the slice's file offset is kept: every later read adds it, and nothing maps the file.

That is a deliberate choice. All reads go through `read_exact`, a loop over `pread` at explicit offsets that turns a short read into `OCERZ_EFORMAT`. A truncated or malformed image therefore fails while it is being read, with a message naming what was missing, instead of faulting later inside guest memory. The header must then say `MH_MAGIC_64`, CPU type x86-64 and file type `MH_EXECUTE` or `MH_DYLINKER`; `sizeofcmds` must be non-zero and at most 16 MB; and every load command's size must be at least a load command and must not run past the end of the commands. Each check names the failure.

The dynamic path has its own slice selector in `src/dyld.c`, which reads the whole file into a buffer once and works from that buffer for mapping, binding and symbol lookup; Chapter 11 picks up there.

The static path

A static program gets the oldest and simplest memory layout AArchX has: `ocerz_mem_init(0x100000000, 0x900000000)` reserves an arena from 4 GB to 36 GB of guest address space, and guest addresses translate to host addresses by adding a single base (Chapter 8). The loader then makes two passes over the load commands.

First pass: measure

Each `LC_SEGMENT_64` contributes its `[vmaddr, vmaddr + vmsize)` to one overall range. `__PAGEZERO` is recognized by having a zero `vmaddr` and no protection, and is skipped: it reserves the bottom 4 GB on a real system and has nothing to map. The segment whose file offset is zero contains the Mach header, and its `vmaddr` becomes the image's header address. An image with no mappable segment, or none at file offset zero, is refused.

The entry point comes from one of two commands. `LC_UNIXTHREAD` carries one or more flavor blocks of thread state; the loader walks them, checking that each block fits inside the command, and for flavor 4, `x86_THREAD_STATE64`, takes register index 16, which is `rip`. `LC_MAIN` carries an offset, and the entry is the header address plus that offset. A thread state wins if both are present. A static image's slide is always zero: it is placed exactly where it was linked.

Second pass: map, copy, protect

The whole range is mapped read-write with `ocerz_map_fixed`, each segment's file bytes are read straight into place, and only then are protections applied. They are applied in two rounds, read-only and executable segments first and writable ones second. Writing everything before protecting anything means the loader never has to write into a page it has already made read-only. Note that "executable" here describes the guest's view only: guest code is never executed by the host, so the host pages stay readable and the JIT reads them (Chapter 6).

The initial stack

The kernel's exec path leaves a precise structure on the new process's stack, and both C runtime start code and dyld read it positionally. So the order and alignment are a contract, not a detail. `ocerz_setup_stack` builds it in an 8 MB stack mapped anywhere in the arena:

1. sixteen zero bytes at the very top;
2. the strings, pushed downwards: the single `apple[0]` string `executable_path=<path>`, then every environment string, then every argument string;
3. below the strings, aligned down to 16 bytes, the pointer area, in ascending order: the Mach header address (only when the image entered through `LC_MAIN`, where dyld's convention puts it), `argc`, the `argv` pointers and a null, the `envp` pointers and a null, the `apple` pointer and a null;
4. `rsp` pointing at the first word of that area, itself 16-byte aligned, and `rbp` zero.

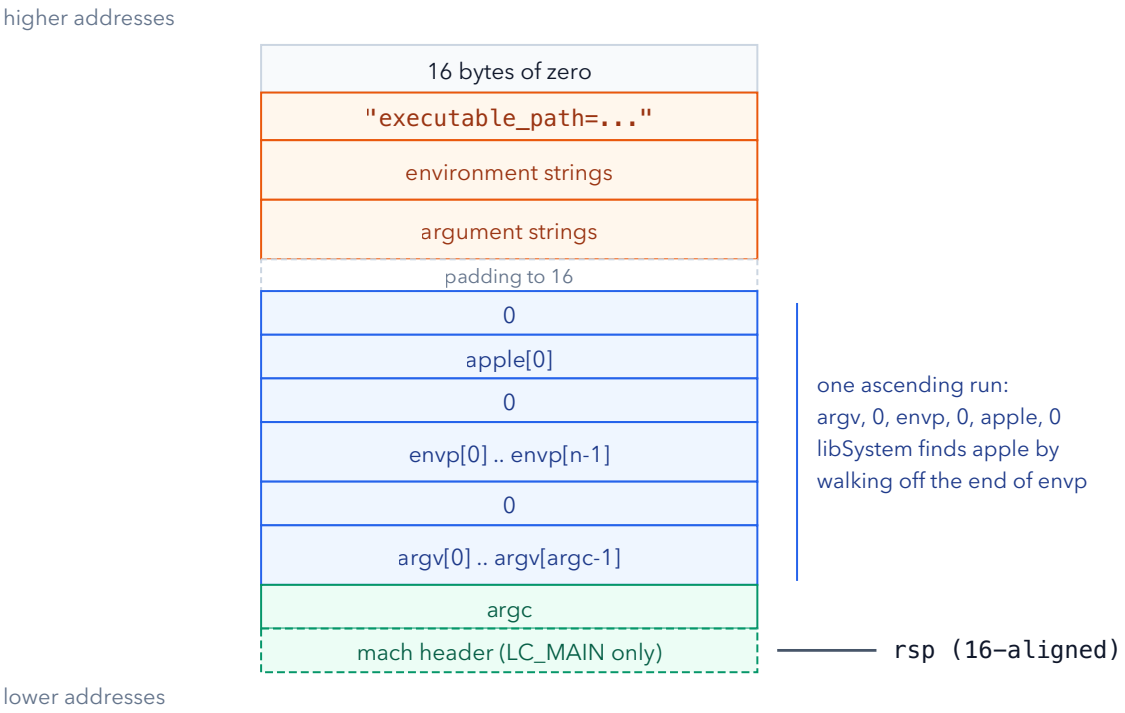


Figure 10.2. The initial stack as `ocerz_setup_stack` builds it for a static program. The dynamic frame (Figure 10.3) has the same vector layout with five more `apple` strings.

The three vectors must form one contiguous ascending run. libSystem never receives the `apple` array as an argument: it finds it by walking past the null that ends `envp`. An earlier version of the dynamic frame stacked the vectors downwards instead, which put the `argv` vector where `apple` belongs. libSystem's walk then ran past `argv`'s terminator into the string area and handed `strlen` the bytes of `"th_port=0x..."` as a pointer, and `python3` died there.

The dynamic frame

A dynamic program's frame is built by `build_frame` in `src/dyld.c`, not by `stack.c`, because it has more to carry. Its stack is also 8 MB, and it holds:

- **Six `apple` strings** instead of one: `executable_path=`, `stack_guard=`, `ptr_munge=`, `malloc_entropy=`, `main_stack=` with the stack's top and size, and `th_port=` with the main thread's Mach port. libSystem's initializer reads these.
- **A ProgramVars block**, the structure handed to libSystem's initializer and to every image initializer, pointing at the real `NXArgc`, `NXArgv`, `environ` and `__progname`: libdyld's in cache mode, the host's in native mode. It used to point at cells of its own beside the vectors, and then `_NSGetEnviron()` and the `environ` a program reads came apart at the first `setenv`: a program that called `setenv` and then `execve` with `environ` handed its child an environment without the variable.
- **A return stub at the top of the stack.** In cache mode AArchX bypasses dyld's own start routine and calls `main` directly, so `main` returns into nine bytes AArchX writes 64 bytes below the stack top: `mov edi, eax; mov eax, 0x2000001; syscall`, the exit system call with `main`'s result. In native mode the same place calls C's `exit` (the symbol `_exit`) in the synthesized libSystem instead, so that `atexit` handlers run and stdio is flushed, as `start` would do; the exit system call stays behind that call only for the case where the export is missing.

Every argument and environment entry is counted first and the frame is sized to fit. The arrays used to be fixed at 64 entries with the environment cut at 60, which silently dropped the last variables of a large environment. Wine's loader marks its one-time re-exec by appending `WINELOADERNOEXEC=1` as the last variable, so a launch with one variable too many re-executed every Wine process forever. It looked like an intermittent hang and was a counting bug (2026-09-06, found under Steam).

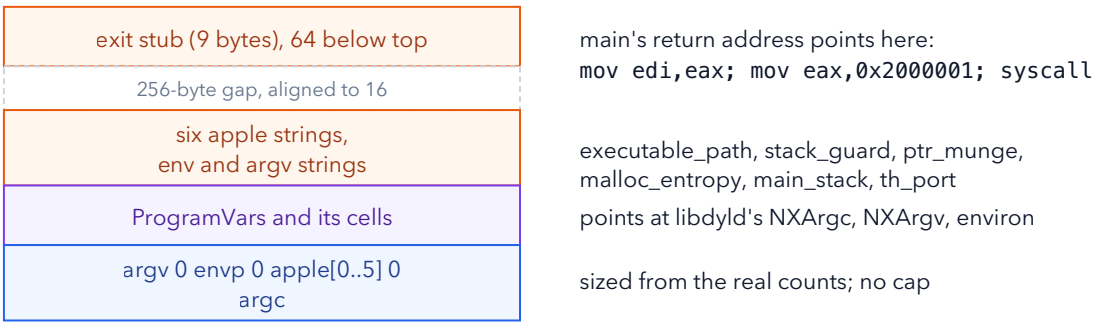


Figure 10.3. What the dynamic frame adds to the static layout, top of stack first.

The executable's own path

The dynamic loader `realpath`s the program's path before anything else uses it, so the guest always sees an absolute executable path, in `executable_path=`, in the image list and in `_NSGetExecutablePath`. A real system resolves it the same way. Without it, a program started as `./steam_osx` got a relative path back from `_dyld_get_image_name(0)`, and Steam's tier1 library built `"/../Steam.AppBundle/..."` out of it and asserted. The dynamic test `exec_abspath` pins it.

Where a dynamic image lands

The dynamic path does not use the static arena. `ocerz_dyld_run` first reserves a 256 GB **identity** arena, where guest addresses are host addresses, which is the fast case for the JIT. It is that large because JavaScriptCore's Gigacage asks for a single 128 GB mapping and Chromium's PartitionAlloc reserves 32 GB at startup. Then:

- A **position-independent** main image is mapped anywhere inside the arena, and the difference from its linked address becomes its slide. Every library is placed the same way.
- A main image that is **not** position-independent goes exactly where it was linked. If any of its segments lies below 12 GB (the constant `OCERZ_LOW_LIMIT`, 0x300000000), the process switches to the low-shadow layout, where those low guest addresses live at a separate host base, because the host will not hand out the bottom of its own address space. Wine's preloader is such a program, which is why Wine processes run in that layout. A segment elsewhere outside the arena is registered as an extra guest range and mapped fixed.
- One exception, for **native mode** only (275843d4, 2026-10-09, from the MacShack patch of Jfishin): a non-PIE main image that lies below 12 GB and keeps the default 4 GB page zero is slid into the arena like a PIE. In native mode the guest shares its address space with host frameworks whose heap and shared-cache pointers it is handed directly, and under the low shadow such a program's Objective-C classes crashed. A non-PIE executable has no rebase information (linking with -no_pie drops it), so `scan_rebase` rewrites its absolute pointers: every aligned word in the file-backed sections of its

__DATA segments that points into one of the image's own sections. Requiring a section rejects look-alikes, such as a method list's entry size and count of 1 read together as the image base plus 24; Aragami has 183 such words against 50,800 real pointers. Wine's loader, whose page zero is 4 KB so that Windows' low addresses stay free, keeps the low shadow, and so does any program run with `OCERZ_NOPIE_LOW=1`. `tests/dynamic/native_nopie` is a -no_pie Objective-C program compared with Rosetta.

- A process whose loader is Wine reserves its arena at a fixed base, so that `ntdll.so` lands at the same address in every process of a Wine session (Chapter 8 and Chapter 15 explain why that matters).

From here to the mini-dyld

For a static program the story ends here: `vm.cpu.rip` is set to the entry point and `ocerz_vm_run` starts the run loop, which is Chapter 2's subject. For a dynamic program, `main()` only routes. It hands `ocerz_dyld_run` the path, the guest's arguments (after the Steam Helper rewrite) and the environment snapshot, and the mini-dyld takes over: mapping the shared cache in cache mode, loading every dependency, applying fixups, running libSystem's initializer and the rest in dependency order, and finally jumping to the program's entry point with the return stub on the stack. Chapter 11 covers all of that.

> **WHAT THE LOADER DOES NOT DO**
>
> Its own paths read only 64-bit x86 Mach-O. A standalone i386 Mach-O is not something current macOS can run, and since 0ca24d64 (2026-10-10) it is handed to m32 (Chapter 13) instead of being refused; the i386 code of Wine's WoW64 is a different matter, a Windows program inside a 64-bit process (Chapter 15). It does not check code signatures, since the guest's code is never handed to the host to execute. And it maps nothing from the file: every byte is read and copied, which costs a little at startup and means the image in guest memory never depends on the file staying unchanged.

CHAPTER 11

The mini-dyld and the shared cache

Every real macOS program is dynamically linked, and on a current Mac its system libraries do not exist as files: they live inside the dyld shared cache. AArchX never runs Apple's dyld. It brings its own dynamic linker, the mini-dyld, which maps the x86-64 shared cache, loads and binds every image, runs initializers in the order dyld would, and answers the API that libdyld exposes to programs. This chapter is about that linker in cache mode; Chapter 12 covers how the same loader serves native mode.

6,563	2,731	1,591	59
LINES IN DYLD.C	LINES IN DYLDAPI.C	LINES IN CACHE.C	DYLD API SLOTS ANSWERED

Why a linker of its own

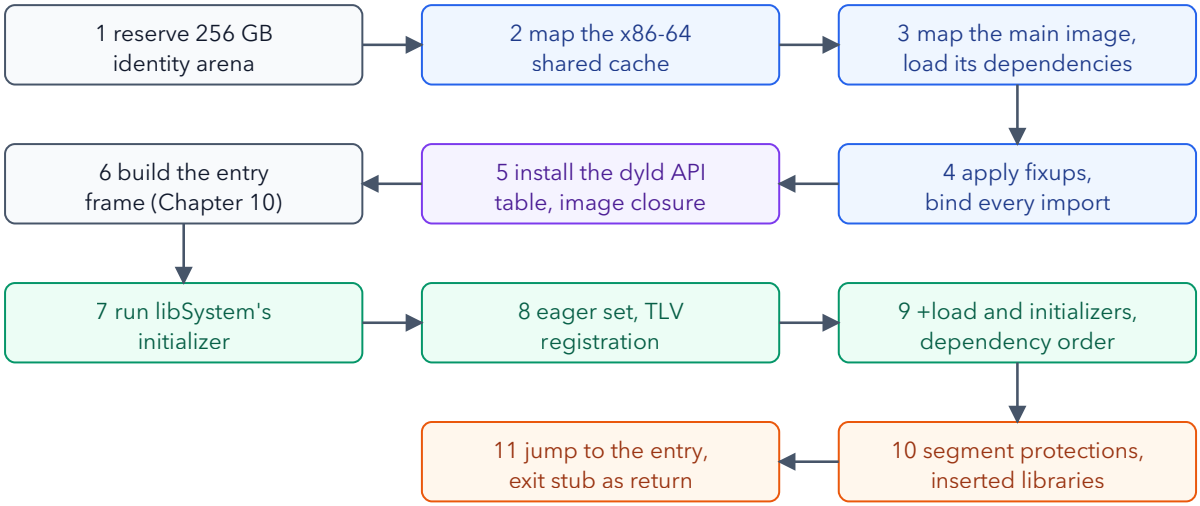
The obvious design is to load Apple's x86-64 `/usr/lib/dyld` as a second image and let it do everything, the way the kernel would. AArchX does not, and the reason is control. A translator has to know which code is which image, when an image appears, what its segments are, when its initializers run and on which stack, and it has to answer the questions programs ask about images, because some of those answers differ under translation. Owning the linker means every one of those facts is ocerz's own data rather than something reconstructed from the outside, and it means each bug in this area could be fixed where it lived. The cost is that the linker must reproduce dyld's observable behaviour exactly, and most of this chapter is about where that turned out to matter.

The mini-dyld lives in three files. `src/cache.c` maps the shared cache and resolves symbols out of it. `src/dyld.c` loads images, applies fixups, orders initializers and implements `dlopen`. `src/dyldapi.c` answers the dyld API that libdyld's exported functions call into, and builds the image list that libobjc and every other reader of loaded images walk.

Two other loaders exist, and this chapter is not about either. Native mode (Chapter 12) reuses src/dyld.c with synthesized libraries in place of the cache. An i386 program never reaches this code at all: m32, the 32-bit runtime of Chapter 13, has its own linker, src/m32_dyld.c, and for a UPX-packed program that loads itself it even makes up the dyld file the program's stub expects to open (Chapter 10 describes how that hand-off starts).

From exec to main

`ocerz_dyld_run` is the whole startup sequence, and its order is the order dyld uses where order is observable.



Native mode skips 2, 5 and 7: no cache, no libdyld, no libSystem initializer. Its images come from Chapter 12's synthesizer.

Figure 11.1. The startup sequence of `ocerz_dyld_run` in cache mode.

Two details of the sequence are worth stating now. libSystem's initializer runs first and alone, called through `ocerz_vm_call` with five arguments, `argc`, `argv`, `envp`, `apple` and the ProgramVars block (Chapter 10), and when it returns, its whole dependency closure is marked initialized. And the program's entry point is entered with the nine-byte exit stub as its return address, because dyld's own `start` routine, which would normally call `main` and then `exit`, is never run.

Finding and mapping the cache

Where the cache is

The x86-64 shared cache is shipped by the Rosetta package, not by the base system, which is why cache mode needs Rosetta installed even though Rosetta's translator never runs. Its location moved. macOS 27 keeps it in the Rosetta cryptex, `/System/Volumes/Preboot/Cryptexes/Rosetta/System/Library/dyld/`; macOS 26 keeps it in the OS cryptex, `/System/Volumes/Preboot/Cryptexes/OS/System/Library/dyld/`. `ocerz_cache_dir` tries the Rosetta location first and the OS one only when the first holds no cache, so a macOS 27 machine maps exactly what it always did. Between 2026-09-19 and 2026-10-03 only the Rosetta location was tried, and every dynamic program on macOS 26 stopped with "cannot map shared cache". The unit test `test_cache` used to skip whenever mapping failed, which is how that went unnoticed; it now checks that the first directory holding a cache is the one in use, and fails rather than skips when a cache exists but will not map.

Main file and subcaches

The cache is a main file, `dyld_shared_cache_x86_64`, and numbered subcaches beside it, `.01`, `.02` and so on, mapped from the same directory until one is missing (at most sixteen). Everything ocerz reads from them is from the published format, `dyld_cache_format.h`: each file's header lists its mappings in an array

of `dyld_cache_mapping_and_slide_info` records, 56 bytes each, giving the address, size, file offset, slide-info offset and size, flags and maximum and initial protections. The image table (`imagesOffset` and `imagesCount`) lists about 3,600 images, each with its address and path.

Every mapping is mapped `MAP_FIXED` at its preferred address, starting at `0x7ff800000000`; the cache is never slid. Execute permission is turned into read: guest code is never executed by the host, only read by the decoder (Chapter 4). A mapping that carries slide information is mapped writable, because its pointers have to be rewritten.

Lazy slide unpacking

The slide information (version 2 in this cache) stores every pointer in the data regions as an offset plus a chain of deltas to the next pointer in the same page. So even at slide zero the DATA and DATA_CONST regions, about 500 MB, must be unpacked before anything reads them. Doing it eagerly touched and copied every page at process start, about 200 ms per process, and Steam under Wine starts dozens of processes.

So those regions are mapped `PROT_NONE`, and each 16 KB host page is unpacked on its first touch, from the fault handler, then given its final protection. The interesting part is how the unpacked page appears.

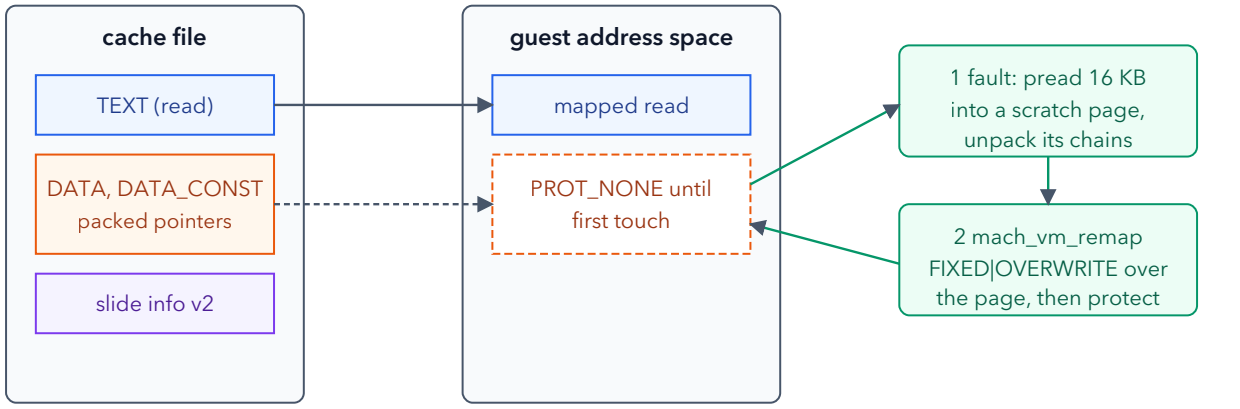


Figure 11.2. Lazy slide unpacking. The real page is never writable while half unpacked.

The first lazy version did `mprotect(RW)` on the page and rebased it in place. Every other thread could then read raw pointer chains mid-unpack, and libswiftCore inside a Steam process dereferenced one of those half-built pointers. Now the page is read from the file into a private scratch mapping, its chains are unpacked there, and the finished page is installed over the real one with `mach_vm_remap(VM_FLAGS_FIXED | VM_FLAGS_OVERWRITE)` while the real page is still `PROT_NONE`. A concurrent reader simply faults, waits on the unpack lock and retries against the finished page. A per-thread note of the last page retried tells a genuine second fault, an alignment or protection fault on an already unpacked page, apart from a lazy one, so those still reach the real handler. The lock is held across a whole page rebuild, so `fork` takes it like the other emulator locks, or a child could inherit a lock nobody will release. `OCERZ_EAGER_SLIDE=1` unpacks everything up front, `OCERZ_NO_LAZY_REMAP=1` goes back to in-place rebasing, and `OCERZ_LAZYCHECK=1` unpacks every page twice and compares.

Pages a program patches

Some engines make a libSystem page writable and patch it in place. So every cache mapping is recorded (which also tells an address in the cache apart from a wild one), along with write-watch state for any page a guest has made writable. A lazily slid page must be unpacked before such an `mprotect`, because once it is accessible the fault that would have unpacked it never comes. When code has been translated out of a patched page, write permission is taken back, so the next patch faults and the translations made from the old bytes can be retired (Chapter 7).

Resolving symbols

Export tries and the two-level namespace

Every image publishes its exports as a trie, a prefix tree of names whose terminal nodes carry the symbol's offset and flags. The walker has one subtle case: a terminal of size zero with the name fully consumed is not a miss, because the node carries an empty edge to a terminal child whenever a symbol is a strict prefix of others, as `_libiconv` is of `_libiconv_open`.

macOS links in a **two-level namespace**: each import records which library it came from, as an ordinal into the importing image's list of dependencies. The mini-dyld honours it. A cache dependency is resolved in the specific library the image linked, following re-exports, rather than by a search of every image. This matters more than it sounds: `openssl` links `libcrypto.46.dylib` (LibreSSL 3.3.6) while the cache also carries `libcrypto.44` (2.8.3) exporting the same names, and a flat search bound `OpenSSL_version` in whichever came first, so `openssl version` printed the wrong version. A library lookup follows `LC_REEXPORT_DYLIB` as well as re-export entries in the trie, because an umbrella such as libSystem answers for its members only through those load commands.

The per-process index

All resolution runs on an index built once per process: install path to image and header to image in open-addressed tables, and, filled per image on first use, its export trie and its linked images in ordinal order with their kind. The index is published lock-free with a compare-and-swap, so there is no lock for `fork` to catch half-held. A breadth-first search from one image walks the index with a visited bitmap and memoizes its answer per image and name.

It replaced a 512-slot direct-mapped memo of paths. Under Wine, which resolves thousands of OpenGL and AppKit names through `dlsym`, colliding paths fell back to a `strcmp` over all ~3,600 images, and a Steam web helper spent a quarter of its startup in the resolver. A benchmark of 4,500 framework lookups through `dlsym` went from 8.17 s to 0.12 s; Rosetta takes 0.21 s. A miss on a framework handle went from 2 ms to about 10 µs.

Paths the image table does not list

The image table holds each library once, under its install name. Every other path for it, `/usr/lib/libz.dylib` for `libz.1.dylib` or `libgcc_s.1.dylib` for libSystem, is only in the cache's dylibs trie, which the published header locates. ocerz failed 108 of the SDK's 111 aliased `/usr/lib` names that Rosetta loads until all three path lookups fell back to that trie, and Steam's `steamloader.dylib`, which links `libgcc_s.1.dylib`, could not load. Of the 10,595 paths the macOS 27 Intel cache answers to, 6,677 are reached that way.

A dependency is also looked for in the cache both before and after its symlinks are followed. Eleven of the twelve dylibs in `/usr/lib/swift` are symlinks into frameworks whose binaries exist only in the cache, so following the symlink first led to a file that is not on disk: Tailscale lost Network.framework through Sparkle that way, with a "Library not loaded" error naming a path the cache was holding all along.

Libraries without a trie, and imports that may be missing

A dylib built by an old enough toolchain carries its exports only in the classic symbol table. Resolving through the trie alone found nothing in it at all: 21 of the 60 dylibs Photoshop ships are of that kind, and its OpenCV could not find a single Intel IPP entry point. Such an image now gets an index, built once, from the symbols `LC_DYSYMTAB` marks as external definitions. Only an image with no trie is indexed, because where there is one it is the whole truth.

A symbol imported with `-undefined dynamic_lookup` is resolved from the flat namespace and is allowed to be absent; dyld does not refuse to load over one. ocerz called each a fatal unresolved import, and Intel MKL's `libmkl_core` has 2,161 of them, because MKL puts its kernels in whichever ISA-specific library it loads at run time; one Photoshop launch printed fifteen thousand lines. They are now counted quietly (`OCERZ_DYNLOOKUPLOG=1` lists them). A genuinely unresolved import names the symbol, the image that wanted it and the library it was looked for in, and the list stops after 24 unless `OCERZ_ALLMISS=1`.

Weak coalescing

A weak-coalescing bind (ordinal −3) asks whether any loaded image already defines the name, so that every copy of a C++ inline function or template instantiation resolves to one definition. dyld answers it only from images that define weak symbols. ocerz walked all ~3,600 cache images, and Electron Framework's ~2,560 such binds, all its own C++ names that no system library has, cost 1.8 ms each: 4.6 s in every Discord process. Now the walk covers the ~300 weak-defining images, and after 128 lookups a 1 MB bloom filter of everything they and their re-exports export turns a miss into one probe. The names are hashed along the trie's edges rather than rebuilt, which keeps the build near 40 ms, and the filter is published with a release store because readers test it without a lock.

The newest rule (2026-10-03) is that only images *already loaded in the guest* count, again as dyld does: it coalesces among the images loaded so far, so a weak definition in a cache image nobody loaded never wins. Searching every cache image let a library that carries its own copy of a C++ template bind to Apple's copy instead. DXMT's `winemetal.so` statically links its own LLVM, and among its 4,481 weak binds is an LLVM template function that an LLVM inside Apple's GPU compiler framework also exports; DXMT's shader compiler then called the other LLVM's code on its own LLVM's objects, failed every shader, and R.E.P.O. aborted. The main program and its startup dependencies are bound before the set of loaded images exists, so until then every cache image still counts. `OCERZ_WEAK_ALL_CACHE=1` keeps the old rule.

Two more rules came on 2026-10-09 (230da272), both again from dyld's published behaviour (dyld-1378, `Loader::resolveSymbol`). A weak-definition lookup takes the first image in load order that has weak definitions and exports the name, and the *main executable comes first*. ocerz asked the shared cache first, so a program that defines its own `operator new(size_t, nothrow_t)`, as libc++ also does, got libc++'s; under Rosetta it gets its own. resolve_import now asks the main executable first when its header carries `MH_WEAK_DEFINES`, and that includes the time while disk dylibs are bound, before the main image is registered. `OCERZ_NO_WEAK_MAIN_FIRST`=1 asks the cache first again. The second rule is about the weak-bind stream of a binary with `LC_DYLD_INFO`. The stream names no library: each of its binds is a weak-definition lookup, which is how dyld runs it. ocerz took them as ordinal 0, the image itself, but searched every image

in the cache before the image, so each template copy a C++ program binds to itself searched 3,900 images and found nothing. `ffmpeg -version` did that 1,743 times, 621 ms. classic_bind_stream now runs the stream as weak-definition lookups (`OCERZ_NO_WEAK_STREAM`=1 keeps the old ordinal), and ClipGrab's 161 MB ffmpeg took 0.11 s instead of 0.71 s, against Rosetta's 0.028 s. The dynamic test `weak_main_first` builds a program with its own weak operator new twice, once with chained fixups, where these binds are weak lookups by ordinal, and once with -no_fixup_chains, where they are in the weak-bind stream; both failed before the change.

The same ordering applies to an import with the main-executable ordinal (−1): the main executable is asked first, as dyld does. A bundle's import of `Sleep` is its launcher's, not the legacy CoreServices `Sleep` that the flat search later in the table would find (275843d4, 2026-10-09).

The weak filter described above no longer has to be rebuilt by every process. It depends on nothing but the shared cache, yet each process that reached 128 weak lookups built it again, 990,000 names from 308 images and what they re-export, 37 ms. Since 94614e9c (2026-10-09) it is kept in the translation store's directory (Chapter 7), under the cache's UUID, with a sum over its bytes, written to a temporary name and renamed into place; it is read at the first weak lookup, which also spares the 128 lookups made without it. `OCERZ_TCACHE`=off reads and keeps nothing, and the store's free-space floor stops the write. Under `OCERZ_TCACHE_LOG`, reading, rejecting and keeping it are logged. `ollama --version`, whose x86 library binds many weak names, took 0.183 s instead of 0.217 s (Rosetta 0.026 s). The dynamic case `dweak_bloom` binds a library's 200 weak definitions three times against one fresh store: the first run must keep the filter, the second must read it, and the third, after the file has been corrupted, must reject it and keep a good one again.

The order of a lookup

Step	When it applies	Where it looks
1	positive ordinal	the named dependency: a loaded disk image by install name or path (with `@`-paths expanded), otherwise that library in the cache
2	ordinal −3 (weak), main executable has weak definitions	the main executable first (230da272, 2026-10-09)
3	ordinal −1	the main executable first (275843d4, 2026-10-09)
4	ordinal −3 (weak)	the weak-defining cache images already loaded
5	otherwise, still unfound	the cache's flat namespace
6	flat, weak or main-executable ordinals	the importing image itself
7	still unfound	every loaded disk image
8	ordinal −1, −2, −3	the main executable (how a plug-in binds to its host program)
9	native mode only	synthesized libraries opened on demand
miss	not weak	dynamic-lookup count, native-mode report (exit 71), or a named fatal

Fixups

Binding writes the resolved addresses into the image; rebasing adjusts the image's own pointers by its slide. The mini-dyld handles all three generations of the format.

- **Chained fixups** (`LC_DYLD_CHAINED_FIXUPS`), what every modern linker emits. Each page's chain starts from a per-page offset and links pointer to pointer; a 64-bit entry is either a bind, with an import ordinal and an addend, or a rebase, with a target. Formats 2 (`DYLD_CHAINED_PTR_64`, target is a virtual address) and 6 (`DYLD_CHAINED_PTR_64_OFFSET`, target is an offset from the image base) are supported, which is everything x86-64 uses; anything else is refused by name. An import is named by ordinal at every location that uses it, so each ordinal is resolved once and the answer reused.
- **Classic opcodes** (`LC_DYLD_INFO`): rebase and bind streams of byte-coded instructions. Binds repeat a symbol across consecutive locations, so the last answer is reused while the symbol, library and weakness stay the same. Lazy binds are applied eagerly, so no lazy-binding helper ever runs.
- **Pre-10.6 relocations.** A dylib built for macOS 10.5 or earlier has no compressed fixup information at all: local relocations are rebased by the slide, external relocations add a symbol's address to the value in place, and the indirect symbol table fills the lazy and non-lazy pointer sections. On x86-64 a relocation's address counts from the first writable segment. ocerz applied none of this, so Steam's `steamloader.dylib`, which Steam injects into every game it launches, jumped to zero on its first call.

Protections after binding

Every image is mapped writable so the copy and the fixups can land; then each segment gets its declared protection. That used to be done for `__TEXT` alone, and everything else stayed writable and said so: `mach_vm_region` reported read-write for `__DATA_CONST` and `__LINKEDIT`. That was not cosmetic. Electron asks about a region it expects to be read-only and executes a trap instruction when the answer is anything else, which is why Discord's renderers died at startup. Now the protection read is `initprot` at offset 60 of the segment command (not `maxprot` at 56, which differs in old images), and a segment flagged `SG_READ_ONLY`, which is how `__DATA_CONST` is spelled, ends up read-only whatever its `initprot` says. It cannot happen at load time, because Objective-C has not finished writing selector references into those segments, so images are queued and protected in a batch after the initializer phase and after every `dlopen`; a `dlopen` that fails takes its images out of the queue.

Initializers

Getting initializer order right is most of `dyld.c`, and it has produced more application-level bugs than any other part of the linker. The rule dyld follows is simple to state: each image's Objective-C `+load` methods and then its own C and C++ initializers run after every one of its dependencies has finished, bottom-up. Neither global ordering works. Running all `+load` methods first runs one framework's `+load` ahead of libc++'s initializer; running all initializers first recurses `libsystem_malloc` into its own zone setup.

Pruning, and who counts as a Cocoa program

libSystem's closure is marked done the moment its initializer returns, so a library whose only dependency is libSystem stops the walk right there instead of descending through libxpc into Foundation and dragging half the system into its subtree. Whether a program needs the full dependency-ordered phase at all cannot be decided from the main executable's own load commands: an application is often a small stub linking

one umbrella framework, and Safari, a Cocoa application by any measure, once read as "not a CoreFoundation program" and skipped the phase entirely. But libSystem's closure cannot be followed either, because libxpc weak-links a library that links Foundation, which made every program, `/usr/bin/sort` included, look like a Cocoa application. The test is made on the program's closure with libSystem's left out.

The set of images whose initializers run eagerly is closed under dependencies, because the walk recurses into every dependency but only runs the initializers of images in the set. Without that, an image in the set could be initialized on top of an uninitialized dependency, which is how Safari aborted in a static initializer that called `operator new` before libc++abi had been initialized.

Upward links

An upward link (`LC_LOAD_UPWARD_DYLIB`) is how a library declares the back edge of a dependency cycle: CoreFoundation links Foundation upward, because Foundation depends on CoreFoundation. It is an ordering edge for nothing, and it took three attempts to treat it exactly as dyld does.

1. **Followed on the way down.** Treating upward links as ordinary dependencies made CoreFoundation's upward link to CoreServicesInternal a real edge, which put several unrelated frameworks inside libc++abi's subtree. libc++abi then sat unfinished on the recursion stack while another framework's initializer called `operator new` into a libc++ that had not been initialized.
2. **Never followed.** Leaving upward targets out entirely broke `sw_vers` on macOS 27. Foundation's code was reachable, since the whole cache is mapped, and its classes registered, but its own initializer never ran. `NSString` therefore stayed an abstract class cluster, and the first `+[NSString stringWithFormat:]` fell into the method that reports an abstract class, which builds its message with `+[NSString stringWithFormat:]`, until the 8 MB guest stack ran out.
3. **Followed right after the declaring image.** Better, and wrong on macOS 26.7.1, where libobjc links libswiftCore upward and libswiftCore links Foundation upward. libobjc is a dependency of CoreFoundation, so Foundation's whole subtree, SkyLight among it, initialized while CoreFoundation was still unfinished on the recursion stack, and SkyLight's first allocation through CoreFoundation's allocator recursed between `malloc` and `malloc_zone_malloc`.

The rule now (`baf668f3`, 2026-10-03) is that an upward target is initialized after the *whole* walk from the root has finished. Each target goes on a list the root call owns, and the list is worked through after the root's own initializers, including targets it adds on the way. That rule was not derived from reading dyld: it was measured. `tests/dynamic/upward_defer.c` builds three libraries of its own in the same shape and logs the order in which their initializers run.

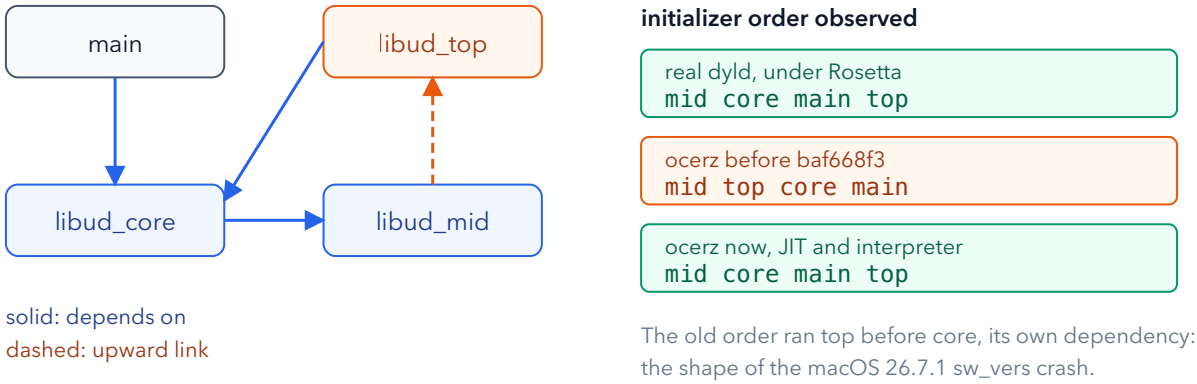


Figure 11.3. The experiment behind the upward-link rule. The libraries are the project's own, so the rule comes from observed behaviour, not from reading Apple's loader.

The dlopen closure

A `dlopen` runs the initializers of the images it adds, through `init_closure`, and three rules apply there that the startup phase did not need:

- **Post-order, never sorted by address.** A dlopened image is mapped below the dependencies it pulls in, so an address sort ran a dependent before its dependency. Steam's `steamui` initialized before `libtier0_s`, and its first memory-container growth called through tier0's allocator singleton while its vtable pointer was still null: a call through offset 0x28 of a null pointer, at exactly address 0x28.
- **`+load` before initializers, per image.** The closure used to run initializers only, so a cache framework first pulled in by a `dlopen`, Foundation under Wine's `ntdll.so`, never had its `+load` run, and `+[NSString alloc]` kept returning the abstract class instead of its placeholder. Wine's `mountmgr` answers Steam's DHCP query through SystemConfiguration; CFBundle built a string on an abstract `NSMutableString`, and the same recursion as above ran `winedevice`'s stack out. Steam then saw no network adapters. (`OCERZ_NO_CLOSURE_LOADS=1` restores the old behaviour; the test is `dlopen_load_phase`.)
- **Upward links followed only to the Objective-C core.** The dlopen closure follows an upward link only when it points at Foundation, CoreFoundation or libobjc, so a program that dlopens SystemConfiguration still gets Foundation initialized. Following every upward link there brought in some 360 more initializers in every Wine process, and Steam's browser process then died in `objc_msgSend`. (`OCERZ_NO_DLOPEN_UPWARD=1`; test `dlopen_objc_core`.)

Initializers run under a dlopen are *tolerant*: if one stops with a fatal fault, ocerz prints `init-skip` with the initializer and the faulting address and carries on, because the process had been running fine and a crash inside one framework's initializer, which a real system might never reach, should not end it. An initializer that fails during startup is fatal, with a backtrace naming each frame's image and offset, and the process exits with 125.

dlopen, dlsym and the image list



A cache image skips mapping and binding: it is already mapped. It still goes through the last two steps, and joining the list must come after its Objective-C mapping, or libobjc takes it as handled and the framework loses its categories. A disk image's bulk callback fires before its initializers.

Figure 11.4. A `dlopen` in cache mode, left to right.

Resolving the path

A path beginning with `@` is resolved against the image that called `dlopen`, as dyld does: `@rpath` through that image's `LC_RPATH` entries and then the main executable's, `@loader_path` against its directory. The caller is the return address of the call into the `dlopen` slot (0x68), or `dlopen_from`'s third argument (slot 0x2e0). Before 2026-09-26 the path went to the loader as written, and the Game Porting Toolkit's `libd3dshared.dylib`, which dlopens `@rpath/D3DMetal.framework/D3DMetal` through its own `@loader_path` rpath, failed and aborted every D3DMetal game at its first Direct3D call (test `dlopen_caller_rpath`).

The same chain, from the image outwards, finds the dependencies of what a dlopen loads (9ce519cd, 2026-10-13). dyld resolves an @rpath name along the load chain: the image's own `LC_RPATH` entries, then those of the image that called dlopen, then the main executable's. Cache mode used only the image's own, so a plug-in with no `LC_RPATH` that links a library from its loader's directory had those imports left unbound. CrossOver's cxcompatdb.so, dlopened by ntdll.so, links @rpath/libgnutls.30.dylib, which only ntdll.so's @loader_path/../../../lib64 reaches, and every CrossOver bottle run under ocerz crashed at startup. A leaf name a program dlopens goes through the same rpaths, after `DYLD_LIBRARY_PATH` and before the working directory; cache mode skipped them, and CrossOver's win32u.so, which dlopens libfreetype.6.dylib, found no FreeType and drew no text. Native mode already did both. The dynamic test dlopen_chain_rpath pins both cases (Rosetta prints OK for it, and 0.7 did not), and `OCERZ_NO_DLOPEN_CALLER_RPATH=1` turns both off along with the caller-rpath path it already covered.

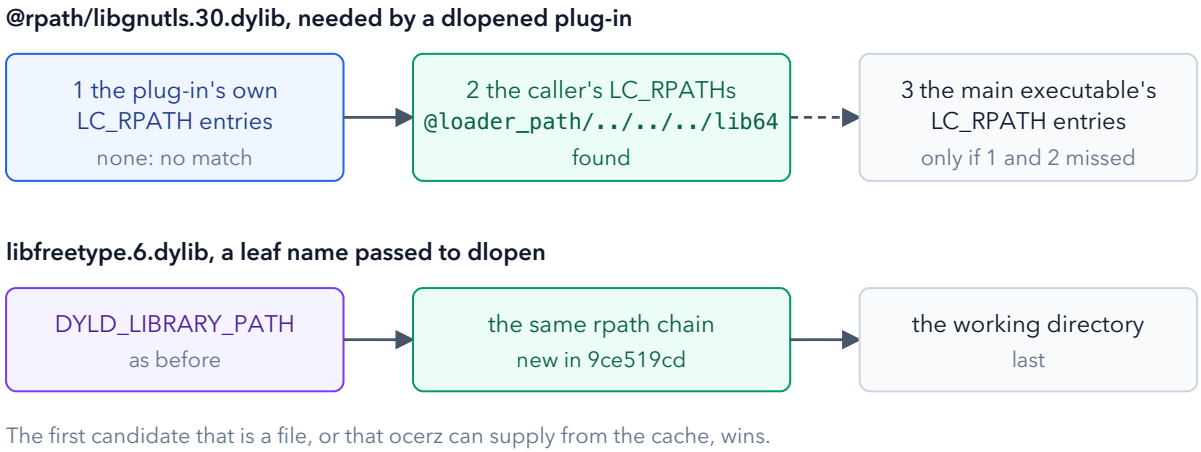


Figure 11.5. Where a dlopen looks for a dependency of the image it loads. Cache mode used only the first box until 2026-10-13.

Which candidate wins matters too. An @rpath name is tried in each directory in order and the first candidate ocerz can supply wins, as dyld's first existing file does (48b4fc08, 2026-10-05). A candidate used to count only if it was a file on disk, and an app built for systems before 10.14.4, which bundles the Swift runtime and lists /usr/lib/swift ahead of its own Frameworks, then loaded its bundled libswiftCore, which stops with "This copy of libswiftCore.dylib requires an OS version prior to 10.14.4" (iGlance). Every later system supplies its own libswiftCore, and it exists only in the shared cache. A candidate now counts when it is a file or an image of the x86 cache; in native mode the guest runtime's copy or an image an API database builds counts too (Chapter 12). dlopen's @rpath expansion uses the same test, and the dynamic test `native_swift_bundled` pins it for both modes.

Some x86-64 libraries exist only in the Rosetta cryptex, and Intel code names them by their system path: Metal opens `/usr/lib/libMTLHud.dylib` for its performance HUD, and there is no such file on disk. So a cache-mode `dlopen` of an absolute path found neither in the cache nor on disk retries it under the Rosetta cryptex.

In native mode a dlopen has one more rule (78d3f9b8, 2026-10-13). A dependency the host has but no API database describes, because it lives in the shared cache or has only an arm64 slice, used to fail the whole dlopen even when nothing imported a symbol from it. Qt's Cocoa platform plugin links AGL.framework, which the SDK no longer describes, and imports nothing from it, so every Qt app (ClipGrab) aborted with "Could not load the Qt platform plugin cocoa". Such a dependency is now passed over, as dyld would load it unused, and an import from it still fails the dlopen, as Symbol not found. `OCERZ_NO_HOST_ONLY_DEP`=1 refuses it as before.

The same image, never twice

A `dlopen` of the main executable's own path returns the main image rather than a second copy. Steam's bootstrapper dlopens its own `steam_osx`, the copy gave duplicate Objective-C classes, and `steamui` crashed on a null vtable (test `dlopen_self`). Loaded images are also matched by device and inode, not only by path: Chromium's sandboxed helpers dlopen the 386 MB Chromium Embedded Framework again through a `Contents/Frameworks` symlink, `realpath` fails inside the sandbox, and each helper loaded the framework twice.

The image list

In cache mode the dyld APIs `_dyld_image_count`, `_dyld_get_image_name` and `_dyld_get_image_header` answer from the image closure `dyldapi.c` keeps. Until 2026-09-22 a `dlopen` of a disk dylib added to that list and a `dlopen` of a cache image did not. The library was loaded, its symbols resolved and `dladdr` knew where they lived, while every walk of the list said it was absent; the count stayed at 45 where a real process went to 348. libobjc keys its per-image queries off that list, and Metal finds its GPU driver through them, so `MTLCreateSystemDefaultDevice` returned nil and OpenGL had no accelerated renderer: every translated program saw no GPU at all, and Brawlhalla drew a window and never drew into it. A cache image now joins the list at the end of its `dlopen`, after its Objective-C mapping and initializers. The order matters: libobjc calls back into the dyld APIs while it maps an image, and an image already in the list when that callback arrives is one it takes as handled, which cost the newly loaded framework its categories (test `dlopen_image_list`).

The initial closure itself was once walked through a fixed 1024-entry queue, which dropped dependencies once full, and it filled early because a library every image links was pushed once per dependent. Safari died in its startup on a method a category should have added. The closure array is now its own work list, deduplicated through an open-addressed set, so each image is walked once; Safari's closure is about 1,500 of the cache's 3,600 images. (Later commits count 3,900 to 3,918 images in the macOS 27 cache: 16f65d59, 2026-10-09.)

Callbacks registered for bulk image loads are kept, not just driven once. libxpc registers one and learns from it which frameworks carry XPC services; driven only at startup, every framework dlopened later had services that launchd answered with "No such process", so a Wine process could not reach the Metal shader compiler service, and `steam.exe` retried a view service a thousand times a second (`OCERZ_NO_BULK_NOTIFY=1` turns this off).

dlsym

`dlsym` on a handle searches the image and then everything it links, breadth first, skipping upward links, which is what dyld answers: a symbol two dependencies down is found, one behind an upward link is not. Measured under Rosetta with the project's own dylibs. The disk-image path used to search the image alone, so `dlsym(handle, "malloc")` failed for every dylib; the cache path followed upward links, so CoreFoundation's handle found an OpenGL function by way of Foundation (test `dlsym_deps`; `OCERZ_NO_DLSYM_DEPS` and `OCERZ_DLSYM_UPWARD` restore the old behaviours).

`OCERZ_DLSYM_UPWARD` is itself read once. Until 36f4297f (2026-10-13) `ocerz_cache_dlsym_image` called getenv for every symbol it resolved through the cache index, which is every dlsym a guest makes into the cache. getenv walks the whole environment, and the commit measured 78 to 203 ns a miss on Linux with 82 to 236 variables; the value is now read at first use into a static, as `OCERZ_NO_LAZY_REMAP` and `OCERZ_EAGER_SLIDE` already were.

The dyld API table

libdyld's exported functions, `_dyld_image_count`, `dlopen`, `dladdr` and the rest, are thin trampolines: each calls through a table of function pointers on an object that dyld provides at startup. In cache mode the mini-dyld provides that object itself. Its table has one entry per eight-byte slot, and entry *n* points at `OCERZ_DYLDAPI_LO + n`, inside a trap window from `0xdda00000` to `0xdda10000` that the decoder, inter-

preter and JIT all watch. A call through slot 0x68 therefore arrives at `0xdda00068`; the run loop sees the address in the window and hands the call to `ocerz_dyldapi_dispatch`, which reads the arguments from the guest's registers, answers, and returns as a function would. ocerz answers 59 slots.

Slots that are code, not traps

Every call through the trap window leaves translated code for `ocerz_dyldapi_dispatch` and comes back through a lookup of the return address. That is cheap once and expensive a million times. CoreFoundation asks slot 0x238 (dyld_program_sdk_at_least) and slot 0x210 in nearly every call: building a 200,000-entry CFDictionary made 1.2 million such calls, 800,000 of them to 0x238 and 400,000 to 0x210, a tenth of the time the build took. Since 90903488 (2026-10-09) five slots whose answer is fixed once the main executable's `LC_BUILD_VERSION` has been read are not traps: `dyldapi_fast_slots` writes a few bytes of x86 code for each into a guest page and points their table entries at it, so the JIT translates and chains into them like any guest function. Slot 0x210 returns 1; 0x188 and 0x190 return the program's SDK and minimum OS versions; 0x238 and 0x240 compare a version asked for against those, for the program's own platform or for any (the platform argument −1). The CoreFoundation build went from 1.05 s to 0.89 s with this change alone (commit message, 2026-10-09). `OCERZ_NO_DYLDAPI_FAST=1`, or `OCERZ_DYLDAPI_TRACE`, keeps every slot trapping, and `tests/dynamic/dyld_apis.c` checks that the answers are true at the program's own versions, false past them and false for another platform, under Rosetta's dyld and under ocerz with and without the fast slots. The dispatcher's switch keeps its own case for each of the five, which is what answers them when the fast slots are off.

Where the leaf routines come from

dyldapi.c also builds the table behind the hand-written string and memory routines of `src/leaf.s` (Chapter 17): it finds where libSystem's _memmove, _strlen and the rest live, so that translated calls to Apple's x86 versions can run the arm64 ones instead, and so that the translator knows which of Apple's routines it may translate with plain accesses. It resolved those names inside libsystem_platform, which found them only through an upward link back to libSystem. On macOS 26.7.1 the x86-64 libsystem_platform links libsystem_kernel alone and exports __platform_memmove and its relatives, which libsystem_c re-exports under the plain names, so every lookup but ___bzero missed: cache mode answered none of the nine entries of the dynamic test sys_strings_inplace, and the translator ran Apple's routines with no plain-access translation. Since 82816498 (2026-10-05) the names are resolved from libSystem, as a program binds them, keeping the answers that land in libsystem_platform's text; on 26.7.1 that gives 10 routines of ocerz's own and 11 chosen through a pointer, and all nine entries are answered in place.

How a slot is known

Which slot is which function is established by running code, never by reading Apple's. `tools/dyldslots.sh` takes every name the SDK's public `libdyld.tbd` stub file exports, calls each one under ocerz from a small probe program with all-zero arguments, and runs with `OCERZ_DYLDAPI_TRACE=1`, which prints every arrival in the table with its arguments and caller. The first slot a call reaches is that export's slot. A slot no export reaches first is named by its caller in a traced run, through `dladdr`: slot 0x358 is called from `_dyld_objc_register_callbacks`. The `--check` mode runs inside the dynamic tests and fails for any slot the dispatcher answers that has no such witness, and for any change in the pinned pairs.

Export	First slot reached	Export	First slot reached
`_dyld_image_count`	0x10	`dladdr`	0x60

Export	First slot reached	Export	First slot reached
`_dyld_get_image_header`	0x18	`dlopen`	0x68
`_dyld_get_image_vmaddr_slide`	0x20	`dlclose`	0x70
`_dyld_get_image_name`	0x28	`dlerror`	0x78
`_dyld_register_func_for_add_image`	0x30	`dlsym`	0x80
`_NSGetExecutablePath`	0x50	`dyld_image_path_containing_address`	0x168

These twelve pairs are pinned in `tools/dyldslots.pinned`. Note that slot 0x30 is witnessed but is not among the 59 the dispatcher answers.

What an unanswered slot returns

A slot ocerz does not implement answers zero in `rax` *and* in `rdx`. The second register matters because a dyld API can return a pair. Slot 0x450, which libobjc's initialization asks for a table of message-send offsets and a count, is exactly that shape: with `rdx` left alone, the count was whatever the guest had in the register, libobjc read a table at address zero, and the damage surfaced much later as a recursive-lock abort inside class realization. That slot is now answered explicitly with an empty table.

Two answers worth knowing

`_NSGetExecutablePath` behaves as dyld's does: a buffer the path fits in gets the path and a size left exactly as the caller set it, and only a too-small buffer has the size rewritten, with -1 returned. It used to write the length back on success too, which an arm64 build of the same program never sees.

Which image holds an address is asked by `_dyld_find_unwind_sections` once for every frame an exception unwinds through, and by the image-containing-address slots. It used to walk the load commands of each image and then of all ~3,600 cache images per call. It is now answered from a sorted table of every mapped segment, searched by bisection with a running maximum of segment ends so overlapping ranges are still found. The table is rebuilt when the closure changes and published with release and acquire ordering, and `__PAGEZERO`, which has no protection, is left out.

Objective-C in cache mode

In cache mode the translated libobjc from the cache is the Objective-C runtime. ocerz's job is to tell it the truth about images, through the callbacks libobjc registers with dyld: the `map_images` batch at startup is the dependency closure, and each image's `+load` methods are delivered before its initializers. Two refinements came from real failures.

Categories of late images. The cache has already merged every startup image's categories into the classes they extend, so startup images never report their category lists. An image mapped after startup does report them, because the classes it extends may already be realized with copied method lists that only a category list can reach (`OCERZ_NO_LATE_CATLIST=1` turns that off).

Categories that must exist before any class is realized. Some images' categories have to be visible before the classes they extend are first used, and a later `dlopen` registers them too late. CoreSpotlight adds coding methods to Foundation's collection classes, and without them the indexing AppKit starts about 20 seconds into a session threw an unrecognized-selector exception inside a dispatch block and ended the process. `OCERZ_PRELOAD_OBJC` moves named cache images' closures into the startup batch; Wine processes get AppKit, QuartzCore, HIToolbox and CoreSpotlight by default, and `@cat` preloads every image that defines categories, at a cost of 3 to 4 s per boot.

Selector canonicalization uses the selector table the cache's published Objective-C optimization header points at. It was once indexed by hand, which took thirteen seconds of startup out of every process that canonicalized a selector.

Thread-local variables

Two descriptor layouts share the same 24 bytes. A static linker emits the classic `tlv_descriptor` of a thunk, a 64-bit key and a 64-bit offset, so the offset is at +0x10. The shared cache ships the packed form, a thunk, a 32-bit key, a 32-bit offset, a signed delta to the initial content and its size, where the offset is the 32-bit word at +0xc and +0x10 is the delta. ocerz always writes the packed form, and it once read +0x10 unconditionally, which took the delta (zero) for a cache image and stored it over the real offset: every thread-local in the image collapsed onto offset zero and they aliased each other. SwiftUI reads a thread-local holding its current property list that way, got a small integer instead, and crashed casting it to a class. A key is small enough that a classic descriptor's 64-bit key leaves +0xc zero, so a non-zero +0xc now means the packed form is already there.

Keeping the launch small

A cache-mode launch used to do work that dyld does not, and on 2026-10-09 three changes cut most of it. Each is pinned by a dynamic test and has a switch that restores the old behaviour. The measure is the launch of a program that links only CoreFoundation, which the commit messages give as 133 ms, then 67 ms, then 42 ms, with Rosetta at 9 ms (Figure 11.6). That is still about 4.7 times Rosetta's launch, so the gap is smaller, not closed.

Delayed-init links stay out

macOS 15 added a flag on a library link, `DYLIB_USE_DELAYED_INIT`. dyld maps and binds such a library, which costs nothing for a cache image, but keeps it off the image list, runs none of its initializers and does not tell libobjc about it, unless an ordinary link reaches it too. A client that needs it dlopens it first, and that dlopen follows only the ordinary links from there (dyld-1378, `RuntimeState::recursiveMarkNonDelayed`). A re-export is never delayed. ocerz followed delayed links like any other. On macOS 27, CoreGraphics links TextRecognition delayed, and that brings CoreML and Vision with it: a program that links only CoreFoundation had 505 images and ran their initializers, where under Rosetta it has 346. `ocerz_dylib_dep_delayed` now leaves those links out of the image closure that dyldapi.c reports as loaded, of the eager set and of the initializer walk with its upward links. After the change the image list of that program is identical to Rosetta's, image for image, and the launch took 67 ms instead of 133 (85b248fc). The dynamic test `delay_init` checks that every image on the list at main is reachable from the main executable through links that are not delayed, and then parses Markdown, which

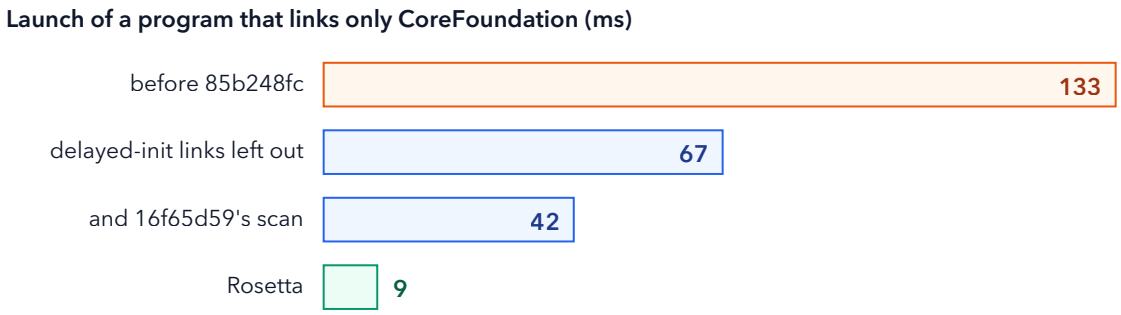
Foundation does through libcmark-gfm on a delayed link: the result must be right, and libcmark-gfm must be on the list afterwards. `OCERZ_NO_DELAY_INIT=1` follows the links again, and the test then fails, naming the same 159 images.

The pointer scan reads less

The eager set, the images whose initializers run at launch, is the dependency closure plus any image a pointer in an eager image leads into, found by `scan_uses`. That scan read every pointer in every eager image's pointer sections: 830,000 pointers for the CoreFoundation-only program, each looked up by binary search in a table of every segment of all 3,918 cache images, and building that table means reading every image's header. Together that was 26 ms of each launch. The scan exists for links the dependency walk did not follow. The cache builder binds a cache image only to the images it links, so when all of an image's links resolve its pointers lead nowhere the walk has not been, except into delayed-init images, which stay out. The scan therefore reads only images that are not in the cache, which ocerz binds itself, sometimes through a lookup across the whole cache, and cache images with a link the walk could not resolve (weak links to images the system lacks, such as libobjc-env, do not count). It searches the eager images' own segments first and builds the table of every segment only for a pointer that leads elsewhere. Programs that link CoreFoundation, Foundation, AppKit and Swift run the same initializers in the same order with and without the change; `OCERZ_SCAN_ALL_USES=1` scans every image again. The launch fell from 67 ms to 42 (16f65d59), and `ollama --version`, with the delayed-init change before it, from 0.304 s to 0.215 s (Rosetta 0.026 s).

What is kept between runs

Three smaller changes sit on the translation store of Chapter 7. The weak-definition filter is kept beside it (94614e9c, above). Every record loaded from the store is summed before it is trusted, to catch a torn or damaged one, and the sum had been a chain of dependent multiplies, one per eight bytes: about 2 ms of the CoreFoundation program's launch for the 9 MB of records it loads. a02fcc3f (2026-10-09) made it CRC32C in three lanes, whose instructions overlap, and the same launch spends 0.9 ms summing, most of it the first touch of pages the sum reads anyway. Each ocerz build keeps its own store, so no record written with the old sum is read with the new one. And since 93b919f9 (2026-10-07) the store leaves free space on its disk: it writes nothing while the volume has less than `OCERZ_TCACHE_MIN_FREE_MB` free (10,240 by default), a process writes at most half of what was free above that floor when it opened the store, and a lone process that finds the volume already under the floor removes the store. That change followed a tester's Mac whose few free gigabytes filled under a Steam session, could no longer swap and was reset by its watchdog. Reading is unaffected, so a full disk costs warm starts, not correctness.



Figures from the commit messages of 2026-10-09; the Rosetta bar is the figure those messages give.

Figure 11.6. The launch of the smallest Cocoa-linked program, before and after two changes of 2026-10-09 that leave out work dyld does not do.

Smaller things that had large effects

The main thread's thread ID. libpthread caches the kernel thread ID next to the thread's TSD, and fills it in only when a thread sets itself up, which the hand-built main thread never did. So `pthread_threadid_np()` returned 0 on the main thread. The mutex protocol stores that ID as the lock owner, and owner 0 looks unlocked, so a mutex the main thread held gave no mutual exclusion at all against other threads: lost wake-ups, corrupted flags, and a Wine explorer that froze. Since 2026-08-18 the loader fills the main thread's cached ID in itself.

Inserted libraries. In cache mode `DYLD_INSERT_LIBRARIES` is honoured: once the startup initializers have run, each library it names is opened as the guest's `dlopen` would, before the program's entry point (Chapter 10 explains how the host's dyld is kept from acting on it first). This, the pre-10.6 relocations and bundle launching together are what let games launched from Steam run under AArchX.

A probe inside the real process. `OCERZ_DLOPEN_PIGGYBACK` loads a chosen dylib, initializers and all, right after the first successful `dlopen` whose path matches: a probe placed inside the real process, on the same thread, at the same point in its life. It fires for cache images too, and it is how a graphics probe walked through Wine's process start showed that the only thing that mattered was the stack the graphics library first ran on (Chapter 15).

> **KEY IDEA**
>
> Almost every rule in this chapter is "do what dyld does", and almost every one was learned the hard way, from an application failing far from the cause. The project's discipline for these rules is to pin each with a dynamic test whose expected output comes from running the same program under Rosetta, and to give each a switch that restores the old behaviour, so that the next regression can be bisected rather than guessed at.

CHAPTER 12

Native mode

Cache mode depends on Apple's x86-64 shared cache, which ships with Rosetta, and Rosetta ends after macOS 27. Native mode is AArchX's answer: the program stays x86-64, but every system library it links is a synthesized x86 image whose exports lead straight into the Mac's own arm64 frameworks. This chapter explains how a call crosses from System V x86-64 into Apple's arm64 ABI and back, how Objective-C, blocks, callbacks, threads and signals cross with it, what it costs, and what still stops it. It is also where the older Intel applications now run: between 2026-10-04 and 2026-10-18 it went from a client and a handful of tools to Swift programs, Objective-C exceptions, 178 library databases and four applications from the Mac OS X 10.5 to 10.8 years.

17,544	178	65,536	87/87
LINES ACROSS ELEVEN FILES	LIBRARIES WITH AN API DATABASE	CALLBACK TRAMPOLINES	NATIVE-MODE GATE, 2026-10-13

The idea

A program compiled for an Intel Mac imports `printf` from `/usr/lib/libSystem.B.dylib` and `CFRelease` from CoreFoundation. In cache mode those names bind to x86 code inside the shared cache, which is translated like the program's own. In native mode there is no x86 system code anywhere: ocerz builds, in memory, an x86-64 Mach-O for each library the program links, with a real header, real segments and a real export trie, but where each exported function is a twelve-byte stub that traps into ocerz. ocerz then performs the call for real, against the host's arm64 implementation of the same function, moving arguments and results between the two calling conventions on the way.

So the program's own code is translated exactly as in cache mode (Chapters 4 to 7), and everything the program asks of the system runs as native arm64 code at native speed. Native mode is selected with `-native` or `OCERZ_MODE=native`; cache mode remains the default because it runs more software. It was built quickly: the first synthesized libSystem landed on 2026-09-16, CoreFoundation, the API database, structures by value, Objective-C and the guest's own classes on 2026-09-17, `NSApplicationMain`, `dlopen` and processes on 2026-09-18, and on 2026-09-21 the x86-64 Steam client came up in native mode on macOS 27, with its GPU process running hardware-accelerated graphics through the Mac's own OpenGL.

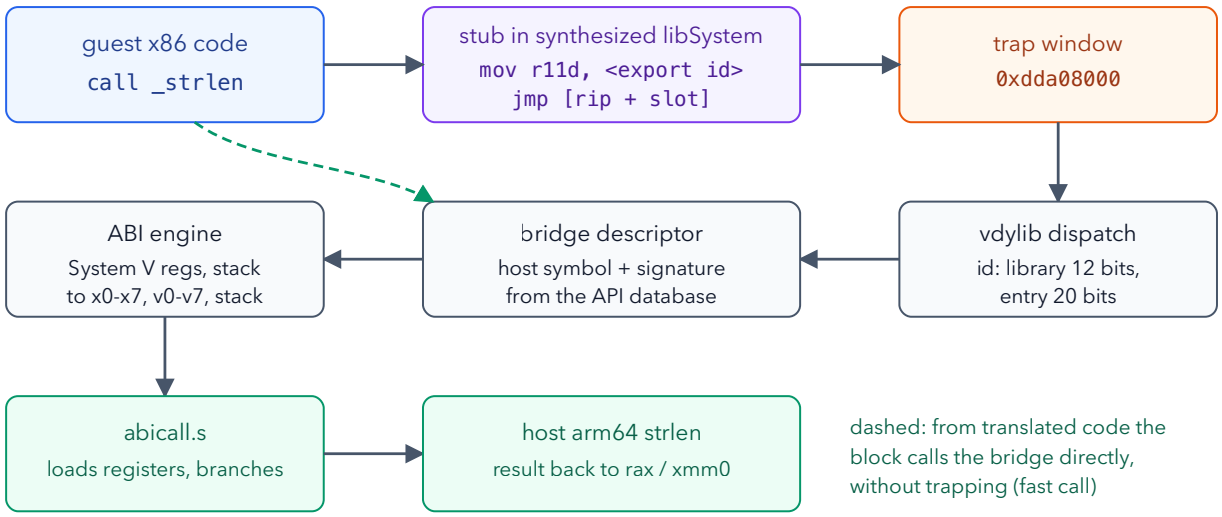


Figure 12.1. A call out, from guest code into a native function. Every synthesized stub jumps through a slot holding the same trap address; `r11` says which export it was.

Two weeks of growth

This chapter was first written at 90fe4d8e, on 2026-10-04. Seventy-nine commits about native mode landed in the fortnight after it. They are described where their mechanisms belong, but the order they came in is itself informative, because each stage was found by running something and reading what stopped it:

Date	Commit	What it added
2026-10-04	`06d1b4da`, `f2ab14b7`	Swift programs on a guest copy of the Swift runtime, then Swift Concurrency
2026-10-04	`2a583d38`, `ef381864`, `643406e5`	databases for 29, then 7, then 74 more libraries, chosen by a census of imports; `long double`, fenv on the guest's own state, typed malloc zones
2026-10-04	`af333877`	Discord's splash screen and gateway
2026-10-05	`46349987`	Objective-C exceptions
2026-10-05	`3dcf68d1`, `045f27e2`, `56edc184`	`imp_implementationWithBlock`, function-pointer results, callback structures for streams, data consumers and patterns
2026-10-05, 06	`4f200419`, `2d59b306`, `22221270`, `0aae84f0`	the calls Wine, Steam, R.E.P.O. and D3DMetal make, under Wine's low address window
2026-10-09	`275843d4`	the Intel-game fixes brought in from a contributor's iOS and iPad project (credit: Jfishin)
2026-10-10	`0ca24d64`	32-bit i386 programs run, through m32 (Chapter 13)
2026-10-13	`f2e350ab`	a `fork` inside a callback is performed
2026-10-16	`334a5e1f`	Transmission, Sequel Pro, HandBrake and VLC 2.2 open their windows
2026-10-18	`ad51f4b3`, `beed19af`	cheap callbacks and a per-thread Objective-C send cache

Synthesized libraries

`src/vdylib.c` builds the images. The design rule is that the loader must not be able to tell a synthesized image from a file. The loader already works out of a host buffer, copying segments out of it and walking the export trie inside it, so the builder produces exactly what a linker would lay out on disk and the buffer *is* the file. Mapping, import resolution, `dlopen` and `dladdr` then handle synthesized images with no special case anywhere.

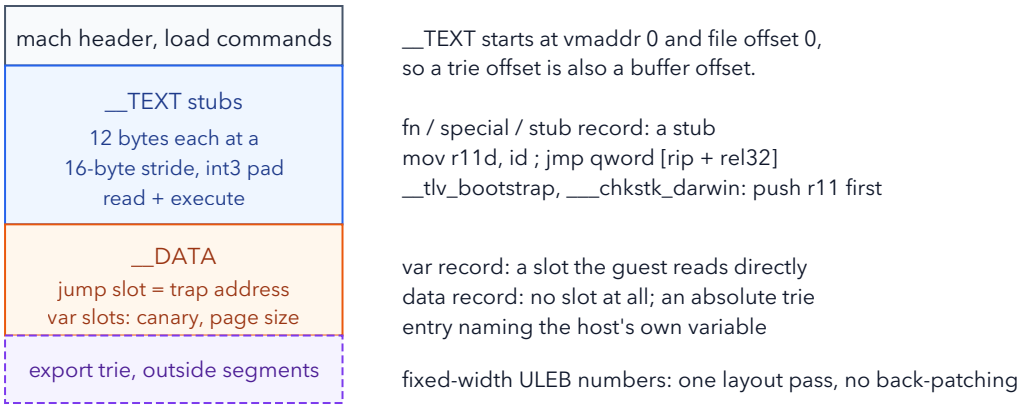


Figure 12.2. A synthesized image. It needs no rebases, binds or chained fixups: the stub's jump is `rip`-relative and its slot holds a constant.

The stub

Every function export is `41 bb <id32>`, `mov r11d, <export id>`, then `ff 25 <rel32>`, `jmp qword [rip + rel32]`. The slot the jump reads lives in `__DATA` and holds one address for the whole process, `OCERZ_DYLDAPI_LO + OCERZ_BRIDGE_OFF`, which is `0xdda08000`, inside the trap window the decoder, the JIT and the interpreter already watch for the dyld API (Chapter 11). So a call into a synthesized framework needs no new instruction and no new range check: the guest jumps to an address that traps, and `r11` says which export it was. `r11` is free to use, because it is a scratch register in System V and every linker stub on the platform clobbers it.

Stubs are twelve bytes placed every sixteen, so a stub's address is its index shifted, which keeps the trie address, the jump displacement and the slot all derivable from one number. The four bytes of slack are `int3`, so a stray branch into the padding stops rather than running zeros.

Two exports cannot spend `r11`. `__tlv_bootstrap`, the thunk thread-local variable descriptors call, must preserve every register but `rax`, and clang does keep live values in `r11` across a thread-local access; a program that loaded one thread-local into `r11` and then touched another read back garbage. `___chkstk_darwin`, the stack probe, has the same contract. Their stubs push `r11` before loading the ID, still within the sixteen-byte stride, and their handlers restore it.

Export IDs

An export ID is the library's ordinal in its top twelve bits and the entry's index in its database file in the low twenty. The ordinal is the position of the library's file among the version directory's files sorted by name. Numbering libraries in load order would have given the same libSystem different IDs, and different bytes,

depending on whether a program named CoreFoundation first; numbered by the directory, an ID means the same export in every process that reads the same database. The last ordinal belongs to no file: it numbers ocerz's own trampolines, such as the x86 entry of a native block handed to guest code.

The export trie

Each child edge in an export trie carries the absolute offset of the child node, so a node's size depends on how wide its children's offsets encode, which depends on the sizes of the nodes before them: minimal encoding needs a fixed-point iteration that is not guaranteed to settle. The builder instead writes every offset and terminal address as a padded ULEB128 of fixed width, which the format allows. Offsets and stub terminals are five bytes, carrying 35 bits; a native data terminal carries a host address and takes ten bytes, every bit a 64-bit value has. With every width known up front, layout is one pass and emission a second, with no back-patching. The trie is a real radix trie, because the walker takes the first child whose edge matches as a prefix: with a flat root, the edge `_write` would swallow a lookup of `_writev`.

Exports that are not functions

A **var** export is a slot in the image's own `__DATA`, filled when the image is built by a named filler, and the trie points at it exactly as it points at a stub. The main one is `___stack_chk_guard`, the canary every stack-protected function copies in its prologue and compares in its epilogue; clang turns the protector on by default, so almost every program imports it. The canary is drawn from `arc4random_buf` with its lowest byte zero, so a string copy climbing out of a buffer, which can write zero only as its terminator, cannot rewrite it and continue. The page-size fillers write 4096, 4095 and 12 behind `vm_page_size` and its relatives, because x86 programs are compiled knowing 4 KB pages and this host's are 16 KB: Chromium's allocator works in 4 KB pages by constant while V8 asks `sysconf`, and stops when the two disagree.

A **data** export takes no slot at all. Its trie entry is absolute, with the host variable's own address as its value, so the guest's reference binds straight to the native object. That matters for identity: a `CFSTR` literal's first word is bound to `___CFConstantStringClassReference`, the isa by which CoreFoundation and the Objective-C runtime recognize it as a string at all, and a guest may hand `&kCFTypeArrayCallBacks` to a CoreFoundation that compares the pointer with its own. A copy would be a second object at a second address that nothing recognizes. It works because native mode runs a position-independent guest in the identity map, where a host address is a guest address. The same applies to `_environ`, `___progname`, `___stdoutp`, `___stderrp` and `__DefaultRuneLocale`: each is a variable native code also reads or writes, so a guest copy would go stale at the first write by either side. A name the host does not have is left out of the trie, so a program importing it fails to bind and says which name it wanted, instead of running with a reference to address zero.

The API database

What each library exports, and how each export crosses, is data rather than code. Each library has one text file, named for the last component of its install name, under `runtime/apis/macos/<version>/`, found beside the `ocerz` binary or under `OCERZ_APIDB`. The version directory is chosen once per process from the minimum macOS version the program's main image declares, compared as packed numbers so that 10.13 sorts above 10.9. The files are generated from the builder's own SDK by `make apis` and are not committed: they are derived from Apple's SDK.

Record	Meaning
`fn <export> <host-symbol> <signature>`	a bridged function: calls cross to the host symbol under the signature
`data <export> <host-symbol>`	an absolute export naming the host's own variable
`var <export> <bytes> <filler>`	a slot of the image's own, filled by a named filler
`special <export> <handler>`	a function ocerz answers itself
`stub <export> <reason>`	binds, but names itself and stops (exit 72) when called; the reason is one word such as `variadic` or `long-double`
`shape`, `struct`	the layout of a structure of function pointers, and which argument of which function carries one
`inplace <export> <arg> <offset> <sig>`	a function pointer inside a guest-owned structure the native function uses where it lies, converted for the length of one call (zlib and bzip2 allocators)

A file that breaks any rule is refused whole, naming its path, line and the rule, because a library half described would bind some imports to the wrong thing. Signatures are checked by the same parser the ABI engine uses, so a signature the file accepts is one the crossing can take.

`tools/sdkgen.sh` generates a file by reading the library's `.tbd` stub for the exact list of x86-64 exports, then parsing its headers with the Command Line Tools' libclang twice, once for x86-64 and once for arm64. An export whose declaration maps onto the engine's classes on both architectures crosses; anything else becomes a stub with a reason. `tools/sdkgen/overrides` holds what no header can say: the functions ocerz answers itself, callback structure shapes, and calls a generic crossing would get wrong inside an emulator. Each run writes its coverage to `tools/sdkgen/baseline` and fails if a crossing was lost.

Library (coverage baseline at 6e6c64b5)	fn	data	special	stub
libSystem.B.dylib	2,871	505	242	5,508
CoreFoundation	803	1,232	9	1,038
CoreGraphics	751	569	2	3,660
Foundation	117	1,548	4	7,345
AppKit	82	4,033	0	4,362
All 177 libraries	25,381	79,738	306	113,108

Foundation and AppKit have few `fn` records because their API is mostly Objective-C methods, which cross through message sends rather than through C functions. Most stubs are exports no public header declares at all. These counts were made by counting record types in the generated files of the checked tree on 2026-10-18 (the totals also include 134 `var` records, 88 `inplace`, 43 `struct` and 30 `shape`). The first edition's table counted a different generation of the same libraries, and since the files depend on the SDK of whichever machine ran `make apis`, a different Mac gives different numbers; compare only counts made from one set of files.

How sdkgen decides

Nothing in the table above was typed by hand. `tools/sdkgen/sdkgen.c` (3,044 lines) is the program behind `tools/sdkgen.sh`, and the way it combines its three inputs is the reason a wrong crossing is so rare. The `.tbd` says which names exist for x86-64; two libclang parses of the headers say what each name is on x86-64 and on arm64; and the overrides file says what neither can. A function crosses only if both parses give a signature the engine's classes can express and the two agree. Everything else is a stub that carries the reason, which is what makes the stub count a to-do list.

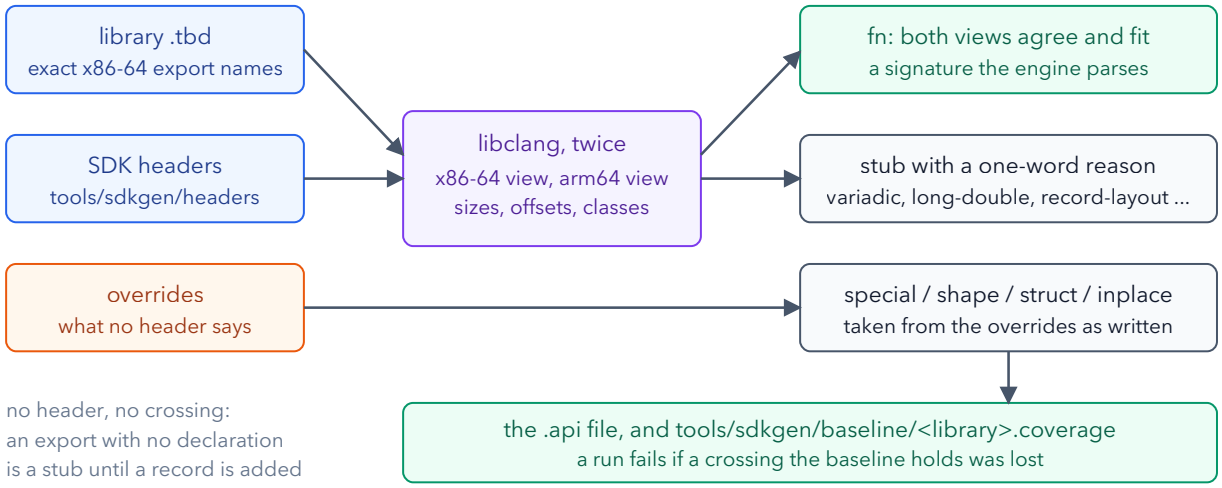


Figure 12.3. How a database file is generated. The baseline is what turns a regression in the SDK, the generator or the overrides into a failed run rather than a program that stops later.

The generator kept learning cases during the fortnight, and each is a rule the headers cannot supply by themselves:

- **Structures that cross although they are packed.** A structure of sixteen bytes or less whose offsets are natural and only whose alignment is smaller crosses by value when every argument of the call fits arm64's eight general and eight floating-point registers, because the alignment is read only on arm64's stack (`e917251d`, 2026-10-05). That let through Carbon's `RegisterEventHotKey` (its `EventHotKeyID` is `pack(2)`) and CoreServices's `AbsoluteTime` calls. A structure over sixteen bytes with natural offsets and size and a smaller alignment alone, CoreMedia's `CMTime` under `pack(4)`, crosses too, since neither ABI reads the alignment at that size (`2a583d38`); every function in CoreMedia and AVFoundation that takes a `CMTime` now crosses.
- **A header that is not the library's own.** OpenGL.framework re-exports libGL, and its database collects libGL's exports, but it parsed only `OpenGL.h`, so every `gl` function a program imports from it was a stub with no declaration. Parsing `gl.h`, `glext.h`, `gl3.h` and `gl3ext.h` as well took the crossing count from 52 to 1,296. CoreAudio's entry is parsed as Objective-C so that the process-tap calls, declared in a header only Objective-C can include, get signatures.
- **Typedefs that hide a different type.** A `dispatch_time_t` is tagged by its typedef with the class `T` (below), `GLhandleARB` crosses as the integer it holds (`d19f1f56`, 2026-10-13), and float and double variables reach i386 code (`c3654064`).
- **Libraries with no headers at all.** libxo, libmd, libarchive, liblzma, libxcselect, libMobileGestalt and libDiagnosticMessagesClient have databases whose exports are all named stubs, so a program

importing them binds and says which call it needed. libffi was left out on purpose: its calls take and make function pointers that it calls itself, which would be x86 code called from arm64 (`643406e5`).

- **Private signatures from open sources.** Where a call is in no SDK header but its signature is fixed in Apple's open-source releases, it gets a record that says so: the `os_variant` queries, the `__ulock` pair, `os_unfair_recursive_lock_trylock`, `voucher_adopt`, `mach_msg2_trap` and others, in `ef381864`, `af333877` and `22221270`. D3DMetal's link to CoreAnalytics got a database from the SDK's `.tbd` with its two event calls declared as Apple's open-source Security and python wrappers declare them (`0aae84f0`, 2026-10-06).

How far it reaches: the import census

Whether a program binds can be measured before running anything, and from 2026-10-04 that measurement chose what to build next. On a macOS 26.7.1 machine with 2,913 x86-64 executables and libraries, every import of each was matched against the databases and the unbound and stub-bound ones were ranked by how many binaries wanted them. Each commit that followed took the largest items it could cross:

Commit	Date	Binaries that bind	Also counted
before `2a583d38`	2026-10-04	1,789	1,095 import nothing that is a stub
`2a583d38`	2026-10-04	1,976	1,238 import nothing that is a stub
`ef381864`	2026-10-04	2,013	71 of 84 command-line tools print what their arm64 slices print
`643406e5`	2026-10-04	2,073	every binary of 35 of 57 third-party applications binds
docs/ compatibility.md	2026-10-05	**2,094**	1,344 import nothing that is a stub; 36 of 57 applications; 1,117 of 1,260 third-party binaries

Binding is not running. The census says that a program will not stop at bind time, not that the first call into a stub will not stop it, which is why the rows also count binaries that import nothing that is only a stub. What kept the rest from binding was mostly Swift: the Swift interface of Foundation and the Swift-only frameworks, SwiftUI and Combine above all, and then private frameworks the SDK describes nothing of. The ranking also found work that no single application would have: 553 binaries import the Objective-C personality routine and 340 `objc_begin_catch` (`46349987`), 22 import `imp_implementationWithBlock` and 38 `__udivti3` (`3dcf68d1`, `e917251d`).

The ABI engine

`src/abi.c` exists because the two calling conventions disagree in ways that make the *n*th argument rarely land in the *n*th register on either side. x86-64 System V passes integers in `rdi, rsi, rdx, rcx, r8, r9` and floating-point values in `xmm0` to `xmm7`; Apple's arm64 passes integers in `x0` to `x7` and floating-point values in `v0` to `v7`. Each side counts its integer and floating-point arguments in separate sequences, so one `double` in the middle of a signature shifts nothing on one side and everything on the other, and the two disagree again about which arguments overflow to the stack and how. So the mapping is computed from a signature.

The notation

A signature is a result class followed by argument classes in parentheses: `i(pp)` for `strcmp`, `p(pLL)` for `memcpy`, `d(d)` for `sin`.

Class	Meaning	Class	Meaning
`v`	nothing (result only)	`p`	pointer, converted between guest and host views
`b B`	8-bit signed, unsigned	`f d`	float, double
`h H`	16-bit signed, unsigned	`{...}`	a structure by value, members in braces
`i u`	32-bit signed, unsigned	`c{sig}`	a callback: a function pointer with its own signature
`l L`	64-bit signed, unsigned	`k{sig}`	a block, with its declared signature
`D`	long double: read in x87 format, passed as a double	`T`	a `dispatch_time_t`, converted between Intel and host ticks

Two classes and one widened rule came from running programs rather than from reading headers. `D`, `long double` (`2a583d38`, 2026-10-04): an argument is read from the guest's 16-byte-aligned stack pair in the x87 format and passed as the double that Apple's arm64 `long double` is, and a result is pushed on the x87 stack, so `strtold` and the `long double` math functions cross; one inside a structure, callback or block is still refused, and the precision beyond a double is lost, which the limits section notes. `T` is for time and is described with Swift below. And a result may now be class `c`, a function pointer (`045f27e2`, 2026-10-05): a function returning one was a stub, which stopped 100 exports, among them Carbon's `New...UPP` constructors and CoreFoundation's `Get...CallBack` calls. The bridge hands the guest its own function when the native one is ocerz's callback onto it, as a constructor that returns its argument does, and otherwise a thunk the guest can call, made once per native function. 66 exports cross that way; only a callback whose own result is a function pointer is still refused, because the notation of a callback has no room for another callback.

The disagreements that matter

- **Narrow integers.** System V leaves the bits above a narrow argument undefined; Apple's arm64 makes the caller extend an argument narrower than 32 bits and the callee extend a narrow result. CoreFoundation passes a `Boolean` (`B`) and a `UniChar` (`H`) almost everywhere. So a narrow value is only ever read as its own 8 or 16 bits and extended to all 64, by sign or by zero, before it is written anywhere. `i` and `u` are treated the same way at 32 bits, which is why a signature must say `int` where the header does, not `long`.
- **The stack.** System V gives every stacked argument an eightbyte; Apple packs a stacked argument at its own size and alignment. Eight `int`s and then `char`, `short`, `char`, `int` land at stack offsets 0, 2, 4 and 8.
- **Floating point** travels as raw bit patterns, never as C doubles, so nothing is converted on the way. A float is the low 32 bits of its slot, which is where the arm64 `s` register lives inside its `v` register.
- **Rounding mode.** A guest may set a rounding mode through MXCSR, which the JIT honours by driving the host FPCR. A host function compiled for the default mode must not inherit it, so the crossing restores the default for the call and puts the guest's back afterwards, writing FPCR only when the bits actually differ.

Most crossings need none of the machinery: a signature of scalars only, within six integer and eight floating-point arguments, goes through `ocerz_abi_perform_registers` straight from guest registers to host registers with no stack block.

Structures by value

Objective-C passes and returns structures by value constantly: `-frame` returns a `CGRect` (`{{dd}{dd}}`), `-rangeOfString:` an `NSRange` (`{LL}`). A structure crosses as bytes: gathered from wherever one ABI put it into a buffer laid out as the structure, pointer members converted in place, then scattered wherever the other ABI wants it. The classifications differ often enough that nothing may be assumed:

Structure	x86-64 System V	Apple arm64
`{LL}` NSRange	`rdi, rsi`; returned in `rax, rdx`	`x0, x1`
`{ff}`	both floats in one `xmm`	two `v` registers (homogeneous aggregate)
`{{dd}{dd}}` CGRect	MEMORY: on the stack, returned through a pointer in `rdi`	`d0` to `d3`
`{df}`	`xmm0, xmm1`	two `x` registers (not an aggregate)
`{fffff}`, `{LLL}`	MEMORY	pointer to a caller-made copy; returned through `x8`

A spilled structure leaves the registers free for later arguments on x86 but closes them on arm64. Over sixteen scalar members (`CATransform3D`'s count) or eight levels of nesting is refused, and a union has no notation at all.

A union whose members are all integers and pointers, with nested structures, unions and arrays of them, is bytes on both sides, because x86-64 classes every eightbyte INTEGER and arm64 sees a composite with no floating-point member. The Objective-C converter lays such a union out and writes it as integers of its own alignment that fill its size (`2c4ba904`, 2026-10-05). It was found when Obsidian stopped as a `<select>` opened, refused by name on a method whose type encoding passes `optional<int>` by value. The class sweep in `tests/unit/test_objcbridge.c` sizes each union with the runtime's own `NSGetSizeAndAlignment` and agrees on all 17,761 method encodings of the host's classes. A union holding a float or a double is still refused, since there the two ABIs can disagree.

Making the call

`src/abicall.s` is the last step, and the one no C prototype can express: it receives two arrays of eight words (the `x` and `v` register contents), a block of stacked bytes, an address for `x8` and two arrays for the result. The registers it must load are the registers the call arrived in, so everything still needed is moved into callee-saved `x19` to `x23` first, and the eight register pairs then load in one run with nothing live in `x0` to `x7`. Done in halves, the call would be right for the arities whose registers happen not to overlap and quietly wrong for the rest. The stacked block is copied to a freshly cut, 16-byte-aligned area at `sp`, and after the call `x0`, `x1` and `d0` to `d3` are stored, which covers every place a result can come back.

Variadic calls

A signature cannot say where a function's named arguments stop, and that is exactly where the ABIs part company. System V passes a variadic argument wherever a fixed one of its class would go; Apple's arm64 passes every variadic argument in its own eight-byte stack slot, doubles included, with no floating-point register used. A variadic function bridged by an ordinary signature would be quietly wrong, so the database carries generic variadics as stubs, and the ones a format string or a sentinel describes are special records with veneers: the `printf` family, `NSLog`, `CFStringCreateWithFormat`, `syslog`, `warn`, the wide `printf`s and the `scanf` family's main members. A veneer reads the format, gathers the variadic arguments from System V's registers and stack through `ocerz_abi_va_start` and `ocerz_abi_va_arg`, packs them into eight-byte slots and calls the function's `v` form with a pointer to the slots as its `va_list`, because an Apple arm64 `va_list` is nothing more than that. Positional arguments, `long double` and `%n` are refused by name, because a guess at any of them reads the wrong slot for every argument after it.

The set of variadic veneers grew in the same fortnight. `err`, `errx`, `errc`, `warnc` and their `v` forms, `NSLogv`, CoreFoundation's `...AndArguments` calls, `sscanf_l`, `asprintf_l` and their `v` forms became format veneers (`2a583d38`, `af333877`), the four `CFCalendar` calls that take one variadic argument per component letter have one, and `glob` is special because the `glob_t` callbacks only matter under `GLOB_ALTDIRFUNC`, which is refused by name. The `fenv.h` functions stopped being plain crossings (`ef381864`, 2026-10-04): the host numbers its exception flags and rounding modes differently from x86, so `fesetround(FE_DOWNWARD)` asked for an arm64 mode that does not exist and the crossing then restored the host's mode anyway. They are special and work on the guest's x87 control and status words and MXCSR, plus the FPSR flags translated arithmetic raises, matching what x86 libm does under Rosetta call for call. The 128-bit helpers `__udivti3`, `__umodti3`, `__divti3` and the rest of compiler-rt's family are special too, computed from the register pairs System V passes them in; division by zero raises `SIGFPE` with `FPE_INTDIV` and out-of-range conversions saturate, which was checked against x86 compiler-rt under Rosetta (`e917251d`).

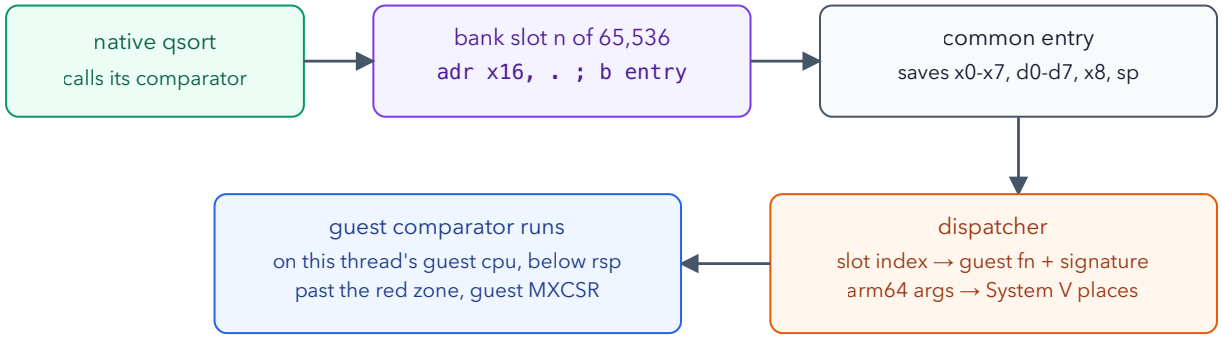
Calls from translated code

The trap path costs a round trip through the run loop: the block exits, the run loop finds `rip` in the trap window, the bridge runs, and the return comes back in through the dispatcher with the host return-address stack abandoned. So when a translated block ends in a synthesized stub's `mov r11d` / `jmp` pair and the slot holds the trap address, the JIT turns the jump into a call to `ocerz_vdylib_fastcall` made from inside the block, followed by the `ret` the export would have ended with. The fast call notes `rsp` and the JIT's retirement count, performs the dispatch, and lets translated code carry on only if the result was an ordinary return: `rip` is the word just below the new `rsp`, `rsp` rose by eight (sixteen for the `r11`-keeping stubs), nothing asked the thread to stop, and no translation was retired meanwhile. A delivered signal, among other things, sends it back through the dispatcher exactly as the trap path would. Only the `xmm` registers the signature reads or writes are spilled around the call, which the API database record supplies.

Eleven routines do not cross at all. `strlen`, `strnlen`, `strcmp`, `strncmp`, `memcmp`, `bcmp`, `strchr`, `memchr`, `memcpy`, `memmove` and `memset` are called constantly and do very little, so `src/leaf.s` holds arm64 versions that read their arguments from the host registers full register pinning keeps `rdi`, `rsi` and `rdx` in, and leave the result in `rax`'s register (Chapter 7). A short `strlen` costs 2 ns that way, against 17 ns for a crossing. A fault inside one of these routines is attributed to the branch that reached it, so a guest that hands `strlen` a bad pointer gets its own `SIGSEGV` handler with its callee-saved registers intact.

Calls back into guest code

Native functions take function pointers and call them: `qsort` its comparator, a run loop its observer, `pthread_create` its start routine. A pointer the guest supplied names x86 code native code cannot jump to. So an argument of class `c` carries the callback's own signature, `qsort` being `v(pLLc{i(pp)})`, and the guest function is **interned**: bound, with that signature, to one slot of a fixed bank of 65,536 arm64 trampolines. Native code receives the slot's address. The same function with the same signature always gets the same slot, so a library that compares callback pointers still sees one function, and a program that submits the same work a million times uses one slot.



A thread with no guest cpu (a libdispatch worker, an audio thread) is given one on its first callback and keeps it. The result goes back in x0/x1, d0-d3, or through the buffer the native caller passed in x8.

Figure 12.4. A call back, from native code into a guest function.

The bank is ordinary code assembled into ocerz, not generated at run time. Generating a trampoline from inside a native callback would race the translator for the one writable JIT arena, a fork child abandons that arena while native code may still hold pointers into it, and a real-time audio callback cannot wait for code generation at all. Each slot is two instructions, an `adr` of its own address into `x16` and a branch to the common entry, so the bank is one repeated block, 512 KB of text, and a slot's index is its distance from the start divided by eight. Exhausting the bank is a named refusal, never a silent reuse.

Not every function pointer a guest passes is guest code. A guest that copies `kCFTypeArrayCallBacks` copies native retain and release functions, and interning one of those would make the translator decode arm64 instructions as x86. So every `c` argument is tested by address: inside the guest reservation is guest code, inside the host's shared cache is not, and anywhere else is guest code exactly when host dyld does not know the address, because ocerz's own loader maps every guest image and never registers it with dyld.

When a slot is called, the dispatcher reads the signature against the native caller's registers and stack, packed Apple's way, places each value where System V puts it, and runs the guest function on the calling thread's guest cpu, below its stack pointer and past its 128-byte red zone, with the guest's MXCSR rounding mode applied for the duration. While it runs, the thread's bridge frame (below) is saved and cleared: a comparator that follows a bad pointer is taking an ordinary guest fault, and reporting it as a fault in `qsort` would kill a process that was fine. On 2026-09-21 a call back cost about 430 ns, against about 17 ns for a call out, and most of it was three system calls that saved and cleared the signal mask and alternate-stack state for the callback's fault-recovery point. That is no longer true; the next section says why and what it is now.

Entering guest code cheaply

A call from native code into guest code is the path of a `qsort` comparator, a Cocoa delegate, a CoreFoundation callback and, for 32-bit programs, every AppKit message (Chapter 13). On 2026-10-18 it cost 536 ns in native mode, where Rosetta runs the same comparator in 4.2 ns inside the x86 `qsort`. Sampling the callback put 59% of its time in three system calls and 18% in copying the 8 KB `OcerzCPU` record. `ad51f4b3` removed both, and a third group of small costs:

- **The signal mask was read on every callback**, to clear what a host thread had blocked. Only a thread's outermost guest loop needs that: a callback nested inside `run_cpu` or another callback was called by a native function that guest code called, which leaves the mask alone. A per-thread count of guest loops (`t_guest_loops`) now says which callback is outermost.
- **The recovery point saved the mask and the alternate-stack state.** `sigsetjmp(jb, 1)` did both on every callback, to serve the rare jump back when a guest fault lands inside it. The point is now taken as `sigsetjmp(jb, 0)`, and `callback_recovered` does after a jump what `siglongjmp` would have: it empties the mask and, unless the point is itself on the signal stack, takes the thread off that stack with sigreturn's `UC_RESET_ALT_STACK` style, which only clears the on-signal-stack flag (this is what `bsd/dev/arm/unix_signal.c` in Apple's published XNU source defines, and what libc's own `longjmp` relies on). The `signal_callback_fault` case exercises this through 338 recoveries.
- **The callback's cpu copied everything.** It now copies its caller's record less the return-address stack, which starts empty (a translated `ret` below its own pushes finds a stack top of 0 and looks its target up; with `OCERZ_NO_HOST_RAS=1` the stack is zeroed instead, because there returns wrap without that check) and less the system-call ring: 2.2 KB, not 8.2.
- **Its exit did work that was almost always nothing.** It added 62 signal counters back with atomics when they were zero, `ocerz_apply_mxcsr_round` called `fesetround`, which wrote FPCR every time, and `guestprof_start` read an environment variable on every callback.

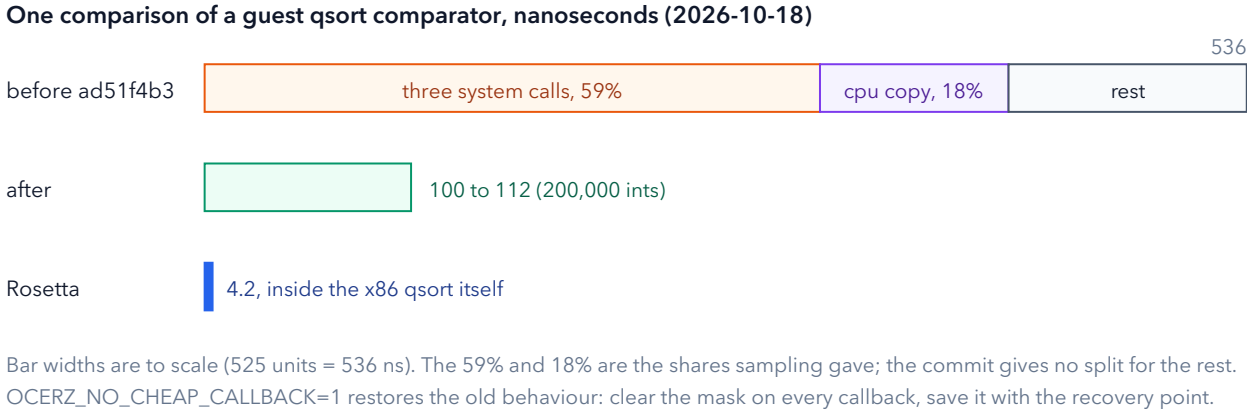


Figure 12.5. What a callback cost before and after `ad51f4b3`. The guest comparator still costs more than twenty times what Rosetta's x86 comparator costs, because here the comparison crosses between two worlds on every call.

A qsort of 200,000 integers with a guest comparator went from 536 to 100 to 112 ns a comparison. The same commit documents the knob: `OCERZ_NO_CHEAP_CALLBACK=1`. A quirk of the measurement worth knowing: the 430 ns in the first edition's table and the 536 ns here come from different loops and different days, so compare a commit's own before and after, not the two figures.

A guard that was Wine's alone

The new `signal_callback_fault` case exists because of a second bug in the same area. `crash_handler` counts faults at one guest address in a row and, past sixteen, took the thread for one of Wine's foreign workers looping on a dispatcher state it cannot repair, and ended it (`WORKER-TERMINATE`), whenever it had no guest signal stack. Every program that is not Wine has none. A handler that unprotects a page and returns, as a garbage collector's write barrier or a JVM safepoint poll does, faults at the same address again the next time, and the seventeenth time its thread ended: a sort whose comparator read a protected page on every comparison exited 241 in cache mode after sixteen of its 338 faults. A Mac delivers every one. `ff6ea313` (2026-10-18) restricts the count to Wine processes, the foreign workers it was written for.

Callbacks that are not quite callbacks

The section above describes a pointer the guest hands over and native code later calls. Older and odder programs found four neighbouring cases:

- **A function the guest stored where native code reads one.** HandBrake points `stderr->_write` at its own x86 function. In native mode `stderr` is the host's `FILE`, so libc jumped into x86 code. That fault now resumes in `ocerz_abi_bounce`, which runs the guest function as a callback with the native call's six integer arguments (`334a5e1f`, 2026-10-16; `OCERZ_NO_NATIVE_BOUNCE=1` stops the process instead). The function is named once on standard error; one that takes floating point or a structure by value gets garbage there. The same commit found that a fatal report from HandBrake had gone into HandBrake's own pipe and faulted again while being written, so a fatal report now goes to a descriptor duplicated before the guest ran, a second fault while reporting ends the process, and ocerz's other diagnostics go through a `FILE` of their own on descriptor 2 (`include/ocerz/types.h` redefines `stderr` for ocerz's own sources).
- **Structures of callbacks beyond CoreFoundation's.** CFReadStream and CFWriteStream clients, `CTRunDelegateCreate`, `CGDataConsumerCreate`, `CGPatternCreate` (whose structure arrives in `rcx` behind two structures System V passes in memory), CFMachPort, CFMessagePort, CFSocket and CFFileDescriptor contexts, VideoToolbox's decompression callback record and CoreMedia's block sources all copy a structure of function pointers when they are created. Each has a `shape` or a handler that binds a copy by offset; CoreMedia's source is under `pack(4)` on both architectures with its function pointers at offsets 4 and 12 (`56edc184`, `045f27e2`, `e1395215`, 2026-10-05). A `struct` record may now name a stack slot, because `CFSocketCreate`'s context is its seventh argument, which System V passes on the stack; the bridge puts its converted copy's address in the guest's slot. Where the header has a versioned structure, a shape of version *N* is checked against the structure named with *N* appended.
- **Audio threads.** `AudioDeviceCreateIOProcID` and its block form were stubs because their out-parameter is typed as a function pointer, although the guest only hands it back to `AudioDeviceStart` and its siblings. It is now taken as a plain pointer. The ID is a small number, `0xa` on the machine it was found on, and the address test had called it guest code (anything host dyld does not know is guest code) and handed `AudioDeviceStart` a trampoline, which CoreAudio refused; the first 64 KB now counts as nobody's code (`d0ade5bb`). `AudioUnitSetProperty` converts the render and input callbacks that CoreAudio's I/O thread calls (`22221270`).
- **A `fork` inside a callback.** It had been refused by name whenever a translated block's frame lay beneath the callback on the host stack, on the grounds that the child could not return into translations it did not inherit. It can: the child forgets the JIT without unmapping its code (`ocerz_jit_forget`)

and translates into a fresh arena, so the code that frame returns into is still there, unchanged, and its exits come back to the child's own JIT (`f2e350ab` , 2026-10-13). Qt starts every `QProcess` that way from an AppKit callback, so ClipGrab stopped as soon as it ran its downloader.
`OCERZ_FORK_CALLBACK_REFUSE=1` refuses the fork again. See Chapter 9 for the fork child's side of this.

Threads the guest never created

Native frameworks call back on threads of their own: libdispatch runs work on its workers, an audio device has a render thread. Such a thread has no x86 registers and no guest stack, and borrowing another thread's cpu would overwrite a guest thread that is running. So the first time such a thread calls a guest function, `ocerz_thread_attach` gives it a guest personality of its own, a cpu, a guest stack and a guest thread block behind `gs` , registered like any guest thread and reused on every later call, and torn down when the host thread exits. A second thread running guest code is a second observer of guest memory, so attaching one also retires plain memory mode, exactly as starting a guest thread does (Chapter 7).

That machinery makes guest threads free. `pthread_create` 's start routine is a callback held past the call and run on a thread the guest never created, so the native `pthread_create` starts a host thread, which is given a guest personality on its way into the start routine. Mutexes and condition variables are forwarded unchanged, which is sound only because `pthread_mutex_t` , `pthread_cond_t` and the rest have the same size, alignment and initializer values on x86-64 and arm64, checked by compiling both rather than assumed. Thread-specific data keys are ocerz's own, numbered 256 to 767 as the real library's dynamic keys are, with values in the guest thread block that `gs` points into, because Chromium's code reads them with inline `gs` -relative loads, about eighty of them in CEF. `errno` is the same kind of data: `___error` answers a slot of the guest thread block, copied to and from the host's `errno` at the edges of every native call, because between two guest instructions ocerz's own code runs on the same host thread and leaves its own errors behind.

Stopping a thread

`thread_suspend` , `thread_resume` and `thread_get_state` are bridged, because garbage collectors use them: JavaScriptCore's allocator and Chromium's samplers stop every other thread and read its registers. The kernel call is the real one, since a cooperative stop leaves the suspend count where the caller can see it. What makes it safe is refusing to leave a thread stopped where that would deadlock the caller: holding the JIT lock or the map lock, which a per-thread critical depth records, or anywhere inside the host's own allocator, which is read off its stack, since the suspender's next `malloc` would wait for a lock the stopped thread holds. Such a target is resumed and tried again. `thread_get_state` answers the guest's x86 registers, reconstructed from the arm64 registers the stopped thread holds them in.

Objective-C

In native mode there is no x86 Objective-C runtime at all. The guest's classes are the host's: `_OBJC_CLASS_$_NSString` binds to the native class object, and any object a native method returns is an address the guest can hold. What the guest cannot do on its own is send a message, because `objc_msgSend` jumps into whatever implementation the selector finds, and that implementation's arguments are in x86 registers. So every send is a crossing whose signature is not known until it arrives.

Selectors

A selector is compared by address. The guest's `@selector(length)` is a word in its `__objc_selrefs` pointing at the string inside its own image, while the native runtime's selector for the same name is a different address. So after an image's fixups and before any guest code runs, every selector reference is rewritten to `sel_registerName` of its string, which makes every selector the guest holds the native one.

A send

x86-64's `objc_msgSend` takes the receiver in `rdi` and the selector in `rsi`, and the method's own arguments after them in the usual places. The signature comes from the native runtime: `object_getClass` of the receiver (which understands tagged pointers, because it made them), then `class_getInstanceMethod` of that class and selector, which also runs a lazy method resolver, and `method_getTypeEncoding`. A class receiver's class is its metaclass, so class methods are found the same way. The call then goes to the host's `objc_msgSend` under that signature: native `objc_msgSend` leaves every argument register untouched and jumps to the implementation, which is what makes one engine crossing correct for every method. The encoding is turned into the notation once per class and selector and parsed once per distinct notation; most of `NSString` is `p(pp)`, `L(pp)` or `p(ppp)`, so parsed signatures are shared heavily.

The encoding converts class by class: `q` to `l`, `Q` to `L`, `l` and `L` (32-bit in an encoding) to `i` and `u`, any pointer to `p`, a structure to braces. `B` becomes `b`, because arm64's `BOOL` is `bool` while an x86 guest's is a `signed char`; they agree on every canonical value. A union, bitfield, `long double`, complex number or 128-bit integer stops the send with the class, selector, encoding and reason.

The special cases each have a rule. A structure result larger than sixteen bytes is sent by an x86 compiler through `objc_msgSend_stret`, with the hidden result pointer first; Apple's arm64 has no `_stret` form and returns through `x8`, so a `_stret` send is an ordinary send under the same signature, checked for agreement. `objc_msgSendSuper` and `objc_msgSendSuper2` pass their `objc_super` structure through, since it is guest memory the native runtime can read. A message to nil zeroes `rax`, `rdx`, `xmm0` and `xmm1`. A selector with no method is forwarded natively: the receiver is asked `forwardingTargetForSelector:` (up to eight targets deep) and otherwise `methodSignatureForSelector:`, never cached, since a proxy's answer can depend on the instance. Foundation's variadic methods, `stringWithFormat:`, `arrayWithObjects:` and the rest, are listed by selector and each checked against the SDK's declaration, since no encoding says a method is variadic.

Exceptions

The first edition said that an Objective-C exception cannot unwind through guest frames, because the unwinder walks the host stack while the guest's frames are x86 frames on the guest's own stack, and that all ocerz could do was print the exception's name and reason and the crossing it escaped from. Until `46349987` (2026-10-05) `objc_exception_throw`, `objc_begin_catch`, `objc_end_catch`, `objc_terminate` and `__objc_personality_v0` were stubs, so native mode stopped a program at its first `@throw`; the census counted 553 binaries importing the personality routine. It is now handled in two halves.

Guest exceptions are built on the guest's own libc++abi from `runtime/guest` (`make guest-cxx`), the way Apple's open-source libobjc builds them on the C++ runtime. Each entry point is a special that jumps to the matching `__cxa_` call or to `__gxx_personality_v0`, so no host frame sits under the guest's unwinding. Binding any of them, or libSystem's `_Unwind_` calls, loads that guest library at bind time. The C++ runtime matches a `@catch` through the thrown type's `type_info` vtable, so `objc_ehtype_vtable`, `_OBJC_EHTYPE_id` and the `_OBJC_EHTYPE_$_` exports of CoreFoundation and CloudKit are `var` records of

guest memory whose entries lead to ocerz trampolines; `can_catch` walks the thrown object's native class chain. The first throw installs a guest terminate handler, which hands an uncaught Objective-C exception to the uncaught-exception handler and aborts, as the native runtime does. These tables hold guest addresses, which an image must not contain while it is being built, so a `var` filler is now either a byte filler, run when the image is built, or a late filler, whose slot is zeros until the loader maps the image and tells it the slot's guest address (`76a84e8b`, 2026-10-05; `test_vdylib`'s determinism checks had caught the first form).

Native exceptions are the other half. An exception a native method raises, `[NSException raise:...]` or one of Foundation's range and argument checks, is caught where the send returns to the guest. `src/objcguard.s` (60 lines) is a frame with its own personality which every send uses once the guest C++ runtime is loaded: the native frames' cleanups run on the way to it, the object is retained through the host C++ runtime's catch calls and thrown on into the guest from the send's call site. The guard cost about 1 to 3 ns on a 59 ns send. What still does not work is a guest exception leaving a callback that native code made into the guest, and a native C++ exception reaching the guest; both end as uncaught. Without the guest runtime the Objective-C exception calls stop the program with a named message. `tests/dynamic/native_objc_exceptions.m` compares `@throw`, native raises from Foundation, subclass and `id` clauses, rethrow, `@finally`, `@synchronized`, 2,000 throws and uncaught exits against the arm64 build, under both engines.

Remembering a send

A guest's `objc_msgSend` was correct from the first day and slow from the same one. Every send asked libobjc for the implementation, asked whether that implementation was a guest function bound into the callback bank or guest code, looked the method's signature up, fetched the selector's name and checked it against the selectors that need converting, and labelled the bridge frame twice, before the message was sent. A send to a native object cost 44 to 50 ns, against 2.7 ns under Rosetta (`beed19af`, 2026-10-18).

What depends only on the class, the selector and the implementation libobjc answers for them now lives in a per-thread table, `t_ob_fast`, of 256 slots (`OB_FAST_SLOTS`) indexed by a hash of the class and selector. An entry records whether that implementation is guest code to jump to and, for a method with nothing to convert beyond its registers, its signature and its selector name. A repeated send asks libobjc for the implementation and makes the call. Staleness is handled by two checks. `g_ob_generation` is an atomic counter that every bridged change to a class's methods bumps, and an entry records the generation it was made under; and an entry also records the implementation, so a different one for the same pair (a guest class defined late, a host swizzle) misses. A slot of the callback bank is never rebound, so the guest function behind an implementation stays what it was.

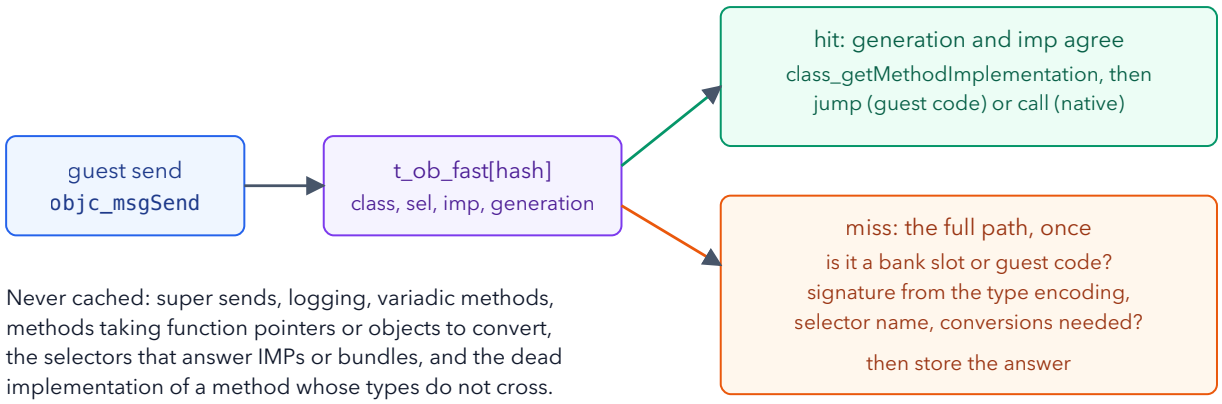


Figure 12.6. The send cache. The slow path of the earlier sections still exists and still decides every first send; the table only remembers its answer.

The measured effect is modest and was reported as such, on 2026-10-18: `[NSString length]` from 49.5 to 40.9 ns, `[NSArray count]` from 45.5 to 39.4 ns and a guest class's method from 37.4 to 35.8 ns. What remains is the cost of the native `objc_msgSend` itself and of entering and leaving the bridge. `OCERZ_NO_OBJC_SEND_CACHE=1` decides every send in full again, which is the way to check whether a cached answer is the cause of a misbehaviour.

Classes of the guest's own

`src/objcclass.c` makes the guest's classes, categories and protocols the native runtime's own, after the image's fixups and selector rewrite and before any guest code runs. `objc_readClassPair` realizes a compiler-emitted class pair in writable memory without moving it, so a guest class keeps its address and no reference to it is rewritten; the runtime slides the instance variables past the native superclass's arm64 size and writes the offsets the guest's code reads. An `NSView` subclass whose ivar offset held 152 had it slid to 536, arm64 `NSView`'s size, and the view's `drawRect:` was called. Class and method structures are laid out identically on x86-64 and arm64, both being LP64, so the runtime reads the guest's own.

Three words of each class and metaclass are written first (the cache word, the word after it, and the data pointer, which is pointed at a host-heap copy of the read-only class data). Every method list is copied into an absolute list whose implementations are callback slots bound to the guest's x86 methods under the notation of their type encodings, so native code calling a guest `-hash` or `-drawRect:` lands in guest code. A method whose types cannot cross is installed as a function that names it and stops if it is ever called, so one odd method does not keep its class out. A class whose superclass the host lacks is refused by name, as the native runtime itself leaves such a class unrealized. Swift classes and root classes were also refused in the first edition; since 06d1b4da (2026-10-04) both are accepted, because the Swift runtime's own root class, `Swift._SwiftObject`, is one (see Swift below). Categories are applied with `class_replaceMethod` in list order, protocols are matched to native ones by name or built and registered, and `+load` methods are queued per image and run before `main`, dependencies first.

What older Objective-C programs needed

Programs from the Mac OS X 10.5 to 10.8 years use parts of the runtime that newer ones have left, and each was found by one application stopping. They are collected here because they are all small and all follow the same rule, to cross what can be crossed correctly and to name the rest.

- **Methods that take a `va_list`.** Mousecape crashed inside `-[NSString initWithFormat:arguments:]` at address `0x3000000008`: the arm64 Foundation read the x86-64 `va_list`, a structure of `gp_offset`, `fp_offset` and two pointers, as the arm64 `va_list`, a plain pointer to the next argument. The variadic table has a third kind, `OCERZ_OBJC_VA_LIST`, naming the format argument and the `va_list` argument; the send gathers the arguments from the guest's list with the classifier the variadic sends use and replaces the register with the address of the gathered slots, which is an arm64 `va_list` (`5250a363`, 2026-10-05). It covers `initWithFormat:arguments:`, the locale and validated-format forms, `raise:format:arguments:` and the predicate and expression forms. A method is only treated this way when every argument up to the `va_list` is in an integer register.
- **Variadic methods ending in a nil, with pairs.** `NSGradient`'s `initWithColorsAndLocations:` takes color and location pairs ending in nil: a new kind, `OCERZ_OBJC_VA_OBJECT_DOUBLE` (`334a5e1f`).
- **Positional format arguments.** Sparkle's localized strings use `%2$@`, `%1$d` and `*3$`. `ocerz_objc_format_classes` orders the classes by position, and refuses a format that mixes the two forms, leaves a position out or converts one position as two types. Plain C `printf` still refuses positional formats.
- **Garbage-collector write barriers.** Sequel Pro's FeedbackReporter is built with `-fobjc-gc` and calls `objc_assign_*` and `objc_read_weak` for every object store. arm64's libobjc still exports them, so the overrides give them `fn` records.
- **Where a class came from.** Sequel Pro loads its nibs through `[NSBundle bundleForClass:]`, which asks `class_getImageName`. A guest class is realized in place, so a hook, `objc_setHook_getImageName`, answers with the real path of the image ocerz mapped it from (`ocerz_dyld_path_containing`). The install name was not enough, since it is `@executable_path/...` in a framework. D3DMetal's use of `+[NSBundle bundleWithIdentifier:]` needed the same idea in the other direction: the host knows only bundles host dyld loaded, so when it finds none, the frameworks ocerz's loader mapped are asked through their `Info.plist` (`0aae84f0`).
- **Instance variables by symbol.** VLC reads `NSWindow`'s `_styleMask` through `_OBJC_IVAR_$_NSWindow._styleMask`, which the databases omit because the symbol belongs to no header. The import now binds to a 64-bit cell holding the native runtime's offset (`OCERZ_NO_NATIVE_IVAR=1` refuses it). Mousecape's import of `_objc_empty_vtable`, which older toolchains put in the word after a class's cache and only the x86-64 libobjc exports, is a variable of eight zero bytes from a new `zero` filler (`e3d7b974`).
- **A method that cannot cross, called by the guest itself.** VLC initializes its menu items through a method that takes a union by value. Native code still cannot call such a method, but a guest send of it jumps to the guest function the buried method's record keeps (`ocerz_objcbridge_dead_guest_imp`), so the program that defines the method and calls it works.
- **Methods made from blocks.** `imp_implementationWithBlock` was a stub that 22 of the census's third-party binaries import. Its implementation is now guest code, as libobjc's own trampolines are: a stub in a page of guest memory that moves `self` over `_cmd`, loads the block into the first argument and jumps to the block's invoke, one register further on for a block that returns a structure in memory. Being guest code, it crosses `class_addMethod` and native callers like any function the guest wrote; the

block is copied first, guest-side for a guest block. `imp_getBlock` and `imp_removeBlock` answer for those stubs and ask the host about any other implementation (`3dcf68d1`, 2026-10-05). A stub is never rewritten, since a translation of it may exist.

- **Runtime-created classes.** `objc_allocateClassPair`, `class_addIvar`, `class_addMethod` and `objc_registerClassPair` work, added methods get callback trampolines from their type encodings, and adding an override or allocating a class invalidates cached signatures so an old one is not reused after a method changes or a disposed class's address is recycled. `docs/native-mode.md` describes this with the `native_objc_runtime` fixture; the generation counter of the send cache above is the same mechanism seen from the other side.
- **Category properties** no longer rewrite existing declarations (`c6cdd9cc`), and an `@rpath` candidate counts when ocerz can supply it (`48b4fc08`).

Blocks

A block is an object whose third word is the function that runs it, called with the block itself as its first argument, and whose descriptor holds its size, copy and dispose helpers and signature. The layout, flags and signature strings are the same on both architectures, and in native mode the guest's block classes bind to the native ones, so a guest block is a well-formed native block in every word except its functions, which are x86 code. A block therefore crosses as a pointer that is converted, class `k`, with the block's declared signature in braces: `dispatch_async` is `v(pk{v()})`.

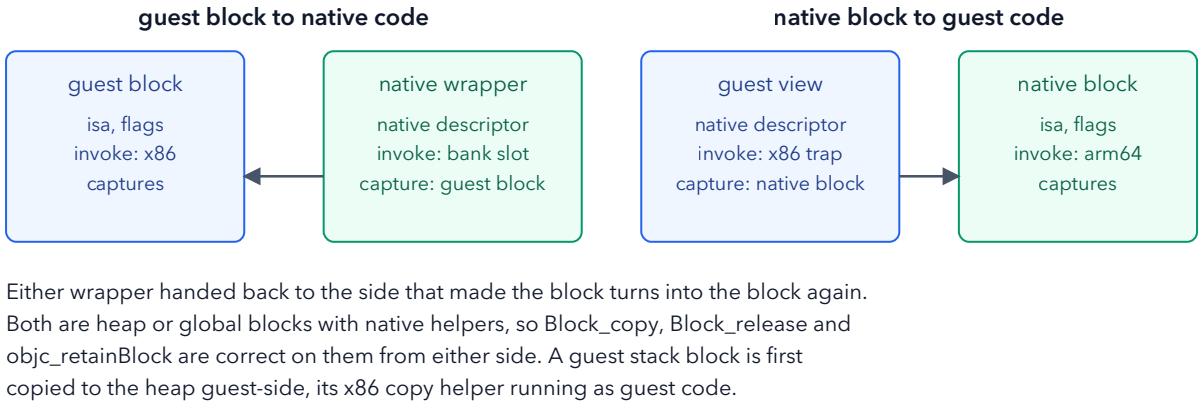


Figure 12.7. The two block wrappers. Each holds only a reference to what it wraps.

A guest block handed to native code becomes a **native wrapper**: a block whose invoke is a bank slot bound to the guest block's own function under the guest block's own signature, whose first argument the slot turns back into the guest block, and whose dispose helper releases the guest block. A global guest block gets one global wrapper for the life of the process; a heap block gets the same wrapper for as long as that wrapper lives. A native block handed to guest code, a completion handler say, becomes a **guest view**: a heap block whose invoke is a twelve-byte x86 trampoline in a guest page, so guest code calls it like any block, and the trampoline calls the native block under its own signature.

The guest's block runtime is libclosure's own except where libclosure would run a guest helper as arm64 code: `_Block_copy`, `objc_retainBlock` and `_Block_object_assign` copy a guest stack block, or a `__block` variable still on the stack, guest-side, and give the heap copy helpers that are callback slots, so the native `_Block_release` that finally frees it runs the guest's dispose helper as guest code. An `id` argu-

ment that holds a guest block where the receiver's method signature says `@?` crosses as a block, which is what an `NSXPCConnection` proxy's methods look like, and a guest protocol carries its extended method types into the native runtime, which `NSXPCInterface` needs to describe a reply block. Stickies, which used to stop at its first `dispatch_once`, runs because of this.

Swift

The first edition listed Swift among the things native mode could not run. It runs now, within limits, and the way it runs is a design decision of its own. A Swift program carries no runtime. It links `/usr/lib/swift/libswiftCore.dylib`, which on a current macOS is arm64 code inside the shared cache, and Swift's calling convention, metadata and generics give no signature a crossing could convert. So native mode runs the swift.org runtime as **guest code**: `make guest-swift` downloads the 6.4.0 toolchain package, checks its SHA-256 and its Developer ID signature, and copies the x86_64 slices of the thirteen runtime libraries into `runtime/guest/usr/lib/swift`. The guest-library lookup (`OCERZ_GUEST_ROOT`, or `runtime/guest` beside the binary) then loads them in place of the native databases. Every Swift call stays x86 code, and only the runtime's own calls into libSystem and libobjc cross, as a C program's do (`06d1b4da`, 2026-10-04).

Four things in the runtime's use of libobjc needed native mode's help:

- **Swift classes are Objective-C classes.** A guest image's Swift classes are defined in the native runtime like its Objective-C ones, with the Swift bits kept in each class's data word. A class whose metadata the Swift runtime has to complete first waits until the runtime registers it through `_objc_realizeClassFromSwift`; classes built while running arrive through `objc_readClassPair`; and a generic class the compiler specialized ahead of time, which no class list names, is defined the first time it reaches the native runtime.
- **Hooks.** The runtime installs `objc_setHook_getClass`, `objc_setHook_getImageName` and `objc_setHook_lazyClassNamer` and asks through `objc_addLoadImageFunc` to see every image. Each hook runs behind a native wrapper that calls the guest's function and falls back to the hook it displaced, and the image function is called for every guest image, including each one a `dlopen` loads later.
- **Reserved thread-specific keys.** The runtime keeps its exclusivity-checking state in one of Darwin's reserved keys, which it uses by number without creating it. Native mode backs keys 10 to 255 with the guest thread block, as it does the keys `pthread_key_create` makes.
- **Releasing an object.** Darwin's Swift runtime releases an `AnyObject` value with a plain `objc_release`, and every element of an array of class instances is stored as one; libobjc hands a Swift object on to `swift_release`. The native libobjc would hand it to the native Swift runtime, so the guest's `objc_release` of a guest Swift object goes to the guest's own `swift_release` instead. When native code drops the last reference itself, as an autorelease pool the native runtime drains does, the native `swift_release` calls the class's destroy function, which is x86 code. The jump into guest memory faults, and ocerz recognises the destroy function by the object Swift passes in a context register, runs it as guest code, and returns to the native caller. Freeing a million objects through arrays takes 130 ms by this path, against 5.5 s when each went through the fault alone and 52 ms under Rosetta (`06d1b4da`).

Swift Concurrency (`f2ab14b7`, the same day) needed two further things. An asynchronous `main` ends in `dispatch_main`, which in libdispatch ends the main thread and leaves the main queue to the thread pool; in ocerz the main thread is the process's own, so `dispatch_main` is a special that runs the native `CFRunLoopRun` there for good, draining the main queue the way an application's main thread does. And

every `Task.sleep` waited about forty times the machine's uptime. The x86 Concurrency runtime builds its deadlines in nanoseconds and hands them to libdispatch as a `dispatch_time_t`, because on an Intel Mac a tick is a nanosecond; on Apple silicon a tick is 125/3 of one, and the native libdispatch read the guest's numbers as 24 MHz ticks. Rosetta and cache mode show x86 code a 1/1 timebase. The notation class `T`, which the generator gives every `dispatch_time_t` by its typedef, converts the clock value between the guest's nanoseconds and the host's ticks in both directions, keeps the clock bit, and passes `NOW`, `FOREVER` and wall-clock times through unchanged. `mach_absolute_time` and `mach_timebase_info` stay the host's, so tick values native frameworks hand back, a CoreAudio timestamp or a display link's, still compare with them. The Concurrency runtime also calls `voucher_adopt`, `voucher_copy` and the `os_unfair_recursive_lock` pair, which no SDK header declares; they cross under the signatures Apple's open sources give.

`make native-swift` runs `tests/run_native_swift_tests.sh`. It compiles each fixture for x86_64 and arm64 from the same source and module name, and the x86_64 build under `-native` must print exactly what the arm64 build prints, under the JIT and under the interpreter. The fixtures cover classes, generics, protocols with associated types, enums, errors, closures, `Mirror`, key paths, Unicode strings, a `Regex` built at run time and a weak reference that must read nil once its object is gone; the release paths above, requiring the destroy crossing for three objects whose last release comes from a drained pool; actors, task groups, `async let`, detached tasks, the main actor, cancellation and `AsyncStream`; and a C fixture that times one call of each libdispatch function taking or returning a `dispatch_time_t`, so a failure points at the boundary rather than at the Swift runtime.

What does not run is the Swift API of any system framework. `import Foundation` and `import Dispatch` bring Swift overlays whose exports are Swift functions, `String._bridgeToObjectiveC` or a `DispatchQueue` method, and those have no crossing: the program stops with exit 72 or 71 naming the first one. SwiftUI, Combine, CryptoKit and every other framework with a Swift interface are the same, and they have no headers to generate a database from either. Many newer applications are written that way.

CoreFoundation and callback structures

A `CFTypeRef` crosses unchanged in both directions, and that is a consequence of the address map, not a shortcut: in the identity map the native object a function returns is an address the guest can hold, compare and hand back, and the same object fetched twice is the same pointer. No handle table stands between the two.

What needs converting is a structure of function pointers passed by address: an array's or dictionary's callbacks, a timer's, observer's or source's context. CoreFoundation copies each of them when it creates the object, so the bridge copies the guest's structure first, converts every function-pointer word, and passes the copy's address for the length of the call. Which argument holds such a structure is a `struct` record, and what each word of each version is, a `shape` record; a version no shape describes stops by name, because guessing a context's layout would put a guest pointer where native code will call it. CoreGraphics' structures start with a 32-bit version and four bytes of padding that a structure filled field by field leaves as stack garbage, so a word that matches no version whole is matched again on its low half.

Everything else a process does

Signals. A guest signal handler is x86 code, so a native `sigaction` cannot be handed one. `sigaction`, `signal`, `sigprocmask`, `pthread_sigmask`, `sigaltstack`, `raise`, `kill` and `pthread_kill` are answered against the same handler table, masks and alternate stacks the system calls change in cache mode (Chapter 9), with a trampoline in guest memory in place of an x86 libc's `_sigtramp`. A handler runs on the state after the call that raised it, so `raise` returns with its handler already run. A signal from outside is delivered when the receiving thread's next crossing returns, the way cache mode delivers one at the next system call; checking costs a thread-local load, two loads and a branch per crossing. A thread that makes no crossing at all and loops in translated code is reached by the run loops instead: since `a7fd0f3f` (2026-10-14) a pending signal is delivered between two steps and the loop's blocks are stopped by unchaining their edges (Chapters 7 and 9).

Files, memory, jumps and processes (`src/sysbridge.c`). `open`, `fcntl`, `ioctl`, `sem_open` and the like are declared variadic but take one optional argument whose presence the named arguments decide; each handler reads it where x86-64 left it, only when the call uses it, and calls the host function through its real variadic prototype. Every structure those calls point at, `struct flock`, `struct termios`, `struct winsize` and the rest, was checked to have the same layout on both architectures. `mmap`, `munmap`, `mprotect` and the Mach VM calls run the same code the cache-mode system calls run, so guest memory comes out of the guest arena in 4 KB pages, and code written into a mapping and made executable is translated afresh. `setjmp` and `longjmp` save and restore the guest's registers, signal mask and MXCSR in Apple's x86-64 `jmp_buf` layout; a jump out of a native callback to a `setjmp` taken before it would leave native frames behind and is refused by name. `fork`, the `exec` family, `posix_spawn`, `system` and `popen` go through cache mode's own fork, exec and spawn, so every program a guest starts runs under ocerz in native mode, except a system tool with an arm64 slice that is not a shell or launcher, which runs as itself (`OCERZ_NO_NATIVE_CHILDREN=1` changes that).

Loading code at run time. Host dyld has never heard of an image ocerz mapped, and the host's `dlopen` would load arm64 code, so `dlopen`, `dlsym`, `dladdr`, `dlerror` and the image-list calls are answered by ocerz's loader. A load binds its whole closure before anything in it runs, and one that cannot bind fails with dyld's own wording, "Symbol not found" or "Library not loaded", leaving nothing mapped: a class handed to the native runtime cannot be taken back, so the Objective-C definitions wait until the bind has succeeded. `dlopen(NULL)` answers `RTLD_DEFAULT`, the value an arm64 program on the host printed.

The guest's identity. Native frameworks ask the process who it is, and the process is ocerz. So before any framework is opened, the host's `NXArgc`, `NXArgv` and program name become the guest's, and the first framework opened opens CoreFoundation with `CFProcessPath` naming the guest's executable, which CoreFoundation honours and keeps; the variable is removed again the moment that initializer returns, so no child inherits it. `[NSBundle mainBundle]` is then the application's bundle, and `NSApplicationMain` crosses as an ordinary call: it reads the guest's `Info.plist`, finds the principal class by name, loads the main storyboard instantiating the guest's own classes, and runs the application with every delegate method a call back into guest code. Image Capture's x86-64 slice runs this way and quits with status 0 when asked; Stickies opens a note and stays up.

Malloc zones. Chromium saves the default zone's function pointers and writes its own over them. Written into the host's zone, those x86 pointers would be the next thing the host's own `calloc` jumps to. So the guest never holds a host zone: every call that answers one answers a view of it in a page of the guest's own, whose pointers are thunks the guest can call. What the guest writes into a view changes only its own calls.

When a crossing fails

A crossing is the one place a thread the guest drives runs native code, so for its duration the thread carries a **bridge frame**: a thread-local record naming the library, the export, the signature and the host address, raised immediately before the call and lowered immediately after, by one small function with no other way out. A fault during a crossing is then reported as such, naming the call and saying whether the faulting address was in guest space (the guest passed a bad pointer) or outside it (ocerz marshalled the call wrong). The process stops there, because a native frame can be neither resumed nor unwound:

```
ocerz: BRIDGE-FAULT[35366] SIGBUS inside a bridged call, not in guest code
ocerz:   call=/usr/lib/libSystem.B.dylib:_strlen sig='L(p)' host_fn=0x18980eac0 depth=1
ocerz:   cause: the fault address is in guest space, so the guest passed a bad pointer to
_strlen
```

Two exit codes keep the other failures apart. An import that never bound stops the process with **71** before the guest runs, after a report naming every missing (library, symbol) pair at once. An export that bound but has no crossing names itself and stops with **72** when called. In both cases the process stops rather than running with a guessed answer; that is native mode's central rule.

What it costs

Guest loop, ns per iteration (Apple M5)	Trap path	Default	arm64 build
`getpid`	40.5	17.0	0.7
`strlen` of a 12-byte string	42.0	**2.0**	0.9
`memcpy` of 48 bytes	45.6	**3.0**	2.7
`-[NSString length]`	51.3	28.3	3.6
`qsort` comparator, per call back	426	430	3.1

The trap path is `OCERZ_NO_BRIDGE_FASTCALL=1`. With `OCERZ_NO_LEAF_INPLACE=1` the `strlen` and `memcpy` rows read 17.3 and 20.8, which is what an ordinary crossing costs. Against cache mode the cost shows only in code that crosses in a tight loop: on the benchmark suite's `str` kernel native mode measures 1.03x of cache mode, `memcpy` 0.99x, and kernels that make no calls are at parity. One kernel is slower for a different reason: `fpvec` runs at 1.05x of Rosetta in native mode, against 0.80x in cache mode, because native mode keeps the translated NaN checks rather than set the processor's alternate floating-point mode, since the host's own code runs on guest threads and two FPCR writes per crossing would cost about as much as the crossing (Chapter 7).

Two costs have moved since that table was taken (2026-09-21), both on 2026-10-18 and both reported by the commits that moved them as a before and an after of the same loop, which is why they are given as pairs and not as new rows of the table above:

Loop	Before	After	Commit
`-[NSString length]`, ns per send	49.5	**40.9**	`beed19af`
`-[NSArray count]`, ns per send	45.5	**39.4**	`beed19af`
a guest class's method, ns per send	37.4	**35.8**	`beed19af`
`qsort` of 200,000 ints, ns per guest comparison	536	**100 to 112**	`ad51f4b3`

The sends are still several times slower than the 2.7 ns a send costs under Rosetta, and a callback is still more than twenty times slower than Rosetta's 4.2 ns comparator. Native mode's speed comes from the library calls that never leave arm64, such as a decoder, a compositor or a GPU driver, not from the crossings themselves. A program that sends millions of messages a second to small objects is the case where cache mode can still win.

Steam in native mode

On 2026-09-21, on macOS 27, the x86-64 Steam client came up in native mode: the login window, then the 1280 by 800 Steam window with its menus and store view, the GPU process running hardware ANGLE over the host's own OpenGL, and two renderers, stable for minutes. From the web helper starting to the main window took 23 s in native mode and 34 s in cache mode on the same machine; the arm64 Steam takes 2 to 4 s. Reaching it needed, in the order the failures appeared: x86 thunks for native method implementations handed to the guest, since Chromium swizzles `allocWithZone:`; the guest-owned `errno` and thread-specific data keys described above; `bzero` answering its destination in `rax`; system tools with an arm64 slice running as themselves; the SystemConfiguration exports IOKit still carries; every API database loaded before the sandbox closes, because from then on the kernel may refuse ocerz the read of its own data files; a libGL database; Steam's rewritten `ipcserver` job started in its parent's mode; the `syslog`, `warn` and wide `printf` veneers; the malloc zone views; a page size of 4096 everywhere; the guest's own stack bounds for V8's stack scanner; and legacy CoreGraphics display and CoreServices calls that are exported but gone from the headers.

Older Mac applications

On 2026-10-16, `334a5e1f` made four x86-only applications from the Mac OS X 10.5 to 10.8 years open their windows: Transmission, Sequel Pro, HandBrake and VLC 2.2. The commit message says they open in two seconds under `-native` with no switches. They are a better test of the design than the modern programs before them, because each is built against runtime and library behaviour that newer software has dropped, and each stopped on something newer programs never reach. The causes, in the order the commit gives them, are all described where their mechanism belongs, and the table collects them by application:

Application	What stopped it	What native mode does now
Sequel Pro	garbage-collector write barriers; `bundleForClass:` for its nibs; Sparkle's positional format strings	`fn` records for `objc_assign_*` and `objc_read_weak`; an image-name hook; positional formats
Sequel Pro and Transmission	Sparkle links the 0.9.x libcrypto, which no Mac with Apple silicon has	imports of removed system libraries bind to named stubs
HandBrake	a guest function stored in `stderr`'s `_write`; a crash report that went into its own pipe	`ocerz_abi_bounce`; a report descriptor taken before the guest ran
VLC 2.2	301 plug-ins; `NSMapTable` and `NSHashTable` callback structures passed by value; a union-by-value method; `_styleMask` by ivar symbol; QTKit; GNU libstdc++ in five plug-ins	an image table of 2048 (was 256); specials in `bridge.c`; the buried method's guest function; an offset cell; a stubbed removed library; `runtime/guest-gnu`
(variadic) `NSGradient`	color and location pairs ending in nil	`OCERZ_OBJC_VA_OBJECT_DOUBLE`

Libraries no Mac with Apple silicon has

Some system libraries exist in Rosetta's cache and nowhere else: QTKit, VideoDecodeAcceleration and the 0.9.x libcrypto. They are not on disk, not in the host's cache, not in a guest root, and in no database. Transmission and Sequel Pro link the old libcrypto through Sparkle, and VLC's interface links QTKit for its capture panel alone. Refusing the whole program over a library only a side feature wants was the wrong rule, so their imports now bind by default to stubs answering 0 or nil, each named on standard error as it binds, for the program and for a plug-in it opens (`334a5e1f`). `OCERZ_NO_STUB_REMOVED=1` refuses them again, and `OCERZ_STUB_MISSING=1`, which came from the MacShack work below, passes over any import that does not bind and, since this commit, over a `dlopen` dependency that is nowhere. The distinction between a library that never existed here and one that is merely not described by a database is the point: the first answers with a stub, the second is still refused with `native library without an API database`.

GNU libstdc++

VLC's Matroska, DASH, live555, UPnP and taglib plug-ins import GNU libstdc++ from `/usr/lib/libstdc++.6.dylib`. Rosetta's cache still has it; on Apple silicon only an arm64 build exists, whose objects, vtables and unwinder x86 code cannot share. `tools/fetch_guest_gnu_cxx.sh` fetches the x86_64 libstdc++ and libgcc_s of GCC 15.2 and GNU libiconv 1.19 from MacPorts, pinned by SHA-256, into `runtime/guest-gnu`, which the loader searches after `runtime/guest` (`OCERZ_GUEST_GNU_ROOT` names another directory, and an empty value disables it). GCC 15's libstdc++ still exports the old `std::string` and iostream ABI those programs were built against. The directory is not tracked and no release carries it; without it, a program importing libstdc++ stops with exit 71 and a line naming the script. The commit says plainly that this path has no test of its own, since no compiler available there targets it, so VLC's plug-ins were the check.

Intel games

MacShack (github.com/Jfishin/MacShack) runs Mac games on iPhone and iPad, and Intel ones under AArchX's native mode. Jfishin found and fixed a set of problems bringing up Unity, Feral, MonoKickstart and LOVE games there, and `275843d4` (2026-10-09, co-authored) brought in the part that native mode on macOS needs. The i386 personality and its JIT stayed out because the x87 work on main already covered what they would add; so did the pieces that only make sense on iOS. What came in:

- **A non-PIE main below 12 GB** with the default 4 GB page zero is slid into the arena like a PIE, and `scan_rebase` then rewrites the absolute pointers in its `__DATA` sections, for which `-no_pie` leaves no rebase information. Under the low shadow, such a program's Objective-C classes had crashed. Wine's loader, with its 4 KB page zero, keeps the low shadow, and so does any program under `OCERZ_NOPIE_LOW=1`.
- **Binding order.** Lazy flat-namespace binds wait until all of the main executable's dependencies are loaded, as dyld resolves them on first call (LOVE's framework imports Lua flat); a main-executable ordinal binds against the main executable first; weak references from the guest C++ runtime resolve from the main executable and the other guest-root images only. Before that last rule, libc++ had bound to UnityPlayer's `operator new` and libsteam_api's `free` then took its strings.
- **`dlsym` and `dlopen`.** `dlsym` on a system library's handle also searches the libraries it depends on, `RTLD_DEFAULT` loads a framework the program never linked on demand, a bare leaf name is tried in the caller's and the main executable's `LC_RPATH`s before the working directory, and a framework path with no file behind it is matched against the databases with the `Versions/<x>` segment ignored.
- **Bridge.** `__divdc3` and `__divsc3`, which the arm64 libSystem lacks, are defined in `src/complexdiv.c` and bridged beside the multiplications; `CFUUIDGetConstantUUIDWithBytes` passes its stack bytes as `UInt8`, because arm64 packs them a byte apiece and bytes 7 to 15 had come out wrong; `pthread_exit` runs the guest's `pthread_cleanup_push` handlers as guest code first; `abort` and `__stack_chk_fail` end the process at once, since a Mono crash handler that aborted inside the collector's stop-the-world had left the other threads suspended; and `IOCreatePlugInInterfaceForService` is refused, as for a service with no plug-in, so a HID pad scan skips the pad (a real pad's plug-in would hand x86 code a table of arm64 functions).
- **Only the arena holds guest code.** A function pointer outside it, such as one of libobjc's `imp_implementationWithBlock` trampolines, is native code; `dladdr` knew nothing of it, so it used to be taken for x86.

Under Wine, and for 32-bit programs

Native mode runs MacNdCheese's Wine in the same way it runs any other x86-64 program, and Wine is a hard guest: it lives below 12 GB under a low shadow window, hands native code host pointers all the time, and uses the Mach and libSystem calls an emulator does not often see. The work of 2026-10-05 and 2026-10-06 is a list of what it took for `wine-unified` to start every service with msync, for Notepad to show its window, for Steam to log on and show its client, and for R.E.P.O. to reach its main menu on DXMT. The mechanisms worth knowing:

- **Low host memory is aliased and pinned** (`2d59b306`). Readable host memory below 12 GB, namely the main image, the first heap, the main stack and the shared cache, is aliased into the shadow at the same offset and pinned: `ocerz_g2h` answers the host address for it, fixed maps over it are refused, and a forked child aliases it again. The free host space left there is reserved once pinning is done, so a later

host allocation cannot land where the guest sees the shadow. A value in the first 64 KB crosses unchanged, because Objective-C's empty-pool token is 1 and the shadow alias of 1 crashed `objc_autoreleasePoolPop`. `__block` variables on a Wine stack forward to a guest address, so the forwarding word is rewritten to its host alias before libclosure follows it.

- **`mach_vm_map` chooses addresses as Wine and Chromium need.** A fixed map in the low window maps its object over the shadow; an anonymous fixed map above it is claimed in ocerz's map as cache mode does, which Chromium's 16 GB PartitionAlloc pools needed; an anywhere map searches from 12 GB up so wineserver's msync pages are one page to guest and host; and an anywhere hint above user space, which only an uninitialised variable passes, is dropped.
- **`i386_set_ldt` and `i386_get_ldt`** exist only in the x86_64 libSystem, so the arm64 SDK the databases are read from has neither. They are specials on the same table that cache mode's machdep calls 5 and 6 keep (`4f200419`, 2026-10-05); `i386_get_ldt` answers how many entries the table holds, as Rosetta does. `wine --version` then printed its version in native mode, and a census of the 94 x86_64 Mach-O files in the wine-unified tree found no other missing symbol.
- **`__ulock_wait` and `__ulock_wait2` are specials** (`22221270`, 2026-10-06). A guest signal that lands after msync last looked at its word and before the wait reaches the kernel was held until the wait ended, so the SIGUSR1 APC that would set a service's reply event never ran. The wait is now marked kickable-if-signalled and the unstick monitor interrupts it once a guest signal is pending. The same commit made `thread_get_state` answer the x86 debug-register flavors (all zero) instead of ending the process, because wineserver asks for them on context requests and so does Chromium's stack sampler; copied `iovec` and `msghdr` for `readv`, `writev`, `sendmsg` and `recvmsg`; and let a guest function pointer cross to a method of a guest-defined class (Wine's own `WineEventQueue`).
- **`IsWindowVisible`** (`1c67c487`, 2026-10-06). Steam's web helper ended whenever its window was dragged: `winemac.drv` tests `!IsWindowVisible(hwnd)` for `SC_MOVE`, and the name bound to a Carbon export no header declares any more, so the database had it as a no-declaration stub. It is now an `fn` record as Apple's Window Manager reference gives it, and called with a null or other non-window value it answers 0 on arm64 and under Rosetta without reading through the pointer.
- **D3DMetal** (`0aae84f0`). MacNdCheese's D3DMetal backend, built on Apple's Game Porting Toolkit, runs under native mode: a D3D11 device, swap chain and presents, and R.E.P.O. to its main menu. The commit says it was worked out black-box, from what ocerz's loader and bridge report and from return values, never from the binaries. What it needed, beyond the database for CoreAnalytics and the bundle lookup above, was `_dyld_get_image_uuid` answered from the image's own `LC_UUID`, two undeclared Metal texture-swizzle functions crossing as scalars, and a new `nothing` special answering 0 for a function whose arguments nothing declares and which only shares the shader cache.

32-bit programs. From 2026-10-10 (`0ca24d64`) native mode also runs i386 Mac programs, through m32, the 32-bit Mac runtime, with Carbon, QuickDraw, AGL and ATSUI among its parts. It shares this chapter's loader, bridge and databases, and since `1ae3140a` (2026-10-18) it is the default for i386 programs. Chapter 13 describes it, including the Steam bridge for 32-bit games; this chapter's mechanisms are the 64-bit half of the same design. Chapter 15 covers Wine itself.

What native mode cannot run yet

Each of these stops a program with a named message, rather than running it wrongly. The first edition's list has shrunk: Swift programs, Objective-C exceptions, `imp_implementationWithBlock` and most of the unbound frameworks have left it.

- **Frameworks without a database.** 177 libraries have one, including libSystem, libobjc, CoreFoundation, Foundation, AppKit, CoreGraphics, Carbon, Security, IOKit, Metal, OpenGL, WebKit, CoreData, JavaScriptCore, SceneKit, SpriteKit, MapKit and the audio and video frameworks. Other system libraries still need one, and unsupported exports inside these databases remain stubs. `dlopen` of a native library without a database answers NULL with "native library without an API database". libffi is left out on purpose (`643406e5`).
- **Swift's framework APIs.** Swift programs, Concurrency included, run on a guest copy of the Swift runtime, but the Swift APIs of the system frameworks, Foundation's and Dispatch's overlays, SwiftUI, Combine and the rest, have no crossing.
- **C++ across the boundary.** The guest runtime (`make guest-cxx`, and `runtime/guest-gnu` for GNU libstdc++) runs guest C++, exceptions between guest frames included, but guest C++ objects are not interchangeable with native arm64 C++ objects, and an exception cannot unwind through a bridge frame or leave a callback native code made into the guest.
- **Variadic calls without a veneer.** The `scanf` family except `scanf`, `fscanf`, `sscanf`, `sscanf_l` and their `v` forms, and `asl_log`, are stubs. Plain C `printf` still refuses positional formats, `long double` and `%n`.
- **Odd argument types.** A union that holds a float or a double, or a bitfield structure, by value is a stub; a `long double` inside a structure, callback or block is refused, and one that crosses as an argument or result is a double, so it loses the x87 format's extra precision.
- **Callbacks whose results are callbacks** are refused, and a `longjmp` out of a callback native code made, to a `setjmp` taken before it, is refused by name.
- **GSS and Kerberos descriptors.** They lay their structures out under `pack(2)` on x86, which moves a `gss_OID_desc`'s pointer, so a call that passes or returns a descriptor is still a stub (`b8932fef`).
- **Bundled XPC services.** A service inside the application's own `Contents/XPCServices` is not reached, most likely because launchd looks it up for the executable that asks, which is ocerz.
- **Blocks that cross as plain objects.** A block crosses as a block only where a type says it is one; as a plain `id` it is passed as the pointer it is, so only the side that made it can call it.
- **`fenv`.** The functions work on the guest's own state, but an exception the guest unmasks does not trap, and `stmxcsr` does not show the flags arithmetic raised, though `fetestexcept` does.

> **KEY IDEA**
>
> Native mode is a translator for the program and a foreign-function interface for everything else. The FFI is driven entirely by data generated from the SDK's own declarations, so supporting another framework is a regeneration, not a rewrite, and every export that cannot be crossed correctly is a named refusal rather than a guess.

CHAPTER 13

32-bit Mac programs (m32)

macOS stopped running i386 Mach-O programs in Catalina, and Rosetta, which translates x86-64 only, never ran them. m32 runs them on Apple silicon: their i386 code goes through the 32-bit engines that serve Wine's WoW64, and every call they make into the system crosses into the Mac's own arm64 frameworks, as native mode's calls do. This chapter follows an i386 program into its 4 GB window, explains how calls cross in both directions between two ABIs whose pointers and longs differ in width, and describes what m32 rebuilds because Apple silicon no longer has it. Nearly all of it was written between 10 and 18 October 2026.

18,387
LINES OF M32 IN 37 FILES, WITHOUT DLMALLOC

33
LIBRARIES WITH AN I386 DATABASE

79
I386 TEST PROGRAMS

61 fps
BATMAN, M4 MAX, 2026-10-10

Where m32 comes from

Until 10 October 2026, `main()` refused any executable with no x86-64 slice. m32 arrived that day in `0ca24d64`, authored by Jfishin. It comes from MacShack, a project that runs Mac games on iPhone and iPad, where it was written to bring Feral's Batman: Arkham Asylum (2014, i386 only: Unreal Engine 3, Objective-C 1, tcmalloc, Bink, FMOD Ex, OpenGL) to Apple silicon. On landing, the game played its New Game opening at about 31 frames a second on an M4 Max. The i386 JIT was main's own; none of MacShack's i386 JIT work came with it. The rest arrived in four stretches, each driven by the programs that stopped next:

Dates	Author	What arrived	Driven by
2026-10-10/11	Jfishin	m32 (pull request #8); the 32-bit JIT speed work (#9)	Batman: Arkham Asylum
2026-10-11/13	Jfishin	the Steam bridge and `steamgen`; libSystem, CoreFoundation, OpenGL and font fixes	Modern Warfare 3, Left 4 Dead 2
2026-10-16	mont127	self-loading programs, Carbon, QuickDraw, AGL, ATS, CoreGraphics callbacks	Kega Fusion, VLC 1.1.12, Firefox 3.6
2026-10-17/18	mont127	SDL 1.2 support, Objective-C twins and outlets, packed structures, trap blocks, native mode by default	free SDL games, Adium, Nexuiz

The design fits in a paragraph. The guest runs as i386 code in a flat 4 GB window and never holds a host address. No i386 system library exists on the machine, so each import from one is bound to a trap address, one per export. A trap crosses to the host's arm64 function, converting arguments and result between i386 `cdecl` and Apple's arm64 ABI by a pair of notations generated from the SDK's headers, compiled once for i386 and once for arm64. Calls no notation describes have hand-written *specials*. Host objects reach the guest as 8-byte handle cells, and guest objects the host must see get host twins. m32 shares the 32-bit de-

coder, interpreter and JIT with WoW64 (Chapter 15), the trap window with the dyld API (Chapter 11), and the callback bank and call assembly with native mode (Chapter 12). It adds its own loader, Objective-C 1 runtime, C++ unwinder, QuickDraw, ATSUI and Steam client. At `6e6c64b5` it is 30 C files, one Objective-C file and an unmodified dlmalloc 2.8.6 under `src/` (`git ls-files 'src/m32*'` counts 32), plus six headers: 18,387 lines without dlmalloc's 6,282, against 9,412 when it landed.

How a program arrives at m32

`main()` turns an application bundle into its `CFBundleExecutable`, handles launch scripts, and then asks whether the file is an i386 program. `ocerz_peek_dynamic` answers −2 for a Mach-O file with no x86-64 slice, and `m32_is_i386` reads the first page, where even a fat file's architecture table must lie. A thin i386 file qualifies, and so does a fat one that lists i386 and not x86-64; a file with both slices takes the x86-64 path. Anything else stops with "has no x86_64 slice" and exit 64. Chapters 2 and 10 trace the same route from the loader's side.

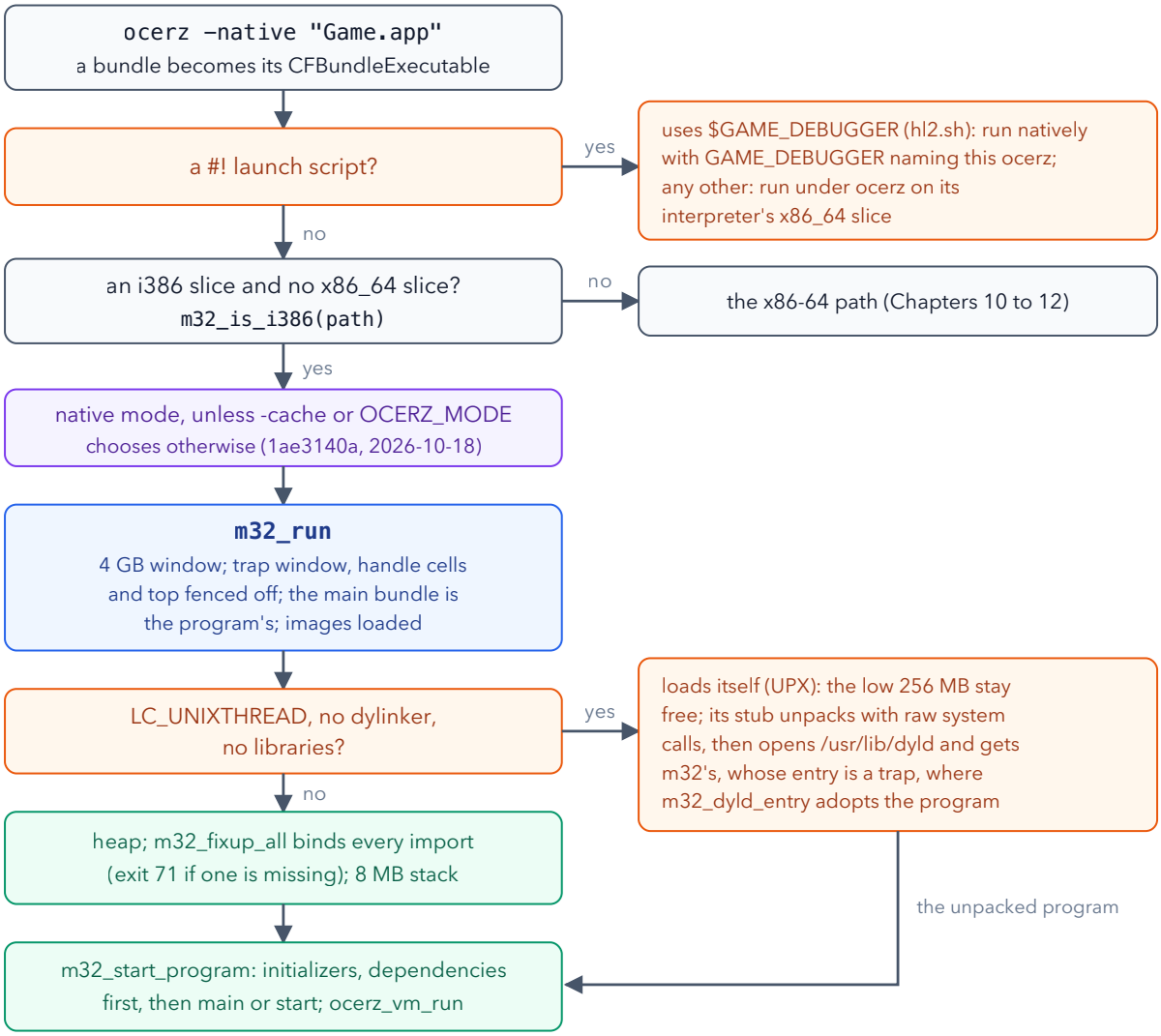


Figure 13.1. How a program arrives at m32. Launch scripts are handled before the file is examined, and an i386 program runs in native mode unless told otherwise.

Launch scripts

Run natively, a launch script ends with the kernel refusing the i386 program it starts. Valve's `hl2.sh`, which Steam runs for a Source game, sets `DYLD_LIBRARY_PATH` and starts the game as `${GAME_DEBUGGER} <game> <args>`. Since `46fade2e` (2026-10-13) a file beginning with `#!` runs natively with `GAME_DEBUGGER` naming this ocerz and the flags before the script; a value already set wins. The scripts expand the variable unquoted, so an ocerz path containing a space is refused. Any other script, such as `Nexuiz.app`'s executable (`exec "$0"-bin "$@"`), runs since `88b7e20c` (2026-10-18) on its interpreter's x86-64 slice under ocerz, so every program the shell starts is ocerz's. Before that, `/bin/sh` handed the game to the kernel and got "Bad CPU type in executable". `execv` and `execve` from i386 code also start the new program under ocerz (`1affbd39`), which is how Modern Warfare 3's launcher starts the game.

m32_run

`m32_run` (`src/m32.c`) reserves the window and fences three ranges in it (next section). It names the program to CoreFoundation as the main bundle, loads the program and its libraries, starts the heap and binds every import. An import with no answer anywhere ends the process with exit 71 ("required imports are unresolved") unless `OCERZ_M32_KEEP_GOING` is set, in which case it returns 0 when called. An LDT descriptor gives selector `0x0f` a flat 32-bit code segment. An 8 MB stack receives the frame the kernel and dyld built: `argc`, `argv`, `envp` and an `apple` vector with `executable_path=`. `gs` points at 768 thread-specific data slots, Darwin's key count. An `LC_MAIN` program is called as `main(argc, argv, envp, apple)`, with `esp + 4` aligned to 16 and a return address of `M32_TRAP(M32_ID_EXIT)`, a trap that exits with `eax`. An `LC_UNIXTHREAD` program starts at its own entry with `argc` at the stack pointer. Initializers run first, dependencies before dependents. `m32_dep_order` has been a depth-first post-order since `aa51c8bb` (2026-10-18); reverse load order, used before, left Adium's classes without their superclasses.

Images

`src/m32_dyld.c` is m32's dyld. The main executable is not position-independent and is linked at `0x1000`, so it maps at its own addresses; dylibs slide. Every bind, lazy ones included, is applied at load, so `dyld_stub_binder` is never reached. Images with dyld info and images with only classic relocations both load. Particular programs added rules:

- Images linked flat look every import up in every image and then the system (`281e193d`). Before that, 664 of Wesnoth's GLib and Pango imports failed.
- `LC_RPATH` entries are directories (`aa51c8bb`), and a bare leaf name passed to `dlopen` searches `DYLD_LIBRARY_PATH` and then `DYLD_FALLBACK_LIBRARY_PATH`, as dyld does (`471ba306`, `88b7e20c`).
- dyld's NSModule calls load and look up on m32's loader (`c42a4d64`; Firefox finds XUL that way).
- A program linked for 10.4 reaches dyld through two words of `__DATA,__dyld` that held dyld's fixed addresses of the time, `0x8fe01000` and `0x8fe01008`. They now point at m32's own entry points (`487c886a`).

Guest libraries macOS no longer has, such as GCC's libstdc++ and LLVM's i386 libc++, come from `runtime/guest32`, which `tools/fetch_guest32_cxx.sh` fills from MacPorts, each file pinned by SHA-256.

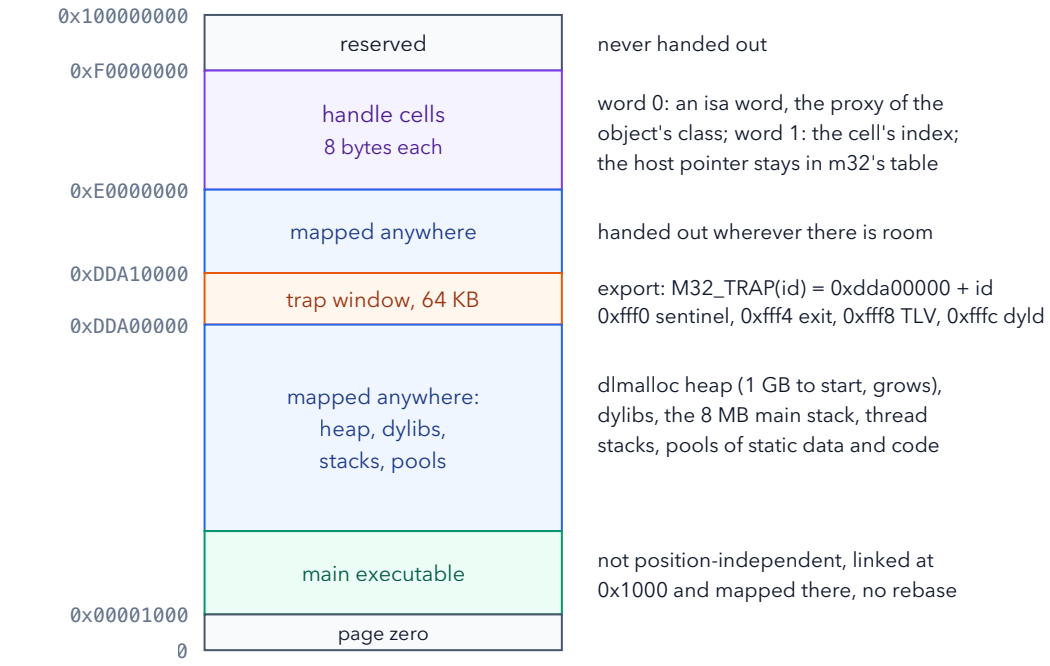
Programs that load themselves

A UPX-packed executable, such as Kega Fusion 3.63's, has one segment, an `LC_UNIXTHREAD` entry, no libraries and no dynamic linker. Its stub unpacks the program with raw system calls, opens `/usr/lib/dyld`, maps it and jumps to its entry with the program's Mach header pushed above `argc`, as the kernel would. Under m32 the stub died at its first `sysenter`, and there is no i386 dyld to open anyway. `0144d669` (2026-10-16) fixed both. `sysenter` now decodes, and in an m32 process the interpreter answers it and `int 0x80` as Darwin system calls. The number is in the low 16 bits of `eax`, the arguments sit above the return address at `ecx` or `esp`, and a failure sets the carry flag with `errno` in `eax`. Only the twelve calls a loader stub makes exist, such as `open`, `read`, `mmap` and `mprotect`; the rest fail with `ENOSYS`.

Opening `dyld` returns a file m32 writes and unlinks at once: an i386 `MH_DYLINKER` with one empty 16 KB segment and a thread state whose `eip` is the trap `M32_ID_DYLD`. The stub maps it and jumps there, and `m32_dyld_entry` takes the program from the Mach header the stub pushed. It rebuilds the program's file from its segments in the window and starts it like any other, sliding and rebasing it if UPX 5 placed it elsewhere. Until then the low 256 MB around the packed segments are kept free for the unpacking. The test `selfload` is a test program packed with UPX 5.2.1.

The 4 GB window

`ocerz_mem_init(0, 4 GB)` reserves the window, and guest address `g` is host address `ocerz_guest_base + g`. Before anything else is placed, `m32_run` fences three ranges: the trap window at `0xdda00000`, mapped with no access; the handle cells from `0xE0000000`; and the top 256 MB, never mapped. Later mappings go through `m32_map`, which gives back any that reaches past the handle cells (`e246ed8a`).



Not to scale. Guest address g is host address guest_base + g; the guest never holds a host address.

Figure 13.2. The window an i386 program lives in. Imports point into the trap window, host objects are handle cells near the top, and everything else is ordinary guest memory.

The pointer rule

Every pointer that crosses obeys one rule (`src/m32_handle.c`). `m32_handle` turns a host pointer into a guest one. NULL becomes 0, a pointer inside the window becomes its offset, and a host object with a guest twin becomes the twin. Anything else gets a handle cell: 8 bytes whose first word is an isa word, filled with the guest proxy of the object's class, and whose second is the cell's index. A guest that reads through a handle, as Objective-C code does when it checks a class, sees a sane isa and nothing more. `m32_host` goes back: a cell to its host pointer, an aliased guest address to its host object, anything else to the window address it names. The range holds 33,554,432 cells, and they are never freed. Aliases run the other way. Each i386 `CFSTR` literal (16 bytes in `__DATA,__cfstring`) is aliased to an immortal host `CFString`, so passing it to CoreFoundation reaches a real string while the guest keeps its pointer. Guest Objective-C objects, guest blocks, Steam proxies, guest `FILE`s and placeholders for host variables such as `&kCFTypeArrayCallBacks` are aliases too.

Memory the guest asks for

`malloc` starts on a dlmalloc mspace in the window (`OCERZ_M32_HEAP_MB`, default 1024, growing). When a game registers a malloc zone of its own, as Batman does with tcmalloc, `malloc` runs that zone's functions through small generated i386 thunks, and `free` goes by who owns the pointer. Generated code gets its own static pool, because the JIT write-protects pages it believes hold code. Since the Intel Macs had 4 KB pages, `MAP_FIXED` at a 4 KB boundary maps host pages wholly inside the range afresh and zeroes only the covered part of the others (`cb5bd4a6`; Modern Warfare 3 commits single pages). `mach_vm_region` reports only the window (`8f8bb942`).

Running i386 code

Once started, an m32 program runs in `ocerz_vm_run` on the ordinary interpreter and JIT with `cpu->mode32` set, using the 32-bit translations built for Wine's WoW64 (Chapters 6 and 15). The i386 differential gate compares the two engines on 40,044 generated sequences in each of two memory layouts (`95db4cfc`).

The speed work

Batman's first 31 frames a second came mostly from instructions the 32-bit JIT still declined, each a call-out to the interpreter. Jfishin's second pull request removed them in order of frequency in the render thread, then cut the cost of calls. Everything landed on 2026-10-10 and was measured on the New Game opening on an M4 Max:

Commits	What changed	Share of what was interpreted	fps	ms CPU per frame
`0ca24d64`	m32 lands		31	32
`45ed22a9`, `f8fa8aaa`, `9523b275`	shifts by `cl`; MMX; `shld` and `shrd`	43%; 14%; a third of the rest	40	24
`ef750e2c`	imports called from inside the block	(run loop: 17% of the thread)	51	19
`e6e93ecb`, `358e33fb`, `e7cd1400`	memory shifts by an immediate; `ah`, `ch`, `dh`, `bh`	a fifth; a quarter; a quarter	57	17

Commits	What changed	Share of what was interpreted	fps	ms CPU per frame
`b92a3c9d`	leaf exports, no crossing		61	16

The fourth row's commits state no frame rate; 57 and 17 are where `b92a3c9d` says it started. `OCERZ_NO_HIGH8_ARITH` and `OCERZ_NO_SHIFT_MEM_IMM` turn two of these translations off, in 64-bit code too. On 2026-10-18 `b77d67dc` translated `adc` and `sbb` into memory with dead flags, i386's way of adding a 64-bit value to memory. DarkPlaces averages its textures that way, and Nexuiz's benchmark demo went from 104.1 to 96.9 billion cycles.

Native mode by default

An i386 program has no x86 shared cache, so for m32 the mode decides little but how x86 NaN results are made. In cache mode, translated SSE code relies on `FPCR.AH` for them (Chapter 7), and host frameworks must not run with that bit set. So since `281e193d` (2026-10-18) `m32_trap` clears it for every call into the host and `m32_call` sets it for every callback. Each write costs about 8.5 ns on an M5, so a call paid about 17 ns, more than the NaN checks the bit saves. Hours later, `1ae3140a` made an i386 program run in native mode unless `-cache` or `OCERZ_MODE` says otherwise. That means translated NaN checks and no FPCR writes, and Nexuiz's benchmark demo took 8% fewer cycles. The same commit stopped m32's in-block calls from waiting for `-native`, a condition inherited from the x86-64 bridge. `tests/run_m32_tests.sh` checks the default.

> **HOW IT WAS FOUND: THE FAULT IN APPKIT**
>
> Wesnoth stopped while AppKit loaded its main nib, in host code with no guest frame near. `OCERZ_WILDDUMP=1` was taught to walk the host's frames at a fault in host code, reading each through the kernel (`281e193d`). The frames led into AppKit's TextKit upgrade of an archived `NSTextView`, which loads natively; the difference was the floating-point mode bit that translated code had left set.

Threads

A guest `pthread_create` makes a host thread with a 32-bit CPU attached, at user-interactive quality of service; at the default, macOS sometimes leaves a game's render thread on the efficiency cores for good. A thread the host started gets a CPU and a 2 MB guest stack the first time it calls guest code. i386 pthread objects are smaller than arm64's (a mutex is 44 bytes, not 64), so on first use a guest object's signature word becomes a magic value and the next 8 bytes point at a host object built from the guest's initializer. i386 Darwin's `gs` points at the thread's specific data, and V8 in Left 4 Dead 2's Chromium checks that it can read `gs:[key * 4]`, so `pthread_setspecific` writes there too (`7cab7590`). Signal handlers are recorded but not delivered, since m32 builds no i386 signal frames.

Calls into the host

An i386 program calls a system function through an import stub, a lone `jmp dword [slot]` whose slot dyld filled. Under m32 the slot holds a trap address.

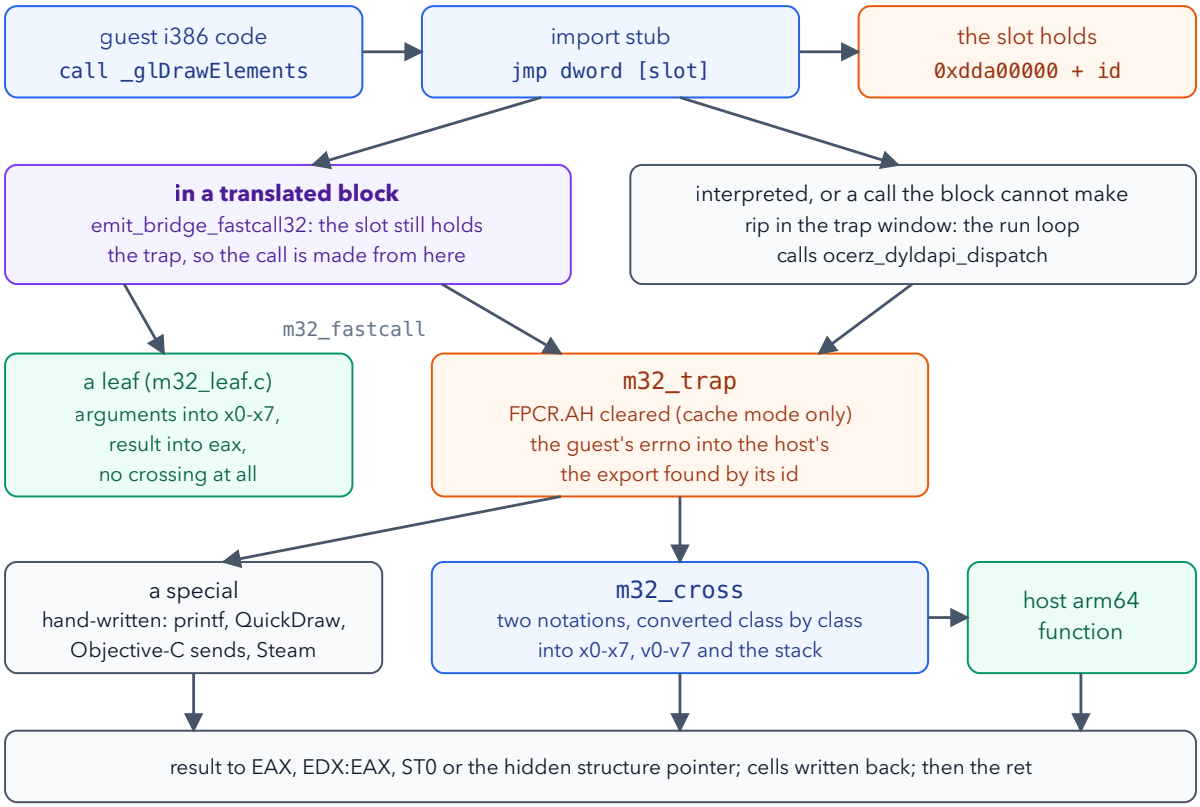


Figure 13.3. A call out of i386 code. From a translated block the call is made in place, as a leaf when it can be; otherwise the run loop dispatches the trap. Either way the export runs as a special or through the generic crossing.

Imports are traps

`m32_export` (`src/m32_bridge.c`) gives each symbol name one export with an ID from `0x10` and answers `M32_TRAP(id) = 0xdda00000 + id`. The table is a fixed array indexed by ID, read by the trap without a lock; entries are only added, under one. A bridged function's host entry point is found in the framework the x86-64 bridge opened by its 64-bit database, so `make apis32` builds those too (`a62822eb`). Set up with `make apis32` alone, Batman had quit with "CoreFoundation has no CFStringCreateWithBytes". A data import is a guest variable. A pointer variable such as `NSApp` may change after load, so live variables are rescanned after crossings: every crossing at first, every 64th after a million, because the scan had cost 3% of Batman's render thread. A variable's guest address is aliased to the host's, since GLUT's fonts are compared by address.

When either engine reaches the trap window, `ocerz_dyldapi_dispatch` hands an m32 thread's trap to `m32_trap`; WoW64 threads stay out. `m32_trap` clears `FPCR.AH` if cache mode uses it, loads the guest's `errno` into the host's, and runs the export's special or the generic crossing. An export with neither logs its name once and returns 0. Loading `errno` first keeps C's contract that a call which sets no `errno` leaves it alone. Until `28657cdf` (2026-10-17) m32 copied the host's last `errno` back after every call. SuperTuxKart's music never loaded: libvorbisfile clears `errno`, reads, and takes a short read with a non-zero `errno` for an error.

The i386 API databases

`make apis32` runs `tools/sdkgen.sh --guest i386` for 33 libraries, from libSystem and AppKit to Carbon, OpenAL and GLUT. Each library's headers are parsed through libclang twice, as i386 macOS 10.14, the last macOS that ran i386, and as arm64. The output, not committed because it derives from Apple's SDK, is `runtime/apis32/macos/<version>/<leaf>.api`, with a `.objc32` file beside it holding the i386 encodings of host classes' methods. Lookups strip variant suffixes such as `$UNIX2003`. The records are:

```
fn32   <export> <host symbol> <guest notation> <host notation>
data32 <export> <host symbol> <guest size> <host size> ptr|int|blob
bad32  <export> <reason>
```

Many fixes went into the generator. A lone `double` variable, aligned to 4 on i386 and 8 on arm64, was marked "layout", so `NSAppKitVersionNumber` read 0 and SDL 2 found no displays for MW3, until `c3654064` made such a variable a blob. `GLhandleARB` is a `void *` holding an object name, so it now crosses as an integer (`d19f1f56`). As a handle it had made Left 4 Dead 2's frames black.

The notation

Notations use native mode's letters (Chapter 12), read with i386 sizes on the guest side: pointers are 4 bytes, and 8-byte members of structures are aligned to 4. A C `long` is `i` or `u` in a guest notation and `l` or `L` in a host one. `@`, `#` and `:` are an object, a class and a selector. m32 adds classes for what a 32-bit caller passes by pointer:

Class	Meaning on the guest side
`s`	a C string; a host result is copied into the window once per distinct string
`P`, `W`, `V`	one pointer, signed or unsigned long written back through a pointer
`S`	a 16-bit integer the host writes as 32 bits (`FSIORefNum *`)
`O`, `E`	an object read and written through a pointer, or only written (`NSError **`)
`R{…}`, `r{…}`	a pointer to one structure whose layouts differ, copied in and back (`r`: in only)
`Q`	other data whose layouts differ; only a handle may pass
`G`, `T`	an AppKit tag; a `dispatch_time_t`, converted between Intel and Apple silicon clocks
digit after `{`	the structure is packed to that many bytes

The generic crossing

`m32_cross` parses an export's two notations on its first call and keeps them. It reads the arguments from the guest stack starting at `esp + 4`, converts each class by class, and places it as Apple's arm64 ABI wants it. Integers go in `x0` to `x7` and floating-point values in `v0` to `v7`. An aggregate of up to four floats or doubles goes in `v` registers, another structure of at most 16 bytes in `x` registers, and a larger one by reference. The overflow goes on the stack at natural alignment. Function pointers become callback slots and blocks become host blocks (below). The call goes through `abicall.s` under a bridge frame, so a fault inside it is reported as that call's (Chapter 12). The result comes back the i386 way: `EAX`, `EDX:EAX`, floats and doubles in `ST0`, and structures of 1, 2, 4 or 8 bytes in registers. A structure whose only member is a float or double comes back in `ST0`, and any other through a hidden pointer that the callee pops. Written-

back cells are copied home first; an `E` only if the callee stored into it, since `NSError *error;` is often uninitialized. If the host raised an Objective-C exception, `src/m32_catch.m`, m32's only Objective-C file, stops it at the crossing and it is rethrown into the guest's innermost `@try`.

Where the two widths differ

- **NSNotFound** is `INT64_MAX` on the host and `INT32_MAX` in the guest. Truncated, it read −1, and Adium raised `NSRangeException` from `indexOfObject:`. Since `aa51c8bb` (2026-10-18) each crosses as the other, inside an `NSRange` too; for arguments, only in messages.
- **id * arguments** crossed as plain pointers, and the host stored an 8-byte id over two guest words. AIUtilities read a truncated `NSError` and crashed. They are now `E` or `O`.
- **NSRange pointers.** `longestEffectiveRange:` got 16 bytes where the guest keeps 8, answered a length of 0, and Adium's link filter looped forever. Such a pointer is now `R{uu}` in the guest and `R{LL}` on the host.
- **AppKit tags** are 64-bit, and AppKit raised "Truncated the NSTrackingRectTag to 32bit". The guest gets a small number for each (`G`).
- **Arrays of ids** (`arrayWithObjects:count:`) cross as copies of host ids (`8ae32ab4`), and CFNumber's `long` and `CGFloat` types become the 32-bit ones (`577730e3`).

The host can call a guest method with such a pointer too, as AppKit does with a guest `validateValue:forKey:error:`. Handed the host address, the guest wrote over a handle cell. Since `b9684609` it gets a guest copy, written back after the call.

Structures packed to 2

Carbon's headers declare their structures under `#pragma pack(2)`, which on arm64 leaves a pointer after a 4-byte field at offset 4: an `AEDesc` is 12 bytes, not 16. The generator could not express that and gave the host side a plain pointer, so 143 calls copied their structure into a host buffer of size zero, among them the Apple Event Manager, OSA and `GetMenuTrackingData`. Since `88b7e20c` a digit after the brace caps member alignment, as in `R{2up}`. The generator writes it when a record's offsets match clang's only so capped. 163 notations changed, and the 64-bit databases stayed byte for byte the same.

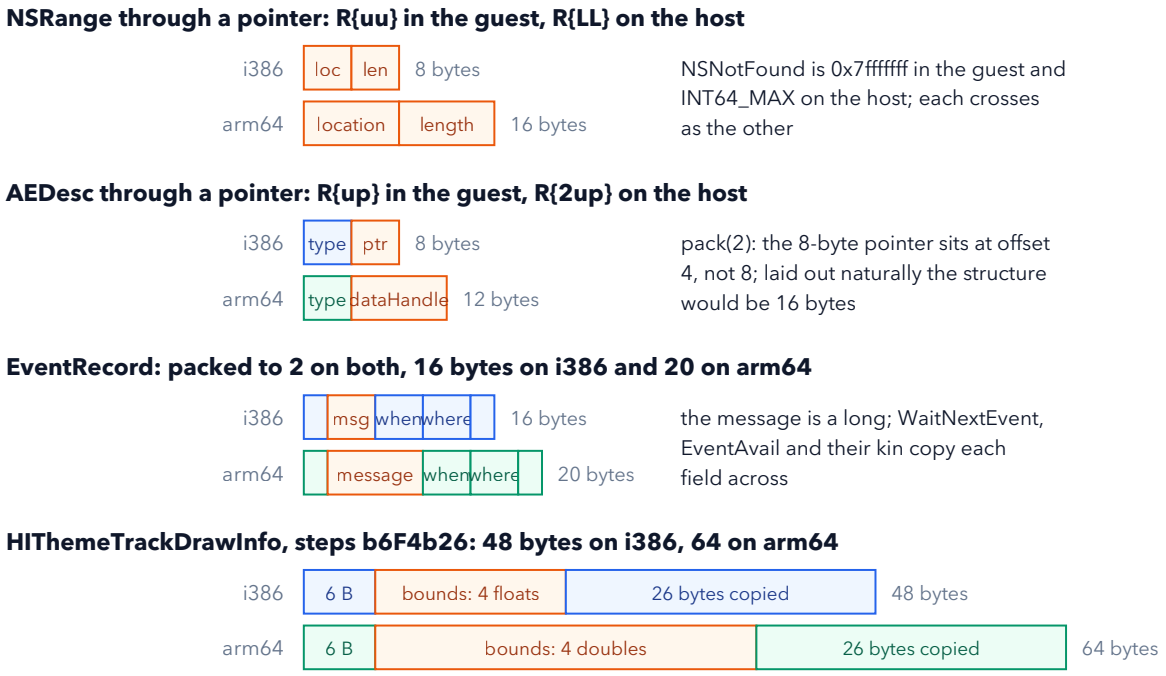


Figure 13.4. Four structures as each side lays them out, at 6 units per byte. Orange fields change width or place and are converted; the others are copied as they are.

Some structures no notation describes, because they hold a union or widen inside a packed layout. Most of HIToolbox's theme structures are laid out alike on both sides, as measured with `clang -arch i386` and `-arch arm64`; six are not. `HIThemeTrackDrawInfo` has its `HIRect` after six bytes, so HIToolbox read Firefox's scroll bars 16 bytes off. Since `c64cca9f` the six are converted for the 22 calls that take them by strings of steps: `b` copies bytes, `F` widens floats, `L` words and `P` handles (`OCERZ_NO_M32_PACKED=1`). Specials convert the rest field by field: `EventRecord` (16 bytes against 20), `ProcessInfoRec`, `z_stream` (56 against 112), `struct lconv`, `addrinfo`, `rusage`, `statfs`, `dirent`, `AudioQueueBuffer` and `aiocb`.

Arrays of CGFloats

`CGFloat` is a float on i386 and a double on arm64, and CoreGraphics takes arrays of them, of points and of rectangles everywhere. A table in `src/m32_cross.c` says how long each array is: a fixed count, a count argument, a color space's components plus alpha, or a `CFArray`'s count. The array crosses as a copy of doubles, and output arrays come back as floats. It covers colors, dashes, point and rectangle lists, glyph positions, gradients and CoreText's advances (`34b6f7fb`). Before it, Kega Fusion's `CGColorCreate` passed floats read as doubles, and `CGContextFillRects` crossed only its first rectangle.

Specials

Calls whose forms differ in layout, that allocate for the caller, that are variadic or that exist only on i386 have hand-written specials, found by exact name, then without the variant suffix, then by Carbon name pattern. Variadics rest on `m32_printf_slots`. An i386 `va_list` is a pointer to the next stacked argument, while an arm64 one points at 8-byte slots. The walker reads the format as `printf` will, takes each argument at its i386 size, fills the slots, and calls the host's `v`-form. Positional arguments fill slots in position order (`d0143034`; VLC's localized strings), and `scanf` writes through scratch cells narrowed afterwards. `NSLogv`, `CFStringCreateWithFormat`, the wide `printf`s and `ioctl` use the same walker.

Calls from inside translated code

The in-block crossing

Through the run loop, a call costs a block exit, a trap, and a return through the dispatcher that abandons the caller's host return-address stack entry (Chapter 6). Batman's render thread made about 30,000 such calls a frame. `emit_bridge_fastcall32` (`ef750e2c`), the i386 form of native mode's fast call, turns a block's final `jmp dword [slot]` into a call to `m32_fastcall` from inside the block when the slot holds a trap, followed by the export's `ret` through the return-address stack. The slot and the host stack depth are checked again at run time. Only `xmm0` to `xmm7` are spilled, since i386 code names no others. The block continues only if the export returned like a function: `rip` the word that was at `esp`, `esp` up by 4 to 260 for a `stdcall` callee, nothing pending and no translation retired. Otherwise it leaves for the dispatcher.

Leaf calls

Batman's threads call small libc functions millions of times a second, and Feral's IndirectX wraps every OpenGL call in CGL context calls. `src/m32_leaf.c` (`b92a3c9d`) answers 21 exports as leaves, with no crossing. They include `memcpy`, `memset`, `memcmp`, `strlen`, `wcscmp`, the pthread-specific calls, `mach_absolute_time`, `OSMemoryBarrier` (a `dmb`) and four CGL context calls, the last of which, `CGLGetCurrentContext`, joined in `9c8001ca` because Valve's ToGL asks for it about 500,000 times a second. The JIT loads the arguments off the guest stack into `x0` to `x7`, adding the guest base to pointers, and puts the result in `eax`. Then it returns through the return-address stack. Guest registers stay pinned in `x21` to `x28`, which the call preserves. The file is compiled with `-mgeneral-regs-only`, because the guest's `xmm` registers stay pinned in vector registers, and the JIT saves `xmm0` to `xmm7` around leaves that run OpenGL's code. Beyond the table, any OpenGL function that takes and returns only integers and pointers, with matching notations, is a leaf through `m32_leaf_tramp`. That trampoline clears `FPCR.AH`, without which Apple's GL engine faulted in Nexuiz. `memcmp` and `strlen` are libc's own since `b77d67dc`, after a byte loop cost DarkPlaces 6% of its benchmark. `OCERZ_NO_M32_LEAF=1` turns leaves off.

Trap blocks

A function pointer to an export, from `dlsym` or a GL extension lookup, is the export's trap address. A call through it went back to the run loop every time, and DarkPlaces calls all of OpenGL that way, about 250,000 calls a second. Since `1ae3140a` (2026-10-18), `jit_decode` asked to decode a trap address in an m32 process produces `ff 25 <slot>`. That is a `jmp dword [slot]` through a guest word holding the trap, allocated once per export by `m32_trap_slot`, so the call is made from inside a block like any import, as a leaf when it can be. A trap block that cannot make the call hands the trap to the run loop with `OCERZ_EUNSUP`, since its own jump would find the same block again. With the commit's changes together, trap blocks, in-block calls in either mode and native mode by default, Nexuiz's benchmark demo (3,104 frames) went from 133.6-138.8 to about 107 billion cycles, and from about 100 to 114 frames a second on an idle M5. Since `c64cca9f`, `dlsym` answers NULL for a function m32 cannot call, so a program probing for an optional function takes its own fallback.

Calls back into guest code

Host code calls function pointers: `qsort` its comparator, CoreAudio its render callback, AppKit every method of a guest class. A class `c{…}` carries the callback's guest and host notations. `m32_callback` (`src/m32_callback.c`) binds the guest function with that pair to a slot of the callback bank of 65,536 arm64 trampolines that native mode uses (Chapter 12), reserved through `ocerz_abi_callback_reserve` so that calls reach m32's dispatcher. The same function and notations always get the same slot, and a system function handed back is its own host function.

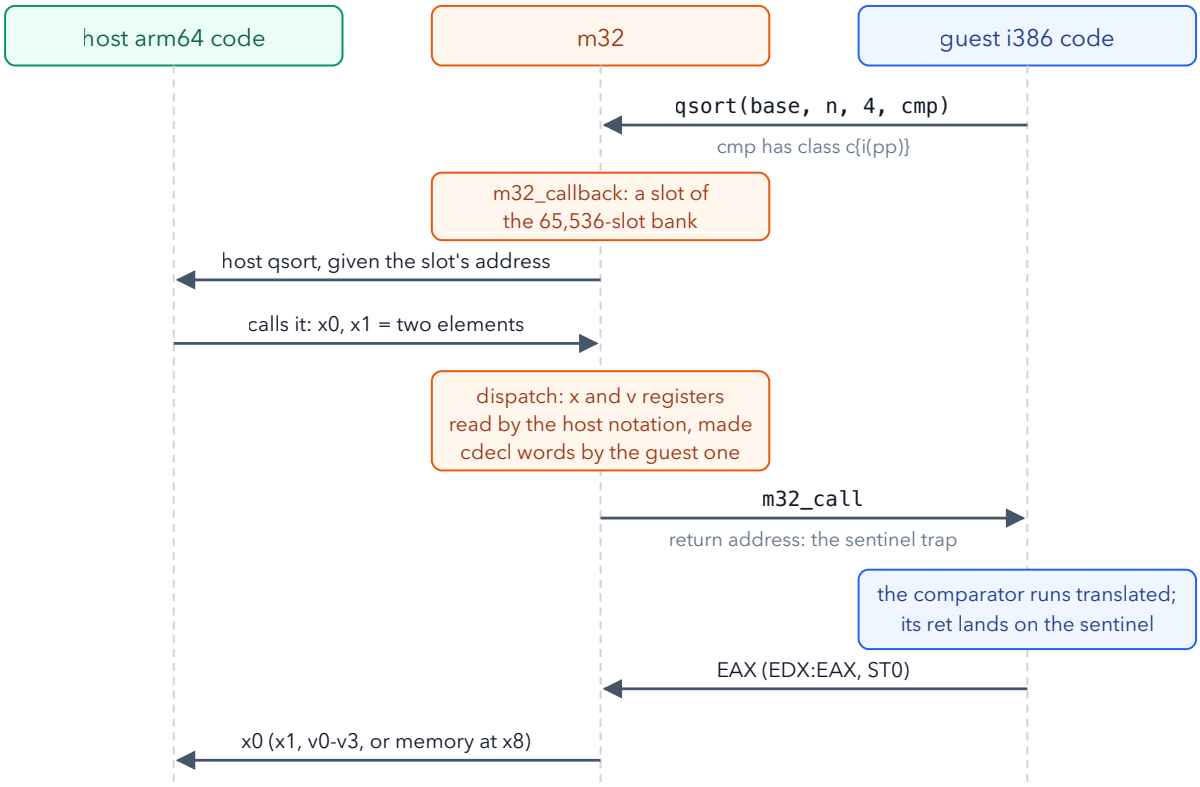


Figure 13.5. A callback round trip. The guest's function pointer becomes a slot of the callback bank on the way out; when the host calls the slot, m32 converts the arguments, runs the guest function on the thread's 32-bit CPU until it returns into the sentinel, and converts the result back.

When the host calls the slot, the dispatcher reads the arm64 arguments by the host notation and turns them into `cdecl` words by the guest one. A structure becomes the words of its i386 layout, and pointers of class `E`, `O` or `R` become guest copies. `m32_call` finds the thread's 32-bit CPU, attaching one if needed, and calls `ocerz_vm_call32`. That function pushes the words with the sentinel trap as return address and runs a nested guest loop until the function returns into it. Structure results follow both conventions: i386 returns up to 8 bytes in `EAX:EDX` and the rest through memory, while arm64 wants `v0` to `v3`, `x0` and `x1`, or `x8`. That case arrived with Firefox's window delegate (`c64cca9f`): given {0, 0} from `windowWillResize:toSize:`, AppKit sized the window to nothing. The nested loop is native mode's, which `ad51f4b3` (2026-10-18) made cheaper, measuring an x86-64 `qsort` comparator at 536 ns a call before and 100-112 ns after. No m32 measurement is recorded.

Callbacks inside structures needed specials. Run loop and SystemConfiguration contexts hold 4-byte fields with callbacks among them (`dc674f29`, `f5a1889e`). CoreAudio's render callback structure is two 4-byte words, which the host rejected, so FMOD Ex never started and Batman ran silent. The host now gets a

thunk that, on the I/O thread, hands the guest copies of the buffers and copies the samples back (`src/m32_audio.c`). `tsearch` trees live in guest memory, with libc's 8-byte key word replaced by an AVL tree of 16-byte guest nodes (`src/m32_tsearch.c`; VLC's `tfind` faulted).

CoreGraphics callbacks

`src/m32_cg.c` came with Firefox 3.6, which draws gradients, patterns and glyph outlines through cairo. `CGFunctionCreate`'s and `CGPatternCreate`'s callback structures are three words on i386 and three doublewords on arm64. Crossed as they were, CoreGraphics read a float of Firefox's domain as the evaluate function and jumped to it. The host now gets m32's callbacks, which call the guest's as guest code with float values in a guest buffer, under a lock because CoreGraphics may evaluate on any thread. `CGPathApply` hands the applier guest copies of each element. cairo's own declarations of `CGContextSetCompositeOperation` and `CGContextSetCTM` become a blend mode and a concatenation. Its private glyph calls are made with CoreText; without `CGFontGetGlyphPath`, Firefox stopped with "unable to find a usable font".

Blocks and C++ exceptions

A guest block crossing to the host becomes a global host block whose invoke is a callback slot, aliased to the guest block. A stack block is first copied to the guest heap, since the host may run it later. C++ exceptions use `src/m32_unwind.c`, a two-phase Itanium unwinder over the guest's `__eh_frame` that calls each frame's personality routine in the guest. GCC-built programs import `_Unwind_*` from `libgcc_s`, whose unwinder knew nothing of m32's, and SuperTuxKart aborted after its first throw. Since `71b7c555` those imports bind to m32's.

Objective-C 1

A 32-bit Mac program uses the original Objective-C runtime, with fragile instance variables and classes laid out as the compiler wrote them. The host has only the modern runtime. `src/m32_objc.c` (1,960 lines) is an Objective-C 1 runtime of its own, joined to the host's at the edges.

Proxies, pairs and twins

Guest classes stay the guest's structures, and m32 searches their method lists. A host class is stood for by a *proxy*, an `old_class` in the window flagged `OC_CLS_HOST`. A guest class whose superclass chain reaches a proxy is *paired* at load with a host class made by `objc_allocateClassPair`. The host class holds each guest method as a callback slot into the guest implementation and adopts the guest's protocols, because the host checks conformance itself. An instance is a pair: a host object and a guest twin of the guest's instance size, aliased both ways.

Twins once lived forever. A twin stayed aliased to a freed object's address, so the next object allocated there inherited it, isa and all. In Adium an AutoHyperlinks scanner came back as an icon source and answered "unrecognized selector -linkifiedString", in some runs. Since `aa51c8bb` each twin hangs a reaper on its host object as an associated object, which takes the alias back and frees the twin when the object dies (`OCERZ_NO_M32_REAP=1`).

A message

`objc_msgSend` is a special. A nil receiver answers 0. For a guest class or guest object, m32 looks the method up in the guest's lists, runs `+initialize` if needed, and on a hit sets `rip` to the implementation, which then runs on the caller's own frame with no crossing. Anything else goes to the host, with arguments converted by the method's two encodings: the host's from the runtime, the i386 one from the `.objc32` files or the guest's method list. `src/m32_objc_types.c` turns an encoding such as `v24@0:4{_NSRect={_NSPoint=ff}{_NSSize=ff}}8` into the notation `v(@:{ffff})`. The two encodings pair up because `l` in an encoding is always 32-bit, LP64 spelling a 64-bit long `q`. A few messages are diverted. `-[NSOpenGLContext setView:]` and `-update` trap off the main thread in today's AppKit, and SDL 2 sends both from its render thread, so they run on the main thread (`6c375f24`). `systemFontOfSize:` answers Lucida Grande, the system font these programs were written for, because Gecko found no usable font in the modern one (`c42a4d64`).

Nibs, bindings and bundles

Three AppKit habits assume a class's instance variables and image belong to the runtime, and a paired class broke each:

- **Bundles.** Foundation names a class's bundle with `class_getImageName`, and a class made at run time has no image, so Sparkle looked for its nib in the main bundle. Paired classes now name the guest image through `objc_setHook_getImageName` (`d0143034`).
- **Key-value coding** reaches an instance variable directly when no accessor answers, and the host class has none of the guest's. Pairs now answer `valueForUndefinedKey:` and `setValue:forUndefinedKey:` from the twin's variable `key` or `_key`, boxing numbers by type (`OCERZ_NO_M32_KVC=1`).
- **Nib outlets.** `NSNibOutletConnector` sets the instance variable of the outlet's name when the owner has no setter, so every such outlet of a guest owner stayed nil. Sparkle's prompt opened 0 pixels high. m32 replaces `establishConnection` to set the twin's variable (`aa51c8bb`).

Categories, `+load`, swizzling and `method_exchangeImplementations` of guest methods work. Guest protocols get host twins of the same name. The fragile runtime's `@try` is `setjmp`-based, so m32 keeps per-thread exception frames and its own `_setjmp` and `_longjmp` (`src/m32_objc_exc.c`). Code built for the garbage collector, as Adium is, calls `objc_assign_global` and its relatives for every object store; without a collector they are plain stores (`b9684609`).

Carbon on what Apple silicon keeps

Most 32-bit Mac programs were written against Carbon, which Apple never made fully 64-bit. m32's rule, since 2026-10-16, is to use whatever arm64 HIToolbox still exports and rebuild the rest on its successors.

What HIToolbox still has

Much of Carbon's window, menu, control and event API is declared for 32-bit code only (`#if !__LP64__`), yet arm64 HIToolbox still exports it, and a native probe showed it working. m32 had answered all of it with 0, 158 of Kega Fusion's imports among it. Since `3e692ce8` the generator declares each such function again in a second arm64 translation unit, written from the i386 type spellings. Carbon's database went from 364 crossed functions to 847. What still differs is handled at the host function (`src/m32_carbon.c`):

- **Windows.** 64-bit HIToolbox makes only compositing windows, and `CreateNewWindow` without `kWindowCompositingAttribute` fails with -5601. Every window an i386 program creates gets the attribute.
- **UPPs** were plain function pointers on Intel, but 64-bit Carbon exports no `New…UPP`. m32 answered 0, so every Kega Fusion handler was NULL. Each now returns its argument.
- **Event parameters** are laid out for the process's architecture. Reference types become 4-byte handles, of which only the low halves had come back before; `HIPoint` and `HIRect` become floats; a draw event's `GrafPtr` becomes the window's QuickDraw port.

QuickDraw, menus and controls

Apple silicon keeps QuickDraw's region and rectangle arithmetic and cursor visibility; a header the generator supplies declares those 42 functions. Ports, GWorlds, drawing, cursors and devices are gone, and `src/m32_qd.c` (`8b8d0288`) rebuilds them on CoreGraphics with i386 layouts measured against the SDK's headers. A window's port is a bitmap context over pixels in guest memory. On the first drawing, m32 installs a `kEventControlDraw` handler on the content view that draws the bitmap, and each drawing call asks for a redraw. A GWorld is a bitmap context over guest pixels, xRGB or x555 big-endian as on Intel Macs, so a program writing through `GetPixBaseAddr` draws. Pens, lines, rectangles, regions, clipping and `CopyBits` work. `src/m32_controls.c` (`bde6faf2`) makes the dropped menu and control calls from HIView and menu successors. User panes are views of a class m32 registers, calling the guest's draw, hit-test and tracking procedures, and a pane shows what its draw procedure drew with QuickDraw.

AGL

Apple silicon still exports AGL and it works, but no SDK has its headers. `src/m32_agl.c` (`34b6f7fb`) crosses its 38 calls by hand with `agl.h`'s documented signatures. An `AGLDrawable` is a QuickDraw port, and 64-bit `aglSetDrawable` refuses every one. So a window's port is set as its window with `aglSetWindowRef`, and a GWorld's as its pixels with `aglSetOffScreen`.

ATSUI and ATS on CoreText

Left 4 Dead 2's vgui finds its fonts with ATS and draws all its text with ATSUI. Today ATSUI rejects every font, ATS deliberately traps in a function whose name asks for a bug report, and the SDK declares none of ATSUI's functions. `src/m32_atsu.c` (`48d41344`, Jfishin, 2026-10-13) keeps the documented contracts on CoreText. Fonts are activated with `CTFontManagerRegisterGraphicsFont`, styles carry font, size, face and color, lines are drawn into the layout's context, and layout records come back in their 14-byte i386 form. Later commits added font tables, glyph bounds and the Font Manager's families with their classic IDs (Geneva 3, Helvetica 21, the system font Lucida Grande). CoreText's layout is not ATSUI's to the pixel.

HOW IT WAS FOUND: KEGA FUSION, ONE STOP AT A TIME

Kega Fusion 3.63, a UPX-packed Carbon emulator drawing through AGL, drove 2026-10-16, each fix the next thing that stopped it. Its stub died at `sysenter` (`0144d669`). Unpacked, its start code, linked for 10.4, jumped to the fixed address dyld had then (`487c886a`). It then reached `main` and stopped at its first window, with 242 imports from the 32-bit-only Carbon UI. HIToolbox turned out to export 158 of them (`3e692ce8`). With QuickDraw rebuilt it stopped at `glViewport` for want of AGL. Along the way AppKit crashed making a window sized from garbage an unanswered call had left. lldb could not attach, and `OCERZ_ABORT_REPORT=1`, which names an abort's host frames and guest instruction, was written to find it. With AGL, UPPs, event parameters and `CGFloat` arrays (`34b6f7fb`), it opened its 640 by 502 window two seconds after start.

OpenGL

Drawables in points. Linked with today's SDK, `wantsBestResolutionOpenGLSurface` defaults to YES, and a drawable on a Retina display has twice the window's points. The i386 programs were linked when it was NO, so Left 4 Dead 2 filled a quarter of its window. Since `81a1ba9f` a view given to an `NSOpenGLContext` gets NO unless the program chose itself.

Buffer offsets. With a buffer bound to `GL_ARRAY_BUFFER`, a `gl*Pointer`'s pointer is an offset into it, and with one bound to `GL_ELEMENT_ARRAY_BUFFER` so is a `glDraw*Elements` index pointer. m32 turned them into window addresses, and Nexuiz's benchmark faulted. Since `1ae3140a`, `src/m32_gl.c` tracks each context's two bindings as the guest makes them, without asking GL, which would wait on its multithreaded engine, and passes such pointers through.

Mapped buffers. `glMapBuffer` returns driver memory outside the window, so a mapping gets a shadow in the guest heap, copied out as ranges are flushed or at unmap. At first each map copied the whole buffer in and asked `glGetBufferParameteriv` for its size, which waits for every queued command. In large fights that was about half of Left 4 Dead 2's render thread. Since `e7a180f2` a shadow is the buffer's mirror: `glBufferData` and `glBufferSubData` keep it true, so a map copies nothing in and asks the driver nothing. Writers m32 cannot see exist, so only array and element-array buffers are mirrored.

The libc and libSystem gaps

libSystem is where the two widths meet most often. Typical gaps the programs found, each now a special or a generator fix:

Call	What went wrong	Found by	Commit
`bootstrap_look_up`	its `name_t` had no notation; `eax` came back as it was	Steam's library	`d2eab4cc`
`dispatch_once`	a 4-byte predicate; m32 runs the work itself	MW3's first run	`9b2eb1e6`
`getaddrinfo`	48-byte host records passed as a handle	MW3	`eabdc7b5`
`scandir`	host dirents; no sound files found	Left 4 Dead 2	`bbb3afe2`
`finite`	no declaration, so 0: ragdolls froze	Left 4 Dead 2	`a1c3dff6`
`fopen`	a handle whose `_file` read 0; settings saved empty	Left 4 Dead 2	`fadef476`
`__srget`	libc's refilled the buffer for every character	Pango	`281e193d`

Call	What went wrong	Found by	Commit
`localeconv`	56-byte i386 `lconv` against 96	GLib, in Wesnoth	`281e193d`

A guest `FILE` is now an 88-byte i386 `__sFILE` aliased to the host stream with only `_file` set, which is why `getc` calls `__srget` for every character. Clocks follow the Intel Macs: `mach_absolute_time` counts nanoseconds with a 1/1 timebase, as Bink's timer expects.

The Steam bridge

Valve has shipped no i386 `steamclient` since 2019. A 32-bit game's own i386 `libsteam_api` asks the running Steam where its client lives, loads that `steamclient.dylib`, finds no i386 slice and quits. Jfishin's bridge (`825fe3a8`, 2026-10-11) answers that load. Nothing of Valve's is shipped: the game talks to the user's own Steam, which still serves every interface version back to `SteamClient006`.

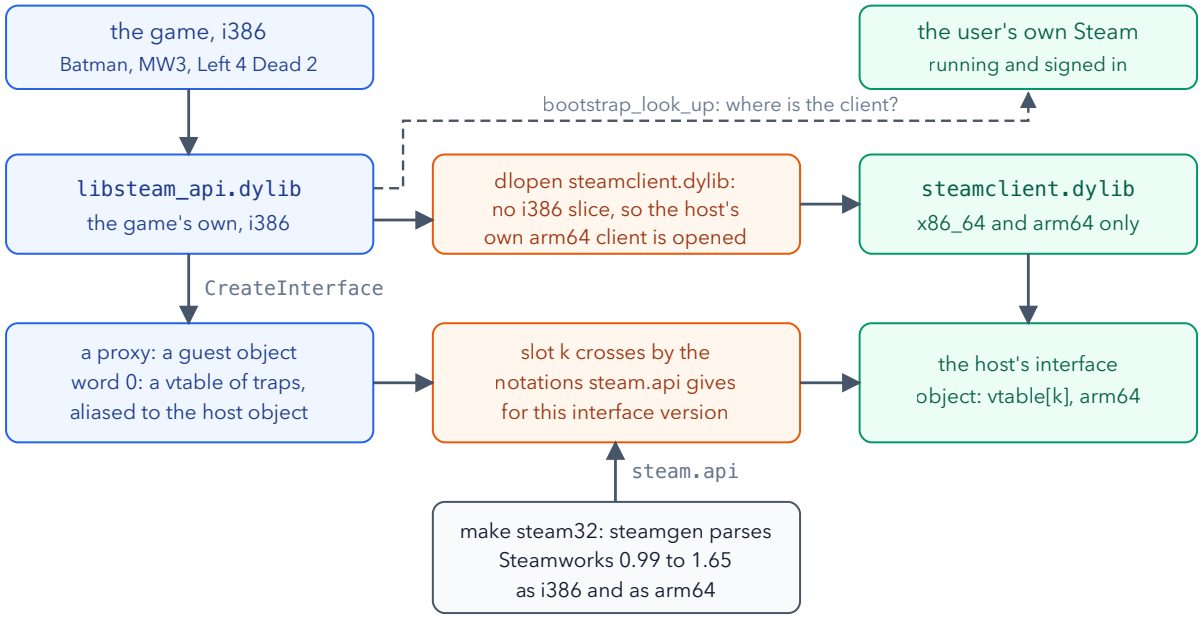


Figure 13.6. The Steam bridge. The game's own i386 library finds the running Steam, m32 opens the host's arm64 client in place of the i386 one that no longer exists, and every interface the game gets is a guest proxy whose virtual methods cross into the host object's.

Steam tells only processes it launched where the client is, so a game must be started from Steam with `ocerz -native %command%` as its launch options. Its `libsteam_api` asks with `bootstrap_look_up`, which needed a special first (`d2eab4cc`). Then `m32_steam_dlopen` opens the host's arm64 client and hands back a handle whose `dlsym` answers the client's exports itself (`src/m32_steam.c`). `CreateInterface`, and every method that returns an interface, return a proxy: a guest object whose first word points at a vtable of trap addresses, aliased to the host object, so the generic crossing turns a proxy `this` into the host object. Each slot crosses to the host vtable's function at the same index by the notations for that interface version. Proxies are keyed by object and version (`68cda65c`). Returned strings are copied into guest memory under a lock (`05c04778`). `Steam_BGetCallback`'s message holds a pointer and is converted; Steamworks 1.23 and later lay callbacks out with `pack(4)`, which both sides share.

`make steam32` runs `tools/steamgen.sh`, which fetches the Steamworks headers of every SDK from 0.99 to 1.65 from Valve's Proton repository at a pinned commit. `tools/sdkgen/steamgen.c` parses each set as i386 and as arm64 C++ and writes `steam.api`: an `iface` line per version, then each method's slot, name and two notations, flagged when it returns an interface or a string, or `bad` with a reason. Only names and notations go in. On 2026-10-11 (`ff020772`) it found 211 interface versions, 6,303 methods and 101 bad slots. Since then records returned by value cross as `R<size><class>`, 35 slots (`80d2da59`). i386 passes a hidden pointer for such a record ahead of `this` and the method pops it, so answering 0 had sent Left 4 Dead 2 to address `0x1001` once a controller was connected. Methods taking a matchmaking response object are bad, 28 slots, so the game finds no Internet servers (`b819a13e`). A bad slot answers 0. The `steambridge` test runs against `tests/m32/steamfake`, an arm64 stand-in for the client.

What runs

These are hand-run results from the commit messages and documentation; none of the programs is in a gate (Chapter 16). The Steam games were measured on an M4 Max, Nexuiz on an M5.

Program	Result	Commit, date
Batman: Arkham Asylum (Feral, 2014)	about 31, then 61 fps; from Steam, signed in, 21 interfaces bridged	`b92a3c9d` (10-10), `825fe3a8` (10-11)
Call of Duty: Modern Warfare 3 (Aspyr, 2011)	launcher, movies with sound and the campaign, from Steam	`a322c200` (10-12)
Left 4 Dead 2 (Valve)	campaigns from Steam at 80-120 fps; a timedemo went from 32-41 to 75-77 fps once the game was brought to the front	`0dd3cb74`, `d20ee9a7` (10-13)
Kega Fusion 3.63	runs; window two seconds after start	`34b6f7fb` (10-16)
VLC 1.1.12	controller window opens and runs	`d0143034` (10-16)
Firefox 3.6.28	draws its window, which does not reach the screen	`c64cca9f` (10-16)
AssaultCube, SuperTuxKart	play with sound and music	`71b7c555`, `28657cdf` (10-17)
Neverball, Neverputt, Battle for Wesnoth	run; Wesnoth's title screen and test scenario	`281e193d` (10-18)
Adium 1.4.5, Teeworlds 0.5.2	Adium's setup assistant, contact list and update prompt; Teeworlds plays	`aa51c8bb` (10-18)
Nexuiz 2.5.2	loads a map and its bots play; benchmark about 100 to 114 fps	`88b7e20c`, `1ae3140a` (10-18)

The Left 4 Dead 2 row carries a lesson. Its launcher calls `SetFrontProcess` before its `NSApplication` exists, and today's cooperative activation ignores that, so the game ran unfocused and throttled. `0dd3cb74` also asks the application to activate once its run loop turns. A frame rate measured in the background means little on this machine.

Testing

Only Xcode 26's `ld-classic` links i386; Command Line Tools 27 dropped it. The 79 programs in `tests/m32` are therefore committed as binaries, and `M32_REBUILD=1` rebuilds them against stub `.tbd` files. Each `.expected` file holds the output of the same source built for x86-64 under Rosetta, or for arm64 natively, or written by hand. `tests/unit/test_m32.c` (1,725 lines) calls specials directly on a 32-bit CPU, for what no linked program can pin. Its `carbon` mode crosses calls through the generated databases as translated code does. `make check-m32` runs the database spot checks, `test_m32 carbon`, the programs, and `high8` and `shiftmem` again with their translations off. On the MacBook Air (M5, macOS 27) on 2026-10-16, at `c64cca9f`, `test_m32` passed 424 checks and `test_m32 carbon` 347, none failing. The README of `95db4cfc` gives the program gate as 79 / 79, with one more skipped without a Steam database. Chapter 16 has the detail.

What does not work yet

- **Signals** are recorded but never delivered: there are no i386 signal frames.
- **Carbon UI that is gone.** Of the control kinds, arm64 keeps push buttons, check boxes, static text and icons. Sliders, tabs, pop-up buttons, edit text, scroll bars and the Data Browser are not made yet, nor Navigation Services dialogs or MLTE.
- **QuickDraw** draws no text, copies for every transfer mode, and refuses GWorlds of 8 bits or fewer.
- **No crossing.** `long double` calls stay unresolved, IOKit plug-in interfaces answer unsupported (no old-style HID joysticks), QuickTime's calls fail as on a Mac that cannot open the movie, and OpenCL is not bridged.
- **Firefox's window** is visible to AppKit but off screen to the window server, for reasons not yet known; the process has a second window server connection that Gecko's Carbon event handler opens.
- **Steam.** Methods taking function pointers or response objects answer 0, and achievements and cloud saves are unverified. In Left 4 Dead 2 exit to the main menu crashes and console letters double. Batman's options window, a legacy WebView, opens black.
- **Lifetimes and limits.** Handle cells are never freed, blocks copied for the host leak, and zlib streams and GL shadows are capped at 256 each. The unwinder reads only `__eh_frame`.

> **KEY IDEA**
>
> m32 treats a 32-bit Mac program as two problems. Its code is ordinary i386 and goes through the translator AArchX already had for WoW64. Its calls are the hard part, because every pointer, long and float it hands the system means something different at 64 bits. Notations generated from the SDK handle most of that mechanically, specials handle the rest one layout at a time, and what Apple silicon dropped is rebuilt on what it kept.

PART V

Real software

Cocoa applications, the macOS Steam client, games, the GPU, and Windows software through Wine.

CHAPTER 14

macOS applications, Steam, games and the GPU

The earlier chapters describe mechanisms; this one describes what they add up to when real software is pointed at them. It walks through the command-line tools, the Cocoa applications, the GPU, the macOS Steam client and the first games, and for each one says what broke on the way and what that taught the translator. Since October it also covers the AArchX.app library that wraps the command line, 32-bit Mac games played from Steam, and Windows programs started from CrossOver. Every result here was run by hand and dated; none of it is an automated gate.

---

16 / 16
CLI TOOLS BYTE-IDENTICAL TO NATIVE

74
SYSTEM TOOLS AGREEING WITH ROSETTA

14
FIXES TO BRING UP MACOS STEAM

0.54 s
ROSETTA'S FIRST BRAWLHALLA FRAME (AARCHX: 5.3 S)

How an application is run

AArchX runs the x86-64 slice of a Mach-O executable. For a bundled application that used to mean naming the binary inside the bundle, for example `./ocerz /System/Applications/Chess.app/Contents/MacOS/Chess`. Since 2026-09-23 (`cd1f9c1c`) the bundle itself can be given: AArchX reads the bundle's `Info.plist`, runs the executable it names, and passes that full path as the guest's `argv[0]`, which matters because applications derive their resource paths from it. That change came from games, which Steam launches by bundle (see *Games* below).

Two more kinds of program can be named since October. A program that has only an i386 slice, which no current macOS runs, goes to native mode, where m32 (Chapter 13) runs it; the mode is chosen by what the file carries. And a file that begins with `#!` runs as itself: Steam's `%command%` for a Source game on the Mac is a shell script, `hl2.sh`, that starts the game as `${GAME_DEBUGGER} <game> <args>`. Until `46fade2e` (2026-10-13) `ocerz` took that script for a program and stopped with "no x86_64 slice"; now it runs the script with `GAME_DEBUGGER` naming itself and the flags that came before the script, so the one launch option `ocerz -native %command%` serves a script-started game as it serves an app. A `GAME_DEBUGGER` that is already set wins, and because the scripts expand the variable unquoted, an `ocerz` path with a space in it is refused with a message.

"Works" in this chapter means a command-line tool printed exactly what it prints natively, or an application drew its main window on screen and stayed up. `tools/apptest.sh cli` automates the first check by diffing each tool's output against the native run; `tools/apptest.sh gui` automates the survival check. Neither is part of `make check`, because both depend on the machine's installed software.

macOS 27 changes what there is to run

On macOS 27 Apple ships the bundled applications, Chess, Calculator, Dictionary, Font Book, Grapher, Digital Color Meter, Activity Monitor, Console, TextEdit, Preview and Script Editor among them, as arm64 only. There is no x86-64 slice to translate. AArchX says so and exits, and since 2026-09-23 (`29159dba`) a guest that spawns or execs an arm64-only program gets the native program, exactly as it would under Rosetta. That rule came from Steam, not from the bundled apps: the Steam client spawns `/usr/bin/open` three times per launch, `open` is arm64-only on macOS 27, and every one of those children used to die on "cannot read" before doing anything. The application results below therefore say which macOS they were measured on. Safari, Steam, Discord and Brawlhalla still carry Intel code and can be run on macOS 27.

Command-line tools

Command-line tools are the most reliable case and the cheapest regression check. Sixteen are diffed against native output by `tools/apptest.sh cli`, and all sixteen match on macOS 26.7.1 and on macOS 27 since 2026-10-03:

Tools	What they exercise
`uname`, `sw_vers`, `id`, `echo`, `basename`	process identity, sysctls, CoreFoundation and, on macOS 27, Swift
`ls`, `wc`, `sort`, `uniq`, `head`, `grep`	file I/O, locale, `sysconf`, the C library's string and memory routines
`file`, `xxd`, `nm`, `plutil`	Mach-O and property-list parsing, larger dependency graphs
`openssl version`, `openssl dgst -sha256`	two-level namespace binding, vector crypto code

Two of them earned their place by failing. `openssl` links `libcrypto.46.dylib` while the shared cache also carries `libcrypto.44.dylib`, which exports the same names; a flat search bound `OpenSSL_version` to whichever image came first and printed the wrong version. Imports are now resolved in the specific library the binary named (two-level namespace binding, Chapter 11). `sort` needed the `sigaction` structure translated between its two layouts and the 64-bit width of the `hw.pagesize` sysctl: writing four bytes into an eight-byte answer had made `sysconf(_SC_PHYS_PAGES)` return zero.

Beyond the sixteen, a wider sweep of 74 system tools run with read-only arguments in cache mode was compared against each tool's own x86-64 slice under Rosetta, and every one agreed: `sed`, `awk`, `otool`, `shasum`, `curl`, `sqlite3`, `perl` and the rest. The only disagreements were tools whose output changes between two runs anyway, such as `ps` and `vm_stat`. One tool cannot be run on a Mac without Xcode: `/usr/bin/python3` there is a stub that asks `xcrun` for the real interpreter, and `xcrun` has no x86-64 slice.

HOW IT WAS FOUND: SW_VERS, TWICE

On macOS 27 `sw_vers` is a Swift program, and from 2026-09-21 it crashed under AArchX with a stack overflow, briefly making the CLI score 15 of 16. CoreFoundation links Foundation as an *upward* dependency, the way a library declares the back edge of a dependency cycle, and AArchX left upward links out of its initializer walk entirely. Foundation's code was reachable anyway, because the whole shared cache is mapped, and its classes registered, so nothing looked missing; what never ran was Foundation's initializer, so `NSString` stayed an abstract class cluster and the first `+[NSString stringWithFormat:]` recursed until the 8 MB guest stack ran out. `6f01add3` (2026-09-22) initializes an upward dependency after the image that declares it.

On macOS 26.7.1 the same tool crashed again, in a different place: memory allocation recursing into itself. There libobjc links libswiftCore upward and libswiftCore links Foundation upward, and libobjc is a dependency of CoreFoundation, so following each upward link right after the declaring library initialized Foundation's whole subtree while CoreFoundation was still unfinished. A small experiment with three libraries of the same shape showed Rosetta's own order, `mid core main top`: an upward dependency waits until the entire walk from the program is done. `baf668f3` (2026-10-03) does exactly that, and `tests/dynamic/upward_defer.c` pins the order.

Ollama

Ollama's command-line binary (`Contents/Resources/ollama`, a Go program using cgo) has worked since 2026-09-13. `ollama --version` prints what it prints natively, `ollama serve` answers its HTTP API (`/api/version`, `/api/tags`, `/api/show`), and `llama-server --list-devices` lists the same devices as natively. It took five fixes, each general:

- **AVX2.** Go's runtime turns on its AVX2 code paths under Rosetta without checking CPUID, so AArchX has to implement them whether or not it advertises them (Chapter 4).
- **`sigaltstack` reporting `SS_DISABLE`** when no alternate stack has been set, which Go checks before installing its own.
- **`dlopen` refusing a thin Mach-O of another architecture**, rather than mapping arm64 code it could never run.
- **Bare `@loader_path` and `@executable_path` rpaths**, with nothing after the token.
- **Constructors in programs that do not link CoreFoundation**, which the initializer phase had only been run for.

The Ollama menu-bar application runs too: in a 30-second run it started its own server, served its settings page into its window and shut down cleanly on SIGTERM. Before that, WebKit's allocator stopped it within ten seconds, because `thread_suspend` returned `MACH_SEND_INVALID_DEST`: workqueue threads that AArchX started ended without running the guest's thread-exit path, so the allocator was suspending threads that no longer existed (Chapter 9). Nobody has yet run a model under AArchX.

Applications in cache mode

Application	Result	Measured
Chess	works and plays: board window, and the `sjeng` engine subprocess answers moves	macOS 26.6, 2026-09-11

Application	Result	Measured
Calculator, Dictionary, Font Book, Grapher, Digital Color Meter	main window on screen	macOS 26.6, 2026-09-11
Activity Monitor, Console	window on screen; each logs one optional library missing from the x86-64 cache	macOS 26.6, 2026-09-11
TextEdit, Preview, Script Editor	run, and open a document window when given a file	macOS 26.6, 2026-09-11
Safari	runs and browses	2026-09-21, owner-confirmed
Steam (x86-64 macOS client)	full UI with `-cef-disable-gpu`	2026-09-12
Discord	the Electron client reaches its signed-in view in a 1280 by 720 window, seven processes, as under Rosetta	2026-09-22, owner-confirmed
Brawlhalla	the Adobe AIR game reaches its menus with a Steam client running	2026-09-22, owner-confirmed
Photoshop	stops during startup, later than before	2026-09-22
Photos	aborted on a graphics call; all three causes fixed by 2026-09-22, not run again	2026-09-13

In native mode (Chapter 12) the list is shorter: Image Capture opens its main window and quits cleanly, Stickies opens a note and stays up, and the x86-64 Steam client comes up as described below. In October it grew to older Intel applications: Transmission, Sequel Pro, HandBrake and VLC 2.2 open their windows (`README.md`, October 2026), and 32-bit applications run through m32: Adium, VLC 1.1.12 and Kega Fusion open theirs (Chapter 13).

Safari

Safari had the longest bring-up of any application and is the best map of what a large Cocoa program needs. Its executable is only a launcher that links one umbrella framework, so the dependency-ordered initializer phase was at first skipped for it entirely. Behind that came, in turn: the Objective-C image closure capped at 1024 entries, so a method on `NSBundle` defined in a late image was missing; an initializer in one framework calling `operator new` before libc++abi had been initialized; `mach_vm_map` taking the 8 KB below a request that was 4 KB-aligned but not 16 KB-aligned; a Mach reply landing on top of the shared cache; and JavaScriptCore's Gigacage, a single 128 GB reservation, which is why the guest arena is 256 GB.

The last wall was quieter. On 2026-09-13 Safari started and never showed a window, and the cause was WebKit's allocator failing to suspend a thread and stopping the process. The guest's main thread was missing from AArchX's emulation of `thread_suspend` and `thread_get_state`, and the workqueue threads AArchX starts were leaving without running the guest's thread-exit path. Both were fixed that day; the macOS 27 work continued through `f2ceb4f9` (2026-09-19), and Safari was confirmed browsing on 2026-09-21.

Discord

Discord's renderers died the moment they started. Electron asks `mach_vm_region` about a region it expects to be read-only, and stops deliberately when the answer says otherwise. AArchX answered with the host's view of its own memory, where a whole arena is one read-write-execute reservation. Since `81fa93d2` (2026-09-22) the query is answered from AArchX's own map of the guest in every memory layout, and the loader applies each segment's declared protection, its `initprot`, with `SG_READ_ONLY` honoured for `__DATA_CONST`, so cache mode reports the same three protections for `__TEXT`, `__DATA_CONST` and `__LINKEDIT` that a real system reports. Protecting at load time turned out to be too early, because the Objective-C runtime still writes selector references into `__DATA_CONST`, so images are queued and protected in a batch once Objective-C is done with them.

Three follow-ups made Discord fast rather than merely working. Electron's framework performs about 2,560 weak-coalescing binds for its own C++ names; each one walked every one of roughly 3,600 cache images at 1.8 ms, 4.6 s per process, until `1a650e42` limited the walk to the roughly 300 images that define weak symbols and put a 1 MB Bloom filter in front of it. Alignment faults always hotpatch now (`20e684f4`), which took Discord's first ten seconds from 5,391 retranslations to none. And reserving and releasing a gigabyte, which V8 does at startup, went from 1.8 ms to 0.45 ms when a mapping onto free slots became a single host `mmap` (`8cf3468c`).

In native mode Discord's x86-64 slice stopped binding on 41 imports from eight frameworks that no API database described. With databases for ServiceManagement, UniformTypeIdentifiers, CoreMediaIO, Network, MediaPlayer, StoreKit, QuickLookUI and Accessibility it starts its browser process, its network and utility services and two renderers, shows its splash screen and connects to Discord's gateway (`af333877`, macOS 26.7.1, 2026-10-04). Its main window had not finished loading when that run was stopped at 90 seconds, and its five processes held about 11 GB between them, so native mode has not matched cache mode here. The walls on the way were mostly places where Chromium assumes it owns the process, as with Steam below: a guest thread that asks for a 16 KB stack gets at least 512 KB, because the translator alone needs 40 KB of the host stack under the guest's frames; the four Secure Transport calls that take an `SSLCipherSuite`, 32 bits on x86-64 and 16 on arm64, copy through a buffer of the host's width; `-[NSView sortSubviewsUsingFunction:context:]` takes a C comparator that no type encoding can describe, so a table of selectors with documented function arguments makes those callbacks; and `mach_vm_region` answers from AArchX's own map in native mode too, without which every renderer stopped at an `int3` (the cache-mode version of that fix is above).

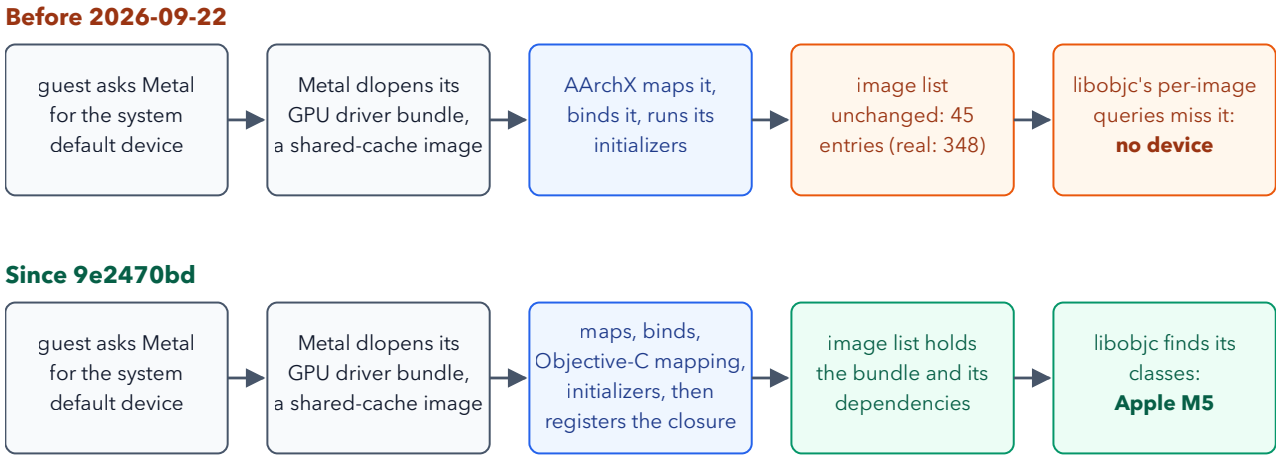
Photoshop and Photos

Photoshop still stops during startup, but finding where produced four general fixes in one commit (`6f01add3`, 2026-09-22). Upward dependencies are initialized (the `sw_vers` story above). A dylib built by an old enough toolchain has no export trie at all, only a classic symbol table, so resolving through the trie found nothing in it; 21 of the 60 dylibs Photoshop ships are of that kind, which is why its OpenCV could not find a single Intel IPP entry point. Such an image now gets an index built once from the symbols its `LC_DYSYMTAB` marks as external. Imports declared with `-undefined dynamic_lookup` are allowed to be missing, as dyld allows; Intel's MKL has 2,161 of them, and a single launch printed fifteen thousand lines of false alarms. And every dyld API entry AArchX does not implement now answers zero in both result registers, because a caller that expected a pair read a table through a register it had never been given. Photoshop now stops later, when an initializer runs while the Objective-C runtime holds its lock and the recursive acquisition aborts, which is re-entrancy and wants its own change.

Photos aborted because `CGLChoosePixelFormat` returned error 10002 for every attribute set. The cause was first blamed on a Rosetta-only GPU service, and it was in AArchX's loader three times over: `_dyld_shared_cache_contains_path` rejected the software renderer's plugin path because it runs through a symlink, `dlsym` on a cache image searched the whole cache instead of that image (both fixed 2026-09-13), and the third cause is the subject of the next section.

The GPU

Until 2026-09-22 a translated program saw no GPU at all. `MTLCreateSystemDefaultDevice` returned nil and `MTLCopyAllDevices` found no device, on an Apple M5 that Rosetta and a native arm64 build both report. OpenGL had only its software renderer left to offer, so `CGLChoosePixelFormat` failed for every accelerated attribute set, the same 10002 that had already been blamed on two other causes. Brawlhalla put a window on screen, ran its ActionScript and never drew into it.



Registering before the Objective-C mapping would be wrong in the other direction: libobjc calls back into the dyld APIs while mapping, and an image already listed then is treated as handled and loses its categories.

Figure 14.1. Why a translated program saw no GPU until 2026-09-22, and the order in which a `dlopen`ed shared-cache image is now registered.

One bug caused all of it. In cache mode the dyld APIs that walk the loaded images, `_dyld_image_count`, `_dyld_get_image_name` and `_dyld_get_image_header`, are answered from the image list AArchX keeps. A `dlopen` of a dylib on disk added to that list; a `dlopen` of an image that lives in the shared cache returned its Mach header and added nothing. The library was loaded, its symbols resolved and `dladdr` knew where they lived, while every walk of the list said it was not there: the count stayed at 45 where a real system goes to 348. The Objective-C runtime keys its per-image queries off that list, so asking it for a bundle's classes returned nothing for a bundle whose classes `objc_getClass` could already find, and Metal, which loads its GPU driver bundle exactly that way, concluded there was no device.

`9e2470bd` (2026-09-22) registers a cache image and its dependency closure at the end of the `dlopen`, after the Objective-C mapping and the initializer phase. The order is the whole fix: registering earlier made libobjc, which calls back into the dyld APIs while it maps an image, treat the new image as already handled, and the framework lost its categories, which are usually what a framework is `dlopen`ed for. `tests/dynamic/dlopen_image_list.c` opens two cache libraries and checks the list before and after each,

comparing the header the list reports against the image a symbol really lives in rather than against the opaque handle. Metal, `CGLQueryRendererInfo`, `CGLChoosePixelFormat` and `CGLCreateContext` now answer as under Rosetta, with `GL_RENDERER` reading `Apple M5` and `GL_VERSION` reading `2.1 Metal - 91.7`.

A smaller GPU-adjacent fix followed the next day. Some x86-64 libraries are in neither the shared cache nor on disk but only in the Rosetta cryptex, and Intel code names them by their system path; Metal opens `/usr/lib/libMTLHud.dylib` when `MTL_HUD_ENABLED=1`. Since `7d918e73` a cache-mode `dlopen` of an absolute path found nowhere else retries it under `/System/Volumes/Preboot/Cryptexes/Rosetta`, so the Metal performance HUD appears under AArchX too.

The GPU since September

The fix above gave cache mode the same Metal and OpenGL that Rosetta has. Three later pieces of GPU work belong to other modes.

- **Native mode and Direct3D through Wine.** On 2026-10-06 (`0aae84f0`) MacNdCheese's D3DMetal backend, Apple's Game Porting Toolkit 4, ran under native mode: a Direct3D 11 device, a swap chain and presents through the toolkit's `d3d11` and `dxgi`, and R.E.P.O. to its main menu on it. The commit records that this was worked out black-box, from what AArchX's loader and bridge report and from return values. It needed an API database for CoreAnalytics, the private framework the backend links, built from the SDK's text stub; `+[NSBundle bundleWithIdentifier:]` to ask the frameworks AArchX's own loader mapped through their `Info.plist` when the host's dyld knows no such bundle, because the backend finds its shader library through its own bundle; `_dyld_get_image_uuid` answering from the image's own `LC_UUID`; and Metal's two undeclared texture-swizzle conversion functions crossing as scalars, which is how all 1,296 swizzles round-trip on arm64 and under Rosetta. The same week (`22221270`) declared `MTLSetShaderCachePath` and `MTLGetShaderCachePath` as DXMT uses them, and R.E.P.O. reached its main menu on DXMT. Chapter 15 has the Wine side.
- **32-bit OpenGL.** m32 runs 32-bit games on the Mac's own OpenGL (Chapter 13). Two October fixes show what a real game asks of it. A map used to copy the whole buffer into its guest shadow and ask `glGetBufferParameteriv` for the size, which on Apple's multithreaded GL engine waits for every queued command first; that was about half of Left 4 Dead 2's render thread in large fights, where Valve's GL layer refills its dynamic buffers several times a frame. A shadow is now the buffer's mirror once it has been mapped through (`e7a180f2`, 2026-10-13), so a map copies nothing in and asks the driver nothing, and only array and element-array buffers are mirrored because writers m32 cannot see, such as pixel packs, cannot be followed. And `-[NSOpenGLContext setView:]` and `-update` trap off the main thread in today's AppKit, as they do for native arm64 code, though the Intel Macs these games ran on allowed them and SDL 2 makes both from its render thread; m32 runs those two calls on the main thread (`6c375f24`, 2026-10-12).
- **A frame counter for them.** `OCERZ_M32_HUD` draws an overlay on 32-bit games. It was loaded when a game bound `CGLFlushDrawable`, which Left 4 Dead 2 never imports, since it presents with `-[NSOpenGLContext flushBuffer]`; with the variable set m32 now loads AppKit and the overlay at start (`1f2e3035`, 2026-10-13).

The macOS Steam client

The x86-64 macOS Steam client is the largest macOS program AArchX runs, and it is a family of programs rather than one. It came up with its full UI on 2026-09-12 in cache mode: the bootstrapper, `ipcserver`, the client, and the CEF web helper with its GPU, utility and renderer processes, all translated, with the Steam window drawn through SwiftShader, Chromium's software renderer. After its first self-update the client lives in Application Support:

```
./ocerz "$HOME/Library/Application Support/Steam/Steam.AppBundle/Steam/Contents/MacOS/
steam_osx" -cef-disable-gpu
```

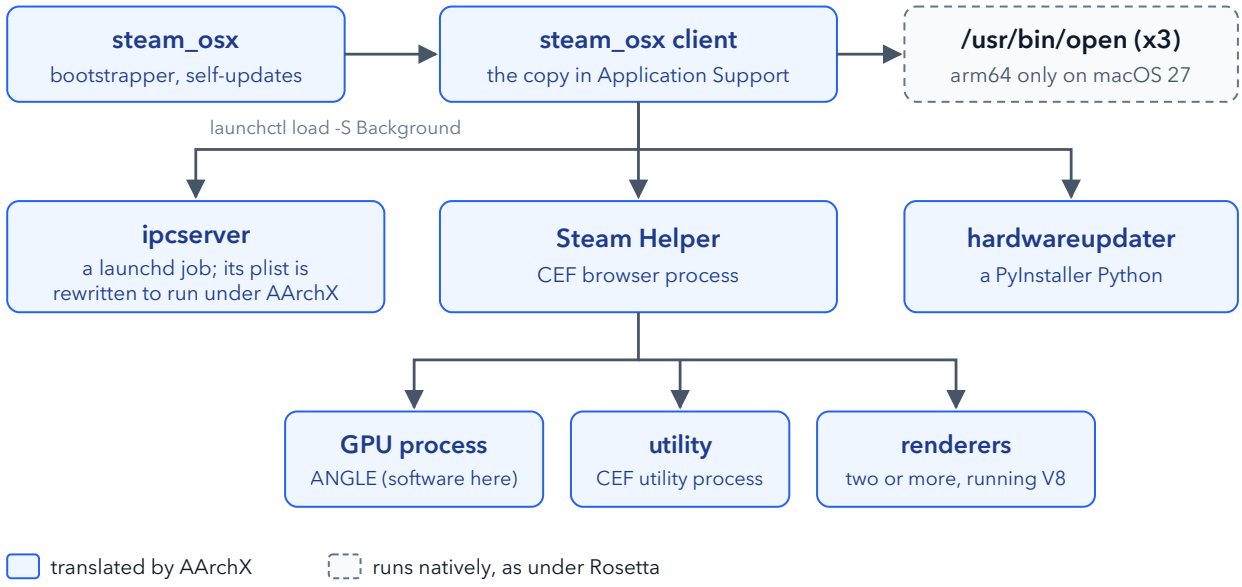


Figure 14.2. The macOS Steam client's processes under AArchX in cache mode. Every Intel process in the tree is translated; the arm64-only `/usr/bin/open` children on macOS 27 run as themselves.

What it took, in order

Each step below was found because the previous one let Steam get further. Read in order, they are a fair cross-section of what a large real program asks of a translator that a benchmark never does.

#	Date	Symptom	Fix
1	09-11	bootstrapper stops before updating	System V semaphores, which Steam's tier0 threading code uses (`3e3756a6` , with the rest of SysV IPC); with them it runs and self-updates
2	09-12	an assertion on a path beginning `/../Steam.AppBundle`	the guest gets an absolute executable path, as the kernel gives one; Steam derives its bundle root from it
3	09-12	duplicate Objective-C classes	`dlopen` of the executable's own path returns the image already loaded instead of loading a second copy
4	09-12	a crash at address `0x28` after 38 s	`dlopen` initializers run in dependency order (`def0fce4`); see the callout
5	09-12	Chromium's allocator cannot reserve its region	a guest arena big enough for PartitionAlloc's 32 GB reservation, later 256 GB for JavaScriptCore

#	Date	Symptom	Fix
6	09-12	`ipcserver init failed`	`ipcserver` run under AArchX: the launchd plist Steam writes is rewritten on the way through, and the job's label re-enabled first
7	09-12	whole-process stalls during translation	the JIT lock is taken without ever parking, so a lost wakeup cannot wedge translation
8	09-12	the web helper never reports ready	a page refused translation for churning is retried after 4,096 refusals as well as after 1.5 quiet seconds
9	09-12	a fixed translation thrown away again	a fault from an already retired translation just recovers instead of invalidating its page again
10	09-12	`hardwareupdater` crashes	upward links in `dylib_use_command` form stop counting as initializer-order edges
11	09-12	sandboxed helpers load the 386 MB CEF framework twice	loaded images are matched by device and inode, as dyld does, because `realpath` fails inside the sandbox
12	09-13	renderers die on a libc++ comparator assertion	a self-loop compare whose loop head reads the flags is no longer fused (`tests/dynamic/cef_partition.s`)
13	09-13	macOS asks for the login password once per helper	`Steam Helper` runs with `--use-mock-keychain`
14	09-23	three children die per launch on macOS 27	arm64-only children run natively

HOW IT WAS FOUND: THE CRASH AT 0X28

Step 4 is the clearest example of an initialization-order bug. `steamui.dylib` depends on `libtier0_s.dylib` , Valve's low-level runtime library, which owns the global allocator, but AArchX sorted a `dlopen` 's initializer closure by load address, and `steamui` happened to be mapped below `tier0` . Its first container growth therefore called through Valve's global allocator object before tier0's initializer had set the object's vtable pointer: a call through a null vtable at offset `0x28` , a fault at exactly that address. The fix collects the closure in dependency post-order and never sorts it. It was found independently by two agents on two machines the same day, line for line identical, and with it Steam ran its full 100-second test window instead of dying at 38 seconds, its peak memory going from 0.64 GB to 2.62 GB as it got further.

Two of those steps deserve more detail. **`ipcserver`** is started by `steam_osx` with `launchctl load -S Background` on a plist it writes into Application Support, which would make launchd run it natively, outside the translator. AArchX rewrites that plist as it passes through, putting itself in front of the program in `ProgramArguments` and in `Program` (Steam's own plist only has `Program` , so the array is built from it), and enables the job's label first: a legacy `launchctl unload` leaves the label disabled, and every later load then fails with an I/O error while the client reports `ipcserver init failed` . The Mach service lookup itself was never the problem, since Mach traps go straight to the host kernel.

The mock keychain is a trade-off rather than a fix. Chromium reads its "Steam Safe Storage" item from the login keychain at startup; that item does not trust the `ocerz` binary, so macOS asked for the password once per helper process, and each helper blocked until the prompt was answered. With `--use-mock-keychain` , which AArchX adds whether `Steam Helper` is launched by Steam or directly, cookies CEF stores under AArchX are encrypted with the mock key, so they are not readable by a native Steam, nor the reverse. `OCERZ_NO_MOCK_KEYCHAIN=1` turns it off.

Steam in native mode

On 2026-09-21, on macOS 27, the same client came up in native mode (`./ocerz -native ...steam_osx`), where the guest's system libraries are synthesized stubs into the Mac's own arm64 frameworks (Chapter 12). It reached the login window, then the 1280 by 800 Steam window with its menus built and the store view loading, with the GPU process running **hardware ANGLE over the host's own OpenGL** and two renderers, stable for minutes with nothing reported. From the web helper starting to the main window took 23 s in native mode and 34 s in cache mode on the same machine; the arm64 Steam takes 2 to 4 s. The work that got it there (`b0aa6e70`) is a list of places where Chromium assumes it owns the process:

- x86 thunks for native Objective-C implementations handed to the guest, because Chromium swizzles `allocWithZone:` and then calls the original through a pointer;
- a guest-owned `errno` slot and guest-owned thread-specific-data keys, because AArchX's own code changes the host's between any two guest instructions, and CEF makes eighty inline `gs`-relative reads;
- views of the host's malloc zones, so that Chromium writing x86 hooks over the default zone cannot send the next native `calloc` into x86 code;
- a page size of 4,096 everywhere, which V8 checks against PartitionAlloc's compile-time constant, and the guest's own stack bounds for V8's stack scanner;
- bridged `thread_suspend` and `thread_resume` that refuse to leave a thread stopped while it holds the JIT lock or the map lock or is anywhere inside the host's allocator.

`hardwareupdater` still fails to run its script in native mode, and several imports Steam has not reached yet are stubs. Neither run is part of a gate.

Games

Launching a game from Steam

Steam runs a game through a tool when the game's launch options are set to `/path/to/ocerz %command%`. It then passes the game's `.app` bundle as the argument and puts its loader and overlay libraries in `DYLD_INSERT_LIBRARIES`. Three things stood in the way, all fixed in `cd1f9c1c` (2026-09-23):



Figure 14.3. What happens when Steam launches a game through `ocerz %command%`.

- **A bundle as the program.** Given a bundle, AArchX runs the executable the bundle's `Info.plist` names, with that path as the guest's `argv[0]`.

- **`DYLD_INSERT_LIBRARIES` in cache mode.** The libraries are opened inside the guest before its entry point runs. The catch is that the host's own dyld reads the same variable when it starts `ocerz`, and would load the arm64 slices of those libraries into the translator itself. So `ocerz`, started with the variable set, moves it aside, re-executes itself without it, and restores it for the guest.
- **Libraries in the pre-10.6 format.** A dylib built for Mac OS X 10.5 has local and external relocations and an indirect symbol table instead of compressed fixups, and AArchX applied no fixups to it at all, so Steam's `steamloader.dylib` called address zero at once. Those relocations are now applied (dynamic test `dlegacy_format`, whose library is itself built for 10.5). The same library links `libgcc_s.1.dylib`, which the shared cache knows only as an alias in its dylibs trie; 108 of the SDK's 111 aliased `/usr/lib` names failed to load until `655a1cac` made every path lookup fall back to that trie.

Steam's own frame counter in its overlay relies on dyld interposing, which AArchX does not do, so `OCERZ_FPS=1` (`5f1d5530`) puts four instructions at the head of the translated block that begins the Intel `CGLFlushDrawable` export, counting calls, and a thread prints frames per second once a second. The guest sees nothing of it.

Brawlhalla

Brawlhalla, an Adobe AIR game with an x86-64 macOS build, is the first game in the record. It needed two things that were new on 2026-09-22: the GPU fix above, without which it ran its ActionScript into a window it never drew, and System V `shmat` and `shmdt` (`b7e28bed`), which were not in the syscall table at all. Steam's client talks to a running game through a System V shared-memory segment, so the game asked for one, got `ENOSYS`, carried on with a null pointer and segfaulted at address zero. The host kernel refuses a fixed address for `shmat` even over address space the arena has reserved, so the segment is attached wherever the host wants it and `mach_vm_remap` carries that mapping into guest space, sharing the pages rather than copying them (Chapter 8).

Measured the same day against Rosetta, with one Steam client serving both: **the first frame took 5.3 s against Rosetta's 0.54 s**, and the game's main thread saturated a full core where Rosetta used a quarter to half of one. Three JIT changes that evening are a good example of how a real program finds costs a benchmark suite does not (Chapter 6 has the mechanisms):

Change	What was wrong	Measured effect
A return that misses the host return-address stack stays in the body (`ad681d96`)	Every trip out of translated code resets the host stack and so empties the return-address stack. Engine code calls every ten instructions and trips out every few hundred thousand blocks, so the stack was permanently empty: 3.2 billion misses in 88 s, each one leaving through the epilogue.	2.6-2.9 to 3.85 billion guest instructions a second

Change	What was wrong	Measured effect
Veneer pools (`ace6eee0`)	A chain is a `b` patched into the exiting block, and `b` reaches 128 MB. The arena fills front to back, so script code translated late could not chain to engine helpers translated early; 4.5-5.8% of chains were silently dropped and a quarter of the main thread's time was in prologues and epilogues. A 256 KB pool every 64 MB of arena now holds indirect jumps that reach anywhere.	71% of a core, 60% more blocks a second; every chain activates
Constant addresses resolved at translation time (`3c9fe1b2`)	A global load or store paid a commpage range check and, in ordered mode, an alignment test at run time, for an address that is a constant of the translation.	14-18 host words become 4; 13% more blocks a second

Taken together the main thread went from a saturated core to about three quarters of one while doing about 75% more work per second, and at the menu it now spends most of its time waiting on the display, as it does under Rosetta. `OCERZ_TRIPSTAT=1`, added on 2026-09-24 to see what remains, measured 6.6 million exits to the dispatcher a second at the menu, ten destinations making 92% of them, which is where the 14% of the main thread still spent outside translated code goes.

32-bit Mac games from Steam

Steam for Mac still lists 32-bit games, warns that the current macOS cannot run them, installs them and lets the player press Play. Since October they play. Chapter 13 explains how m32 runs the program; this section is only about the Steam end of it. Played from Steam with `ocerz -native %command%` as the launch option, three games are recorded in `docs/32-bit-steam-games.md`:

Game	Result as recorded	Documented
Batman: Arkham Asylum Game of the Year Edition (Feral, 2014)	plays, about 60 frames a second on an M4 Max; its options window opens black and the game starts after it	2026-10-11 (`e0577d07`)
Call of Duty: Modern Warfare 3 (Aspyr, 2011), campaign	launcher, intro movies with sound and the campaign play; multiplayer, a separate Steam app, was not tried	2026-10-12 (`a322c200`)
Left 4 Dead 2 (Valve)	campaigns play with sound and music, 80 to 120 frames a second at the default settings on an M4 Max, fewer in the largest fights	2026-10-13 (`d20ee9a7`)

All three sign in to Steam. The reason needs the bridge, which is the part specific to Steam. Valve has shipped no i386 `steamclient` since 2019. A 32-bit game's own `libsteam_api` asks the running Steam where its client lives, which Steam answers only to processes it launched (a game started from a terminal cannot find it, and asks Steam to relaunch it), opens `steamclient.dylib`, finds no i386 slice and quits. When an i386 program opens such a file, m32 opens the host's own copy, its arm64 slice, and answers the client's exports itself (`825fe3a8`, 2026-10-11). `CreateInterface`, and every method that the interface database flags as returning an interface, return a proxy: a guest object whose first word is a vtable of trap

addresses, aliased to the host object, so that crossing a call turns the proxy `this` into the real one. Each slot crosses by the notations that `make steam32` writes from Valve's Steamworks headers, which are fetched at a pinned version and never committed; a returned C string is copied into guest memory once per host string, and the callback queue's messages are converted. Nothing of Valve's is shipped, and no Steam emulator is involved: the game talks to the user's own Steam.

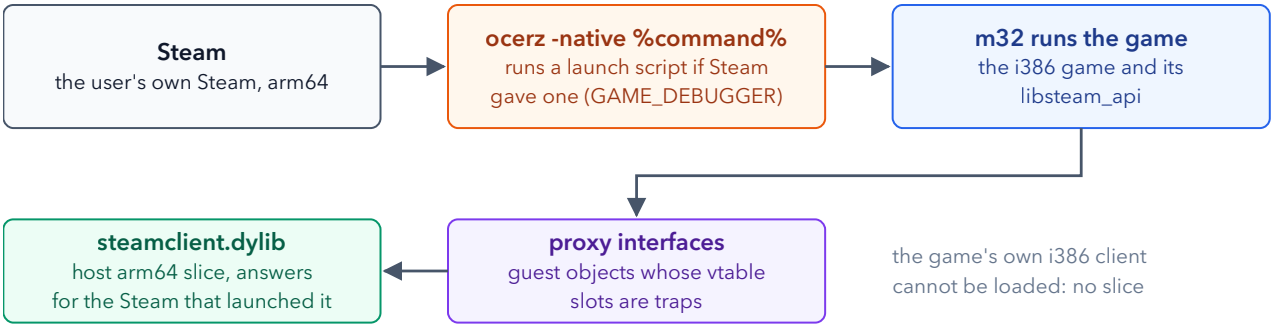


Figure 14.4. A 32-bit Mac game started from Steam. The game's own Steam library is i386; its calls cross into the arm64 client of the Steam that launched it.

Four details came from the three games. A Steam object handed out under two version strings gets one proxy per version, since keying proxies on the host object alone gave a second caller the first version's vtable (`68cda65c`, 2026-10-12). Methods that return a record by value cross (`80d2da59`, 2026-10-13), string results are copied safely from several game threads at once (`05c04778`), and `OCERZ_M32LOG=steam` names the SDK a game's client interface comes from and each callback it receives. The documentation's advice for a game that quits at once is to read the log: a missing class or symbol means the framework databases are absent (`make apis32`), and "no Steam interface database" means `make steam32` has not been run. And Left 4 Dead 2 had a problem that only a real window system shows. Its launcher brings itself to the front before its `NSApplication` exists, and today's cooperative activation ignores that while another application is in front, so the game ran unfocused, throttled by its own no-focus sleep, with no mouse until the player switched away and back. For the current process `SetFrontProcess` now also sends the application an activation request once its run loop turns (`0dd3cb74`, 2026-10-13); a timedemo started from a shell went from 32-41 to 75-77 frames a second.

What is not working is listed in the documentation and repeated here as recorded on 2026-10-13: Left 4 Dead 2's menu background movies need QuickTime, which macOS no longer has; typing in its developer console doubles each letter; "Exit to main menu" crashes, so the game has to be quit from its menu; and if it must rebuild its sound cache it stays on the loading poster until it is started again. Achievements, statistics and cloud saves have not been verified through the bridge for Batman or Modern Warfare 3, and the Steam overlay is not expected to appear. The prebuilt release has no Steam interface database and no 32-bit GNU C++ runtime, so these games still need a build from source (`docs/32-bit-steam-games.md`).

WINDOWS GAMES

Counter-Strike 2 and R.E.P.O. are Windows games and run through Wine; they are covered with the rest of the Wine work in Chapter 15, including how R.E.P.O. runs under native mode.

The AArchX app

Everything above is run from a terminal. On 2026-10-13 (`2dce5f07`) AArchX.app arrived for people who would rather not use one, and within the day it grew from a drop target into a library. It is a window around `ocerz`, not a second implementation: the documentation says that `ocerz` works the same with or without it and that the app only finds and starts it. It is one Objective-C file, `app/AArchX.m` (1,636 lines), with `app/icon.m` (70 lines) that draws the icon, an AX in the system font's black compressed cut on a near-black tile, when `make app` runs; no image is checked in. There is no Xcode project, and `make app` signs the bundle ad hoc, which seals it.

The library

The app keeps a library of the user's programs, shown as a grid of tiles with each program's own icon and name and one word for what it is: Intel, 32-bit or Windows, or that it is running. A program is added by dropping it on the window (one program dropped is added and run, several are only added), by +, by Open With in Finder, or by dropping it on the Dock icon. Double-clicking a tile runs it; its menu runs it in native mode, stops it, opens its last log, shows it in Finder or removes it. An Activity tab lists every run with how it ended, and the logs are in `~/Library/Logs/AArchX`. Each run is `ocerz [-native] -- <path>`. Open With lists the app for app bundles and Unix executables at the Alternate rank, so it never becomes anyone's default.

The app chooses the mode from what the file carries, reading only its header (`macho_arches` in the source). A 32-bit program goes to native mode by itself, because only m32 runs it, and so does everything when Rosetta's x86-64 cache is missing, because cache mode needs it (Chapter 11). An arm64-only program is turned away with a note that it runs by itself. `ocerz` is looked for inside the bundle, beside it, in `~/.aarchx` and in Homebrew's `bin`, or at the `OcerzPath` default of the app's preferences. The release also ships `AArchX.zip`, a self-contained copy with `ocerz` and its runtime inside; because it is not notarized, macOS refuses to open it the first time, and the user clears that in System Settings under Privacy and Security.

Windows programs

A Windows `.exe` or `.msi` runs as `ocerz <wine> <program>` with a Wine build the app finds on the Mac: WineHQ's Stable, Staging and Devel apps (MacNdCheese's copies included), Whisky's Wine, the Game Porting Toolkit, or a `wine` in Homebrew's `bin`. It runs in cache mode, in the program's own folder, in a prefix of the app's own, `~/Library/Application Support/AArchX/Wine`, so that no other Wine setup's prefix is touched. An `.msi` is installed with `msiexec /i`. Stop also stops that prefix's `wineserver`, which ends every Windows program running in it. Chapter 15 is about what Wine asks of the translator.

CrossOver bottles

When CrossOver is installed, a sheet lists its bottles with a box each, and ticking a bottle runs its Windows programs under AArchX instead of Rosetta. Nothing inside CrossOver.app is changed. Ticking does two things, and unticking undoes both (`a246dc47` and `e3a26326`, 2026-10-13).

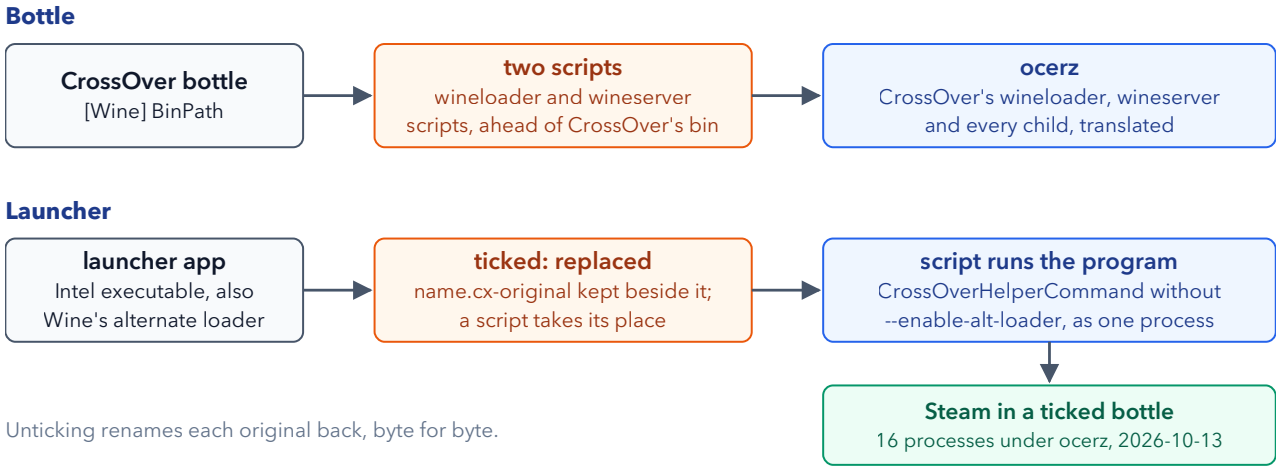


Figure 14.5. What ticking a CrossOver bottle changes. The bottle's `BinPath` sends Wine's loader and server through `ocerz`; each launcher is replaced by a script so that its program's window processes are not left to run under Rosetta.

- **The bottle.** The box sets the bottle's `[Wine] BinPath`, CrossOver's documented way to use another Wine, to two small scripts the app writes into `~/Library/Application Support/AArchX/CrossOver/bin`, ahead of CrossOver's own `bin`. They run CrossOver's `wineloader` and `wineserver` under `ocerz`, or as they are if `ocerz` has gone.
- **The launchers.** That alone was not enough, and the first version showed it: Steam's window still ran under Rosetta. CrossOver writes a launcher app for each program into `~/Applications/CrossOver`, and the launcher is also Wine's alternate loader (`--enable-alt-loader macdrv`), so the program's window processes run as copies of the launcher's own Intel executable. Under `ocerz` those processes do not run yet. So ticking a bottle keeps each of its launchers' executables beside the launcher as `<name>.cx-original` and puts a script in its place. The script runs the command CrossOver wrote for the program, its `CrossOverHelperCommand`, without the `--enable-alt-loader` the launcher would add, and becomes it, so the launcher stays running while the program does. Steam then starts with all 16 of its processes under `ocerz`. Given arguments, which means CrossOver is asking for the launcher itself, the script runs the original.

Unticking renames the originals back, byte for byte, and a launcher CrossOver has written again since is left alone. Two costs are documented: a ticked bottle's programs show Wine's own Dock icon instead of the launcher's, and links such as `steam://` from a browser do not reach the program through its launcher. If CrossOver rewrites a launcher, it is back on Rosetta until the sheet is opened again, which switches it again; programs of a bottle should be quit before it is switched, because the change applies when the bottle next starts.

One more change made this work. CrossOver starts each process through a link named after the Windows program, and for a hard link `realpath` keeps that name, so `ocerz`, which marked a process as Wine's only when its loader was named `wine` or `wine64`, started the bottle's `cmd.exe` in the ordinary memory layout, and it died reserving the low-shadow base its Wine parent had handed down. Since `74e6140d` (2026-10-13) Wine's loader is any program with a `WINE_RESERVE` segment, the range its preloader keeps for Windows, whatever it is called; `tests/dynamic/wine_segment.c` pins it and `OCERZ_NO_WINE_SEGMENT_DETECT=1` goes back to the name alone (Chapter 15).

Getting it

The same week the project gained a prebuilt release: a one-line installer, `docs/install.sh`, that downloads the latest release archive, checks its SHA-256 and unpacks it into `~/.aarchx` (`ocerz`, the API databases for both modes and the guest C++ runtime), and a Homebrew cask in the `mont127/aarchx` tap. The installer moves AArchX.app into `~/Applications` and registers it with Launch Services so that Open With lists it (`b1c999d4`, 2026-10-13); `AARCHX_NO_APP=1` leaves it out and `--uninstall` removes only a copy that is ours. The cask puts the app in `/Applications`.

What is still open

- **Steam's GPU process in cache mode.** CEF's GPU process initializes ANGLE through CGL, and every accelerated pixel format failed until the loader fixes of 2026-09-13 and 2026-09-22, so cache-mode Steam runs with `-cef-disable-gpu`. It has not been retried with acceleration since; native mode already runs the GPU process on hardware ANGLE.
- **Launch stalls.** Some launches stall before the web helper starts its child processes, with the JIT lock held inside translation; relaunching gets past it.
- **A Chromium initializer that traps.** On 2026-09-12, on both test machines and at the same address, one static initializer in the Chromium Embedded Framework hit an `int3` after about 2.3 million instructions. AArchX runs initializers reached through `dlopen` in a tolerant mode, so Steam carried on; which Chromium check fires has not been identified.
- **Startup time.** Large applications start several times slower than under Rosetta, because everything they run at startup is translated the first time it runs. The translation cache (Chapter 7) helps on later launches, but does not close the gap. Translation itself has got cheaper: on an M2 Max on 2026-10-13, a cold start of the system's `perl` loading ten modules, with the cache off, took about 275 ms in the morning and 208 ms after one barrier pair per chain install (`f4ef8900`); Ruby loading its standard library went from 185 to 168 ms.
- **32-bit games.** The Steam bridge answers 0 for a method it cannot cross, mostly one that takes a function pointer. Only three games are recorded, and the known faults of each are listed above.
- **CrossOver.** The alternate loader's window processes do not run under `ocerz` yet, which is why the launchers are replaced.
- **Discord in native mode** reaches its splash screen and gateway but had not shown its main window when last run.
- **Photoshop** stops on Objective-C runtime-lock re-entrancy during initialization; **Photos** has not been rerun since its causes were fixed; nobody has run a model under **Ollama**.

CHAPTER 15

Wine and WoW64

Wine is the hardest guest AArchX runs and the one that has taught it the most. A Wine session is a dozen or more cooperating processes, each mixing Unix code, Windows PE code and Objective-C, with its own contract for the segment registers, its own exception model layered on Unix signals, and a 32-bit Windows world inside a 64-bit process. This chapter explains what Wine asks of a translator, how AArchX answers, and the long series of walls between the first Notepad window in July and R.E.P.O. rendering at roughly 45-65 frames a second in October. Since 2026-10-06 it has also meant Wine under native mode, and the October JIT work has been driven by 7-Zip running under Wine; the chapter covers both, then CrossOver and the signal rules that Wine's thread suspension depends on.

~15 s
WINEBOOT ON THE FIXED ARENA BASE

33 s
WINDOWS STEAM TO ITS MAIN WINDOW (ROSETTA 11-12 S)

19
EMULATED PROCESSES IN ONE STEAM SESSION

45-65 fps
R.E.P.O.'S MAIN MENU, 2026-10-04 (ROSETTA ~178)

Why Wine is the hardest guest

Wine is not one program. A Wine process on macOS is a Unix-side loader and `ntdll.so`, which talk to the host kernel and to the rest of the session; a set of Windows PE modules, `ntdll.dll`, `kernel32.dll` and the application's own DLLs, running as ordinary x86 code; and a display driver, `winemac.drv`, written in Objective-C against AppKit and Metal. Every process of a session talks over sockets to `wineserver`, which keeps the Windows kernel objects, and a prefix starts its own service processes at boot. All of it is x86-64 code, and under AArchX all of it is translated, `wineserver` included.

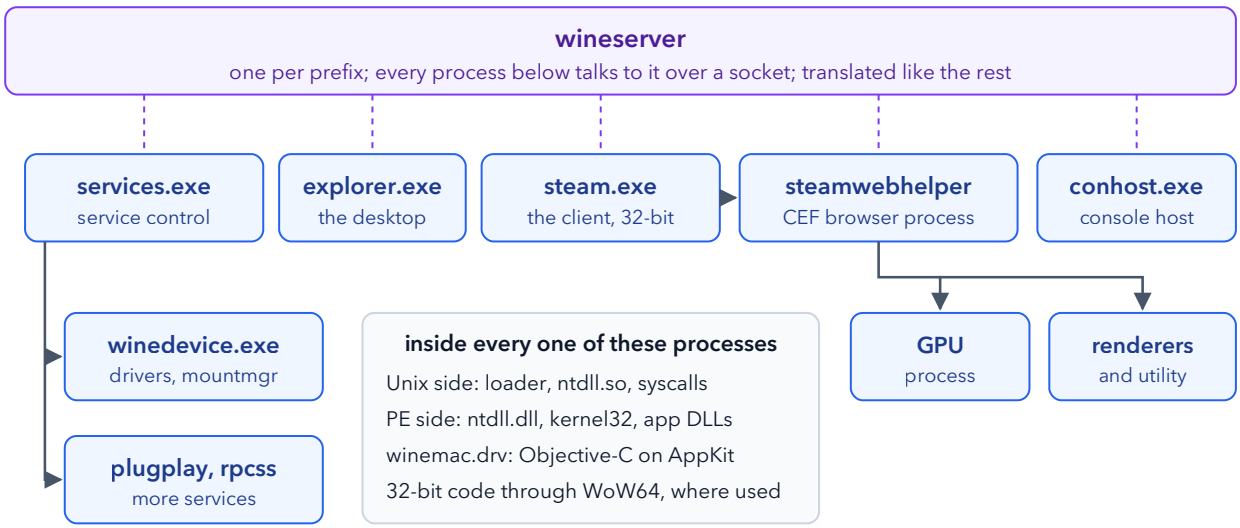


Figure 15.1. A Windows Steam session under Wine, simplified. On 2026-09-05 such a session was nineteen emulated processes.

That structure stresses a translator in ways a single program never does:

- **Two ABIs and two segment-register contracts in one thread.** The same thread runs Unix code, where `gs` points at the Darwin thread-specific data (TSD), and Windows code, where `gs` points at the Windows thread environment block (TEB), and switches between them on every system call.
- **Windows exceptions built on Unix signals.** A Windows access violation is a host `SIGSEGV` that Wine's handler turns into a Windows exception, walks Windows unwind data for, and resumes. The translator's signal frames, register recovery and `sigreturn` all have to be exactly what Wine expects.
- **Low addresses.** Windows programs, and the whole 32-bit WoW64 world, live below 4 GB, where a 64-bit macOS process has nothing mapped and cannot map its own code. AArchX gives Wine processes a separate memory layout for that, described below.
- **Many processes that share memory.** The processes of a prefix share files, shared-memory sections, a page of kernel data and, through `wineserver`, each other's address spaces, which `wineserver` reads and writes with Mach calls aimed at another task.
- **Cocoa and Metal underneath.** `winemac.drv` brings in AppKit, Metal and their XPC services, so every Wine GUI process is also a Cocoa application.

That is why Wine has been a better test than any benchmark: each of its parts reaches a different corner of the emulator, and the bugs it found were almost all general.

Running it

```
WINE="/path/to/Wine Devel.app/Contents/Resources/wine"
export WINEARCH=wow64
export WINEPREFIX="$HOME/.wine-ocerz"

./ocerz "$WINE/bin/wine" wineboot -u
./ocerz "$WINE/bin/wine" notepad
```

Three Wine builds have been run: Wine Devel 11.8, the build the README describes; MacNdCheese's unified Wine, which Windows Steam and the games are measured with; and, since 2026-10-01, stock Wine Staging 11.16 with DXMT. Two more have been used since: Wine Staging 11.18, for the 7-Zip measurements of 2026-10-14, and CrossOver's own Wine, through AArchX.app (Chapter 14). Children inherit everything they need, the translator included: a Wine process that starts another goes through AArchX's own `exec` and `posix_spawn`, so the whole tree stays translated.

> **ALWAYS UNDER A WATCHDOG**
>
> On 2026-08-17 an unwatched `wineboot` under AArchX went into Wine's service respawn loop, seventeen `explorer.exe` processes at 4 to 8 GB each, took the machine past 100 GB and killed WindowServer. Since then every Wine session is run under `tests/safe_run.py`, which puts the command in its own process group, checks the group's process count, physical footprint and wall clock every 0.3 s, and kills the group, plus anything that daemonized out of it, when a limit is crossed. The footprint, not RSS, is what is measured, because RSS counts the resident shared-cache pages again in every process.

The segment-register contract

Wine on macOS uses `gs` for two things. While Unix code runs, the `gs` base is the Darwin TSD, where the C library keeps `errno` and pthread data. While Windows code runs, it is the TEB, where Windows code expects `gs:[0x30]` to be the TEB's own address and `gs:[0x58]` to be the thread's TLS array. Wine switches between the two at each of seven transitions in its code, each through the `_thread_set_tsd_base` machine-dependent call, and it saves the pthread base in the TEB at offset `0x320` so that its assembly can find its way back. Its own code caches the TEB in `r13` before switching.

In AArchX the `gs` base is ordinary CPU state, a field of the guest CPU structure that every `gs`-relative access adds in (Chapter 6), and `_thread_set_tsd_base` sets it. The difficulty is everything else that touches it:

- **Signal return.** When AArchX builds a signal frame it stashes the interrupted `fs` and `gs` bases past the end of the Darwin `ucontext`. Measured on the real kernel, a base that a handler sets survives `sigreturn` when the task has no LDT. Wine relies on exactly that: its handler installs the TEB as the `gs` base before returning to Windows code. Restoring `gs` from the stash undid that, and Windows code then ran on the pthread base until something read `gs:[0x30]`. The restore was removed in July (`b634d124`), came back with a broader change in August (`467542b9`), and since `e5679eb5` (2026-09-26) only `fs` is restored.
- **The TLS array.** Wine's macOS port copies the TEB fields Windows code reads into reserved TSD slots when a thread starts, so `gs:[0x58]` under the TSD base gives the TLS array. But the TLS array is the one field that changes after that copy is taken, and a thread whose copy preceded its TLS setup read zero, which crashed MSVC `__declspec(thread)` code. TSD slot 6, at `gs:[0x30]`, holds the TEB's own address on Wine threads and zero on every other thread, so AArchX special-cases the absolute `gs:[0x58]` form: when slot 6 is non-zero it reads the live field through the TEB instead of the stale copy. The test is self-selecting, so nothing else is affected, and both the interpreter and the JIT implement it.
- **Machine-dependent calls.** The Unix-class system calls clear `rdx` on return under Rosetta, and AArchX does the same. Machine-dependent calls such as `_thread_set_tsd_base` leave `rdx` and the flags alone. Applying the Unix rule to them crashed every Wine process at its first return to Windows code, because Wine keeps its frame pointer in `rdx` across that call (fixed 2026-09-04, test `machdep_rdx`).

- **Thread identity.** `thread_info` with `THREAD_IDENTIFIER_INFO` reports a thread's TSD address. Under Wine Staging, whose `gs` moves between TSD and TEB on every system call, AArchX answered with the host TSD of whichever host thread ran the guest thread, and on a host stack above 12 GB that leaked AArchX's own TSD into Wine, which then lost its TEB. Since `2e5f0d20` (2026-10-01) the answer is mapped back to the owning guest thread's TSD.

WoW64 and 32-bit code

Modern Wine runs 32-bit Windows programs the way 64-bit Windows does, through WoW64: the process is 64-bit, the 64-bit `ntdll` and `wow64cpu` stay in charge, and 32-bit code runs in a 32-bit code segment inside it. Wine defines that segment in the process's local descriptor table, and `wow64cpu` reaches 32-bit code with a far transfer through its selector. Rosetta runs 32-bit code inside WoW64 too, so this is parity rather than novelty; standalone 32-bit Mach-O programs are not supported by current macOS at all, and run through m32 instead (Chapter 13).

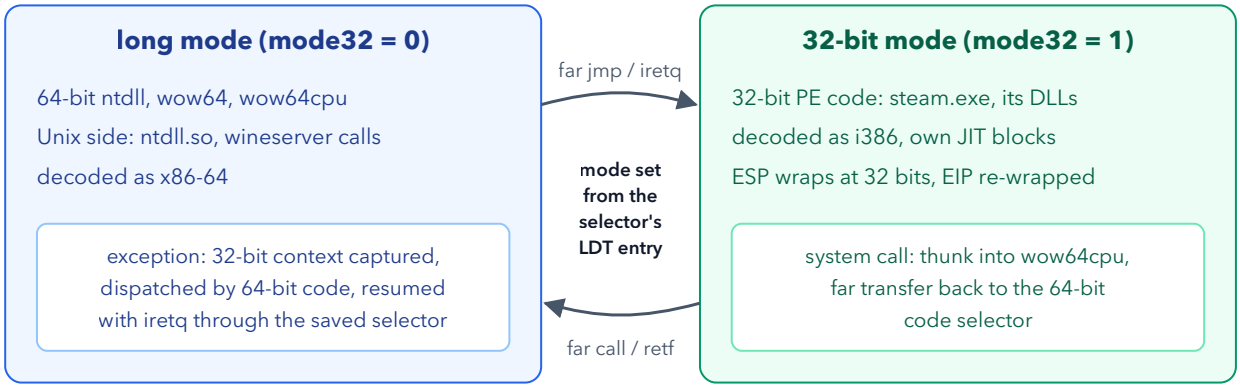


Figure 15.2. How a WoW64 process moves between long mode and 32-bit mode under AArchX. Every far transfer funnels through one helper that sets the selector, the mode and the instruction pointer together.

AArchX keeps its own copy of the LDT entries Wine installs and decides the CPU's mode from the descriptor a far transfer loads: a selector whose descriptor is a 64-bit code segment means long mode, one whose descriptor has the 32-bit default size means 32-bit mode. Every far transfer, `jmp far`, `call far`, `retf` and `iret`, goes through a single helper in the interpreter, so the selector, the mode and the instruction pointer can never disagree, and the decoder is told the mode in exactly one place. The JIT keys its blocks on the mode they were translated for, so the same bytes can have a 32-bit and a 64-bit translation. The i386 support itself, the decoder's 32-bit operand and address rules, the interpreter's BCD and segment instructions, the JIT's 32-bit forms, is covered in Chapters 4 to 6, and its differential gate in Chapter 16. That gate had 20,033 generated cases in the first edition of this book; on 2026-10-16 it was 40,044 sequences, run in each memory layout since `0d296dbd` (2026-10-07), so the Wine layout's 32-bit code is checked against the same cases.

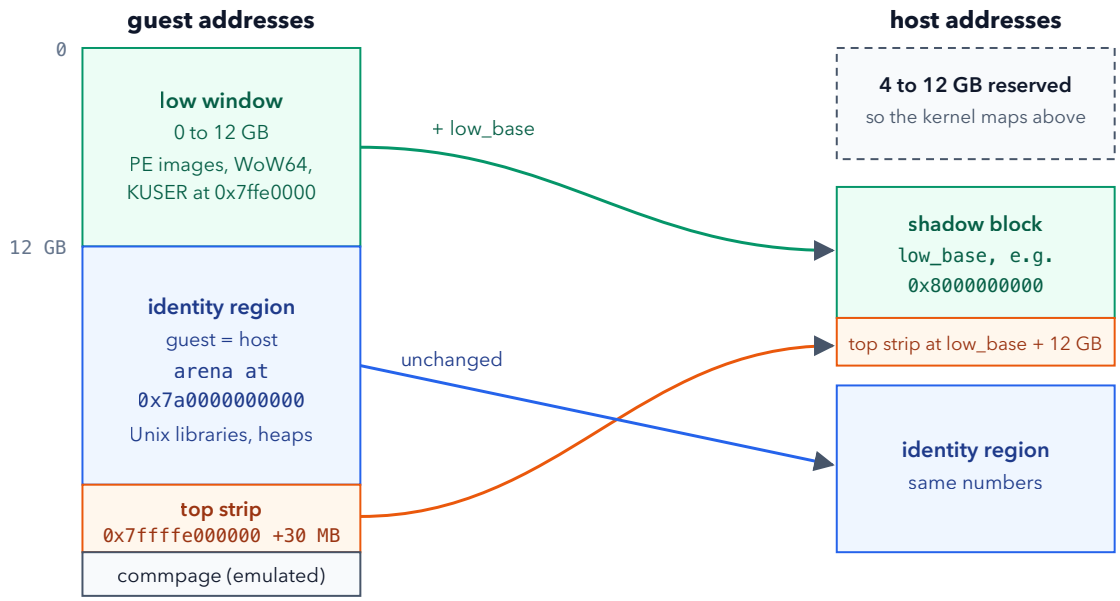
Three costs that every WoW64 program paid were taken out on 2026-10-08 (`e7de713b`), measured with the 32-bit benchmark `winbench32` under Wine. An address in 32-bit code went through the whole low-window test although it is below 4 GB by construction and so, in the low window, is simply `low_base` plus itself; it now costs one `orr`. An `esp`-relative operand of 32-bit code took the acquire load or release store, while `push` and `pop` in the same code, and `rsp`-relative operands of 64-bit code, were plain under the

stack policy; they are plain too, (`OCERZ_TSO_STRICT` orders them all; Chapter 7). And a 32-bit `call` and `ret` now use the host shadow stack as 64-bit ones do, with the return address tagged so the two modes' entries stay apart. In milliseconds, before and after, with Rosetta's time in brackets: virtual calls into small frames 461 to 166 (158), `qsort` 159 to 142 (156), strings 303 to 286 (104), hashing 30 to 26 (20), recursion 12.2 to 11.6 (6.4). Strings are still 2.8 times Rosetta's time and recursion 1.8 times, so 32-bit code is not finished.

> **HOW IT WAS FOUND: THE RESTART STORM**
>
> In early September every Steam process died within a second of starting, at its first `OutputDebugString`, and Steam relaunched itself in a loop. `OutputDebugString` raises a Windows exception, so the first exception was killing it. The decoder had folded `mov r/m, sreg` into the long-mode selector constants, so the 32-bit `RtlCaptureContext` recorded the 64-bit code selector in the context it captured; WoW64 copied that selector into the frame it restores with `iretq`, and the 32-bit exception dispatcher then ran as 64-bit code until its stack overflowed. Segment selectors are CPU state now (2026-09-05), read and written like any register.
>
> A second detail came out of the same hunt. A selector slot is 16 bits, and the hardware ignores the rest; Wine's 32-bit context structure stores `SegCs` and `SegSs` as 32-bit fields that a 16-bit store fills, so their upper halves are stack garbage by the time `wow64cpu` pushes them for an `iretq`. A far transfer now takes the low 16 bits only. A 14-second MinGW reproduction and `OCERZ_MODELOG=1`, which prints every far transfer with its selector and target mode, found it. The guest tests `mov_sreg` and `far_sel_bits` and the i386 differential case `mov-sreg` pin both.

Memory: the low shadow

A normal 64-bit macOS process reserves its first 4 GB as `__PAGEZERO`, and AArchX itself lives in the host address space too, so a guest that needs memory at low addresses cannot simply have it at the same numbers. Windows code needs exactly that: PE images at 32-bit bases, `KUSER_SHARED_DATA` at `0x7ffe0000`, and the entire 32-bit address space of WoW64 code. When the program being loaded has a segment below 12 GB, AArchX switches the process to its third memory layout, the **low shadow** (Chapter 8 describes all three).



Not to scale. Every process of a prefix uses the same low_base (inherited in OCERZ_LOWBASE) and the same arena base.

Figure 15.3. The low-shadow layout of a Wine process: where guest addresses live on the host.

The rules, from `include/ocerz/mem.h` and `src/mem.c`:

- **The low window.** Guest addresses below 12 GB (`OCERZ_LOW_LIMIT`, `0x300000000`) are stored at `low_base` plus the address. One host block holds the 12 GB window and the top strip; AArchX tries the bases `0x8000000000`, `0x10000000000`, `0x500000000` and `0x600000000000` in turn, or exactly the base its parent passed in `OCERZ_LOWBASE`, so every process of a prefix agrees. That agreement matters because `wineserver` reads and writes its clients' memory with Mach calls aimed at their tasks, and those calls must move a low address by the same base (`2e5f0d20`).
- **The top strip.** The 30 MB below the commpage, `[0x7ffffe000000, 0x7fffffe00000)`, is also redirected, to `low_base` plus 12 GB, because Windows puts things there that the host cannot give at those numbers.
- **The identity region.** Everything else is at its own number, guest equal to host. The Unix side of Wine, the shared cache and the heaps live there.
- **The fixed arena base.** Since `26b1ccba` (2026-09-26) a process whose loader is named `wine` reserves its identity arena at `OCERZ_WINE_ARENA_BASE`, `0x7a0000000000`, falling back to the usual placement only if that range is taken. The callout below explains why. Since `74e6140d` (2026-10-13) a process counts as Wine's loader when it carries a `WINE_RESERVE` segment, the address range Wine's preloader keeps for Windows, whatever the file is called; the name `wine` or `wine64` had been the only test, and it failed for CrossOver, whose processes start through a link named after the Windows program (see *CrossOver* below). `OCERZ_NO_WINE_SEGMENT_DETECT=1` goes back to the name alone.

HOW IT WAS FOUND: ONE PAGE, EVERY PROCESS

Every Wine process writes the address of its `__wine_syscall_dispatcher` into a slot at `0x7ffe1000`, next to `KUSER_SHARED_DATA`. Under AArchX that slot shares a 16 KB host page with the file-backed `KUSER_SHARED_DATA`, so it is physically one page shared by every process of the prefix. That is harmless only while the value is the same everywhere, that is, while `ntdll.so` loads at the same address in every process.

It was found twice. On 2026-09-06 Steam's CEF children crashed reading address zero: a new process zero-filling its fresh pages zeroed the shared slot for every process, and until it stored its own pointer a few instructions later every system call elsewhere called NULL. The zero-fill now leaves that slot alone. On 2026-09-26 `wine notepad` died while `wineboot` was still starting services: with the arena wherever the host put it, `ntdll.so` loaded at a different address in each process, the last process to write the slot won, and every other process sent its system calls into memory it had never mapped. With the fixed base, `ntdll.so` sits at `0x7a0041c1c000` in all of them, the whole service tree starts, `wineboot` finishes in about 15 s, and Notepad opens.

Host pointers into the low window

The low shadow has a cost on the boundary with the host. When the kernel or a system service hands the guest a host pointer below 12 GB, that number names the guest's own memory at that address, not the host memory it points to. AArchX handles this with **aliases**: the host region behind such a pointer is mapped again into the shadow at the same guest number, so the guest can read it. IOKit replies that carry addresses of kernel-mapped GPU memory were the first case, in July. Four refinements came in October, all from R.E.P.O.:

- **Clipping** (`2e5f0d20`). An alias stops at the first page that holds guest data. Aliasing a 2 GB slice of the host's arm64 shared cache once replaced Wine's loader header, and the windowing system walked that header for ten seconds until the GPU watchdog fired.
- **Inner pointers** (`cc4ea96c`). Creating an IOSurface answers with records inside memory the kernel shares with the process, at addresses that are not page-aligned. The reply scan only took page-aligned values, so those records were read through whatever the low shadow held at their numbers; a buffer size came back as four gigabytes with the right size in its low bits, and R.E.P.O. aborted on a nil drawable in about two launches of three. Eight-byte-aligned values inside IOKit or shared host memory are now candidates too, unless the guest has its own memory at that number.
- **Refresh** (`13e7b14a`). Metal frees a buffer and maps the next one at the same host address. The alias kept the freed buffer's memory object alive, so the guest read the previous compute job's results and a job could wait forever. Each alias now records the memory object it shows, and is mapped again when the host has put a different object there.
- **Keeping the window clear** (`334da007`). The identity arena starts at 12 GB, so the kernel put its own mappings in the free host space below it, exactly where Wine and Unity keep their memory: IOSurface's records landed on numbers the guest already used, where they cannot be aliased, and R.E.P.O. failed to start in three launches of five. A low-shadow process now reserves every free host range between 4 GB and 12 GB, at startup and after each alias, so the kernel maps above the arena, where guest and host numbers agree. With it, eight launches of ten reached the menu with no wild pointer.

Pointers that point the other way have the same problem. `__mac_syscall` passes the kernel a policy-defined structure that points into the caller's stack, and under Wine the Unix side runs on a per-thread stack inside the low window. The kernel received guest addresses, the sandbox check failed with `EFAULT`, and the Metal OpenGL renderer read that as no access to the GPU: every pixel-format request from a Wine thread failed, and wined3d found no GL adapter. Since `fdb259d2` (2026-09-26) words in the first twelve of that structure that point near the caller's stack pointer are rewritten to host addresses for the call and restored afterwards; `dxdiag` under Wine reports "GL renderer: Apple M5" with a 4.1 core context. The voucher-creation Mach trap had the same flaw from a low stack, and libdispatch, which creates vouchers for the work it queues, crashed on purpose until `f81c8478` (2026-10-03) translated its pointers too.

The cost of the low window, and how it came down

In the low shadow every guest memory access needs a translation, because the guest number may be in the window, in the identity region or in the top strip. Until October that translation built three 64-bit constants and tested three ranges, about twenty host instructions per load, and the JIT's fastest memory forms, which assume an address can be used as it is, were switched off entirely. Two rounds of work narrowed the gap. In September (`53df4711`, `d307ec19`) memory operands, calls, returns, the host return-address stack and spliced calls learned to take the guarded path instead of falling back to the interpreter, which took Steam's web helper from 47-67% of its samples in translated code to 78-81%. In October (`1b636806`) the guard itself shrank: when the window's host base has no bits below 16 GB and is encodable as an arm64 logical immediate, as `0x8000000000` and `0x10000000000` are, an address below 12 GB becomes a host address with a shift, a compare and an `orr`, and one in the identity region is recognized with two shifts and an add. In the Wine layout that took `memcpy` from 6.1 times Rosetta's time to 3.1, a mixed workload from 3.0 to 1.65 and an interpreter loop from 1.5 to 1.0.

The Wine layout's JIT work, driven by 7-Zip

Between 2026-10-04 and 2026-10-11 more than twenty more commits narrowed the gap, and the last six were driven by one program. 7-Zip 25.01 under Wine Staging is a good probe because its two hot loops, LZMA's range decoder and its binary-tree match finder, are small, memory-bound and unforgiving. Chapter 8 describes the guards and the base hoist, Chapter 6 the flag handling and Chapter 7 the memory ordering; this section is the history and the measurements.

A wrong turn, dated. On 2026-10-04 (`4450788a`) the host block behind the low window was made 16 GB, of which the guest can own only the first 12, so that one `tst` of the high bits told a low address from an identity one. R.E.P.O.'s menu measured the same, at 100.2 and 93.8 frames a second against 106.8 and 92.1 in alternating runs, and `xbench`'s `memcpy` went from 0.089 to 0.074 s. It was reverted the next day (`85687480`, 2026-10-05): on macOS 26.7.1 the host places IOSurface and Metal memory between 12 and 16 GB, the guest reaches it at its own address, and with the 16 GB window those addresses landed in the reserved part, so two tests, `dmetal_nocopy_low` and `diosurface_low_stack`, died with an alignment fault inside CarbonCore. That is the path a Wine game drawing through DXMT takes. macOS 27 places that memory elsewhere, which is why the gate on that machine had passed.

The guard stages. On 2026-10-10 the Wine layout's memory operand lost most of its cost in a series of commits, each measured on a single kernel and quoted as the commit gave it:

Change	Kernel	Before to after
In-place string and memory routines take translated pointers (`a0375541`)	`memcpy` , `xbench` , Rosetta 0.563 s	1.274 to 0.429 s
The hoist takes three bases, indexed operands, either side of 12 GB (`ff3e02ea`)	`fpvec` , time relative to the same kernel outside the Wine layout	2.63 to 1.07
A base register is guarded once per block, the displacement rides in the access (`41a5b5e9`)	Lua 5.4 mixed script, Rosetta 0.625 s	0.870 to 0.826 s
`rsp` 's register holds the host address it points at (`ab86121d`)	fewer instructions retired in an object-update loop	not timed in the message
12 GB lives in `x29` , and each guard lays out the side its address was on (`5b02be18`)	object-update loop, Rosetta 0.278 s	0.369 to 0.334 s
Hoisted indexed operands are one register-offset access (`8365b47d`)	`chase` , Rosetta 0.523 s	0.670 to 0.553 s

The first row matters most for what it says about the layout: until it, every Wine-layout process ran Apple's x86 `memcpy` , `memset` and `strlen` , translated, with the low-window test on every access, because the in-place routines (`src/leaf.s`) refused the layout outright (`memcpy` took 2.26 times Rosetta's time against 0.72 outside it). In cache mode the block at a routine's entry now hands it translated pointers, after checking that the range it may touch lies on one side of 12 GB; a failed check, a routine that declines and a fault inside one all arrive after the call with the arguments put back, and the x86 translation runs as before.

7-Zip, 2026-10-11. The decompression benchmark (`b -mmt1 -md22`), which ran at half of Rosetta's speed under Wine, was worked on for an afternoon. The cause of each step was visible in the profile: the decoder decodes each bit with a `sub` , two `cmovae` , two `cmovb` and an `sbb` , the last of them eight instructions after the `sub` , so the `sub` wrote its flag record and each consumer read it back at fifteen to eighteen instructions. The steps were: the window through which a compare hands its flags on grew from 3 instructions to 8 (`e853c450`); a plain load may sit in it (`a8773e55`); an `adc` or `sbb` of an immediate whose carry comes from a compare is one `csinc` (`d68898a5`); the flags live at a loop's own start became the least fixed point over the loop, which stopped the LZ copy loop's back edge from materializing the flags on every trip (`5fc69e49` , where 136 of 150 samples in the materializing routine came from that one edge); a `lea` with an address-size prefix may sit in the window (`571a7295`); and the hoist checks an index that two operands share once a trip (`e8a26292`), which is both of 7-Zip's hottest loops.

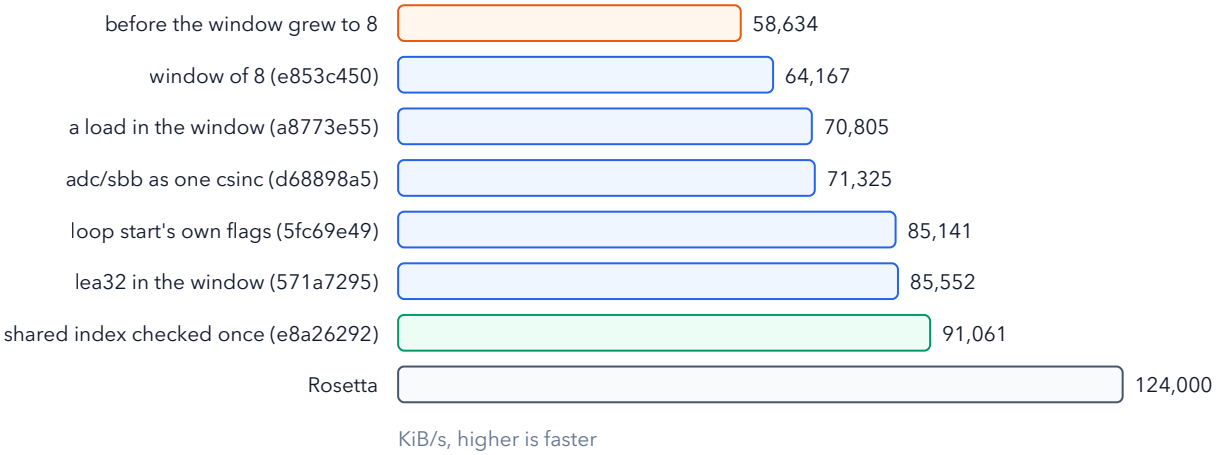


Figure 15.4. 7-Zip 25.01 decompression (`b -mmt1 -md22`) under Wine Staging on 2026-10-11, in KiB/s after each commit as that commit's message reported it. Each figure is a separate pair of runs, so the steps carry noise of a few percent; Rosetta's 124,000 was measured the same day.

Compression is the harder half. It did not move in any of those commits, and on 2026-10-14, on the MacBook Air, it was about 8,200 KiB/s under AArchX against 11,600 under Rosetta, one thread: a 26 MB archive took 6.85 s against 4.8 s. Seventy percent of it is LZMA's binary-tree match finder. Ordered memory, which Wine always needs, costs that kernel about 17% against plain memory, and the archive comes out identical either way, so ordering changes the speed and not the answer. Splitting the 17% shows two different pictures. In 7-Zip's own benchmark the acquire loads cost 14% and the release stores 2.6%. In the real run the barriers cost only 1-3%, and most of the rest is code generation: the ordered load and store forms of arm64 take no index register, so an indexed access has to form its address first. Chapter 7's rule that scalars stay ordered is untouched; the lever is cheaper address formation under ordering, not weaker ordering.

The walls, in order

The Wine bring-up ran from June to October. Most of its walls were not where they first appeared to be, and the way each was found is as instructive as the fix.

June and July: the first windows

- **libdispatch.** `CFRunLoopRunInMode` never returned, which blocked `winemac.drv` 's Cocoa startup. Four bugs, fixed in July; bridging guest work queues to the host's real workqueue mechanism is what made dispatch work at all (Chapter 9).
- **The display configuration.** Hundreds of "Failed to read display config" lines came from `MTLCreateSystemDefaultDevice` returning NULL, because one dyld API, `dlopen_preflight` , was unimplemented and Metal concluded its compiler was missing. A dozen lines of code fixed it, and the real Apple GPU appeared. Display problems are logged under `WINEDEBUG=+display` , not `+macdrv` .
- **Invalid exception frames** (`3317e905` , 2026-07-31). `mach_vm_region` replies described the host's map instead of the guest's, so Wine's address-space reservation recorded garbage areas and later put a thread stack at a shadow host address, and exception dispatch rejected every frame. Region queries now answer from AArchX's own accounting.

- **The explorer deadlock** (resolved 2026-08-18). Three threads waited on a mutex owned by a thread waiting on a Win32 handle. The real cause was older: the hand-built main thread's cached thread id had never been filled in, so `pthread_threadid_np` returned 0, mutexes recorded owner 0, which reads as unlocked, and the main thread's mutexes provided no mutual exclusion at all.

August: the Steam client starts

Date	Wall	What it really was
08-17	Wine sees no GPU	IOKit out-buffer pointers inside a request were not translated under the low shadow (`663ec40c`)
08-19	32-bit code	the i386 decoder, interpreter, JIT and WoW64 host support
08-24	sessions ending 15-60 s after a window appeared	AArchX rewrote an interrupted Mach receive as a timeout, which CoreFoundation took as its run loop timing out, and applications exited with status 0 (`3e46c6f4`)
08-25	a desynchronized wineserver protocol	the guest's signal mask was never enforced, so a `SIGUSR1` landed in the middle of a `wineserver` round trip
08-25	a fast failure after 23 s	the `gs:[0x58]` TLS copy described above
08-25	Steam exits silently with status 14	the bootstrapper requires SSE4.2, so it is implemented and advertised; status 14 is the client asking to be restarted
08-25	**first window**	Steam's rescue dialog, rendered and clickable through `winemac.drv` ; a fresh prefix boot fell from 43 s to 17 s
08-26	the web helper times out at startup	V8 reserves a 32 GB sandbox with a fixed-address `mach_vm_map` , and AArchX's region table held 16 entries and leaked one on overflow, refusing 642 million calls in one run
08-26	two real crashes	an `os_unfair_lock` whose owner died, now broken as a robust mutex is, and guest self-kills sent to the host instead of the guest
08-26	profile database errors	813 refused fixed-address mappings that two instrumentation bugs had hidden

September: to the login window

The first week of September was a series of short, sharp bugs, most found by bisecting the JIT against the interpreter (Chapter 18):

- **Black windows** (introduced by `60f961d6` on 2026-08-25, fixed by `6073adee` on 2026-09-05). Every Wine window painted black. clang's outliner produces helpers that end in a compare and a `ret` and return their result in the flags, and the JIT assumed no flags survive a return. A pure flag producer that reaches a `ret` now keeps its flags live (test `ret_flags`).
- **Self-exiting windows** (`ef67bb06`). Notepad quit after about 24 seconds with status 0. A category that never attached made an Objective-C method missing inside a dispatch block, and a page the kernel mapped for an IOSurface sat at a host address the guest could not see. The first is why CoreSpotlight is in the default Objective-C preload list.
- **The restart storm**, described in the WoW64 section above.
- **An endless re-exec.** The initial stack capped the environment at 60 entries and dropped Wine's trailing `WINELOADERNOEXEC=1` , so the loader re-executed itself forever.

- **A stream that never updated.** A read-only, unaligned shared mapping became a private snapshot, so `steam.exe` never saw the browser's writes to its UI stream; real shared overlays are tried first now.
- **SQLite "out of memory" in CEF.** One miscompiled block: a fused compare-and-branch whose taken side's flag record was bypassed by chaining (`8ce197f2`).
- **Misaligned locked instructions.** A Windows critical section inside a packed structure made `lock cmpxchg` misaligned, and arm64's atomics fault on misalignment, so those go through an address-striped lock.
- **Self-modifying code.** V8 writes its code with plain stores, and the JIT ran stale translations until code pages were write-armed (Chapter 7).

On 2026-09-08 a last run of seven fixes, among them a V8 safepoint deadlock, `pop [rsp+disp]` computing its address before the pop, AVX decoding for Metal's shader paths and `punpckhqdq` decoded as `punpcklqdq`, which had squashed every Cocoa window to 74 pixels high, brought up Steam's 700 by 440 "Sign in to Steam" window at full size, painted, with software compositing.

Late September: to the main window

- **The fixed arena base** (`26b1ccba`), described above.
- "**No network adapters**" (`50a4749f`). Foundation, first pulled in by a `dlopen` under Wine's `ntdll.so`, never had its `+load` methods run, so `+[NSString alloc]` returned an abstract class rather than a placeholder. Wine's mount manager answers Steam's network query through SystemConfiguration, a string method then recursed until `winedevice`'s stack ran out, and Steam saw no adapters and stopped at its connection dialog. `dlopen` closures now run `+load` before initializers.
- **No localization** (`d1cf1c49`). `pcmpistri` must count a needle that runs past the end of its 16-byte block as a partial match; AArchX cleared the bit, so Steam's SSE4.2 `strstr` never found `%language%` in a file name, loaded no localization, and stopped on an assertion right after login.
- **Wine-only JIT gaps** (`53df4711`), found with the new `OCERZ_GUESTPROF` sampler, which names the instruction forms a process is interpreting: Steam's web helper interpreted `pinsrw` from memory for 15% of its samples, because every emitter that needed a plain address handed its memory forms to the interpreter in the low window.

On 2026-09-27 Windows Steam reached its **main window in about 33 s with 58 s of CPU**, where Rosetta takes 11-12 s and 17-21 s. A profile of one startup split the time: translation took 12.6 s of 54, over 2.18 million blocks; Windows code spent 14 s in translated code; and a short Wine process's startup is about 70% translation, most of it while `winemac.drv` brings in AppKit and Foundation. That is what the translation cache addresses (Chapter 7): a Wine process's code sits at the same address every run, so a block one process translated is loaded, not translated again, by the next. Windows Steam's window came up in 46.9 s from an empty store and in 35.2 s and 33.7 s from a warm one. Three and a half minutes after launch the two busiest web-helper processes used 31% and 10% of a core where, the day before, they had used 197% and 167%; once Steam settles, the whole Wine tree uses about a tenth of a core.

Stock Wine Staging 11.16 with DXMT

On 2026-10-01 Steam was run on stock Wine Staging 11.16 with DXMT, the Direct3D-to-Metal translation layer. Staging moves `gs` between the TSD and the TEB on every system call and runs Chromium's sandbox, and it found six more bugs (`2e5f0d20`, `6064e90e`): the TSD identity leak described above; signals aimed at one thread delivered to whichever thread reached a system call first, which broke Wine's

`SIGUSR1` and `SIGQUIT`; `wineserver`'s Mach calls into another task not moving low addresses by the shadow base, which failed every GPU-process launch under the sandbox with error 43; a fixed-address `mach_vm_map` over pages in use being moved over them instead of refused; aliases that reached into guest data; and libxpc's bulk image-load callback being called only once, at registration, so XPC services of frameworks loaded later were unreachable: shaders failed to compile, text input could not reach its service, and `steam.exe` retried a view bridge a thousand times a second. All but the TSD mapping have an `OCERZ_NO_*` switch (Appendix A).

October: what a Steam session leaves behind

Two costs of running Windows Steam repeatedly showed up in October. The first was blocks that were already dead still running. A chain to a target out of a branch's reach goes through a veneer, a literal load and a `br`, and the chained branch lands on the veneer. Unchaining a retired block compared each branch's target with the block's own code, a veneer is not in it, and so a chain through a veneer was never cut; the retired block went on running from it. In a CEF process with over 700,000 blocks live many chains are far. Each pass out through a tripped probe then went to the runtime, found the block no longer live, and asked for its retirement again, and the retirement code searched the whole live list for it, never finding it. `bfaca27a` (2026-10-11) follows a veneer to the address it loads and keeps each block's index into the live list, which says at once whether it is live. Counted in the three busiest CEF processes of Steam under Wine, probe exits from retired blocks fell from 1,905,762, 1,159,135 and 527,613 to 323,620, 67,584 and 24,701; the browser process had made 169,984 searches of a 716,148-block live list, 29.4 s of its CPU, and none found its block. The dynamic tests `smc_chain`, `smc_ras`, `flip_phase` and `xlive_self` now also run with `OCERZ_FORCE_VENEER=1`, which chains everything through veneers; the commit says plainly that they pass without the fix too, because a retired block is the same guest code, and that the counts above are the evidence.

The second was disk. The translation cache never looked at free space, a Wine session uses several stores, one per memory mode, and each new build adds more. A tester's Mac with a few gigabytes free filled up under a Steam session, could no longer swap, froze and panicked on its watchdog; the development machine's volume held 6.6 GB of cache in 80 stores with 777 MB left. Since `93b919f9` (2026-10-07) nothing is written while the volume has less than `OCERZ_TCACHE_MIN_FREE_MB` free, 10,240 by default; a process writes at most half of what was free above that floor when it opened the store; and a lone process that finds the volume already under the floor removes the store. Reading is unaffected, so a full disk costs warm starts and not correctness.

Wine under native mode

Everything above ran Wine in cache mode, where Wine's x86 code binds against Apple's x86-64 shared cache. On 2026-10-05 and 06 MacNdCheese's unified Wine ran under native mode (Chapter 12), where Wine's code runs translated but its system calls into frameworks go to the Mac's own arm64 libraries through the bridge. The commit of 2026-10-06 (`22221270`) states the result: every service starts with `msync`, Notepad shows its window, Steam logs on and shows its client, and R.E.P.O. reaches its main menu on DXMT. Native mode adds a difficulty of its own: it hands the guest host pointers all the time, and under the low shadow a number below 12 GB names the guest's memory, not the host's.

- **The two LDT calls** (`4f200419`, 2026-10-05). Wine 11.16 under `-native` stopped at once, with `Symbol not found: _i386_set_ldt` expected in `libSystem`. Both functions exist only in the x86-64

libSystem, so the arm64 SDK that the API databases are read from has neither. They became specials on the same table that cache mode's machine-dependent calls 5 and 6 already keep, and `i386_get_ldt` now answers how many entries the table holds, as Rosetta does, where it had answered how many it copied. A census of the 94 x86-64 Mach-O files in MacNdCheese's Wine found no other missing symbol, and `wine --version` prints `wine-11.16 (Staging)`.

- **Memory below 12 GB** (`2d59b306`, 2026-10-06). Readable host memory below 12 GB, the main image, the first heap, the main stack and the shared cache, is aliased into the shadow at the same offset and pinned: `ocerz_g2h` answers its host address, fixed maps over it are refused, and a forked child aliases it again. The free host space left there is reserved once pinning is done, so a later host allocation cannot land where the guest sees the shadow. `wine_main_preload_info` is rewritten to leave pinned ranges out, because Wine asserts on refused reservations. A value in the first 64 KB crosses unchanged, since Objective-C's empty-pool token is 1 and its shadow alias crashed the pool pop. A `__block` variable on a Wine stack forwards to a guest address, and the forwarding word is rewritten to its host alias before libclosure follows it. `mach_vm_map` maps a fixed request in the low window over the shadow, claims an anonymous fixed one above it as cache mode does, and searches an anywhere request from 12 GB up, so that `wineserver`'s `msync` pages are one page to guest and host.
- **A signal landing before a wait.** Wine's `msync` waits on a word in a shared page with `__ulock_wait`. A guest signal that landed after `msync` last looked at the word and before the wait reached the kernel was held until the wait ended, so the `SIGUSR1` APC that would set a service's RPC reply event never ran. The wait is now marked as one that a signal may kick, and the unstick monitor interrupts it once a guest signal is pending.
- **Debug registers.** `thread_get_state` answers the x86 debug-register flavors, all zero, instead of ending the process: `wineserver` asks for them on context requests, and Chromium's stack-sampling profiler took the server down with it.
- **A Carbon call** (`1c67c487`, 2026-10-06). Steam's web helper ended whenever its window was dragged. `winemac.drv`'s `move_window` tests `!IsWindowVisible(hwnd)` for a window move, the name binds to a Carbon export that no header declares any more, so the API database held it as a stub, and the stub ended the process; Steam then started another web helper. It is now a record as Apple's Window Manager reference gives it, and called with a null pointer, a zeroed block or an address in page zero it answers 0, as arm64 and Rosetta do.

The tests for this live in `tests/win`, Windows programs that print OK or BAD: waits woken by another process or by `wineserver` through one handle and through several, named shared memory mapped by two processes, and fresh processes whose first act is an RPC to `services.exe` (`1a01a968`, 2026-10-06). They are how a lost `msync` wake was told apart from a broken shared mapping. The native-mode tests `native_wine_imports` and `native_low_wine`, the second linked as Wine's loader is, under a low shadow, compare against Rosetta's output.

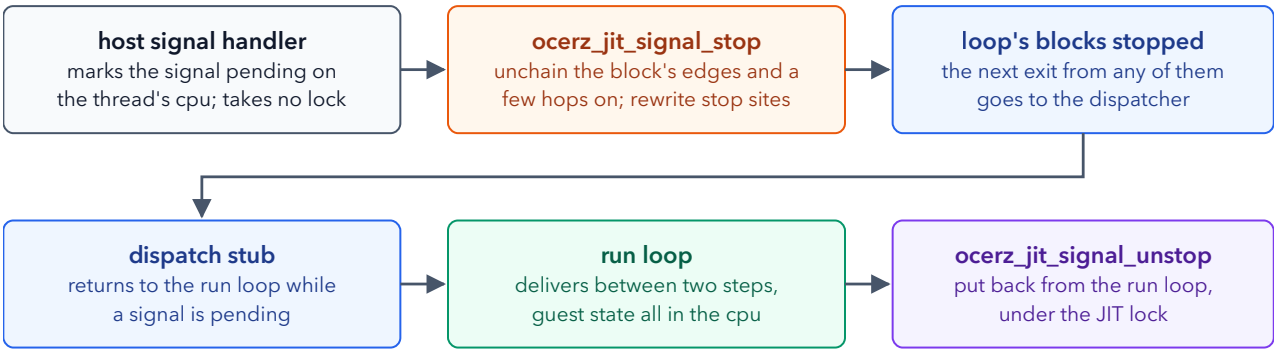
Signals, thread suspension and fault loops

Two rules about signals changed in October, and Wine depends on both.

A signal reaches a thread in a loop

The host's signal handler only marks a signal pending on the thread's cpu structure. Until 2026-10-14 the guest got it after the thread's next system call or bridged call and nowhere else, so a thread spinning in a loop never saw it. That held in the interpreter as much as in the JIT: a loop of `dec` and `jnz` with a 5 ms `ITIMER_REAL` got no `SIGALRM` until it ended. Under Wine this is not a curiosity. `SuspendThread` is a `SIGUSR1` sent to the thread, so Mono's garbage collector, which suspends every thread before it collects, waited for any busy thread to finish its loop.

`a7fd0f3f` changed the rule: both run loops deliver a pending signal that the thread does not block between two steps, where the guest state is all in the cpu. That was enough for the interpreter. Translated code reaches the run loop only when a block exits to it, so there are two more pieces.



Before 2026-10-14 a thread in a loop got a signal only after its next system call or bridged call.
The interpreter needs only the run loop's delivery; translated code needs the stop and the stub.
OCERZ_NO_SIGNAL_STOP=1 restores the old rule; m32 processes keep it.

Figure 15.5. How a signal reaches a thread that is inside a translated loop (`a7fd0f3f` , 2026-10-14).

- **The handler stops the blocks.** It calls `ocerz_jit_signal_stop` with the interrupted host pc. That does to the block the thread is in, and to the blocks its chained edges lead to a few hops on, what a retire does: it unchains their chained edges and rewrites their stop sites, so the thread's next jump out of any of them goes to the dispatcher. The thread records each block and edge it changed, and `ocerz_jit_signal_unstop` puts them back from the run loop: a stop site goes back to the word it was translated with, now kept as `stop_run`, and an edge is chained again the way a chain install would.
- **The dispatch stub returns to C.** The stub that block exits jump into enters the next block without returning; while such a signal is pending it goes back to the run loop instead.

Stop sites alone were not enough. A loop whose backward branch is a side exit chained through its conditional site has no stop site anywhere, which is why the walk unchains edges. The handler takes no lock, and that was learned from a test: the thread holding the JIT lock was the sender, translating its own wait loop, so a trylock lost the signal. The handler can race with a chain install on a word, but every word either side writes is a whole valid branch, the restore runs under the lock, and the blocks the handler reads stay allocated, because retired blocks are freed only with the JIT. A block whose stop sites are not in their running state is no longer saved to the translation cache, since it would come back stopped for good. The dynamic test `loop_signal` pins the rule under both engines: seven loop shapes, with plain and with ordered memory, and a thread that 20 `pthread_kill` calls must each reach while it spins, every handler

ending its loop by rewriting the interrupted context. `OCERZ_NO_SIGNAL_STOP=1` goes back to the old delivery, and m32 processes keep it, because delivery there would enter a 64-bit handler (Chapter 13). Chapter 9 has the general signal machinery.

A fault guest code recovers from again and again

The crash handler counts the faults at one guest address in a row. Past sixteen (`OCERZ_SIG_MAX_REPEAT`) it took the thread for one of Wine's foreign workers, looping on a dispatcher state it cannot repair, and ended it with a `WORKER-TERMINATE` message, whenever the thread had no guest signal stack. Every program that is not Wine has none. But a handler that unprotects a page and returns, as a garbage collector's write barrier or a JVM safepoint poll does, faults at the same address again the next time it is protected, and the seventeenth time its thread ended. A sort whose comparator reads a protected page on every comparison exited 241 in cache mode after sixteen of its 338 faults, and under 0.8 as well, where a Mac delivers every one. `ff6ea313` (2026-10-18) applies the count only to Wine processes, the foreign workers it was written for. The new native case `signal_callback_fault` pins it: in native mode under both engines, where every fault lands inside a callback from the host's `qsort`, and in cache mode.

CrossOver

CrossOver's Wine is another Wine build, and on 2026-10-13 AArchX.app learned to run its bottles (Chapter 14 explains the mechanism: a `BinPath` pair of scripts and a script in place of each launcher). What it taught the translator was small and specific. Wine's alternate loader, `--enable-alt-loader macdrv`, runs a program's window processes as copies of the launcher's own Intel executable, and under `ocerz` those processes do not run yet; the launcher is therefore replaced by a script that starts the program through CrossOver's `wine` without the option, and Steam in a ticked bottle runs all 16 of its processes under AArchX (`e3a26326`). And processes started through a hard link named after the Windows program were not recognized as Wine's, which `74e6140d` fixed with the `WINE_RESERVE` test described under the fixed arena base. CrossOver runs in cache mode as the Wine builds above do. Nothing here has been timed against CrossOver on Rosetta.

Games through Wine

Counter-Strike 2

Counter-Strike 2, launched from Windows Steam with D3DMetal from Apple's Game Porting Toolkit, was the first game run through Wine under AArchX. D3DMetal did not load at first: its bridge library `dlopen`s `@rpath/D3DMetal.framework/D3DMetal` through its own `@loader_path` rpath, and in cache mode AArchX passed `@`-paths to the loader as written. The bridge asserted "Failed to dlopen D3DMetal" at the first Direct3D call, Wine's abort handling spun in its unwinder, and the game sat at 100% CPU with no window. `5606e53e` (2026-09-26) resolves `@rpath` and `@loader_path` against the image that called `dlopen`, as dyld does; with that and the `__mac_syscall` fix above, which cleared an error box from wined3d, CS2 loads D3DMetal and reaches its own window.

It then stops on a fatal "Multiple entity classes have the same designer name" error that Rosetta never reaches, which points at a translation problem rather than at D3DMetal. Finding it means running `cs2.exe` with JIT features switched off, but switching them off for the whole process tree slows Steam so

much that CS2 never starts, so `OCERZ_EXE_ENV="cs2.exe:K=V,..."` (`7ea8fc70`) sets variables only in the processes whose command line matches. The same runs print hundreds of "dropping stale oolslow arm" warnings whose source is not yet known. That is where CS2 stands.

R.E.P.O.

R.E.P.O., a Unity game, run through MacNdCheese's Wine with DXMT, is the first game to render under AArchX through Wine, as of 2026-10-03. Unity brings two things Steam does not: a JIT of its own, Mono, which writes and patches code constantly, and a renderer that pushes real GPU traffic through DXMT and Metal. Getting it to render, then to run well, took thirteen commits over 3 and 4 October:

Symptom	Cause	Commit
livelock at about nine cores; every shader failed	DXMT links its own LLVM statically, and a weak-symbol bind took the definition from Apple's LLVM in an unloaded cache image; weak binds now count only images already loaded	`dba5478d`
GPU address fault on the first frame	Metal's no-copy buffers handed the kernel a low-window guest address; recognized by the call's shape and translated for the call	`f21f8236`
nil drawable after a failed IOSurface query	the voucher-creation trap's pointers were not translated from a low stack	`f81c8478`
stale compute results, jobs waiting forever	aliases kept freed GPU buffers alive; refreshed now	`13e7b14a`
nil drawable in two launches of three	unaligned pointers in IOKit replies; aliased now	`cc4ea96c`
failed to start in three launches of five	the kernel's mappings landed in the low window; 4-12 GB reserved now	`334da007`
30% of the process in invalidation	Mono writes code all through startup, with about 390,000 blocks live; invalidations now use per-granule block lists	`3a4eb467`
a crash on the profiler thread	the sampler read a cpu that had just been freed	`94d4b08a`
Mono crashes; new code interpreted after two minutes	the 1 GB code arena filled; it is now flushed and refilled	`7b82e872`
low frame rate	the low-window address guard; now a shift, compare and `orr`	`1b636806`
(a test, not a symptom)	`OCERZ_TSO_NARROW` orders `movd` and `movq`; off by default	`975c18fb`
every Mono call-site patch retired four times too much	retirement touches only the blocks in the written range	`b8ba4796`
4% of the process interpreting MMX	UnityPlayer's video decoder is MMX; MMX is translated now	`b7449a73`

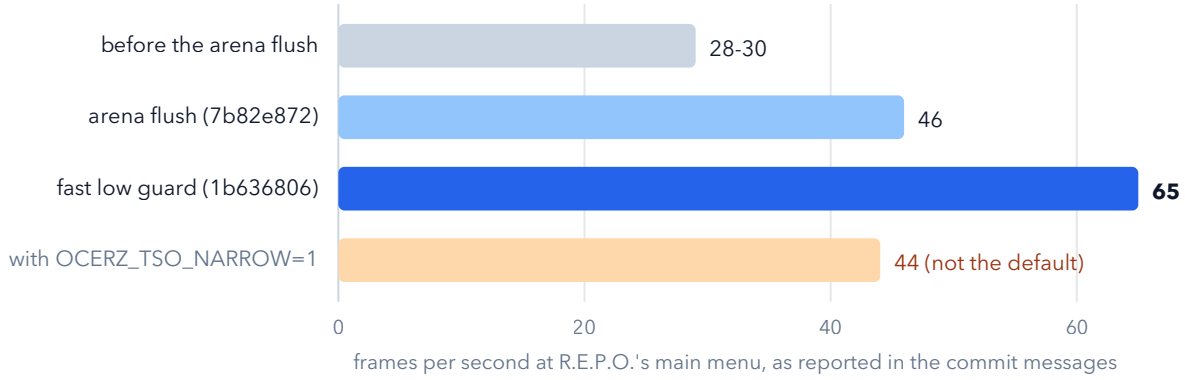


Figure 15.6. R.E.P.O.'s main-menu frame rate across the October changes. The ordered `movd`/`movq` mode is shown for scale; it is correct for a pattern no known program depends on, and is off by default. These are single runs as each commit reported them. With the build at 90fe4d8e the menu's median varies from about 40 to 65 frames a second between runs, a singleplayer level runs at about 50, and Rosetta draws the same menu at about 178.

On 2026-10-06 R.E.P.O. also reached its main menu under native mode, on DXMT and then on D3DMetal, MacNdCheese's backend built on Apple's Game Porting Toolkit 4 (`22221270` and `0aae84f0`; Chapter 14 has the GPU side). Its sound needed a change that applies in both modes. Real-time threads ask the kernel for a time-constraint policy whose period and computation are in `mach_absolute_time` units; the guest sees a 1/1 timebase, as under Rosetta, so x86 code computes them in nanoseconds, and for a Rosetta process the kernel converts them but for AArchX nothing did. CoreAudio's 2 ms of computation was read as 2,000,000 ticks, 83 ms, the policy was refused with `KERN_INVALID_ARGUMENT`, the audio hardware layer's I/O thread gave up after 52 attempts, and every output unit failed with error 89: Wine's audio driver, and through it the game's FMOD, found no devices. `b52597f3` (2026-10-06) sends the three values in host ticks and converts a get reply back to nanoseconds; an output unit now starts and calls its render callback as often as under Rosetta, 56 times in 0.6 s, and R.E.P.O. plays sound. `OCERZ_NO_TC_XLATE=1` sends the values as they are.

Two of those changes say something general about running a JIT on top of a JIT. Mono generates code for the whole of a game's run, and AArchX never reuses a retired translation's space in place, so the 1 GB code arena filled within two minutes and every block after that ran in the interpreter; worse, a translation that overflowed the arena left the translator in a state that damaged later translations, which is why Mono crashed whenever the arena filled while the game was still loading. Since `7b82e872`, when less than an eighth of the arena, at most 8 MB, is left, the next translation miss flushes it: every thread is brought out of translated code at its next block edge, every table pointing into the arena is emptied, and translation starts again at the front (Chapter 7). And Mono patches call sites in code it has already run, over a thousand times a minute, each a write to a write-armed code page; rounding each such write up to a 64 KB granule and clearing every cache in the process had made every patch cost four times what it should and made every thread mispredict its returns afterwards. Retirement is now limited to the blocks the write overlaps, with only the cached entries that point into them cleared (`b8ba4796`).

A MEMORY-ORDERING RESULT WORTH KNOWING

SSE loads and stores stay plain even in ordered mode, the 8- and 4-byte `movq` and `movd` forms included, and compilers do move pointers and counters through xmm registers. `tests/dynamic/tso_narrow.c` shows a reader seeing a published counter newer than the data written before it hundreds to over a thousand times in three million handoffs, which x86 never allows. `OCERZ_TSO_NARROW=1` orders those two instructions and closes the gap, at the price shown in Figure 15.6. No program is known to depend on it, so it is not the default.

What is still open

- **Startup time.** Windows Steam needs about three times Rosetta's time to its main window (2026-09-27; not re-measured since). Translation is the largest part, the translation cache helps on later launches, and the low window still makes translated code slower than in the other memory layouts.
- **Windows Steam with GPU compositing.** In September, without its software-rendering flags, Steam's GPU process asked DXMT for a swap chain on a window owned by another process, and `winemac.drv` refuses that ("Cross-process child window Metal swapchains are not implemented"), under Rosetta as well. Under Rosetta the GPU process then exits and Chromium falls back to software; under AArchX its main thread stayed parked and the browser waited on it. Two later fixes bear on that, signals delivered to the thread they were aimed at and sandboxed GPU-process launches no longer failing, but whether the GPU process now survives has not been recorded.
- **Counter-Strike 2** stops on the designer-name error Rosetta never reaches, and the "stale oolslow arm" warnings are unexplained.
- **7-Zip's compression.** It runs at about 70% of Rosetta's speed under Wine (8,200 against 11,600 KiB/s on 2026-10-14). Decompression closed most of its gap in one afternoon but was still 91,061 against Rosetta's 124,000 KiB/s at the end of it (2026-10-11).
- **Node under Wine.** On 2026-10-18 a function-churn benchmark ran at 2.2 times Rosetta's time on the MacBook Air, 90% of it in translated V8 code rather than in translation. That is an open lead, not a result.
- **CrossOver's alternate loader** does not run under `ocerz`, so every launcher of a ticked bottle is replaced.
- **Arena overflow.** Why an overflowing translation damages later ones is not understood; the flush avoids the condition rather than fixing its cause.

PART VI

Quality, speed and tooling

How correctness is held, how speed is measured, how bugs are found, and what is still missing.

CHAPTER 16

Testing

A translator that is fast and occasionally wrong is worse than useless, because its mistakes surface hours later as a crash in somebody else's code. AArchX is held to correctness by a chain of gates that all reduce to one idea: the interpreter is the reference, the JIT must agree with it byte for byte, and the interpreter itself must agree with real x86 behaviour as Rosetta or the Intel manual defines it. This chapter walks through every gate, what it can and cannot catch, where its expected outputs come from, and how a new test is added.

30
UNIT HARNESSES AT 6E6C64B5

134
GUEST-GATE CASES PER ENGINE, AIR, 2026-10-16

40,044
I386 DIFFERENTIAL CASES, 2026-10-16

406
DYNAMIC-SUITE CASES, AIR, 2026-10-16

The shape of the problem

Every mechanism in Chapters 4 to 9 has two kinds of failure. The first is a wrong answer: a flag computed one way by the interpreter and another by the JIT, a NaN with the wrong sign, a store that becomes visible in the wrong order. The second is a wrong *environment*: a system call that returns the host's view of memory instead of the guest's, an initializer that runs before its dependency, a Mach message whose out-of-line buffer was never translated. The first kind is local and can be pinned by a few instructions. The second kind only shows up when real libraries run, and needs real dynamically linked programs to reproduce.

The gates are arranged accordingly. Unit harnesses drive one component directly. Freestanding guest programs exercise instruction semantics with nothing between them and the translator. Differential gates compare the two engines on the same input, so that a JIT bug is caught even where nobody wrote a golden for it. The dynamic suite builds real Mach-O programs against the system libraries. The native suite does the same for native mode and compares against a separately compiled arm64 build. A separate gate, `make check-m32`, runs i386 programs through m32 (Chapter 13). Above all of these sit two tools that run real software: `tools/apptest.sh` and the benchmark harness, whose kernels checksum their own work.

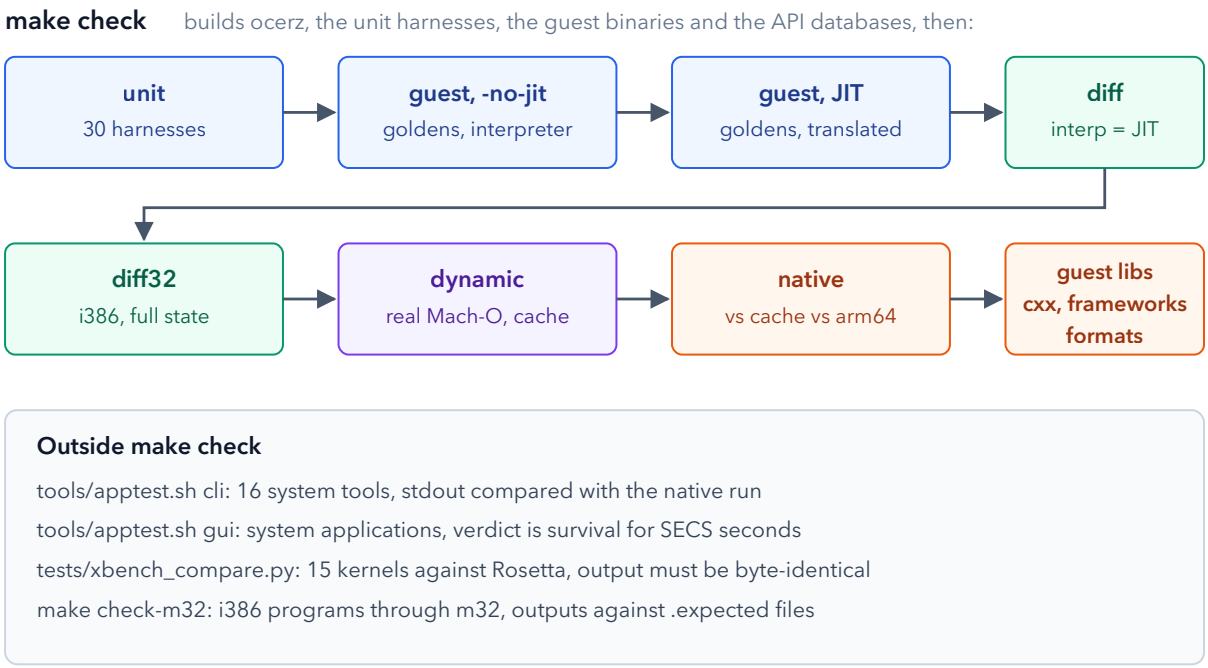


Figure 16.1. The gates in the order `make check` runs them (the guest-library, C++, framework, format and Swift runners that follow the native suite are left out of the picture). Blue gates compare against stored goldens, green gates compare the two engines with each other, purple and orange gates run real dynamically linked programs. Each runs to completion and fails the build on any mismatch.

Unit harnesses

`tests/unit` holds 30 C programs at `6e6c64b5` (counted with `ls tests/unit/*.c`; the Makefile builds one harness from each file it finds there). The newest, `test_m32`, runs with the rest in `make check`; `make check-m32` runs it again with the argument `carbon`, described below. Each links the translator's own object files (everything except `main.o`) and drives one component directly, without loading a guest program at all. The `unit` target in the Makefile runs them one after another with `OCERZ_NO_ARM_EXEC=1` and stops at the first that exits non-zero. Their binaries are committed under `tests/unit/bin` and refreshed by separate "tests: rebuild unit binaries" commits, a convention discussed below. Since 2026-10-13 (`5e09eb4b`) they are linked with no debug map, which would otherwise name the absolute path of every object and with it the build machine\'s home directory.

What makes several of them valuable is that they are exhaustive rather than illustrative. The ABI engine (`test_abi.c`) is checked over 28,071 argument arrangements. The callback trampoline bank (`test_callback.c`) is driven through 131,974 cases. The in-place string and memory routines of `src/leaf.s` are compared with the C library over 6,736,902 inputs, and every string is placed against an inaccessible page, so a routine that reads one byte past the terminator dies inside the harness rather than in somebody's application. The memory suite runs 2,692 checks of the 4 KB slot table against 16 KB host pages.

Component	Harness	Checks (Sept. 2026, except test_m32)
arm64 emitter	`test_a64emit`	validated by execution
decoder corpus, assembler as oracle	`test_corpus`	511 instructions

Component	Harness	Checks (Sept. 2026, except test_m32)
x86-64 decode	`test_decode`	246 / 246
i386 decode and addressing	`test_decode32`, `test_decode32_addr`	102 + 26 rejects, 122
extension and SSE semantics	`test_ext`, `test_sse`	237 / 0, 246 / 0
loader, syscall layer	`test_loader`, `test_syscall`	54 / 0, 365 / 0
guest memory, shared mappings	`test_mem`, `test_shared_map`	2,692 / 0, 105 / 0
API database parser	`test_apidb`	334 / 0
synthesized images	`test_vdylib`	142,426 / 0
bridge, ABI engine, callbacks	`test_bridge`, `test_abi`, `test_callback`	1,247, 28,071, 131,974
thread attach, Objective-C, blocks	`test_attach`, `test_objcbridge`, `test_blocks`	270, 125,400, 107
in-place string routines	`test_leaf`	6,736,902 / 0
shared cache mapper	`test_cache`	15 checks
m32 specials, QuickDraw, Carbon crossings	`test_m32`, `test_m32 carbon`	424 / 0, 347 / 0 (2026-10-16)

Other harnesses pin JIT behaviours that are hard to reach from a guest program: `test_jit_exit` and `test_jit_order_transition` drive the dispatcher and the plain-to-ordered memory switch, `test_jit_psc_invalidate` the invalidation of call-site caches, `test_chain_concurrency` the patching of chains while other threads run them, and `test_jit32`, `test_interp32` and `test_wow64` the 32-bit engines and the WoW64 transitions.

> **A TEST THAT USED TO SKIP**
>
> `test_cache` once skipped silently whenever the shared cache could not be mapped. That is how cache mode stopped working on every macOS 26 machine on 2026-09-19 without a single failing test: the cache had moved on macOS 27 and the lookup followed it. Since 2026-10-03 the harness requires the first directory that holds a cache to be the one ocerz maps, and fails rather than skips when a cache exists but cannot be mapped.

The guest suite

`tests/guest` holds 108 freestanding x86-64 source files at `6e6c64b5` (`ls tests/guest/*.c tests/guest/*.s`, which includes the runtime's own `crt0.s` and `libmini.c`), built into 100 binaries. They are built with `clang -arch x86_64 -nostdlib -static -fno-stack-protector -O2` against a tiny runtime of their own (`crt0.s`, `libmini.c` and the syscall wrappers in `gsys.h`), so nothing stands between the test and the translator: no dyld, no libSystem, no shared cache. A guest test is the purest possible statement about instruction semantics. The binaries are static `LC_UNIXTHREAD` images that run both under ocerz and directly on the Mac under Rosetta, which is how their goldens are produced.

`tests/run_guest_tests.sh` runs every binary and compares standard output with `tests/guest/expect/<name>.out`. `make check` invokes it twice, once with `-no-jit` and once without, so the same goldens judge both engines. Inside each invocation it runs the binaries several ways, because each configuration reaches code the others cannot:

Stage	Setting	Cases	What it reaches
main pass	defaults	94	every test, plain memory model, hardware NaN rule where the processor has `FEAT_AFP`
ordered	`OCERZ_NO_PLAIN_MEM=1`	11	the LSE atomics and `ldapur` / `stlur` forms a multi-threaded program uses
translated NaN checks	`OCERZ_NO_AFP=1`	12	the software NaN checks a processor without `FEAT_AFP` needs
forced replay	`OCERZ_FPB_FORCEREPLAY=1`	12	every deferred NaN check taking its replay arm
tiny arena	`OCERZ_JIT_CODE_KB=64`	5	the interpreted-block path, where a taken side exit must stop the block
Wine	`OCERZ_HOSTWQ=1`, 512 KB arena	2	a real dynamic program from the shared-cache world; skipped when no x86-64 Wine is installed

At `6e6c64b5` the main pass names 94 tests (the `NAMES` array in `tests/run_guest_tests.sh`, which has as many goldens in `tests/guest/expect`), and the stages add 11, 12, 12 and 5, so a run is 134 cases per engine, 136 where an x86-64 Wine is installed and the two Wine cases run. The book's first edition counted 133 at `90fe4d8e` (2026-10-04); the newest test since is `pair_flags` (`c5fe05f4`, 2026-10-07). The recorded runs fit that arithmetic: on the MacBook Air (M5, macOS 27) on 2026-10-16, on the m32 commit `c64cca9f`, 134 / 134 under the interpreter and 134 / 134 under the JIT; on the Mac Studio (M2 Max, macOS 27) on 2026-10-13, 136 / 136 both ways.

The tiny-arena stage exists because of a specific bug. On 2026-08-17 a superblock side exit taken in the middle of an interpreted block let the rest of the block run on after the branch, and it crashed roughly one Wine run in 150. A 64 KB code arena makes almost every block take the interpreted path, so the stage reaches it in every run. The NaN stages exist because the hardware NaN rule made the software path unreachable on the machines the project is developed on: on 2026-09-21 AArchX began setting FPCR.AH where the processor reports `FEAT_AFP`, and without the extra stages the translated checks every older processor depends on would have gone untested.

What the guest tests pin

Most guest tests pin one bug that real software hit, which makes the list a history of the project. A few examples, with the failure each one would show:

- `ret_flags`: helpers that end in a compare and a `ret` return their result in EFLAGS. The JIT once treated every `ret` as killing the flags, and every Wine window painted black (2026-09-05).
- `jcc_gap_fuse`: a fused compare, flag-neutral gap and branch, where the gap writes a register the compare read. The fusion once let the gap clobber the compared value, and Steam's browser process crashed on its first navigation (2026-09-06).
- `punpck_qdq`: `0F 6D` decoded as `punpcklqdq` from the initial commit until 2026-09-08, so the high quadword never moved down and every Cocoa window was constrained to a 74-pixel strip.

- `side_exit_l0`: a scalar SSE result still held in the JIT's lane-0 scratch when a hot conditional branch is taken; chaining past the side exit skipped the flush.
- `simd_pack_jit` and `mmx_jit`: every packed, multiply, shuffle, AES, `pclmulqdq` and `crc32` form translated on 2026-09-27, and every MMX form translated on 2026-10-04, on random rows, edge rows, all four rounding modes and every shift count, with outputs recorded under Rosetta.
- `tso_forms`: every load and store form under the ordered memory model, run in the ordered stage too.

The x86-64 differential gate

`tests/run_diff_test.sh` globs `tests/guest/bin/*`, 100 binaries at `6e6c64b5` (99 at `90fe4d8e`), and runs each under `-no-jit` and under the JIT with the same arguments and environment. The two outputs must match byte for byte, and so must the exit codes. The interpreter is the reference implementation, so any divergence is a JIT bug by construction. It passed 99 / 99 on 2026-10-04 and 100 / 100 on the Air on 2026-10-16 (and 100 with no failures on the Studio on 2026-10-13).

The gate needs no golden at all, which is its strength. A test added for one purpose exercises hundreds of other instructions along the way, and the differential checks all of them. The project's notes from early September record JIT work in which this gate caught a regression the guest suite had passed, which is why it is part of the gate list every JIT commit must clear.

> **WHAT A DIFFERENTIAL CANNOT SEE**
>
> A mistake the interpreter and the JIT share is invisible to a differential. The interpreter's own semantics are pinned only by goldens taken from real hardware behaviour, and by the real software in the dynamic suite and in daily use. Several bugs in this book were found exactly there: the `punpckhqdq` decode, the `pcmpistri` partial match that broke Steam's localisation, and the SSE4.2 instructions Steam needed at all.

The i386 differential gate

There is no 32-bit Mach-O for current macOS to load, so the WoW64 32-bit engines cannot be tested by running whole programs. (m32 runs whole i386 programs, but only by building them with an old linker; Chapter 13 and the section below.) `tests/run_diff32.sh` and its harness `tests/diff32.c` build 32-bit instruction sequences directly in guest memory instead, and run each one twice from the same architectural state, once interpreted and once through the JIT and the same dispatcher `vm.c` uses. The comparison is total: all sixteen general register slots at 64 bits wide, EFLAGS after materialisation, EIP, the mode, the segment selectors, the floating-point and SSE register file, and every byte of every memory region the sequence was allowed to touch.

Until 2026-10-07 the default gate was 33 hand-written cases plus 20,000 generated ones, 20,033 in all. It is now 44 hand-written cases, 20,000 generated integer sequences and 20,000 generated x87 sequences, 40,044 (the commit that added the x87 sequences, `9f0b7087`, gives this split). `--cases` sets the first count and `--x87-cases` the second, which defaults to the first. The x87 sequences start from a register file the case chooses, with NaNs, infinities, zeros, double denormals and integers above 2^{53} at stack depths 1 to 8, and the comparison covers each register's double, its exact 80-bit image and its validity bit, the tags, TOP and the control and status words (Chapter 7). `run_diff32.sh` runs the corpus twice, once in an offset arena and once laid out as a Wine process is, the identity arena and low shadow window (`--low`), which is where WoW64 code actually lives; so a run reports 40,044 twice. The Air's run on 2026-10-16

passed 40,044 both times. Every sequence ends in the 32-to-64-bit far jump WoW64 uses, so the terminator is both the stop condition and a test of the mode switch itself, and unreached code is filled with `hlt`, which is fatal in the interpreter, so a sequence that runs off its end is loud rather than silent.

A generator that crashes is worthless, so nothing in it relies on luck. EBP is pinned to the middle of a mapped scratch region and never written; it, rather than EBX, is the pin because the 32-bit byte registers are AL, CL, DL, BL, AH, CH, DH, BH and no byte write can reach it. ESP is changed only by bounded adds and subtracts with stack traffic tracked by depth. Every memory operand is one of a fixed set of forms that provably lands inside a mapped window, and the guards that make it so are ordinary i386 instructions emitted before the operand's instruction, so they are under test too. `div` and `idiv` carry a prologue that makes a divide error impossible, and `pusha` / `popa` and `enter` / `leave` appear only in matched pairs.

Two switches keep the gate honest. `--jit-required`, which `run_diff32.sh` always passes, fails the run if the JIT translated fewer 32-bit blocks than sequences were run, so a future change that quietly declined them could not turn the gate into a second interpreter run that passes every case. `--selftest` corrupts one field of the JIT side's result for each class of compared state and requires the comparator to catch every one, so a green run cannot come from a comparator that compares nothing. The random generator is splitmix64, so a seed and a case index reproduce any failing sequence exactly.

> **HOW IT WAS FOUND**
>
> The switch got its present form because the gate had once failed in exactly the way it was meant to prevent. Since 2026-09-07 a translated block arms the page it came from, so the harness's rewrite of its code region for the next case ran into the self-modifying-code path, which retired the blocks and counted the 64 KB region toward the churn limit; three cases later the region ran interpreted. The 20,000-case gate went on passing while translating 516 blocks, where it translated 50,970 when it landed. `0d296dbd` (2026-10-07) found it with `OCERZ_PERFSTAT=1`, which showed 260,156 of 260,162 executed instructions going through the interpreter's one-instruction path. The harness now retires every block and disarms before it writes a case, and asks for a 2 MB arena, since retiring everything flushes the instruction cache over all code emitted since the last flush, which made the default gate take 117 s in the 1 GB arena and takes 15 s in 2 MB.

The first time it was pointed at a real 32-bit JIT, on 2026-08-20, the gate found eight bugs, among them an effective-address computation that truncated at 4 GB with no case for 16-bit addressing, the BCD instructions declared to read no flags, CL-count shifts that wrote their destination when the count masked to zero, and a `mov A,B` + `op A,A` fusion that miscompiled 64-bit blocks too.

The dynamic suite

Freestanding binaries never reach dyld, libSystem, threads, signals delivered through the system's trampolines, `fork`, or the shared cache. `tests/run_dynamic_tests.sh` builds real dynamically linked x86-64 Mach-O programs with the Command Line Tools' clang, from the sources in `tests/dynamic`, and runs them under ocerz in cache mode. At `6e6c64b5` that directory holds 168 C sources, 11 Objective-C, 7 C++, 3 Swift and 2 assembler files, and the runner has 196 case invocations, counted with `grep -E "^run_[a-z_]+_case "` on `tests/run_dynamic_tests.sh` (the same count gives 88 at `90fe4d8e`; the first edition's 91 counted a little differently). Most cases run under both engines and report one result for each, and some groups run the same source again with a switch or in another address layout, so the gate's own total

is about twice the invocation count: 184 on 2026-10-04, 287 on 2026-10-08 (`d1679446`), 404 on the Studio on 2026-10-13 and 406 on the Air on 2026-10-16, all with no failures. The runner also checks that every dyld API slot ocerz answers has a witness (Chapter 11).

The cases group by what they protect:

- **Threads, signals and processes:** `dfork_signal`, `dthread_signal`, `dsignal_wait`, `dsignal_redzone`, `dsigaltstack_query`, `dthread_suspend`, `dthread_act`, `dspawn_pipe`, `dspawn_arm64_only`, `dwq_thread_exit`, `dwq_overcommit`.
- **Memory ordering and atomics:** `drel_acq_order`, `datomic_counter`, `dbyte_atomics`, `dunaligned_atomics`, `dshmem_coherence`, `dtso_narrow`.
- **Self-modifying code and real engines:** `dsmc_io`, `dsmc_high`, `dsmc_ras`, `dsmc_chain`, and `jsc`, which runs a loop of 50,000 object allocations through the system's JavaScriptCore and checks the sum. `smc_chain` (`60bd6cea`, 2026-10-11) calls a routine on another page 20,000 times, rewrites it twice and requires the calls to reach the code written last: a callee spliced into its caller lies outside the caller's own span, and until that commit a write to it left the callers running the old bytes.
- **Signals reaching a thread in a translated loop:** `loop_signal` (`a7fd0f3f`, 2026-10-14) covers seven loop shapes with plain and ordered memory and a thread that 20 `pthread_kill`s must each reach while it spins, every handler ending the loop by rewriting the interrupted context; `OCERZ_NO_SIGNAL_STOP=1` restores the old delivery (Chapter 9).
- **x87:** `x87_arith`, `x87_compare`, `x87_int`, `x87_trans` and `x87_state` check 1,759 lines of Rosetta's output (`8f310eb8`, 2026-10-07) under both engines in both layouts through `run_golden_case`; `x87_top_entry` calls an x87 loop at four stack depths and checks each result with every x87 switch off in turn (`d1679446`, 2026-10-08); `fist_rc` pins the rounding-control and C1 behaviour of `fist`.
- **Flag forwarding:** `nzcv_window` (the 7-Zip decoder's step exactly as its compiler wrote it, consumers 7, 8 and 9 instructions from a compare, `adc` and `sbb` of immediates of each sign and size) and `xlive_self` (loops whose own back edge must not keep a flag alive), both against Rosetta in both layouts, with planted bugs the commits record the tests catching.
- **The loader:** `dupward_init`, `dupward_defer`, `dinit_order`, `ddlopen_load_phase`, `ddlopen_objc_core`, `ddlopen_caller_rpath`, `ddlsym_deps`, `ddlopen_image_list`, `ddlopen_cache_alias`, `dlegacy_format`, `dapp_bundle`, `dinsert_libraries`, `dweak_unloaded`.
- **Floating point and SIMD:** `dfp_rounding`, `dnan_contexts`, `davx2_ops`, `davx_fma`, `dbmi_ops`, `dmmx_ops`, `dymm_state`.
- **Wine's address layout:** the "low" cases below.
- **The translation cache:** `dtcache` and `dtcache_low`, which run the suite's own workload twice over one store.
- **The veneer paths:** `smc_chain`, `smc_ras`, `flip_phase` and `xlive_self` run a second time with `OCERZ_FORCE_VENEER=1`, which chains every edge through a veneer, as far targets are chained in a big process. The commit that added the second pass (`bfaca27a`, 2026-10-11) first said the tests passed without the fix; `e778db6a` corrected that the same day: `smc_ras` fails with "stale return site" when unchaining does not follow the veneer, so a far chain left in place had reached the old translation after a write to guest code, a correctness bug as well as a speed one.

Low-shadow builds of guest tests

Wine's main image insists on a low address, so a Wine process runs in a third memory layout, the low shadow, where guest addresses below 12 GB live at their own host base and every memory access in translated code is checked (Chapter 8). Code paths that exist only there were long tested only by running Wine. The suite now builds selected guest tests as non-PIE mains at `0x200000000`, which activates the low shadow, and checks them against the same goldens the static gate uses: `dsimd_low`, `dmmx_low`, `dras_stress_low`, `dcallret_fault_low`, `dfault_link_low`, `dstack_test_low`, `djcc_chain_rec_low`, `drsp_ops_low`, `dfault_chain_low`, `dpromo_callout_low` and `dtso_narrow_low`. Three more, `dmac_syscall_low_stack`, `dmetal_nocopy_low` and `diosurface_low_stack`, put a thread's stack or a GPU buffer below 12 GB to pin the system-call translations that Wine depends on. The hoist and stack-pointer work of 2026-10-10 and 2026-10-11 brought two checks that compare with plain C or with Rosetta in both layouts: `low_loop_hoist` (three arrays under one index, an index that hops from a mapping below 12 GB to one far above, records walked by an offset stepped inside the loop, a store into a protected page whose handler must see the guest address) and `lowstack_ptr` (`rsp` written every way an instruction can write it and read back, stacks on both sides of 12 GB, a fault on each and a `fork` from each). Each commit lists the planted bugs its test catches; the stack-pointer one includes a fill that computed the delta in the wrong register and hung every fork in the Wine layout.

The native suite and its companions

`tests/run_native_tests.sh` is the largest gate in the tree, over fifteen thousand lines. Its central argument is in its own header: an exit code cannot prove that a bridged call worked, because a guest handed nonsense by every crossing can still exit 0. So the native cases compare *output*. Each x86-64 fixture is run under native mode with the JIT, under native mode interpreted, and under cache mode, and all three must print what a separately compiled arm64 build of the same source prints. Cache mode reaches the system libraries through the real shared cache, so agreeing with it is agreeing with the real libSystem. `xbench_dyn` is in the suite because its kernels checksum their own work: a `memcpy` that copied the wrong bytes changes the checksum, and the checksum is all it prints.

The suite also pins refusals. Native mode stops with a named message when it cannot make a call correctly (Chapter 12), and the gate checks those messages and their exit codes, 71 for an import that cannot bind and 72 for an export with no crossing, so a refusal cannot silently become a wrong answer. Result: 87 / 87 on macOS 27 on 2026-10-03, and still 87 with no failures on the Mac Studio on 2026-10-13. (On 2026-10-08, with Rosetta uninstalled by an OS update, `sys_proc` failed because its arm64 fixture spawns the x86-64 build; `d1679446` records it as 86 / 87.) The file is now 15,415 lines, 15,261 at `90fe4d8e`. `ad51f4b3` (2026-10-18) added its callback cases here.

Five companion runners extend it, all run by `make check`: `run_guest_library_tests.sh` (guest x86-64 libraries supplied in place of a database), `run_native_cxx_tests.sh` (the optional LLVM guest C++ runtime: containers, streams, regex, RTTI, exceptions, threads and futures, compared with arm64 fixtures and required to show positive JIT and bridge counts), `run_native_framework_tests.sh` (CFNetwork, SystemConfiguration, Security and CoreServices crossings, including a certificate check against an in-memory trust anchor) `run_native_format_tests.sh` (the `va_list` printf veneers past both register limits) and `run_native_swift_tests.sh` (three Swift fixtures and one C fixture compiled for x86-64 and arm64 from

the same source and module name, which must print the same under native mode, with the JIT and interpreted, as the arm64 build does running as itself; it needs the guest Swift runtime from `make guest-swift` and skips without it).

The m32 gate

i386 programs are not in `make check`, because making them needs an i386 linker that current Command Line Tools no longer have. `make check-m32` runs, in order: `tests/m32/dbcheck.sh`, spot checks that the generated i386 API database in `runtime/apis32` declares specific functions with the expected signatures (it needs `make apis32` first); `test_m32 carbon`, which crosses Carbon calls through those databases as translated code does; `tests/run_m32_tests.sh`; and a second run of just the `high8` and `shiftmem` programs with `OCERZ_NO_HIGH8_ARITH=1` and `OCERZ_NO_SHIFT_MEM_IMM=1`, so those two translations are judged against the interpreter's answer as well.

Each of the 79 `tests/m32/*.expected` files (counted with `ls` at `6e6c64b5`) belongs to a program of the same name, built from a source of that name except where noted below, and holds its standard output plus a final `exit <status>` line. The runner starts the program from `tests/m32/bin` under `ocerz -native` with a 60-second alarm and compares. The programs are committed as binaries, as the guest tests are, because linking i386 needs Xcode 26's `ld-classic`: `M32_REBUILD=1` runs `tests/m32/build.sh` first, which makes the stub `.tbd` files the linker needs from each program's own undefined symbols, since the SDK has no i386 libraries any more. They cover the layers Chapter 13 describes: C and CoreFoundation crossings (`cf`, `cfformat`, `cfuuid`), Objective-C 1 (`objc_basic`, `objc_swizzle`, `objc_struct`, `objc_exc`), OpenGL (`glarb`, `glmirror`, `glthread`), Steam (`steambridge`, against `tests/m32/steamfake`, an arm64 stand-in for Steam's client), launch scripts (`launchscript` and `execscript`, scripts rather than C sources), self-loading (`selfload` is `objc_basic` packed by UPX 5.2.1, committed as a binary and rebuilt only where `upx` is installed), and the 32-bit translations (`high8`, `shiftd`, `shiftmem`, `mmxjit`, `jccfuse`, the last a set of compare-and-branch shapes from Batman's hot code) and the thread-local slot at `gs:` (`tsdgs`). The C++ programs (`cxx`, `cxxexc`, `cxxllvm`) need a guest libstdc++ or libc++ that `tools/fetch_guest32_cxx.sh` puts in `runtime/guest32`, and are skipped without it; `steambridge` is skipped without the interface database `make steam32` writes. One more check, `native-by-default`, runs `ret42` without `-native` and requires the verbose log to say that its NaN results come from translated checks, which is how it shows that an i386 program started plainly runs in native mode all the same (`src/main.c`, `1ae3140a`, 2026-10-18).

The big programs of Chapter 13 (Batman, AssaultCube, Adium, VLC 1.1.12, Wesnoth, Nexuiz and the rest) are not in this gate: no committed fixture contains them, and their results are the hand-run claims of `docs/m32.md`. What the gate holds is the smaller programs that each pin one thing those applications needed. The gate figures from the Air on 2026-10-16, on `c64cca9f`: `test_m32` 424 passed and 0 failed, `test_m32 carbon` 347 and 0.

`tests/win` holds seven Win32 programs (`sync_stress`, `timed_wait`, `xproc_shm`, `xproc_wait`, `first_rpc`, `d3d11_present`, `smc_ret32`). They are built with a MinGW cross compiler and run under Wine by hand; no script in the tree runs them and `make check` does not. `sync_stress`, for example, hammers the wait and wake primitives Chromium and V8 park on in five phases and exits 2 with "STALL" if any phase makes no progress for eight seconds.

Real software, outside the gates

`tools/apptest.sh cli` runs sixteen system command-line tools under ocerz and compares their standard output with the native run: `uname`, `sw_vers`, `echo`, `ls`, `id`, `basename`, `wc`, `sort`, `uniq`, `head`, `grep`, `file`, `xxd`, `nm`, `plutil` and `openssl`. It reads 16 of 16 on macOS 26.7.1 since the initializer-order fix of 2026-10-03, and on macOS 27. `tools/apptest.sh gui` launches system applications; a GUI application has no output to diff, so the verdict is survival: "ALIVE" means it was still running after `SECS` seconds.

The benchmark harness, `tests/xbench_compare.py`, is also a correctness check. Each of its fifteen kernels prints a checksum of its own result, and the harness fails a kernel whose output under ocerz differs from its output under Rosetta, so a kernel that is fast because it computed the wrong answer fails rather than scores. All fifteen are bit-identical (Chapter 17).

Where expected outputs come from

A golden is only as good as its source. The project uses three, in a fixed order of preference.

1. **Rosetta.** The guest binaries run directly on an Apple-silicon Mac, where Rosetta translates them, and their output becomes the golden. Rosetta is independent of AArchX, so agreeing with it is evidence about x86 semantics rather than about the interpreter. The SIMD tests store hashes produced this way.
2. **The interpreter**, for tests whose point is the JIT: the golden is what the reference engine prints, and the test fails if the translated run differs.
3. **The architectural semantics** in the Intel manual, for tests written before Rosetta could be consulted, checked by hand; `movhlpd`'s golden was taken this way and later confirmed to match Rosetta.

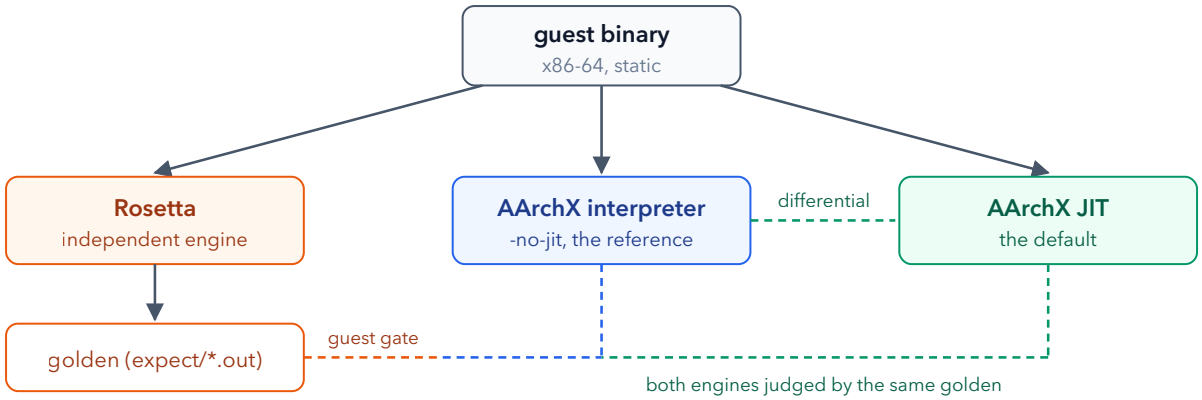


Figure 16.2. Three engines, two kinds of comparison. Goldens come from Rosetta, an engine independent of AArchX, and judge both of AArchX's engines; the differential compares the two engines directly and needs no golden at all.

Contribution rules state the principle bluntly: take expectations from a real run under Rosetta or natively, not from memory. The loader tests follow it the same way. The test for the initializer order of upward dependencies (`dupward_defer`, Chapter 11) was written by building a few libraries of the project's own in the shape that crashed `sw_vers`, logging the order in which their initializers ran under Rosetta, and making that order the expected output.

THE 2026-10-03 REWRITE

Four guest tests and three passages of `jit.c`'s explanation block had described their bugs through code in macOS's own frameworks. On 2026-10-03 (`ae3780b7`) they were rewritten to state each rule in x86 terms: `side_exit_l0` and `xf_epilogue` became new routines built from the rule they pin, `punpck_qdq` gained the REX-register forms, and `jcc_gap_fuse`'s last loop became three scans whose flag-neutral gap writes the register the compare read. The new goldens came from Rosetta. Each rewritten test was then run against a build of ocerz from just before the fix it pins, and fails there exactly as its predecessor did.

Conventions

Binaries are committed. The test binaries under `tests/guest/bin` and `tests/unit/bin` are tracked in git, so a change to the translator is measured against exactly the binaries the previous run used, and a compiler update cannot silently change what the tests exercise. Rebuilt unit binaries are committed separately, with the subject "tests: rebuild unit binaries", which is why that subject appears after nearly every change in the history.

Benchmarks live apart. `tests/guest/benchbin` holds the benchmark binaries, deliberately outside `bin`, because the differential gate globs that directory and a benchmark there would silently become an extra case with a runtime of minutes.

Tests run without the translation cache. Every runner exports `OCERZ_TCACHE=off` unless told otherwise, so no test depends on what an earlier one stored and none writes into `~/Library/Caches`. The two cases that test the cache turn it on against a directory of their own.

Do not load the machine during a run. The dynamic suite has timing-sensitive cases (thread suspension, timed waits, the unstick monitor), and the contribution guide asks that no other ocerz work be built or run beside `make check`.

Current results

Gate	Air, 2026-10-04 (90fe4d8e)	Studio, 2026-10-13	Air, 2026-10-16 (c64cca9f)
unit harnesses	0 failures, 29 programs	green	test_m32 424 / 0, carbon 347 / 0
guest suite, interpreted	133 / 133	**136 / 136**	**134 / 134**
guest suite, translated	133 / 133	**136 / 136**	**134 / 134**
x86-64 differential	99 / 99	**100 / 0**	**100 / 100**
i386 differential	20,033 / 20,033	**40,044 / 0**	**40,044, twice**
dynamic suite	184 / 184	**404 / 0**	**406 / 406**
native suite	87 / 87	**87 / 0**	not recorded
CLI tools against native	16 / 16	not recorded	not recorded
xbench output against Rosetta	15 / 15	not recorded	not recorded

The first column is the full `make check` that gated `90fe4d8e` on the MacBook Air (M5, macOS 27); its two CLI and xbench rows come from the 2026-10-03 run on the tree that merged both machines' work. The second is the Mac Studio (M2 Max, macOS 27) on the translation-speed changes of `f4ef8900` and its neighbours. The third is the Air on `c64cca9f`, the m32 commit of that day; the later 0.8 commits (`1ae3140a` to `a454f2fd`, 2026-10-18) have no recorded gate run beyond what each commit message says. The dynamic counts count each case under each engine separately; "not recorded" means no figure was kept for that run, not a failure. The Air's 134 and the Studio's 136 differ by the two Wine cases, as described above. `docs/testing.md` itself still carries a September table (131, 97, 113 and so on, and 20,033 for the i386 differential, the older figure) and is behind these runs; the README and `95db4cfc` (2026-10-18) give 40,044 in each of two layouts, which the gates of 2026-10-13 and 2026-10-16 ran; the figures here come from the commits and the recorded sessions, not from it.

Conventions

Binaries are committed. The test binaries under `tests/guest/bin`, `tests/unit/bin` and `tests/m32/bin` are tracked in git, so a change to the translator is measured against exactly the binaries the previous run used, and a compiler update cannot silently change what the tests exercise. Rebuilt unit binaries are committed separately, with the subject "tests: rebuild unit binaries", which is why that subject appears after nearly every change in the history. The unit binaries are linked without a debug map, so a committed binary does not carry a build machine's paths (`5e09eb4b`).

Benchmarks live apart. `tests/guest/benchbin` holds the benchmark binaries, deliberately outside `bin`, because the differential gate globs that directory and a benchmark there would silently become an extra case with a runtime of minutes.

Tests run without the translation cache. Every runner exports `OCERZ_TCACHE=off` unless told otherwise, so no test depends on what an earlier one stored and none writes into `~/Library/Caches`. The two cases that test the cache turn it on against a directory of their own. A test that counts translated blocks with the cache on gets a wrong count, since the persistent store hides the JIT's log lines; `f1d4592f` (2026-10-05) made one such count run with the cache off.

Do not load the machine during a run. The dynamic suite has timing-sensitive cases (thread suspension, timed waits, the unstick monitor), and the contribution guide asks that no other ocerz work be built or run beside `make check`. `docs/testing.md` puts a whole `make check` at roughly half an hour.

A switch with a check mode. New translation rules often ship with a debug switch that re-derives their answer and stops on a difference, and the commit that adds one records running the gates with it set. `OCERZ_NZCV_PROD_CHECK=1`, added with the per-block producer table of `6d7e051a`, asks the old function again at every read; the commit records the guest, differential and dynamic suites (136, 100 and 398 cases) and two large starts running with it without a difference. `OCERZ_X87_CHECK=1`, `OCERZ_LOWSTACK_CHECK=1` and `OCERZ_WLIM_CHECK=1` are of the same kind.

Adding a test

Every behavioural change carries the test that would have failed before it. Where the test goes depends on what it needs:

- **Pure instruction semantics** go in `tests/guest`: add the source, its name to `TESTS` in `tests/guest/Makefile` and to `NAMES` in `tests/run_guest_tests.sh`, and a golden from a run under Rosetta in `tests/guest/expect`. If it exercises a path only Wine's layout reaches, also add a `run_low_golden_case` line to the dynamic suite.
- **Anything that needs real libraries** (threads, signals, dyld, Mach) goes in `tests/dynamic` with a case in `tests/run_dynamic_tests.sh`. A test that should print `OK` under both Rosetta and ocerz is the usual form; where the point is an ordering or a value, the program computes it and compares with the Rosetta result it embeds.
- **Component behaviour** that no guest can reach goes in a `tests/unit` harness.
- **Native-mode crossings** go in `tests/run_native_tests.sh` as an output comparison against an arm64 build.
- **An i386 behaviour** goes in `tests/m32`: a source, its `.expected` file (the program's output and an `exit` line), a `.link` file if it needs extra link arguments, and a rebuild of `tests/m32/bin` on a machine with Xcode 26. A pure x87 or 32-bit instruction rule goes in `tests/diff32.c` instead, so both engines are compared on it.

A good test also proves it can fail. The dynamic cases for switchable features name the switch that restores the old behaviour, and record that the case fails under it: `dupward_init` fails with `OCERZ_NO_UPWARD_INIT=1`, `ddlopen_caller_rpath` with `OCERZ_NO_DLOPEN_CALLER_RPATH=1`, `dmac_syscall_low_stack` with `OCERZ_NO_MACSYS_XLATE=1`. A test that passes whether or not the fix is present pins nothing.

CHAPTER 17

Performance

AArchX is measured against Rosetta, on the same machine, running the same x86-64 binary, with the output compared byte for byte. On the project's fifteen-kernel benchmark its translated code is now faster than Rosetta's on most kernels, and its remaining losses are a handful of percent with known causes. Real applications tell a different story: what costs time there is translating code for the first time, Wine's memory layout, and a long tail of system-level work. This chapter covers the method, the results, the techniques that produced them, and the gaps.

0.72x
MEMCPY, LINKED, VS ROSETTA

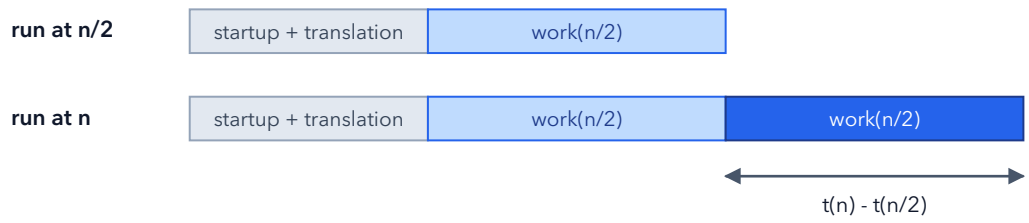
13 / 15
KERNELS UNDER 1.00X, LINKED BUILD

17 ns
A NATIVE-MODE CROSSING

45-65 fps
R.E.P.O. MENU UNDER WINE (ROSETTA ~178)

How it is measured

`tests/xbench_compare.py` runs the fifteen kernels of `xbench`, a single x86-64 program compiled from one source in two builds: a freestanding static binary that makes no library calls at all, and `xbench_dyn`, the same source linked against libSystem. For each kernel the harness first calibrates a size n at which Rosetta takes about a third of a second. It then times both engines at n and at $n/2$ and reports the difference of the two.



The difference keeps only steady-state work: process startup, mapping, dyld and the first translation of every block appear in both runs and cancel. Each kernel is run five times (REPS=5) and the median is reported, separately for AArchX and for Rosetta.

Figure 17.1. The paired-delta method. Subtracting the half-size run removes everything that does not scale with the work, so the ratio compares the engines' translated code rather than their startup.

Both engines' output is compared byte for byte. Each kernel prints a checksum of its own result, so a kernel that is fast because it computed the wrong answer fails rather than scores, and the harness doubles as a correctness check (Chapter 16). The commands:

```
python3 tests/xbench_compare.py                                    # static build, plain
memory
OCERZ_NO_PLAIN_MEM=1 python3 tests/xbench_compare.py               # ordered memory
XB=tests/guest/benchbin/xbench_dyn python3 tests/xbench_compare.py  # linked against libSystem
XB=tests/guest/benchbin/xbench_dyn OCERZ_MODE=native python3 tests/xbench_compare.py
```

Two rules govern how the numbers are read. A ratio within a couple of percent of 1.00x is a tie, which lands on either side from run to run. And the machine must be idle: the project's notes record benchmark sessions spoiled by a stuck background process that held a core for two days, and a loaded machine moves every ratio by several percent.

The kernels

Kernel	What it does	What it stresses
`icall`	indirect calls through a table	indirect branch lookup, return-address stack
`jtab`	a switch compiled to a jump table	indirect jumps, site caches
`depchain`	a dependent chain of arithmetic	register pinning, flag elision
`brmiss`	unpredictable branches	conditional branch layout, flag fusion
`memcpy`	copies of random small sizes	the system's copy routine, alignment
`str`	`strlen` and `strcmp` on short strings	call overhead into tiny routines
`hash`	multiply-shift hashing	multiply latency (a floor on both sides)
`idiv`	integer division	division and its flag rules
`fpsse`	scalar SSE floating point	scalar FP, NaN exactness
`fpvec`	packed single-precision arithmetic	packed FP, NaN checks
`chase`	pointer chasing through a shuffled list	dependent loads (a cache-latency floor)
`qsort`	sorting with a comparator	calls, callbacks, branches
`leafcall`	a tight loop of small function calls	call and return bookkeeping
`mixed`	struct updates, branches and small loops	everything at once
`vm`	a bytecode interpreter loop	indirect dispatch, loads and stores

Results

The current table was taken on an Apple M5 under macOS 27 on 2026-09-21, five repetitions per kernel. The **static** columns measure a binary in which every instruction is the program's own translated code. The **dynamic** columns are the same source linked against libSystem, which is what a real program looks like; its `memcpy` and `str` kernels spend their time in the system's own routines. **Ordered** is `OCERZ_NO_PLAIN_MEM=1`, the stricter memory model every program with a second thread runs under. **Native** binds the dynamic build against the Mac's own arm64 frameworks (Chapter 12).

Kernel	static	static, ordered	dynamic	dynamic, ordered	native
`icall`	0.92x	0.98x	0.93x	0.82x	0.84x

Kernel	static	static, ordered	dynamic	dynamic, ordered	native
`jtab`	**0.89x**	**0.86x**	**0.98x**	**0.92x**	**0.97x**
`depchain`	**0.90x**	**0.90x**	**0.90x**	**0.90x**	**0.90x**
`brmiss`	**0.97x**	**0.99x**	**0.97x**	**0.98x**	**0.96x**
`memcpy`	1.04x	1.22x	**0.72x**	**0.73x**	**0.73x**
`str`	1.05x	1.16x	**0.77x**	**0.96x**	**0.82x**
`hash`	1.00x	1.00x	**0.99x**	**0.99x**	**0.99x**
`idiv`	**0.99x**	**0.99x**	**0.99x**	**0.99x**	**0.99x**
`fpsse`	**0.86x**	**0.88x**	**0.86x**	**0.86x**	**0.85x**
`fpvec`	**0.82x**	**0.82x**	**0.80x**	**0.83x**	1.05x
`chase`	1.00x	1.23x	1.00x	1.22x	1.00x
`qsort`	**0.98x**	1.04x	**0.97x**	1.04x	**0.97x**
`leafcall`	1.03x	1.02x	1.03x	1.03x	1.03x
`mixed`	**0.88x**	**0.88x**	**0.83x**	**0.80x**	**0.90x**
`vm`	**0.85x**	**0.83x**	**0.84x**	**0.76x**	**0.73x**

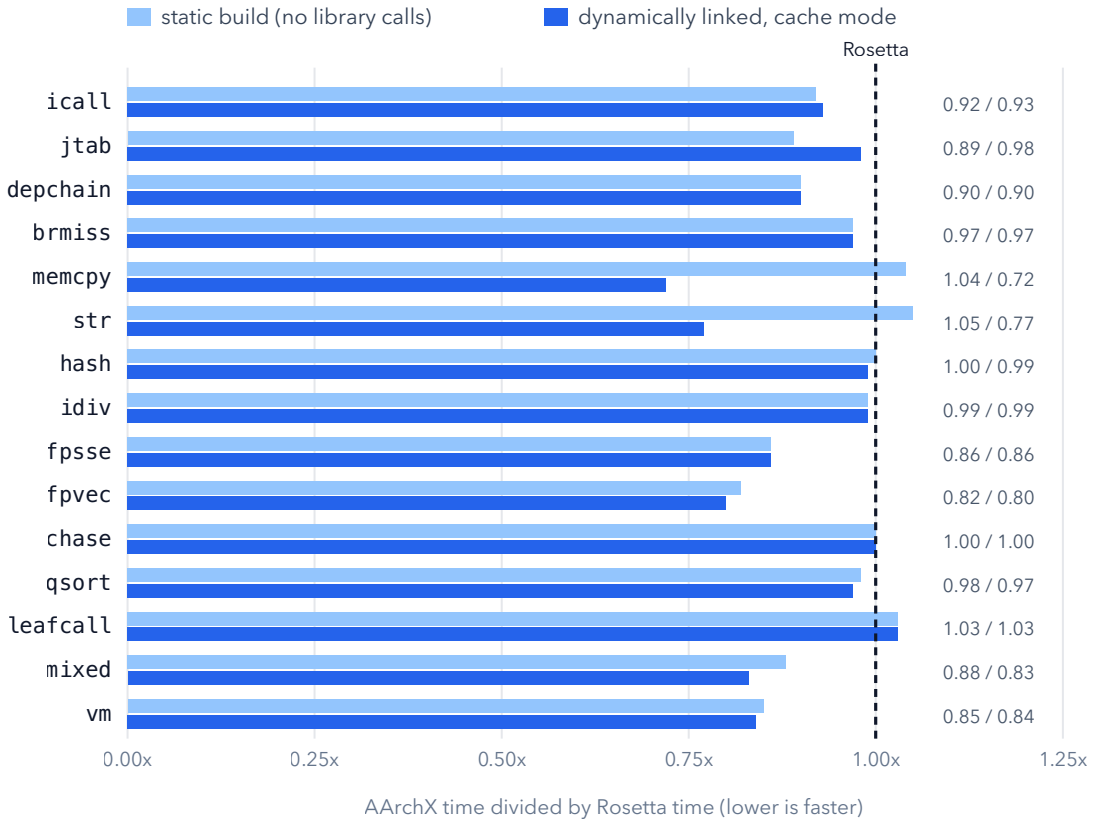


Figure 17.2. The plain-memory columns of the table: static build (light) and dynamically linked build in cache mode (dark), each as AArchX time over Rosetta time. Bars left of the dashed line are faster than Rosetta.

Rounded to two decimals, the dynamically linked build has thirteen kernels under 1.00x, `chase` at parity and `leafcall` 3% behind; the commit that produced the table counted fourteen as faster than Rosetta, so `chase` sits just below parity before rounding. The static build has ten kernels under 1.00x, `hash` and `chase` at parity and three behind by at most 5%. The history is worth knowing: the first commit to claim rough parity with Rosetta was the plain-memory tier of 2026-07-23 (`f5325e6c`); on 2026-08-17 the static build had eight wins and three ties; on 2026-09-04, on an Apple M2 Max, thirteen wins and two ties.

That table, from `docs/performance.md` as it stands at `6e6c64b5`, is the last full run of the fifteen kernels on record; nothing after 2026-09-21 repeats the whole grid. Later commits quote single kernels, on the machine of the day: `d7567fc5` (2026-10-18) took `str` from 1.074x to 0.967x and `brmiss` from 0.980x to 0.963x by retranslating a superblock whose probed branches were all kept without its probes (the probe's fall-through count had become a `nop` run on every pass), and `bench br`, a loop of an LCG and two random-bit branches, from 1.059x to 1.033x of Rosetta (2.436G instructions to 2.137G; 2.870G cycles to 2.722G against Rosetta's 2.64 to 2.74G); `39271608` (2026-10-13) cut the interpreter's CPU time on `depchain` and `mixed` to 0.888 and 0.882 of what it was by making the decoder cheaper (below). Read those as dated single measurements, not as a new table.

What makes it fast

The techniques are described in depth in Chapters 6 and 7. Seen from the benchmark table, each one moves particular kernels.

- **The guest's registers stay in registers.** All sixteen general registers and all sixteen vector registers live in fixed arm64 registers for a whole block, with one layout shared by every block, so control passes from one block's body straight into another's without touching memory. This is most of `depchain`'s 0.90x and much of `mixed`.
- **Flags are computed where they are read.** x86 writes six flags on almost every arithmetic instruction and reads them rarely. AArchX records what produced them and materialises only what a later instruction needs, and a compare followed by a branch becomes one arm64 flag-setting operation and a conditional branch.
- **Blocks are chained and grown.** Direct branches replace dispatch; a host return stack keeps the processor's return predictor aligned with the guest's calls; a block continues past a conditional branch with the rare side out of line, and is retranslated the other way round when the profile says so. `icall`, `jtab` and `vm` are the beneficiaries.
- **The processor produces x86's NaNs.** Where the processor reports `FEAT_AFP`, setting FPCR.AH makes add, subtract, multiply, divide and square root follow SSE's NaN rule, and the checks that otherwise guard every floating-point result disappear. In `fpvec` the end-of-batch check had been three vector operations on top of the eight doing the work, with the vector pipelines the limit: the kernel went from 1.06x to 0.82x on 2026-09-21, and the x86-64-v3 build's from 0.96x to 0.58x.
- **Hot library routines do not go through the library.** `strlen`, `strnlen`, `strcmp`, `strncmp`, `memcmp`, `bcmp`, `strchr`, `memchr`, `memcpy`, `memmove` and `memset` have hand-written arm64 versions in `src/leaf.s` that read their arguments from the registers that already hold `rdi`, `rsi` and `rdx` and leave the result in the one that holds `rax`, so translated code reaches one with a single direct branch and spills nothing. On 2026-09-21 that took the linked build's `str` from 1.08x to 0.77x and `memcpy` from 0.98x to 0.72x. Before them, the linked `memcpy` kernel had been 4.6x slower than Rosetta in ordered mode, because the system's own copy routines are handed buffers at any alignment and an ordered

access that straddles a 16-byte boundary takes a barrier; translating libsystem_platform's string and memory routines with plain accesses, as they are under Rosetta too, brought it to 0.98x by itself (`OCERZ_NO_MEMFN_PLAIN=1` turns that off).

- **A program pays for ordering only when it can observe it.** A process that never starts a thread, forks or maps shared writable memory runs with plain loads and stores (Chapter 7). The benchmark kernels never do, which is why the plain columns are the headline; the ordered columns show the cost when the switch has happened.
- **Translations are kept.** Translated blocks are stored on disk and loaded by every later process that runs the same code. This does nothing for the steady-state table and a great deal for startup, discussed below.

What is still slower

`hash` and `chase` are ties no translation can move. `hash` is a chain of multiply, shift and or per step, and both engines are bound by the multiplier's latency; `chase` is a chain of dependent loads, and both wait on the cache. The losses that remain have known causes:

- **`leafcall`, about 3% behind everywhere.** Both calls in the loop are already inlined into one translated block. What is left is the frame bookkeeping the x86 calling convention asks for.
- **`chase` under ordered memory, 1.22x.** A dependent load through a shifted index is one instruction on x86. Under the ordered model it becomes an acquiring load, and arm64's acquiring loads have no register-index form, so the address is computed first. That is an instruction-set floor, not a missing optimisation.
- **The static `memcpy` and `str` under ordered memory, 1.22x and 1.16x.** These are the program's own byte loops at arbitrary alignments, and an ordered access that straddles a 16-byte boundary pays a barrier.
- **`fpvec` in native mode, 1.05x.** Native mode keeps the software NaN checks, because the host's own code runs on guest threads there, and clearing and restoring FPCR.AH around every crossing would cost about as much as the crossing itself.

SIMD: AVX2, FMA and MMX

Until 2026-09-13 every VEX-encoded instruction left translated code for the interpreter, so AVX2 and FMA loops ran up to a hundred times slower than under Rosetta. Then the JIT learned the instructions these kernels spend their time in. Timings are the best of five on an Apple M5 under macOS 26.6.2, taken on 2026-09-14 and, for nbody and mandelbrot, 2026-09-15; "before" is the build at `767ecf38`.

Kernel	Before	Now	Rosetta
`memclr` 32 MB, AVX2 `vmovdqu`	24.12 ms	0.57 ms	0.56 ms
`indexbyte` 32 MB, AVX2	71.98 ms	**0.85 ms**	1.48 ms
`memeq` 32 MB, AVX2	64.79 ms	**0.86 ms**	1.72 ms
int32 loop 4M, clang AVX2	135.35 ms	**0.44 ms**	1.30 ms
int32 loop 4M, clang SSE4.1	29.48 ms	**0.56 ms**	0.78 ms

Kernel	Before	Now	Rosetta
saxpy 4M, clang AVX2+FMA	37.62 ms	**0.33 ms**	0.48 ms
nbody 200k steps, scalar SSE2	72.90 ms	6.13 ms	5.92 ms
nbody 200k steps, scalar AVX2	1125.37 ms	**6.97 ms**	10.57 ms
nbody 200k steps, scalar AVX2+FMA	904.45 ms	**6.31 ms**	9.47 ms
mandelbrot 400x400, scalar SSE2	19.97 ms	12.07 ms	11.26 ms
mandelbrot 400x400, scalar AVX2	828.24 ms	**10.95 ms**	11.44 ms

The first three are hand-written loops shaped like Go's runtime routines, which matter because Go turns its AVX2 paths on under Rosetta without checking CPUID; the rest are C loops that clang vectorises, except nbody and mandelbrot, which stay scalar. Scalar floating-point loops now take 0.9 to 1.2 times Rosetta's time whether built for SSE2 or for x86-64-v3, because scalar results stay in host lane registers across a loop instead of being merged into the guest register after every operation, and 256-bit loops keep the upper halves of their `ymm` registers in host registers too. The whole fifteen-kernel suite built for x86-64-v3 used to lose ten kernels, three of them by 4x to 13x; on 2026-09-15 it won twelve and lost none by more than 7%.

MMX followed on 2026-10-04 (`b7449a73`). UnityPlayer's video decoder is written in MMX, and in R.E.P.O.'s menu it cost about 4% of the process in the interpreter. A loop of the decoder's moves, adds, multiplies and packs now runs in 0.142 s instead of 1.395 s. Rosetta takes 0.053 s: AArchX keeps the MMX registers in the CPU structure, so every MMX instruction loads and stores its operands, which is the next step if it matters.

x87

32-bit programs do their floating-point arithmetic on the x87 stack, so x87 was where the 32-bit speed gap against Rosetta lay (issue #6). The first state of that story was wrong values rather than slow ones: a comparison with Rosetta on 2026-10-07 (`8f310eb8`) found `fadd st(0), st(i)` computing `st(0)` op `st(0)` (10 + 3 came out as 20), `fcomi` comparing a register with itself, `fcmovcc` copying a register onto itself and `fxam` decoded as `fxch`; the rewrite gave every register a double plus an exact 80-bit image valid while the double is still that image rounded (Chapter 7). Speed came in the next commits, the first measured on an M2 Max and the others on the machine of the day:

- `6c6679f0` (2026-10-07) translated the common instructions. Before it every x87 instruction left translated code through the slow call, spilling and reloading every pinned register. On the new `x87` kernel (22 x87 instructions per element: loads, multiply-adds, `fcomip`/`fcmov` clamps, `fistp`, `fstp`), paired delta for 10,000 rounds: 1.147 s before, 0.161 s after, Rosetta 0.731 s, so 4.5 times faster than Rosetta; the precision-control-24 variant 1.233 s, 0.171 s and 0.756 s. `fsw` stays exact without writing FPSR, whose writes cost 15 ns each: the inexact flag is computed out of line with error-free transformations. The commit records trying to keep ST(i) in vector registers across a run and measuring it 6% slower, so that was left out.
- `9f0b7087` (2026-10-07) did the same in 32-bit blocks.
- `2295ec7f` (2026-10-07) took the control word and TOP off the dependency chain: every instruction had loaded the 8-byte word the one before had just stored, so the tag and image bits lay on the path to

the next address. A run loads it once; TOP inside a run is the run's first TOP plus the pushes and pops counted at translation. `winbench`'s i686 double kernel, under Wine: 2,160 ms before, 1,066 ms after, against 13,050 ms under Rosetta and 483 ms under Rosetta with a separate x87 loader; the float-arithmetic structs kernel 867 to 509 ms (Rosetta 2,041, with the loader 136). By the commit's own numbers AArchX is well ahead of plain Rosetta there and still behind Rosetta with the loader.

- `d1679446` (2026-10-08) keeps the registers of a run on a known TOP in eight vector lanes: dependent x87 adds went from 3.50 to 1.62 ns, independent ones from 1.14 to 1.00; the x87 kernel of `diff32 --bench --low` from 15.0 to 9.3 ns an iteration and the double kernel from 25.5 to 20.0. The commit states that `winbench32` itself was not run, because the test bottle and Rosetta were both gone after the macOS 27 update.

Each step has a switch that restores the older code (`OCERZ_NO_X87_LIVE`, `OCERZ_NO_X87_TOPSPEC`, `OCERZ_NO_X87_LANES`, `OCERZ_NO_X87_KCARRY`, `OCERZ_NO_X87_ST_ELIDE`) and is gated by the 40,044-case i386 differential (Chapter 16).

Exactness and its price

Bit-exact x86 floating point is not free on arm64. The two architectures disagree about NaNs: an invalid operation gives a negative default NaN on one and a positive one on the other, and when both operands are NaNs x86 returns the first where arm64 lets a signalling one win. Programs do notice. `mixed` was a 1.20x loss for a long time, and the whole gap was the cost of checking every packed result before anything could use it.

The fix came in stages. First the check was deferred to the compares that read a value. Then it became one batch per run of floating-point work (2026-09-15, `a16dc15b`): moves, unpacks, stores and `ucomisd` with its branch stay inside the batch, a stored value is checked just before the store, a branch out of the loop carries its check in the exit stub, and only the registers the loop still reads are checked at the end. nbody's SSE2 pair loop went from 107 to 86 host instructions per iteration, 42 of which had been NaN bookkeeping. A store that might alias an earlier load of the batch used to end it, because the replay re-executes the loads; the batch now keeps the memory it is about to overwrite in a spare vector register, so the replay can restore it. Finally the hardware rule above removed the checks altogether on processors with `FEAT_AFP`, except for fused multiply-add, whose negated forms negate a NaN operand that x86 returns as it came.

Memory ordering

A program that has created a second thread, forked or mapped shared memory retires plain mode for good and pays for x86-TSO ordering on every scalar load and store; Wine is always in that mode. Scalar accesses use acquire and release forms, because flags, locks and atomics are scalar, and a release store orders every earlier vector store. SSE loads and stores are left plain, which is also what FEX ships, because ordering them cost 3.3x on `memcpy` and 3.0x on `fpvec`. That one decision, on 2026-09-05 (`36930d64`), took ordered `memcpy` from 3.55x to 1.35x of Rosetta and `fpvec` from 3.67x to 0.99x on an M2 Max.

The narrow SSE moves were left plain too, and on 2026-10-04 a test (`dtso_narrow`) showed that a pointer published through `movq` can be seen out of order hundreds to over a thousand times in three million handoffs, where x86 never shows it. `OCERZ_TSO_NARROW=1` orders `movd` and `movq` without a barrier, but it is not the default: it took R.E.P.O.'s menu from 65 frames a second to 44, and no program is known to depend on it.

Native-mode crossings

In native mode a call from x86 code into a system library crosses into arm64 code (Chapter 12). Measured as a guest loop making ten million calls and timing itself, on the M5:

Guest loop, ns per iteration	through the trap	default	arm64 build
`getpid`	40.5	17.0	0.7
`strlen` of a 12-byte string	42.0	**2.0**	0.9
`memcpy` of 48 bytes	45.6	**3.0**	2.7
`-[NSString length]`	51.3	28.3	3.6
a `qsort` comparator, per call back into guest code	426	430	3.1

"Through the trap" is the original path: the stub traps, the dispatcher finds the bridge, the call is made and the run loop resumes. "Default" calls the bridge from inside the translated block and returns with a real `ret`, keeping the return predictor in step. `strlen` and `memcpy` are low because they do not cross at all; with `OCERZ_NO_LEAF_INPLACE=1` they read 17.3 and 20.8 ns. The callback direction is the expensive one, about 430 ns in this September table, and most of it was three system calls that save and clear the signal mask and alternate-stack state for the callback's fault-recovery point. That was changed on 2026-10-18, with the cost of Objective-C sends, as described under "Callbacks and sends" below. Against cache mode the cost shows only in code that crosses in a tight loop: the linked build's `str` reads 1.03x of cache mode, while kernels that make no calls are at parity.

Wine's layout

Wine's main image insists on a low address, so a Wine process runs in the low shadow (Chapter 8): guest addresses below 12 GB live at their own host base, and every memory access in translated code needs a check to find out which base applies. Until 2026-10-04 that check built three 64-bit constants and tested three ranges, about twenty instructions per load, and the kernels built into that layout ran up to six times slower than under Rosetta. The low shadow also keeps the fast memory forms, base hoisting, paired moves and the batch undo log off, because they need a host address formed without any check.

`1b636806` made the common case short. When the low window's host base has no bits below 16 GB and is encodable as an arm64 logical immediate, an address below 12 GB becomes a host address with a shift, a compare and one `orr`, and an address between 12 GB and the top strip is recognised as identity with two shifts and an add; only the top strip, which holds the commpage, still takes the general form. A debug build that ran both forms at every access and compared the results never saw them differ.

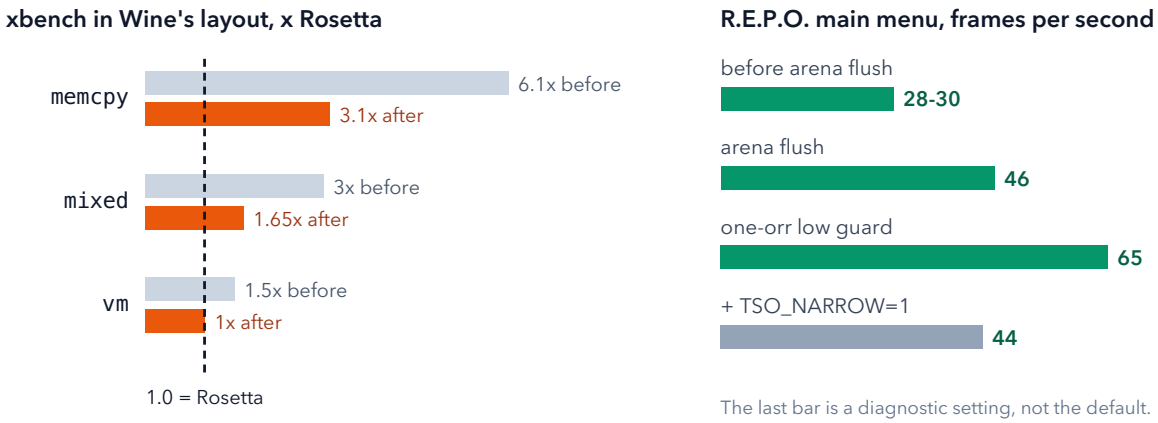


Figure 17.3. Left: three kernels built into Wine's address layout, before and after the short low-window check of 2026-10-04. Right: R.E.P.O.'s main menu under Wine, after the arena flush and then the short check; the grey bar is `OCERZ_TSO_NARROW=1`, which is correct in a case no program is known to need and costs a third of the frame rate.

Before the arena flush of the same day (`7b82e872`) existed, the long checks filled the 1 GB code arena while R.E.P.O. was still loading, and the game crashed at its first scene change in every run. The arena flush matters for speed in its own right: Unity's Mono JIT keeps generating code, and once the arena filled every new block ran interpreted. Flushing it and starting again at its front, about once a minute, took the menu from 28 to 30 frames a second to 46.

The hoist and the stack pointer, 2026-10-08 to 2026-10-11

The short check still ran at every access, and a loop over arrays carries one on each. `ff3e02ea` (2026-10-10) made the base hoist, which settles the low-window conversion once for a base register, work on the shapes loops have: up to three bases a block, operands indexed by one register and scale (the block checks the index each time round, in two instructions), and bases on either side of 12 GB, with a block first translated for the usual case and retranslated in a wide form if a check fails. On `xbench` built as the tests build Wine-layout programs, time relative to the same kernel outside the layout fell from 2.63 to 1.07 on `fpvec` and from 1.42 to 1.18 on `mixed`; the other kernels did not move. In a test loop, 316 blocks had been bailing out of the hoist and being translated a second time, nearly all of them shared-cache code holding addresses above 12 GB; now 17 are.

`ab86121d` (also 2026-10-10) made rsp's register hold the host address rsp points at, so that push, pop, call, ret and rsp-relative operands are single pre- and post-indexed accesses instead of an add and an access; `17bfc689` (2026-10-09) moved rsp once for a run of up to 16 pushes or pops where each had waited for the write-back of the one before (a native replica of `fib`'s prologue ran 0.315 s and then 0.21 s). The commit's own measurement is sober: instructions retired in the Wine layout fell 6.8% on an object-update loop, 9.5% on recursive `fib` and 2.7% on Lua 5.4 running a mixed script, but wall time moved less than one percent (0.333 to 0.331 s, 0.108 to 0.107 s, 0.233 to 0.232 s), because the conversions it removed were not on the loops' critical paths. The object loop takes 0.278 s in the identity layout, so the rest of the Wine layout's cost is elsewhere.

7-Zip under Wine

7-Zip is the project's current stand-in for compute-bound code in Wine's layout, and the instructive case, because every clean kernel of the benchmark table is faster than Rosetta and a real program that is not a clean kernel can still lose. Measurements on 7-Zip 25.01 under Wine Staging 11.18, MacBook Air M5, one thread, 2026-10-14: compression about 8,200 KiB/s under AArchX against 11,600 KiB/s under Rosetta, and a 26 MB archive took 6.85 s against 4.8 s, so AArchX takes about 1.4 times as long. 70% of the compression is LZMA's binary-tree match finder.

The decompressor was attacked first, with `7z b -mmt1 -md22`, and Rosetta decompresses at 124,000 KiB/s on the same benchmark in each commit message. Four commits show what the work consisted of, each time from a profile of the hottest block:

- `e853c450` (2026-10-11). The LZMA decoder decodes each bit with a `sub` and then two `cmovae`, two `cmovb` and an `sbb`, the last eight instructions after the `sub`, with moves between them and the conditional moves rewriting the `sub`'s operands. Only consumers within three instructions took the `sub`'s NZCV; the others read the flag record back and rebuilt the carry, 15 to 18 instructions each, so the `sub` wrote its record too, and over half the code in the hottest blocks was these. The window grew from 3 to 8 (`OCERZ_NZCV_GAP`); each `cmov` became a `csel`, the `sbb` four instructions. Decompression 58,634 to 64,167 KiB/s.
- `a8773e55` (2026-10-11) let a plain load sit in that window, rebuilding NZCV from the producer's record after it (`OCERZ_NO_NZCV_GAP_LOAD`): 64,005 to 70,805 KiB/s.
- `d68898a5` (2026-10-11): `sbb r, -1`, the decoder's symbol step, with the carry a condition on NZCV, is one `csinc` instead of a `cset` and two adds (`OCERZ_NO_ADC_CINC`): 70,549 to 71,325 KiB/s.
- `5fc69e49` (2026-10-11): the LZ copy loop, `mov byte [r14+rdx], bl; movzx ebx, byte [rdi+rdx]; inc rdx; jne`, was the hottest block of decompression at 18% of the time, and 136 of 150 samples in `ocerz_flags_materialize` came from its back edge. The flag look-ahead followed the loop's own back edge like any other, found every flag live past its depth limit, and so kept CF alive round every trip of a loop in which nothing reads CF; the arm back to the start then spilled and filled every pinned register around a C call on each iteration. A block that loops to its own start now begins with no flags live and iterates to the least fixed point (`OCERZ_NO_XLIVE_SELF`): 71,586 to 85,141 KiB/s.

The figures within one commit share a baseline and the figures across commits do not (each pair was measured with its own two or three runs), so they cannot be chained into one series; read end to end, the figures run from 58,634 KiB/s, 47% of Rosetta's 124,000, to 85,141, 69% of it, over the four commits of one day.

Compression is another matter, and the question there was ordering. Wine always needs ordered memory, and it costs 7-Zip's compression about 17% against plain memory; the archive came out identical either way. Split into parts, in 7-Zip's own benchmark the acquire loads cost 14% and the release stores 2.6%, so the barriers themselves are the smaller part. In the real run the barriers cost only 1 to 3%, and most of the rest is code generation: the ordered load and store forms of arm64 take no index register, so every indexed access in the match finder needs its address formed first (the same floor as `chase` above).

A smaller finding from the same week, `2cf48a7e` (2026-10-13), was a correctness bug that also made loops slow: a block that loops on itself and carries a scalar double round the loop keeps it in a fixed lane, but the emitters for a loop ending in `dec` and `jnz` left their lanes unflushed on both arms, so `addsd`/

`mulsd` / `dec rcx` / `jne` over 2,048 doubles gave a different result from the interpreter's, and had since the fixed lanes arrived on 2026-08-25. The repair sends such a loop straight back to its loop entry, which is also faster: 1.99 ns an element against 2.64 through the chained edge on an M2 Max.

> **AN OPEN LEAD, NOT A RESULT**
>
> A function-churn benchmark under Wine, run on the MacBook Air on 2026-10-18 with Node, took 2.2 times Rosetta's time, and 90% of it was in translated V8 code rather than in translation. Where in that code the time goes has not been found.

Startup: where real applications lose

The benchmark table measures steady state. Real applications are dominated by something the table deliberately subtracts: translating code for the first time. Windows Steam under Wine starts a dozen processes that each run the same Wine and system code, and on 2026-09-27 the sampling profiler (Chapter 18) split one startup as follows: of 54 s sampled, translation was 12.6 s across 2.18 million blocks with almost no churn, Windows code 14 s in translated code, and 3.3 s in `libswiftCore` walking every protocol conformance record because the shared cache's precomputed Swift tables are not offered to it.

The response came in two kinds. The first made translation itself cheaper:

- cross-block flag liveness memoises its successor decodes, which had been a fifth of translation time: 5.78 to 4.85 μs per block over a Steam startup (`2987ca8a`);
- a logical-immediate encoding is refused by counting bit transitions before searching, where the search had been a tenth of translation time (`e1ce350c`);
- a block retranslated with its branch flipped leaves the live array by a recorded index instead of a search and a multi-megabyte `memmove`, which had been about 330 ms of every busy process's startup (`215d127f`);
- retiring invalidated blocks walks only their recorded predecessors instead of every live block's edges, which had been 18% of a CEF process's samples (`e2ae267b`).

The second avoided translating at all. Since 2026-09-27 (`93547a4c`, on by default from `08a0458b`) translated blocks are stored on disk and shared between processes. A record names the guest byte ranges the translation read and their hash, which a load checks first; code is emitted so it can move, with every process-specific value a fixed instruction sequence or literal with a relocation record; and only code that sits at the same address every run is kept, which is all of a Wine process and the shared cache otherwise. Records are LZ4-compressed and written by a background thread.

Measurement	Value
Windows Steam to its main window, empty store	46.9 s
the same, warm store (two runs)	**35.2 s, 33.7 s**
the same under Rosetta	11-12 s
`hostname.exe` cold, before / after the store's overheads were cut	0.78 s / 0.61 s
`hostname.exe` with nothing kept	0.58 s
store size after a Steam session	2.1 GB

Other startup costs that were found and removed along the way show how much of the gap is ordinary system work done slowly. The shared cache's selector table was being indexed by hand, thirteen seconds of startup for every process that canonicalised a selector, until it was read from the cache header (2026-09-21). Electron Framework's 2,560 weak-coalescing binds each walked all 3,600 cache images at 1.8 ms apiece, 4.6 s in every Discord process, until the walk was limited to the 300 images that define weak symbols and backed by a bloom filter (2026-09-23). A `dlsym` benchmark of 4,500 framework look-ups took 8.17 s, against Rosetta's 0.21 s, until symbol lookup moved onto a per-process index; it now takes 0.12 s (2026-09-27). With those and the JIT forms of the same week, the two busiest Steam web helpers three and a half minutes after launch went from 197% and 167% of a core to 31% and 10%, and the whole settled Wine process tree uses about a tenth of a core.

Translation time, 2026-10-13

The store only helps the second run. The first run of anything is translation, and on 2026-10-13 a profile on the Mac Studio (M2 Max, macOS 27; sampling at 1 ms, perl and ruby starting in cache mode with the store off, 1,771 to 1,864 samples under `translate`) found four places where it spent its time, all of them in work that produced no code. Taking the cold start of the system's perl loading ten modules, with `OCERZ_TCACHE=off` and `OCERZ_JITMEASURE=1`'s total at 32,768 blocks: about 275 ms of translation that morning and 208 ms after `f4ef8900`; ruby loading its standard library went from 185 ms to 168 ms. These are the figures of the commit messages, best or median of several interleaved runs, on one machine.

- **The decoder** (`39271608`). It runs for every instruction the interpreter executes and several times for each one the JIT translates: the block itself, the successors the flag look-ahead reads, the callees a splice inlines, the fusion look-aheads. A decode cost about 276 arm64 instructions, 158 of them in `ocerz_decode_mode`'s own code rather than any opcode's handler: setting up eighteen integer fields, copying prefix state in and out of the decode state around the prefix loop, a range test and a switch for every byte, and a pass over the operands for a RIP-relative fixup on every instruction. A byte is now classified as a prefix or not by one table load (`prefix_kind`), the prefixes are gathered in locals, the one-byte opcode map is a single switch, and the RIP-relative fixup runs only when `decode_modrm` says it produced such an operand. Counted under qemu over 600 KB of code from bash, libc, python3 and libclang-cpp, a decode takes 213.8 instructions where it took 276.4 in long mode (191.1 against 247.9 in 32-bit mode); over 3.46 million instructions on an x86-64 host it is 1.20 times faster in long mode and 1.18 in 32-bit mode. The interpreter's CPU time on `depchain` and `mixed` is 0.888 and 0.882 of what it was; a cold start improves by between 1% and 3%. Nothing a decode produces changed, and the commit's evidence is large: `decodiff` identical over its 16,777,216 openings, and a harness that linked both decoders and compared every field of the result at every byte offset of 13.7 MB of code in both modes, plus 20 million random byte strings, 49,071,175 decodes with no difference (Chapter 4).
- **The flag look-ahead** (`e34ad68f`). To know which flags are live where a block starts, translation decodes the block's successors and follows their targets down to `OCERZ_XLIVE_DEPTH`. The profile put the look-ahead's decoding at 20% of translation time and 57.6 decodes per translated block. The observation that removes most of them: the fold gives a flag at the entry only if some instruction reads it before any earlier one writes it, or if it is live out and nothing writes it. Once the instructions decoded so far have written all six flags, neither can happen for anything later, so the decode loop now stops at that instruction and the block it hands on ends in a non-terminator, which the code after the loop already treats as every flag live out (`OCERZ_NO_XLIVE_KILL=1` reads every successor to its end). The proof is by exhaustion: from every byte offset of bash's and libc's text, 4.5 million block starts, the loop stopped early in 64% to 66%, and the fold over the prefix agreed with the fold over the whole

block under 64 live-out values each, 187 million comparisons, none different. Decodes per translated block fell from 57.6 to 26.4 for perl and from 43.2 to 23.8 for ruby; the look-ahead's share of translation time from 20.0% to 7.5%; perl's translation from 275 to 237 ms best of three (277 to 242 median) and ruby's from 207 to 181 best (207 to 185 median). With the translation store recording, a record's byte ranges now stop where its look-ahead stopped, which is all its liveness depends on.

- **The producer table** (`6d7e051a`). The fused compare and branch, the flag-forwarding window and the liveness pass each asked, for every instruction, which earlier instruction produces the flags it consumes, scanning back through the window each time (`nzcv_fuse_producer` was 8.6% of translation time inclusive, with `cc_consumer_inline_ok` 4.7% self). The answer depends on the instructions and on state translation sets before the liveness pass and does not change until the block is finished, so `nzcv_prod_fill` now works every instruction's producer out once, and everything reads the table through `nzcv_producer_of`. The function fell from 9.8% of translation time to 2.2%, plus 2.1% for filling the table. `OCERZ_NZCV_PROD_CHECK=1` asks the old function again at every read and stops on a difference; the commit ran the guest, differential and dynamic suites (136, 100 and 398 cases) and two large starts with it set without one. With the chain-install change below, perl's translation was 208 ms against 232 and ruby's 168 against 185.
- **The chain install** (`f4ef8900`). `sys_icache_invalidate` was 10.5% of translation time, about half of it under `blk_chain_install`, and the profile showed that on Apple silicon a call over one word costs about what a call over a whole block costs: the price is the barriers, not the cache lines. The install called it once for every word it had patched, the chained branches and short conditional branches of the new block and of every pending predecessor it drained. `chain_batch_end` now syncs them together with the sequence the stop-site code already used: `dc cvau` on each word, one `dsb ish`, `ic ivau` on each word, then `dsb ish` and `isb`. Every word is still invalidated and the barriers still complete before the install returns, still under the JIT lock. Perl: 208 ms against 215 with `OCERZ_NO_CHAIN_SYNC_BATCH=1`, ruby 168 against 171, best of seven interleaved runs; the flushes leave `sys_icache_invalidate` and come back as 5.0% inside `blk_chain_install`, because a barrier pair costs about what a call did, so what is saved is one pair a block and what is left is one.

Two smaller changes of the same week belong to the same effort: `OCERZ_RET_FLAGS_DEAD`, `OCERZ_FPB_DBGPRINT` (`669bf3df`) and `OCERZ_DLSYM_UPWARD` (`36f4297f`) are read once instead of at every use, as the other switches already were. The pattern across all four is the book's usual one: nothing about the generated code changed, a profile of the translator pointed at the work around it, and each fix is exact and checked by exhaustion or by a switch that re-derives the old answer.

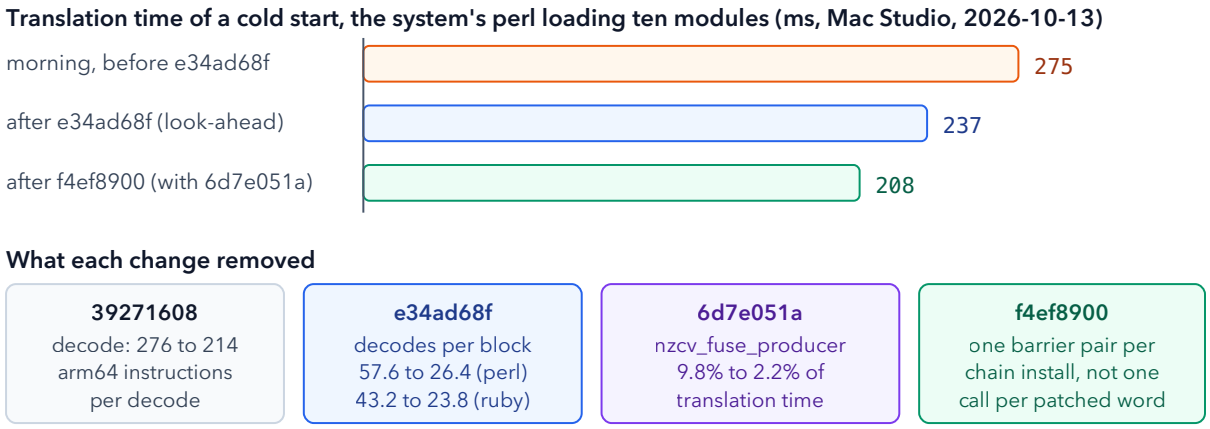


Figure 17.4. Translation time of a start with nothing cached, and the four changes behind it. The bars are the Studio's figures for the system's perl (`OCERZ_TCACHE=off`); the boxes give each commit's own before and after for the quantity it changed.

Games

Brawlhalla, an Adobe AIR game launched from Steam, was the first game measured against Rosetta, on 2026-09-22. Its first frame took 5.3 s against 0.54 s, and its main thread saturated a core where Rosetta used a quarter to half of one. Three changes that evening are instructive because the profile pointed at each one:

- A return whose host return-stack entry had been lost left translated code through the epilogue, and every trip out wiped all entries still outstanding, so the whole call chain then missed on its way up: 3.2 billion misses in 88 s. A miss now stays in the body through the inline hash probe an indirect call uses. Throughput went from 2.6-2.9 to 3.85 billion guest instructions a second.
- A chain is a `b` instruction patched into the exiting block, and it reaches 128 MB. Script code generated late could not reach engine helpers translated early, and the patch was dropped silently, so a quarter of the thread's time was in prologues and epilogues. A 256 KB veneer pool every 64 MB of arena now catches every chain that does not reach; the thread fell from a saturated core to 71% of one while executing 60% more blocks a second.
- A rip-relative or absolute memory operand has a constant address, so its commpage guard and alignment test are answered at translation time: four host words instead of fourteen or eighteen, and 13% more blocks a second.

R.E.P.O., a Unity game, is the current workload under Wine with DXMT. Its Mono JIT patches call sites in code it has already run more than a thousand times a minute, each a write to a protected code page. Invalidation used to round each write out to a 64 KB granule and wipe every thread's return stack and every site cache; since 2026-10-04 only the blocks overlapping the written range are retired, found through per-granule lists, and only the cached entries pointing into them are cleared. A benchmark that rewrites one small function at a time among 300,000 translated ones went from 37.9 to 24.7 μs per rewrite (Rosetta: 14.7).

Native mode on a real application

On 2026-09-21 the x86-64 macOS Steam client came up in native mode on macOS 27. From the web helper starting to the main window took 23 s in native mode and 34 s in cache mode on the same machine, 44 s from launch; the arm64 Steam takes 2 to 4 s. In native mode the system frameworks run as arm64 code and only Steam's own code is translated, which is the likely reason native mode was the faster of the two there.

Callbacks and sends

Native mode has two expensive directions, and the first week of 2026-10-18 cut both. They matter because every Cocoa delegate, CoreFoundation callback and AppKit message to an m32 program is a call from native code into guest code, and every message the guest sends to a native object is a send through the bridge.

Callbacks into guest code (`ad51f4b3`, 2026-10-18). A call from native code into guest code cost 536 ns in native mode where Rosetta runs the same comparator in 4.2 ns inside the x86 `qsort`. The sampling profiler put 59% of the 536 in three system calls and 18% in copying the 8 KB CPU structure:

- The host signal mask was read on every callback to clear what a host thread had blocked. Only a thread's outermost guest loop needs that: a callback nested inside `run_cpu` or another callback is called by a native function the guest called, which leaves the mask alone. A per-thread count of guest loops decides.
- The recovery point, for the rare jump back out of a faulting callback, was taken with `sigsetjmp(jb, 1)`, which saves the mask and asks for the signal-stack state at every callback. It is now taken without them, and `callback_recovered` does what the long jump did: it empties the mask and, unless the point is on the signal stack, takes the thread off it with `sigreturn`'s `UC_RESET_ALT_STACK`, which only clears that flag, as Apple's published XNU source defines it (`bsd/dev/arm/unix_signal.c`) and as libc's long jump does. Over the 338 recoveries of `signal_callback_fault` the thread had been marked on the signal stack before each and off it after.
- The callback's CPU copies its caller's less the return-address stack, which starts empty, and the system-call ring: 2.2 KB instead of 8.2.
- Its exit had added 62 signal counters back with atomics when they were zero, `ocerz_apply_mxcsr_round` had called `fesetround`, which wrote FPCR every time, and the guest profiler's start had read its environment variable on every callback.

A `qsort` of 200,000 integers with a guest comparator went from 536 ns a comparison to 100-112 ns. `OCERZ_NO_CHEAP_CALLBACK=1` restores the mask clearing and the saving recovery point. That is the figure to compare with the 430 ns row of the September table, which was measured on another day with another harness; the two are not one series.

Objective-C sends (`beed19af`, 2026-10-18). Every `objc_msgSend` from guest code in native mode asked libobjc for the implementation, asked whether it was a guest function bound into the callback bank or guest code, looked the method's signature up, fetched the selector's name and checked it against the selectors that need converting, and labelled the bridge frame twice, before the message was sent: 44 to 50 ns for a send to a native object, against 2.7 ns under Rosetta. What depends only on the class, the selector and the implementation libobjc answers now lives in a per-thread table, `t_ob_fast`: whether that imple-

mentation is guest code to jump to and, for a method with nothing to convert beyond its registers, its signature and selector name. A repeated send asks libobjc for the implementation and makes the call. A global generation counter, bumped by every bridged change to a class's methods, retires an entry, and a different implementation for the pair misses it. Super sends, logging, variadic methods, methods taking function pointers or objects to convert, and the selectors that answer IMPs or bundles take the full path as before. Results: `[NSString length]` 49.5 to 40.9 ns, `[NSArray count]` 45.5 to 39.4 ns, a guest class's method 37.4 to 35.8 ns; `OCERZ_NO_OBJC_SEND_CACHE=1` decides every send in full. A send is still about fifteen times Rosetta's 2.7 ns.

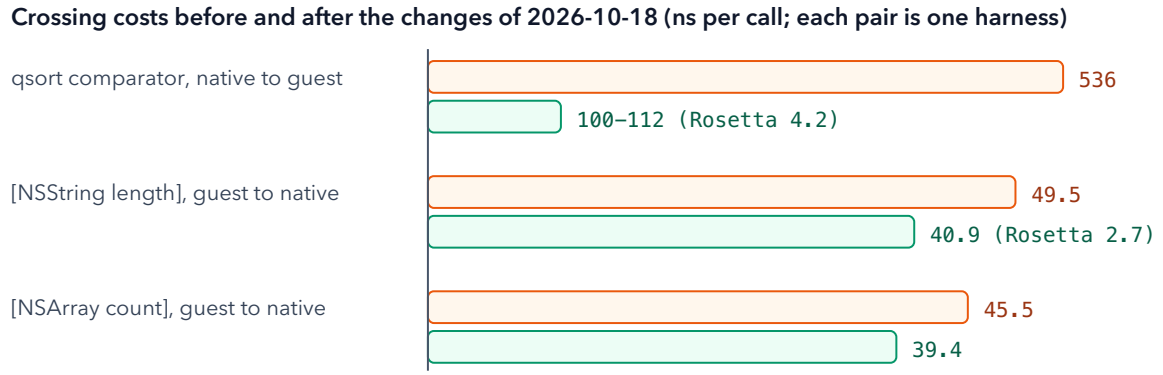


Figure 17.5. Orange: before; green: after. The callback pair is drawn on a different scale from the two send pairs, whose bars share one. The 2.7 ns Rosetta figure for a send is the one the commit quotes for a send to a native object.

32-bit Mac programs

m32 (Chapter 13) runs on the same JIT, so what makes i386 code fast is mostly JIT work, and it was tracked by one game. `docs/m32.md` gives Batman: Arkham Asylum's New Game opening at about 61 frames a second on an M4 Max, from 31 before the 32-bit speed work of 2026-10-10 (recorded in `c716841f`, which updated the README and the documentation). What got it there is listed in the document: shifts by `cl`, MMX, `shld` and `shrd`, and the high-byte registers (`ah`, `ch`, `dh`, `bh`) translated in 32-bit code instead of interpreted; an import stub calling its export from inside the block (`emit_bridge_fastcall32`); and small libc, CGL and OpenGL exports called as leaves with no crossing at all (`src/m32_leaf.c`). The switches that turn each off are `OCERZ_NO_M32_FASTCALL`, `OCERZ_NO_M32_LEAF` and `OCERZ_NO_HIGH8_ARITH`.

The next round, on 2026-10-18 (`1ae3140a`), measured Nexuiz 2.5.2's benchmark demo (3,104 frames): about 20% fewer cycles, 133.6 to 138.8 G before and about 107 G after, with the frame rate on an idle M5 going from about 100 to 114. Three generic changes made it. A function pointer to an export (`dlsym`'s, an OpenGL extension's) is its trap, and a call through it had jumped into the trap window, which sent the dispatch stub back to the run loop, out of translated code and in again for every call; DarkPlaces calls all of OpenGL that way, about 250,000 calls a second. A trap is now translated as the import stub it stands for, so the call is made from inside the block (`OCERZ_NO_M32_TRAP_BLOCKS=1` turns it off). Fast calls no longer wait for native mode, since m32 has no x86 shared cache. And an i386 program now runs in native mode by default: in cache mode the NaN results from FPCR.AH cost m32 two FPCR writes a call, about 8.5 ns each on an M5 (an unchanged write is 2.5 ns, a read 0.34), where native mode uses translated NaN checks and no writes, for 8% fewer cycles.

Reading these numbers honestly

The kernels are microbenchmarks. Each isolates one thing, which is what makes it useful for finding a cost and what makes it a poor predictor of how long an application takes to start or how smoothly it runs. The real-application figures are single measurements on one machine, recorded in commit messages on the day; they show direction and size, not a reproducible score. And everything here is relative to Rosetta on the same hardware: an M5 that runs both engines faster moves no ratio.

CHAPTER 18

The debugging toolkit

A translator's bugs are rarely where they surface. A miscompiled block in a browser engine shows up minutes later as "out of memory" from SQLite; a wrong decode shows up as windows 74 pixels high; an initializer run in the wrong order shows up as a stack overflow in `malloc`. AArchX is built for that. It has a fixed method for narrowing any failure to one cause, and some five hundred switches that each turn one optimisation off, print one kind of event, or select one range. This chapter is the method, the instruments, and six failures traced with them from symptom to fix.

545
OCERZ_ VARIABLES READ AT 6E6C64B5 (410 AT 90FE4D8E)

216
OCERZ_NO_* SWITCHES (124 BEFORE)

94
*LOG DIAGNOSTICS (87 BEFORE)

~1 h
TO FIND ONE BAD BLOCK IN LIBCEF

The method

The contribution guide states the workflow the tree is built for, and it is worth following in order, because each step either names the cause or halves the space it can be in.

1. **Run it under `-no-jit`.** The interpreter is the reference implementation. If the program works interpreted and fails translated, the bug is in the JIT. If it fails both ways, the cause is in the decoder, the interpreter's semantics, the loader, the system-call layer, or the libraries' environment.
2. **Get a control.** Run the same binary under Rosetta, or natively if it has an arm64 slice. A failure that also happens there is not AArchX's. A value that differs is the first concrete fact.
3. **Shrink it.** Reproduce the failure in the smallest program that shows it. Several of the fixes in this book were found with probes that run in five seconds instead of Steam launches that take six minutes.
4. **Narrow with switches.** For a JIT bug, bisect addresses first and optimisations second. For everything else, turn on the log family for the subsystem and read what it says before forming a theory.

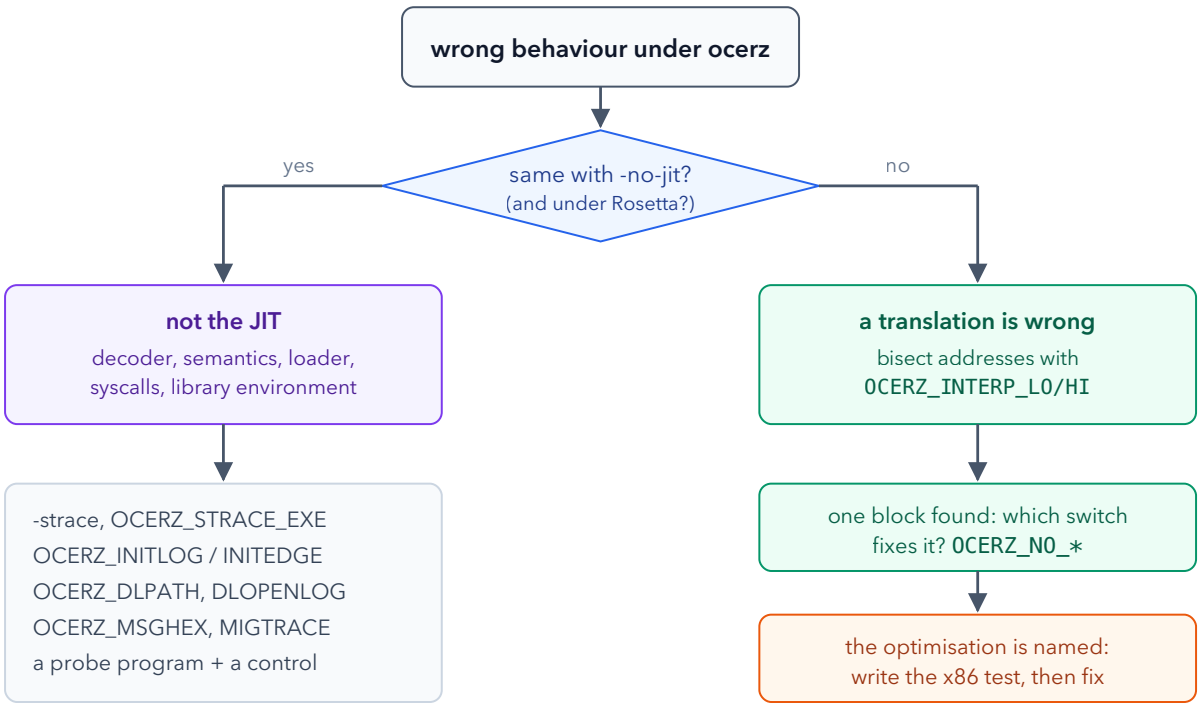
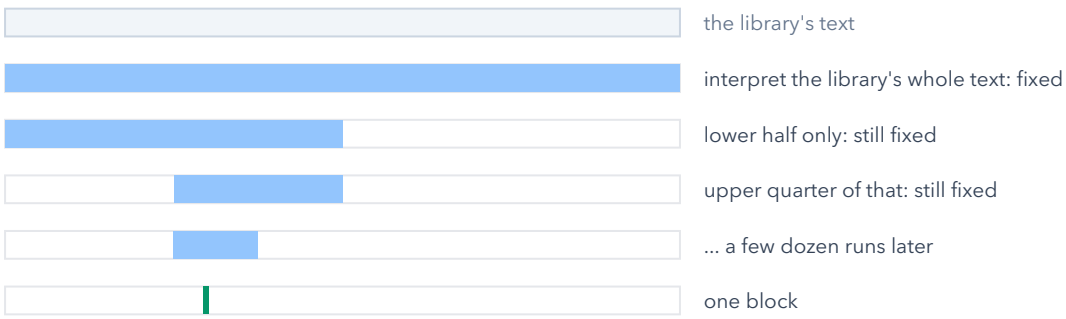


Figure 18.1. The narrowing procedure. The first branch decides which half of the system to look at; the right half ends in a named optimisation and a test written from the instruction semantics, the left half in the subsystem's own log.

Bisecting a JIT bug

Addresses first

When a program works under `-no-jit` and fails translated, some block is wrong, and the first job is to find which one. `OCERZ_INTERP_LO` and `OCERZ_INTERP_HI` name a guest address range whose blocks the JIT declines, so they run in the interpreter; `OCERZ_INTERP_LO2` / `HI2` name a second range. Start with the range of the image under suspicion, typically one library's text, and confirm that interpreting it fixes the program. Then halve the range and keep the half that still fixes it. A few dozen runs take a range of tens of megabytes down to a single block. `OCERZ_INTERP_RIP` then keeps named block starts interpreted, which confirms the block and lets every other block run translated at full speed.



Shaded: the range kept in the interpreter with OCERZ_INTERP_LO/HI.

Figure 18.2. An address bisection. The range whose blocks are interpreted shrinks by half each run, keeping whichever half still fixes the program, until a single translated block is left to blame.

The same idea applies to features that are not "translate or not". Several fixes added their own range switch so the bisection could work on that one decision: the bug that painted every Wine window black was found with `OCERZ_RETFL_LO/HI`, which kept flags live across returns only inside the range, until one return remained.

Optimisations second

With the block known, the next question is which optimisation miscompiles it. The JIT reads 78 `OCERZ_NO_*` switches at `90fe4d8e` and 142 at `6e6c64b5` (the distinct `"OCERZ_NO_...` names in `src/jit.c`, counted with `git grep`), each turning off exactly one transformation: chaining, side-exit fusion, flag forwarding, branch flips, deferred NaN checks, `mov` fusion, lane caches, the in-place string routines, and so on. The 64 added since `90fe4d8e` follow the work of Chapters 6 and 7: the x87 translation and its stages (`OCERZ_NO_JIT_X87`, `OCERZ_NO_X87_LIVE`, `OCERZ_NO_X87_TOPSPEC`, `OCERZ_NO_X87_LANES`, `OCERZ_NO_X87_KCARRY`, `OCERZ_NO_X87_ST_ELIDE`), the Wine layout's hoist and stack handling (`OCERZ_NO_LOW_HOIST`, `OCERZ_NO_LOW_HOIST_INDEX`, `OCERZ_NO_LOW_STACK_PTR`, `OCERZ_NO_LAZY_RSP`, `OCERZ_NO_STACK_RUN`), the flag-forwarding window (`OCERZ_NO_NZCV_GAP_LOAD`, `OCERZ_NO_NZCV_SIBLING`, `OCERZ_NO_ADC_CINC`, `OCERZ_NO_XLIVE_SELF`, `OCERZ_NO_XLIVE_KILL`), translation time (`OCERZ_NO_CHAIN_SYNC_BATCH`), signals (`OCERZ_NO_SIGNAL_STOP`), chains and retirement (`OCERZ_NO_VENEER_UNCHAIN`, `OCERZ_NO_FLIP_KEEP_RETIRE`, `OCERZ_NO_INCDEC_LOOP`) and callbacks and sends (`OCERZ_NO_CHEAP_CALLBACK`, `OCERZ_NO_OBJC_SEND_CACHE`). The convention, visible in the commits quoted in Chapter 17, is that a transformation arrives with its switch. Running the failing program once per switch, or bisecting the set, names the one that fixes it. A switch that fixes the program names the bug's location; it is not a workaround to ship.

For Wine and Steam, where one failing process sits among a dozen healthy ones and turning a feature off for all of them changes timing enough to hide the bug, the switches can be scoped to one process. `OCERZ_EXE_ENV="cs2.exe:OCERZ_NO_CHAIN=1"` sets variables only in processes whose command line contains `cs2.exe`, and the setting is inherited through every exec in Wine's chain. `OCERZ_NOJIT_EXE` and `OCERZ_STRACE_EXE` do the same for the interpreter and the system-call trace.

Looking at the translation

`OCERZ_JITDIS=<path>` writes every translated block to `<path>.<pid>` as the arm64 words ocerz emitted, labelled by the guest instruction each group came from; `OCERZ_JITDIS_LO/HI` limit it to a range and `OCERZ_JITDIS_BRIEF=1` leaves out the hex words. The dump has been fully buffered since 2026-09-22, because the line-buffered version made a write per line, starved the main thread on the JIT lock, and could not be profiled under. `OCERZ_CHAINCHECK`, `OCERZ_INVMAP_CHECK` and `OCERZ_TCACHE=verify` are validators: they check chaining, the code-region map and stored translations against their own invariants as the program runs. Four more validators arrived with the optimisations of October. `OCERZ_NZCV_PROD_CHECK=1` asks, at every use, which instruction's flags a consumer takes by the old function and stops with both answers if they differ from the per-block table that replaced it (`6d7e051a`). `OCERZ_X87_CHECK=1` traps where the x87 state word in its register and in memory disagree, or the TOP counted at translation and the real one do. `OCERZ_LOWSTACK_CHECK=1` traps where, in the Wine layout, the stack-delta register, the CPU's copy of it and rsp's host pointer disagree, and `OCERZ_WLIM_CHECK=1` emits a breakpoint at each guarded access if the register that holds the 12 GB limit no longer holds it. A validator is the right tool when a transformation's correctness is a claim about state the program never prints.

Some switches only make a rare path common. `OCERZ_FORCE_VENEER=1` chains every block through a veneer, as only far targets are in a big process, so the veneer paths run in small tests (the dynamic suite runs four of its cases that way; Chapter 16). `OCERZ_LIVE_SCAN=1` searches the whole live list for a block being retired whose index does not point back at it, the old behaviour that cost one browser process 29.4 s of CPU in `bfaca27a`'s measurement. `OCERZ_LOW_HOIST_WIDE=1` starts every block of a Wine process in the hoist's wide form, `OCERZ_LOW_HOIST_LOG=1` names each block that takes the hoist, moves to the wide form or gives it up, and `OCERZ_LOW_HOIST_SHARE=<n>` sets how many operands must share an index for the block to check it once a trip (default 2; 3 before 2026-10-11).

> **BLACK-BOX BY RULE**
>
> When the block a bisection finds is inside one of Apple's libraries, the investigation stops at the block. The bisection has already named the faulty translation by address and by the optimisation whose switch fixes it, and that is enough to write a test from the x86 semantics alone (Chapter 16) and fix the translator. Reading the library's instructions is neither needed nor done.

Reading a crash report

When a guest thread faults and no guest handler takes the signal, ocerz prints a crash report and exits 139. Each line answers one question.

```
ocerz: guest crash[<pid>] cpu#0x0 SIGBUS host_addr=... host_pc=... host_insn=... host_lr=...
slide=... ocerz_base=...
  host-x: ...                                  the arm64 registers at the fault
 host_bt= guest_rip=... guest_addr=... icount=... cur_rip=... interp_once=... exec_state=...
  regs: rax=... rcx=... rdx=... rbx=... rsi=... rdi=... rbp=... r8=...
  gs_base=... fs_base=... insn@rip-24= ... [bytes at the faulting rip]
  r12=... r13=... r14=... r15=...
  blockhist: ...                               the last guest blocks entered
  fault-page: UNCOMMITTED si_code=... host_prot=... region=[...)
  rip-region=[...) prot=...
  rbp-chain: ...
  bt: ...
    frame 0x... /usr/lib/system/libsystem_malloc.dylib+0x...
```

- **The first line** is the host's view: the signal, the host address that faulted, the host instruction and its word, the link register, and `ocerz_base`, the address ocerz itself is loaded at. Subtracting it from `host_pc` gives an offset that `atos -l` turns into a function in ocerz, which says whether the fault was in translated code, the interpreter, or ocerz's own C.
- **`guest_rip` and `guest_addr`** are the guest's view: the instruction that faulted and the address it touched. `icount` counts interpreted instructions only, so a fully translated loop never advances it.
- **`regs` and `r12`-`r15`** are the full guest register file, recovered from the arm64 registers the JIT keeps them in. The callee-saved registers are printed deliberately: "a register the ABI says survives a call did not" is a whole class of emulation bug.
- **`insn@rip-24`** shows the bytes around the faulting instruction, with the instruction itself in brackets; a decode bug shows here.
- **`blockhist`** is the guest addresses of the last blocks entered, the path that led to the fault.

- **`fault-page`** says whether the faulting address was committed guest memory, uncommitted, or outside the arena altogether, with its host protection. "UNCOMMITTED" at an address just below a stack is a stack overflow; "outside-arena" is a wild pointer.
- **`rbp-chain`, `bt` and the `frame` lines** are the guest backtrace, each frame named by image and offset. A short repeating pair of frames is unbounded recursion.

`OCERZ_ABORT_REPORT=1` covers a case the report above never sees: host code that aborts without a word, an assertion in a framework or an uncaught exception whose log is gone. With the variable set, `SIGABRT` is caught, the host frames it came from and the guest instruction its thread was at are printed, and the abort then proceeds as before (`src/vm.c`).

Other fatal lines name their own cause. `decode failed (... mode) at rip=...` with the bytes means the decoder met an instruction it does not know. `guest breakpoint/interrupt` is an `int3` or `ud2` in guest code, usually an assertion in the program; the report then shows where it was raised. `fatal on guest thread cpu#N; exiting process <pid> "<command line>"` names the process, because under Wine a dozen processes share one standard error. `BRIDGE-FAULT ... inside a bridged call, not in guest code` is native mode's own report (Chapter 12): it names the bridged function, its signature, and whether the faulting address was guest memory (the guest passed a bad pointer) or not (ocerz marshalled the call wrongly). `IMAGE-CLOBBER` is printed, always, when a guest mapping lands on a loaded image or a protection change takes execute away from one; a library unmapped under running code otherwise surfaces only much later as a wild jump. The `WILD-` family (`WILD-LOOKUP`, `WILD-STACK`, `WILD-FAULT-AV`, `WILD-WORKER-TERMINATE`) reports indirect branches and faults whose target lies outside anything the guest mapped.

A hung process

Hangs are harder than crashes, because nothing prints. Three instruments cover them.

`kill -INFO <pid>`. SIGINFO is always armed. It dumps every guest thread: the host thread id, the guest `rip` and `rsp`, the system call it is blocked in, and, in a Wine process, its Windows-side stack. Reading that stack means knowing where Wine keeps things: the TEB is found through a thread-specific-data slot, and the PE caller's registers sit in the syscall frame Wine saves in the TEB, because Wine runs its Unix-side code on a separate stack. A hung Wine process can be inspected this way without attaching a debugger, which matters because the debugger often cannot attach to it.

`OCERZ_JITLOCKLOG=1` reports any wait for the translation lock longer than three seconds, with the holder's thread, where it took the lock, its guest `rip` and its kernel thread state. It found the stalls in Steam's web helper where a thread holding the lock had been suspended. `OCERZ_FLUSHLOG=1` logs every arena flush and names the threads a flush is waiting on.

Mach conversations. A guest thread parked in `mach_msg` or `kevent` is usually waiting for a reply that never came, not stuck in translation. `OCERZ_MACHMSG` prints Mach messages as they pass; `OCERZ_MSGHEX` prints a round trip exactly as the guest saw it, which separates "the kernel replied wrongly" from "the guest read the reply wrongly"; `OCERZ_PORTDUMP` names the conversation behind a lost-wakeup wedge; `OCERZ_ULOCKLOG` traces `ulock` waits and wakes with their owners.

Where the time goes

Switch	What it reports
`OCERZ_PERFSTAT=1`	at exit: blocks translated and executed, instructions executed in translated code and in the slow path, the hottest blocks, the return-address-stack line (misses, slot-pool use, call sites without a slot) and the chain line (chains activated, veneered, out of range). Since 2026-10-04 (`d8af2a18`) return-address misses are split into an empty stack, a null entry and a mismatched address, with the `ret` sites that miss most; on R.E.P.O.'s menu 183.6 of 184.1 million misses were the empty stack, which costs a cache probe rather than a trip out
`OCERZ_JITMEASURE=1`	after every 16,384 blocks translated, how many there were and the time translating them took, in all and for shared-cache code. The cold-start figures of Chapter 17 (275 to 208 ms for perl) are its `xlat_total` at 32,768 blocks, with `OCERZ_TCACHE=off`
`OCERZ_XLIVELOG`	the flag look-ahead's decisions; since `e34ad68f` the byte count it reports stops where the look-ahead stopped
`OCERZ_ORDERLOG=1`	what first made the process use ordered memory, with a backtrace
`OCERZ_GUESTPROF=<usec>`	a sampler that suspends each running guest thread at a jittered interval, reads its host pc and credits the place with the CPU time used since the last look; every `OCERZ_GUESTPROF_PERIOD` seconds (default 10) and at exit it prints the hottest guest addresses with their image, the hottest host symbols, the hottest interpreted instruction forms, the translator's own counts and time, and CPU per image
`OCERZ_TRIPSTAT=1`	exits from translated code to the dispatcher per second, and the twenty commonest destinations with the state of the block at each
`OCERZ_FPS=1`	frames per second in cache mode, counted at the block that begins `CGLFlushDrawable`
`OCERZ_BLACKLOG=1` , `OCERZ_INVSRC=1`	pages refused translation because they keep changing, and which code invalidated them
`OCERZ_XLATPAGES=1`	every distinct 4 KB page a process translates
`OCERZ_TCACHE_LOG=1`	per process, blocks loaded from and stored to the translation cache; `OCERZ_TCACHE_MIN_FREE_MB` sets the free disk space the stores leave (10,240 MB by default; `93b919f9` , 2026-10-07, after a tester's Mac filled up under a Steam session, could not swap and panicked on the watchdog)

GUESTPROF deserves a note on why it weights by CPU time rather than counting samples. Wine's device service process has a poller that sleeps a millisecond at a time and woke on the sampler's own timer ticks: a plain sample count billed it 39% of a core while `ps` measured 4.7%. Crediting each sample with the CPU the thread actually used since the last look removes that bias. The per-form output, the opcode, operand kinds and size of each instruction the interpreter executes, is how the Wine-only JIT gaps of 2026-09-27 were found: Steam's web helper was interpreting `pinsrw` from memory for 15% of its samples, and after those forms were translated the process spent 78-81% of its samples in translated code instead of 47-67%.

Loader and library diagnostics

Switch	What it shows
`OCERZ_DLPATH=1`	every `dlopen` and `dlsym` failure with its reason; unlike `-v`, inherited by child processes
`OCERZ_DLOPENLOG`, `OCERZ_DLSYMLOG=1`	each `dlopen` request and result with its caller; each `dlsym` with handle, result and caller
`OCERZ_INITLOG`, `OCERZ_INITTRACE`, `OCERZ_INITEDGE`	every initializer run, every image the initializer walk enters or skips, and every dependency edge it follows, upward edges marked
`OCERZ_ALLMISS=1`, `OCERZ_DYNLOOKUPLOG=1`	every unresolved import rather than the first 24; symbols imported with `-undefined dynamic_lookup` that nothing defines
`OCERZ_DYLDAPI_TRACE=1`	every call into the dyld API table, with its slot, arguments and caller (Chapter 11)
`OCERZ_EXCLOG=1`	every Objective-C and C++ exception thrown, with the throw site and the exception's reason

System calls, Mach and IOKit

`-strace` traces every guest system call of one process; `OCERZ_STRACE_EXE=<text>` traces only the processes whose command line matches. `OCERZ_MACSYSLOG=1` prints each `__mac_syscall` with its policy, number and argument words. `OCERZ_FAULTLOG=1` prints the mapping state around every guest fault, and `OCERZ_WINEFAULTLOG` adds the faulting thread's first thread-specific-data slots next to its TEB, which tells a missing TEB from a wrong `gs`. `OCERZ_MODELOG=1` prints every far transfer with its selector and target mode, the WoW64 32/64-bit switches. For IOKit, `OCERZ_MIGTRACE` traces the translation of Mach interface messages, `OCERZ_IOKITSEL=<selector>` (or `all`) prints every `io_connect_method` call with that selector, its scalar inputs, the first words of the reply and the guest's `rip`. `OCERZ_IOKITERR=1` prints every such call that comes back with `kIOReturnVMError` or `kIOReturnBadArgument`, with its selector, structure input and out-of-line buffers, which is what names the field a translation missed, and `OCERZ_MSGNEEDLE=<value>` reports every outgoing Mach message that carries a given 64-bit value, which is how the untranslated address in Metal's no-copy buffers was found.

For a value that is wrong without anything crashing, `OCERZ_WATCH` and `OCERZ_WATCHBT` watch a guest address and print who writes it, `OCERZ_REGTRAP` stops at a guest address and prints the registers and the frame-pointer return chain (`OCERZ_REGTRAP_MAX=N` stops after N hits, since a trap in a recursing function otherwise prints until the stack runs out), and `OCERZ_PEEK` answers the first question worth asking when a guest assertion fires on a constant: was this global already wrong before the guest ran?

Two utilities make Wine's process tree manageable. `OCERZ_STDERR_FILE=<file>` appends every process's standard error to one file, and every fatal line carries the process id and command line, so the interleaved output can be separated afterwards.

Diagnosing a 32-bit program

An i386 program under m32 (Chapter 13) fails in ways a 64-bit one does not: an import that never resolved, a crossing that marshalled a structure in the wrong layout, an Objective-C message that went to the wrong twin, a game that quits quietly because one of its own checks failed. The instruments are one switch with many kinds, a few that name what to watch, and a handful of GL tools. All are read in `src/m32*.c`; the table in `docs/m32.md` lists them.

`OCERZ_M32LOG` takes a comma-separated list of kinds, each tested with a substring match. At `6e6c64b5` the source reads these:

Kind	What it prints
`imports`	each import as it resolves
`images`	each image the loader maps
`calls`	every crossing: the function, its string arguments, and, with `frames` as well, the guest's EBP chain, so you see who is calling. `calls` (or `exit`) also names who ended the process, with its caller chain, since games quit quietly when a check fails
`callbacks`	each callback the host makes into guest code, by slot, with the guest function and its word count
`objc`	each Objective-C message the guest sends: selector, receiver and whether the receiver is a host object, a guest class, a proxy or a handle
`throws`	C++ throws and host exceptions raised into the guest
`dlsym`	each `dlsym` that found nothing, with its caller
`files`	which files the guest asks for
`steam`	the Steam bridge: the client it opened and the SDK its interface versions come from, each interface it bridged, every call to a method it cannot cross, each callback it hands the game
`render`	each audio unit render callback the host makes, with its frames and bytes
`cond`, `sched`, `yield`	condition-variable waits and signals by thread with their callers; what a game asks of `pthread_setschedparam`; and, every 4,096th call, who is spinning in `sched_yield`
`vmregion`, `memalign`	the first 64 `vm_region_recurse_64` answers and every 20,000th after; each failed `posix_memalign` and every 100,000th call
`glstat`	how often each OpenGL and CGL call ran in the last ten seconds, most first, which is what a frame does
`glerr`, `glnan`	the GL errors each call leaves and why a shader or program did not build (it drains the error flag the game would read, and stops after 5,000 lines); shader constants that are not finite or absurdly large, the first sign of a translated maths bug when the 3D scene draws nothing while the 2D interface is fine (it stops after 2,000 lines)
`framedump`, `drawtrace`	every 5 seconds the frame about to be shown, read back from the back buffer into `ocerz-frame-<n>.tga` in `OCERZ_M32_FRAMEDIR` (default `/tmp`), so a frame can be looked at without screen capture; and one whole frame, the first after `OCERZ_M32_TRACE_AT` seconds (default 400) following a frame of more than 200 draws, with the framebuffer, viewport, colour and depth at its centre and the vertex program's first constants after each draw

Three other switches pick what to watch. `OCERZ_M32_CALLLOG=<name part>[|<part>...]` prints, for each import whose name contains a part, its first four argument words, its caller and the value left in `eax`; `OCERZ_M32_CALLLOG_DESC=<k>` adds argument `k` described by CoreFoundation (for CF-object arguments), and `OCERZ_M32_CFDESC=<name part>` with `OCERZ_M32LOG=calls` describes what matching calls return. `OCERZ_M32_SENDLOG=<selector part>[|<part>...]` prints each message to a host object whose selector contains a part, with its arguments (a string's characters too) and result, and then every message the objects it returned receive, which is how to see who retains and releases an object. And `OCERZ_M32_HUD=<overlay dylib>` loads an FPS overlay inside the process rather than through `DYLD_INSERT_LIBRARIES`, which every child inherits (Batman then ran a shell and quit), and draws it on each frame the game presents, through `CGLFlushDrawable` or by swizzling `-[NSOpenGLContext flushBuffer]` for games that never import it (Left 4 Dead 2); it prints whether it loaded.

Two more are for measurement. `OCERZ_M32_ACTIVATE=1` brings the game's application to the front five seconds after launch, as a Finder launch would; started from a shell it stays behind the active app, and macOS then runs a background app on the efficiency cores, which makes frame rates meaningless. `OCERZ_M32_SWAPINTERVAL=<n>` replaces the swap interval a game asks for (vsync quantises a frame to 60/*n* frames a second; the game's own request is logged once), and `OCERZ_M32_KEEP_GOING=1` runs a program with unresolved imports, which then return 0, so you can see how far it gets. `OCERZ_M32_SHADERDUMP=<dir>` writes each shader source as `shader-<n>.glsl`.

The switch for each m32 feature is the other half of the toolkit, as in the JIT: `OCERZ_NO_M32_TRAP_BLOCKS`, `OCERZ_NO_M32_FASTCALL`, `OCERZ_NO_M32_LEAF`, `OCERZ_NO_M32_PACKED`, `OCERZ_NO_M32_KVC`, `OCERZ_NO_M32_REAP`, `OCERZ_NO_M32_CARBON_WRAP` and the rest of the `OCERZ_NO_M32_*` family, 21 of them at `6e6c64b5`, each restores one earlier behaviour. `OCERZ_GUEST32_ROOT` and `OCERZ_APIDB32` say where the guest's system libraries and the i386 API databases are looked for. A program started without `-native` is run in native mode regardless (`1ae3140a`); `-v` says so and notes that NaN results come from translated checks.

Native-mode diagnostics

Native mode has its own set, listed in `docs/reference.md`. `OCERZ_BRIDGESTAT=1` prints at exit how many times each bridged function was called, over how many of the bridged exports. `OCERZ_BRIDGELOG=1` names every bridged call as it happens with the guest's return address and first argument; `OCERZ_BRIDGELOG_MATCH="CFBundle|stat"` narrows it to calls whose name contains a word, with the first three arguments (the first two as text when they point at a C string), the result and the caller, which traces a few APIs without slowing a game to a crawl, and `OCERZ_TRACE_CHAIN` adds the guest frame-pointer chain. `OCERZ_SUSPLOG=1` logs each `thread_suspend` and `thread_resume` a guest makes. `OCERZ_STUB_MISSING=1` binds any import that does not resolve to a stub answering 0 or nil, each named as it binds, so a program that would exit 71 can be run to see what it does next; `OCERZ_NO_STUB_REMOVED=1` refuses the system libraries Intel Macs had and Apple silicon does not. `OCERZ_POLICYLOG=1` logs `__mac_syscall` policy calls, and `OCERZ_PINLOG=1` prints each memory range as it is pinned (`OCERZ_NO_LOW_PIN=1` turns that off). `OCERZ_FORK_CALLBACK_REFUSE=1` goes back to refusing a `fork` inside a callback, which is performed since `f2e350ab` (2026-10-13; the case was ClipGrab running yt-dlp).

The messages and exit codes are those of `docs/troubleshooting.md` and Appendix B: 71 for an import with no bridge (the library has no database, or the database does not list the export), 72 for a bound export with no crossing, 64 for a static image given to native mode, 65 for a file that cannot be read or has

no x86-64 slice, 70 for a program that prints nothing because it could not get its arena or its stack, and 139 for a guest crash or a bridge fault. The first line beginning `ocerz:` is nearly always the one that matters; an exit 72 often runs the program's own exit handlers on the way out and produces a second crash report, and the line just above it is the real one. For a hang, `sample <pid>` shows where the threads are, and a thread parked in `mach_msg` or `kevent` is usually waiting for something that never arrived rather than stuck in translation.

Case studies

The six failures below were each traced with the instruments above; the first four are from September and early October, the last two from the following two weeks. They are told from the outside, as the instruments showed them.

One miscompiled block in a browser engine (2026-09-06)

Symptom. Steam's Windows client, running under Wine with Chromium's embedded browser, could not open any of its browser profile databases: SQLite reported "out of memory" for every one, the browser retried forever, and no renderer ever started. Under Rosetta none of this happened.

Narrowing. Under `-no-jit` the browser worked, so a translation was wrong. An address bisection with `OCERZ_INTERP_LO` / `HI` over the browser library's text halved the range until one block remained, in SQLite's B-tree cell walk inside the browser library. A bisection over the switches then found two that each fixed it: `OCERZ_NO_JCCFUSE`, which stops fusing a compare with its branch, and `OCERZ_NO_CHAIN`, which stops chaining. The whole hunt took about an hour.

Cause. The block ended in a fused compare and conditional branch, `cmp ebp,0xb` then `jbe`, and the branch target began with a `ja` that read the same flags. The fused branch recorded the producer's flags on its taken side in a stub, and when the edge was chained the branch was retargeted straight at the successor, past the stub. The successor's `ja` read a stale record. The bug had come in with conditional-branch retargeting on 2026-08-15 (`cd04e9e3`) and was fixed on 2026-09-06 (`8ce197f2`): a stub that records flags, replays lane flushes or checks a floating-point batch must stay on the path. The regression test, `jcc_chain_rec`, puts four producer shapes behind side exits with alternating inputs so that a stale record shows; it fails 30 of its 48 checks without the fix.

Windows 74 pixels high (2026-09-08)

Symptom. Steam's login window logged its size as 700 by 74, and every titled Cocoa window in every emulated process was squashed to the same height.

Narrowing. Steam takes six minutes to reach that point, so the first step was a probe: a small AppKit program that prints `NSScreen.visibleFrame`, run under ocerz in five seconds. It printed a 74-pixel-high frame, under the JIT and under the interpreter alike, so this was not a JIT bug. `OCERZ_MSGHEX` then showed the window server's reply as the guest received it, and the reply was correct. A correct reply turned into a wrong value by both engines points at the one thing they share: the decoder.

Cause. Opcode `66 0F 6D` is `punpckhqdq`, but a default arm in the decoder had mapped both `0F 6C` and `0F 6D` to `punpcklqdq` since the initial commit. The high quadword of a rectangle never moved down, so width and height came back as copies of x and y. Fixed in `0dd7e7e1`; `punpck_qdq` covers every form. Three five-second probes had replaced a day of six-minute Steam runs.

A stack overflow in `malloc` (2026-10-03)

Symptom. `sw_vers`, a two-line Objective-C tool, crashed on macOS 26.7.1 under ocerz. The crash report's backtrace was two frames in `libsystem_malloc` repeating, `malloc` and `malloc_zone_malloc` calling each other, and the fault page was uncommitted memory just below the stack: unbounded recursion.

Narrowing. `-no-jit` crashed the same way, so the JIT was not involved. None of the initializer-order switches changed it. `OCERZ_INITLOG` showed which initializers had run before the crash: libc++abi, libc++, libswiftCore, libswift_Concurrency, DebugSymbols, CoreDisplay and SkyLight, and not CoreFoundation, which `sw_vers` links directly. `OCERZ_INITEDGE` printed every dependency edge the walk followed, and following them back from SkyLight gave the chain: CoreFoundation depends on libobjc; libobjc links libswiftCore *upward*; libswiftCore links Foundation upward; and Foundation's dependencies lead to SkyLight. So SkyLight was being initialized while CoreFoundation was still unfinished on the walk's stack, and its first allocation went through CoreFoundation's allocator before CoreFoundation had set it up.

What real dyld does. AArchX followed each upward link as soon as the library declaring it had been initialized. To learn the right rule without reading Apple's loader, the fix was an experiment: three small libraries of the project's own, `core` depending on `mid`, `mid` linking `top` upward and `top` depending on `core`, each initializer logging its name. Under Rosetta the order was `mid core main top`: the upward target runs last, after the program's own initializer. The old ocerz printed `mid top core main`. Upward targets now wait until the whole walk from the program is done (`baf668f3`), and the experiment became the test `dupward_defer`. `sw_vers` prints what the native tool prints, and `tools/apptest.sh cli` went from 15 to 16 of 16 on that machine.

A nil drawable in a Unity game (2026-10-03)

Symptom. R.E.P.O., a Unity game running under Wine with DXMT, aborted on a nil Metal drawable in about two launches of three.

First cause. Metal frees a buffer and maps the next one at the same host address. Under Wine the guest sees such memory through an alias in the low shadow, and the alias kept the freed buffer's memory object alive: the guest read the previous compute job's results, Metal's completion state went stale, and a job could wait forever. Each alias is now recorded with the memory object it shows, and an IOKit reply that points into an alias whose host memory has changed re-aliases it (`13e7b14a`; `OCERZ_NO_ALIAS_REFRESH=1` turns it off). The game rendered, but the abort still came intermittently.

Second cause. `OCERZ_IOKITSEL` printed the IOKit calls that fetch a new drawable's properties. The buffer size in the failing calls was garbage: four gigabytes, with the right size in its low bits, and the kernel refused it with `kIOReturnVMError`. Creating an IOSurface returns two records inside memory the kernel shares with the process, at addresses that are 8-byte aligned but not page aligned. The IOKit reply scan aliased only page-aligned values into the low shadow, so under Wine those records were read through whatever the shadow held at that number. An 8-byte-aligned value inside IOKit or shared host memory is now a candidate too, skipped when the guest has its own memory at that number (`cc4ea96c`). A companion change keeps the kernel from placing its mappings in the low 12 GB at all, by reserving the free host ranges there, so they land where guest and host addresses are the same and need no alias (`334da007`). In fifteen launches after the change, none failed.

Retired blocks that kept running (2026-10-11)

Symptom. Steam's browser processes under Wine stayed busy running blocks that a branch flip had already retired. The counts came from per-process statistics of the kind `OCERZ_PERFSTAT` prints: in the three busiest CEF processes, 1,905,762, 1,159,135 and 527,613 probe exits from retired blocks, and in the browser process 169,984 searches of a 716,148-block live list, 29.4 s of its CPU, none of which found the block it was looking for.

Cause. Two separate things (`bfaca27a`). A chain to a target out of a branch's reach goes through a veneer, a literal load and a `br`, and the chained branch lands on the veneer; unchaining a retired block compared each branch's target with the block's code, and a veneer is not in it, so a chain through a veneer was never cut and the retired block went on running from it. Each pass through a probe that had tripped then left for the runtime, which found the block no longer live. And each of those exits asked for the block's retirement again, and the retire, not finding it where its live-list index pointed, searched the whole list. The fixes follow the veneer to the address it loads and recognise a block whose live index does not point back at it as not live; `OCERZ_NO_VENEER_UNCHAIN=1` and `OCERZ_LIVE_SCAN=1` restore each. After: 323,620, 67,584 and 24,701 probe exits.

A correction worth keeping. The commit message first said that the tests it added passed even with the fix switched off, and that no test reached a block retired by a write through a stale veneer. `e778db6a`, the same day, withdrew that: `smc_ras` run with `OCERZ_FORCE_VENEER=1` fails with "stale return site 7800, stale indirect target 0" when the unchain does not follow the veneer. A far chain left in place had reached the old translation after a write to guest code, so the change was a correctness fix as well as a speed one. The lesson for the method is to run the claim "this test does not exercise the fix" before writing it down, which is exactly what the switch exists for.

A signal that never reached a spinning thread (2026-10-14)

Symptom. It was found on the way to something else. While fixing a loop miscompile (`2cf48a7e`, 2026-10-13), the author noted that an interval timer's signal reached a thread that spins in one block only rarely: with a 20 ms `ITIMER_REAL`, 2.6 s spent inside `lea; dec; jnz` took one or two `SIGALRM`s, on `main` as with the fix. The consequence under Wine was already known: `SuspendThread` is a `SIGUSR1` sent to the thread, so Mono's garbage collector waited for any busy thread to finish its loop.

Narrowing. The first control was the interpreter: a loop of `dec` and `jnz` with a 5 ms `ITIMER_REAL` got no `SIGALRM` at all until it ended, under `-no-jit` as under the JIT. So the cause was not translation. The host handler only marked the signal pending on the thread's CPU, and the guest saw it after the thread's next system call or bridged call and nowhere else.

Fix (`a7fd0f3f`). Both run loops now deliver a pending, unblocked signal between two steps, where the guest state is all in the CPU, which was enough for the interpreter. Translated code reaches the run loop only when its blocks exit to it, so two more pieces were needed, shown in Figure 18.3.

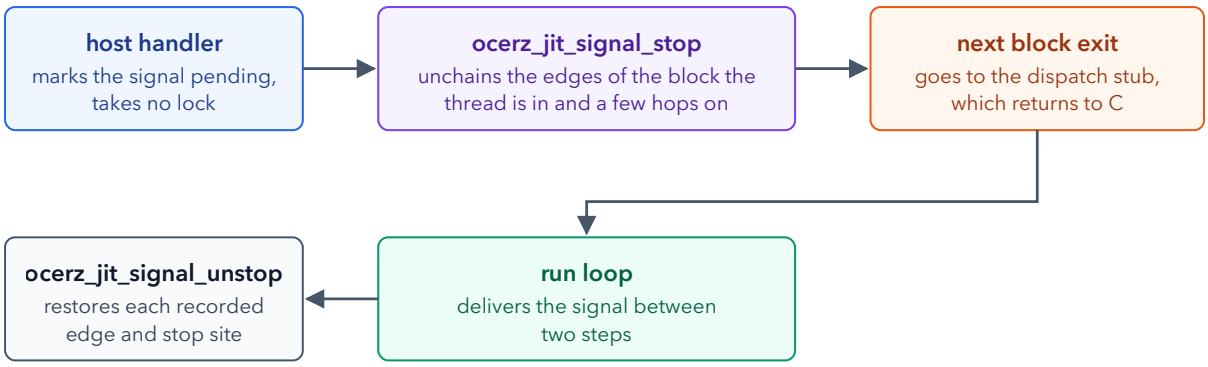


Figure 18.3. How a signal reaches a thread in a translated loop (`a7fd0f3f`). The handler does only what is safe without a lock; the thread's own run loop restores what the handler changed.

The handler calls `ocerz_jit_signal_stop` with the interrupted host pc. It does to the block the thread is in, and to the blocks its chained edges lead to a few hops on, what a retire does: it unchains their chained edges and rewrites their stop sites, so the thread's next jump out of any of them goes to the dispatcher. The thread records each block and edge it changed, and the run loop's `ocerz_jit_signal_unstop` puts them back, a stop site returning to the word it was translated with. Stop sites alone were not enough, because a loop whose backward branch is a side exit chained through its conditional site has none, which is why the walk unchains edges. The handler takes no lock: in the test the thread holding the JIT lock was the sender, translating its own wait loop, so a try-lock lost the signal. It can race a chain install on a word, but every word either side writes is a whole valid branch, and the restore runs under the lock. A block whose stop sites are not in their running state is no longer saved to the translation cache, since it would come back stopped for good. The dynamic test `loop_signal` pins it under both engines; `OCERZ_NO_SIGNAL_STOP=1` goes back to the old delivery, and m32 processes keep it, since delivery there would enter a 64-bit handler.

> **WHAT THEY HAVE IN COMMON**
>
> None of them was found by reading the code that misbehaved. Each was found by asking one question at a time of an instrument that could answer it: does the interpreter agree, does the control agree, which range, which switch, which reply, which order. The switches exist because each of these questions was once asked without one.

CHAPTER 19

Limitations and roadmap

AArchX is experimental software, at version 0.8 as of `6e6c64b5` (2026-10-18). It runs a long list of real programs, and it also has gaps that stop others, deviations from x86 that a careful program could notice, and costs that make some workloads slower than under Rosetta. This chapter is the honest ledger, grouped by how each item bites, followed by the direction the project is taking now that Rosetta's end is in sight. It was first written at `90fe4d8e` (2026-10-04, 0.3-dev); the first section says what closed in the two weeks since, and every later section has been rewritten against the newer tree, saying what an item was, what it is and when it changed.

HOW TO READ THIS LIST

The gates compare the JIT with the interpreter, and the interpreter with goldens taken partly from Rosetta. A deviation the two engines share is invisible to a differential by construction, which is why several items below were found only by running real software. The list is what is known; it is not a guarantee that nothing else is wrong. One case from this fortnight shows the shape of the problem: a loop that carries a `double` and ends in `dec` and `jnz` computed a wrong sum in the JIT from the day the fixed lanes arrived (2026-08-25) until `2cf48a7e` (2026-10-13), while the interpreter was right; the new test `tests/dynamic/incdec_loop.c` is the first that has a loop of that shape.

What closed since 2026-10-04

These were limitations or roadmap items in the first edition's ledger, and are done. Each row gives the commit that did it and its date; the sections after this one describe what is left.

Item	It was	It is
x87 correctness (Chapter 7)	Registers held as doubles, with most of the register forms computing wrong values: D8's `fadd st(0), st(i)` computed `st(0)` with itself, `fcomi` always said equal, `fcmov` copied a register onto itself, `fxam` decoded as `fxch`. A Delphi courier that moves 64-bit integers through `fild` and `fistp` corrupted them.	`8f310eb8` (2026-10-07, issue #6) gave x87 a unit, `src/x87.c`: the registers are still doubles, but each has an exact 80-bit image, valid while the double is still that image rounded, so integer copies, `long double` copies, denormals and NaN payloads are bit for bit. `FXSAVE` now stores the 80-bit registers in stack order and `FXRSTOR` restores TOP. The issue's check reports 0 of 9 quadwords corrupted under both engines. The tests follow Rosetta where it and the manual part.
x87 speed	Every x87 instruction left translated code through a call to the interpreter.	`6c6679f0` (2026-10-07) translates the common instructions; `9f0b7087` (2026-10-07) does so in 32-bit blocks too. On the x87 kernel of `xbench` a paired delta of 10,000 rounds on an M2 Max went from 1.147 s to 0.161 s against Rosetta's 0.731 s (2026-10-07). Operations that meet a NaN, give an infinity or a tiny result, or round other than to nearest still run in the interpreter, with the same results.

Item	It was	It is
Signals for threads in loops (Chapter 9)	A signal reached a guest thread only at its next system call, so a loop of `dec` and `jnz` with a 5 ms `ITIMER_REAL` got no `SIGALRM` until it ended, and under Wine `SuspendThread`, which is a `SIGUSR1`, waited for any busy thread to finish.	`a7fd0f3f` (2026-10-14): both run loops deliver a pending signal between two steps; the handler unchains the edges of the block the thread is in and of the blocks a few hops on, and the dispatch stub returns to the run loop while a signal is pending. `OCERZ_NO_SIGNAL_STOP=1` restores the old delivery. m32 processes keep the old delivery.
Swift in native mode	Swift classes were refused.	`06d1b4da` (2026-10-04, the afternoon after the first edition's commit) runs the swift.org runtime as guest code, installed by `make guest-swift`; `f2ab14b7` (the same day) made Swift Concurrency run and gave `dispatch_time_t` its own notation class, converting Intel nanoseconds to the host's ticks. What is still refused is the Swift APIs of the system frameworks, below.
API databases (Chapter 12)	Fifty-nine libraries had one, and WebKit and GameKit were the gaps that mattered.	177 libraries have one at `6e6c64b5`: the files in `tools/sdkgen/baseline`, one per library, which `83816ab6` (2026-10-05) completed for macOS 27. WebKit and GameKit are among them (`643406e5`, 2026-10-04, added GameKit and 73 other libraries). The census of 2026-10-05 on a macOS 26.7.1 machine found 2,094 of 2,913 x86-64 binaries bind completely, and every binary of 36 of 57 third-party applications does.
Objective-C and C++ in native mode	Native Objective-C exceptions could not be caught by guest handlers; `imp_implementationWithBlock` and `class_replaceMethod` were stubs; `err`, `errx`, a `long double` and a fork inside a callback were refused.	`46349987` (2026-10-05) makes Objective-C exceptions work between guest frames, and an exception a native method raises is caught where the send returns to the guest; `3dcf68d1` (2026-10-05) adds `imp_implementationWithBlock`; `2a583d38` (2026-10-04) `err`, `glob` and `long double` (a double's precision); `f2e350ab` (2026-10-13) performs a fork inside a callback, the child forgetting the JIT's tables without unmapping its code.
Cost of a callback into guest code	About 430 ns, measured 2026-09-21; the commit that fixed it measured 536 ns on the day it was fixed, against 4.2 ns for the same comparator inside Rosetta's x86 `qsort`. Most of it was three system calls (mask, signal-stack state and the jump buffer) and a copy of the 8 KB CPU structure.	`ad51f4b3` (2026-10-18): a nested callback does not read the mask; the recovery point is taken with `sigsetjmp(jb, 0)` and recovery takes the thread off the signal stack with the `UC_RESET_ALT_STACK` flag of `sigreturn`, as Apple's published XNU source defines it; the callback's CPU copy is 2.2 KB. A `qsort` of 200,000 integers with a guest comparator went from 536 to 100-112 ns a comparison. `OCERZ_NO_CHEAP_CALLBACK=1` restores the old path.
Same-address fault loop guard	Applied to every program: a handler that unprotects a page and returns, as a garbage collector's write barrier does, had its thread ended at the seventeenth fault at one address (exit 241 in a test).	`ff6ea313` (2026-10-18): only Wine processes are counted, since the loop the guard was written for is a Wine foreign worker's.
Wine's memory layout (Chapter 8)	The low shadow kept the fast memory forms, base hoisting, paired moves and the batch undo log off.	Done between 2026-10-07 and 2026-10-11, in about twenty-five commits: a hoist that takes three bases and indexed operands (`ff3e02ea`), the stack pointer's register holding the host address in the Wine layout (`ab86121d`), 12 GB kept in `x29` (`5b02be18`), and a hoist that checks an index two operands share once a trip (`e8a26292`). Wine's own cost is now measured below.

Item	It was	It is
Self-modifying code	A write into a callee spliced into its caller left the callers running old code; a chain through a veneer was never cut when its target was retired.	`60bd6cea` (2026-10-11) gives each block the least and greatest guest address it covers; `bfaca27a` (2026-10-11) follows a veneer to its target when unchaining. In the three busiest of Steam's CEF processes under Wine, probe exits from retired blocks fell from 1,905,762, 1,159,135 and 527,613 to 323,620, 67,584 and 24,701.
Retirement cost	Every retire walked all 134 MB of per-site indirect-branch caches in R.E.P.O.'s process, about 4.3 ms with the JIT lock held.	`d8b7073c` (2026-10-05) tags each entry with its column's generation and bumps the generation instead: 6.2 s for 1,446 retires fell to 75 ms for 1,439.
32-bit Mac programs	An executable with only an i386 slice was refused.	`0ca24d64` (2026-10-10) brought in m32 (Chapter 13). What it runs and what it does not is in a section below.
Distribution	Build from source with the Command Line Tools.	A prebuilt release and a one-line installer (`3bf6e6aa` , 2026-10-13), a Homebrew cask (`962f9c7c`), and AArchX.app with a library of programs, Windows programs through Wine and CrossOver bottles (`2dce5f07` , `a246dc47` , `e3a26326` , all 2026-10-13).

Programs that stop

- **Counter-Strike 2 under Wine** has loaded D3DMetal and opened its own window since 2026-09-26, then dies on a fatal "Multiple entity classes have the same designer name" error that Rosetta never reaches. The README at `6e6c64b5` still lists it that way, and no later commit changes it. The cause is believed to be a JIT feature, and `OCERZ_EXE_ENV` exists so that features can be switched off inside `cs2.exe` alone while the rest of the Wine tree runs at full speed. The same runs print hundreds of `dropping stale oolslow arm` warnings whose source is not known yet; the first forty now say which block and instruction the out-of-line arm belonged to.
- **Photoshop** stops during startup when an initializer runs while the Objective-C runtime holds its lock and the recursive acquisition aborts (2026-09-22). That is a re-entrancy problem in how initializers are driven and wants its own change. Three earlier causes found on the way were fixed: upward dependencies never initialized, dylibs with no export trie, and `-undefined dynamic_lookup` imports treated as fatal. The README at `6e6c64b5` still says it stops during startup.
- **Photos** has not been run since the last of the three causes of its graphics failure, the invisible GPU, was fixed on 2026-09-22.
- **Ollama** runs its command line, its HTTP server and its menu-bar app, but nobody has run a model under AArchX yet. Its startup is slow beside Rosetta's: `ollama --version` took 0.183 s against Rosetta's 0.026 s (`94614e9c` , 2026-10-09).
- **Node under Wine** is a lead, not a result. A function-churn benchmark on the MacBook Air on 2026-10-18 ran at 2.2 times Rosetta's time, and 90% of the time was in translated V8 code rather than in translation. That points at the quality of the code generated for V8, not at the cost of making it, and it is open.
- **Firefox 3.6.28 (i386)** starts, finds its fonts and draws its browser window, but the window does not reach the screen: AppKit has it visible and the window server has it off screen. The same window settings made natively show. The process has a second window server connection, opened when Gecko

installs a handler on Carbon's event monitor target; without that handler the window reached the screen in some runs and not in others (`docs/m32.md`).

- **Left 4 Dead 2** plays from Steam under m32, but its menu's background movies do not play (they need QuickTime, which macOS no longer has), typing in its developer console doubles each letter, "Exit to main menu" crashes, and a rebuild of its sound caches stays on the loading poster until it is started again (`docs/32-bit-steam-games.md`).

32-bit programs (m32)

m32 is two weeks old, so its ledger is mostly what it does not do yet. All of this is from `docs/m32.md` at `6e6c64b5` and the tests behind it.

- **Carbon's user interface.** Much of Carbon's window, menu, control and event API is declared for 32-bit code alone and still works in Apple silicon's HIToolbox, and m32 crosses it (Carbon's database crosses 847 functions where it crossed 364), but the rest does not exist on Apple silicon and m32 does not provide it yet: of the control kinds only push buttons, check boxes, static text and icons remain (radio and bevel buttons, sliders, tabs, pop-up buttons, edit text, scroll bars and the Data Browser are gone), and neither are Navigation Services dialogs and MLTE.
- **QuickDraw.** Windows and GWorlds draw into bitmaps over CoreGraphics, but QuickDraw text is not drawn, transfer modes other than `srcCopy` copy, 8-bit and smaller GWorlds are refused with `cDepthErr`, and color cursors from resources show the arrow.
- **Calls with no notation.** `long double` calls stay unresolved, and OpenCL is not bridged (`clCreateContextFromType` answers 0). A Steam method the interface database cannot describe, mostly one taking a function pointer, answers 0.
- **Signals.** Handlers are recorded but not delivered, and a pending signal is delivered between steps only for 64-bit processes.
- **Not yet verified through the Steam bridge:** achievements, statistics and cloud saves (Batman and Modern Warfare 3), Modern Warfare 3's multiplayer, and the Steam overlay, which is not expected to appear.
- **Setup.** A 32-bit game needs `make apis32` and `make steam32`, which download Valve's Steamworks headers from Valve's Proton repository at a pinned commit, and the 32-bit C++ runtimes from MacPorts; none of it is committed. The i386 test programs are committed because linking i386 needs Xcode 26's `ld-classic`, which Command Line Tools 27's linker dropped.

The GPU and Steam

- The macOS Steam client in cache mode still runs with `-cef-disable-gpu`, rendering through software, and has not been retried with acceleration since translated programs began to see the GPU on 2026-09-22. In native mode on macOS 27 its GPU process runs hardware-accelerated graphics. Some cache-mode launches stall before the web helper starts its child processes, with the translation lock held inside translation; relaunching gets past it.
- Under MacNdCheese's Wine, DXMT's swap chain on a window owned by another process is refused by the Wine Mac driver ("Cross-process child window Metal swapchains are not implemented"), the same under Rosetta, which is why the Windows Steam client's browser was brought up with GPU

compositing off. R.E.P.O., a single-process Unity game, renders through DXMT under AArchX since 2026-10-03, and since 2026-10-06 the same Wine runs under native mode, where `22221270` reports Steam logged on and R.E.P.O. at its main menu on DXMT and `0aae84f0` reports it at its main menu on D3DMetal as well.

- Steam's overlay frame counter relies on dyld interposing, which AArchX does not implement; `OCERZ_FPS=1` counts frames instead.
- A 32-bit game reaches Steam only when Steam launched it, because Steam tells a game where its client lives only then; started from a terminal, it asks Steam to relaunch it.

Slower than Rosetta

- **Wine startup.** Windows Steam reaches its main window in about 33 s under AArchX against 11-12 s under Rosetta, using about 58 s of CPU against 17-21 s (2026-09-27; no later measurement is recorded). With the translation cache warm it took 33.7 and 35.2 s in two runs, against 46.9 s from an empty cache. A short Wine process spends about 70% of its startup translating, most of it while the Wine Mac driver brings in AppKit and Foundation.
- **Translation itself** has been cheaper since 2026-10-13: the decoder classifies prefixes with one lookup and the one-byte map is one switch (`39271608`, a decode 1.20x faster in long mode on an x86-64 host), the flag look-ahead stops once every flag is written (`e34ad68f`, 57.6 decodes per translated block before), a block's NZCV producers are found once (`6d7e051a`) and a chain install issues one barrier pair (`f4ef8900`). On the Mac Studio on 2026-10-13, the system's perl loading ten modules with `OCERZ_TCACHE=off` spent about 275 ms in translation that morning and 208 ms after `f4ef8900`; Ruby loading its standard library went from 185 ms to 168 ms. Cache-mode startup is still far from Rosetta's: an empty CoreFoundation program launched in 42 ms against Rosetta's 9 (`16f65d59`, 2026-10-09).
- **Wine's memory layout, and ordering.** Wine always needs ordered memory, and 7-Zip 25.01 under Wine Staging 11.18 on the MacBook Air (M5), one thread, 2026-10-14, compressed at about 8,200 KiB/s against Rosetta's 11,600, a 26 MB archive taking 6.85 s against 4.8 s; 70% of compression is LZMA's binary-tree match finder. Ordered memory costs that kernel about 17% against plain memory, and the archive came out identical either way. In 7-Zip's own benchmark the acquire loads cost 14% and the release stores 2.6%; in the real run the barriers cost 1-3% and most of the rest is code generation, because the ordered load and store forms take no index register. Decompression, which the NZCV work of 2026-10-11 was written for, went from 58,634 KiB/s to 91,061 across the commits of 2026-10-11 (Rosetta 124,000 in the same messages). The gap is narrower than on 2026-10-04 and is still there.
- **Swift conformance lookups.** `libswiftCore` spent 3.3 s of one Steam startup walking every protocol conformance record, because the shared cache's precomputed Swift tables are not offered to it. Not remeasured since 2026-10-04.
- **Games.** Brawlhalla's first frame took 5.3 s against 0.54 s on 2026-09-22. MMX, translated since 2026-10-04, still keeps its registers in memory, so an MMX loop was 0.142 s against Rosetta's 0.053 s (and MMX in 32-bit code is translated since `f8fa8aaa`, 2026-10-10).
- **Self-modifying code.** A program that rewrites code it has run pays a page fault per write and a retirement per block hit. After the targeted retirement of 2026-10-04 a rewrite cost 24.7 μs among

300,000 translated functions, against Rosetta's 14.7; the fixes of 2026-10-11 and `d7567fc5` (2026-10-18, below) change how many blocks run stale but were not remeasured on that benchmark.

- **Calls in native mode.** A call from native code back into guest code cost about 430 ns on 2026-09-21 and 100-112 ns after `ad51f4b3`, against 4.2 ns inside Rosetta's x86 `qsort`. A call out costs about 17 ns, and the eleven string and memory routines answered in place 2 ns. An `objc_msgSend` from guest code to a native object cost 44-50 ns against 2.7 ns under Rosetta; `beed19af` (2026-10-18) lets a thread decide a class's, selector's and implementation's questions once, taking `[NSString length]` from 49.5 to 40.9 ns and `[NSArray count]` from 45.5 to 39.4 ns, which is still ten times Rosetta's.
- **Branchy code with kept side exits.** A probed side exit that the flip profile keeps as it is left its probe behind, a `nop` run on every pass. `d7567fc5` (2026-10-18) retires such a block once no edge still probes and translates it again without the probes: on the branch benchmark the run against Rosetta went from 1.059x to 1.033x, `xbench`'s `str` from 1.074x to 0.967x and `brmiss` from 0.980x to 0.963x.

Semantic deviations

These are places where AArchX does not do exactly what an x86 processor does. Each is deliberate or known, and each is the kind of thing a careful program could detect.

Area	Deviation
x87	Arithmetic is done in 64-bit doubles, so extended precision and range are lost in a result: a product that overflows a double but fits an 80-bit value becomes infinity. Since `8f310eb8` (2026-10-07) a value loaded from memory keeps its exact 80-bit image until an operation changes it, so integer copies, `long double` copies, denormals and NaN payloads survive loads, stores, `FNSAVE` and `FXSAVE` bit for bit, and `FXSAVE` stores the 80-bit registers in stack order. Unmasked x87 exceptions do not trap. The MMX registers are kept apart from the x87 stack, so `FXSAVE` and signal frames do not carry them. (Before 2026-10-07 `FXSAVE` used a double-backed layout.)
Approximations	`rcpps`, `rcpss`, `rsqrtps` and `rsqrtss` return exact results, not x86's 12-bit approximations (`src/interp_sse.c` computes `1.0f / x` and `1.0f / sqrtf(x)`).
Interpreted SIMD	Correct but slow: 256-bit byte and integer shuffles, unpacks, permutes and lane inserts, the 256-bit `vptest` (the 128-bit `ptest` has been translated since `ff8ce8b9`, 2026-10-09), the immediate blends, mask-producing compares, gathers, `bextr`, `pdep`, `pext`, and the MMX forms not yet translated (`pmaddwd`, `pmovmskb`, `emms`, the SSSE3 MMX forms, conversions to and from SSE, shifts by a register count; `emit_mmx` in `src/jit.c` handles none of them at `6e6c64b5`).
Not decoded	AVX-512 and its EVEX encoding, and privileged and I/O instructions, which no user program executes.
NaN payloads	A NaN produced inside a floating-point batch that faults before its check keeps the arm64 payload. On processors with `FEAT_AFP`, cache mode leaves add, subtract, multiply, divide and square root to the hardware's x86 NaN rule; fused multiply-add and native mode use the translated checks. An i386 program runs in native mode by default since `1ae3140a` (2026-10-18), so it uses the translated checks too.
Narrow SSE ordering	In the ordered memory model, SSE loads and stores stay plain, the 8- and 4-byte `movq` and `movd` included, so a counter published through `movq` can become visible before data written earlier with a scalar store. `tests/dynamic/tso_narrow.c` sees it hundreds of times in three million handoffs. `OCERZ_TSO_NARROW=1` fixes it at a third of R.E.P.O.'s frame rate; no program is known to depend on it.

Area	Deviation
Signal timing	Until 2026-10-14 an asynchronous signal reached a guest thread only at its next system call (or bridged call in native mode), so a thread spinning in its own code never saw one. Since `a7fd0f3f` the run loops deliver a pending signal between any two steps, and a thread in a translated loop is stopped by unchaining the edges of its block and the blocks a few hops on. Two limits remain: a signal aimed at another thread reaches its guest handler only for the signals AArchX mirrors onto the host, such as `SIGUSR1`, `SIGTERM` and `SIGALRM`, not for fault signals; and m32 processes keep the old delivery, since delivery there would enter a 64-bit handler.
Protection granularity	Guest protection changes are resolved on 16 KB host page boundaries, though guest pages are 4 KB.
`mach_vm_region`	Answered from the guest's own map since 2026-09-22, with each segment's declared protection, but `max_protection` always reads 7, because AArchX keeps no per-segment maximum. An i386 program's walk covers only its 4 GB window (`8f8bb942`, 2026-10-12).
`ioctl`	The third argument is always translated as a pointer. In the identity map every dynamic program uses that is the identity, but in the static arena and Wine's low shadow a scalar argument gets a base added. `fcntl` translates only for the commands that take a pointer.
Process identity	`proc_pidpath` and `proc_name` name the guest executable only when a process asks about itself in cache mode; other AArchX processes appear as `ocerz`.
`long double` in native mode	Crosses as a double, so it loses the x87 format's extra precision.

Loader gaps in cache mode

- The dyld API object answered 59 slots at `90fe4d8e`, each identified by running code (Chapter 11); the count was not redone at `6e6c64b5`. Five of them, whose answers are fixed once the program's build version is read, are now x86 code in a guest page that the JIT translates and chains into (`90903488`, 2026-10-09), which took a 200,000-entry CoreFoundation dictionary build from 1.05 to 0.89 s. Every other slot answered zero in both result registers. Three that real programs reach had no handler: `+0x30`, which `_dyld_register_func_for_add_image` reaches first, so add-image callbacks are not called in cache mode; and `+0x420` and `+0x250`, which the macOS Steam client calls and tolerates a zero from.
- Upward links are followed during a `dlopen` only to Foundation, CoreFoundation and libobjc. Following all of them was tried and rejected: it pulled about 360 more initializers into every Wine process and killed Steam's browser. Libraries reached only through delayed-init dependencies stay out of the launch since `85b248fc` (2026-10-09), as dyld keeps them; `OCERZ_NO_DELAY_INIT=1` loads them at launch again.
- Not rechecked since the September work on `dlopen`: whether `dlopen(NULL)`'s handle finds the program's own symbols, `RTLD_LOCAL`, and `dlerror` after a failed `dlsym`. Native mode does all of these the way dyld does.
- dyld interposing is not implemented.

Translator internals still open

- **Arena overflow.** A translation that overflows the 1 GB code arena left the translator in a state that damaged later translations, reproducible in a second with a 512 KB arena, and why is not understood.

Since 2026-10-04 the arena is flushed before it can overflow, which removes the symptom; the cause is still open.

- **Arena flush and stuck threads.** A flush needs every thread out of translated code. One that cannot be got out in 500 ms, such as a thread stopped by `thread_suspend` while it spins, makes the flush back off and retry two seconds later.
- **Stale out-of-line arms.** The `dropping stale oolslow arm` warnings seen under CS2 have no known source yet.

Native mode

Native mode stops a program it cannot run correctly with a named message, exit 71 for an import that cannot bind and 72 for an export with no crossing (Appendix B), rather than running it wrongly. Since `334a5e1f` (2026-10-16) it opens the windows of Transmission, Sequel Pro, HandBrake and VLC 2.2 in two seconds with no switches. Its gaps, in order of how much software they stop:

1. **Swift's framework APIs.** Swift programs run on a guest copy of the swift.org runtime, Concurrency included (2026-10-04), as long as they stay within the Swift standard library. The Swift APIs of the system frameworks, Foundation's and Dispatch's overlays, SwiftUI, Combine and the rest, have no crossing. Calculator and most newer system applications are written in Swift against those. This gap grows by itself as Apple moves more system software to Swift.
2. **Libraries without a database.** A hundred and seventy-seven libraries have one, generated from the builder's SDK by `make apis`: libSystem, libobjc, CoreFoundation, Foundation, AppKit, CoreGraphics, CoreServices, Carbon, Security, IOKit, Metal, OpenGL, QuartzCore, WebKit, GameKit, the audio and video frameworks and the small dylibs programs pick up. Other system libraries still need one, and unsupported exports inside the 177 remain stubs; `libxo`, `libmd` and `libarchive` have databases whose exports are all stubs, since no SDK header declares them. A library Apple shipped for Intel Macs that no macOS on Apple silicon has at all (QTKit and the 0.9.x libcrypto) is stubbed by default, each import named as it binds, since refusing a program over it would keep all of it from running (`334a5e1f`; `OCERZ_NO_STUB_REMOVED=1` refuses again).
3. **C++ across the boundary.** Guest C++ works with the optional LLVM guest runtime, exceptions between guest frames included, but guest C++ objects are not interchangeable with native arm64 ones, and no exception may unwind through a bridge or callback frame. A native C++ exception cannot be caught by a guest handler. Programs built with GCC need the GNU libstdc++ from `tools/fetch_guest_gnu_cxx.sh`, which is fetched, pinned by SHA-256, and ships in no release.
4. **Calls with no veneer.** The `scanf` family other than `scanf`, `fscanf`, `sscanf`, `sscanf_l` and their `v` forms, and `asl_log` are stubs; so are functions taking a `va_list` the bridge has no veneer for (apart from the printf family and its relatives) or a union or bitfield structure by value, and `forkpty`, `daemon` and `wordexp`. `err`, `errx` and `long double` were stubs at `90fe4d8e` and no longer are.
5. **Replacing methods.** The bulk method APIs are stubs, `imp_implementationWithBlock` refuses a block native code made, and a guest that reads function pointers out of a structure a native call filled in would call arm64 code. (`class_replaceMethod`, `method_setImplementation` and `imp_implementationWithBlock` for guest blocks were stubs at `90fe4d8e` and work now; an `IMP` the native runtime hands the guest is a thunk it can call.)
6. **Smaller items.** A `longjmp` out of a native callback is refused; an XPC service inside an application bundle is not reached, most likely because launchd looks it up for the executable that asks, which is

`ocerz`; a block that crosses as a plain object can be called only by the side that made it; after a bridged call from translated code, the xmm registers the signature does not use hold what native code left in them, which System V allows; a guest function stored where native code reads a function pointer, as HandBrake stores its own into `stderr`'s `_write`, runs with only the first six integer arguments, and one taking floating point or a structure by value gets garbage (`OCERZ_NO_NATIVE_BOUNCE=1` stops the process instead); a guest that is not position-independent runs with its low addresses shadowed, so a structure holding pointers into its own image would carry addresses native code cannot read.

The direction: after Rosetta

Apple ends general-purpose Rosetta after macOS 27, and with it the `dyld_shared_cache_x86_64` that cache mode binds every program against. macOS 27 already moved that cache out of the operating system's own cryptex into a separate Rosetta cryptex. Cache mode works on both macOS 26 and 27 since 2026-10-03, looking in the Rosetta cryptex first and the old location second, but it cannot outlive the file.

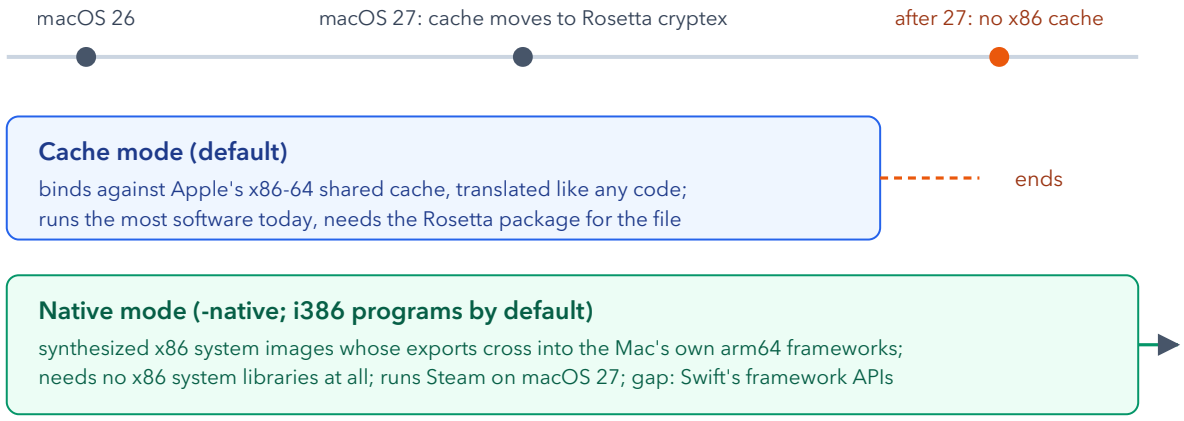


Figure 19.1. The two ways AArchX supplies a program's system libraries, against the end of Rosetta. Cache mode depends on a file Apple ships only while Rosetta does; native mode depends only on the Mac's own frameworks and on databases generated from its SDK.

Native mode is the answer, and the code says so: `include/ocerz/mode.h` describes it as "the mode that survives" the end of Apple's Intel frameworks, "selected explicitly until it is ready to be the default". Built from 2026-09-16, it ran the x86-64 Steam client on macOS 27 by 2026-09-21. It is still not the default for x86-64 programs. Two things have changed that make it the default in practice for some programs: an i386 program has no cache to bind against and so runs in native mode unless `-cache` or `OCERZ_MODE` says otherwise (`1ae3140a`, 2026-10-18), and AArchX.app sends every program to native mode when Rosetta's x86-64 cache is missing (`2dce5f07`, 2026-10-13). Making it the default for everything is a matter of closing the list above, in roughly that order: Swift's framework APIs, because they stop the most software and the share grows; the remaining libraries, which is mostly a data change through `tools/sdkgen.sh` and `make apis` as long as their calls fit the ABI engine; and C++ and exceptions across the boundary. The cost of calling back into guest code, which was three system calls saving signal state for each callback's fault-recovery point, is no longer on the list since `ad51f4b3`.

The second direction is visible in the tree rather than stated. Since 2026-09-23 the project is under the LGPL-2.1-or-later again and carries its own documentation, contribution rules, code of conduct, security policy and issue templates; the commit that brought the documentation in says it moved so the tree would be "self-contained when it becomes public". The README was cut to a short front door on 2026-10-03 with the long material in `docs/`. In 0.7 the project began to be usable without a build: a prebuilt binary, `docs/install.sh`, a Homebrew tap and AArchX.app.

The third direction is old software. m32 (Chapter 13) and the 2026-10-16 native-mode work on Transmission, Sequel Pro, HandBrake and VLC 2.2 make AArchX a way to run programs from the Intel era that macOS no longer runs at all, and Rosetta never ran the 32-bit ones. Nothing about that direction depends on Apple's shared cache.

Open fronts

Front	Where it stands
Games under Wine	R.E.P.O. renders through DXMT, and under native mode through D3DMetal as well (2026-10-06). After the arena flush and the short low-window check of 2026-10-04 its cache-mode menu ran at roughly 45-65 frames a second, varying from run to run, and a singleplayer level at about 50; Rosetta drew the same menu at about 178. Those figures date from 2026-10-04, before the Wine-layout work of 2026-10-07 to 2026-10-11, and no later one is recorded. CS2 reaches its window and stops on the designer-name error.
Wine speed	Ordered memory and code generation for ordered forms (about 17% of a 7-Zip compression, 2026-10-14), translation cost, and Node's translated V8 code (2.2x Rosetta, 2026-10-18).
Steam with the GPU	The macOS client in cache mode has not been retried with acceleration since the GPU became visible.
Application breadth	Photoshop's re-entrancy, Photos untried since its fixes, a model run under Ollama.
Native-mode coverage	Swift's framework APIs, C++ across the boundary, the bulk method APIs, the remaining variadics.
32-bit programs	Carbon's controls that no longer exist, QuickDraw text, Firefox 3.6's window, Left 4 Dead 2's rough edges, the Steam features not yet verified.
Semantic gaps	Extended-precision x87 arithmetic, approximate reciprocals, the SIMD forms still interpreted, add-image callbacks and interposing in cache mode, three unanswered dyld API slots, signals for m32.

The June plan, scored

In June the project wrote down a seven-phase plan from a static-binary translator to Wine and Steam. Every phase has landed or been superseded, and several went differently from the plan in instructive ways.

June phase	Planned	What happened
1. dyld	run Apple's `/usr/lib/dyld` as a second image	Done differently: AArchX never runs Apple's dyld. It wrote its own loader and cache mapper and answers libdyld's API table itself (Chapter 11).
2. Signals	host trampolines, x86 contexts, `sigreturn`	Done, plus mask enforcement, delivery at block entry, mirroring onto the host, frames carrying the upper YMM halves, and since 2026-10-14 delivery to a thread that spins inside a translated loop.

June phase	Planned	What happened
3. Threads	host pthreads, per-thread `gs`, ulock and psynch	Done, plus the host workqueue bridge, emulated thread suspension, the unstick monitor, fork and spawn.
4. Memory model	acquire/release everywhere	Done differently: plain until a second observer appears, then ordered scalars and plain vectors, as FEX ships.
5. JIT tier 2	chaining, inline flags, pinned registers	Exceeded: full register pinning, deferred flags, superblocks and trace inversion, FP batches, AVX and MMX, x87, in-place routines, a translation cache. Faster than Rosetta on most kernels.
6. Wine and Steam	wineserver, Notepad, then D3DMetal	Mostly done: Wine boots, WoW64 runs, Windows Steam reaches its main window, a Unity game renders on DXMT and on D3DMetal; CS2 open.
7. Polish	80-bit x87, MXCSR rounding, exact reciprocals, AVX	MXCSR rounding and AVX done. x87 is half done: exact 80-bit images and a JIT for it since 2026-10-07, but arithmetic still in doubles. Approximate reciprocals open.

Nothing in the June plan anticipated i386 and WoW64, native macOS applications, native mode, the rename to AArchX, the license's round trip through a proprietary text and back, 32-bit Mac programs through m32, or a prebuilt release with an installer and a Homebrew tap. A plan is a list of what is known to be needed; the software decides the rest.

APPENDIX A

Environment variables

AArchX is configured by its command line and by environment variables named `OCERZ_*`, after the project's former name. The first tables below are the variables meant to be used; the later ones are the switches added with particular fixes, the diagnostics, the switches of m32 (Chapter 13), and a census of the several hundred diagnostic knobs that exist for narrowing bugs. The lists agree with `docs/reference.md` and `docs/m32.md` at `6e6c64b5` (2026-10-18), and each name was checked against `src/` with `grep`. Every variable is read once, at first use, and is inherited by child processes unless noted.

Command line

```
usage: ocerz [-v] [-trace] [-strace] [-no-jit] [-native|-cache] [-path file] [--] program
[args...]
       ocerz version
```

Option	Effect
`-v`	more logging: the mode, the arena, the shared cache, image loads, the JIT arena
`-trace`	trace guest instructions
`-strace`	trace guest system calls
`-no-jit`	interpret this process only, without translating
`-native`	bind against the Mac's own arm64 frameworks (Chapter 12)
`-cache`	bind against Apple's x86-64 shared cache; the default
`-path file`	load `file`, and pass every following argument to the guest unchanged
`--`	end of AArchX's own options
`version`	print the name and version, `AArchX 0.8` at `6e6c64b5` (`a454f2fd`, 2026-10-18; it printed `AArchX 0.3-dev` at `90fe4d8e`)

The program may be an executable or an application bundle, in which case the executable its `Info.plist` names is run with that path as `argv[0]`. A program whose executable is a script is run too: Steam's `%command%` for a Source game is `hl2.sh`, which starts the game through `${GAME_DEBUGGER}`, and AArchX runs the script with `GAME_DEBUGGER` naming itself (`46fade2e`, 2026-10-13); any other script runs on its interpreter's x86-64 slice under AArchX, so the programs it starts are AArchX's to run (`88b7e20c`, 2026-10-18). A script whose launcher path contains a space is refused with a message (exit 64). An executable with an i386 slice and no x86-64 one is handed to m32 (Chapter 13) and runs in native mode unless `-cache` or `OCERZ_MODE` says otherwise (`1ae3140a`, 2026-10-18).

Choosing what runs

Variable	Effect
`OCERZ_MODE=native|cache`	pick the mode when no flag does. This is how a program's children inherit it, even when started with an empty environment. An unrecognised value is refused (exit 64).
`OCERZ_NOJIT=1`	interpret the whole process tree
`OCERZ_APIDB=<dir>`	native mode: read the API databases from this directory instead of `runtime/apis` beside the binary
`OCERZ_GUEST_ROOT=<dir>`	native mode: a directory mirroring absolute guest library paths; an x86-64 image found there is used in place of its database. Empty disables the default `runtime/guest`.
`OCERZ_GUEST_GNU_ROOT=<dir>`	native mode: where GCC's libstdc++ and libgcc_s and GNU libiconv are looked for, after `runtime/guest`; the default is `runtime/guest-gnu`, filled by `tools/fetch_guest_gnu_cxx.sh`. Empty disables it.

Process trees

Wine and Steam start many processes, and a setting is usually wanted in one of them. Each of these selects processes by a substring of their command line, and the setting is inherited through every exec in the chain.

Variable	Effect
`OCERZ_EXE_ENV="a.exe:K=V,K=V;b.exe:K=V"`	set variables only in the processes whose command line matches
`OCERZ_STRACE_EXE=<text>`	trace system calls only in matching processes
`OCERZ_NOJIT_EXE=<text>`	interpret only matching processes
`OCERZ_STDERR_FILE=<file>`	append every process's standard error to one file
`OCERZ_ABORT_REPORT=1`	an abort names the host frames it came from and the guest instruction its thread was at
`OCERZ_NO_WINE_SEGMENT_DETECT=1`	recognise Wine's loader by its name alone, not also by a `WINE_RESERVE` segment, as before `74e6140d` (2026-10-13), which CrossOver's loader links need

Keeping translations

Translated blocks are stored under `~/Library/Caches/ocerz`, one directory per build of ocerz and set of `OCERZ_` variables, and loaded by any later process that needs the same code. Only code at the same address in every run is kept: all of a Wine process, the shared cache otherwise. A store that reaches its limit is emptied by the next process that finds no other using it; stores of other builds are removed after a day, or sooner while they add up to more than 2 GB. Records are summed with CRC32C (`a02fcc3f`, 2026-10-09). Each directory also keeps a 1 MB filter of the names the shared cache defines weakly, which a process binding a C++ library would otherwise build for itself (`94614e9c`, 2026-10-09). The test runners set `OCERZ_TCACHE=off`.

Variable	Effect
`OCERZ_TCACHE=off`	translate everything afresh and keep nothing
`OCERZ_TCACHE_DIR=<dir>`	keep the stores under this directory
`OCERZ_TCACHE_MAX_MB=<n>`	the size at which a store stops growing (4096 by default)
`OCERZ_TCACHE_MIN_FREE_MB=<n>`	free space the cache leaves on its disk: it stops writing below this, takes at most half of what is free above it, and clears its stores when the disk is already under it (10240 by default; `93b919f9`, 2026-10-07)
`OCERZ_TCACHE_LOG=1|<file>`	per process, blocks loaded and stored
`OCERZ_TCACHE=verify`	a check: translate everything anyway and compare each block with the stored one
`OCERZ_TCACHE=roundtrip`	a check: store nothing, move every translation to a new address and report anything still pointing at the old one

Memory ordering

Variable	Effect
`OCERZ_NO_PLAIN_MEM=1`	use ordered forms from the start, rather than from the first thread, fork or shared mapping
`OCERZ_NO_REMAP_PLAIN=1`	use ordered forms once any mapping the kernel hands the program has to be moved into guest memory, private or not, as before 2026-10-09 (`618d540f`): a private one, like the logging library's buffers, now leaves memory plain
`OCERZ_TSO_STRICT=1`	order stack-relative accesses too
`OCERZ_TSO_VECTOR=1`	order every SSE load and store, at a large cost
`OCERZ_TSO_NARROW=1`	order `movd` and `movq`; `=2` every SSE access of 8 bytes or fewer

Turning off what makes it fast

Each of these slows AArchX down and is useful for narrowing a suspected miscompilation: if one fixes a program, the bug is in what it controls (Chapter 18).

Variable	Effect
`OCERZ_NO_AFP=1`	check NaN results in translated code even where the processor produces x86's NaNs itself
`OCERZ_NO_JIT_MMX=1`	run MMX instructions in the interpreter
`OCERZ_NO_FAST_LOW_GUARD=1`	in a Wine process, map every guest address with the general three-range check

Variable	Effect
`OCERZ_NO_LEAF_INPLACE=1`	reach `strlen`, `memcpy` and the other nine routines the ordinary way
`OCERZ_NO_MEMFN_PLAIN=1`	translate the system's string and memory routines with ordered accesses when the process has ordering on
`OCERZ_NO_FPB_DEFER=1`	check every floating-point result immediately instead of once per batch
`OCERZ_NO_FLIP=1`	no profile-driven retranslation of superblocks
`OCERZ_NO_MOVFUSE=1`	do not fold a `mov` into a following shift
`OCERZ_NO_CHAIN=1`, `OCERZ_NO_JCCFUSE=1`	no block chaining; no compare-and-branch fusion
`OCERZ_NO_HOST_RAS=1`	use the software return-address stack in the CPU structure instead of the host stack
`OCERZ_NO_XLIVE_MEMO=1`	do not memoise cross-block flag liveness
`OCERZ_NO_LOW_RAS=1`, `OCERZ_NO_LOW_SPLICE=1`	in Wine's layout, no host return stack, no spliced calls
`OCERZ_NO_BRIDGE_FASTCALL=1`	native mode: reach every bridged call through the trap and dispatcher
`OCERZ_NO_COMPACT=1`	keep every block's decoded instructions after translation
`OCERZ_NO_JIT_FLUSH=1`	when the code arena fills, interpret everything new instead of flushing the arena
`OCERZ_NO_HIGH8_ARITH=1`, `OCERZ_NO_SHIFT_MEM_IMM=1`	run arithmetic, compares, moves and `setcc` on `ah`, `ch`, `dh` and `bh`, and shifts of memory by an immediate, in the interpreter, in 64-bit code as in 32-bit (`358e33fb`, `e6e93ecb`, 2026-10-10)
`OCERZ_NO_INLINE_PTEST=1`, `OCERZ_NO_PTEST_FUSE=1`	128-bit `ptest` goes to the interpreter; a `jcc`, `setcc` or `cmovcc` after it no longer tests the registers itself (`ff8ce8b9`, 2026-10-09)
`OCERZ_NO_ADC_MEM=1`	an `adc` or `sbb` into memory whose flags are dead goes to the interpreter again (`b77d67dc`, 2026-10-18)
`OCERZ_NZCV_GAP=<n>`	hand a compare's flags straight to the `setcc`, `cmov`, `adc`, `sbb` or `jcc` reading them only across at most *n* instructions between: 0 to 8, default 8 (3 was the limit before `e853c450`, 2026-10-11)
`OCERZ_NO_NZCV_GAP_LOAD=1`	end the window at a load, instead of letting a load sit in it and building the flags again from the compare's record after it
`OCERZ_NO_ADC_CINC=1`	set the carry in a register and add it for an `adc` or `sbb` of an immediate whose carry comes straight from a compare, instead of one `csinc`
`OCERZ_NO_LEA32_GAP=1`	keep a `lea` of a 32-bit address in 64-bit code from sitting between a compare and the instructions taking its flags
`OCERZ_NO_XLIVE_SELF=1`, `OCERZ_NO_XLIVE_KILL=1`	follow a block's jump back to its own start as any other edge when working out live flags, instead of the least fixed point over the loop; decode a successor to its end instead of stopping at the instruction by which it has written every flag (`e34ad68f`)
`OCERZ_NO_INCDEC_LOOP=1`	leave a block that loops on itself through `inc` or `dec` and `jz`/`jnz` by a chained edge to its own start, instead of branching straight to its loop entry (`2cf48a7e`)
`OCERZ_NO_LAZY_RSP=1`	move `rsp`'s register at every push, pop, call and ret of a call spliced into its caller, instead of leaving it where it was and giving each stack access a fixed offset (`256faca8`, 2026-10-11)

Variable	Effect
`OCERZ_NO_WINE_LIM=1`, `OCERZ_WLIM_CHECK=1`	in a Wine process, test each address against 12 GB with a shift and a compare again, instead of comparing it with `x29`, which holds 12 GB; the check traps before every guard whose `x29` is not 12 GB (a debugging aid)
`OCERZ_LOW_HOIST_SHARE=<n>`	in a Wine process, check an index register once per trip round a block when *n* of its memory operands share it (default 2; 3 before `e8a26292`, 2026-10-11)
`OCERZ_NO_LOW_STACK_PTR=1`, `OCERZ_LOWSTACK_CHECK=1`	in a Wine process, keep `rsp`'s own value in its register and add the stack's offset on every stack access, instead of keeping the host address; the check traps before any instruction where the offset kept for `rsp` no longer matches it (a debugging aid)
`OCERZ_NO_WINE_BASE=1`, `OCERZ_NO_LOW_NZCV_MEM=1`, `OCERZ_LOW_TOP_GUARD=1`	in a Wine process: guard every memory operand's whole address instead of its base register once per block; keep the flags of a compare with a memory operand in the flag record; test every access for the top strip instead of learning which blocks reach it from their first fault
`OCERZ_ALIGN_TEST_ALL=1`	test every ordered access for a 16-byte crossing instead of patching the sites that fault
`OCERZ_NO_LEAF_LOW=1`	in the Wine layout, leave the in-place routines to their translated x86 code instead of handing them translated pointers
`OCERZ_NO_FLIP_KEEP_RETIRE=1`	a superblock whose branches were all kept as they were keeps its probes (`nop`s on the fall-through path) instead of being translated again without them (`d7567fc5`, 2026-10-18)
`OCERZ_NO_VENEER_UNCHAIN=1`, `OCERZ_FORCE_VENEER=1`	when a block is retired, leave chains that reach it through a veneer in place, as before `bfaca27a` (2026-10-11); chain every block through a veneer, for tests
`OCERZ_NO_CHAIN_SYNC_BATCH=1`	after chaining a new block, flush each patched branch from the instruction cache with its own call, instead of one barrier pair for all (`f4ef8900`, 2026-10-13)
`OCERZ_NO_CHEAP_CALLBACK=1`	native mode: when native code calls guest code, clear the host signal mask on every call and save it with the recovery point, as 0.8 did, instead of only on a thread's outermost guest call (`ad51f4b3`)
`OCERZ_NO_OBJC_SEND_CACHE=1`	native mode: decide every `objc_msgSend` in full instead of reusing what the thread decided for the same class, selector and implementation (`beed19af`)
`OCERZ_NO_DYLDAPI_FAST=1`	every dyld API slot traps, including the five whose answers are fixed (`90903488`, 2026-10-09)

Behaviour

Variable	Effect
`OCERZ_STRICT_SYSCALL=1`	stop on an unimplemented system call instead of returning `ENOSYS`
`OCERZ_NO_HOSTWQ=1`	turn off the host workqueue bridge, which is on by default
`OCERZ_NO_UNSTICK=1`	never interrupt a guest thread out of a long wait
`OCERZ_UNSTICK_ALL=1`	let the unstick monitor interrupt every blocking call, `read`, `recvmsg` and `poll` included
`OCERZ_NO_THREADACT=1`	hand `thread_suspend`, `thread_resume` and `thread_get_state` on guest threads to the kernel
`OCERZ_NO_FILEMAP=1`	read every private file mapping into anonymous memory

Variable	Effect
`OCERZ_PRELOAD_OBJC=<paths>`	put matching shared-cache Objective-C images into the startup batch; `@cat` does it for every image that defines categories
`OCERZ_NO_DELAY_INIT=1`	load and initialize at launch the libraries reached only through delayed-init dependencies, as before `85b248fc` (2026-10-09)
`OCERZ_NO_WEAK_MAIN_FIRST=1`	look a weak definition up in the shared cache before the main executable, as before `230da272` (2026-10-09)
`OCERZ_NO_SIGNAL_STOP=1`	deliver a signal only at its thread's next system call or bridged call, as before `a7fd0f3f` (2026-10-14), instead of also between any two blocks
`OCERZ_NO_LOADMAP=1`	do not map a cache image's Objective-C classes before its `+load` runs
`OCERZ_NO_LATE_CATLIST=1`	do not report category lists for cache images loaded after startup
`OCERZ_NO_VMMAP_STEER=1`	let `mach_vm_map` place mappings anywhere in host space
`OCERZ_REFAULT_INVAL=1`	invalidate again on every repeated alignment or commpage fault
`OCERZ_NO_MOCK_KEYCHAIN=1`	launch Steam's helper without `--use-mock-keychain`

Native mode

Variable	Effect
`OCERZ_BRIDGESTAT=1`	print how many times each bridged function was called, at exit
`OCERZ_BRIDGELOG=1`	name every bridged call as it happens, with the guest's return address and first argument; `=2` also prints what each crossing returns (`106f0eef`, 2026-10-05)
`OCERZ_NO_NATIVE_CHILDREN=1`	run the system tools a guest starts under AArchX too; by default one that has an arm64 slice and is not a shell or launcher runs as itself
`OCERZ_NO_TSD_DTORS=1`	do not run the destructors of the guest's thread-specific data keys at thread exit
`OCERZ_NO_NATIVE_BOUNCE=1`	when native code jumps into a guest function stored where it expects a function pointer, stop the process instead of running the function as a callback with six integer arguments (`334a5e1f`, 2026-10-16)
`OCERZ_STUB_MISSING=1`	bind an import no virtual library exports to a placeholder instead of refusing the load: a class becomes an empty Objective-C class answering every message with nil, a constant a string of its own name, anything else a function returning 0 (or -1 for OpenCL's status calls); each is logged once at bind time
`OCERZ_NO_STUB_REMOVED=1`	refuse a program importing a library Apple shipped for Intel Macs that no macOS on Apple silicon has (QTKit and the like), instead of stubbing its imports by default
`OCERZ_NO_HOST_ONLY_DEP=1`	a dependency the host has but no API database describes, which nothing imports from, fails a native `dlopen` again, instead of being passed over as dyld would load it unused (`78d3f9b8`, 2026-10-13)

Variable	Effect
`OCERZ_FORK_CALLBACK_REFUSE=1`	refuse a `fork` made inside a callback translated code made, as before `f2e350ab` (2026-10-13)

32-bit programs (m32)

These are the switches of `docs/m32.md`, which describes each mechanism (Chapter 13). The `OCERZ_NO_M32_*` names turn one rule off and go back to what it replaced.

Variable	Effect
`OCERZ_GUEST32_ROOT=<dir>`	where guest system libraries are looked for, instead of `runtime/guest32`
`OCERZ_M32LOG=<list>`	comma-separated: `imports` as they resolve, `calls` for every crossing, `images`, `objc` sends, `throws`, `glstat` (OpenGL calls and frames per 10 s), `steam` (the bridge: the client it opened, each interface it bridged, every call to a method it cannot cross, each callback it hands the game), `dlsym` (each lookup that found nothing, with its caller) and `render` (each audio-unit render callback)
`OCERZ_M32_SENDLOG=<part>[|<part>...]`	each message to a host object whose selector contains a part: its arguments and result, then every message the objects it returned receive
`OCERZ_M32_KEEP_GOING=1`	run with unresolved imports, which then return 0, instead of exiting 71
`OCERZ_M32_HEAP_MB=<n>`	the guest heap's first size (1024 by default; it grows on demand)
`OCERZ_M32_MUTE=1`	the CoreAudio render callback outputs silence
`OCERZ_M32_HUD=<overlay dylib>`	an FPS overlay exporting `fake_dlsym` draws on each frame the game presents, through `CGLFlushDrawable` or `-[NSOpenGLContext flushBuffer]`
`OCERZ_M32_NO_SELFLOAD=1`	start a program with no libraries and no dyld at its entry like any other, instead of as one that loads itself, as before 2026-10-16 (`0144d669`)
`OCERZ_NO_M32_STEAM=1`	a `steamclient.dylib` with no i386 slice is left to the ordinary loader, which refuses it
`OCERZ_NO_M32_FASTCALL=1`, `OCERZ_NO_M32_LEAF=1`, `OCERZ_NO_M32_TRAP_BLOCKS=1`	imports leave the block for the trap instead of crossing from inside it; leaf exports cross like any other import; a call through a function pointer to an export goes through the run loop
`OCERZ_NO_M32_CARBON_WRAP=1`, `OCERZ_NO_M32_UPP=1`, `OCERZ_NO_M32_EVENT_PARAMS=1`, `OCERZ_NO_M32_FLOAT_ARRAYS=1`, `OCERZ_NO_M32_PACKED=1`	Carbon: a window made without `kWindowCompositingAttribute`; the `New...UPP` calls left unresolved; event parameters in their raw layouts; `CGFloat` arrays as the guest's floats; HIToolbox's theme structures as the guest's bytes
`OCERZ_NO_M32_LUCIDA=1`, `OCERZ_NO_M32_KVC=1`, `OCERZ_NO_M32_REAP=1`, `OCERZ_NO_M32_IMAGE_NAME=1`	the system font answered as today's system font; no key-value coding from guest instance variables and nib outlets with no setter left unset; a twin outlives its host object; `class_getImageName` knows no image for a paired class
`OCERZ_NO_M32_SYSTEM_OPENAL=1`, `OCERZ_NO_M32_DLSYM_NULL=1`	a program's own OpenAL library is loaded instead of the system's; `dlsym` hands out the trap of a function m32 cannot call, which answers 0

`OCERZ_NO_HIGH8_ARITH` and `OCERZ_NO_SHIFT_MEM_IMM` above, and `OCERZ_NO_ADC_MEM`, belong to m32's speed work as well; the 32-bit code uses the same translator.

Diagnostics

None of these changes what a program does; each prints something.

Variable	Effect
`OCERZ_DLPATH=1`	print every `dlopen` and `dlsym` failure with its reason; inherited by child processes, which `-v` is not
`OCERZ_PERFSTAT=1`	every 15 seconds and at exit, print block counts, the hottest blocks, where instructions went, and return-address stack misses split by cause with the `ret` sites that miss most
`OCERZ_FAULTLOG=1`, `OCERZ_EXCLOG=1`, `OCERZ_WILDLOG=1`	the mapping state around every guest fault; every Objective-C and C++ exception with its throw site; indirect branches whose target is outside the guest address space
`OCERZ_JITLOCKLOG=1`, `OCERZ_JITMEASURE=1`	JIT lock waits over three seconds with the holder's state; after every 16,384 blocks translated, how many there were and the time translating them took, in all and for shared-cache code
`OCERZ_NZCV_PROD_CHECK=1`	work out again at every use which instruction's flags each consumer takes, and stop with the difference if it disagrees with the table made before the block's liveness pass (`6d7e051a`)
`OCERZ_GUESTPROF=<usec>`, `OCERZ_GUESTPROF_PERIOD`, `OCERZ_GUESTPROF_HOT=1`	sample every running guest thread at about that interval and, every period (10 seconds by default) and at exit, print the hottest guest code, host symbols and interpreted instruction forms; with `_HOT`, sample in each period only the thread that used the most CPU in the one before (`d0408936`, 2026-10-04)
`OCERZ_TRIPSTAT=1`, `OCERZ_BLACKLOG=1`, `OCERZ_INVSRC=1`, `OCERZ_LIVE_SCAN=1`	count exits from translated code to the dispatcher and print the commonest destinations every ten seconds; print the pages most often refused translation because they kept changing and attribute each refusal to the code that invalidated the page; search the whole live list for a block being retired whose index does not point back at it
`OCERZ_ORDERLOG=1`, `OCERZ_XLATPAGES=1`	print what first made the process use ordered memory, with a backtrace; log each distinct 4 KB page the process translates
`OCERZ_SUSPLOG=1`, `OCERZ_MODELOG=1`, `OCERZ_FPS=1`	native mode: log each `thread_suspend` and `thread_resume`; every far transfer with its selector, target mode and address (the WoW64 switches); cache mode: frames per second, counted at `CGLFlushDrawable`, once a second
`OCERZ_ALLMISS=1`, `OCERZ_DYNLOOKUPLOG=1`, `OCERZ_DLSYMLOG=1`, `OCERZ_MACSYSLOG=1`	list every unresolved import rather than the first two dozen; name each symbol imported with `-undefined dynamic_lookup` that nothing defines; print each `dlsym` with its handle, result and caller; print each `__mac_syscall` with its policy, number and argument words
`OCERZ_IPCLOG=1`, `OCERZ_SELPOOLLOG=1`, `OCERZ_SELVERIFY=1`	send the translated Steam `ipcserver`'s output to a file under `/tmp`; describe the selector index built when the shared cache's own table cannot be used, and check the cache's answers against it

Switches that restore old behaviour

Most fixes that change a rule add a switch that puts the old rule back. They exist for A/B measurement and to prove the regression test fails without the fix, not for daily use.

Variable	Restores
`OCERZ_NO_UPWARD_INIT=1`	upward dependencies never initialized (before 2026-09-22)

Variable	Restores
`OCERZ_NO_CLOSURE_LOADS=1`	no `+load` phase in a `dlopen` closure
`OCERZ_NO_DLOPEN_UPWARD=1`	no following of upward links to Foundation, CoreFoundation and libobjc during a `dlopen`
`OCERZ_NO_DLOPEN_CALLER_RPATH=1`	`@`-paths in `dlopen` not resolved against the caller
`OCERZ_NO_DLSYM_DEPS=1`, `OCERZ_DLSYM_UPWARD=1`	image-only `dlsym` on disk images; upward links followed by `dlsym` in the cache
`OCERZ_NO_BULK_NOTIFY=1`	bulk image-load callbacks called once, at registration
`OCERZ_NO_MACSYS_XLATE=1`	no rewriting of stack pointers in `__mac_syscall` arguments
`OCERZ_NO_THREAD_SIGNALS=1`	signals delivered from a shared pool, not kept on the receiving thread's cpu
`OCERZ_NO_PEER_VM=1`	no translation of `mach_vm_*` calls aimed at another task
`OCERZ_NO_VMMAP_TAKEN=1`	a fixed `mach_vm_map` may move over guest pages in use
`OCERZ_NO_ALIAS_CLIP=1`, `OCERZ_NO_ALIAS_REFRESH=1`	host-memory aliases not clipped at guest data; not refreshed when the host maps new memory there
`OCERZ_NO_IOKIT_INNER_PTR=1`	only page-aligned values in IOKit replies aliased
`OCERZ_NO_LOW_HOLE_FILL=1`	the kernel free to map below 12 GB in a Wine process
`OCERZ_NO_GPU_NOCOPY_XLATE=1`	Metal no-copy buffers passed with guest addresses
`OCERZ_INV_SCAN=1`, `OCERZ_INV_GRANULE=1`, `OCERZ_RETIRE_SCAN=1`, `OCERZ_FAULT_INV_RANGE=1`	the older invalidation strategies: walk every block, round to 64 KB granules, scan every block's edges, range-invalidate on a first fault
`OCERZ_RET_FLAGS_DEAD=1`	flags treated as dead at every `ret`; `ret_flags` fails under it
`OCERZ_NO_LAZY_REMAP=1`, `OCERZ_EAGER_SLIDE=1`	in-place rebasing of shared-cache pages; rebasing all of them at startup

The diagnostic knobs, counted

At `6e6c64b5` the source reads 545 distinct `OCERZ_*` names, found as quoted names in `src/` and `include/` (each is passed to `getenv`, `OCERZ_ENV_ON`, `jit.c`'s own `ENV_ON` or the `ENV_SET` macro, apart from `OCERZ_GUEST_ROOT` and `OCERZ_GUEST_GNU_ROOT`, which are kept in an array). The same count at `90fe4d8e` was 410. Each is documented where it is read, in that file's explanation block or beside the code. They fall into families by name:

Family	Names	At 90fe4d8e	Purpose
`OCERZ_NO_*`	216	124	turn one optimisation or behaviour off; 142 of them in `jit.c` (78 before)
`*LOG`	94	87	print one kind of event as it happens
`*TRACE`	20	18	finer-grained tracing of one subsystem
`*DUMP`	14	13	dump a structure or state on a trigger
`*LO` / `*HI`	14	14	address ranges for bisection

Family	Names	At 90fe4d8e	Purpose
`*CHECK` / `*VERIFY`	13	9	run-time validators of internal invariants
`*_EXE`	6	6	select processes by command line
`OCERZ_TCACHE*`	6	5	the translation cache
`*STAT`	5	5	counters printed periodically or at exit
`OCERZ_TSO_*`	3	3	memory-ordering strength

By file, `jit.c` reads 239 of them (162 before), `syscall.c` 116, `vm.c` 59, `dyld.c` 34, `dyldapi.c` 25, `mem.c` 14 and `main.c` 13; the files whose names begin `m32` read 33 between them. The growth is almost all `OCERZ_NO_*`: 92 new ones in a fortnight, because each rule the JIT work of 2026-10-07 to 2026-10-18 changed came with a switch that puts the old rule back, for A/B measurement and to prove the regression test fails without the fix. The count grows by a few with most fixes, because the instrument that found a bug usually stays in the tree afterwards, inert when its variable is unset.

APPENDIX B

Exit codes and messages

When a program runs to completion under AArchX, the process exits with the program's own status. When AArchX itself cannot go on, it prints at least one line on standard error beginning `ocerz:` and exits with one of a small set of codes. The first such line is nearly always the one that matters. This appendix lists the codes, then the messages by where they come from, with what each means and what to do about it. It is current to `6e6c64b5` (2026-10-18); the codes and messages that arrived since `90fe4d8e` are m32's (Chapter 13), exit 134 for an abort in native mode, a launch-script refusal, and a few native-mode lines.

Exit codes

Code	Meaning	Typical first line
guest's	the program ran and exited	none
64	AArchX's own arguments were wrong; `OCERZ_MODE` held an unknown value; the file is neither an Intel x86-64 program nor an i386 one; native mode was given a statically linked program; a launch script was given with an AArchX path that contains a space	`has no x86_64 slice: it is not an Intel program, so there is nothing to translate`
65	the program could not be read or, on the static path, loaded; for an i386 program (m32), no i386 slice, an image that cannot be mapped, a missing dependency or no entry point	`cannot read <path>`, `m32: <path>: no i386 slice`
70	static path: the guest arena or the guest stack could not be set up; i386 program: the 4 GB window, the guest heap or the stack could not be	`cannot build guest stack`, `m32: no room for a <n> MB guest heap`
71	native mode: imports could not be bound, before the guest ran an instruction; i386 program: required imports are unresolved (unless `OCERZ_M32_KEEP_GOING=1`)	`native: no bridge for <symbol> in <library>`, `m32: <path>: required imports are unresolved`
72	native mode: a bound export with no crossing was called	`bridge: <library> <symbol> not implemented`
125	a call AArchX made into guest code, such as an initializer, hit a fatal error outside tolerant mode	`initializer call to <addr> aborted after <n> instructions`
126	the diagnostic instruction cap `OCERZ_ICAP` was reached	`ICAP hit at <n> instructions`
128 + n	the guest sent itself signal *n* and had no handler for it	`guest self-signal <n>, no handler; exiting <128+n>`
134	native mode: the guest called `abort` or overran a stack guard, which end the process at once (`275843d4`, 2026-10-09); i386 program: an abort-like condition in m32 (a stack protector tripped, a trap with no export, a lazy import left unbound, a bad thread-local descriptor); `abort_with_payload`	`bridge: the guest called abort`, `m32: stack protector tripped (called from <addr>)`

Code	Meaning	Typical first line
139	a fatal fault: a guest crash with no guest handler, or a fault inside a bridged native call	`guest crash[...]` or `BRIDGE-FAULT[...]`
250-252	the dynamic loader returned an internal error before the program started; the status is the low byte of a negative code: 252 for an I/O failure such as an unmappable shared cache, 251 for a malformed image, 250 for an arena that could not be reserved	`cannot map shared cache for dynamic loading`

Codes 64, 65 and 70 come from `src/main.c`, and for i386 programs 65, 70 and 71 from `m32_run` in `src/m32.c` and `src/m32_selfload.c`; 71 otherwise from the loader at the end of a native-mode program's fixups; 72 is `OCERZ_BRIDGE_UNIMPL_EXIT` in `include/ocerz/vdylib.h`; 125, 126 and 139 from `src/vm.c`; 134 from `src/bridge.c`, `src/syscall.c` and the `m32_*` files; 128 + *n* from `src/syscall.c`. The 250-252 range is derived from the code rather than documented: `main` returns the loader's status, and the loader's error codes are small negative numbers.

Starting up

`unknown OCERZ_MODE value '<v>', want native or cache` (64)

The variable is refused rather than ignored, so a typo cannot silently select the wrong mode.

`<path> has no x86_64 slice: it is not an Intel program, so there is nothing to translate` (64)

An arm64-only binary was given at the top level. A guest that *spawns* such a binary is not refused: the child runs natively, as it would under Rosetta. Many bundled applications are arm64-only on macOS 27.

`<path> is a launch script, which starts its game through GAME_DEBUGGER, and ocerz's path has a space in it (<path>): move ocerz to a path without one` (64)

Steam's `%command%` for a Source game such as Left 4 Dead 2 is a script that expands `${GAME_DEBUGGER}` unquoted, which would split an AArchX path at the space. AArchX sets `GAME_DEBUGGER` to itself when it runs such a script, so the folder holding it must have no space in its path. A `GAME_DEBUGGER` already set wins.

`cannot read <path>` (65)

The path is wrong or the file is not a Mach-O AArchX can parse.

`native mode cannot run the static image <path>` (64)

Native mode stands in for the libraries a program links against, so a statically linked program has nothing for it to do. Run it in cache mode.

`cannot find the x86_64 shared cache in <Rosetta cryptex> or <OS cryptex>`

Cache mode found no `dyld_shared_cache_x86_64` in either location. This normally means Rosetta is not installed; `softwareupdate --install-rosetta` installs it. Native mode needs no cache. It is followed by `cannot map shared cache for dynamic loading` and exit 252.

Loading and binding

`Library not loaded: <path> (no such file)`, `Symbol not found`

dyld's own wording, used by AArchX's loader for a dependency that cannot be found or an import nothing exports. In a `dlopen` the load fails cleanly and leaves nothing mapped; at startup it names the image that wanted it. `OCERZ_DLPATH=1` prints every such failure with its reason.

`native: no bridge for <symbol> in <library>` … `native: N unresolved imports, which no virtual library exports` (71)

Native mode could not bind imports: the library has no API database, or its database does not list that export. The message suggests supplying an x86-64 build of the library through `OCERZ_GUEST_ROOT` or `runtime/guest`. Cache mode will usually run the program.

`native library without an API database`

A native-mode `dlopen` of a host library no database describes; the call returns NULL with this as its `dlerror`.

`init-skip: initializer <addr> faulted at rip=<addr> (skipped, process continues)`

An initializer run for a `dlopen` faulted. Those run in a tolerant mode, so the process continues; a framework whose initializer was skipped may misbehave later.

`initializer call to <addr> aborted after <n> instructions` (125)

The same outside tolerant mode, preceded by `initabort-bt` and a backtrace with each frame named by image and offset.

Running

`guest crash[<pid>] cpu#N <signal> host_addr=... host_pc=... ocerz_base=...` (139)

The guest faulted and had no handler that took the signal, which is what would happen on an x86 Mac too. The full report is described in Chapter 18: guest registers, the last blocks entered, the state of the faulting page and a symbolised backtrace.

`fatal: decode failed (<n>, <mode> mode) at rip=<addr>` followed by `bytes:`

The decoder met an instruction it does not know, or a byte sequence that is not an instruction at all. The bytes say which; a wild jump into data looks like the second.

`guest breakpoint/interrupt`

An `int3` or `ud2` in guest code, almost always an assertion in the program or a library. The report shows where; compare with the program under Rosetta.

`fatal on guest thread cpu#N; exiting process <pid> "<command line>"`

A guest thread hit a fatal condition and takes the whole process with it. The process id and command line are printed because under Wine a dozen processes share one standard error.

`guest self-signal <n>, no handler; exiting <128+n>`

The guest raised a signal at itself, typically `abort`, and had no handler. The frame-pointer chain printed before it shows who raised it.

`unimplemented BSD syscall num=<n> -> ENOSYS rip=...`

A system call AArchX does not implement; the guest gets `ENOSYS` and continues. With `OCERZ_STRICT_SYSCALL=1` it stops instead.

`unknown Mach trap: class=1 num=<n> name=<name> ...`

A Mach trap with no handler and no safe default.

`IMAGE-CLOBBER[<pid>] guest <call> addr=... len=... lands on <image>`

Always printed, never fatal by itself: a guest mapping landed on a loaded image, or a protection change took read or execute away from one. A library unmapped under running code otherwise surfaces only much later as a wild jump.

`gs0x320 WORKER-TERMINATE pid=<pid> gaddr=... gs=... icount=...`

A thread faulted at one address more than sixteen times in a row on Wine's foreign-worker pattern (no guest signal stack and no Wine thread block) and was ended. Since `ff6ea313` (2026-10-18) the count applies only to Wine processes: a handler that unprotects a page and returns, as a garbage collector's write barrier does, is entitled to fault at the same address as often as it likes, and before that change the seventeenth fault ended its thread.

`WILD-LOOKUP`, `WILD-STACK`, `WILD-FAULT-AV`, `WILD-WORKER-TERMINATE`

An indirect branch or fault whose target lies outside anything the guest mapped, with the source instruction and the guest registers. `OCERZ_WILDLOG=1` reports such branches as they happen.

Native mode

`bridge: <library> <symbol> not implemented` (72)

The program called an export that binds but has no crossing: a stub, because its signature cannot cross correctly (a variadic without a veneer, a `long double`, a union by value) or because nobody has written the crossing yet. An exit 72 often runs the program's own exit handlers on the way out and produces a second, confusing report; the line naming the stub is the real one.

`BRIDGE-FAULT[<pid>] <signal> inside a bridged call, not in guest code` (139)

A fault inside the Mac's own arm64 code while it ran on a guest thread. The following lines name the call, its signature, the native function and the nesting depth (`call=<library>:<symbol> sig='...' host_fn=... depth=N`), the fault address and guest `rip`, and a `cause:` line saying which side of the guest boundary the faulting address was on: inside guest space means the guest passed a bad pointer; outside means AArchX marshalled the call wrongly. The process stops because a native frame can be neither resumed nor unwound. A fault inside one of the eleven in-place string routines is not reported this way: it is delivered to the guest as its own fault at the calling instruction.

`bridge: the guest called abort` and `bridge: the guest overran a stack guard (__stack_chk_fail)` (134)

In native mode these end the process at once with `_exit(134)`: a request for the VM to exit waits for the other threads, and a thread that aborted inside a garbage collector's stop-the-world (Mono's crash handler) had left them suspended with the game frozen (`275843d4`, 2026-10-09).

`native: <library> is not on this system; <image> loads without it`

A library Apple shipped for Intel Macs that no macOS on Apple silicon has (QTKit, the 0.9.x libcrypto and the like): its imports are bound to placeholders that return 0 or nil, each named as it binds, instead of refusing the program. `OCERZ_NO_STUB_REMOVED=1` refuses it again (`334a5e1f`, 2026-10-16).

`native: native code called guest function <addr> directly; it runs with its integer arguments`

Native code jumped into a guest function stored where it expected a function pointer, as HandBrake stores its own into `stderr`'s write hook. The function runs as a callback with the first six integer arguments, named once. One taking floating point or a structure by value gets garbage; `OCERZ_NO_NATIVE_BOUNCE=1` stops the process instead.

`abi: the callback bank is exhausted`, `abi: native code called guest function ... and a long double does not cross in that direction`

The 65,536 trampolines are all bound, so a further guest function handed to native code gets none; or a callback's signature has a `long double`, which crosses calls out but not calls back.

`guest divided a 128-bit integer by zero, near rip=<addr>`

A native 128-bit division helper trapped on a zero divisor. It is delivered to the guest as `SIGFPE`; the line is printed, and the process dies of the signal, only when the guest has no handler.

A fatal report written to a descriptor duplicated before the guest ran

Since `334a5e1f` a fatal report goes to a descriptor duplicated before the guest ran, and a second fault while reporting ends the process with 139; a guest that redirects standard error into a pipe it then crashes on no longer swallows its own crash report.

32-bit programs (m32)

Every line begins `ocerz: m32:`. The first four are the ones a user meets on a first run; each names the command that fixes it.

`no i386 API database at <dir> (make apis32)`

The i386 databases under `runtime/apis32` have not been generated from the SDK. `make apis32` writes them and also makes the 64-bit ones, which m32 needs too.

`no Steam interface database at <path> (make steam32)`

A game asked for Steam's client and the bridge's interface database has not been made. `make steam32` fetches Valve's Steamworks headers at a pinned commit and writes it.

`<path>: required imports are unresolved` (71) and `<image>: unresolved <symbol> from <library>`

An import no database exports. The second form names each one, in the image that wanted it. `OCERZ_M32_KEEP_GOING=1` runs anyway with the import returning 0; `OCERZ_M32LOG=imports` shows how each resolved.

`<image>: cannot find <dependency>` (65)

A guest dylib is looked for in `@executable_path`, `@loader_path`, `@rpath` and `runtime/guest32` (or `OCERZ_GUEST32_ROOT`). A game built with GCC needs `tools/fetch_guest32_cxx.sh` for its libstdc++.

`<path>: no i386 slice`, `<path>: cannot read`, `<path>: cannot map the image`, `<path> has no entry point` (65)

The program is not an i386 Mach-O, or its layout cannot be mapped into the 4 GB window.

`<path>: what it unpacked at <addr> is not an i386 program m32 can load` (65)

A UPX-packed program's stub finished and its mach header is not one m32 can adopt (`src/m32_selfload.c`). `OCERZ_M32_NO_SELFLOAD=1` starts it as an ordinary program instead.

`no room for a <n> MB guest heap`, `window full (static allocation of <n> bytes)`, `out of handles`, `out of export ids at <name>`

The 4 GB window or one of its tables is exhausted. `OCERZ_M32_HEAP_MB` sets the heap's first size (1024 by default, growing on demand); the handle cells (8 bytes each between 0xE0000000 and 0xF0000000) are never freed.

`call to trap id <id> with no export (from <addr>)`, `a lazy symbol was left unbound (dyld_stub_binding_helper from <addr>)`, `dyld_stub_binder reached: an import was left lazy`, `stack protector tripped (called from <addr>)`, `bad thread-local descriptor <addr>` (134)

Conditions m32 treats as an abort. The third and fourth mean m32 bound every lazy pointer at load and the program still reached dyld's binding helper, so an import was missing; the return address names where.

`C++ exception not caught (<n>), thrown from <addr>`, `no eh_frame unwind information for <addr> (<image>)`, `_Unwind_Resume could not reach the catching frame`, `unwinding: DW_CFA <op> not handled`

The two-phase unwinder over the guest's `__eh_frame` (`src/m32_unwind.c`) could not find a handler or met a frame it cannot read.

`uncaught Objective-C exception: <text>`, `guest threw: "<text>"`, `<export> raised <exception> (caller <addr>)`

An Objective-C exception no guest `@catch` took, or one the host raised into the guest.

`steam: <method>, which the bridge cannot cross, was called (from <addr>), answering 0`, `steam: <version> is not in the interface database, answering 0`, `steam: <method> in <interface> has slot <n> where <m> belongs`

The Steam bridge could not carry a call, usually one that registers a function, or a game asked for an interface version the database lacks. A game that misbehaves after the first is worth a compatibility report with `OCERZ_M32LOG=steam`; the second has the same advice; the third says the database has a slot out of order.

`steam: cannot open <path>: <reason>`

The host's own `steamclient.dylib` could not be opened. Steam tells a game where its client lives only when Steam launched the game, so a game started from a terminal gets here or asks Steam to relaunch it.

`glMapBuffer: no guest shadow for <n> bytes`, `AGL: the host's AGL is missing a function`, `QuickDraw: a CoreGraphics, HIToolbox or QuickDraw function is missing`, `Carbon controls: a HIToolbox function is missing`, `CoreText is missing a function the ATSUI stand-in needs`

The host framework m32 builds a Carbon or OpenGL call on lacks an export it looks up with `dlsym`; nothing in the program is at fault, and the macOS version is the thing to check.

APPENDIX C

Glossary

Terms used in this book and in AArchX's source, with the meaning they have there. Where a term names a mechanism, the chapter that explains it is given in parentheses.

ABI engine

`src/abi.c` and `src/abicall.s`: moves arguments and results between the System V x86-64 calling convention and Apple's arm64 one, in both directions, from a function's signature. Native mode's crossings are built on it (Chapter 12).

Acquire and release forms

The arm64 loads and stores with ordering guarantees (`ldapur` / `stlur` in AArchX's ordered mode) used to give scalar accesses x86's total store order (Chapter 7).

AFP (`FEAT_AFP`, FPCR.AH)

An arm64 feature whose control bit makes add, subtract, multiply, divide and square root produce x86's NaN results. Where the processor has it, cache mode sets it and drops the software NaN checks.

Alias (of host memory)

In Wine's low shadow, a host region a service or the kernel handed the guest, remapped into the shadow at the same number so guest code can read it. Since 2026-10-03 each alias records the memory object it shows and is refreshed when the host maps new memory there (Chapter 8).

API database

One text file per library under `runtime/apis/macos/<sdk>/`, generated from the builder's own SDK by `make apis`, listing every export native mode synthesizes and how it crosses. Not committed, because it is derived from the SDK (Chapter 12).

Arena

Either the guest arena, the reserved region guest memory lives in, or the code arena, the 1 GB region translated code is emitted into. Context says which.

Arena flush

Since 2026-10-04, when less than an eighth of the code arena (at most 8 MB) is left, the next translation miss retires every block, empties every table that points into the arena and starts translating again at its front, once every thread has left translated code (Chapter 7).

Base hoist (Wine layout)

In the low shadow, checking a base register once per block, or an index register once per trip round a block when two or more of its operands share it, and putting each operand's displacement in the access, instead of guarding every operand's whole address (`ff3e02ea`, `e8a26292`, 2026-10-10 to 2026-10-11).

Batch (floating-point batch)

A run of floating-point work whose NaN checks are deferred to its end, with a checkpoint and an undo log so that it can be replayed exactly through the interpreter if a NaN appears.

Block

The unit of translation: guest code from an entry point to the instruction that ends it. A superblock continues past conditional branches.

Bridge, bridged call, crossing

In native mode, the path from an x86 stub to the host's real arm64 function: the arguments are read from the guest's registers by signature, the native function is called, and the result is put back in `rax` or `xmm0`.

Cache mode

The default mode: guest programs bind against Apple's x86-64 dyld shared cache, which AArchX maps and translates like any other code. Needs the Rosetta package installed for the file, but never runs Rosetta.

Callback trampoline bank

A fixed, pre-assembled bank of 65,536 arm64 trampolines. A guest function handed to native code is bound to one slot, and native code calls the slot's address; the same function always gets the same address.

Chaining

Patching a block's exit into a direct branch to its successor's translated body once the successor exists, so control passes without the dispatcher.

Churn reprieve

A page whose translations are invalidated repeatedly is left to the interpreter for a while; since 2026-09-12 it is retranslated after 4,096 refusals even if it never goes quiet, because hot library pages never did.

Commpage

The page the kernel maps at a fixed address for fast user-space reads of values such as the time base. AArchX emulates the x86-64 guest's commpage at `[0x7fffffe00000, 0x7fffffe04000)`.

Cryptex

A signed, separately updatable disk image macOS mounts under `/System/Volumes/Preboot/Cryptexes`. macOS 26 keeps the x86-64 shared cache in the OS cryptex, macOS 27 in a separate Rosetta cryptex; AArchX looks in the Rosetta one first.

Deferred flags

Instead of computing x86's six status flags after every instruction, the JIT records what produced them and materialises only what a later reader needs (Chapter 6).

Differential gate

A test that runs the same input through the interpreter and the JIT and requires identical results. No golden is needed, so it checks every instruction a test happens to execute (Chapter 16).

Dispatcher

The loop that finds or creates the translation for a guest address and enters it. A "trip" is an exit from translated code to the dispatcher.

dyld API table

The object through which libdyld's exported functions reach the dynamic loader, called by byte offset into a table of methods. AArchX installs its own and answers 59 slots, each identified by running code (Chapter 11).

dyld shared cache

A single file in which Apple pre-links every system library at fixed addresses. The x86-64 one is what cache mode maps.

DXMT

A Direct3D 11 implementation over Metal used by MacNdCheese's Wine builds; R.E.P.O. renders through it under AArchX.

D3DMetal

Apple's Direct3D-to-Metal layer from the Game Porting Toolkit, which Counter-Strike 2 uses under Wine.

Exact image (x87)

The 80-bit value each x87 register keeps beside its double, valid while the double is still that image rounded. FILD, FBLD, the constants, m80 loads and the FRSTOR and FXRSTOR images fill it; integer stores, FBSTP, the 80-bit images and FXAM read it (`src/x87.c` , `8f310eb8` , 2026-10-07).

Explanation block

The prose comment at the top of every AArchX source file, which says what the file owns and why it is shaped as it is. The tree has no inline comments; the blocks are the specification (Chapter 3).

Flag record

The saved description of a deferred flag producer (operation, operands, result) from which any flag can later be computed.

Flip, trace inversion

Retranslating a superblock with a conditional branch the other way round when the profile shows the side exit is the hot path.

Gap load

A plain load into a pinned register that sits inside the NZCV window. Since `a8773e55` (2026-10-11) it may, when the producer is a `cmp` , `sub` or `add` whose record holds both operands: the flags are built again from the record after the load, before the next consumer.

Golden

A test's stored expected output, preferably recorded by running the same binary under Rosetta.

gs base, TEB

On macOS the `gs` segment base points at the thread's pthread data; under Wine, Windows code expects it to point at the Thread Environment Block. Wine switches between the two, and AArchX keeps the base as guest CPU state (Chapter 15).

Guest, host

The guest is the x86 program being run and everything it sees; the host is the real arm64 Mac and AArchX itself.

Handle cell

An 8-byte cell in `[0xE0000000, 0xF0000000)` that stands for a host pointer an i386 guest may hold: word 0 is an isa word the Objective-C layer fills, word 1 the cell's index. Reading through one shows a sane isa and nothing of the host object.

Identity map

The layout used for dynamically linked programs, where guest addresses are host addresses, so a translation needs no address arithmetic. The identity arena is 256 GB, starting at 12 GB.

In-place routines (leaf routines)

Hand-written arm64 versions of `strlen` , `strnlen` , `strcmp` , `strncmp` , `memcmp` , `bcmp` , `strchr` , `memchr` , `memcpy` , `memmove` and `memset` in `src/leaf.s` , called from translated code with the guest's registers left where they are.

Initializer

Code a library runs before `main` : C and C++ constructors, and Objective-C `+load` methods. AArchX runs them in dyld's order, dependencies first and upward dependencies after the whole walk (Chapter 11).

i386 differential corpus

The generated 32-bit sequences that the i386 differential gate runs through both engines and compares in full architectural state (Chapter 16): 20,033 (20,000 generated and 33 hand-written) on 2026-10-04, and since the x87 work of 2026-10-07 (`9f0b7087`) 40,044 in each of two memory layouts: 44 hand-written, 20,000 random integer sequences and 20,000 random x87 ones.

Image list

The ordered list of loaded images that `_dyld_image_count` and its siblings walk. A shared-cache image reached by `dlopen` joins it since 2026-09-22; that is what made the GPU visible to translated programs.

Interpreter

AArchX's reference engine, which executes decoded instructions one at a time in C. Everything the JIT declines runs here, and every JIT result is checked against it.

JIT lock

The lock held while translating. It never parks a waiting thread, because a lost wakeup once wedged translation for a whole process.

Keep-retire

After the flip profile keeps every probed branch of a superblock as it was, the block is retired once no edge still probes or watches through its stub, and translated again without the probes, which would otherwise leave a `nop` run on every pass (`d7567fc5`, 2026-10-18; `OCERZ_NO_FLIP_KEEP_RETIRE=1`).

Lane cache

Inside a translated loop, scalar SSE results and the upper halves of `ymm` registers are kept in spare host vector registers instead of being merged back after every instruction.

Launch script

A program whose executable is a `#!` script. Steam's `%command%` for a Source game is one (`hl2.sh`), starting the game through `${GAME_DEBUGGER}`; AArchX runs it with `GAME_DEBUGGER` naming itself, and any other script on its interpreter's x86-64 slice.

Lazy rsp

In a call spliced into its caller whose stack traffic is its own, `rsp`'s register keeps its value from before the call, the pushes, pops, calls and rets move a translation-time offset, and each stack access is at a fixed offset from the register (`256faca8`, 2026-10-11; `OCERZ_NO_LAZY_RSP=1`).

Leaf call (m32)

An m32 export small enough, and taking only integers, booleans and pointers, to be called straight from an import stub's block with no crossing (`src/m32_leaf.c`): the small libc functions, the CGL context calls and many OpenGL functions. Not to be confused with the in-place string and memory routines of `src/leaf.s`, which are also called leaf routines. Built with `-mgeneral-regs-only` so the guest's vector registers stay pinned across the call.

Low shadow

The third memory layout, used by Wine processes and other non-PIE programs with low addresses: guest addresses below 12 GB and a strip at the top live at their own host bases, and every memory access is checked to choose the base (Chapter 8).

Low-shadow guard

The check a translated memory access performs in the low shadow. Since 2026-10-04 an address below 12 GB becomes a host address with a shift, a compare and one `orr`.

m32

The runtime for 32-bit (i386) Mac programs, `src/m32*.c`: an executable with an i386 slice and no x86-64 one is handed to `m32_run` and runs in native mode against the Mac's own arm64 frameworks. It came from the MacShack project into the tree on 2026-10-10 (`0ca24d64`) (Chapter 13).

m32 window

The flat 4 GB guest address space of an i386 program, `ocerz_mem_init(0, 4 GB)`. Guest address *g* is host base plus *g*, and the guest never holds a host address. The trap window, the handle cells and the top of the window are never handed out.

MacNdCheese

A distribution of Wine with graphics back ends for macOS, whose Wine builds run under AArchX.

KUSER_SHARED_DATA

The Windows page of shared system values at a fixed low address, which Wine maps in every process. Under AArchX its 16 KB host page is shared by the whole Wine prefix, which is why Wine processes reserve their arena at a fixed base.

Mach trap, Mach message

The Mach half of macOS's kernel interface: traps are direct kernel entries, messages carry data and port rights between tasks. Both often carry pointers AArchX must translate.

MIG

The Mach Interface Generator, whose message formats IOKit and many system services use; AArchX translates their in-line and out-of-line pointers.

Mini-dyld

AArchX's own dynamic loader in `src/dyld.c`, which loads, binds and initializes a dynamically linked program without running Apple's dyld (Chapter 11).

Native mode

The mode selected by `-native`: no shared cache is mapped, and the guest's system libraries are synthesized x86 images whose exports cross into the Mac's own arm64 frameworks (Chapter 12).

Non-PIE

A program linked to load at a fixed address rather than anywhere. A non-PIE main with a low base activates the low shadow; the test suite builds guest tests that way to reach Wine's layout.

NZCV forwarding

Emitting an x86 compare, test or arithmetic instruction as a flag-setting arm64 instruction when its consumer is a branch or conditional select that can read arm64's NZCV flags directly.

NZCV window

The number of instructions between a flag-producing instruction and the `setcc`, `cmov`, `adc`, `sbb` or `jcc` that reads its flags through which arm64's NZCV is handed on instead of rebuilt from the flag record: 3 until 2026-10-11, 8 since `e853c450` (`OCERZ_NZCV_GAP`).

Ordered mode, plain mode

Plain mode emits ordinary loads and stores; ordered mode uses acquire and release forms for scalar accesses to give x86's ordering. A process starts plain and switches to ordered for good at its first thread, fork or shared writable mapping.

Out-of-line arm (oolslow)

A rarely taken path, such as an unaligned access or a NaN fix-up, emitted after a block's body instead of inline.

Packed structure

A structure declared under `#pragma pack(2)`, as Carbon's headers do, whose i386 and arm64 layouts differ because an arm64 pointer after a 4-byte field sits at offset 4. A notation says so with a digit after the brace, `R{2up}`, and the crossing copies it between the two layouts.

Paired class, proxy

In m32's Objective-C 1 runtime, a *proxy* is an `old_class` in the window that stands for a host class; a guest class whose chain reaches a proxy is *paired* with a host class made at load, holding every guest method as a callback slot into the guest's implementation.

Paired delta

The benchmark method: time a kernel at size n and at n/2 and report the difference, which removes startup and first translation (Chapter 17).

Pinning, pin class

Keeping guest registers permanently in fixed host registers. In pin class 3, guest `rax`-`rdi` live in `x21`-`x28`, `r8`-`r13` in `x3`-`x8`, `r14`-`r15` in `x1`-`x2`, and `xmm0`-`xmm15` in `v16`-`v31` (Chapter 6).

PE

Portable Executable, the Windows binary format. Under Wine, "PE code" is Windows code as opposed to Wine's own Unix-side code.

RAS (return-address stack)

A stack of guest return addresses and their translated continuations, kept on the host stack so a guest `ret` becomes a real `ret` and the processor's return predictor stays accurate.

Retire generation

A counter per column of the per-site indirect-branch caches, carried in each entry's address word. Retiring a block bumps the columns of the blocks it removed instead of walking every table (`d8b7073c`, 2026-10-05).

Retirement

Marking a translated block dead when the guest code it came from changed or its translation must be redone, and unchaining branches into it.

Rosetta 2

Apple's x86-64 translator for Apple silicon, the baseline every AArchX measurement and most goldens are taken against, and the source of the x86-64 shared cache package.

Selector

Either an x86 segment selector, which WoW64 uses to switch between 32- and 64-bit code and which AArchX keeps as CPU state, or an Objective-C method selector. Context says which.

Self-loading program

An i386 program with one segment, no libraries and no dylinker, such as a UPX-packed executable, whose stub unpacks the real program with raw system calls and then opens `/usr/lib/dyld`. m32 answers that open with a dyld of its own whose entry is a trap, takes the program from memory and starts it like any other (`src/m32_selfload.c`).

Send cache

Native mode's per-thread table (`t_ob_fast`) of what `objc_msgSend` needs to know about a class, selector and implementation: whether the implementation is guest code to jump to and, for a method with nothing to convert, its signature and selector name. A repeated send asks libobjc for the implementation and makes the call; a generation bumped by every bridged change to a class's methods retires entries (`beed19af`, 2026-10-18).

Side exit

The out-of-line stub through which a superblock leaves on its rarely taken conditional branch.

Signal stop

How a signal reaches a thread spinning in a translated loop: the host handler unchains the edges of the block the thread is in and of the blocks a few hops on and rewrites their stop sites, so the thread's next jump out goes

to the dispatcher, which returns to the run loop, which delivers the signal between two steps (`a7fd0f3f`, 2026-10-14).

Slot (4 KB slot)

Guest pages are 4 KB and host pages 16 KB, so AArchX tracks protection and ownership per 4 KB slot inside each host page and asks the kernel for the union.

SMC, write-arming

Self-modifying code. Pages that hold translated code are write-protected (armed); a guest write faults, and the blocks overlapping the written range are retired.

Splicing

Translating a small called function into its caller's block, with matched pushes and pops renamed into registers.

Static arena

The layout for statically linked programs: a fixed guest region `[0x100000000, 0x900000000)` at a host base.

Steam bridge

m32's answer to Valve having shipped no i386 `steamclient` since 2019: when an i386 program dlopens a `steamclient.dylib` with no i386 slice, m32 opens the host's own and answers its exports with proxies whose vtable slots cross into the host objects' by the notations `make steam32` writes (`src/m32_steam.c`). It works only for a game Steam launched.

Stub, trap window

Each export of a synthesized library is twelve bytes of x86: load the export's id into `r11d`, jump to an address inside `[0xdda00000, 0xdda10000)`. Jumping there traps into AArchX, which performs the call.

Superblock

A translated block that continues past forward conditional branches along the likely path, with the other path in side exits.

Synthesized image (vdylib)

An x86-64 Mach-O built in memory by `src/vdylib.c` for a system library that has no file in native mode: real headers, an export trie and a stub per export.

Thread-specific data (TSD)

macOS's per-thread slots reached through the `gs` base. AArchX maps a host thread's TSD back to the guest thread it belongs to when answering thread queries.

Tolerant initializer mode

Initializers run for a `dlopen` may fault without ending the process: the faulting initializer is skipped with an `init-skip` message.

Translation cache (tcache)

Translated blocks stored on disk under `~/Library/Caches/ocerz` and loaded by later processes, on by default since 2026-09-27 (Chapter 7).

Trap block

A call through a function pointer to an m32 export (`dlsym`'s, a GL extension's) lands in the trap window. Since `1ae3140a` (2026-10-18) the JIT translates such a trap as the import stub it stands for, a `jmp` through a guest word holding the trap, so the call is made from inside the block, as a leaf when it can be, instead of leaving translated code for the run loop on every call.

TSO

Total store order, x86's memory model, stronger than arm64's.

Twin

The guest-side partner of a host object in m32. A guest instance of a paired class is a host object and a guest twin of the guest's `instance_size`, aliased both ways; guest CFSTR literals, CF-typed data and guest blocks have host twins too. A twin goes with its host object (`aa51c8bb`); `OCERZ_NO_M32_REAP=1` lets it outlive it.

Two-level namespace

Mach-O's rule that an import names the library it comes from, so two libraries can export the same name without conflict.

Unstick monitor

A thread that interrupts guest threads parked for more than 800 ms in waits that tolerate a spurious `EINTR`, so a lost wakeup does not hang a process forever.

Upward dependency

A library link marked upward, the back edge of a dependency cycle. It orders nothing, but the target is still loaded and initialized, after the whole initializer walk.

Veneer

A small stub in one of the 256 KB pools placed every 64 MB in the code arena, through which a chain reaches a target more than 128 MB away.

Workqueue bridge

Forwarding libdispatch's kernel workqueue to the host and running each work item on a guest personality, so Cocoa applications do not deadlock. On by default since 2026-09-10.

WoW64

Windows-on-Windows 64, Wine's way of running 32-bit Windows code in a 64-bit process, switching between 32- and 64-bit code segments (Chapter 15).

xbench

The benchmark in `tests/guest`, in a static and a dynamically linked build, compared against Rosetta by `tests/xbench_compare.py`: fifteen kernels until 2026-10-07, seventeen since the x87 kernels `x87` and `x87pc24` were added (`6c6679f0`).

xlive

Cross-block flag liveness: an exit's flags are dead when the successor, decoded up to three blocks deep, overwrites them before reading them.

x86-64-v3

The x86 feature level with AVX2, FMA, BMI1/BMI2, F16C, LZCNT and MOVBE, which Rosetta runs since macOS 15 and AArchX implements.

APPENDIX D

Timeline

Four and a half months from an initial commit to a translator that runs Steam, Wine, a Unity game, and 32-bit Mac programs that macOS itself no longer runs. The milestones below are taken from the git history and the project's notes; each hash is a commit that can be read for the full story. The repository's history was rewritten twice: once between 2026-09-19 and 2026-10-02, and again on 2026-10-14, when every commit ID changed and the contents did not. Every hash in this book is from the history as it now stands.

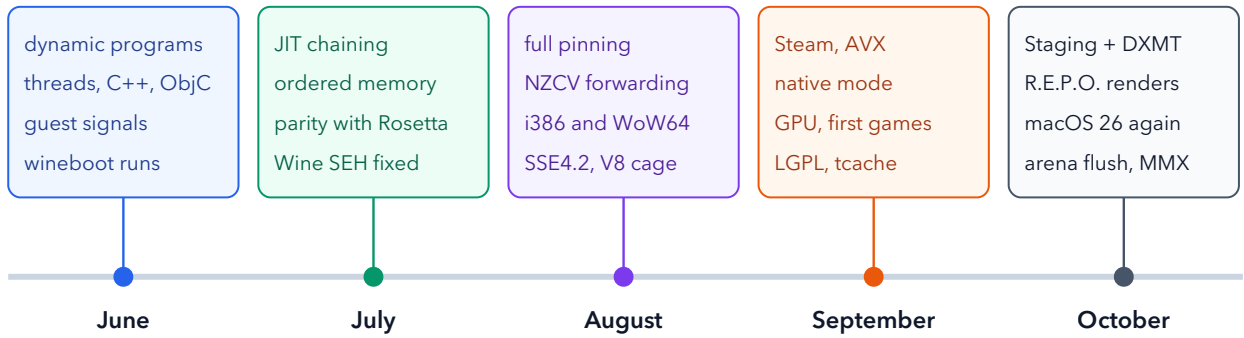


Figure D.1. The first four months at a glance, to 2026-10-04. September carried the largest share of the work: the macOS Steam client, AVX, native mode, the GPU and the first games all arrived in it. Figure D.2 continues from 2026-10-05.

June: from static binaries to Wine

Date	Commit	Milestone
2026-06-03	`d4d3cae1`	Initial commit; LGPL-2.1 license file the same day.
2026-06-05	`f7ab7804`	Real Objective-C programs run end to end against the x86-64 shared cache.
2026-06-06	`f2805908`, `0350 6214`	Real guest threads, Swift metadata and the C++ standard library; C++ exceptions through the dyld unwind-sections slot.
2026-06-08	`28b23115`	`@rpath` expansion and thread-local variables: Homebrew binaries and C++ `threa d_local` work.
2026-06-11	`2c353689`	Guest signals, and the start of Mach thread semantics.
2026-06-13	`63ee8a2c`	Wine's `wineboot` runs end to end into the GUI layer.
2026-06-14	`dcf8e6cd`	Kernel-allocated Mach messages bridged to the guest's libdispatch: Notepad builds its GUI.

July: a JIT that keeps up

Date	Commit	Milestone
2026-07-16	`1c60c5bc`	x86-TSO memory ordering.
2026-07-17	`2d2e08fc`	JIT: lazy flags, call and return inlining, chaining infrastructure.
2026-07-18	`8680d8b9`, `525285e9`	Block chaining and two-way conditional-branch linking.
2026-07-23	`f5325e6c`	The plain-memory tier: rough parity with Rosetta for the first time.
2026-07-25	`9d8fe914`	Ordered memory 37% faster with `ldapr` and relocated unaligned arms.
2026-07-26	`fbd57531`	`FXSAVE` / `FXRSTOR` save the XMM registers: `wineboot` stops crashing.
2026-07-30	`81478bfe`, `2d0dfe65`	Host memory regions aliased into Wine's low shadow; `dlopen_preflight` implemented for Metal's device discovery.
2026-07-31	`3317e905`	The Windows exception blocker resolved: `vm_region` replies describe the guest's memory, not the host's.

August: the pinned JIT and 32-bit code

Date	Commit	Milestone
2026-08-15	`2699e16d`	The JIT rewrite begins: all sixteen `xmm` registers pinned in every block, value-based conditions, in-place SSE on pinned registers.
2026-08-16	`b38ddd0a`, `86fa88f2`, `1233dfe5`	NZCV forwarding; plain memory until a second observer appears, so dynamic programs run at static speed; shared-cache pages rebased lazily per page.
2026-08-17	`2f9087c7`, `3e2e44f6`	Cheap ordered memory with `ldapur` / `stlur`; superblocks past backward branches.
2026-08-20	`0d8c5593`	i386: WoW64 host support, 32-bit JIT code generation and the i386 differential gate.
2026-08-25	`20347e5b`, `1ed5e8bb`, `3302e368`	SSE4.2 implemented and advertised (Steam checks for it); `rsp` held as a host pointer in pin class 3; Chromium's V8 cage lands.
2026-08-26	`393ef104`	`os_unfair_lock` made robust against owner death: a Steam crash fixed.

September: Steam, AVX, native mode and games

Date	Commit	Milestone
2026-09-04	`dd4a764f`, `04ca5ab2`	The project is called AArchX; `ocerz version` says AArchX 0.1.
2026-09-05	`6073adee`, `ef67bb06`, `24b194e9`	Flags survive a return after a compare: Wine windows stop painting black. Notepad stays up. 32-bit exceptions reach the 32-bit dispatcher, ending Steam's restart storm.
2026-09-05	`36930d64`, `78ffa897`	Vector accesses left plain in ordered mode; the license changed to a proprietary text.

Date	Commit	Milestone
2026-09-06	`8ce197f2`, `536602e8`	The SQLite miscompile fixed; misaligned locked atomics under an address-striped lock.
2026-09-07	`a80ba2ed`	Plain guest stores into translated code are trapped: self-modifying code handled.
2026-09-08	`0dd7e7e1`, `b3e11a15`	`punpckhqdq` decoded correctly, ending the 74-pixel windows; VEX decoded and interpreted. Windows Steam's login window renders with software compositing.
2026-09-10	`b17c4915`	Initializers run in dyld's order.
2026-09-11	`039558c8`, `3e3756a6`	Chess plays and eight more system applications open their windows; System V IPC lets Steam's bootstrapper run.
2026-09-12	`3037b859`, `d4b0a469`, `59ee19c0`	One explanation block per file, no inline comments; Steam's `ipcserver` runs translated; the macOS Steam client comes up with its full UI.
2026-09-13	`14b14157`, `35511297`, `459cd391`	AVX2 and FMA translated; the 256 GB guest arena; Ollama's command line works.
2026-09-15	`a16dc15b`	One NaN batch per run of floating-point work.
2026-09-16	`12e7bd98`	Native mode: a process that maps no shared cache.
2026-09-19	`6f97a322`	macOS 27 fixes; the shared cache is looked for in the Rosetta cryptex.
2026-09-21	`b0aa6e70`, `b3990360`	Native mode runs Steam; the in-place string routines and hardware NaNs; Safari works.
2026-09-22	`9e2470bd`, `ace6eee0`, `b7e28bed`	Translated programs see the GPU; chains through veneers; `shmat`. Brawlhalla reaches its menus.
2026-09-23	`bf5fd902`, `cd1f9c1c`, `2fb36e21`	LGPL-2.1-or-later again, with the documentation in the tree; games launched from Steam run under AArchX; every dyld API slot identified by running code.
2026-09-26	`26b1ccba`, `5606e53e`, `fdb259d2`	Wine processes reserve their arena at a fixed base; Counter-Strike 2 loads D3DMetal; OpenGL works from Wine threads.
2026-09-27	`e9732558`, `93547a4c`, `08a0458b`	The sampling profiler across a process tree; translations kept on disk, on by default.

October: games under Wine, and 32-bit programs

Date	Commit	Milestone
2026-10-01	`2e5f0d20`, `6064e90e`	Steam on stock Wine Staging 11.16 with DXMT; dlopened frameworks reach their XPC services.
2026-10-03	`13e7b14a`, `cc4ea96c`, `334da007`	Host-memory aliases refreshed, IOKit records aliased, the low 12 GB kept for the guest: R.E.P.O. renders under Wine through DXMT.
2026-10-03	`ae3780b7`, `2f81fcd1`	Guest tests restated in x86 terms; the README cut to a front door with the long material in `docs/`.
2026-10-03	`416248f4`, `baf668f3`	Cache mode on macOS 26 and 27; upward dependencies initialized after the whole walk, fixing `sw_vers` on macOS 26.7.1.

Date	Commit	Milestone
2026-10-04	`7b82e872`, `1b636806`	The code arena flushes instead of filling; a Wine address below 12 GB becomes a host address with one `orr`. R.E.P.O.'s menu goes from 28-30 frames a second to roughly 45-65 (single runs reached 65).
2026-10-04	`b7449a73`, `975c18fb`	MMX translated; `OCERZ_TSO_NARROW` for narrow SSE ordering.
2026-10-04	`06d1b4da`, `f2ab14b7`, `af333877`	Swift programs run on a guest copy of the swift.org runtime, Swift Concurrency included; Discord's x86-64 slice reaches its splash screen and gateway in native mode.
2026-10-05	`de872c96`	Release 0.4: `ocerz version` prints AArchX 0.4. The same day: Objective-C exceptions in native mode (`46349987`), a per-column retire generation instead of a walk of every table (`d8b7073c`), 177 API databases with macOS 27 baselines (`83816ab6`).
2026-10-06	`33e00a13`, `22221270`, `0aae84f0`	Release 0.5. MacNdCheese's Wine runs under native mode: Steam logs on and R.E.P.O. reaches its main menu on DXMT, then on D3DMetal.
2026-10-07	`8f310eb8`, `6c6679f0`, `7fc00672`	x87 gets a unit with exact 80-bit images, and its common instructions are translated (issue #6): the x87 kernel falls from 1.147 s to 0.161 s against Rosetta's 0.731 s. Release 0.6.
2026-10-08 to 2026-10-11	`ff3e02ea`, `ab86121d`, `e853c450`, `256faca8`	The Wine layout's JIT catches up: base hoists, the stack pointer as a host address, an NZCV window of 8 instructions, `adc` and `sbb` through `csinc`, lazy `rsp`; driven by 7-Zip's LZMA decoder under Wine.
2026-10-09	`85b248fc`, `90903488`, `ff8ce8b9`	Delayed-init dependencies stay out of the launch; fixed dyld API slots become translated x86; `ptest` translated.
2026-10-10	`0ca24d64`	m32: 32-bit Mac programs run, brought in from MacShack; Batman: Arkham Asylum plays its opening at about 31 frames a second on an M4 Max.
2026-10-11	`825fe3a8`, `ff020772`	The Steam bridge: an i386 game's own Steam API reaches the user's own Steam client; `make steam32` writes the interface database.
2026-10-12 to 2026-10-13	`a322c200`, `d20ee9a7`	Call of Duty: Modern Warfare 3 and Left 4 Dead 2 play from Steam under m32.
2026-10-13	`9a4da5bf`, `3bf6e6aa`, `962f9c7c`, `2dce5f07`	Release 0.7: a one-line installer for the prebuilt release, a Homebrew cask in the `mont127/aarchx` tap, and AArchX.app.
2026-10-13	`1c4811dc`, `2206cc7a`, `1a316328`, `a246dc47`	Releases 0.7.1, 0.7.2 and 0.7.3 the same day; the app gains a library, Windows programs through Wine and CrossOver bottles.
2026-10-13	`2cf48a7e`, `f4ef8900`	The dec/jnz fixed-lane miscompile, in the JIT since 2026-08-25, is fixed; a chain install needs one barrier pair (translation of a cold perl start 275 ms to 208 ms on the Mac Studio).
2026-10-14	`a7fd0f3f`	A pending signal is delivered between two steps, and a thread in a translated loop is stopped so it gets there. The same day the repository's history was rewritten: every commit ID changed and the contents did not.
2026-10-16	`0144d669`, `34b6f7fb`, `334a5e1f`	UPX-packed programs unpacked by their own stub; Kega Fusion runs on AGL; Transmission, Sequel Pro, HandBrake and VLC 2.2 open their windows in native mode.

Date	Commit	Milestone
2026-10-17 to 2026-10-18	`71b7c555`, `aa51c8bb`, `88b7e20c`	AssaultCube and SuperTuxKart, then Neverball, Wesnoth, Adium and Nexuiz run under m32.
2026-10-18	`1ae3140a`, `ad51f4b3`, `beed19af`	i386 programs run in native mode by default and calls through function pointers stay in translated code; a callback enters guest code without three system calls (536 to 100-112 ns a comparison); a per-thread `objc_msgSend` send cache.
2026-10-18	`a454f2fd`, `6e6c64b5`	Release 0.8; the tree this edition of the book describes.

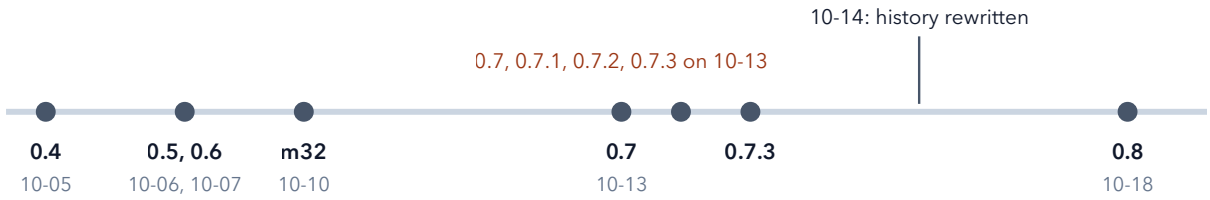


Figure D.2. Releases 0.4 to 0.8, 2026-10-05 to 2026-10-18, with the start of m32 and the history rewrite. Four releases were cut on 2026-10-13 alone.